

WebAssembly Specification

Release 3.0 (2026-11-06)

WebAssembly Community Group

Andreas Rossberg (editor)

Nov 06, 2026

Contents

1 Introduction **1**
- 1.1 Introduction . . . 1
- 1.2 Overview . . . 3

2 Structure **5**
- 2.1 Conventions . . . 5
- 2.2 Values . . . 7
- 2.3 Types . . . 9
- 2.4 Instructions . . . 14
- 2.5 Modules . . . 23

3 Validation **29**
- 3.1 Conventions . . . 29
- 3.2 Types . . . 34
- 3.3 Matching . . . 39
- 3.4 Instructions . . . 47
- 3.5 Modules . . . 73

4 Execution **81**
- 4.1 Conventions . . . 81
- 4.2 Runtime Structure . . . 83
- 4.3 Numerics . . . 91
- 4.4 Types . . . 119
- 4.5 Values . . . 119
- 4.6 Instructions . . . 122
- 4.7 Modules . . . 168

5 Binary Format **179**
- 5.1 Conventions . . . 179
- 5.2 Values . . . 181
- 5.3 Types . . . 182
- 5.4 Instructions . . . 185
- 5.5 Modules . . . 200

6 Text Format **207**
- 6.1 Conventions . . . 207
- 6.2 Lexical Format . . . 209
- 6.3 Values . . . 211
- 6.4 Types . . . 214
- 6.5 Instructions . . . 217
- 6.6 Modules . . . 234

7 Appendix **241**
7.1 Embedding . . . 241
7.2 Profiles . . . 249
7.3 Implementation Limitations . . . 251
7.4 Type Soundness . . . 254
7.5 Type System Properties . . . 268
7.6 Validation Algorithm . . . 270
7.7 Custom Sections and Annotations . . . 278
7.8 Change History . . . 283
7.9 Index of Types . . . 294
7.10 Index of Instructions . . . 294
7.11 Index of Semantic Rules . . . 306

Index **309**

CHAPTER 1

Introduction

1.1 Introduction

WebAssembly (abbreviated Wasm[2]) is a *safe, portable, low-level code format* designed for efficient execution and compact representation. Its main goal is to enable high performance applications on the Web, but it does not make any Web-specific assumptions or provide Web-specific features, so it can be employed in other environments as well.

WebAssembly is an open standard developed by a W3C Community Group[1].

This document describes version 3.0 (2026-11-06) of the core WebAssembly standard. It is intended that it will be superseded by new incremental releases with additional features in the future.

1.1.1 Design Goals

The design goals of WebAssembly are the following:

- Fast, safe, and portable *semantics*:
  - **Fast**: executes with near native code performance, taking advantage of capabilities common to all contemporary hardware.
  - **Safe**: code is validated and executes in a memory-safe[3], sandboxed environment preventing data corruption or security breaches.
  - **Well-defined**: fully and precisely defines valid programs and their behavior in a way that is easy to reason about informally and formally.
  - **Hardware-independent**: can be compiled on all modern architectures, desktop or mobile devices and embedded systems alike.
  - **Language-independent**: does not privilege any particular language, programming model, or object model.
  - **Platform-independent**: can be embedded in browsers, run as a stand-alone VM, or integrated in other environments.
  - **Open**: programs can interoperate with their environment in a simple and universal manner.

[2] A contraction of "WebAssembly", not an acronym, hence not using all-caps.

[1] https://www.w3.org/community/webassembly/

[3] No program can break WebAssembly's memory model. Of course, it cannot guarantee that an unsafe language compiling to WebAssembly does not corrupt its own memory layout, e.g. inside WebAssembly's linear memory.

- Efficient and portable *representation*:
  - **Compact**: has a binary format that is fast to transmit by being smaller than typical text or native code formats.
  - **Modular**: programs can be split up in smaller parts that can be transmitted, cached, and consumed separately.
  - **Efficient**: can be decoded, validated, and compiled in a fast single pass, equally with either just-in-time (JIT) or ahead-of-time (AOT) compilation.
  - **Streamable**: allows decoding, validation, and compilation to begin as soon as possible, before all data has been seen.
  - **Parallelizable**: allows decoding, validation, and compilation to be split into many independent parallel tasks.
  - **Portable**: makes no architectural assumptions that are not broadly supported across modern hardware.

WebAssembly code is also intended to be easy to inspect and debug, especially in environments like web browsers, but such features are beyond the scope of this specification.

1.1.2 Scope

At its core, WebAssembly is a *virtual instruction set architecture (virtual ISA)*. As such, it has many use cases and can be embedded in many different environments. To encompass their variety and enable maximum reuse, the WebAssembly specification is split and layered into several documents.

This document is concerned with the core ISA layer of WebAssembly. It defines the instruction set, binary encoding, validation, and execution semantics, as well as a textual representation. It does not, however, define how WebAssembly programs can interact with a specific environment they execute in, nor how they are invoked from such an environment.

Instead, this specification is complemented by additional documents defining interfaces to specific embedding environments such as the Web. These will each define a WebAssembly *application programming interface (API)* suitable for a given environment.

1.1.3 Security Considerations

WebAssembly provides no ambient access to the computing environment in which code is executed. Any interaction with the environment, such as I/O, access to resources, or operating system calls, can only be performed by invoking functions provided by the embedder and imported into a WebAssembly module. An embedder can establish security policies suitable for a respective environment by controlling or limiting which functional capabilities it makes available for import. Such considerations are an embedder's responsibility and the subject of API definitions for a specific environment.

Because WebAssembly is designed to be translated into machine code running directly on the host's hardware, it is potentially vulnerable to side channel attacks on the hardware level. In environments where this is a concern, an embedder may have to put suitable mitigations into place to isolate WebAssembly computations.

1.1.4 Dependencies

WebAssembly depends on two existing standards:

- IEEE 754[4], for the representation of floating-point data and the semantics of respective numeric operations.
- Unicode[5], for the representation of import/export names and the text format.

However, to make this specification self-contained, relevant aspects of the aforementioned standards are defined and formalized as part of this specification, such as the binary representation and rounding of floating-point values, and the value range and UTF-8 encoding of Unicode characters.

[4] https://ieeexplore.ieee.org/document/8766229
[5] https://www.unicode.org/versions/latest/

Note

The aforementioned standards are the authoritative source of all respective definitions. Formalizations given in this specification are intended to match these definitions. Any discrepancy in the syntax or semantics described is to be considered an error.

1.2 Overview

1.2.1 Concepts

WebAssembly encodes a low-level, assembly-like programming language. This language is structured around the following concepts.

Values

WebAssembly provides only four basic *number types*. These are integers and IEEE 754[6] numbers, each in 32 and 64 bit width. 32-bit integers also serve as Booleans and as memory addresses. The usual operations on these types are available, including the full matrix of conversions between them. There is no distinction between signed and unsigned integer types. Instead, integers are interpreted by respective operations as either unsigned or signed in two's complement representation.

In addition to these basic number types, there is a single 128 bit wide vector type representing different types of packed data. The supported representations are four 32-bit, or two 64-bit IEEE 754[7] numbers, or different widths of packed integer values, specifically two 64-bit integers, four 32-bit integers, eight 16-bit integers, or sixteen 8-bit integers.

Finally, values can consist of opaque *references* that represent pointers towards different sorts of entities. Unlike with other types, their size or representation is not observable.

Instructions

The computational model of WebAssembly is based on a *stack machine*. Code consists of sequences of *instructions* that are executed in order. Instructions manipulate values on an implicit *operand stack*[8] and fall into two main categories. *Simple* instructions perform basic operations on data. They pop arguments from the operand stack and push results back to it. *Control* instructions alter control flow. Control flow is *structured*, meaning it is expressed with well-nested constructs such as blocks, loops, and conditionals. Branches can only target such constructs.

Traps

Under some conditions, certain instructions may produce a *trap*, which immediately aborts execution. Traps cannot be handled by WebAssembly code, but are reported to the outside environment, where they typically can be caught.

Functions

Code is organized into separate *functions*. Each function takes a sequence of values as parameters and returns a sequence of values as results. Functions can call each other, including recursively, resulting in an implicit call stack that cannot be accessed directly. Functions may also declare mutable *local variables* that are usable as virtual registers.

Tables

A *table* is an array of opaque values of a particular *reference type*. It allows programs to select such values indirectly through a dynamic index operand. Thereby, for example, a program can call functions indirectly through a dynamic index into a table. This allows emulating function pointers by way of table indices.

Linear Memory

A *linear memory* is a contiguous, mutable array of raw bytes. Such a memory is created with an initial size but can be grown dynamically. A program can load and store values from/to a linear memory at any byte address (including unaligned). Integer loads and stores can specify a *storage size* which is smaller than the

[6] https://ieeexplore.ieee.org/document/8766229

[7] https://ieeexplore.ieee.org/document/8766229

[8] In practice, implementations need not maintain an actual operand stack. Instead, the stack can be viewed as a set of anonymous registers that are implicitly referenced by instructions. The type system ensures that the stack height, and thus any referenced register, is always known statically.

size of the respective value type. A trap occurs if an access is not within the bounds of the current memory size.

Modules
A WebAssembly binary takes the form of a *module* that contains definitions for functions, tables, and linear memories, as well as mutable or immutable *global variables*. Definitions can also be *imported*, specifying a module/name pair and a suitable type. Each definition can optionally be *exported* under one or more names. In addition to definitions, modules can define initialization data for their memories or tables that takes the form of *segments* copied to given offsets. They can also define a *start function* that is automatically executed.

Embedder
A WebAssembly implementation will typically be *embedded* into a *host* environment. This environment defines how loading of modules is initiated, how imports are provided (including host-side definitions), and how exports can be accessed. However, the details of any particular embedding are beyond the scope of this specification, and will instead be provided by complementary, environment-specific API definitions.

1.2.2 Semantic Phases

Conceptually, the semantics of WebAssembly is divided into three phases. For each part of the language, the specification specifies each of them.

Decoding
WebAssembly modules are distributed in a *binary format*. *Decoding* processes that format and converts it into an internal representation of a module. In this specification, this representation is modelled by *abstract syntax*, but a real implementation could compile directly to machine code instead.

Validation
A decoded module has to be *valid*. Validation checks a number of well-formedness conditions to guarantee that the module is meaningful and safe. In particular, it performs *type checking* of functions and the instruction sequences in their bodies, ensuring for example that the operand stack is used consistently.

Execution
Finally, a valid module can be *executed*. Execution can be further divided into two phases:

Instantiation. A module *instance* is the dynamic representation of a module, complete with its own state and execution stack. Instantiation executes the module body itself, given definitions for all its imports. It initializes globals, memories and tables and invokes the module's start function if defined. It returns the instances of the module's exports.

Invocation. Once instantiated, further WebAssembly computations can be initiated by *invoking* an exported function on a module instance. Given the required arguments, that executes the respective function and returns its results.

Instantiation and invocation are operations within the embedding environment.

CHAPTER 2

Structure

2.1 Conventions

WebAssembly is a programming language that has multiple concrete representations (its binary format and the text format). Both map to a common structure. For conciseness, this structure is described in the form of an *abstract syntax*. All parts of this specification are defined in terms of this abstract syntax.

2.1.1 Grammar Notation

The following conventions are adopted in defining grammar rules for abstract syntax.

- Terminal symbols (atoms) are written in sans-serif font or in symbolic form: i32, nop, $\to$, $[$, $]$.
- Nonterminal symbols are written in italic font: $\mathit{valtype}$, instr.
- A^n is a sequence of $n \geq 0$ iterations of A.
- A^* is a possibly empty sequence of iterations of A. (This is a shorthand for A^n used where n is not relevant.)
- A^+ is a non-empty sequence of iterations of A. (This is a shorthand for A^n where $n \geq 1$.)
- $A^?$ is an optional occurrence of A. (This is a shorthand for A^n where $n \leq 1$.)
- Productions are written $\mathit{sym} ::= A_1 \mid \ldots \mid A_n$.
- Large productions may be split into multiple definitions, indicated by ending the first one with explicit ellipses, $\mathit{sym} ::= A_1 \mid \ldots$, and starting continuations with ellipses, $\mathit{sym} ::= \ldots \mid A_2$.
- Some productions are augmented with side conditions, "if $\mathit{condition}$", that provide a shorthand for a combinatorial expansion of the production into many separate cases.
- If the same meta variable or non-terminal symbol appears multiple times in a production, then all those occurrences must have the same instantiation. (This is a shorthand for a side condition requiring multiple different variables to be equal.)

2.1.2 Auxiliary Notation

When dealing with syntactic constructs the following notation is also used:

- ϵ denotes the empty sequence.
- $|s|$ denotes the length of a sequence s.
- $s[i]$ denotes the i-th element of a sequence s, starting from 0.

- $s[i : n]$ denotes the sub-sequence $s[i] \dots s[i + n - 1]$ of a sequence s.
- $s[[i] = A]$ denotes the same sequence as s, except that the i-th element is replaced with A.
- $s[[i : n] = A^n]$ denotes the same sequence as s, except that the sub-sequence $s[i : n]$ is replaced with A^n.
- $s_1 \oplus s_2$ denotes the sequence s_1 concatenated with s_2; this is equivalent to $s_1\ s_2$, but used for clarity.
- $\bigoplus s^*$ denotes the flattened sequence, formed by concatenating all sequences s_i in s^*.
- $A \in s$ denotes that A is a member of the sequence s, that is, s is of the form $s_1\ A\ s_2$ for some sequences s_1, s_2.

Moreover, the following conventions are employed:

- The notation x^n, where x is a non-terminal symbol, is treated as a meta variable ranging over respective sequences of x (similarly for x^*, x^+, $x^?$).
- When given a sequence x^n, then the occurrences of x in an iterated sequence $(\dots x \dots)^n$ are assumed to denote the individual elements of x^n, respectively (similarly for x^*, x^+, $x^?$). This implicitly expresses a form of mapping syntactic constructions over a sequence.
- $e^{i<n}$ denotes the same sequence as e^n, but implicitly also defines i^n to be the sequence of values 0 to $(n-1)$.

Note

For example, if x^n is the sequence $a\ b\ c$, then $(x + 1)^n$ denotes the sequence $(a + 1)\ (b + 1)\ (c + 1)$.

The form $e^{i<n}$ additionally gives access to an index variable inside the iteration. For example, $(x + i)^{i<n}$ denotes the sequence $(a + 0)\ (b + 1)\ (c + 2)$.

Productions of the following form are interpreted as *records* that map a fixed set of fields field_i to "values" A_i, respectively:

$$r \quad ::= \quad \{\mathsf{field}_1\ A_1, \mathsf{field}_2\ A_2, \dots\}$$

The following notation is adopted for manipulating such records:

- Where the type of a record is clear from context, empty fields with value ϵ are often omitted.
- $r.\mathsf{field}$ denotes the contents of the field component of r.
- $r[.\mathsf{field} = A]$ denotes the same record as r, except that the value of the field component is replaced with A.
- $r[.\mathsf{field} =\!\oplus A^*]$ denotes the same record as r, except that A^* is appended to the sequence value of the field component, that is, it is short for $r[.\mathsf{field} = r.\mathsf{field} \oplus A^*]$.
- $r_1 \oplus r_2$ denotes the composition of two identically shaped records by concatenating each field of sequences point-wise:

$$\{\mathsf{field}_1\ A_1^*, \mathsf{field}_2\ A_2^*, \dots\} \oplus \{\mathsf{field}_1\ B_1^*, \mathsf{field}_2\ B_2^*, \dots\} = \{\mathsf{field}_1\ (A_1^* \oplus B_1^*), \mathsf{field}_2\ (A_2^* \oplus B_2^*), \dots\}$$

- $\bigoplus r^*$ denotes the composition of a sequence of records, respectively; if the sequence is empty, then all fields of the resulting record are empty.

The update notation for sequences and records generalizes recursively to nested components accessed by "paths" $pth ::= ([i]\ \mid\ .\mathsf{field})^+$:

- $s[[i]pth = A]$ is short for $s[[i] = s[i][pth = A]]$,
- $r[.\mathsf{field}\ pth = A]$ is short for $r[.\mathsf{field} = r.\mathsf{field}[pth = A]]$.

2.1.3 Lists

Lists are bounded sequences of the form A^n (or A^*), where the A can either be values or complex constructions. A list can have at most $2^{32} - 1$ elements.

$$list(X) \quad ::= \quad X^* \quad \text{if } |X^*| < 2^{32}$$

2.2 Values

WebAssembly programs operate on primitive numeric *values*. Moreover, in the definition of programs, immutable sequences of values occur to represent more complex data, such as text strings or other vectors.

2.2.1 Bytes

The simplest form of value are raw uninterpreted *bytes*. In the abstract syntax they are represented as hexadecimal literals.

$$byte \quad ::= \quad \texttt{0x00} \mid \ldots \mid \texttt{0xFF}$$

Conventions

- The meta variable b ranges over bytes.
- Bytes are sometimes interpreted as natural numbers $n < 256$.

2.2.2 Integers

Different classes of *integers* with different value ranges are distinguished by their *bit width* N and by whether they are *unsigned* or *signed*.

$$\begin{array}{lcl} uN & ::= & 0 \mid \ldots \mid 2^N - 1 \\ sN & ::= & -2^{N-1} \mid \ldots \mid -1 \mid 0 \mid +1 \mid \ldots \mid +2^{N-1} - 1 \\ iN & ::= & uN \end{array}$$

The class i defines *uninterpreted* integers, whose signedness interpretation can vary depending on context. In the abstract syntax, they are represented as unsigned values. However, some operations convert them to signed based on a two's complement interpretation.

Note

The main integer types occurring in this specification are $u8$, $u32$, $u64$, and $u128$. However, other sizes occur as auxiliary constructions, e.g., in the definition of floating-point numbers.

Conventions

- The meta variables m, n, i, j range over integers.
- Numbers may be denoted by simple arithmetics, as in the grammar above. In order to distinguish arithmetics like 2^N from sequences like $(1)^N$, the latter is distinguished with parentheses.

2.2.3 Floating-Point

Floating-point data represents 32 or 64 bit values that correspond to the respective binary formats of the IEEE 754[9] standard (Section 3.3).

Every value has a *sign* and a *magnitude*. Magnitudes can either be expressed as *normal* numbers of the form $m_0 \,.\, m_1\, m_2 \,\ldots\, m_{\mathsf{m}} \cdot 2^e$, where e is the exponent and m is the *significand* whose most significant bit m_0 is 1, or as a *subnormal* number where the exponent is fixed to the smallest possible value and m_0 is 0; among the subnormals

[9] https://ieeexplore.ieee.org/document/8766229

are positive and negative zero values. Since the significands are binary values, normals are represented in the form $(1 + m \cdot 2^{-M}) \cdot 2^e$ in the abstract syntax, where M is the bit width of m; similarly for subnormals.

Possible magnitudes also include the special values ∞ (infinity) and nan (*NaN*, not a number). NaN values have a *payload* that describes the mantissa bits in the underlying binary representation. No distinction is made between signalling and quiet NaNs.

$$
\begin{array}{rcll}
fN & ::= & +fmag_N \mid -fmag_N & \\
fmag_N & ::= & (1 + m \cdot 2^{-M}) \cdot 2^e & \text{if } m < 2^M \wedge 2 - 2^{E-1} \leq e \leq 2^{E-1} - 1 \\
& \mid & (0 + m \cdot 2^{-M}) \cdot 2^e & \text{if } m < 2^M \wedge 2 - 2^{E-1} = e \\
& \mid & \infty & \\
& \mid & \mathsf{nan}(m) & \text{if } 1 \leq m < 2^M
\end{array}
$$

where $M = \mathrm{signif}(N)$ and $E = \mathrm{expon}(N)$ with

$$
\begin{array}{rcl}
\mathrm{signif}(32) & = & 23 \\
\mathrm{signif}(64) & = & 52 \\
\mathrm{expon}(32) & = & 8 \\
\mathrm{expon}(64) & = & 11
\end{array}
$$

A *canonical NaN* is a floating-point value $\pm\mathsf{nan}(\mathrm{canon}_N)$ where canon_N is a payload whose most significant bit is 1 while all others are 0:

$$
\mathrm{canon}_N = 2^{\mathrm{signif}(N)-1}
$$

An *arithmetic NaN* is a floating-point value $\pm\mathsf{nan}(m)$ with $m \geq \mathrm{canon}_N$, such that the most significant bit is 1 while all others are arbitrary.

Note

In the abstract syntax, subnormals are distinguished by the leading 0 of the significand. The exponent of subnormals has the same value as the smallest possible exponent of a normal number. Only in the binary representation the exponent of a subnormal is encoded differently than the exponent of any normal number.

The notion of canonical NaN defined here is unrelated to the notion of canonical NaN that the IEEE 754[10] standard (Section 3.5.2) defines for decimal interchange formats.

Conventions

- The meta variable z ranges over floating-point values where clear from context.
- Where clear from context, shorthands like $+1$ denote floating point values like $+(1 + 0 \cdot 2^{-M}) \cdot 2^0$.

2.2.4 Vectors

Numeric vectors are 128-bit values that are processed by vector instructions (also known as *SIMD* instructions, single instruction multiple data). They are represented in the abstract syntax using $u128$. The interpretation of lane types (integer or floating-point numbers) and lane sizes are determined by the specific instruction operating on them.

2.2.5 Names

Names are sequences of *characters*, which are *scalar values* as defined by Unicode[11] (Section 2.4).

$$
\begin{array}{rcll}
name & ::= & char^* & \text{if } |\mathrm{utf8}(char^*)| < 2^{32} \\
char & ::= & \text{U+00} \mid \ldots \mid \text{U+D7FF} \mid \text{U+E000} \mid \ldots \mid \text{U+10FFFF} &
\end{array}
$$

Due to the limitations of the binary format, the length of a name is bounded by the length of its UTF-8 encoding.

[10] https://ieeexplore.ieee.org/document/8766229
[11] https://www.unicode.org/versions/latest/

Convention

- Characters (Unicode scalar values) are sometimes used interchangeably with natural numbers $n < 1114112$.

2.3 Types

Various entities in WebAssembly are classified by types. Types are checked during validation, instantiation, and possibly execution.

2.3.1 Number Types

Number types classify numeric values.

$$\mathit{numtype} \quad ::= \quad \mathsf{i32} \mid \mathsf{i64} \mid \mathsf{f32} \mid \mathsf{f64}$$

The types $\mathsf{i32}$ and $\mathsf{i64}$ classify 32 and 64 bit integers, respectively. Integers are not inherently signed or unsigned, their interpretation is determined by individual operations.

The types $\mathsf{f32}$ and $\mathsf{f64}$ classify 32 and 64 bit floating-point data, respectively. They correspond to the respective binary floating-point representations, also known as *single* and *double* precision, as defined by the IEEE 754[12] standard (Section 3.3).

Number types are *transparent*, meaning that their bit patterns can be observed. Values of number type can be stored in memories.

Conventions

- The notation $|t|$ denotes the *bit width* of a number type t. That is, $|\mathsf{i32}| = |\mathsf{f32}| = 32$ and $|\mathsf{i64}| = |\mathsf{f64}| = 64$.

2.3.2 Vector Types

Vector types classify vectors of numeric values processed by vector instructions (also known as *SIMD* instructions, single instruction multiple data).

$$\mathit{vectype} \quad ::= \quad \mathsf{v128}$$

The type $\mathsf{v128}$ corresponds to a 128 bit vector of packed integer or floating-point data. The packed data can be interpreted as signed or unsigned integers, single or double precision floating-point values, or a single 128 bit type. The interpretation is determined by individual operations.

Vector types, like number types are *transparent*, meaning that their bit patterns can be observed. Values of vector type can be stored in memories.

Conventions

- The notation $|t|$ for bit width extends to vector types as well, that is, $|\mathsf{v128}| = 128$.

2.3.3 Type Uses

A *type use* is the use site of a type index referencing a composite type defined in a module. It classifies objects of the respective type.

$$\mathit{typeuse} \quad ::= \quad \mathit{typeidx} \mid \dots$$

The syntax of type uses is extended with additional forms for the purpose of specifying validation and execution.

[12] https://ieeexplore.ieee.org/document/8766229

2.3.4 Heap Types

Heap types classify objects in the runtime store. There are three disjoint hierarchies of heap types:

- *function types* classify functions,
- *aggregate types* classify dynamically allocated *managed* data, such as *structures*, *arrays*, or *unboxed scalars*,
- *external types* classify *external* references possibly owned by the embedder.

The values from the latter two hierarchies are interconvertible by ways of the `extern.convert_any` and `any.convert_extern` instructions. That is, both type hierarchies are inhabited by an isomorphic set of values, but may have different, incompatible representations in practice.

$$
\begin{array}{rcl}
\mathit{absheaptype} & ::= & \mathsf{any} \mid \mathsf{eq} \mid \mathsf{i31} \mid \mathsf{struct} \mid \mathsf{array} \mid \mathsf{none} \\
& \mid & \mathsf{func} \mid \mathsf{nofunc} \\
& \mid & \mathsf{exn} \mid \mathsf{noexn} \\
& \mid & \mathsf{extern} \mid \mathsf{noextern} \\
& \mid & \dots \\
\mathit{heaptype} & ::= & \mathit{absheaptype} \mid \mathit{typeuse}
\end{array}
$$

A heap type is either *abstract* or *concrete*. A concrete heap type consists of a type use that classifies an object of the respective type defined in a module. Abstract types are denoted by individual keywords.

The type `func` denotes the common supertype of all function types, regardless of their concrete definition. Dually, the type `nofunc` denotes the common subtype of all function types, regardless of their concrete definition. This type has no values.

The type `exn` denotes the common supertype of all exception references. This type has no concrete subtypes. Dually, the type `noexn` denotes the common subtype of all forms of exception references. This type has no values.

The type `extern` denotes the common supertype of all external references received through the embedder. This type has no concrete subtypes. Dually, the type `noextern` denotes the common subtype of all forms of external references. This type has no values.

The type `any` denotes the common supertype of all aggregate types, as well as possibly abstract values produced by *internalizing* an external reference of type `extern`. Dually, the type `none` denotes the common subtype of all forms of aggregate types. This type has no values.

The type `eq` is a subtype of `any` that includes all types for which references can be compared, i.e., aggregate values and `i31`.

The types `struct` and `array` denote the common supertypes of all structure and array aggregates, respectively.

The type `i31` denotes *unboxed scalars*, that is, integers injected into references. Their observable value range is limited to 31 bits.

Note

Values of type `i31` are not actually allocated in the store, but represented in a way that allows them to be mixed with actual references into the store without ambiguity. Engines need to perform some form of *pointer tagging* to achieve this, which is why one bit is reserved. Since this type is to be reliably unboxed on all hardware platforms supported by WebAssembly, it cannot be wider than 32 bits minus the tag bit.

Although the types `none`, `nofunc`, `noexn`, and `noextern` are not inhabited by any values, they can be used to form the types of all null references in their respective hierarchy. For example, `(ref null nofunc)` is the generic type of a null reference compatible with all function reference types.

The syntax of abstract heap types is extended with additional forms for the purpose of specifying validation and execution.

2.3.5 Reference Types

Reference types classify values that are first-class references to objects in the runtime store.

$$\begin{array}{lcl} \mathit{reftype} & ::= & \mathsf{ref\ null}^{?}\ \mathit{heaptype} \end{array}$$

A reference type is characterised by the heap type it points to.

In addition, a reference type of the form ref null ht is *nullable*, meaning that it can either be a proper reference to ht or null. Other references are *non-null*.

Reference types are *opaque*, meaning that neither their size nor their bit pattern can be observed. Values of reference type can be stored in tables but not in memories.

Conventions

- The reference type anyref is an abbreviation for (ref null any).
- The reference type eqref is an abbreviation for (ref null eq).
- The reference type i31ref is an abbreviation for (ref null i31).
- The reference type structref is an abbreviation for (ref null struct).
- The reference type arrayref is an abbreviation for (ref null array).
- The reference type funcref is an abbreviation for (ref null func).
- The reference type exnref is an abbreviation for (ref null exn).
- The reference type externref is an abbreviation for (ref null extern).
- The reference type nullref is an abbreviation for (ref null none).
- The reference type nullfuncref is an abbreviation for (ref null nofunc).
- The reference type nullexnref is an abbreviation for (ref null noexn).
- The reference type nullexternref is an abbreviation for (ref null noextern).

2.3.6 Value Types

Value types classify the individual values that WebAssembly code can compute with and the values that a variable accepts. They are either number types, vector types, or reference types.

$$\begin{array}{lcl} \mathit{consttype} & ::= & \mathit{numtype} \mid \mathit{vectype} \\ \mathit{valtype} & ::= & \mathit{numtype} \mid \mathit{vectype} \mid \mathit{reftype} \mid \dots \end{array}$$

The syntax of value types is extended with additional forms for the purpose of specifying validation.

Conventions

- The meta variable t ranges over value types or subclasses thereof where clear from context.

2.3.7 Result Types

Result types classify the result of executing instructions or functions, which is a sequence of values, written with brackets.

$$\begin{array}{lcl} \mathit{resulttype} & ::= & \mathit{list}(\mathit{valtype}) \end{array}$$

2.3.8 Block Types

Block types classify the *input* and *output* of structured control instructions delimiting blocks of instructions.

$$\begin{array}{rcl} \mathit{blocktype} & ::= & \mathit{valtype}^? \\ & | & \mathit{typeidx} \end{array}$$

They are given either as a type index that refers to a suitable function type reinterpreted as an instruction type, or as an optional value type inline, which is a shorthand for the instruction type $\epsilon \to \mathit{valtype}^?$.

2.3.9 Composite Types

Composite types are all types composed from simpler types, including *function types*, *structure types* and *array types*.

$$\begin{array}{rcl} \mathit{comptype} & ::= & \mathsf{struct}\ \mathit{list}(\mathit{fieldtype}) \\ & | & \mathsf{array}\ \mathit{fieldtype} \\ & | & \mathsf{func}\ \mathit{resulttype} \to \mathit{resulttype} \\ \mathit{fieldtype} & ::= & \mathsf{mut}^?\ \mathit{storagetype} \\ \mathit{storagetype} & ::= & \mathit{valtype} \mid \mathit{packtype} \\ \mathit{packtype} & ::= & \mathsf{i8} \mid \mathsf{i16} \end{array}$$

Function types classify the signature of functions, mapping a list of parameters to a list of results. They are also used to classify the inputs and outputs of instructions.

Aggregate types like structure or array types consist of a list of possibly mutable, possibly packed *field types* describing their components. Structures are heterogeneous, but require static indexing, while arrays need to be homogeneous, but allow dynamic indexing.

Conventions

- The notation $|t|$ for the bit width of a value type t extends to packed types as well, that is, $|\mathsf{i8}| = 8$ and $|\mathsf{i16}| = 16$.
- The auxiliary function unpack maps a storage type to the value type obtained when accessing a field:

$$\begin{array}{rcl} \mathrm{unpack}(\mathit{valtype}) & = & \mathit{valtype} \\ \mathrm{unpack}(\mathit{packtype}) & = & \mathsf{i32} \end{array}$$

2.3.10 Recursive Types

Recursive types denote a group of mutually recursive composite types, each of which can optionally declare a list of type uses of supertypes that it matches. Each type can also be declared *final*, preventing further subtyping.

$$\begin{array}{rcl} \mathit{rectype} & ::= & \mathsf{rec}\ \mathit{list}(\mathit{subtype}) \\ \mathit{subtype} & ::= & \mathsf{sub\ final}^?\ \mathit{typeuse}^*\ \mathit{comptype} \end{array}$$

In a module, each member of a recursive type is assigned a separate type index.

2.3.11 Address Types

Address types are a subset of number types that classify the values that can be used as offsets into memories and tables.

$$\begin{array}{rcl} \mathit{addrtype} & ::= & \mathsf{i32} \mid \mathsf{i64} \end{array}$$

Conventions

The *minimum* of two address types is defined as the address type whose bit width is the minimum of the two.

$$\begin{array}{rcll} \min(at_1, at_2) & = & at_1 & \text{if } |at_1| \leq |at_2| \\ \min(at_1, at_2) & = & at_2 & \text{otherwise} \end{array}$$

2.3.12 Limits

Limits classify the size range of resizeable storage associated with memory types and table types.

$$\mathit{limits} \quad ::= \quad [\mathit{u64} \,..\, \mathit{u64}^?]$$

If no maximum is present, then the respective storage can grow to any valid size.

2.3.13 Tag Types

Tag types classify the signature tags with a type use referring to the definition of a function type that declares the types of parameter and result values associated with the tag. The result type is empty for exception tags.

$$\mathit{tagtype} \quad ::= \quad \mathit{typeuse}$$

2.3.14 Global Types

Global types classify global variables, which hold a value and can either be mutable or immutable.

$$\mathit{globaltype} \quad ::= \quad \mathsf{mut}^? \; \mathit{valtype}$$

2.3.15 Memory Types

Memory types classify linear memories and their size range.

$$\mathit{memtype} \quad ::= \quad \mathit{addrtype} \; \mathit{limits} \; \mathsf{page}$$

The limits constrain the minimum and optionally the maximum size of a memory. The limits are given in units of page size.

2.3.16 Table Types

Table types classify tables over elements of reference type within a size range.

$$\mathit{tabletype} \quad ::= \quad \mathit{addrtype} \; \mathit{limits} \; \mathit{reftype}$$

Like memories, tables are constrained by limits for their minimum and optionally maximum size. The limits are given in numbers of entries.

2.3.17 Data Types

Data types classify data segments. Since the contents of a data segment requires no further classification, they merely consist of a universal marker ok indicating well-formedness.

$$\mathit{datatype} \quad ::= \quad \mathsf{ok}$$

2.3.18 Element Types

Element types classify element segments by the reference type of its elements.

$$\mathit{elemtype} \quad ::= \quad \mathit{reftype}$$

2.3.19 External Types

External types classify imports and external addresses with their respective types.

$$\begin{array}{lcl} \mathit{externtype} & ::= & \mathsf{tag}\ \mathit{tagtype} \mid \mathsf{global}\ \mathit{globaltype} \mid \mathsf{mem}\ \mathit{memtype} \mid \mathsf{table}\ \mathit{tabletype} \mid \mathsf{func}\ \mathit{typeuse} \end{array}$$

For functions, the type use has to refer to the definition of a function type.

Note

Future versions of WebAssembly may have additional uses for tags, and may allow non-empty result types in the function types of tags.

Conventions

The following auxiliary notation is defined for sequences of external types. It filters out entries of a specific kind in an order-preserving fashion:

$$\begin{array}{lcll} \mathrm{funcs}(\epsilon) & = & \epsilon & \\ \mathrm{funcs}((\mathsf{func}\ dt)\ xt^*) & = & dt\ \mathrm{funcs}(xt^*) & \\ \mathrm{funcs}(\mathit{externtype}\ xt^*) & = & \mathrm{funcs}(xt^*) & \text{otherwise} \\ \mathrm{tables}(\epsilon) & = & \epsilon & \\ \mathrm{tables}((\mathsf{table}\ tt)\ xt^*) & = & tt\ \mathrm{tables}(xt^*) & \\ \mathrm{tables}(\mathit{externtype}\ xt^*) & = & \mathrm{tables}(xt^*) & \text{otherwise} \\ \mathrm{mems}(\epsilon) & = & \epsilon & \\ \mathrm{mems}((\mathsf{mem}\ mt)\ xt^*) & = & mt\ \mathrm{mems}(xt^*) & \\ \mathrm{mems}(\mathit{externtype}\ xt^*) & = & \mathrm{mems}(xt^*) & \text{otherwise} \\ \mathrm{globals}(\epsilon) & = & \epsilon & \\ \mathrm{globals}((\mathsf{global}\ gt)\ xt^*) & = & gt\ \mathrm{globals}(xt^*) & \\ \mathrm{globals}(\mathit{externtype}\ xt^*) & = & \mathrm{globals}(xt^*) & \text{otherwise} \\ \mathrm{tags}(\epsilon) & = & \epsilon & \\ \mathrm{tags}((\mathsf{tag}\ jt)\ xt^*) & = & jt\ \mathrm{tags}(xt^*) & \\ \mathrm{tags}(\mathit{externtype}\ xt^*) & = & \mathrm{tags}(xt^*) & \text{otherwise} \end{array}$$

2.4 Instructions

WebAssembly code consists of sequences of *instructions*. Its computational model is based on a *stack machine* in that instructions manipulate values on an implicit *operand stack*, consuming (popping) argument values and producing or returning (pushing) result values.

In addition to dynamic operands from the stack, some instructions also have static *immediate* arguments, typically indices or type annotations, which are part of the instruction itself.

Some instructions are structured in that they contain nested sequences of instructions.

The following sections group instructions into a number of different categories.

The syntax of instruction is further extended with additional forms for the purpose of specifying execution.

2.4.1 Parametric Instructions

Instructions in this group can operate on operands of any value type.

$$\begin{array}{lcl} \mathit{instr} & ::= & \mathsf{nop} \\ & \mid & \mathsf{unreachable} \\ & \mid & \mathsf{drop} \\ & \mid & \mathsf{select}\ (\mathit{valtype}^*)^? \\ & \mid & \dots \end{array}$$

The nop instruction does nothing.

The unreachable instruction causes an unconditional trap.

The drop instruction simply throws away a single operand.

The select instruction selects one of its first two operands based on whether its third operand is zero or not. It may include a value type determining the type of these operands. If missing, the operands must be of numeric or vector type.

Note

In future versions of WebAssembly, the type annotation on select may allow for more than a single value being selected at the same time.

2.4.2 Control Instructions

Instructions in this group affect the flow of control.

$$
\begin{array}{rcl}
\mathit{instr} & ::= & \dots \\
& | & \mathsf{block}\ \mathit{blocktype}\ \mathit{instr}^* \\
& | & \mathsf{loop}\ \mathit{blocktype}\ \mathit{instr}^* \\
& | & \mathsf{if}\ \mathit{blocktype}\ \mathit{instr}^*\ \mathsf{else}\ \mathit{instr}^* \\
& | & \mathsf{br}\ \mathit{labelidx} \\
& | & \mathsf{br_if}\ \mathit{labelidx} \\
& | & \mathsf{br_table}\ \mathit{labelidx}^*\ \mathit{labelidx} \\
& | & \mathsf{br_on_null}\ \mathit{labelidx} \\
& | & \mathsf{br_on_non_null}\ \mathit{labelidx} \\
& | & \mathsf{br_on_cast}\ \mathit{labelidx}\ \mathit{reftype}\ \mathit{reftype} \\
& | & \mathsf{br_on_cast_fail}\ \mathit{labelidx}\ \mathit{reftype}\ \mathit{reftype} \\
& | & \mathsf{call}\ \mathit{funcidx} \\
& | & \mathsf{call_ref}\ \mathit{typeuse} \\
& | & \mathsf{call_indirect}\ \mathit{tableidx}\ \mathit{typeuse} \\
& | & \mathsf{return} \\
& | & \mathsf{return_call}\ \mathit{funcidx} \\
& | & \mathsf{return_call_ref}\ \mathit{typeuse} \\
& | & \mathsf{return_call_indirect}\ \mathit{tableidx}\ \mathit{typeuse} \\
& | & \mathsf{throw}\ \mathit{tagidx} \\
& | & \mathsf{throw_ref} \\
& | & \mathsf{try_table}\ \mathit{blocktype}\ \mathit{list}(\mathit{catch})\ \mathit{instr}^* \\
& | & \dots \\
\mathit{catch} & ::= & \mathsf{catch}\ \mathit{tagidx}\ \mathit{labelidx} \\
& | & \mathsf{catch_ref}\ \mathit{tagidx}\ \mathit{labelidx} \\
& | & \mathsf{catch_all}\ \mathit{labelidx} \\
& | & \mathsf{catch_all_ref}\ \mathit{labelidx}
\end{array}
$$

The block, loop, if and try_table instructions are *structured* instructions. They bracket nested sequences of instructions, called *blocks*. As the grammar prescribes, they must be well-nested.

A structured instruction can consume *input* and produce *output* on the operand stack according to its annotated block type.

Each structured control instruction introduces an implicit *label*. Labels are targets for branch instructions that reference them with label indices. Unlike with other index spaces, indexing of labels is relative by nesting depth, that is, label 0 refers to the innermost structured control instruction enclosing the referring branch instruction, while increasing indices refer to those farther out. Consequently, labels can only be referenced from *within* the associated structured control instruction. This also implies that branches can only be directed outwards, "breaking" from the block of the control construct they target. The exact effect depends on that control construct. In case of block or if it is a *forward jump*, resuming execution after the end of the block. In case of loop it is a *backward jump* to the beginning of the loop.

Note

This enforces *structured control flow*. Intuitively, a branch targeting a block or if behaves like a break statement in most C-like languages, while a branch targeting a loop behaves like a continue statement.

Branch instructions come in several flavors: br performs an unconditional branch, br_if performs a conditional branch, and br_table performs an indirect branch through an operand indexing into the label list that is an immediate to the instruction, or to a default target if the operand is out of bounds. The br_on_null and br_on_non_null instructions check whether a reference operand is null and branch if that is the case or not the case, respectively. Similarly, br_on_cast and br_on_cast_fail attempt a downcast on a reference operand and branch if that succeeds, or fails, respectively.

The return instruction is a shortcut for an unconditional branch to the outermost block, which implicitly is the body of the current function. Taking a branch *unwinds* the operand stack up to the height where the targeted structured control instruction was entered. However, branches may additionally consume operands themselves, which they push back on the operand stack after unwinding. Forward branches require operands according to the output of the targeted block's type, i.e., represent the values produced by the terminated block. Backward branches require operands according to the input of the targeted block's type, i.e., represent the values consumed by the restarted block.

The call instruction invokes another function, consuming the necessary arguments from the stack and returning the result values of the call. The call_ref instruction invokes a function indirectly through a function reference operand. The call_indirect instruction calls a function indirectly through an operand indexing into a table that is denoted by a table index and must contain function references. Since it may contain functions of heterogeneous type, the callee is dynamically checked against the function type indexed by the instruction's second immediate, and the call is aborted with a trap if it does not match.

The return_call, return_call_ref, and return_call_indirect instructions are *tail-call* variants of the previous ones. That is, they first return from the current function before actually performing the respective call. It is guaranteed that no sequence of nested calls using only these instructions can cause resource exhaustion due to hitting an implementation's limit on the number of active calls.

The instructions throw, throw_ref, and try_table are concerned with *exceptions*. The throw and throw_ref instructions raise and reraise an exception, respectively, and transfers control to the innermost enclosing exception handler that has a matching catch clause. The try_table instruction installs an exception *handler* that handles exceptions as specified by its catch clauses.

2.4.3 Variable Instructions

Variable instructions are concerned with access to local or global variables.

$$
\begin{array}{rcl}
\mathit{instr} & ::= & \dots \\
& | & \mathsf{local.get}\ \mathit{localidx} \\
& | & \mathsf{local.set}\ \mathit{localidx} \\
& | & \mathsf{local.tee}\ \mathit{localidx} \\
& | & \mathsf{global.get}\ \mathit{globalidx} \\
& | & \mathsf{global.set}\ \mathit{globalidx} \\
& | & \dots
\end{array}
$$

These instructions get or set the values of respective variables. The local.tee instruction is like local.set but also returns its argument.

2.4.4 Table Instructions

Instructions in this group are concerned with tables table.

$$
\begin{array}{rcl}
\mathit{instr} & ::= & \dots \\
& | & \mathsf{table.get}\ \mathit{tableidx} \\
& | & \mathsf{table.set}\ \mathit{tableidx} \\
& | & \mathsf{table.size}\ \mathit{tableidx} \\
& | & \mathsf{table.grow}\ \mathit{tableidx} \\
& | & \mathsf{table.fill}\ \mathit{tableidx} \\
& | & \mathsf{table.copy}\ \mathit{tableidx}\ \mathit{tableidx} \\
& | & \mathsf{table.init}\ \mathit{tableidx}\ \mathit{elemidx} \\
& | & \mathsf{elem.drop}\ \mathit{elemidx} \\
& | & \dots
\end{array}
$$

The table.get and table.set instructions load or store an element in a table, respectively.

The table.size instruction returns the current size of a table. The table.grow instruction grows table by a given delta and returns the previous size, or -1 if enough space cannot be allocated. It also takes an initialization value for the newly allocated entries.

The table.fill instruction sets all entries in a range to a given value. The table.copy instruction copies elements from a source table region to a possibly overlapping destination region; the first index denotes the destination. The table.init instruction copies elements from a passive element segment into a table.

The elem.drop instruction prevents further use of a passive element segment. This instruction is intended to be used as an optimization hint. After an element segment is dropped its elements can no longer be retrieved, so the memory used by this segment may be freed.

Note

An additional instruction that accesses a table is the control instruction call_indirect.

2.4.5 Memory Instructions

Instructions in this group are concerned with linear memory.

$$
\begin{array}{rcll}
\mathit{memarg} & ::= & \{\mathsf{align}\ u32, \mathsf{offset}\ u64\} & \\
\mathit{loadop}_{\mathrm{iN}} & ::= & sz_sx & \text{if } sz < N \\
\mathit{storeop}_{\mathrm{iN}} & ::= & sz & \text{if } sz < N \\
\mathit{vloadop}_{\mathit{vectype}} & ::= & sz \mathsf{x} M_sx & \text{if } sz \cdot M = |\mathit{vectype}|/2 \\
& | & sz_\mathsf{splat} & \\
& | & sz_\mathsf{zero} & \text{if } sz \geq 32 \\
\mathit{instr} & ::= & \dots & \\
& | & \mathit{numtype}.\mathsf{load}\mathit{loadop}^{?}_{\mathit{numtype}}\ \mathit{memidx}\ \mathit{memarg} & \\
& | & \mathit{numtype}.\mathsf{store}\mathit{storeop}^{?}_{\mathit{numtype}}\ \mathit{memidx}\ \mathit{memarg} & \\
& | & \mathit{vectype}.\mathsf{load}\mathit{vloadop}^{?}_{\mathit{vectype}}\ \mathit{memidx}\ \mathit{memarg} & \\
& | & \mathit{vectype}.\mathsf{load}sz_\mathsf{lane}\ \mathit{memidx}\ \mathit{memarg}\ \mathit{laneidx} & \\
& | & \mathit{vectype}.\mathsf{store}\ \mathit{memidx}\ \mathit{memarg} & \\
& | & \mathit{vectype}.\mathsf{store}sz_\mathsf{lane}\ \mathit{memidx}\ \mathit{memarg}\ \mathit{laneidx} & \\
& | & \mathsf{memory.size}\ \mathit{memidx} & \\
& | & \mathsf{memory.grow}\ \mathit{memidx} & \\
& | & \mathsf{memory.fill}\ \mathit{memidx} & \\
& | & \mathsf{memory.copy}\ \mathit{memidx}\ \mathit{memidx} & \\
& | & \mathsf{memory.init}\ \mathit{memidx}\ \mathit{dataidx} & \\
& | & \mathsf{data.drop}\ \mathit{dataidx} & \\
& | & \dots &
\end{array}
$$

Memory is accessed with load and store instructions for the different number types and vector types. They all take a memory index and a *memory argument* $memarg$ that contains an address *offset* and the expected *alignment* (expressed as the exponent of a power of 2).

Integer loads and stores can optionally specify a *storage size* sz that is smaller than the bit width of the respective value type. In the case of loads, a sign extension mode sx is then required to select appropriate behavior.

Vector loads can specify a shape that is half the bit width of $\mathsf{v128}$. Each lane is half its usual size, and the sign extension mode sx then specifies how the smaller lane is extended to the larger lane. Alternatively, vector loads can perform a *splat*, such that only a single lane of the specified storage size is loaded, and the result is duplicated to all lanes.

The static address offset is added to the dynamic address operand, yielding a 33-bit or 65-bit *effective address* that is the zero-based index at which the memory is accessed. All values are read and written in little endian[13] byte order. A trap results if any of the accessed memory bytes lies outside the address range implied by the memory's current size.

The memory.size instruction returns the current size of a memory. The memory.grow instruction grows a memory by a given delta and returns the previous size, or -1 if enough memory cannot be allocated. Both instructions operate in units of page size.

The memory.fill instruction sets all values in a region of memory to a given byte. The memory.copy instruction copies data from a source memory region to a possibly overlapping destination region in another or the same memory; the first index denotes the destination. The memory.init instruction copies data from a passive data segment into a memory.

The data.drop instruction prevents further use of a passive data segment. This instruction is intended to be used as an optimization hint. After a data segment is dropped its data can no longer be retrieved, so the memory used by this segment may be freed.

2.4.6 Reference Instructions

Instructions in this group are concerned with accessing references.

$$
\begin{array}{rcl}
instr & ::= & \dots \\
& | & \mathsf{ref.func}\ funcidx \\
& | & \mathsf{ref.null}\ heaptype \\
& | & \mathsf{ref.is_null} \\
& | & \mathsf{ref.as_non_null} \\
& | & \mathsf{ref.eq} \\
& | & \mathsf{ref.test}\ reftype \\
& | & \mathsf{ref.cast}\ reftype \\
& | & \dots
\end{array}
$$

The ref.null and ref.func instructions produce a null reference or a reference to a given function, respectively.

The instruction ref.is_null checks for null, while ref.as_non_null converts a nullable to a non-null one, and traps if it encounters null.

The ref.eq compares two references.

The instructions ref.test and ref.cast test the dynamic type of a reference operand. The former merely returns the result of the test, while the latter performs a downcast and traps if the operand's type does not match.

Note

The br_on_null and br_on_non_null instructions provide versions of ref.as_non_null that branch depending on the success or failure of a null test instead of trapping. Similarly, the br_on_cast and br_on_cast_fail instructions provides versions of ref.cast that branch depending on the success of the downcast instead of trapping.

An additional instruction operating on function references is the control instruction call_ref.

[13] https://en.wikipedia.org/wiki/Endianness#Little-endian

2.4.7 Aggregate Instructions

Instructions in this group are concerned with creating and accessing references to aggregate types.

$$
\begin{array}{rcl}
\mathit{instr} & ::= & \dots \\
& | & \mathsf{struct.new}\ \mathit{typeidx} \\
& | & \mathsf{struct.new_default}\ \mathit{typeidx} \\
& | & \mathsf{struct.get_}\mathit{sx}^?\ \mathit{typeidx}\ \mathit{fieldidx} \\
& | & \mathsf{struct.set}\ \mathit{typeidx}\ \mathit{fieldidx} \\
& | & \mathsf{array.new}\ \mathit{typeidx} \\
& | & \mathsf{array.new_default}\ \mathit{typeidx} \\
& | & \mathsf{array.new_fixed}\ \mathit{typeidx}\ \mathit{u32} \\
& | & \mathsf{array.new_data}\ \mathit{typeidx}\ \mathit{dataidx} \\
& | & \mathsf{array.new_elem}\ \mathit{typeidx}\ \mathit{elemidx} \\
& | & \mathsf{array.get_}\mathit{sx}^?\ \mathit{typeidx} \\
& | & \mathsf{array.set}\ \mathit{typeidx} \\
& | & \mathsf{array.len} \\
& | & \mathsf{array.fill}\ \mathit{typeidx} \\
& | & \mathsf{array.copy}\ \mathit{typeidx}\ \mathit{typeidx} \\
& | & \mathsf{array.init_data}\ \mathit{typeidx}\ \mathit{dataidx} \\
& | & \mathsf{array.init_elem}\ \mathit{typeidx}\ \mathit{elemidx} \\
& | & \mathsf{ref.i31} \\
& | & \mathsf{i31.get_}\mathit{sx} \\
& | & \mathsf{extern.convert_any} \\
& | & \mathsf{any.convert_extern} \\
& | & \dots
\end{array}
$$

The instructions struct.new and struct.new_default allocate a new structure, initializing them either with operands or with default values. The remaining instructions on structs access individual fields, allowing for different sign extension modes in the case of packed storage types.

Similarly, arrays can be allocated either with an explicit initialization operand or a default value. Furthermore, array.new_fixed allocates an array with statically fixed size, and array.new_data and array.new_elem allocate an array and initialize it from a data or element segment, respectively. The instructions array.get, array.get sx, and array.set access individual slots, again allowing for different sign extension modes in the case of a packed storage type; array.len produces the length of an array; array.fill fills a specified slice of an array with a given value and array.copy, array.init_data, and array.init_elem copy elements to a specified slice of an array from a given array, data segment, or element segment, respectively.

The instructions ref.i31 and i31.get sx convert between type i32 and an unboxed scalar.

The instructions any.convert_extern and extern.convert_any allow lossless conversion between references represented as type $(\mathsf{ref\ null\ extern})$ and as $(\mathsf{ref\ null\ any})$.

2.4.8 Numeric Instructions

Numeric instructions provide basic operations over numeric values of specific type. These operations closely match respective operations available in hardware.

$$
\begin{array}{rcll}
sz & ::= & 8 \mid 16 \mid 32 \mid 64 & \\
sx & ::= & \mathsf{u} \mid \mathsf{s} & \\
num_{\mathsf{i}N} & ::= & iN & \\
num_{\mathsf{f}N} & ::= & fN & \\
instr & ::= & \dots & \\
 & \mid & numtype.\mathsf{const}\ num_{numtype} & \\
 & \mid & numtype.unop_{numtype} & \\
 & \mid & numtype.binop_{numtype} & \\
 & \mid & numtype.testop_{numtype} & \\
 & \mid & numtype.relop_{numtype} & \\
 & \mid & numtype_1.cvtop_{numtype_2,numtype_1}_numtype_2 & \\
 & \mid & \dots & \\
unop_{\mathsf{i}N} & ::= & \mathsf{clz} \mid \mathsf{ctz} \mid \mathsf{popcnt} \mid \mathsf{extend}sz_\mathsf{s} & \text{if } sz < N \\
unop_{\mathsf{f}N} & ::= & \mathsf{abs} \mid \mathsf{neg} \mid \mathsf{sqrt} \mid \mathsf{ceil} \mid \mathsf{floor} \mid \mathsf{trunc} \mid \mathsf{nearest} & \\
binop_{\mathsf{i}N} & ::= & \mathsf{add} \mid \mathsf{sub} \mid \mathsf{mul} \mid \mathsf{div}_sx \mid \mathsf{rem}_sx & \\
 & \mid & \mathsf{and} \mid \mathsf{or} \mid \mathsf{xor} \mid \mathsf{shl} \mid \mathsf{shr}_sx \mid \mathsf{rotl} \mid \mathsf{rotr} & \\
binop_{\mathsf{f}N} & ::= & \mathsf{add} \mid \mathsf{sub} \mid \mathsf{mul} \mid \mathsf{div} \mid \mathsf{min} \mid \mathsf{max} \mid \mathsf{copysign} & \\
testop_{\mathsf{i}N} & ::= & \mathsf{eqz} & \\
relop_{\mathsf{i}N} & ::= & \mathsf{eq} \mid \mathsf{ne} \mid \mathsf{lt}_sx \mid \mathsf{gt}_sx \mid \mathsf{le}_sx \mid \mathsf{ge}_sx & \\
relop_{\mathsf{f}N} & ::= & \mathsf{eq} \mid \mathsf{ne} \mid \mathsf{lt} \mid \mathsf{gt} \mid \mathsf{le} \mid \mathsf{ge} & \\
cvtop_{\mathsf{i}N_1,\mathsf{i}N_2} & ::= & \mathsf{extend}_sx & \text{if } N_1 < N_2 \\
 & \mid & \mathsf{wrap} & \text{if } N_1 > N_2 \\
cvtop_{\mathsf{i}N_1,\mathsf{f}N_2} & ::= & \mathsf{convert}_sx & \\
 & \mid & \mathsf{reinterpret} & \text{if } N_1 = N_2 \\
cvtop_{\mathsf{f}N_1,\mathsf{i}N_2} & ::= & \mathsf{trunc}_sx & \\
 & \mid & \mathsf{trunc_sat}_sx & \\
 & \mid & \mathsf{reinterpret} & \text{if } N_1 = N_2 \\
cvtop_{\mathsf{f}N_1,\mathsf{f}N_2} & ::= & \mathsf{promote} & \text{if } N_1 < N_2 \\
 & \mid & \mathsf{demote} & \text{if } N_1 > N_2
\end{array}
$$

Numeric instructions are divided by number type. For each type, several subcategories can be distinguished:

- *Constants*: return a static constant.
- *Unary operations*: consume one operand and produce one result of the respective type.
- *Binary operations*: consume two operands and produce one result of the respective type.
- *Tests*: consume one operand of the respective type and produce a Boolean integer result.
- *Comparisons*: consume two operands of the respective type and produce a Boolean integer result.
- *Conversions*: consume a value of one type and produce a result of another (the source type of the conversion is the one after the "_").

Some integer instructions come in two flavors, where a signedness annotation sx distinguishes whether the operands are to be interpreted as unsigned or signed integers. For the other integer instructions, the use of two's complement for the signed interpretation means that they behave the same regardless of signedness.

2.4.9 Vector Instructions

Vector instructions (also known as *SIMD* instructions, *single instruction multiple data*) provide basic operations over values of vector type.

$$
\begin{array}{rcll}
\mathit{lanetype} & ::= & \mathit{numtype} \mid \mathit{packtype} & \\
\mathit{dim} & ::= & 1 \mid 2 \mid 4 \mid 8 \mid 16 & \\
\mathit{shape} & ::= & \mathit{lanetype}\mathsf{x}\mathit{dim} & \text{if } |\mathit{lanetype}| \cdot \mathit{dim} = 128 \\
\mathit{ishape} & ::= & \mathit{shape} & \text{if } \mathrm{lanetype}(\mathit{shape}) = \mathsf{i}N \\
\mathit{bshape} & ::= & \mathit{shape} & \text{if } \mathrm{lanetype}(\mathit{shape}) = \mathsf{i8} \\
\mathit{half} & ::= & \mathsf{low} \mid \mathsf{high} & \\
\mathit{zero} & ::= & \mathsf{zero} & \\
\mathit{laneidx} & ::= & u8 & \\
\mathit{instr} & ::= & \ldots & \\
& \mid & \mathit{vectype}.\mathsf{const}\ \mathit{vec}_{\mathit{vectype}} & \\
& \mid & \mathit{vectype}.\mathit{vvunop} & \\
& \mid & \mathit{vectype}.\mathit{vvbinop} & \\
& \mid & \mathit{vectype}.\mathit{vvternop} & \\
& \mid & \mathit{vectype}.\mathit{vvtestop} & \\
& \mid & \mathit{shape}.\mathit{vunop}_{\mathit{shape}} & \\
& \mid & \mathit{shape}.\mathit{vbinop}_{\mathit{shape}} & \\
& \mid & \mathit{shape}.\mathit{vternop}_{\mathit{shape}} & \\
& \mid & \mathit{shape}.\mathit{vtestop}_{\mathit{shape}} & \\
& \mid & \mathit{shape}.\mathit{vrelop}_{\mathit{shape}} & \\
& \mid & \mathit{ishape}.\mathit{vshiftop}_{\mathit{ishape}} & \\
& \mid & \mathit{ishape}.\mathsf{bitmask} & \\
& \mid & \mathit{bshape}.\mathit{vswizzlop}_{\mathit{bshape}} & \\
& \mid & \mathit{bshape}.\mathsf{shuffle}\ \mathit{laneidx}^* & \text{if } |\mathit{laneidx}^*| = \dim(\mathit{bshape}) \\
& \mid & \mathit{ishape}_1.\mathit{vextunop}_{\mathit{ishape}_2,\mathit{ishape}_1_\mathit{ishape}_2} & \\
& \mid & \mathit{ishape}_1.\mathit{vextbinop}_{\mathit{ishape}_2,\mathit{ishape}_1_\mathit{ishape}_2} & \\
& \mid & \mathit{ishape}_1.\mathit{vextternop}_{\mathit{ishape}_2,\mathit{ishape}_1_\mathit{ishape}_2} & \\
& \mid & \mathit{ishape}_1.\mathsf{narrow}_\mathit{ishape}_2_\mathit{sx} & \text{if } |\mathrm{lanetype}(\mathit{ishape}_2)| = 2 \cdot |\mathrm{lanetype}(\mathit{ishape}_1)| \leq 32 \\
& \mid & \mathit{shape}_1.\mathit{vcvtop}_{\mathit{shape}_2,\mathit{shape}_1_\mathit{shape}_2} & \\
& \mid & \mathit{shape}.\mathsf{splat} & \\
& \mid & \mathit{shape}.\mathsf{extract_lane}_\mathit{sx}^?\ \mathit{laneidx} & \text{if } \mathit{sx}^? \neq \epsilon \Leftrightarrow \mathrm{lanetype}(\mathit{shape}) \neq \mathrm{unpack}(\mathrm{lanetype}(\mathit{shape})) \\
& \mid & \mathit{shape}.\mathsf{replace_lane}\ \mathit{laneidx} & \\
& \mid & \ldots &
\end{array}
$$

Vector instructions have a naming convention involving a *shape* prefix that determines how their operands will be interpreted, written $t\mathsf{x}N$, and consisting of a *lane type* t, a possibly *packed* numeric type, and its *dimension* N, which denotes the number of lanes of that type. Operations are performed point-wise on the values of each lane.

Instructions prefixed with v128 do not involve a specific interpretation, and treat the v128 as either an $i128$ value or a vector of 128 individual bits.

Note

For example, the shape i32x4 interprets the operand as four $i32$ values, packed into an $i128$. The bit width of the lane type t times N always is 128.

$vvunop$	::=	not	
$vvbinop$	::=	and | andnot | or | xor	
$vvternop$	::=	bitselect	
$vvtestop$	::=	any_true	
$vunop_{\mathsf{i}N\times M}$	::=	abs | neg	
	|	popcnt	if $N = 8$
$vunop_{\mathsf{f}N\times M}$	::=	abs | neg | sqrt	
	|	ceil | floor | trunc | nearest	
$vbinop_{\mathsf{i}N\times M}$	::=	add	
	|	sub	
	|	add_sat_sx	if $N \leq 16$
	|	sub_sat_sx	if $N \leq 16$
	|	mul	if $N \geq 16$
	|	avgr_u	if $N \leq 16$
	|	q15mulr_sat_s	if $N = 16$
	|	relaxed_q15mulr_s	if $N = 16$
	|	min_sx	if $N \leq 32$
	|	max_sx	if $N \leq 32$
$vbinop_{\mathsf{f}N\times M}$	::=	add | sub | mul | div	
	|	min | max | pmin | pmax	
	|	relaxed_min | relaxed_max	
$vternop_{\mathsf{i}N\times M}$	::=	relaxed_laneselect	
$vternop_{\mathsf{f}N\times M}$	::=	relaxed_madd | relaxed_nmadd	
$vtestop_{\mathsf{i}N\times M}$	::=	all_true	
$vrelop_{\mathsf{i}N\times M}$	::=	eq | ne	
	|	lt_sx	if $N \neq 64 \vee sx = \mathsf{s}$
	|	gt_sx	if $N \neq 64 \vee sx = \mathsf{s}$
	|	le_sx	if $N \neq 64 \vee sx = \mathsf{s}$
	|	ge_sx	if $N \neq 64 \vee sx = \mathsf{s}$
$vrelop_{\mathsf{f}N\times M}$	::=	eq | ne | lt | gt | le | ge	
$vswizzlop_{\mathsf{i8}\times M}$	::=	swizzle | relaxed_swizzle	
$vshiftop_{\mathsf{i}N\times M}$	::=	shl | shr_sx	
$vextunop_{\mathsf{i}N_1\times M_1,\mathsf{i}N_2\times M_2}$	::=	extadd_pairwise_sx	if $16 \leq 2 \cdot N_1 = N_2 \leq 32$
$vextbinop_{\mathsf{i}N_1\times M_1,\mathsf{i}N_2\times M_2}$	::=	extmul_$half$_sx	if $2 \cdot N_1 = N_2 \geq 16$
	|	dot_s	if $2 \cdot N_1 = N_2 = 32$
	|	relaxed_dot_s	if $2 \cdot N_1 = N_2 = 16$
$vextternop_{\mathsf{i}N_1\times M_1,\mathsf{i}N_2\times M_2}$	::=	relaxed_dot_add_s	if $4 \cdot N_1 = N_2 = 32$
$vcvtop_{\mathsf{i}N_1\times M_1,\mathsf{i}N_2\times M_2}$	::=	extend_$half$_sx	if $N_2 = 2 \cdot N_1$
$vcvtop_{\mathsf{i}N_1\times M_1,\mathsf{f}N_2\times M_2}$	::=	convert_$half^?$_sx	if $N_2 = N_1 = 32 \wedge half^? = \epsilon \vee N_2 = 2 \cdot N_1 \wedge half^? = \mathsf{low}$
$vcvtop_{\mathsf{f}N_1\times M_1,\mathsf{i}N_2\times M_2}$	::=	trunc_sat_sx_$zero^?$	if $N_1 = N_2 = 32 \wedge zero^? = \epsilon \vee N_1 = 2 \cdot N_2 \wedge zero^? = \mathsf{zero}$
	|	relaxed_trunc_sx_$zero^?$	if $N_1 = N_2 = 32 \wedge zero^? = \epsilon \vee N_1 = 2 \cdot N_2 \wedge zero^? = \mathsf{zero}$
$vcvtop_{\mathsf{f}N_1\times M_1,\mathsf{f}N_2\times M_2}$	::=	demote_zero	if $N_1 = 2 \cdot N_2$
	|	promote_low	if $2 \cdot N_1 = N_2$

Vector instructions can be grouped into several subcategories:

- *Constants*: return a static constant.
- *Unary Operations*: consume one v128 operand and produce one v128 result.
- *Binary Operations*: consume two v128 operands and produce one v128 result.
- *Ternary Operations*: consume three v128 operands and produce one v128 result.
- *Tests*: consume one v128 operand and produce a Boolean integer result.

- *Shifts*: consume a v128 operand and an i32 operand, producing one v128 result.
- *Splats*: consume a value of numeric type and produce a v128 result of a specified shape.
- *Extract lanes*: consume a v128 operand and return the numeric value in a given lane.
- *Replace lanes*: consume a v128 operand and a numeric value for a given lane, and produce a v128 result.

Some vector instructions have a signedness annotation sx which distinguishes whether the elements in the operands are to be interpreted as unsigned or signed integers. For the other vector instructions, the use of two's complement for the signed interpretation means that they behave the same regardless of signedness.

Conventions

- The function $\mathrm{lanetype}(shape)$ extracts the lane type of a shape.
- The function $\mathrm{dim}(shape)$ extracts the dimension of a shape.
- The function $\mathrm{zeroop}(vcvtop)$ extracts the zero flag from a vector conversion operator, or returns ϵ if it does not contain any.
- The function $\mathrm{halfop}(vcvtop)$ extracts the $half$ flag from a vector conversion operator, or returns ϵ if it does not contain any.

2.4.10 Expressions

Function bodies, initialization values for globals, elements and offsets of element segments, and offsets of data segments are given as expressions, which are sequences of instructions.

$$expr \quad ::= \quad instr^*$$

In some places, validation restricts expressions to be *constant*, which limits the set of allowable instructions.

2.5 Modules

WebAssembly programs are organized into *modules*, which are the unit of deployment, loading, and compilation. A module collects definitions for types, tags, and globals, memories, tables, functions. In addition, it can declare imports and exports and provide initialization in the form of data and element segments, or a start function.

$$\begin{array}{lll} module & ::= & \mathsf{module} \\ & & list(type) \\ & & list(import) \\ & & list(tag) \\ & & list(global) \\ & & list(mem) \\ & & list(table) \\ & & list(func) \\ & & list(data) \\ & & list(elem) \\ & & start^? \\ & & list(export) \end{array}$$

Each of the lists — and thus the entire module — may be empty.

2.5.1 Indices

Definitions are referenced with zero-based *indices*. Each class of definition has its own *index space*, as distinguished by the following classes.

$$\begin{array}{rcl} idx & ::= & u32 \\ typeidx & ::= & idx \\ funcidx & ::= & idx \\ globalidx & ::= & idx \\ tableidx & ::= & idx \\ memidx & ::= & idx \\ tagidx & ::= & idx \\ elemidx & ::= & idx \\ dataidx & ::= & idx \\ labelidx & ::= & idx \\ localidx & ::= & idx \\ fieldidx & ::= & idx \end{array}$$

The index space for tags, globals, memories, tables, and functions includes respective imports declared in the same module. The indices of these imports precede the indices of other definitions in the same index space.

Data indices reference data segments and element indices reference element segments.

The index space for locals is only accessible inside a function and includes the parameters of that function, which precede the local variables.

Label indices reference structured control instructions inside an instruction sequence.

Each aggregate type provides an index space for its fields.

Conventions

- The meta variable l ranges over label indices.
- The meta variables x, y range over indices in any of the other index spaces.
- For every index space $abcidx$, the notation $\mathrm{abcidx}(A)$ denotes the set of indices from that index space occurring free in A. Sometimes this set is reinterpreted as the list of its elements.

Note

For example, if $instr^*$ is $(\mathsf{data.drop}\ 1)\ (\mathsf{memory.init}\ 2\ 3)$, then $\mathrm{dataidx}(instr^*) = 1\ 3$, or equivalently, the set $\{1, 3\}$.

2.5.2 Types

The $type$ section of a module defines a list of recursive types, each consisting of a list of sub types referenced by individual type indices. All function, structure, or array types used in a module must be defined in this section.

$$type \quad ::= \quad \mathsf{type}\ rectype$$

2.5.3 Tags

The tag section of a module defines a list of *tags*:

$$tag \quad ::= \quad \mathsf{tag}\ tagtype$$

The type index of a tag must refer to a function type that declares its tag type.

Tags are referenced through tag indices, starting with the smallest index not referencing a tag import.

2.5.4 Globals

The *global* section of a module defines a list of *global variables* (or *globals* for short):

$$\mathit{global} \quad ::= \quad \text{global}\ \mathit{globaltype}\ \mathit{expr}$$

Each global stores a single value of the type specified in the global type. It also specifies whether a global is immutable or mutable. Moreover, each global is initialized with a value given by a constant initializer expression.

Globals are referenced through global indices, starting with the smallest index not referencing a global import.

2.5.5 Memories

The *mem* section of a module defines a list of *linear memories* (or *memories* for short) as described by their memory type:

$$\mathit{mem} \quad ::= \quad \text{memory}\ \mathit{memtype}$$

A memory is a list of raw uninterpreted bytes. The minimum size in the limits of its memory type specifies the initial size of that memory, while its maximum, if present, restricts the size to which it can grow later. Both are in units of page size.

Memories can be initialized through data segments.

Memories are referenced through memory indices, starting with the smallest index not referencing a memory import. Most constructs implicitly reference memory index 0.

2.5.6 Tables

The *table* section of a module defines a list of *tables* described by their table type:

$$\mathit{table} \quad ::= \quad \text{table}\ \mathit{tabletype}\ \mathit{expr}$$

A table is an array of opaque values of a particular reference type that is specified by the table type. Each table slot is initialized with a value given by a constant initializer expression. Tables can further be initialized through element segments.

The minimum size in the limits of the table type specifies the initial size of that table, while its maximum restricts the size to which it can grow later.

Tables are referenced through table indices, starting with the smallest index not referencing a table import. Most constructs implicitly reference table index 0.

2.5.7 Functions

The *func* section of a module defines a list of *functions* with the following structure:

$$\begin{array}{lcl} \mathit{func} & ::= & \text{func}\ \mathit{typeidx}\ \mathit{local}^*\ \mathit{expr} \\ \mathit{local} & ::= & \text{local}\ \mathit{valtype} \end{array}$$

The type index of a function declares its signature by reference to a function type defined in the module. The parameters of the function are referenced through 0-based local indices in the function's body; they are mutable.

The locals declare a list of mutable local variables and their types. These variables are referenced through local indices in the function's body. The index of the first local is the smallest index not referencing a parameter.

A function's expression is an instruction sequence that represents the body of the function. Upon termination it must produce a stack matching the function type's result type.

Functions are referenced through function indices, starting with the smallest index not referencing a function import.

2.5.8 Data Segments

The *data* section of a module defines a list of *data segments*, which can be used to initialize a range of memory from a static list of bytes.

$$\begin{array}{rcl} \mathit{data} & ::= & \mathsf{data}\ \mathit{byte}^*\ \mathit{datamode} \\ \mathit{datamode} & ::= & \mathsf{active}\ \mathit{memidx}\ \mathit{expr} \mid \mathsf{passive} \end{array}$$

Similar to element segments, data segments have a mode that identifies them as either *active* or *passive*. A passive data segment's contents can be copied into a memory using the memory.init instruction. An active data segment copies its contents into a memory during instantiation, as specified by a memory index and a constant expression defining an offset into that memory.

Data segments are referenced through data indices.

2.5.9 Element Segments

The *elem* section of a module defines a list of *element segments*, which can be used to initialize a subrange of a table from a static list of elements.

$$\begin{array}{rcl} \mathit{elem} & ::= & \mathsf{elem}\ \mathit{reftype}\ \mathit{expr}^*\ \mathit{elemmode} \\ \mathit{elemmode} & ::= & \mathsf{active}\ \mathit{tableidx}\ \mathit{expr} \mid \mathsf{passive} \mid \mathsf{declare} \end{array}$$

Each element segment defines a reference type and a corresponding list of constant element expressions.

Element segments have a mode that identifies them as either *active*, *passive*, or *declarative*. A passive element segment's elements can be copied to a table using the table.init instruction. An active element segment copies its elements into a table during instantiation, as specified by a table index and a constant expression defining an offset into that table. A declarative element segment is not available at runtime but merely serves to forward-declare references that are formed in code with instructions like ref.func. The offset is given by another constant expression.

Element segments are referenced through element indices.

2.5.10 Start Function

The *start* section of a module declares the function index of a *start function* that is automatically invoked when the module is instantiated, after tables and memories have been initialized.

$$\mathit{start} ::= \mathsf{start}\ \mathit{funcidx}$$

Note

The start function is intended for initializing the state of a module. The module and its exports are not accessible externally before this initialization has completed.

2.5.11 Imports

The *import* section of a module defines a set of *imports* that are required for instantiation.

$$\mathit{import} ::= \mathsf{import}\ \mathit{name}\ \mathit{name}\ \mathit{externtype}$$

Each import is labeled by a two-level name space, consisting of a *module name* and an *item name* for an entity within that module. Importable definitions are tags, globals, memories, tables, and functions. Each import is specified by a respective external type that a definition provided during instantiation is required to match.

Every import defines an index in the respective index space. In each index space, the indices of imports go before the first index of any definition contained in the module itself.

Note

Unlike export names, import names are not necessarily unique. It is possible to import the same module/item name pair multiple times; such imports may even have different type descriptions, including different kinds of entities. A module with such imports can still be instantiated depending on the specifics of how an embedder allows resolving and supplying imports. However, embedders are not required to support such overloading, and a WebAssembly module itself cannot implement an overloaded name.

2.5.12 Exports

The *export* section of a module defines a set of *exports* that become accessible to the host environment once the module has been instantiated.

$$
\begin{array}{rcl}
\mathit{export} & ::= & \mathsf{export}\ \mathit{name}\ \mathit{externidx} \\
\mathit{externidx} & ::= & \mathsf{func}\ \mathit{funcidx} \mid \mathsf{global}\ \mathit{globalidx} \mid \mathsf{table}\ \mathit{tableidx} \mid \mathsf{memory}\ \mathit{memidx} \mid \mathsf{tag}\ \mathit{tagidx}
\end{array}
$$

Each export is labeled by a unique name. Exportable definitions are tags, globals, memories, tables, and functions, which are referenced through a respective index.

Conventions

The following auxiliary notation is defined for sequences of exports, filtering out indices of a specific kind in an order-preserving fashion:

$$
\begin{array}{lcll}
\mathrm{funcs}(\epsilon) & = & \epsilon & \\
\mathrm{funcs}((\mathsf{func}\ x)\ xx^*) & = & x\ \mathrm{funcs}(xx^*) & \\
\mathrm{funcs}(\mathit{externidx}\ xx^*) & = & \mathrm{funcs}(xx^*) & \text{otherwise} \\
\mathrm{tables}(\epsilon) & = & \epsilon & \\
\mathrm{tables}((\mathsf{table}\ x)\ xx^*) & = & x\ \mathrm{tables}(xx^*) & \\
\mathrm{tables}(\mathit{externidx}\ xx^*) & = & \mathrm{tables}(xx^*) & \text{otherwise} \\
\mathrm{mems}(\epsilon) & = & \epsilon & \\
\mathrm{mems}((\mathsf{memory}\ x)\ xx^*) & = & x\ \mathrm{mems}(xx^*) & \\
\mathrm{mems}(\mathit{externidx}\ xx^*) & = & \mathrm{mems}(xx^*) & \text{otherwise} \\
\mathrm{globals}(\epsilon) & = & \epsilon & \\
\mathrm{globals}((\mathsf{global}\ x)\ xx^*) & = & x\ \mathrm{globals}(xx^*) & \\
\mathrm{globals}(\mathit{externidx}\ xx^*) & = & \mathrm{globals}(xx^*) & \text{otherwise} \\
\mathrm{tags}(\epsilon) & = & \epsilon & \\
\mathrm{tags}((\mathsf{tag}\ x)\ xx^*) & = & x\ \mathrm{tags}(xx^*) & \\
\mathrm{tags}(\mathit{externidx}\ xx^*) & = & \mathrm{tags}(xx^*) & \text{otherwise}
\end{array}
$$

CHAPTER 3

Validation

3.1 Conventions

Validation checks that a WebAssembly module is well-formed. Only valid modules can be instantiated.

Validity is defined by a *type system* over the abstract syntax of a module and its contents. For each piece of abstract syntax, there is a typing rule that specifies the constraints that apply to it. All rules are given in two *equivalent* forms:

1. In *prose*, describing the meaning in intuitive form.
2. In *formal notation*, describing the rule in mathematical form.[14]

Note

The prose and formal rules are equivalent, so that understanding of the formal notation is *not* required to read this specification. The formalism offers a more concise description in notation that is used widely in programming languages semantics and is readily amenable to mathematical proof.

In both cases, the rules are formulated in a *declarative* manner. That is, they only formulate the constraints, they do not define an algorithm. The skeleton of a sound and complete algorithm for type-checking instruction sequences according to this specification is provided in the appendix.

3.1.1 Types

To define the semantics, the definition of some sorts of types is extended to include additional forms. By virtue of not being representable in either the binary format or the text format, these forms cannot be used in a program; they only occur during validation or execution.

$$
\begin{array}{rcl}
\mathit{valtype} & ::= & \ldots \mid \mathsf{bot} \\
\mathit{absheaptype} & ::= & \ldots \mid \mathsf{bot} \\
\mathit{typeuse} & ::= & \ldots \mid \mathit{deftype} \mid \mathsf{rec}.n
\end{array}
$$

The unique value type bot is a *bottom type* that matches all value types. Similarly, bot is also used as a bottom type of all heap types.

[14] The semantics is derived from the following article: Andreas Haas, Andreas Rossberg, Derek Schuff, Ben Titzer, Dan Gohman, Luke Wagner, Alon Zakai, JF Bastien, Michael Holman. Bringing the Web up to Speed with WebAssembly[15]. Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017). ACM 2017.

[15] https://dl.acm.org/citation.cfm?doid=3062341.3062363

Note

No validation rule uses bottom types explicitly, but various rules can pick any value or heap type, including bottom. This ensures the existence of principal types, and thus a validation algorithm without back tracking.

A type use can consist directly of a defined type. This occurs as the result of substituting a type index with its definition.

A type use may also be a *recursive type index*. Such an index refers to the i-th component of a surrounding recursive type. It occurs as the result of rolling up the definition of a recursive type.

Both extensions affect occurrences of type uses in concrete heap types, in sub types and in instructions.

A type of any form is *closed* when it does not contain a heap type that is a type index or a recursive type index without a surrounding recursive type, i.e., all type indices have been substituted with their defined type and all free recursive type indices have been unrolled.

Note

It is an invariant of the semantics that sub types occur only in one of two forms: either as "syntactic" types as in a source module, where all supertypes are type indices, or as "semantic" types, where all supertypes are resolved to either defined types or recursive type indices.

Recursive type indices are local to a recursive type. They are distinguished from regular type indices and represented such that two closed types are syntactically equal if and only if they have the same recursive structure.

Convention

- The *difference* $rt_1 \setminus rt_2$ between two reference types is defined as follows:

$$\begin{array}{lcl} (\mathsf{ref}\ \mathsf{null}^?_1\ ht_1) \setminus (\mathsf{ref}\ \mathsf{null}\ ht_2) & = & (\mathsf{ref}\ ht_1) \\ (\mathsf{ref}\ \mathsf{null}^?_1\ ht_1) \setminus (\mathsf{ref}\ ht_2) & = & (\mathsf{ref}\ \mathsf{null}^?_1\ ht_1) \end{array}$$

Note

This definition computes an approximation of the reference type that is inhabited by all values from rt_1 except those from rt_2. Since the type system does not have general union types, the definition only affects the presence of null and cannot express the absence of other values.

3.1.2 Defined Types

Defined types denote the individual types defined in a module. Each such type is represented as a projection from the recursive type group it originates from, indexed by its position in that group.

$$\mathit{deftype} \quad ::= \quad \mathit{rectype}.n$$

Defined types do not occur in the binary or text format, but are formed by rolling up the recursive types defined in a module.

Note

It is an invariant of the semantics that all recursive types occurring in defined types are rolled up.

Conventions

- $t[x^* := dt^*]$ denotes the parallel *substitution* of type indices x^* with corresponding defined types dt^* in type t, provided $|x^*| = |dt^*|$.
- $t[(\mathsf{rec}\ i)^* := dt^*]$ denotes the parallel substitution of recursive type indices $(\mathsf{rec}\ i)^*$ with defined types dt^* in type t, provided $|(\mathsf{rec}\ i)^*| = |dt^*|$. This substitution does not proceed under recursive types, since they are considered local *binders* for all recursive type indices.
- $t[:= dt^*]$ is shorthand for the substitution $t[x^* := dt^*]$, where $x^* = 0\ \ldots\ (|dt^*| - 1)$.

Note

All recursive types formed by the semantics are closed with respect to recursive type indices that occur inside them. Hence, substitution of recursive type indices never needs to modify the bodies of recursive types. In addition, all types used for substitution are closed with respect to recursive type indices, such that name capture of recursive type indices cannot occur.

3.1.3 Rolling and Unrolling

In order to allow comparing recursive types for equivalence, their representation is changed such that all type indices internal to the same recursive type are replaced by recursive type indices.

Note

This representation is independent of the type index space, so that it is meaningful across module boundaries. Moreover, this representation ensures that types with equivalent recursive structure are also syntactically equal, hence allowing a simple equality check on (closed) types. It gives rise to an *iso-recursive* interpretation of types.

The representation change is performed by two auxiliary operations on the syntax of recursive types:

- *Rolling up* a recursive type substitutes its internal type indices with corresponding recursive type indices.
- *Unrolling* a recursive type substitutes its recursive type indices with the corresponding defined types.

These operations are extended to defined types and defined as follows:

$$
\begin{array}{lcll}
\mathrm{roll}_x(rectype) & = & \mathsf{rec}\ (subtype[(x+i)^{i<n} := (\mathsf{rec}.i)^{i<n}])^n & \text{if } rectype = \mathsf{rec}\ subtype^n \\
\mathrm{unroll}(rectype) & = & \mathsf{rec}\ (subtype[(\mathsf{rec}.i)^{i<n} := (rectype.i)^{i<n}])^n & \text{if } rectype = \mathsf{rec}\ subtype^n \\
\mathrm{roll}^*_x(rectype) & = & ((\mathsf{rec}\ subtype^n).i)^{i<n} & \text{if } \mathrm{roll}_x(rectype) = \mathsf{rec}\ subtype^n \\
\mathrm{unroll}(rectype.i) & = & subtype^*[i] & \text{if } \mathrm{unroll}(rectype) = \mathsf{rec}\ subtype^*
\end{array}
$$

In addition, the following auxiliary relation denotes the *expansion* of a defined type or type use:

$$
\begin{array}{lcll}
deftype & \approx & comptype & \text{if } \mathrm{unroll}(deftype) = \mathsf{sub}\ \mathsf{final}^?\ typeuse^*\ comptype \\
deftype & \approx_C & comptype & \text{if } deftype \approx comptype \\
typeidx & \approx_C & comptype & \text{if } C.\mathsf{types}[typeidx] \approx comptype
\end{array}
$$

3.1.4 Instruction Types

Instruction types classify the behaviour of instructions or instruction sequences, by describing how they manipulate the operand stack and the initialization status of locals:

$$
instrtype \quad ::= \quad resulttype \to_{localidx^*} resulttype
$$

An instruction type $t_1^* \to_{x^*} t_2^*$ describes the required input stack with argument values of types t_1^* that an instruction pops off and the provided output stack with result values of types t_2^* that it pushes back. Moreover, it enumerates the indices x^* of locals that have been set by the instruction or sequence.

Note

Instruction types are only used for validation, they do not occur in programs.

3.1.5 Local Types

Local types classify locals, by describing their value type as well as their *initialization status*:

$$\begin{array}{rcl} \mathit{localtype} & ::= & \mathit{init}\ \mathit{valtype} \\ \mathit{init} & ::= & \mathsf{set} \mid \mathsf{unset} \end{array}$$

Note

Local types are only used for validation, they do not occur in programs.

3.1.6 Contexts

Validity of an individual definition is specified relative to a *context*, which collects relevant information about the surrounding module and the definitions in scope:

- *Types*: the list of types defined in the current module.
- *Functions*: the list of functions declared in the current module, represented by a defined type that expands to their function type.
- *Tables*: the list of tables declared in the current module, represented by their table type.
- *Memories*: the list of memories declared in the current module, represented by their memory type.
- *Globals*: the list of globals declared in the current module, represented by their global type.
- *Tags*: the list of tags declared in the current module, represented by their tag type.
- *Element Segments*: the list of element segments declared in the current module, represented by the elements' reference type.
- *Data Segments*: the list of data segments declared in the current module, each represented by an ok entry.
- *Locals*: the list of locals declared in the current function (including parameters), represented by their local type.
- *Labels*: the stack of labels accessible from the current position, represented by their result type.
- *Return*: the return type of the current function, represented as an optional result type that is absent when no return is allowed, as in free-standing expressions.
- *References*: the list of function indices that occur in the module outside functions and can hence be used to form references inside them.

In other words, a context contains a sequence of suitable types for each index space, describing each defined entry in that space. Locals, labels and return type are only used for validating instructions in function bodies, and are left empty elsewhere. The label stack is the only part of the context that changes as validation of an instruction sequence proceeds.

More concretely, contexts are defined as records C with abstract syntax:

$$
\begin{array}{lll}
\mathit{context} & ::= & \{\mathsf{types}\ \mathit{deftype}^* \\
& & \ \mathsf{tags}\ \mathit{tagtype}^* \\
& & \ \mathsf{globals}\ \mathit{globaltype}^* \\
& & \ \mathsf{mems}\ \mathit{memtype}^* \\
& & \ \mathsf{tables}\ \mathit{tabletype}^* \\
& & \ \mathsf{funcs}\ \mathit{deftype}^* \\
& & \ \mathsf{datas}\ \mathit{datatype}^* \\
& & \ \mathsf{elems}\ \mathit{elemtype}^* \\
& & \ \mathsf{locals}\ \mathit{localtype}^* \\
& & \ \mathsf{labels}\ \mathit{resulttype}^* \\
& & \ \mathsf{return}\ \mathit{resulttype}^? \\
& & \ \mathsf{refs}\ \mathit{funcidx}^* \\
& & \ \dots\}
\end{array}
$$

Note

The definition of contexts needs to be extended with additional fields for the purpose of proving type soundness.

Convention

A type of any shape can be *closed* to bring it into closed form relative to a context it is valid in, by substituting each type index x occurring in it with its own corresponding defined type $C.\mathsf{types}[x]$, after first closing the types in $C.\mathsf{types}$ themselves.

$$
\begin{array}{lcll}
\mathsf{clos}_C(t) & = & t[:= dt^*] & \text{if } dt^* = \mathsf{clos}^*(C.\mathsf{types}) \\
\mathsf{clos}^*(\epsilon) & = & \epsilon & \\
\mathsf{clos}^*(dt^*\ dt_n) & = & dt'^*\ dt_n[:= dt'^*] & \text{if } dt'^* = \mathsf{clos}^*(dt^*)
\end{array}
$$

Note

Free type indices referring to types within the same recursive type are handled separately by rolling up recursive types before closing them.

3.1.7 Prose Notation

Validation is specified by stylised rules for each relevant part of the abstract syntax. The rules not only state constraints defining when a phrase is valid, they also classify it with a type. The following conventions are adopted in stating these rules.

- A phrase A is said to be "valid with type T" if and only if all constraints expressed by the respective rules are met. The form of T depends on the syntactic class of A.

  Note

  For example, if A is a function, then T is a defined function type; for an A that is a global, T is a global type; and so on.

- The rules implicitly assume a given context C.
- In some places, this context is locally extended to a context C' with additional entries. The formulation "Under context C', … *statement* …" is adopted to express that the following statement must apply under the assumptions embodied in the extended context.

3.1.8 Formal Notation

Note

This section gives a brief explanation of the notation for specifying typing rules formally. For the interested reader, a more thorough introduction can be found in respective text books.[16]

The proposition that a phrase A has a respective type T is written $A : T$. In general, however, typing is dependent on a context C. To express this explicitly, the complete form is a *judgement* $C \vdash A : T$, which says that $A : T$ holds under the assumptions encoded in C.

The formal typing rules use a standard approach for specifying type systems, rendering them into *deduction rules*. Every rule has the following general form:

$$\frac{premise_1 \qquad premise_2 \qquad \dots \qquad premise_n}{conclusion}$$

Such a rule is read as a big implication: if all premises hold, then the conclusion holds. Some rules have no premises; they are *axioms* whose conclusion holds unconditionally. The conclusion always is a judgment $C \vdash A : T$, and there usually is one respective rule for each relevant construct A of the abstract syntax.

Note

For example, the typing rule for the i32.add instruction can be given as an axiom:

$$\frac{}{C \vdash \mathsf{i32.add} : \mathsf{i32}\ \mathsf{i32} \to \mathsf{i32}}$$

The instruction is always valid with type $\mathsf{i32}\ \mathsf{i32} \to \mathsf{i32}$ (saying that it consumes two i32 values and produces one), independent of any side conditions.

An instruction like global.get can be typed as follows:

$$\frac{C.\mathsf{globals}[x] = \mathsf{mut}\ t}{C \vdash \mathsf{global.get}\ x : \epsilon \to t}$$

Here, the premise enforces that the immediate global index x exists in the context. The instruction produces a value of its respective type t (and does not consume any values). If $C.\mathsf{globals}[x]$ does not exist then the premise does not hold, and the instruction is ill-typed.

Finally, a structured instruction requires a recursive rule, where the premise is itself a typing judgement:

$$\frac{C \vdash blocktype : t_1^* \to t_2^* \qquad \{\mathsf{labels}\ (t_2^*)\} \oplus C \vdash instr^* : t_1^* \to t_2^*}{C \vdash \mathsf{block}\ blocktype\ instr^* : t_1^* \to t_2^*}$$

A block instruction is only valid when the instruction sequence in its body is. Moreover, the result type must match the block's annotation $blocktype$. If so, then the block instruction has the same type as the body. Inside the body an additional label of the corresponding result type is available, which is expressed by extending the context C with the additional label information for the premise.

3.2 Types

Simple types, such as number types are universally valid. However, restrictions apply to most other types, such as reference types, function types, as well as the limits of table types and memory types, which must be checked during validation.

Moreover, block types are converted to instruction types for ease of processing.

[16] For example: Benjamin Pierce. Types and Programming Languages[17]. The MIT Press 2002

[17] https://www.cis.upenn.edu/~bcpierce/tapl/

3.2.1 Number Types

The number type $numtype$ is always valid.

$$\frac{}{C \vdash numtype : \mathsf{ok}}$$

3.2.2 Vector Types

The vector type $vectype$ is always valid.

$$\frac{}{C \vdash vectype : \mathsf{ok}}$$

3.2.3 Type Uses

The type use $typeidx$ is valid if:

- The type $C.\mathsf{types}[typeidx]$ exists.

$$\frac{C.\mathsf{types}[typeidx] = dt}{C \vdash typeidx : \mathsf{ok}}$$

3.2.4 Heap Types

The heap type $absheaptype$ is always valid.

$$\frac{}{C \vdash absheaptype : \mathsf{ok}}$$

3.2.5 Reference Types

The reference type $(\mathsf{ref}\ \mathsf{null}^?\ heaptype)$ is valid if:

- The heap type $heaptype$ is valid.

$$\frac{C \vdash heaptype : \mathsf{ok}}{C \vdash \mathsf{ref}\ \mathsf{null}^?\ heaptype : \mathsf{ok}}$$

3.2.6 Value Types

The value type $valtype$ is valid if:

- Either:
  - The value type $valtype$ is of the form $numtype$.
  - The number type $numtype$ is valid.
- Or:
  - The value type $valtype$ is of the form $vectype$.
  - The vector type $vectype$ is valid.
- Or:
  - The value type $valtype$ is of the form $reftype$.
  - The reference type $reftype$ is valid.
- Or:

- The value type $valtype$ is of the form bot.

$$\frac{}{C \vdash \mathsf{bot} : \mathsf{ok}}$$

3.2.7 Result Types

The result type t^* is valid if:

- For all t in t^*:
  - The value type t is valid.

$$\frac{(C \vdash t : \mathsf{ok})^*}{C \vdash t^* : \mathsf{ok}}$$

3.2.8 Block Types

Block types may be expressed in one of two forms, both of which are converted to instruction types by the following rules.

The block type $typeidx$ is valid as the instruction type $t_1^* \ \rightarrow \ t_2^*$ if:

- The type $C.\mathsf{types}[typeidx]$ exists.
- The expansion of $C.\mathsf{types}[typeidx]$ is $(\mathsf{func}\ t_1^* \ \rightarrow \ t_2^*)$.

$$\frac{C.\mathsf{types}[typeidx] \approx \mathsf{func}\ t_1^* \rightarrow t_2^*}{C \vdash typeidx : t_1^* \rightarrow t_2^*}$$

The block type $valtype^?$ is valid as the instruction type $\epsilon \ \rightarrow \ valtype^?$ if:

- If $valtype$ is defined, then:
  - The value type $valtype$ is valid.

$$\frac{(C \vdash valtype : \mathsf{ok})^?}{C \vdash valtype^? : \epsilon \rightarrow valtype^?}$$

3.2.9 Instruction Types

The instruction type $t_1^* \rightarrow_{x^*} t_2^*$ is valid if:

- The result type t_1^* is valid.
- The result type t_2^* is valid.
- For all x in x^*:
  - The local $C.\mathsf{locals}[x]$ exists.

$$\frac{C \vdash t_1^* : \mathsf{ok} \qquad C \vdash t_2^* : \mathsf{ok} \qquad (C.\mathsf{locals}[x] = lt)^*}{C \vdash t_1^* \rightarrow_{x^*} t_2^* : \mathsf{ok}}$$

3.2.10 Composite Types

The composite type $(\mathsf{struct}\ fieldtype^*)$ is valid if:

- For all $fieldtype$ in $fieldtype^*$:
  - The field type $fieldtype$ is valid.

$$\frac{(C \vdash fieldtype : \mathsf{ok})^*}{C \vdash \mathsf{struct}\ fieldtype^* : \mathsf{ok}}$$

The composite type $(\mathsf{array}\ fieldtype)$ is valid if:

- The field type *fieldtype* is valid.

$$\frac{C \vdash \mathit{fieldtype} : \mathsf{ok}}{C \vdash \mathsf{array}\ \mathit{fieldtype} : \mathsf{ok}}$$

The composite type $(\mathsf{func}\ t_1^*\ \to\ t_2^*)$ is valid if:

- The result type t_1^* is valid.
- The result type t_2^* is valid.

$$\frac{C \vdash t_1^* : \mathsf{ok} \qquad C \vdash t_2^* : \mathsf{ok}}{C \vdash \mathsf{func}\ t_1^* \to t_2^* : \mathsf{ok}}$$

The field type $(\mathsf{mut}^?\ \mathit{storagetype})$ is valid if:

- The storage type *storagetype* is valid.

$$\frac{C \vdash \mathit{storagetype} : \mathsf{ok}}{C \vdash \mathsf{mut}^?\ \mathit{storagetype} : \mathsf{ok}}$$

The packed type *packtype* is always valid.

$$\frac{}{C \vdash \mathit{packtype} : \mathsf{ok}}$$

3.2.11 Recursive Types

Recursive types are validated with respect to the first type index defined by the recursive group.

The recursive type $(\mathsf{rec}\ \mathit{subtype}^*)$ is valid for the type index x if:

- Either:
  - The sub type sequence $\mathit{subtype}^*$ is empty.
- Or:
  - The sub type sequence $\mathit{subtype}^*$ is of the form $\mathit{subtype}_1\ \mathit{subtype}'^*$.
  - The sub type $\mathit{subtype}_1$ is valid for the type index x.
  - The recursive type $(\mathsf{rec}\ \mathit{subtype}'^*)$ is valid for the type index $x+1$.

$$\frac{}{C \vdash \mathsf{rec}\ \epsilon : \mathsf{ok}(x)} \qquad \frac{C \vdash \mathit{subtype}_1 : \mathsf{ok}(x) \qquad C \vdash \mathsf{rec}\ \mathit{subtype}^* : \mathsf{ok}(x+1)}{C \vdash \mathsf{rec}\ (\mathit{subtype}_1\ \mathit{subtype}^*) : \mathsf{ok}(x)}$$

The sub type $(\mathsf{sub\ final}^?\ x^*\ \mathit{comptype})$ is valid for the type index x_0 if:

- The length of x^* is less than or equal to 1.
- For all x in x^*:
  - The index x is less than x_0.
  - The type $C.\mathsf{types}[x]$ exists.
  - The sub type $\mathrm{unroll}(C.\mathsf{types}[x])$ is of the form $(\mathsf{sub}\ y^*\ \mathit{comptype}')$.
- $\mathit{comptype}'^*$ is the concatenation of all such $\mathit{comptype}'$.
- The composite type *comptype* is valid.
- For all $\mathit{comptype}'$ in $\mathit{comptype}'^*$:
  - The composite type *comptype* matches the composite type $\mathit{comptype}'$.

$$\frac{|x^*| \leq 1 \qquad (x < x_0)^* \qquad (\mathrm{unroll}(C.\mathsf{types}[x]) = \mathsf{sub}\ y^*\ \mathit{comptype}')^* \qquad C \vdash \mathit{comptype} : \mathsf{ok} \qquad (C \vdash \mathit{comptype} \leq \mathit{comptype}')^*}{C \vdash \mathsf{sub\ final}^?\ x^*\ \mathit{comptype} : \mathsf{ok}(x_0)}$$

Note

The side condition on the index ensures that a declared supertype is a previously defined types, preventing cyclic subtype hierarchies.

Future versions of WebAssembly may allow more than one supertype.

3.2.12 Limits

Limits must have meaningful bounds that are within a given range.

The limits range $[n\,..\,m^?]$ is valid within k if:

- n is less than or equal to k.
- If m is defined, then:
  - n is less than or equal to m.
  - m is less than or equal to k.

$$\frac{n \leq k \qquad (n \leq m \leq k)^?}{C \vdash [n\,..\,m^?] : k}$$

3.2.13 Tag Types

The tag type $typeuse$ is valid if:

- The type use $typeuse$ is valid.
- The expansion of C is $(\mathsf{func}\ t_1^*\ \rightarrow)$.

$$\frac{C \vdash typeuse : \mathsf{ok} \qquad typeuse \approx_C \mathsf{func}\ t_1^* \rightarrow [] }{C \vdash typeuse : \mathsf{ok}}$$

3.2.14 Global Types

The global type $(\mathsf{mut}^?\ t)$ is valid if:

- The value type t is valid.

$$\frac{C \vdash t : \mathsf{ok}}{C \vdash \mathsf{mut}^?\ t : \mathsf{ok}}$$

3.2.15 Memory Types

The memory type $(addrtype\ limits\ \mathsf{page})$ is valid if:

- The limits range $limits$ is valid within $2^{|addrtype|-16}$.

$$\frac{C \vdash limits : 2^{|addrtype|-16}}{C \vdash addrtype\ limits\ \mathsf{page} : \mathsf{ok}}$$

3.2.16 Table Types

The table type $(addrtype\ limits\ reftype)$ is valid if:

- The limits range $limits$ is valid within $2^{|addrtype|} - 1$.
- The reference type $reftype$ is valid.

$$\frac{C \vdash limits : 2^{|addrtype|} - 1 \qquad C \vdash reftype : \mathsf{ok}}{C \vdash addrtype\ limits\ reftype : \mathsf{ok}}$$

3.2.17 External Types

The external type $(\mathsf{tag}\ tagtype)$ is valid if:

- The tag type $tagtype$ is valid.

$$\frac{C \vdash tagtype : \mathsf{ok}}{C \vdash \mathsf{tag}\ tagtype : \mathsf{ok}}$$

The external type $(\mathsf{global}\ globaltype)$ is valid if:

- The global type $globaltype$ is valid.

$$\frac{C \vdash globaltype : \mathsf{ok}}{C \vdash \mathsf{global}\ globaltype : \mathsf{ok}}$$

The external type $(\mathsf{mem}\ memtype)$ is valid if:

- The memory type $memtype$ is valid.

$$\frac{C \vdash memtype : \mathsf{ok}}{C \vdash \mathsf{mem}\ memtype : \mathsf{ok}}$$

The external type $(\mathsf{table}\ tabletype)$ is valid if:

- The table type $tabletype$ is valid.

$$\frac{C \vdash tabletype : \mathsf{ok}}{C \vdash \mathsf{table}\ tabletype : \mathsf{ok}}$$

The external type $(\mathsf{func}\ typeuse)$ is valid if:

- The type use $typeuse$ is valid.
- The expansion of C is $(\mathsf{func}\ t_1^*\ \rightarrow\ t_2^*)$.

$$\frac{C \vdash typeuse : \mathsf{ok} \qquad typeuse \approx_C \mathsf{func}\ t_1^* \rightarrow t_2^*}{C \vdash \mathsf{func}\ typeuse : \mathsf{ok}}$$

3.3 Matching

On most types, a notion of *subtyping* is defined that is applicable in validation rules, during module instantiation when checking the types of imports, or during execution, when performing casts.

3.3.1 Number Types

The number type $numtype$ matches only itself.

$$\frac{}{C \vdash numtype \leq numtype}$$

3.3.2 Vector Types

The vector type $vectype$ matches only itself.

$$\frac{}{C \vdash vectype \leq vectype}$$

3.3.3 Heap Types

The heap type $heaptype_1$ matches the heap type $heaptype_2$ if:

- Either:
  - The heap type $heaptype_2$ is of the form $heaptype_1$.
- Or:
  - The heap type $heaptype'$ is valid.
  - The heap type $heaptype_1$ matches the heap type $heaptype'$.
  - The heap type $heaptype'$ matches the heap type $heaptype_2$.
- Or:
  - The heap type $heaptype_1$ is of the form eq.
  - The heap type $heaptype_2$ is of the form any.
- Or:
  - The heap type $heaptype_1$ is of the form i31.
  - The heap type $heaptype_2$ is of the form eq.
- Or:
  - The heap type $heaptype_1$ is of the form struct.
  - The heap type $heaptype_2$ is of the form eq.
- Or:
  - The heap type $heaptype_1$ is of the form array.
  - The heap type $heaptype_2$ is of the form eq.
- Or:
  - The heap type $heaptype_1$ is of the form $deftype$.
  - The heap type $heaptype_2$ is of the form struct.
  - The expansion of $deftype$ is $(\mathsf{struct}\ fieldtype^*)$.
- Or:
  - The heap type $heaptype_1$ is of the form $deftype$.
  - The heap type $heaptype_2$ is of the form array.
  - The expansion of $deftype$ is $(\mathsf{array}\ fieldtype)$.
- Or:
  - The heap type $heaptype_1$ is of the form $deftype$.
  - The heap type $heaptype_2$ is of the form func.
  - The expansion of $deftype$ is $(\mathsf{func}\ t_1^* \to t_2^*)$.
- Or:
  - The heap type $heaptype_1$ is of the form $deftype_1$.
  - The heap type $heaptype_2$ is of the form $deftype_2$.
  - The defined type $deftype_1$ matches the defined type $deftype_2$.
- Or:
  - The heap type $heaptype_1$ is of the form $typeidx$.
  - The type $C.\mathsf{types}[typeidx]$ exists.

  - The type C.types$[typeidx]$ matches the heap type $heaptype_2$.
- Or:
  - The heap type $heaptype_2$ is of the form $typeidx$.
  - The type C.types$[typeidx]$ exists.
  - The heap type $heaptype_1$ matches the type C.types$[typeidx]$.
- Or:
  - The heap type $heaptype_1$ is of the form $(\mathsf{rec}.i)$.
  - The heap type $heaptype_2$ is of the form struct.
  - The recursive type C.recs$[i]$ exists.
  - The recursive type C.recs$[i]$ is of the form $(\mathsf{sub\ final}^?\ (\mathsf{struct}\ fieldtype^*))$.
- Or:
  - The heap type $heaptype_1$ is of the form $(\mathsf{rec}.i)$.
  - The heap type $heaptype_2$ is of the form array.
  - The recursive type C.recs$[i]$ exists.
  - The recursive type C.recs$[i]$ is of the form $(\mathsf{sub\ final}^?\ (\mathsf{array}\ fieldtype))$.
- Or:
  - The heap type $heaptype_1$ is of the form $(\mathsf{rec}.i)$.
  - The heap type $heaptype_2$ is of the form func.
  - The recursive type C.recs$[i]$ exists.
  - The recursive type C.recs$[i]$ is of the form $(\mathsf{sub\ final}^?\ (\mathsf{func}\ t_1^*\ \to\ t_2^*))$.
- Or:
  - The heap type $heaptype_1$ is of the form $(\mathsf{rec}.i)$.
  - The length of $typeuse^*$ is greater than j.
  - The heap type $heaptype_2$ is of the form $typeuse^*[j]$.
  - The recursive type C.recs$[i]$ exists.
  - The recursive type C.recs$[i]$ is of the form $(\mathsf{sub\ final}^?\ typeuse^*\ ct)$.
- Or:
  - The heap type $heaptype_1$ is of the form none.
  - The heap type $heaptype_2$ matches the heap type any.
  - The heap type $heaptype_2$ is not of the form bot.
- Or:
  - The heap type $heaptype_1$ is of the form nofunc.
  - The heap type $heaptype_2$ matches the heap type func.
  - The heap type $heaptype_2$ is not of the form bot.
- Or:
  - The heap type $heaptype_1$ is of the form noexn.
  - The heap type $heaptype_2$ matches the heap type exn.
  - The heap type $heaptype_2$ is not of the form bot.
- Or:

  - The heap type $heaptype_1$ is of the form noextern.
  - The heap type $heaptype_2$ matches the heap type extern.
  - The heap type $heaptype_2$ is not of the form bot.
- Or:
  - The heap type $heaptype_1$ is of the form bot.

$$\frac{}{C \vdash heaptype \leq heaptype}$$

$$\frac{C \vdash heaptype' : \mathsf{ok} \qquad C \vdash heaptype_1 \leq heaptype' \qquad C \vdash heaptype' \leq heaptype_2}{C \vdash heaptype_1 \leq heaptype_2}$$

$$\frac{}{C \vdash \mathsf{eq} \leq \mathsf{any}} \qquad \frac{}{C \vdash \mathsf{i31} \leq \mathsf{eq}} \qquad \frac{}{C \vdash \mathsf{struct} \leq \mathsf{eq}} \qquad \frac{}{C \vdash \mathsf{array} \leq \mathsf{eq}}$$

$$\frac{deftype \approx \mathsf{struct}\ fieldtype^*}{C \vdash deftype \leq \mathsf{struct}} \qquad \frac{deftype \approx \mathsf{array}\ fieldtype}{C \vdash deftype \leq \mathsf{array}} \qquad \frac{deftype \approx \mathsf{func}\ t_1^* \to t_2^*}{C \vdash deftype \leq \mathsf{func}}$$

$$\frac{C \vdash C.\mathsf{types}[typeidx] \leq heaptype}{C \vdash typeidx \leq heaptype} \qquad \frac{C \vdash heaptype \leq C.\mathsf{types}[typeidx]}{C \vdash heaptype \leq typeidx}$$

$$\frac{C \vdash heaptype \leq \mathsf{any} \qquad heaptype \neq \mathsf{bot}}{C \vdash \mathsf{none} \leq heaptype} \qquad \frac{C \vdash heaptype \leq \mathsf{func} \qquad heaptype \neq \mathsf{bot}}{C \vdash \mathsf{nofunc} \leq heaptype}$$

$$\frac{C \vdash heaptype \leq \mathsf{exn} \qquad heaptype \neq \mathsf{bot}}{C \vdash \mathsf{noexn} \leq heaptype} \qquad \frac{C \vdash heaptype \leq \mathsf{extern} \qquad heaptype \neq \mathsf{bot}}{C \vdash \mathsf{noextern} \leq heaptype}$$

$$\frac{}{C \vdash \mathsf{bot} \leq heaptype}$$

3.3.4 Reference Types

The reference type $(\mathsf{ref}\ \mathsf{null}_1^?\ ht_1)$ matches the reference type $(\mathsf{ref}\ \mathsf{null}_2^?\ ht_2)$ if:

- The heap type ht_1 matches the heap type ht_2.
- Either:
  - $\mathsf{null}_1^?$ is absent.
  - $\mathsf{null}_2^?$ is absent.
- Or:
  - $\mathsf{null}_1^?$ is of the form $\mathsf{null}^?$.
  - $\mathsf{null}_2^?$ is of the form null.

$$\frac{C \vdash ht_1 \leq ht_2}{C \vdash \mathsf{ref}\ ht_1 \leq \mathsf{ref}\ ht_2} \qquad \frac{C \vdash ht_1 \leq ht_2}{C \vdash \mathsf{ref}\ \mathsf{null}^?\ ht_1 \leq \mathsf{ref}\ \mathsf{null}\ ht_2}$$

3.3.5 Value Types

The value type $valtype_1$ matches the value type $valtype_2$ if:

- Either:
  - The value type $valtype_1$ is of the form $numtype_1$.
  - The value type $valtype_2$ is of the form $numtype_2$.
  - The number type $numtype_1$ matches the number type $numtype_2$.
- Or:
  - The value type $valtype_1$ is of the form $vectype_1$.

  - The value type $valtype_2$ is of the form $vectype_2$.
  - The vector type $vectype_1$ matches the vector type $vectype_2$.
- Or:
  - The value type $valtype_1$ is of the form $reftype_1$.
  - The value type $valtype_2$ is of the form $reftype_2$.
  - The reference type $reftype_1$ matches the reference type $reftype_2$.
- Or:
  - The value type $valtype_1$ is of the form bot.

$$\frac{}{C \vdash \mathsf{bot} \leq valtype}$$

3.3.6 Result Types

Subtyping is lifted to result types in a pointwise manner.

The result type t_1^* matches the result type t_2^* if:

- For all t_1 in t_1^*, and corresponding t_2 in t_2^*:
  - The value type t_1 matches the value type t_2.

$$\frac{(C \vdash t_1 \leq t_2)^*}{C \vdash t_1^* \leq t_2^*}$$

3.3.7 Instruction Types

Subtyping is further lifted to instruction types.

The instruction type $t_{11}^* \to_{x_1^*} t_{12}^*$ matches the instruction type $t_{21}^* \to_{x_2^*} t_{22}^*$ if:

- The result type t_{21}^* matches the result type t_{11}^*.
- The result type t_{12}^* matches the result type t_{22}^*.
- The local index sequence x^* is of the form $x_2^* \setminus x_1^*$.
- For all x in x^*:
  - The local $C.\mathsf{locals}[x]$ exists.
  - The local $C.\mathsf{locals}[x]$ is of the form $(\mathsf{set}\ t)$.

$$\frac{C \vdash t_{21}^* \leq t_{11}^* \qquad C \vdash t_{12}^* \leq t_{22}^* \qquad x^* = x_2^* \setminus x_1^* \qquad (C.\mathsf{locals}[x] = \mathsf{set}\ t)^*}{C \vdash t_{11}^* \to_{x_1^*} t_{12}^* \leq t_{21}^* \to_{x_2^*} t_{22}^*}$$

Note

Instruction types are contravariant in their input and covariant in their output. Moreover, the supertype may ignore variables from the init set x_1^*. It may also *add* variables to the init set, provided these are already set in the context, i.e., are vacuously initialized.

3.3.8 Composite Types

The composite type $comptype_1$ matches the composite type $comptype_2$ if:

- Either:
  - The composite type $comptype_1$ is of the form $(\mathsf{struct}\ ft_1^*\ ft'^*_1)$.
  - The composite type $comptype_2$ is of the form $(\mathsf{struct}\ ft_2^*)$.
  - For all ft_1 in ft_1^*, and corresponding ft_2 in ft_2^*:
    - The field type ft_1 matches the field type ft_2.
- Or:
  - The composite type $comptype_1$ is of the form $(\mathsf{array}\ ft_1)$.
  - The composite type $comptype_2$ is of the form $(\mathsf{array}\ ft_2)$.
  - The field type ft_1 matches the field type ft_2.
- Or:
  - The composite type $comptype_1$ is of the form $(\mathsf{func}\ t_{11}^*\ \to\ t_{12}^*)$.
  - The composite type $comptype_2$ is of the form $(\mathsf{func}\ t_{21}^*\ \to\ t_{22}^*)$.
  - The result type t_{21}^* matches the result type t_{11}^*.
  - The result type t_{12}^* matches the result type t_{22}^*.

$$\frac{(C \vdash ft_1 \leq ft_2)^*}{C \vdash \mathsf{struct}\ (ft_1^*\ ft'^*_1) \leq \mathsf{struct}\ ft_2^*} \qquad \frac{C \vdash ft_1 \leq ft_2}{C \vdash \mathsf{array}\ ft_1 \leq \mathsf{array}\ ft_2} \qquad \frac{C \vdash t_{21}^* \leq t_{11}^* \qquad C \vdash t_{12}^* \leq t_{22}^*}{C \vdash \mathsf{func}\ t_{11}^* \to t_{12}^* \leq \mathsf{func}\ t_{21}^* \to t_{22}^*}$$

3.3.9 Field Types

The field type $(\mathsf{mut}_1^?\ zt_1)$ matches the field type $(\mathsf{mut}_2^?\ zt_2)$ if:

- The storage type zt_1 matches the storage type zt_2.
- Either:
  - $\mathsf{mut}_1^?$ is absent.
  - $\mathsf{mut}_2^?$ is absent.
- Or:
  - $\mathsf{mut}_1^?$ is of the form mut.
  - $\mathsf{mut}_2^?$ is of the form mut.
  - The storage type zt_2 matches the storage type zt_1.

$$\frac{C \vdash zt_1 \leq zt_2 \qquad C \vdash zt_2 \leq zt_1}{C \vdash \mathsf{mut}\ zt_1 \leq \mathsf{mut}\ zt_2}$$

The storage type $storagetype_1$ matches the storage type $storagetype_2$ if:

- Either:
  - The storage type $storagetype_1$ is of the form $valtype_1$.
  - The storage type $storagetype_2$ is of the form $valtype_2$.
  - The value type $valtype_1$ matches the value type $valtype_2$.
- Or:
  - The storage type $storagetype_1$ is of the form $packtype_1$.
  - The storage type $storagetype_2$ is of the form $packtype_2$.

- The packed type $packtype_1$ matches the packed type $packtype_2$.

The packed type $packtype$ matches only itself.

$$\frac{}{C \vdash packtype \leq packtype}$$

3.3.10 Defined Types

The defined type $deftype_1$ matches the defined type $deftype_2$ if:

- Either:
  - The defined type $\mathsf{clos}_C(deftype_1)$ is of the form $\mathsf{clos}_C(deftype_2)$.
- Or:
  - The sub type $\mathrm{unroll}(deftype_1)$ is of the form $(\mathsf{sub\ final}^?\ typeuse^*\ ct)$.
  - The length of $typeuse^*$ is greater than i.
  - The type use $typeuse^*[i]$ matches the heap type $deftype_2$.

$$\frac{\mathsf{clos}_C(deftype_1) = \mathsf{clos}_C(deftype_2)}{C \vdash deftype_1 \leq deftype_2}$$

$$\frac{\mathrm{unroll}(deftype_1) = \mathsf{sub\ final}^?\ typeuse^*\ ct \qquad C \vdash typeuse^*[i] \leq deftype_2}{C \vdash deftype_1 \leq deftype_2}$$

Note

Note that there is no explicit definition of type *equivalence*, since it coincides with syntactic equality, as used in the premise of the former rule above.

3.3.11 Limits

The limits range $[n_1 \,..\, u64_1^?]$ matches the limits range $[n_2 \,..\, u64_2^?]$ if:

- n_1 is greater than or equal to n_2.
- Either:
  - $u64_1^?$ is of the form m_1.
  - If $u64_2$ is defined, then:
    - m_1 is less than or equal to $u64_2$.
- Or:
  - $u64_1^?$ is absent.
  - $u64_2^?$ is absent.

$$\frac{n_1 \geq n_2 \qquad (m_1 \leq m_2)^?}{C \vdash [n_1 \,..\, m_1] \leq [n_2 \,..\, m_2^?]} \qquad \frac{n_1 \geq n_2}{C \vdash [n_1 \,..\, \epsilon] \leq [n_2 \,..\, \epsilon]}$$

3.3.12 Tag Types

The tag type $deftype_1$ matches the tag type $deftype_2$ if:

- The defined type $deftype_1$ matches the defined type $deftype_2$.
- The defined type $deftype_2$ matches the defined type $deftype_1$.

$$\frac{C \vdash deftype_1 \leq deftype_2 \qquad C \vdash deftype_2 \leq deftype_1}{C \vdash deftype_1 \leq deftype_2}$$

Note

Although the conclusion of this rule looks identical to its premise, they in fact describe different relations: the premise invokes subtyping on defined types, while the conclusion defines it on tag types that happen to be expressed as defined types.

3.3.13 Global Types

The global type $(\mathsf{mut}_1^? \ valtype_1)$ matches the global type $(\mathsf{mut}_2^? \ valtype_2)$ if:

- The value type $valtype_1$ matches the value type $valtype_2$.
- Either:
  - $\mathsf{mut}_1^?$ is absent.
  - $\mathsf{mut}_2^?$ is absent.
- Or:
  - $\mathsf{mut}_1^?$ is of the form mut.
  - $\mathsf{mut}_2^?$ is of the form mut.
  - The value type $valtype_2$ matches the value type $valtype_1$.

$$\frac{C \vdash valtype_1 \leq valtype_2 \qquad C \vdash valtype_2 \leq valtype_1}{C \vdash \mathsf{mut}\ valtype_1 \leq \mathsf{mut}\ valtype_2}$$

3.3.14 Memory Types

The memory type $(addrtype \ limits_1 \ \mathsf{page})$ matches the memory type $(addrtype \ limits_2 \ \mathsf{page})$ if:

- The limits range $limits_1$ matches the limits range $limits_2$.

$$\frac{C \vdash limits_1 \leq limits_2}{C \vdash addrtype \ limits_1 \ \mathsf{page} \leq addrtype \ limits_2 \ \mathsf{page}}$$

3.3.15 Table Types

The table type $(addrtype \ limits_1 \ reftype_1)$ matches the table type $(addrtype \ limits_2 \ reftype_2)$ if:

- The limits range $limits_1$ matches the limits range $limits_2$.
- The reference type $reftype_1$ matches the reference type $reftype_2$.
- The reference type $reftype_2$ matches the reference type $reftype_1$.

$$\frac{C \vdash limits_1 \leq limits_2 \qquad C \vdash reftype_1 \leq reftype_2 \qquad C \vdash reftype_2 \leq reftype_1}{C \vdash addrtype \ limits_1 \ reftype_1 \leq addrtype \ limits_2 \ reftype_2}$$

3.3.16 External Types

The external type $(\mathsf{tag} \ tagtype_1)$ matches the external type $(\mathsf{tag} \ tagtype_2)$ if:

- The tag type $tagtype_1$ matches the tag type $tagtype_2$.

$$\frac{C \vdash tagtype_1 \leq tagtype_2}{C \vdash \mathsf{tag}\ tagtype_1 \leq \mathsf{tag}\ tagtype_2}$$

The external type $(\mathsf{global} \ globaltype_1)$ matches the external type $(\mathsf{global} \ globaltype_2)$ if:

- The global type $globaltype_1$ matches the global type $globaltype_2$.

$$\frac{C \vdash globaltype_1 \leq globaltype_2}{C \vdash \mathsf{global}\ globaltype_1 \leq \mathsf{global}\ globaltype_2}$$

The external type $(\mathsf{mem}\ memtype_1)$ matches the external type $(\mathsf{mem}\ memtype_2)$ if:

- The memory type $memtype_1$ matches the memory type $memtype_2$.

$$\frac{C \vdash memtype_1 \leq memtype_2}{C \vdash \mathsf{mem}\ memtype_1 \leq \mathsf{mem}\ memtype_2}$$

The external type $(\mathsf{table}\ tabletype_1)$ matches the external type $(\mathsf{table}\ tabletype_2)$ if:

- The table type $tabletype_1$ matches the table type $tabletype_2$.

$$\frac{C \vdash tabletype_1 \leq tabletype_2}{C \vdash \mathsf{table}\ tabletype_1 \leq \mathsf{table}\ tabletype_2}$$

The external type $(\mathsf{func}\ deftype_1)$ matches the external type $(\mathsf{func}\ deftype_2)$ if:

- The defined type $deftype_1$ matches the defined type $deftype_2$.

$$\frac{C \vdash deftype_1 \leq deftype_2}{C \vdash \mathsf{func}\ deftype_1 \leq \mathsf{func}\ deftype_2}$$

3.4 Instructions

Instructions are classified by instruction types that describe how they manipulate the operand stack and initialize locals: A type $t_1^* \to_{x^*} t_2^*$ describes the required input stack with argument values of types t_1^* that an instruction pops off and the provided output stack with result values of types t_2^* that it pushes back. Moreover, it enumerates the indices x^* of locals that have been set by the instruction. In most cases, this is empty.

Note

For example, the instruction i32.add has type $\mathsf{i32}\ \mathsf{i32} \to \mathsf{i32}$, consuming two i32 values and producing one. The instruction $(\mathsf{local.set}\ x)$ has type $t \to_x \epsilon$, provided t is the type declared for the local x.

Typing extends to instruction sequences $instr^*$. Such a sequence has an instruction type $t_1^* \to_{x^*} t_2^*$ if the accumulative effect of executing the instructions is consuming values of types t_1^* off the operand stack, pushing new values of types t_2^*, and setting all locals x^*.

For some instructions, the typing rules do not fully constrain the type, and therefore allow for multiple types. Such instructions are called *polymorphic*. Two degrees of polymorphism can be distinguished:

- *value-polymorphic*: the value type t of one or several individual operands is unconstrained. That is the case for all parametric instructions like drop and select.
- *stack-polymorphic*: the entire (or most of the) instruction type $t_1^* \to t_2^*$ of the instruction is unconstrained. That is the case for all control instructions that perform an *unconditional control transfer*, such as unreachable, br, or return.

In both cases, the unconstrained types or type sequences can be chosen arbitrarily, as long as they are valid in the current context and meet the constraints imposed for the surrounding parts of the program.

Note

For example, the select instruction is valid with type $t\ t\ \mathsf{i32} \to t$, for any possible number type t. Consequently, both instruction sequences

$$(\mathsf{i32.const}\ 1)\ (\mathsf{i32.const}\ 2)\ (\mathsf{i32.const}\ 3)\ (\mathsf{select})$$

and

$$(\mathsf{f64.const}\ {+1})\ (\mathsf{f64.const}\ {+2})\ (\mathsf{i32.const}\ 3)\ (\mathsf{select})$$

are valid, with t in the typing of select being instantiated to i32 or f64, respectively.

The unreachable instruction is stack-polymorphic, and hence valid with type $t_1^* \to t_2^*$ for any possible sequences of value types t_1^* and t_2^*. Consequently,

$$(\mathsf{unreachable})\ (\mathsf{i32.add})$$

is valid by assuming type $\epsilon \to \mathsf{i32}$ for the unreachable instruction. In contrast,

$$(\mathsf{unreachable})\ (\mathsf{i64.const}\ 0)\ (\mathsf{i32.add})$$

is invalid, because there is no possible type to pick for the unreachable instruction that would make the sequence well-typed.

The Appendix describes a type checking algorithm that efficiently implements validation of instruction sequences as prescribed by the rules given here.

3.4.1 Parametric Instructions

nop

The instruction nop is valid with the instruction type $\epsilon \to \epsilon$.

$$\frac{}{C \vdash \mathsf{nop} : \epsilon \to \epsilon}$$

unreachable

The instruction unreachable is valid with the instruction type $t_1^* \to t_2^*$ if:

- The instruction type $t_1^* \to t_2^*$ is valid.

$$\frac{C \vdash t_1^* \to t_2^* : \mathsf{ok}}{C \vdash \mathsf{unreachable} : t_1^* \to t_2^*}$$

Note

The unreachable instruction is stack-polymorphic.

drop

The instruction drop is valid with the instruction type $t \to \epsilon$ if:

- The value type t is valid.

$$\frac{C \vdash t : \mathsf{ok}}{C \vdash \mathsf{drop} : t \to \epsilon}$$

Note

Both drop and select without annotation are value-polymorphic instructions.

select $(t^*)^?$

The instruction (select $valtype^?$) is valid with the instruction type $t\ t\ \mathsf{i32} \to t$ if:

- The value type t is valid.
- Either:
  - The value type sequence $valtype^?$ is of the form t.
- Or:
  - The value type sequence $valtype^?$ is absent.
  - The value type t matches the value type t'.
  - The value type t' is of the form $numtype$ or t' is of the form $vectype$.

$$\frac{C \vdash t : \mathsf{ok}}{C \vdash \mathsf{select}\ t : t\ t\ \mathsf{i32} \to t} \qquad \frac{C \vdash t : \mathsf{ok} \qquad C \vdash t \leq t' \qquad t' = numtype \vee t' = vectype}{C \vdash \mathsf{select} : t\ t\ \mathsf{i32} \to t}$$

Note

In future versions of WebAssembly, select may allow more than one value per choice.

3.4.2 Control Instructions

block $blocktype\ instr^*$

The instruction (block $bt\ instr^*$) is valid with the instruction type $t_1^* \to t_2^*$ if:

- The block type bt is valid as the instruction type $t_1^* \to t_2^*$.
- Let C' be the same context as C, but with the result type sequence t_2^* prepended to the field labels.
- Under the context C', the instruction sequence $instr^*$ is valid with the instruction type $t_1^* \to_{x^*} t_2^*$.

$$\frac{C \vdash bt : t_1^* \to t_2^* \qquad \{\mathsf{labels}\ (t_2^*)\} \oplus C \vdash instr^* : t_1^* \to_{x^*} t_2^*}{C \vdash \mathsf{block}\ bt\ instr^* : t_1^* \to t_2^*}$$

Note

The notation $\{\mathsf{labels}\ (t^*)\} \oplus C$ inserts the new label type at index 0, shifting all others. The same applies to all other block instructions.

loop $blocktype\ instr^*$

The instruction (loop $bt\ instr^*$) is valid with the instruction type $t_1^* \to t_2^*$ if:

- The block type bt is valid as the instruction type $t_1^* \to t_2^*$.
- Let C' be the same context as C, but with the result type sequence t_1^* prepended to the field labels.
- Under the context C', the instruction sequence $instr^*$ is valid with the instruction type $t_1^* \to_{x^*} t_2^*$.

$$\frac{C \vdash bt : t_1^* \to t_2^* \qquad \{\mathsf{labels}\ (t_1^*)\} \oplus C \vdash instr^* : t_1^* \to_{x^*} t_2^*}{C \vdash \mathsf{loop}\ bt\ instr^* : t_1^* \to t_2^*}$$

if $blocktype\ instr_1^*$ else $instr_2^*$

The instruction (if $bt\ instr_1^*$ else $instr_2^*$) is valid with the instruction type $t_1^*\ \mathsf{i32} \to t_2^*$ if:

- The block type bt is valid as the instruction type $t_1^* \to t_2^*$.
- Let C' be the same context as C, but with the result type sequence t_2^* prepended to the field labels.

- Under the context C', the instruction sequence $instr_1^*$ is valid with the instruction type $t_1^* \to_{x_1^*} t_2^*$.
- Under the context C', the instruction sequence $instr_2^*$ is valid with the instruction type $t_1^* \to_{x_2^*} t_2^*$.

$$\frac{C \vdash bt : t_1^* \to t_2^* \qquad \{\mathsf{labels}\ (t_2^*)\} \oplus C \vdash instr_1^* : t_1^* \to_{x_1^*} t_2^* \qquad \{\mathsf{labels}\ (t_2^*)\} \oplus C \vdash instr_2^* : t_1^* \to_{x_2^*} t_2^*}{C \vdash \mathsf{if}\ bt\ instr_1^*\ \mathsf{else}\ instr_2^* : t_1^*\ \mathsf{i32} \to t_2^*}$$

br l

The instruction $(\mathsf{br}\ l)$ is valid with the instruction type $t_1^*\ t^*\ \to\ t_2^*$ if:

- The label $C.\mathsf{labels}[l]$ exists.
- The label $C.\mathsf{labels}[l]$ is of the form t^*.
- The instruction type $t_1^*\ \to\ t_2^*$ is valid.

$$\frac{C.\mathsf{labels}[l] = t^* \qquad C \vdash t_1^* \to t_2^* : \mathsf{ok}}{C \vdash \mathsf{br}\ l : t_1^*\ t^* \to t_2^*}$$

Note

The label index space in the context C contains the most recent label first, so that $C.\mathsf{labels}[l]$ performs a relative lookup as expected. This applies to other branch instructions as well.

The br instruction is stack-polymorphic.

br_if l

The instruction $(\mathsf{br_if}\ l)$ is valid with the instruction type $t^*\ \mathsf{i32}\ \to\ t^*$ if:

- The label $C.\mathsf{labels}[l]$ exists.
- The label $C.\mathsf{labels}[l]$ is of the form t^*.

$$\frac{C.\mathsf{labels}[l] = t^*}{C \vdash \mathsf{br_if}\ l : t^*\ \mathsf{i32} \to t^*}$$

br_table $l^*\ l_N$

The instruction $(\mathsf{br_table}\ l^*\ l')$ is valid with the instruction type $t_1^*\ t^*\ \mathsf{i32}\ \to\ t_2^*$ if:

- For all l in l^*:
  - The label $C.\mathsf{labels}[l]$ exists.
  - The result type t^* matches the label $C.\mathsf{labels}[l]$.
- The label $C.\mathsf{labels}[l']$ exists.
- The result type t^* matches the label $C.\mathsf{labels}[l']$.
- The instruction type $t_1^*\ t^*\ \mathsf{i32}\ \to\ t_2^*$ is valid.

$$\frac{(C \vdash t^* \leq C.\mathsf{labels}[l])^* \qquad C \vdash t^* \leq C.\mathsf{labels}[l'] \qquad C \vdash t_1^*\ t^*\ \mathsf{i32} \to t_2^* : \mathsf{ok}}{C \vdash \mathsf{br_table}\ l^*\ l' : t_1^*\ t^*\ \mathsf{i32} \to t_2^*}$$

Note

The br_table instruction is stack-polymorphic.

Furthermore, the result type t^* is also chosen non-deterministically in this rule. Although it may seem necessary to compute t^* as the greatest lower bound of all label types in practice, a simple sequential algorithm does not require this.

br_on_null l

The instruction (br_on_null l) is valid with the instruction type t^* (ref null ht) $\to$ t^* (ref ht) if:

- The label $C.\mathsf{labels}[l]$ exists.
- The label $C.\mathsf{labels}[l]$ is of the form t^*.
- The heap type ht is valid.

$$\frac{C.\mathsf{labels}[l] = t^* \qquad C \vdash ht : \mathsf{ok}}{C \vdash \mathsf{br_on_null}\ l : t^*\ (\mathsf{ref\ null}\ ht) \to t^*\ (\mathsf{ref}\ ht)}$$

br_on_non_null l

The instruction (br_on_non_null l) is valid with the instruction type t^* (ref null ht) $\to$ t^* if:

- The label $C.\mathsf{labels}[l]$ exists.
- The label $C.\mathsf{labels}[l]$ is of the form t^* (ref null$^?$ ht).

$$\frac{C.\mathsf{labels}[l] = t^*\ (\mathsf{ref\ null}^?\ ht)}{C \vdash \mathsf{br_on_non_null}\ l : t^*\ (\mathsf{ref\ null}\ ht) \to t^*}$$

br_on_cast l rt_1 rt_2

The instruction (br_on_cast l rt_1 rt_2) is valid with the instruction type t^* rt_1 $\to$ t^* *reftype* if:

- The label $C.\mathsf{labels}[l]$ exists.
- The label $C.\mathsf{labels}[l]$ is of the form t^* rt.
- The reference type rt_1 is valid.
- The reference type rt_2 is valid.
- The reference type rt_2 matches the reference type rt_1.
- The reference type rt_2 matches the reference type rt.
- The reference type *reftype* is $rt_1 \setminus rt_2$.

$$\frac{C.\mathsf{labels}[l] = t^*\ rt \qquad C \vdash rt_1 : \mathsf{ok} \qquad C \vdash rt_2 : \mathsf{ok} \qquad C \vdash rt_2 \leq rt_1 \qquad C \vdash rt_2 \leq rt}{C \vdash \mathsf{br_on_cast}\ l\ rt_1\ rt_2 : t^*\ rt_1 \to t^*\ (rt_1 \setminus rt_2)}$$

br_on_cast_fail l rt_1 rt_2

The instruction (br_on_cast_fail l rt_1 rt_2) is valid with the instruction type t^* rt_1 $\to$ t^* rt_2 if:

- The label $C.\mathsf{labels}[l]$ exists.
- The label $C.\mathsf{labels}[l]$ is of the form t^* rt.
- The reference type rt_1 is valid.
- The reference type rt_2 is valid.
- The reference type rt_2 matches the reference type rt_1.
- The reference type $rt_1 \setminus rt_2$ matches the reference type rt.

$$\frac{C.\mathsf{labels}[l] = t^*\ rt \qquad C \vdash rt_1 : \mathsf{ok} \qquad C \vdash rt_2 : \mathsf{ok} \qquad C \vdash rt_2 \leq rt_1 \qquad C \vdash rt_1 \setminus rt_2 \leq rt}{C \vdash \mathsf{br_on_cast_fail}\ l\ rt_1\ rt_2 : t^*\ rt_1 \to t^*\ rt_2}$$

call x

The instruction $(\mathsf{call}\ x)$ is valid with the instruction type $t_1^*\ \to\ t_2^*$ if:

- The function $C.\mathsf{funcs}[x]$ exists.
- The expansion of $C.\mathsf{funcs}[x]$ is $(\mathsf{func}\ t_1^*\ \to\ t_2^*)$.

$$\frac{C.\mathsf{funcs}[x] \approx \mathsf{func}\ t_1^* \to t_2^*}{C \vdash \mathsf{call}\ x : t_1^* \to t_2^*}$$

call_ref x

The instruction $(\mathsf{call_ref}\ x)$ is valid with the instruction type $t_1^*\ (\mathsf{ref\ null}\ x)\ \to\ t_2^*$ if:

- The type $C.\mathsf{types}[x]$ exists.
- The expansion of $C.\mathsf{types}[x]$ is $(\mathsf{func}\ t_1^*\ \to\ t_2^*)$.

$$\frac{C.\mathsf{types}[x] \approx \mathsf{func}\ t_1^* \to t_2^*}{C \vdash \mathsf{call_ref}\ x : t_1^*\ (\mathsf{ref\ null}\ x) \to t_2^*}$$

call_indirect $x\ y$

The instruction $(\mathsf{call_indirect}\ x\ y)$ is valid with the instruction type $t_1^*\ at\ \to\ t_2^*$ if:

- The table $C.\mathsf{tables}[x]$ exists.
- The table $C.\mathsf{tables}[x]$ is of the form $(at\ lim\ rt)$.
- The reference type rt matches the reference type $(\mathsf{ref\ null\ func})$.
- The type $C.\mathsf{types}[y]$ exists.
- The expansion of $C.\mathsf{types}[y]$ is $(\mathsf{func}\ t_1^*\ \to\ t_2^*)$.

$$\frac{C.\mathsf{tables}[x] = at\ lim\ rt \qquad C \vdash rt \leq (\mathsf{ref\ null\ func}) \qquad C.\mathsf{types}[y] \approx \mathsf{func}\ t_1^* \to t_2^*}{C \vdash \mathsf{call_indirect}\ x\ y : t_1^*\ at \to t_2^*}$$

return

The instruction return is valid with the instruction type $t_1^*\ t^*\ \to\ t_2^*$ if:

- The result type $C.\mathsf{return}$ is of the form t^*.
- The instruction type $t_1^*\ \to\ t_2^*$ is valid.

$$\frac{C.\mathsf{return} = (t^*) \qquad C \vdash t_1^* \to t_2^* : \mathsf{ok}}{C \vdash \mathsf{return} : t_1^*\ t^* \to t_2^*}$$

Note

The return instruction is stack-polymorphic.

$C.\mathsf{return}$ is absent (set to ϵ) when validating an expression that is not a function body. This differs from it being set to the empty result type $[\epsilon]$, which is the case for functions not returning anything.

return_call x

The instruction $(\mathsf{return_call}\ x)$ is valid with the instruction type $t_3^*\ t_1^*\ \to\ t_4^*$ if:

- The function $C.\mathsf{funcs}[x]$ exists.
- The expansion of $C.\mathsf{funcs}[x]$ is $(\mathsf{func}\ t_1^*\ \to\ t_2^*)$.
- The result type $C.\mathsf{return}$ is of the form $t_2'^*$.
- The result type t_2^* matches the result type $t_2'^*$.

- The instruction type $t_3^* \ \to \ t_4^*$ is valid.

$$\frac{C.\mathsf{funcs}[x] \approx \mathsf{func}\ t_1^* \to t_2^* \qquad C.\mathsf{return} = (t'^*_2) \qquad C \vdash t_2^* \leq t'^*_2 \qquad C \vdash t_3^* \to t_4^* : \mathsf{ok}}{C \vdash \mathsf{return_call}\ x : t_3^*\ t_1^* \to t_4^*}$$

Note

The return_call instruction is stack-polymorphic.

return_call_ref x

The instruction $(\mathsf{return_call_ref}\ x)$ is valid with the instruction type $t_3^*\ t_1^*\ (\mathsf{ref\ null}\ x) \ \to \ t_4^*$ if:

- The type $C.\mathsf{types}[x]$ exists.
- The expansion of $C.\mathsf{types}[x]$ is $(\mathsf{func}\ t_1^* \ \to \ t_2^*)$.
- The result type $C.\mathsf{return}$ is of the form t'^*_2.
- The result type t_2^* matches the result type t'^*_2.
- The instruction type $t_3^* \ \to \ t_4^*$ is valid.

$$\frac{C.\mathsf{types}[x] \approx \mathsf{func}\ t_1^* \to t_2^* \qquad C.\mathsf{return} = (t'^*_2) \qquad C \vdash t_2^* \leq t'^*_2 \qquad C \vdash t_3^* \to t_4^* : \mathsf{ok}}{C \vdash \mathsf{return_call_ref}\ x : t_3^*\ t_1^*\ (\mathsf{ref\ null}\ x) \to t_4^*}$$

Note

The return_call_ref instruction is stack-polymorphic.

return_call_indirect $x\ y$

The instruction $(\mathsf{return_call_indirect}\ x\ y)$ is valid with the instruction type $t_3^*\ t_1^*\ at \ \to \ t_4^*$ if:

- The table $C.\mathsf{tables}[x]$ exists.
- The table $C.\mathsf{tables}[x]$ is of the form $(at\ lim\ rt)$.
- The reference type rt matches the reference type $(\mathsf{ref\ null\ func})$.
- The type $C.\mathsf{types}[y]$ exists.
- The expansion of $C.\mathsf{types}[y]$ is $(\mathsf{func}\ t_1^* \ \to \ t_2^*)$.
- The result type $C.\mathsf{return}$ is of the form t'^*_2.
- The result type t_2^* matches the result type t'^*_2.
- The instruction type $t_3^* \ \to \ t_4^*$ is valid.

$$\frac{\begin{array}{c} C.\mathsf{tables}[x] = at\ lim\ rt \qquad C \vdash rt \leq (\mathsf{ref\ null\ func}) \\ C.\mathsf{types}[y] \approx \mathsf{func}\ t_1^* \to t_2^* \qquad C.\mathsf{return} = (t'^*_2) \qquad C \vdash t_2^* \leq t'^*_2 \qquad C \vdash t_3^* \to t_4^* : \mathsf{ok} \end{array}}{C \vdash \mathsf{return_call_indirect}\ x\ y : t_3^*\ t_1^*\ at \to t_4^*}$$

Note

The return_call_indirect instruction is stack-polymorphic.

throw x

The instruction $(\mathsf{throw}\ x)$ is valid with the instruction type $t_1^*\ t^*\ \rightarrow\ t_2^*$ if:

- The tag $C.\mathsf{tags}[x]$ exists.
- The expansion of $C.\mathsf{tags}[x]$ is $(\mathsf{func}\ t^*\ \rightarrow)$.
- The instruction type $t_1^*\ \rightarrow\ t_2^*$ is valid.

$$\frac{C.\mathsf{tags}[x] \approx \mathsf{func}\ t^* \rightarrow \epsilon \qquad C \vdash t_1^* \rightarrow t_2^* : \mathsf{ok}}{C \vdash \mathsf{throw}\ x : t_1^*\ t^* \rightarrow t_2^*}$$

Note

The throw instruction is stack-polymorphic.

throw_ref

The instruction throw_ref is valid with the instruction type $t_1^*\ (\mathsf{ref\ null\ exn})\ \rightarrow\ t_2^*$ if:

- The instruction type $t_1^*\ \rightarrow\ t_2^*$ is valid.

$$\frac{C \vdash t_1^* \rightarrow t_2^* : \mathsf{ok}}{C \vdash \mathsf{throw_ref} : t_1^*\ (\mathsf{ref\ null\ exn}) \rightarrow t_2^*}$$

Note

The throw_ref instruction is stack-polymorphic.

try_table $\mathit{blocktype}\ \mathit{catch}^*\ \mathit{instr}^*$

The instruction $(\mathsf{try_table}\ bt\ \mathit{catch}^*\ \mathit{instr}^*)$ is valid with the instruction type $t_1^*\ \rightarrow\ t_2^*$ if:

- The block type bt is valid as the instruction type $t_1^*\ \rightarrow\ t_2^*$.
- Let C' be the same context as C, but with the result type sequence t_2^* prepended to the field labels.
- Under the context C', the instruction sequence instr^* is valid with the instruction type $t_1^* \rightarrow_{x^*} t_2^*$.
- For all catch in catch^*:
  - The catch clause catch is valid.

$$\frac{C \vdash bt : t_1^* \rightarrow t_2^* \qquad \{\mathsf{labels}\ (t_2^*)\} \oplus C \vdash \mathit{instr}^* : t_1^* \rightarrow_{x^*} t_2^* \qquad (C \vdash \mathit{catch} : \mathsf{ok})^*}{C \vdash \mathsf{try_table}\ bt\ \mathit{catch}^*\ \mathit{instr}^* : t_1^* \rightarrow t_2^*}$$

catch $x\ l$

The catch clause $(\mathsf{catch}\ x\ l)$ is valid if:

- The tag $C.\mathsf{tags}[x]$ exists.
- The expansion of $C.\mathsf{tags}[x]$ is $(\mathsf{func}\ t^*\ \rightarrow)$.
- The label $C.\mathsf{labels}[l]$ exists.
- The result type t^* matches the label $C.\mathsf{labels}[l]$.

$$\frac{C.\mathsf{tags}[x] \approx \mathsf{func}\ t^* \rightarrow \epsilon \qquad C \vdash t^* \leq C.\mathsf{labels}[l]}{C \vdash \mathsf{catch}\ x\ l : \mathsf{ok}}$$

catch_ref x l

The catch clause (catch_ref x l) is valid if:

- The tag $C.\mathsf{tags}[x]$ exists.
- The expansion of $C.\mathsf{tags}[x]$ is (func t^* $\rightarrow$).
- The label $C.\mathsf{labels}[l]$ exists.
- The result type t^* (ref exn) matches the label $C.\mathsf{labels}[l]$.

$$\frac{C.\mathsf{tags}[x] \approx \mathsf{func}\ t^* \rightarrow \epsilon \qquad C \vdash t^*\ (\mathsf{ref}\ \mathsf{exn}) \leq C.\mathsf{labels}[l]}{C \vdash \mathsf{catch_ref}\ x\ l : \mathsf{ok}}$$

catch_all l

The catch clause (catch_all l) is valid if:

- The label $C.\mathsf{labels}[l]$ exists.
- The result type ϵ matches the label $C.\mathsf{labels}[l]$.

$$\frac{C \vdash \epsilon \leq C.\mathsf{labels}[l]}{C \vdash \mathsf{catch_all}\ l : \mathsf{ok}}$$

catch_all_ref l

The catch clause (catch_all_ref l) is valid if:

- The label $C.\mathsf{labels}[l]$ exists.
- The result type (ref exn) matches the label $C.\mathsf{labels}[l]$.

$$\frac{C \vdash (\mathsf{ref}\ \mathsf{exn}) \leq C.\mathsf{labels}[l]}{C \vdash \mathsf{catch_all_ref}\ l : \mathsf{ok}}$$

3.4.3 Variable Instructions

local.get x

The instruction (local.get x) is valid with the instruction type $\epsilon \rightarrow t$ if:

- The local $C.\mathsf{locals}[x]$ exists.
- The local $C.\mathsf{locals}[x]$ is of the form (set t).

$$\frac{C.\mathsf{locals}[x] = \mathsf{set}\ t}{C \vdash \mathsf{local.get}\ x : \epsilon \rightarrow t}$$

local.set x

The instruction (local.set x) is valid with the instruction type $t \rightarrow_x \epsilon$ if:

- The local $C.\mathsf{locals}[x]$ exists.
- The local $C.\mathsf{locals}[x]$ is of the form (init t).

$$\frac{C.\mathsf{locals}[x] = \mathit{init}\ t}{C \vdash \mathsf{local.set}\ x : t \rightarrow_x \epsilon}$$

local.tee x

The instruction $(\mathsf{local.tee}\ x)$ is valid with the instruction type $t \to_x t$ if:

- The local $C.\mathsf{locals}[x]$ exists.
- The local $C.\mathsf{locals}[x]$ is of the form $(\mathit{init}\ t)$.

$$\frac{C.\mathsf{locals}[x] = \mathit{init}\ t}{C \vdash \mathsf{local.tee}\ x : t \to_x t}$$

global.get x

The instruction $(\mathsf{global.get}\ x)$ is valid with the instruction type $\epsilon \to t$ if:

- The global $C.\mathsf{globals}[x]$ exists.
- The global $C.\mathsf{globals}[x]$ is of the form $(\mathsf{mut}^?\ t)$.

$$\frac{C.\mathsf{globals}[x] = \mathsf{mut}^?\ t}{C \vdash \mathsf{global.get}\ x : \epsilon \to t}$$

global.set x

The instruction $(\mathsf{global.set}\ x)$ is valid with the instruction type $t \to \epsilon$ if:

- The global $C.\mathsf{globals}[x]$ exists.
- The global $C.\mathsf{globals}[x]$ is of the form $(\mathsf{mut}\ t)$.

$$\frac{C.\mathsf{globals}[x] = \mathsf{mut}\ t}{C \vdash \mathsf{global.set}\ x : t \to \epsilon}$$

3.4.4 Table Instructions

table.get x

The instruction $(\mathsf{table.get}\ x)$ is valid with the instruction type $at \to rt$ if:

- The table $C.\mathsf{tables}[x]$ exists.
- The table $C.\mathsf{tables}[x]$ is of the form $(at\ \mathit{lim}\ rt)$.

$$\frac{C.\mathsf{tables}[x] = at\ \mathit{lim}\ rt}{C \vdash \mathsf{table.get}\ x : at \to rt}$$

table.set x

The instruction $(\mathsf{table.set}\ x)$ is valid with the instruction type $at\ rt \to \epsilon$ if:

- The table $C.\mathsf{tables}[x]$ exists.
- The table $C.\mathsf{tables}[x]$ is of the form $(at\ \mathit{lim}\ rt)$.

$$\frac{C.\mathsf{tables}[x] = at\ \mathit{lim}\ rt}{C \vdash \mathsf{table.set}\ x : at\ rt \to \epsilon}$$

table.size x

The instruction $(\mathsf{table.size}\ x)$ is valid with the instruction type $\epsilon \to at$ if:

- The table $C.\mathsf{tables}[x]$ exists.
- The table $C.\mathsf{tables}[x]$ is of the form $(at\ \mathit{lim}\ rt)$.

$$\frac{C.\mathsf{tables}[x] = at\ \mathit{lim}\ rt}{C \vdash \mathsf{table.size}\ x : \epsilon \to at}$$

table.grow x

The instruction (table.grow x) is valid with the instruction type $rt\ at\ \rightarrow\ at$ if:

- The table $C.\mathsf{tables}[x]$ exists.
- The table $C.\mathsf{tables}[x]$ is of the form $(at\ lim\ rt)$.

$$\frac{C.\mathsf{tables}[x] = at\ lim\ rt}{C \vdash \mathsf{table.grow}\ x : rt\ at \rightarrow at}$$

table.fill x

The instruction (table.fill x) is valid with the instruction type $at\ rt\ at\ \rightarrow\ \epsilon$ if:

- The table $C.\mathsf{tables}[x]$ exists.
- The table $C.\mathsf{tables}[x]$ is of the form $(at\ lim\ rt)$.

$$\frac{C.\mathsf{tables}[x] = at\ lim\ rt}{C \vdash \mathsf{table.fill}\ x : at\ rt\ at \rightarrow \epsilon}$$

table.copy $x\ y$

The instruction (table.copy $x_1\ x_2$) is valid with the instruction type $at_1\ at_2\ addrtype\ \rightarrow\ \epsilon$ if:

- The table $C.\mathsf{tables}[x_1]$ exists.
- The table $C.\mathsf{tables}[x_1]$ is of the form $(at_1\ lim_1\ rt_1)$.
- The table $C.\mathsf{tables}[x_2]$ exists.
- The table $C.\mathsf{tables}[x_2]$ is of the form $(at_2\ lim_2\ rt_2)$.
- The reference type rt_2 matches the reference type rt_1.
- The address type $addrtype$ is $\min(at_1, at_2)$.

$$\frac{C.\mathsf{tables}[x_1] = at_1\ lim_1\ rt_1 \qquad C.\mathsf{tables}[x_2] = at_2\ lim_2\ rt_2 \qquad C \vdash rt_2 \leq rt_1}{C \vdash \mathsf{table.copy}\ x_1\ x_2 : at_1\ at_2\ \min(at_1, at_2) \rightarrow \epsilon}$$

table.init $x\ y$

The instruction (table.init $x\ y$) is valid with the instruction type $at\ \mathsf{i32}\ \mathsf{i32}\ \rightarrow\ \epsilon$ if:

- The table $C.\mathsf{tables}[x]$ exists.
- The table $C.\mathsf{tables}[x]$ is of the form $(at\ lim\ rt_1)$.
- The element segment $C.\mathsf{elems}[y]$ exists.
- The element segment $C.\mathsf{elems}[y]$ is of the form rt_2.
- The reference type rt_2 matches the reference type rt_1.

$$\frac{C.\mathsf{tables}[x] = at\ lim\ rt_1 \qquad C.\mathsf{elems}[y] = rt_2 \qquad C \vdash rt_2 \leq rt_1}{C \vdash \mathsf{table.init}\ x\ y : at\ \mathsf{i32}\ \mathsf{i32} \rightarrow \epsilon}$$

elem.drop x

The instruction (elem.drop x) is valid with the instruction type $\epsilon\ \rightarrow\ \epsilon$ if:

- The element segment $C.\mathsf{elems}[x]$ exists.

$$\frac{C.\mathsf{elems}[x] = rt}{C \vdash \mathsf{elem.drop}\ x : \epsilon \rightarrow \epsilon}$$

3.4.5 Memory Instructions

Memory instructions use memory arguments, which are classified by the address type and the and bit width of the access they are suitable for.

memarg

$\{\mathsf{align}\ n,\ \mathsf{offset}\ m\}$ is valid for at and N if:

- 2^n is less than or equal to $N/8$.
- m is less than $2^{|at|}$.

$$\frac{2^n \leq N/8 \qquad m < 2^{|at|}}{\vdash \{\mathsf{align}\ n,\ \mathsf{offset}\ m\} : at \to N}$$

$t.\mathsf{load}\ x\ memarg$

The instruction $(nt.\mathsf{load}\ x\ memarg)$ is valid with the instruction type $at \to nt$ if:

- The memory $C.\mathsf{mems}[x]$ exists.
- The memory $C.\mathsf{mems}[x]$ is of the form $(at\ lim\ \mathsf{page})$.
- $memarg$ is valid for at and $|nt|$.

$$\frac{C.\mathsf{mems}[x] = at\ lim\ \mathsf{page} \qquad \vdash memarg : at \to |nt|}{C \vdash nt.\mathsf{load}\ x\ memarg : at \to nt}$$

$t.\mathsf{load}N_sx\ x\ memarg$

The instruction $(\mathrm{i}N.\mathsf{load}K_sx\ x\ memarg)$ is valid with the instruction type $at \to \mathrm{i}N$ if:

- The memory $C.\mathsf{mems}[x]$ exists.
- The memory $C.\mathsf{mems}[x]$ is of the form $(at\ lim\ \mathsf{page})$.
- $memarg$ is valid for at and K.

$$\frac{C.\mathsf{mems}[x] = at\ lim\ \mathsf{page} \qquad \vdash memarg : at \to K}{C \vdash \mathrm{i}N.\mathsf{load}K_sx\ x\ memarg : at \to \mathrm{i}N}$$

$t.\mathsf{store}\ x\ memarg$

The instruction $(nt.\mathsf{store}\ x\ memarg)$ is valid with the instruction type $at\ nt \to \epsilon$ if:

- The memory $C.\mathsf{mems}[x]$ exists.
- The memory $C.\mathsf{mems}[x]$ is of the form $(at\ lim\ \mathsf{page})$.
- $memarg$ is valid for at and $|nt|$.

$$\frac{C.\mathsf{mems}[x] = at\ lim\ \mathsf{page} \qquad \vdash memarg : at \to |nt|}{C \vdash nt.\mathsf{store}\ x\ memarg : at\ nt \to \epsilon}$$

$t.\mathsf{store}N\ x\ memarg$

The instruction $(\mathrm{i}N.\mathsf{store}K\ x\ memarg)$ is valid with the instruction type $at\ \mathrm{i}N \to \epsilon$ if:

- The memory $C.\mathsf{mems}[x]$ exists.
- The memory $C.\mathsf{mems}[x]$ is of the form $(at\ lim\ \mathsf{page})$.
- $memarg$ is valid for at and K.

$$\frac{C.\mathsf{mems}[x] = at\ lim\ \mathsf{page} \qquad \vdash memarg : at \to K}{C \vdash \mathrm{i}N.\mathsf{store}K\ x\ memarg : at\ \mathrm{i}N \to \epsilon}$$

v128.load x $memarg$

The instruction (v128.load x $memarg$) is valid with the instruction type $at \rightarrow$ v128 if:

- The memory $C.\mathsf{mems}[x]$ exists.
- The memory $C.\mathsf{mems}[x]$ is of the form $(at\ lim\ \mathsf{page})$.
- $memarg$ is valid for at and $|\mathsf{v128}|$.

$$\frac{C.\mathsf{mems}[x] = at\ lim\ \mathsf{page} \qquad \vdash memarg : at \rightarrow |\mathsf{v128}|}{C \vdash \mathsf{v128.load}\ x\ memarg : at \rightarrow \mathsf{v128}}$$

v128.load$N$xM_sx x $memarg$

The instruction (v128.load$N$xM_sx x $memarg$) is valid with the instruction type $at \rightarrow$ v128 if:

- The memory $C.\mathsf{mems}[x]$ exists.
- The memory $C.\mathsf{mems}[x]$ is of the form $(at\ lim\ \mathsf{page})$.
- $memarg$ is valid for at and $N \cdot M$.

$$\frac{C.\mathsf{mems}[x] = at\ lim\ \mathsf{page} \qquad \vdash memarg : at \rightarrow N \cdot M}{C \vdash \mathsf{v128.load}N\mathsf{x}M\mathsf{_}sx\ x\ memarg : at \rightarrow \mathsf{v128}}$$

v128.load$N$_splat x $memarg$

The instruction (v128.load$N$_splat x $memarg$) is valid with the instruction type $at \rightarrow$ v128 if:

- The memory $C.\mathsf{mems}[x]$ exists.
- The memory $C.\mathsf{mems}[x]$ is of the form $(at\ lim\ \mathsf{page})$.
- $memarg$ is valid for at and N.

$$\frac{C.\mathsf{mems}[x] = at\ lim\ \mathsf{page} \qquad \vdash memarg : at \rightarrow N}{C \vdash \mathsf{v128.load}N\mathsf{_splat}\ x\ memarg : at \rightarrow \mathsf{v128}}$$

v128.load$N$_zero x $memarg$

The instruction (v128.load$N$_zero x $memarg$) is valid with the instruction type $at \rightarrow$ v128 if:

- The memory $C.\mathsf{mems}[x]$ exists.
- The memory $C.\mathsf{mems}[x]$ is of the form $(at\ lim\ \mathsf{page})$.
- $memarg$ is valid for at and N.

$$\frac{C.\mathsf{mems}[x] = at\ lim\ \mathsf{page} \qquad \vdash memarg : at \rightarrow N}{C \vdash \mathsf{v128.load}N\mathsf{_zero}\ x\ memarg : at \rightarrow \mathsf{v128}}$$

v128.load$N$_lane x $memarg$ $laneidx$

The instruction (v128.load$N$_lane x $memarg$ i) is valid with the instruction type at v128 $\rightarrow$ v128 if:

- The memory $C.\mathsf{mems}[x]$ exists.
- The memory $C.\mathsf{mems}[x]$ is of the form $(at\ lim\ \mathsf{page})$.
- $memarg$ is valid for at and N.
- i is less than $128/N$.

$$\frac{C.\mathsf{mems}[x] = at\ lim\ \mathsf{page} \qquad \vdash memarg : at \rightarrow N \qquad i < 128/N}{C \vdash \mathsf{v128.load}N\mathsf{_lane}\ x\ memarg\ i : at\ \mathsf{v128} \rightarrow \mathsf{v128}}$$

v128.store x $memarg$

The instruction (v128.store x $memarg$) is valid with the instruction type at v128 $\rightarrow$ ϵ if:

- The memory C.mems$[x]$ exists.
- The memory C.mems$[x]$ is of the form $(at\ lim\ \text{page})$.
- $memarg$ is valid for at and $|\text{v128}|$.

$$\frac{C.\text{mems}[x] = at\ lim\ \text{page} \qquad \vdash memarg : at \rightarrow |\text{v128}|}{C \vdash \text{v128.store}\ x\ memarg : at\ \text{v128} \rightarrow \epsilon}$$

v128.store$N$_lane x $memarg$ $laneidx$

The instruction (v128.store$N$_lane x $memarg$ i) is valid with the instruction type at v128 $\rightarrow$ ϵ if:

- The memory C.mems$[x]$ exists.
- The memory C.mems$[x]$ is of the form $(at\ lim\ \text{page})$.
- $memarg$ is valid for at and N.
- i is less than $128/N$.

$$\frac{C.\text{mems}[x] = at\ lim\ \text{page} \qquad \vdash memarg : at \rightarrow N \qquad i < 128/N}{C \vdash \text{v128.store}N\text{_lane}\ x\ memarg\ i : at\ \text{v128} \rightarrow \epsilon}$$

memory.size x

The instruction (memory.size x) is valid with the instruction type ϵ $\rightarrow$ at if:

- The memory C.mems$[x]$ exists.
- The memory C.mems$[x]$ is of the form $(at\ lim\ \text{page})$.

$$\frac{C.\text{mems}[x] = at\ lim\ \text{page}}{C \vdash \text{memory.size}\ x : \epsilon \rightarrow at}$$

memory.grow x

The instruction (memory.grow x) is valid with the instruction type at $\rightarrow$ at if:

- The memory C.mems$[x]$ exists.
- The memory C.mems$[x]$ is of the form $(at\ lim\ \text{page})$.

$$\frac{C.\text{mems}[x] = at\ lim\ \text{page}}{C \vdash \text{memory.grow}\ x : at \rightarrow at}$$

memory.fill x

The instruction (memory.fill x) is valid with the instruction type at i32 at $\rightarrow$ ϵ if:

- The memory C.mems$[x]$ exists.
- The memory C.mems$[x]$ is of the form $(at\ lim\ \text{page})$.

$$\frac{C.\text{mems}[x] = at\ lim\ \text{page}}{C \vdash \text{memory.fill}\ x : at\ \text{i32}\ at \rightarrow \epsilon}$$

memory.copy x y

The instruction (memory.copy x_1 x_2) is valid with the instruction type at_1 at_2 $addrtype$ $\rightarrow$ ϵ if:

- The memory C.mems$[x_1]$ exists.
- The memory C.mems$[x_1]$ is of the form $(at_1\ lim_1\ \text{page})$.
- The memory C.mems$[x_2]$ exists.

- The memory $C.\mathsf{mems}[x_2]$ is of the form $(at_2\ lim_2\ \mathsf{page})$.
- The address type $addrtype$ is $\min(at_1, at_2)$.

$$\frac{C.\mathsf{mems}[x_1] = at_1\ lim_1\ \mathsf{page} \qquad C.\mathsf{mems}[x_2] = at_2\ lim_2\ \mathsf{page}}{C \vdash \mathsf{memory.copy}\ x_1\ x_2 : at_1\ at_2\ \min(at_1, at_2) \to \epsilon}$$

memory.init $x\ y$

The instruction $(\mathsf{memory.init}\ x\ y)$ is valid with the instruction type $at\ \mathsf{i32}\ \mathsf{i32} \to \epsilon$ if:

- The memory $C.\mathsf{mems}[x]$ exists.
- The memory $C.\mathsf{mems}[x]$ is of the form $(at\ lim\ \mathsf{page})$.
- The data segment $C.\mathsf{datas}[y]$ exists.
- The data segment $C.\mathsf{datas}[y]$ is of the form ok.

$$\frac{C.\mathsf{mems}[x] = at\ lim\ \mathsf{page} \qquad C.\mathsf{datas}[y] = \mathsf{ok}}{C \vdash \mathsf{memory.init}\ x\ y : at\ \mathsf{i32}\ \mathsf{i32} \to \epsilon}$$

data.drop x

The instruction $(\mathsf{data.drop}\ x)$ is valid with the instruction type $\epsilon \to \epsilon$ if:

- The data segment $C.\mathsf{datas}[x]$ exists.
- The data segment $C.\mathsf{datas}[x]$ is of the form ok.

$$\frac{C.\mathsf{datas}[x] = \mathsf{ok}}{C \vdash \mathsf{data.drop}\ x : \epsilon \to \epsilon}$$

3.4.6 Reference Instructions

ref.null ht

The instruction $(\mathsf{ref.null}\ ht)$ is valid with the instruction type $\epsilon \to (\mathsf{ref\ null}\ ht)$ if:

- The heap type ht is valid.

$$\frac{C \vdash ht : \mathsf{ok}}{C \vdash \mathsf{ref.null}\ ht : \epsilon \to (\mathsf{ref\ null}\ ht)}$$

ref.func x

The instruction $(\mathsf{ref.func}\ x)$ is valid with the instruction type $\epsilon \to (\mathsf{ref}\ dt)$ if:

- The function $C.\mathsf{funcs}[x]$ exists.
- The function $C.\mathsf{funcs}[x]$ is of the form dt.
- x is contained in $C.\mathsf{refs}$.

$$\frac{C.\mathsf{funcs}[x] = dt \qquad x \in C.\mathsf{refs}}{C \vdash \mathsf{ref.func}\ x : \epsilon \to (\mathsf{ref}\ dt)}$$

ref.is_null

The instruction ref.is_null is valid with the instruction type $(\mathsf{ref\ null}\ ht) \to \mathsf{i32}$ if:

- The heap type ht is valid.

$$\frac{C \vdash ht : \mathsf{ok}}{C \vdash \mathsf{ref.is_null} : (\mathsf{ref\ null}\ ht) \to \mathsf{i32}}$$

ref.as_non_null

The instruction ref.as_non_null is valid with the instruction type $(\mathsf{ref\ null}\ ht) \to (\mathsf{ref}\ ht)$ if:

- The heap type ht is valid.

$$\frac{C \vdash ht : \mathsf{ok}}{C \vdash \mathsf{ref.as_non_null} : (\mathsf{ref\ null}\ ht) \to (\mathsf{ref}\ ht)}$$

ref.eq

The instruction ref.eq is valid with the instruction type $(\mathsf{ref\ null\ eq})\ (\mathsf{ref\ null\ eq}) \to \mathsf{i32}$.

$$\frac{}{C \vdash \mathsf{ref.eq} : (\mathsf{ref\ null\ eq})\ (\mathsf{ref\ null\ eq}) \to \mathsf{i32}}$$

ref.test rt

The instruction $(\mathsf{ref.test}\ rt)$ is valid with the instruction type $rt' \to \mathsf{i32}$ if:

- The reference type rt is valid.
- The reference type rt' is valid.
- The reference type rt matches the reference type rt'.

$$\frac{C \vdash rt : \mathsf{ok} \qquad C \vdash rt' : \mathsf{ok} \qquad C \vdash rt \leq rt'}{C \vdash \mathsf{ref.test}\ rt : rt' \to \mathsf{i32}}$$

Note

The liberty to pick a supertype rt' allows typing the instruction with the least precise super type of rt as input, that is, the top type in the corresponding heap subtyping hierarchy.

ref.cast rt

The instruction $(\mathsf{ref.cast}\ rt)$ is valid with the instruction type $rt' \to rt$ if:

- The reference type rt is valid.
- The reference type rt' is valid.
- The reference type rt matches the reference type rt'.

$$\frac{C \vdash rt : \mathsf{ok} \qquad C \vdash rt' : \mathsf{ok} \qquad C \vdash rt \leq rt'}{C \vdash \mathsf{ref.cast}\ rt : rt' \to rt}$$

Note

The liberty to pick a supertype rt' allows typing the instruction with the least precise super type of rt as input, that is, the top type in the corresponding heap subtyping hierarchy.

3.4.7 Aggregate Reference Instructions

struct.new x

The instruction $(\mathsf{struct.new}\ x)$ is valid with the instruction type $t^* \to (\mathsf{ref}\ x)$ if:

- The type $C.\mathsf{types}[x]$ exists.
- The expansion of $C.\mathsf{types}[x]$ is $(\mathsf{struct}\ (\mathsf{mut}^?\ zt)^*)$.
- The value type sequence t^* is $\mathrm{unpack}(zt)^*$.

$$\frac{C.\mathsf{types}[x] \approx \mathsf{struct}\ (\mathsf{mut}^?\ zt)^*}{C \vdash \mathsf{struct.new}\ x : \mathrm{unpack}(zt)^* \to (\mathsf{ref}\ x)}$$

struct.new_default x

The instruction (struct.new_default x) is valid with the instruction type $\epsilon \to (\mathsf{ref}\ x)$ if:

- The type $C.\mathsf{types}[x]$ exists.
- The expansion of $C.\mathsf{types}[x]$ is $(\mathsf{struct}\ (\mathsf{mut}^?\ zt)^*)$.
- For all zt in zt^*:
  - A default value for $\mathrm{unpack}(zt)$ is defined.

$$\frac{C.\mathsf{types}[x] \approx \mathsf{struct}\ (\mathsf{mut}^?\ zt)^* \qquad (\mathrm{default}_{\mathrm{unpack}(zt)} \neq \epsilon)^*}{C \vdash \mathsf{struct.new_default}\ x : \epsilon \to (\mathsf{ref}\ x)}$$

struct.get_$sx^?$ x y

The instruction (struct.get_$sx^?$ x i) is valid with the instruction type $(\mathsf{ref\ null}\ x) \to t$ if:

- The type $C.\mathsf{types}[x]$ exists.
- The expansion of $C.\mathsf{types}[x]$ is $(\mathsf{struct}\ ft^*)$.
- The length of ft^* is greater than i.
- The field type $ft^*[i]$ is of the form $(\mathsf{mut}^?\ zt)$.
- The signedness $sx^?$ is present if and only if zt is a packed type.
- The value type t is $\mathrm{unpack}(zt)$.

$$\frac{C.\mathsf{types}[x] \approx \mathsf{struct}\ ft^* \qquad ft^*[i] = \mathsf{mut}^?\ zt \qquad sx^? \neq \epsilon \Leftrightarrow zt \neq \mathrm{unpack}(zt)}{C \vdash \mathsf{struct.get_}sx^?\ x\ i : (\mathsf{ref\ null}\ x) \to \mathrm{unpack}(zt)}$$

struct.set x y

The instruction (struct.set x i) is valid with the instruction type $(\mathsf{ref\ null}\ x)\ t \to \epsilon$ if:

- The type $C.\mathsf{types}[x]$ exists.
- The expansion of $C.\mathsf{types}[x]$ is $(\mathsf{struct}\ ft^*)$.
- The length of ft^* is greater than i.
- The field type $ft^*[i]$ is of the form $(\mathsf{mut}\ zt)$.
- The value type t is $\mathrm{unpack}(zt)$.

$$\frac{C.\mathsf{types}[x] \approx \mathsf{struct}\ ft^* \qquad ft^*[i] = \mathsf{mut}\ zt}{C \vdash \mathsf{struct.set}\ x\ i : (\mathsf{ref\ null}\ x)\ \mathrm{unpack}(zt) \to \epsilon}$$

array.new x

The instruction (array.new x) is valid with the instruction type $t\ \mathsf{i32} \to (\mathsf{ref}\ x)$ if:

- The type $C.\mathsf{types}[x]$ exists.
- The expansion of $C.\mathsf{types}[x]$ is $(\mathsf{array}\ (\mathsf{mut}^?\ zt))$.
- The value type t is $\mathrm{unpack}(zt)$.

$$\frac{C.\mathsf{types}[x] \approx \mathsf{array}\ (\mathsf{mut}^?\ zt)}{C \vdash \mathsf{array.new}\ x : \mathrm{unpack}(zt)\ \mathsf{i32} \to (\mathsf{ref}\ x)}$$

array.new_default x

The instruction (array.new_default x) is valid with the instruction type i32 $\rightarrow$ (ref x) if:

- The type $C.\mathsf{types}[x]$ exists.
- The expansion of $C.\mathsf{types}[x]$ is $(\mathsf{array}\ (\mathsf{mut}^?\ zt))$.
- A default value for $\mathrm{unpack}(zt)$ is defined.

$$\frac{C.\mathsf{types}[x] \approx \mathsf{array}\ (\mathsf{mut}^?\ zt) \qquad \mathrm{default}_{\mathrm{unpack}(zt)} \neq \epsilon}{C \vdash \mathsf{array.new_default}\ x : \mathsf{i32} \rightarrow (\mathsf{ref}\ x)}$$

array.new_fixed x n

The instruction (array.new_fixed x n) is valid with the instruction type t^n $\rightarrow$ (ref x) if:

- The type $C.\mathsf{types}[x]$ exists.
- The expansion of $C.\mathsf{types}[x]$ is $(\mathsf{array}\ (\mathsf{mut}^?\ zt))$.
- The value type t is $\mathrm{unpack}(zt)$.

$$\frac{C.\mathsf{types}[x] \approx \mathsf{array}\ (\mathsf{mut}^?\ zt)}{C \vdash \mathsf{array.new_fixed}\ x\ n : \mathrm{unpack}(zt)^n \rightarrow (\mathsf{ref}\ x)}$$

array.new_elem x y

The instruction (array.new_elem x y) is valid with the instruction type i32 i32 $\rightarrow$ (ref x) if:

- The type $C.\mathsf{types}[x]$ exists.
- The expansion of $C.\mathsf{types}[x]$ is $(\mathsf{array}\ (\mathsf{mut}^?\ rt))$.
- The element segment $C.\mathsf{elems}[y]$ exists.
- The element segment $C.\mathsf{elems}[y]$ matches the reference type rt.

$$\frac{C.\mathsf{types}[x] \approx \mathsf{array}\ (\mathsf{mut}^?\ rt) \qquad C \vdash C.\mathsf{elems}[y] \leq rt}{C \vdash \mathsf{array.new_elem}\ x\ y : \mathsf{i32}\ \mathsf{i32} \rightarrow (\mathsf{ref}\ x)}$$

array.new_data x y

The instruction (array.new_data x y) is valid with the instruction type i32 i32 $\rightarrow$ (ref x) if:

- The type $C.\mathsf{types}[x]$ exists.
- The expansion of $C.\mathsf{types}[x]$ is $(\mathsf{array}\ (\mathsf{mut}^?\ zt))$.
- The value type $\mathrm{unpack}(zt)$ is of the form $\mathit{numtype}$ or $\mathrm{unpack}(zt)$ is of the form $\mathit{vectype}$.
- The data segment $C.\mathsf{datas}[y]$ exists.
- The data segment $C.\mathsf{datas}[y]$ is of the form ok.

$$\frac{C.\mathsf{types}[x] \approx \mathsf{array}\ (\mathsf{mut}^?\ zt) \qquad \mathrm{unpack}(zt) = \mathit{numtype} \vee \mathrm{unpack}(zt) = \mathit{vectype} \qquad C.\mathsf{datas}[y] = \mathsf{ok}}{C \vdash \mathsf{array.new_data}\ x\ y : \mathsf{i32}\ \mathsf{i32} \rightarrow (\mathsf{ref}\ x)}$$

array.get_$sx^?$ x

The instruction (array.get_$sx^?$ x) is valid with the instruction type (ref null x) i32 $\rightarrow$ t if:

- The type $C.\mathsf{types}[x]$ exists.
- The expansion of $C.\mathsf{types}[x]$ is $(\mathsf{array}\ (\mathsf{mut}^?\ zt))$.
- The signedness $sx^?$ is present if and only if zt is a packed type.
- The value type t is $\mathrm{unpack}(zt)$.

$$\frac{C.\mathsf{types}[x] \approx \mathsf{array}\ (\mathsf{mut}^?\ zt) \qquad sx^? \neq \epsilon \Leftrightarrow zt \neq \mathrm{unpack}(zt)}{C \vdash \mathsf{array.get_}sx^?\ x : (\mathsf{ref\ null}\ x)\ \mathsf{i32} \to \mathrm{unpack}(zt)}$$

array.set x

The instruction $(\mathsf{array.set}\ x)$ is valid with the instruction type $(\mathsf{ref\ null}\ x)\ \mathsf{i32}\ t\ \to\ \epsilon$ if:

- The type $C.\mathsf{types}[x]$ exists.
- The expansion of $C.\mathsf{types}[x]$ is $(\mathsf{array}\ (\mathsf{mut}\ zt))$.
- The value type t is $\mathrm{unpack}(zt)$.

$$\frac{C.\mathsf{types}[x] \approx \mathsf{array}\ (\mathsf{mut}\ zt)}{C \vdash \mathsf{array.set}\ x : (\mathsf{ref\ null}\ x)\ \mathsf{i32}\ \mathrm{unpack}(zt) \to \epsilon}$$

array.len

The instruction array.len is valid with the instruction type $(\mathsf{ref\ null\ array})\ \to\ \mathsf{i32}$.

$$\frac{}{C \vdash \mathsf{array.len} : (\mathsf{ref\ null\ array}) \to \mathsf{i32}}$$

array.fill x

The instruction $(\mathsf{array.fill}\ x)$ is valid with the instruction type $(\mathsf{ref\ null}\ x)\ \mathsf{i32}\ t\ \mathsf{i32}\ \to\ \epsilon$ if:

- The type $C.\mathsf{types}[x]$ exists.
- The expansion of $C.\mathsf{types}[x]$ is $(\mathsf{array}\ (\mathsf{mut}\ zt))$.
- The value type t is $\mathrm{unpack}(zt)$.

$$\frac{C.\mathsf{types}[x] \approx \mathsf{array}\ (\mathsf{mut}\ zt)}{C \vdash \mathsf{array.fill}\ x : (\mathsf{ref\ null}\ x)\ \mathsf{i32}\ \mathrm{unpack}(zt)\ \mathsf{i32} \to \epsilon}$$

array.copy $x\ y$

The instruction $(\mathsf{array.copy}\ x_1\ x_2)$ is valid with the instruction type $(\mathsf{ref\ null}\ x_1)\ \mathsf{i32}\ (\mathsf{ref\ null}\ x_2)\ \mathsf{i32}\ \mathsf{i32}\ \to\ \epsilon$ if:

- The type $C.\mathsf{types}[x_1]$ exists.
- The expansion of $C.\mathsf{types}[x_1]$ is $(\mathsf{array}\ (\mathsf{mut}\ zt_1))$.
- The type $C.\mathsf{types}[x_2]$ exists.
- The expansion of $C.\mathsf{types}[x_2]$ is $(\mathsf{array}\ (\mathsf{mut}^?\ zt_2))$.
- The storage type zt_2 matches the storage type zt_1.

$$\frac{C.\mathsf{types}[x_1] \approx \mathsf{array}\ (\mathsf{mut}\ zt_1) \qquad C.\mathsf{types}[x_2] \approx \mathsf{array}\ (\mathsf{mut}^?\ zt_2) \qquad C \vdash zt_2 \leq zt_1}{C \vdash \mathsf{array.copy}\ x_1\ x_2 : (\mathsf{ref\ null}\ x_1)\ \mathsf{i32}\ (\mathsf{ref\ null}\ x_2)\ \mathsf{i32}\ \mathsf{i32} \to \epsilon}$$

array.init_elem $x\ y$

The instruction $(\mathsf{array.init_elem}\ x\ y)$ is valid with the instruction type $(\mathsf{ref\ null}\ x)\ \mathsf{i32}\ \mathsf{i32}\ \mathsf{i32}\ \to\ \epsilon$ if:

- The type $C.\mathsf{types}[x]$ exists.
- The expansion of $C.\mathsf{types}[x]$ is $(\mathsf{array}\ (\mathsf{mut}\ zt))$.
- The element segment $C.\mathsf{elems}[y]$ exists.
- The element segment $C.\mathsf{elems}[y]$ matches the storage type zt.

$$\frac{C.\mathsf{types}[x] \approx \mathsf{array}\ (\mathsf{mut}\ zt) \qquad C \vdash C.\mathsf{elems}[y] \leq zt}{C \vdash \mathsf{array.init_elem}\ x\ y : (\mathsf{ref\ null}\ x)\ \mathsf{i32}\ \mathsf{i32}\ \mathsf{i32} \to \epsilon}$$

array.init_data x y

The instruction (array.init_data x y) is valid with the instruction type (ref null x) i32 i32 i32 $\rightarrow$ ϵ if:

- The type $C.\mathsf{types}[x]$ exists.
- The expansion of $C.\mathsf{types}[x]$ is (array (mut zt)).
- The value type $\mathrm{unpack}(zt)$ is of the form $numtype$ or $\mathrm{unpack}(zt)$ is of the form $vectype$.
- The data segment $C.\mathsf{datas}[y]$ exists.
- The data segment $C.\mathsf{datas}[y]$ is of the form ok.

$$\frac{C.\mathsf{types}[x] \approx \mathsf{array}\ (\mathsf{mut}\ zt) \qquad \mathrm{unpack}(zt) = numtype \vee \mathrm{unpack}(zt) = vectype \qquad C.\mathsf{datas}[y] = \mathsf{ok}}{C \vdash \mathsf{array.init_data}\ x\ y : (\mathsf{ref\ null}\ x)\ \mathsf{i32}\ \mathsf{i32}\ \mathsf{i32} \rightarrow \epsilon}$$

3.4.8 Scalar Reference Instructions

ref.i31

The instruction ref.i31 is valid with the instruction type i32 $\rightarrow$ (ref i31).

$$\frac{}{C \vdash \mathsf{ref.i31} : \mathsf{i32} \rightarrow (\mathsf{ref}\ \mathsf{i31})}$$

i31.get_sx

The instruction (i31.get_sx) is valid with the instruction type (ref null i31) $\rightarrow$ i32.

$$\frac{}{C \vdash \mathsf{i31.get_}sx : (\mathsf{ref\ null}\ \mathsf{i31}) \rightarrow \mathsf{i32}}$$

3.4.9 External Reference Instructions

any.convert_extern

The instruction any.convert_extern is valid with the instruction type (ref null$_1^?$ extern) $\rightarrow$ (ref null$_2^?$ any) if:

- null$_1^?$ is of the form null$_2^?$.

$$\frac{\mathsf{null}_1^? = \mathsf{null}_2^?}{C \vdash \mathsf{any.convert_extern} : (\mathsf{ref}\ \mathsf{null}_1^?\ \mathsf{extern}) \rightarrow (\mathsf{ref}\ \mathsf{null}_2^?\ \mathsf{any})}$$

extern.convert_any

The instruction extern.convert_any is valid with the instruction type (ref null$_1^?$ any) $\rightarrow$ (ref null$_2^?$ extern) if:

- null$_1^?$ is of the form null$_2^?$.

$$\frac{\mathsf{null}_1^? = \mathsf{null}_2^?}{C \vdash \mathsf{extern.convert_any} : (\mathsf{ref}\ \mathsf{null}_1^?\ \mathsf{any}) \rightarrow (\mathsf{ref}\ \mathsf{null}_2^?\ \mathsf{extern})}$$

3.4.10 Numeric Instructions

t.const c

The instruction (nt.const c_{nt}) is valid with the instruction type ϵ $\rightarrow$ nt.

$$\frac{}{C \vdash nt.\mathsf{const}\ c_{nt} : \epsilon \rightarrow nt}$$

$t.unop$

The instruction $(nt.unop_{nt})$ is valid with the instruction type $nt \ \rightarrow \ nt$.

$$\frac{}{C \vdash nt.unop_{nt} : nt \rightarrow nt}$$

$t.binop$

The instruction $(nt.binop_{nt})$ is valid with the instruction type $nt \ nt \ \rightarrow \ nt$.

$$\frac{}{C \vdash nt.binop_{nt} : nt \ nt \rightarrow nt}$$

$t.testop$

The instruction $(nt.testop_{nt})$ is valid with the instruction type $nt \ \rightarrow \ \mathsf{i32}$.

$$\frac{}{C \vdash nt.testop_{nt} : nt \rightarrow \mathsf{i32}}$$

$t.relop$

The instruction $(nt.relop_{nt})$ is valid with the instruction type $nt \ nt \ \rightarrow \ \mathsf{i32}$.

$$\frac{}{C \vdash nt.relop_{nt} : nt \ nt \rightarrow \mathsf{i32}}$$

$t_1.cvtop_t_2_sx^?$

The instruction $(nt_1.cvtop_nt_2)$ is valid with the instruction type $nt_2 \ \rightarrow \ nt_1$.

$$\frac{}{C \vdash nt_1.cvtop_nt_2 : nt_2 \rightarrow nt_1}$$

3.4.11 Vector Instructions

Vector instructions can have a prefix to describe the shape of the operand. Packed numeric types, $\mathsf{i8}$ and $\mathsf{i16}$, are not value types. An auxiliary function maps such packed type shapes to value types:

$$\mathrm{unpack}(\mathsf{i}N\mathsf{x}M) \quad = \quad \mathrm{unpack}(\mathsf{i}N)$$

$\mathsf{v128.const}\ c$

The instruction $(\mathsf{v128.const}\ c)$ is valid with the instruction type $\epsilon \ \rightarrow \ \mathsf{v128}$.

$$\frac{}{C \vdash \mathsf{v128.const}\ c : \epsilon \rightarrow \mathsf{v128}}$$

$\mathsf{v128}.vvunop$

The instruction $(\mathsf{v128}.vvunop)$ is valid with the instruction type $\mathsf{v128} \ \rightarrow \ \mathsf{v128}$.

$$\frac{}{C \vdash \mathsf{v128}.vvunop : \mathsf{v128} \rightarrow \mathsf{v128}}$$

$\mathsf{v128}.vvbinop$

The instruction $(\mathsf{v128}.vvbinop)$ is valid with the instruction type $\mathsf{v128}\ \mathsf{v128} \ \rightarrow \ \mathsf{v128}$.

$$\frac{}{C \vdash \mathsf{v128}.vvbinop : \mathsf{v128}\ \mathsf{v128} \rightarrow \mathsf{v128}}$$

v128.$vvternop$

The instruction ($\mathsf{v128}.vvternop$) is valid with the instruction type $\mathsf{v128}\ \mathsf{v128}\ \mathsf{v128} \to \mathsf{v128}$.

$$\frac{}{C \vdash \mathsf{v128}.vvternop : \mathsf{v128}\ \mathsf{v128}\ \mathsf{v128} \to \mathsf{v128}}$$

v128.$vvtestop$

The instruction ($\mathsf{v128}.vvtestop$) is valid with the instruction type $\mathsf{v128} \to \mathsf{i32}$.

$$\frac{}{C \vdash \mathsf{v128}.vvtestop : \mathsf{v128} \to \mathsf{i32}}$$

$shape.vunop$

The instruction ($sh.vunop$) is valid with the instruction type $\mathsf{v128} \to \mathsf{v128}$.

$$\frac{}{C \vdash sh.vunop : \mathsf{v128} \to \mathsf{v128}}$$

$shape.vbinop$

The instruction ($sh.vbinop$) is valid with the instruction type $\mathsf{v128}\ \mathsf{v128} \to \mathsf{v128}$.

$$\frac{}{C \vdash sh.vbinop : \mathsf{v128}\ \mathsf{v128} \to \mathsf{v128}}$$

$shape.vternop$

The instruction ($sh.vternop$) is valid with the instruction type $\mathsf{v128}\ \mathsf{v128}\ \mathsf{v128} \to \mathsf{v128}$.

$$\frac{}{C \vdash sh.vternop : \mathsf{v128}\ \mathsf{v128}\ \mathsf{v128} \to \mathsf{v128}}$$

$shape.vtestop$

The instruction ($sh.vtestop$) is valid with the instruction type $\mathsf{v128} \to \mathsf{i32}$.

$$\frac{}{C \vdash sh.vtestop : \mathsf{v128} \to \mathsf{i32}}$$

$shape.vrelop$

The instruction ($sh.vrelop$) is valid with the instruction type $\mathsf{v128}\ \mathsf{v128} \to \mathsf{v128}$.

$$\frac{}{C \vdash sh.vrelop : \mathsf{v128}\ \mathsf{v128} \to \mathsf{v128}}$$

$ishape.vishiftop$

The instruction ($sh.vshiftop$) is valid with the instruction type $\mathsf{v128}\ \mathsf{i32} \to \mathsf{v128}$.

$$\frac{}{C \vdash sh.vshiftop : \mathsf{v128}\ \mathsf{i32} \to \mathsf{v128}}$$

$ishape$.bitmask

The instruction (sh.bitmask) is valid with the instruction type $\mathsf{v128} \to \mathsf{i32}$.

$$\frac{}{C \vdash sh.\mathsf{bitmask} : \mathsf{v128} \to \mathsf{i32}}$$

i8x16.$vswizzlop$

The instruction ($sh.vswizzlop$) is valid with the instruction type $\mathsf{v128}\ \mathsf{v128} \to \mathsf{v128}$.

$$\frac{}{C \vdash sh.vswizzlop : \mathsf{v128}\ \mathsf{v128} \to \mathsf{v128}}$$

i8x16.shuffle $laneidx^{16}$

The instruction $(sh.\text{shuffle}\ i^*)$ is valid with the instruction type $\text{v128}\ \text{v128} \to \text{v128}$ if:

- For all i in i^*:
  - The lane index i is less than $2 \cdot \dim(sh)$.

$$\frac{(i < 2 \cdot \dim(sh))^*}{C \vdash sh.\text{shuffle}\ i^* : \text{v128}\ \text{v128} \to \text{v128}}$$

$shape$.splat

The instruction $(sh.\text{splat})$ is valid with the instruction type $numtype \to \text{v128}$ if:

- The number type $numtype$ is $\text{unpack}(sh)$.

$$\frac{}{C \vdash sh.\text{splat} : \text{unpack}(sh) \to \text{v128}}$$

$shape$.extract_lane_$sx^?$ $laneidx$

The instruction $(sh.\text{extract_lane_}sx^?\ i)$ is valid with the instruction type $\text{v128} \to numtype$ if:

- The lane index i is less than $\dim(sh)$.
- The number type $numtype$ is $\text{unpack}(sh)$.

$$\frac{i < \dim(sh)}{C \vdash sh.\text{extract_lane_}sx^?\ i : \text{v128} \to \text{unpack}(sh)}$$

$shape$.replace_lane $laneidx$

The instruction $(sh.\text{replace_lane}\ i)$ is valid with the instruction type $\text{v128}\ numtype \to \text{v128}$ if:

- The lane index i is less than $\dim(sh)$.
- The number type $numtype$ is $\text{unpack}(sh)$.

$$\frac{i < \dim(sh)}{C \vdash sh.\text{replace_lane}\ i : \text{v128}\ \text{unpack}(sh) \to \text{v128}}$$

$ishape_1.vextunop_ishape_2$

The instruction $(sh_1.vextunop_sh_2)$ is valid with the instruction type $\text{v128} \to \text{v128}$.

$$\frac{}{C \vdash sh_1.vextunop_sh_2 : \text{v128} \to \text{v128}}$$

$ishape_1.vextbinop_ishape_2$

The instruction $(sh_1.vextbinop_sh_2)$ is valid with the instruction type $\text{v128}\ \text{v128} \to \text{v128}$.

$$\frac{}{C \vdash sh_1.vextbinop_sh_2 : \text{v128}\ \text{v128} \to \text{v128}}$$

$ishape_1.vextternop_ishape_2$

The instruction $(sh_1.vextternop_sh_2)$ is valid with the instruction type $\text{v128}\ \text{v128}\ \text{v128} \to \text{v128}$.

$$\frac{}{C \vdash sh_1.vextternop_sh_2 : \text{v128}\ \text{v128}\ \text{v128} \to \text{v128}}$$

$ishape_1$.narrow_$ishape_2$_sx

The instruction $(sh_1.\text{narrow_}sh_2\text{_}sx)$ is valid with the instruction type $\text{v128}\ \text{v128} \to \text{v128}$.

$$\frac{}{C \vdash sh_1.\text{narrow_}sh_2\text{_}sx : \text{v128}\ \text{v128} \to \text{v128}}$$

$shape.vcvtop_half^?_shape_sx^?_\mathsf{zero}^?$

The instruction $(sh_1.vcvtop_sh_2)$ is valid with the instruction type $\mathsf{v128} \to \mathsf{v128}$.

$$\frac{}{C \vdash sh_1.vcvtop_sh_2 : \mathsf{v128} \to \mathsf{v128}}$$

3.4.12 Instruction Sequences

Typing of instruction sequences is defined recursively.

The instruction sequence $instr^*$ is valid with the instruction type it if:

- Either:
  - The instruction sequence $instr^*$ is empty.
  - The instruction type it is of the form $\epsilon \to \epsilon$.
- Or:
  - The instruction sequence $instr^*$ is of the form $instr'$.
  - The instruction type it is of the form $t_1^* \to_{x^*} t_2^*$.
  - The instruction $instr'$ is valid with the instruction type $t_1^* \to_{x^*} t_2^*$.
- Or:
  - The instruction sequence $instr^*$ is of the form $instr_1^*\ instr_2^*$.
  - The instruction type it is of the form $t_1^* \to_{x_1^*\ x_2^*} t_3^*$.
  - The instruction sequence $instr_1^*$ is valid with the instruction type $t_1^* \to_{x_1^*} t_2^*$.
  - For all x_1 in x_1^*:
    - The local $C.\mathsf{locals}[x_1]$ exists.
    - The local $C.\mathsf{locals}[x_1]$ is of the form $(init\ t)$.
  - Under the context C with the local types of x_1^* updated to $(\mathsf{set}\ t)^*$, the instruction sequence $instr_2^*$ is valid with the instruction type $t_2^* \to_{x_2^*} t_3^*$.
- Or:
  - The instruction sequence $instr^*$ is valid with the instruction type it''.
  - The instruction type it'' matches the instruction type it.
  - The instruction type it is valid.
- Or:
  - The instruction type it is of the form $t^*\ t_1^* \to_{x^*} t^*\ t_2^*$.
  - The instruction sequence $instr^*$ is valid with the instruction type $t_1^* \to_{x^*} t_2^*$.
  - The result type t^* is valid.

$$\frac{}{C \vdash \epsilon : \epsilon \to \epsilon}$$

$$\frac{C \vdash instr_1^* : t_1^* \to_{x_1^*} t_2^* \qquad (C.\mathsf{locals}[x_1] = init\ t)^* \qquad C[.\mathsf{local}[x_1^*] = (\mathsf{set}\ t)^*] \vdash instr_2^* : t_2^* \to_{x_2^*} t_3^*}{C \vdash instr_1^*\ instr_2^* : t_1^* \to_{x_1^*\ x_2^*} t_3^*}$$

$$\frac{C \vdash instr^* : it \qquad C \vdash it \leq it' \qquad C \vdash it' : \mathsf{ok}}{C \vdash instr^* : it'}$$

Note

This *subsumption rule* allows to weaken the type of an instruction sequence to a supertype, which includes the ability to drop init variables x^* from the instruction type in a context where they are not needed, for example, at the end of the body of a block.

$$\frac{C \vdash instr^* : t_1^* \to_{x^*} t_2^* \qquad C \vdash t^* : \mathsf{ok}}{C \vdash instr^* : (t^*\ t_1^*) \to_{x^*} (t^*\ t_2^*)}$$

Note

In combination with the previous two rules, this *frame rule* allows to compose instructions whose types would not directly fit otherwise. For example, consider the instruction sequence

$$(\mathsf{i32.const}\ 1)\ (\mathsf{i32.const}\ 2)\ (\mathsf{i32.add})$$

To type this sequence, its subsequence $(\mathsf{i32.const}\ 2)\ (\mathsf{i32.add})$ needs to be valid with an intermediate type. But the direct type of $(\mathsf{i32.const}\ 2)$ is $\epsilon \to \mathsf{i32}$, not matching the two inputs expected by $\mathsf{i32.add}$. The rule allows to weaken the type of $(\mathsf{i32.const}\ 2)$ to the supertype $\mathsf{i32} \to \mathsf{i32}\ \mathsf{i32}$, such that it can be composed with $\mathsf{i32.add}$ and yields the intermediate type $\mathsf{i32} \to \mathsf{i32}\ \mathsf{i32}$ for the subsequence. That can in turn be composed with the first constant.

3.4.13 Expressions

Expressions $expr$ are classified by result types t^*.

The expression $instr^*$ is valid with the result type t^* if:

- The instruction sequence $instr^*$ is valid with the instruction type $\epsilon \to t^*$.

$$\frac{C \vdash instr^* : \epsilon \to t^*}{C \vdash instr^* : t^*}$$

Constant Expressions

In a *constant* expression, all instructions must be constant.

$instr^*$ is constant if:

- For all $instr$ in $instr^*$:
  - $instr$ is constant.

$instr$ is constant if:

- Either:
  - The instruction $instr$ is of the form $(nt.\mathsf{const}\ c_{nt})$.
- Or:
  - The instruction $instr$ is of the form $(vt.\mathsf{const}\ c_{vt})$.
- Or:
  - The instruction $instr$ is of the form $(\mathsf{ref.null}\ ht)$.
- Or:
  - The instruction $instr$ is of the form $\mathsf{ref.i31}$.
- Or:
  - The instruction $instr$ is of the form $(\mathsf{ref.func}\ x)$.

- Or:
  - The instruction $instr$ is of the form $(\mathsf{struct.new}\ x)$.
- Or:
  - The instruction $instr$ is of the form $(\mathsf{struct.new_default}\ x)$.
- Or:
  - The instruction $instr$ is of the form $(\mathsf{array.new}\ x)$.
- Or:
  - The instruction $instr$ is of the form $(\mathsf{array.new_default}\ x)$.
- Or:
  - The instruction $instr$ is of the form $(\mathsf{array.new_fixed}\ x\ n)$.
- Or:
  - The instruction $instr$ is of the form $\mathsf{any.convert_extern}$.
- Or:
  - The instruction $instr$ is of the form $\mathsf{extern.convert_any}$.
- Or:
  - The instruction $instr$ is of the form $(\mathsf{global.get}\ x)$.
  - The global $C.\mathsf{globals}[x]$ exists.
  - The global $C.\mathsf{globals}[x]$ is of the form $(\epsilon\ t)$.
- Or:
  - The instruction $instr$ is of the form $(\mathsf{i}N.binop)$.
  - $\mathsf{i}N$ is contained in [i32; i64].
  - $binop$ is contained in [add; sub; mul].

$$\frac{(C \vdash instr\ \mathsf{const})^*}{C \vdash instr^*\ \mathsf{const}}$$

$$\frac{}{C \vdash (nt.\mathsf{const}\ c_{nt})\ \mathsf{const}} \qquad \frac{}{C \vdash (vt.\mathsf{const}\ c_{vt})\ \mathsf{const}} \qquad \frac{\mathsf{i}N \in \mathsf{i32}\ \mathsf{i64} \qquad binop \in \mathsf{add}\ \mathsf{sub}\ \mathsf{mul}}{C \vdash (\mathsf{i}N.binop)\ \mathsf{const}}$$

$$\frac{}{C \vdash (\mathsf{ref.null}\ ht)\ \mathsf{const}} \qquad \frac{}{C \vdash (\mathsf{ref.i31})\ \mathsf{const}} \qquad \frac{}{C \vdash (\mathsf{ref.func}\ x)\ \mathsf{const}}$$

$$\frac{}{C \vdash (\mathsf{struct.new}\ x)\ \mathsf{const}} \qquad \frac{}{C \vdash (\mathsf{struct.new_default}\ x)\ \mathsf{const}}$$

$$\frac{}{C \vdash (\mathsf{array.new}\ x)\ \mathsf{const}} \qquad \frac{}{C \vdash (\mathsf{array.new_default}\ x)\ \mathsf{const}} \qquad \frac{}{C \vdash (\mathsf{array.new_fixed}\ x\ n)\ \mathsf{const}}$$

$$\frac{}{C \vdash (\mathsf{any.convert_extern})\ \mathsf{const}} \qquad \frac{}{C \vdash (\mathsf{extern.convert_any})\ \mathsf{const}}$$

$$\frac{C.\mathsf{globals}[x] = t}{C \vdash (\mathsf{global.get}\ x)\ \mathsf{const}}$$

Note

Currently, constant expressions occurring in globals are further constrained in that contained global.get instructions are only allowed to refer to *imported* or *previously defined* globals. Constant expressions occurring in tables may only have global.get instructions that refer to *imported* globals. This is enforced in the validation rule for modules by constraining the context C accordingly.

The definition of constant expression may be extended in future versions of WebAssembly.

3.5 Modules

Modules are valid when all the components they contain are valid. To verify this, most definitions are themselves classified with a suitable type.

3.5.1 Types

The sequence of types defined in a module is validated incrementally, yielding a sequence of defined types representing them individually.

The type definition $(\mathsf{type}\ \mathit{rectype})$ is valid with the defined type sequence dt^* if:

- The length of $C.\mathsf{types}$ is equal to x.
- The defined type sequence dt^* is of the form $\mathrm{roll}^*_x(\mathit{rectype})$.
- Let C' be the same context as C, but with the defined type sequence dt^* appended to the field types.
- Under the context C', the recursive type $\mathit{rectype}$ is valid for the type index x.

$$\frac{x = |C.\mathsf{types}| \qquad dt^* = \mathrm{roll}^*_x(\mathit{rectype}) \qquad C \oplus \{\mathsf{types}\ dt^*\} \vdash \mathit{rectype} : \mathsf{ok}(x)}{C \vdash \mathsf{type}\ \mathit{rectype} : dt^*}$$

The type definition sequence type^* is valid with the defined type sequence $\mathit{deftype}^*$ if:

- Either:
  - The type definition sequence type^* is empty.
  - The defined type sequence $\mathit{deftype}^*$ is empty.
- Or:
  - The type definition sequence type^* is of the form $\mathit{type}_1\ \mathit{type}'^*$.
  - The defined type sequence $\mathit{deftype}^*$ is of the form $dt_1^*\ dt^*$.
  - The type definition type_1 is valid with the defined type sequence dt_1^*.
  - Let C' be the same context as C, but with the defined type sequence dt_1^* appended to the field types.
  - Under the context C', the type definition sequence type'^* is valid with the defined type sequence dt^*.

$$\frac{}{C \vdash \epsilon : \epsilon} \qquad \frac{C \vdash \mathit{type}_1 : dt_1^* \qquad C \oplus \{\mathsf{types}\ dt_1^*\} \vdash \mathit{type}^* : dt^*}{C \vdash \mathit{type}_1\ \mathit{type}^* : dt_1^*\ dt^*}$$

3.5.2 Tags

Tags tag are classified by their tag types, which are defined types expanding to function types.

The tag $(\mathsf{tag}\ \mathit{tagtype})$ is valid with the tag type $\mathit{tagtype}'$ if:

- The tag type $\mathit{tagtype}$ is valid.
- The tag type $\mathit{tagtype}'$ is $\mathsf{clos}_C(\mathit{tagtype})$.

$$\frac{C \vdash \mathit{tagtype} : \mathsf{ok}}{C \vdash \mathsf{tag}\ \mathit{tagtype} : \mathsf{clos}_C(\mathit{tagtype})}$$

3.5.3 Globals

Globals global are classified by global types.

The global $(\mathsf{global}\ \mathit{globaltype}\ \mathit{expr})$ is valid with the global type $\mathit{globaltype}$ if:

- The global type $\mathit{globaltype}$ is valid.
- The global type $\mathit{globaltype}$ is of the form $(\mathsf{mut}^?\ t)$.

- The expression $expr$ is valid with the value type t.
- $expr$ is constant.

$$\frac{C \vdash globaltype : \mathsf{ok} \qquad globaltype = \mathsf{mut}^?\ t \qquad C \vdash expr : t\ \mathsf{const}}{C \vdash \mathsf{global}\ globaltype\ expr : globaltype}$$

Sequences of globals are handled incrementally, such that each definition has access to previous definitions.

The global sequence $global^*$ is valid with the global type sequence $globaltype^*$ if:

- Either:
  - The global sequence $global^*$ is empty.
  - The global type sequence $globaltype^*$ is empty.
- Or:
  - The global sequence $global^*$ is of the form $global_1\ global'^*$.
  - The global type sequence $globaltype^*$ is of the form $gt_1\ gt^*$.
  - The global $global_1$ is valid with the global type gt_1.
  - Let C' be the same context as C, but with the global type sequence gt_1 appended to the field globals.
  - Under the context C', the global sequence $global'^*$ is valid with the global type sequence gt^*.

$$\frac{}{C \vdash \epsilon : \epsilon} \qquad \frac{C \vdash global_1 : gt_1 \qquad C \oplus \{\mathsf{globals}\ gt_1\} \vdash global^* : gt^*}{C \vdash global_1\ global^* : gt_1\ gt^*}$$

3.5.4 Memories

Memories mem are classified by memory types.

The memory $(\mathsf{memory}\ memtype)$ is valid with the memory type $memtype$ if:

- The memory type $memtype$ is valid.

$$\frac{C \vdash memtype : \mathsf{ok}}{C \vdash \mathsf{memory}\ memtype : memtype}$$

3.5.5 Tables

Tables $table$ are classified by table types.

The table $(\mathsf{table}\ tabletype\ expr)$ is valid with the table type $tabletype$ if:

- The table type $tabletype$ is valid.
- The table type $tabletype$ is of the form $(at\ lim\ rt)$.
- The expression $expr$ is valid with the value type rt.
- $expr$ is constant.

$$\frac{C \vdash tabletype : \mathsf{ok} \qquad tabletype = at\ lim\ rt \qquad C \vdash expr : rt\ \mathsf{const}}{C \vdash \mathsf{table}\ tabletype\ expr : tabletype}$$

3.5.6 Functions

Functions *func* are classified by defined types that expand to function types of the form $\mathsf{func}\ t_1^* \to t_2^*$.

The function $(\mathsf{func}\ x\ local^*\ expr)$ is valid with the type $C.\mathsf{types}[x]$ if:

- The type $C.\mathsf{types}[x]$ exists.
- The expansion of $C.\mathsf{types}[x]$ is $(\mathsf{func}\ t_1^*\ \to\ t_2^*)$.
- For all *local* in $local^*$:
  - The local *local* is valid with the local type lt.
- lt^* is the concatenation of all such lt.
- Under the context C with the field locals appended by $(\mathsf{set}\ t_1)^*\ lt^*$ and the field labels appended by t_2^* and the field return appended by t_2^*, the expression *expr* is valid with the result type t_2^*.

$$\frac{C.\mathsf{types}[x] \approx \mathsf{func}\ t_1^* \to t_2^* \qquad (C \vdash local : lt)^* \qquad C \oplus \{\mathsf{locals}\ (\mathsf{set}\ t_1)^*\ lt^*,\ \mathsf{labels}\ (t_2^*),\ \mathsf{return}\ (t_2^*)\} \vdash expr : t_2^*}{C \vdash \mathsf{func}\ x\ local^*\ expr : C.\mathsf{types}[x]}$$

3.5.7 Locals

Locals *local* are classified with local types.

The local $(\mathsf{local}\ t)$ is valid with the local type $(init\ t)$ if:

- The value type t is valid.
- Either:
  - The initialization status *init* is of the form set.
  - A default value for t is defined.
- Or:
  - The initialization status *init* is of the form unset.
  - A default value for t is not defined.

$$\frac{C \vdash t : \mathsf{ok} \qquad \mathrm{default}_t \neq \epsilon}{C \vdash \mathsf{local}\ t : \mathsf{set}\ t} \qquad \frac{C \vdash t : \mathsf{ok} \qquad \mathrm{default}_t = \epsilon}{C \vdash \mathsf{local}\ t : \mathsf{unset}\ t}$$

Note

For cases where both rules are applicable, the former yields the more permissable type.

3.5.8 Data Segments

Data segments *data* are classified by the singleton data type, which merely expresses well-formedness.

The memory segment $(\mathsf{data}\ b^*\ datamode)$ is valid if:

- The data mode *datamode* is valid.

$$\frac{C \vdash datamode : \mathsf{ok}}{C \vdash \mathsf{data}\ b^*\ datamode : \mathsf{ok}}$$

The data mode *datamode* is valid if:

- Either:
  - The data mode *datamode* is of the form passive.
- Or:
  - The data mode *datamode* is of the form $(\mathsf{active}\ x\ expr)$.

- The memory $C.\mathsf{mems}[x]$ exists.
- The memory $C.\mathsf{mems}[x]$ is of the form $(at\ lim\ \mathsf{page})$.
- The expression $expr$ is valid with the value type at.
- $expr$ is constant.

$$\frac{}{C \vdash \mathsf{passive} : \mathsf{ok}} \qquad \frac{C.\mathsf{mems}[x] = at\ lim\ \mathsf{page} \qquad C \vdash expr : at\ \mathsf{const}}{C \vdash \mathsf{active}\ x\ expr : \mathsf{ok}}$$

3.5.9 Element Segments

Element segments $elem$ are classified by their element type.

The table segment $(\mathsf{elem}\ elemtype\ expr^*\ elemmode)$ is valid with the element type $elemtype$ if:

- The reference type $elemtype$ is valid.
- For all $expr$ in $expr^*$:
  - The expression $expr$ is valid with the value type $elemtype$.
  - $expr$ is constant.
- The element mode $elemmode$ is valid with the element type $elemtype$.

$$\frac{C \vdash elemtype : \mathsf{ok} \qquad (C \vdash expr : elemtype\ \mathsf{const})^* \qquad C \vdash elemmode : elemtype}{C \vdash \mathsf{elem}\ elemtype\ expr^*\ elemmode : elemtype}$$

The element mode $elemmode$ is valid with the element type rt if:

- Either:
  - The element mode $elemmode$ is of the form passive.
- Or:
  - The element mode $elemmode$ is of the form declare.
- Or:
  - The element mode $elemmode$ is of the form $(\mathsf{active}\ x\ expr)$.
  - The table $C.\mathsf{tables}[x]$ exists.
  - The table $C.\mathsf{tables}[x]$ is of the form $(at\ lim\ rt')$.
  - The reference type rt matches the reference type rt'.
  - The expression $expr$ is valid with the value type at.
  - $expr$ is constant.

$$\frac{}{C \vdash \mathsf{passive} : rt} \qquad \frac{}{C \vdash \mathsf{declare} : rt}$$

$$\frac{C.\mathsf{tables}[x] = at\ lim\ rt' \qquad C \vdash rt \leq rt' \qquad C \vdash expr : at\ \mathsf{const}}{C \vdash \mathsf{active}\ x\ expr : rt}$$

3.5.10 Start Function

The start function $(\mathsf{start}\ x)$ is valid if:

- The function $C.\mathsf{funcs}[x]$ exists.
- The expansion of $C.\mathsf{funcs}[x]$ is $(\mathsf{func}\ \to)$.

$$\frac{C.\mathsf{funcs}[x] \approx \mathsf{func}\ \epsilon \to \epsilon}{C \vdash \mathsf{start}\ x : \mathsf{ok}}$$

3.5.11 Imports

Imports $import$ are classified by external types.

The import $(\mathsf{import}\ name_1\ name_2\ xt)$ is valid with the external type $externtype$ if:

- The external type xt is valid.
- The external type $externtype$ is $\mathsf{clos}_C(xt)$.

$$\frac{C \vdash xt : \mathsf{ok}}{C \vdash \mathsf{import}\ name_1\ name_2\ xt : \mathsf{clos}_C(xt)}$$

3.5.12 Exports

Exports $export$ are classified by their external type.

The export $(\mathsf{export}\ name\ externidx)$ is valid with the name $name$ and the external type xt if:

- The external index $externidx$ is valid with the external type xt.

$$\frac{C \vdash externidx : xt}{C \vdash \mathsf{export}\ name\ externidx : name\ xt}$$

$\mathsf{tag}\ x$

The external index $(\mathsf{tag}\ x)$ is valid with the external type $(\mathsf{tag}\ jt)$ if:

- The tag $C.\mathsf{tags}[x]$ exists.
- The tag $C.\mathsf{tags}[x]$ is of the form jt.

$$\frac{C.\mathsf{tags}[x] = jt}{C \vdash \mathsf{tag}\ x : \mathsf{tag}\ jt}$$

$\mathsf{global}\ x$

The external index $(\mathsf{global}\ x)$ is valid with the external type $(\mathsf{global}\ gt)$ if:

- The global $C.\mathsf{globals}[x]$ exists.
- The global $C.\mathsf{globals}[x]$ is of the form gt.

$$\frac{C.\mathsf{globals}[x] = gt}{C \vdash \mathsf{global}\ x : \mathsf{global}\ gt}$$

$\mathsf{memory}\ x$

The external index $(\mathsf{memory}\ x)$ is valid with the external type $(\mathsf{mem}\ mt)$ if:

- The memory $C.\mathsf{mems}[x]$ exists.
- The memory $C.\mathsf{mems}[x]$ is of the form mt.

$$\frac{C.\mathsf{mems}[x] = mt}{C \vdash \mathsf{memory}\ x : \mathsf{mem}\ mt}$$

$\mathsf{table}\ x$

The external index $(\mathsf{table}\ x)$ is valid with the external type $(\mathsf{table}\ tt)$ if:

- The table $C.\mathsf{tables}[x]$ exists.
- The table $C.\mathsf{tables}[x]$ is of the form tt.

$$\frac{C.\mathsf{tables}[x] = tt}{C \vdash \mathsf{table}\ x : \mathsf{table}\ tt}$$

$\mathsf{func}\ x$

The external index $(\mathsf{func}\ x)$ is valid with the external type $(\mathsf{func}\ dt)$ if:

- The function $C.\mathsf{funcs}[x]$ exists.
- The function $C.\mathsf{funcs}[x]$ is of the form dt.

$$\frac{C.\mathsf{funcs}[x] = dt}{C \vdash \mathsf{func}\ x : \mathsf{func}\ dt}$$

3.5.13 Modules

Modules are classified by their mapping from the external types of their imports to those of their exports.

A module is entirely *closed*, that is, its components can only refer to definitions that appear in the module itself. Consequently, no initial context is required. Instead, the context C for validation of the module's content is constructed from the definitions in the module.

The module $(\mathsf{module}\ type^*\ import^*\ tag^*\ global^*\ mem^*\ table^*\ func^*\ data^*\ elem^*\ start^?\ export^*)$ is valid with the module type $moduletype$ if:

- Under the context $\{\mathsf{return}\ \epsilon\}$, the type definition sequence $type^*$ is valid with the defined type sequence dt'^*.
- For all $import$ in $import^*$:
  - Under the context $\{\mathsf{types}\ dt'^*,\ \mathsf{return}\ \epsilon\}$, the import $import$ is valid with the external type xt_i.
- xt_i^* is the concatenation of all such xt_i.
- For all tag in tag^*:
  - Under the context C', the tag tag is valid with the tag type jt.
- jt^* is the concatenation of all such jt.
- Under the context C', the global sequence $global^*$ is valid with the global type sequence gt^*.
- For all mem in mem^*:
  - Under the context C', the memory mem is valid with the memory type mt.
- mt^* is the concatenation of all such mt.
- For all $table$ in $table^*$:
  - Under the context C', the table $table$ is valid with the table type tt.
- tt^* is the concatenation of all such tt.
- For all $func$ in $func^*$:
  - The function $func$ is valid with the defined type dt.
- dt^* is the concatenation of all such dt.
- For all $data$ in $data^*$:
  - The memory segment $data$ is valid.
- ok^* is the concatenation of all such ok.
- For all $elem$ in $elem^*$:
  - The table segment $elem$ is valid with the element type rt.
- rt^* is the concatenation of all such rt.
- If $start$ is defined, then:
  - The start function $start$ is valid.
- For all $export$ in $export^*$:

  - The export $export$ is valid with the name nm and the external type xt_{e}.

- nm^* is the concatenation of all such nm.
- xt_{e}^* is the concatenation of all such xt_{e}.
- nm^* disjoint is true.
- The context C is of the form C' with the field tags appended by $jt_{\mathsf{i}}^*\ jt^*$ and the field globals appended by gt^* and the field mems appended by $mt_{\mathsf{i}}^*\ mt^*$ and the field tables appended by $tt_{\mathsf{i}}^*\ tt^*$ and the field datas appended by ok^* and the field elems appended by rt^*.
- The context C' is of the form $\{\mathsf{types}\ dt'^*,\ \mathsf{globals}\ gt_{\mathsf{i}}^*,\ \mathsf{funcs}\ dt_{\mathsf{i}}^*\ dt^*,\ \mathsf{return}\ \epsilon,\ \mathsf{refs}\ x^*\}$.
- The function index sequence x^* is of the form $\mathrm{funcidx}(global^*\ mem^*\ table^*\ elem^*\ export^*)$.
- The tag type sequence jt_{i}^* is of the form $\mathrm{tags}(xt_{\mathsf{i}}^*)$.
- The global type sequence gt_{i}^* is of the form $\mathrm{globals}(xt_{\mathsf{i}}^*)$.
- The memory type sequence mt_{i}^* is of the form $\mathrm{mems}(xt_{\mathsf{i}}^*)$.
- The table type sequence tt_{i}^* is of the form $\mathrm{tables}(xt_{\mathsf{i}}^*)$.
- The defined type sequence dt_{i}^* is of the form $\mathrm{funcs}(xt_{\mathsf{i}}^*)$.
- The module type $moduletype$ is $\mathsf{clos}_C(xt_{\mathsf{i}}^* \to xt_{\mathsf{e}}^*)$.

$$\frac{\begin{array}{c}\{\} \vdash type^* : dt'^* \qquad (\{\mathsf{types}\ dt'^*\} \vdash import : xt_{\mathsf{i}})^* \\ (C' \vdash tag : jt)^* \qquad C' \vdash global^* : gt^* \qquad (C' \vdash mem : mt)^* \qquad (C' \vdash table : tt)^* \qquad (C \vdash func : dt)^* \\ (C \vdash data : ok)^* \qquad (C \vdash elem : rt)^* \qquad (C \vdash start : \mathsf{ok})^? \qquad (C \vdash export : nm\ xt_{\mathsf{e}})^* \qquad nm^*\ \text{disjoint} \\ C = C' \oplus \{\mathsf{tags}\ jt_{\mathsf{i}}^*\ jt^*,\ \mathsf{globals}\ gt^*,\ \mathsf{mems}\ mt_{\mathsf{i}}^*\ mt^*,\ \mathsf{tables}\ tt_{\mathsf{i}}^*\ tt^*,\ \mathsf{datas}\ ok^*,\ \mathsf{elems}\ rt^*\} \\ C' = \{\mathsf{types}\ dt'^*,\ \mathsf{globals}\ gt_{\mathsf{i}}^*,\ \mathsf{funcs}\ dt_{\mathsf{i}}^*\ dt^*,\ \mathsf{refs}\ x^*\} \qquad x^* = \mathrm{funcidx}(global^*\ mem^*\ table^*\ elem^*\ export^*) \\ jt_{\mathsf{i}}^* = \mathrm{tags}(xt_{\mathsf{i}}^*) \qquad gt_{\mathsf{i}}^* = \mathrm{globals}(xt_{\mathsf{i}}^*) \qquad mt_{\mathsf{i}}^* = \mathrm{mems}(xt_{\mathsf{i}}^*) \qquad tt_{\mathsf{i}}^* = \mathrm{tables}(xt_{\mathsf{i}}^*) \qquad dt_{\mathsf{i}}^* = \mathrm{funcs}(xt_{\mathsf{i}}^*)\end{array}}{\vdash \mathsf{module}\ type^*\ import^*\ tag^*\ global^*\ mem^*\ table^*\ func^*\ data^*\ elem^*\ start^?\ export^* : \mathsf{clos}_C(xt_{\mathsf{i}}^* \to xt_{\mathsf{e}}^*)}$$

Note

All functions in a module are mutually recursive. Consequently, the definition of the context C in this rule is recursive: it depends on the outcome of validation of the function, table, memory, and global definitions contained in the module, which itself depends on C. However, this recursion is just a specification device. All types needed to construct C can easily be determined from a simple pre-pass over the module that does not perform any actual validation.

Globals, however, are not recursive but evaluated sequentially, such that each constant expressions only has access to imported or previously defined globals.

CHAPTER 4

Execution

4.1 Conventions

WebAssembly code is *executed* when instantiating a module or invoking an exported function on the resulting module instance.

Execution behavior is defined in terms of an *abstract machine* that models the *program state*. It includes a *stack*, which records operand values and control constructs, and an abstract *store* containing global state.

For each instruction, there is a rule that specifies the effect of its execution on the program state. Furthermore, there are rules describing the instantiation of a module. As with validation, all rules are given in two *equivalent* forms:

1. In *prose*, describing the execution in intuitive form.
2. In *formal notation*, describing the rule in mathematical form.[18]

Note

As with validation, the prose and formal rules are equivalent, so that understanding of the formal notation is *not* required to read this specification. The formalism offers a more concise description in notation that is used widely in programming languages semantics and is readily amenable to mathematical proof.

4.1.1 Prose Notation

Execution is specified by stylised, step-wise rules for each instruction of the abstract syntax. The following conventions are adopted in stating these rules.

- The execution rules implicitly assume a given store s.
- The execution rules also assume the presence of an implicit stack that is modified by *pushing* or *popping* values, labels, and frames.
- Certain rules require the stack to contain at least one frame. The most recent frame is referred to as the *current* frame.

[18] The semantics is derived from the following article: Andreas Haas, Andreas Rossberg, Derek Schuff, Ben Titzer, Dan Gohman, Luke Wagner, Alon Zakai, JF Bastien, Michael Holman. Bringing the Web up to Speed with WebAssembly[19]. Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017). ACM 2017.

[19] https://dl.acm.org/citation.cfm?doid=3062341.3062363

- Both the store and the current frame are mutated by *replacing* some of their components. Such replacement is assumed to apply globally.
- The execution of an instruction may *trap*, in which case the entire computation is aborted and no further modifications to the store are performed by it. (Other computations can still be initiated afterwards.)
- The execution of an instruction may also end in a *jump* to a designated target, which defines the next instruction to execute.
- Execution can *enter* and *exit* instruction sequences that form blocks.
- Instruction sequences are implicitly executed in order, unless a trap, jump, or exception occurs.
- In various places the rules contain *assertions* expressing crucial invariants about the program state.

4.1.2 Formal Notation

Note

This section gives a brief explanation of the notation for specifying execution formally. For the interested reader, a more thorough introduction can be found in respective text books.[20]

The formal execution rules use a standard approach for specifying operational semantics, rendering them into *reduction rules*. Every rule has the following general form:

$$configuration \hookrightarrow configuration$$

A *configuration* is a syntactic description of a program state. Each rule specifies one *step* of execution. As long as there is at most one reduction rule applicable to a given configuration, reduction – and thereby execution – is *deterministic*. WebAssembly has only very few exceptions to this, which are noted explicitly in this specification.

For WebAssembly, a configuration typically is a tuple $(s; f; instr^*)$ consisting of the current store s, the call frame f of the current function, and the sequence of instructions that is to be executed. (A more precise definition is given later.)

To avoid unnecessary clutter, the store s and the frame f are often combined into a *state* z, which is a pair $(s; f)$. Moreover, z is omitted from reduction rules that do not touch them.

There is no separate representation of the stack. Instead, it is conveniently represented as part of the configuration's instruction sequence. In particular, values are defined to coincide with const and ref instructions, and a sequence of such instructions can be interpreted as an operand "stack" that grows to the right.

Note

For example, the reduction rule for the i32.add instruction can be given as follows:

$$(\mathsf{i32.const}\ n_1)\ (\mathsf{i32.const}\ n_2)\ (\mathsf{i32.add}) \hookrightarrow (\mathsf{i32.const}\ (n_1 + n_2) \bmod 2^{32})$$

Per this rule, two const instructions and the add instruction itself are removed from the instruction stream and replaced with one new const instruction. This can be interpreted as popping two values off the stack and pushing the result.

When no result is produced, an instruction reduces to the empty sequence:

$$\mathsf{nop} \hookrightarrow \epsilon$$

Labels and frames are similarly defined to be part of an instruction sequence.

[20] For example: Benjamin Pierce. Types and Programming Languages[21]. The MIT Press 2002
[21] https://www.cis.upenn.edu/~bcpierce/tapl/

The order of reduction is determined by the details of the reduction rules. Usually, the left-most instruction that is not a constant will be the subject of the next reduction *step*.

Reduction *terminates* when no more reduction rules are applicable. Soundness of the WebAssembly type system guarantees that this is only the case when the original instruction sequence has either been reduced to a sequence of value instructions, which can be interpreted as the values of the resulting operand stack, or if an exception or trap occurred.

Note

For example, the following instruction sequence,

$$(\mathsf{f64.const}\ q_1)\ (\mathsf{f64.const}\ q_2)\ (\mathsf{f64.neg})\ (\mathsf{f64.const}\ q_3)\ (\mathsf{f64.add})\ (\mathsf{f64.mul})$$

terminates after three steps:

$$\begin{array}{ll} \hookrightarrow & (\mathsf{f64.const}\ q_1)\ (\mathsf{f64.const}\ q_4)\ (\mathsf{f64.const}\ q_3)\ (\mathsf{f64.add})\ (\mathsf{f64.mul}) \\ \hookrightarrow & (\mathsf{f64.const}\ q_1)\ (\mathsf{f64.const}\ q_5)\ (\mathsf{f64.mul}) \\ \hookrightarrow & (\mathsf{f64.const}\ q_6) \end{array}$$

where $q_4 = -q_2$ and $q_5 = -q_2 + q_3$ and $q_6 = q_1 \cdot (-q_2 + q_3)$.

4.2 Runtime Structure

Store, stack, and other *runtime structure* forming the WebAssembly abstract machine, such as values or module instances, are made precise in terms of additional auxiliary syntax.

4.2.1 Values

WebAssembly computations manipulate *values* of either the four basic number types, i.e., integers and floating-point data of 32 or 64 bit width each, or vectors of 128 bit width, or of reference type.

In most places of the semantics, values of different types can occur. In order to avoid ambiguities, values are therefore represented with an abstract syntax that makes their type explicit. It is convenient to reuse the same notation as for the const instructions and ref.null producing them.

References other than null are represented with additional administrative instructions. They either are *scalar references*, containing a 31-bit integer, *null references*, *structure references*, pointing to a specific structure address, *array references*, pointing to a specific array address, *function references*, pointing to a specific function address, *exception references*, pointing to a specific exception address, or *host references* pointing to an uninterpreted form of host address defined by the embedder. Any of the aformentioned references can furthermore be wrapped up as an *external reference*.

$$\begin{array}{rcl} \mathit{val} & ::= & \mathit{num} \mid \mathit{vec} \mid \mathit{ref} \\ \mathit{num} & ::= & \mathit{numtype}.\mathsf{const}\ \mathit{num}_{\mathit{numtype}} \\ \mathit{vec} & ::= & \mathit{vectype}.\mathsf{const}\ \mathit{vec}_{\mathit{vectype}} \\ \mathit{ref} & ::= & \mathsf{ref.i31}\ \mathit{u31} \\ & \mid & \mathsf{ref.null} \\ & \mid & \mathsf{ref.struct}\ \mathit{structaddr} \\ & \mid & \mathsf{ref.array}\ \mathit{arrayaddr} \\ & \mid & \mathsf{ref.func}\ \mathit{funcaddr} \\ & \mid & \mathsf{ref.exn}\ \mathit{exnaddr} \\ & \mid & \mathsf{ref.host}\ \mathit{hostaddr} \\ & \mid & \mathsf{ref.extern}\ \mathit{ref} \end{array}$$

Note

Future versions of WebAssembly may add additional forms of values.

Value types can have an associated *default value*; it is the respective value 0 for number types, 0 for vector types, and null for nullable reference types. For other references, no default value is defined, $\mathrm{default}_t$ hence is an optional value $val^?$.

$$\begin{array}{lcl} \mathrm{default}_{\mathsf{i}N} & = & (\mathsf{i}N.\mathsf{const}\ 0) \\ \mathrm{default}_{\mathsf{f}N} & = & (\mathsf{f}N.\mathsf{const}\ +0) \\ \mathrm{default}_{\mathsf{v}N} & = & (\mathsf{v}N.\mathsf{const}\ 0) \\ \mathrm{default}_{\mathsf{ref\ null}\ ht} & = & \mathsf{ref.null} \\ \mathrm{default}_{\mathsf{ref}\ ht} & = & \epsilon \end{array}$$

Convention

- The meta variable r ranges over reference values where clear from context.

4.2.2 Results

A *result* is the outcome of a computation. It is either a sequence of values, a thrown exception, or a trap.

$$result \quad ::= \quad val^* \ \mid\ (\mathsf{ref.exn}\ exnaddr)\ \mathsf{throw_ref} \ \mid\ \mathsf{trap}$$

4.2.3 Store

The *store* represents all global state that can be manipulated by WebAssembly programs. It consists of the runtime representation of all *instances* of functions, tables, memories, globals, tags, element segments, data segments, and structures, arrays or exceptions that have been allocated during the life time of the abstract machine.

It is an invariant of the semantics that no element or data instance is addressed from anywhere else but the owning module instances.

Syntactically, the store is defined as a record listing the existing instances of each category:

$$\begin{array}{lcl} store & ::= & \{\mathsf{tags}\ taginst^* \\ & & \ \mathsf{globals}\ globalinst^* \\ & & \ \mathsf{mems}\ meminst^* \\ & & \ \mathsf{tables}\ tableinst^* \\ & & \ \mathsf{funcs}\ funcinst^* \\ & & \ \mathsf{datas}\ datainst^* \\ & & \ \mathsf{elems}\ eleminst^* \\ & & \ \mathsf{structs}\ structinst^* \\ & & \ \mathsf{arrays}\ arrayinst^* \\ & & \ \mathsf{exns}\ exninst^*\} \end{array}$$

Note

In practice, implementations may apply techniques like garbage collection or reference counting to remove objects from the store that are no longer referenced. However, such techniques are not semantically observable, and hence outside the scope of this specification.

Convention

- The meta variable s ranges over stores where clear from context.

4.2.4 Addresses

Function instances, table instances, memory instances, global instances, tag instances, element instances, data instances and structure, array or exception instances in the store are referenced with abstract *addresses*. These are simply indices into the respective store component. In addition, an embedder may supply an uninterpreted set of *host addresses*.

$$
\begin{array}{rcl}
addr & ::= & 0 \mid 1 \mid 2 \mid \dots \\
funcaddr & ::= & addr \\
tableaddr & ::= & addr \\
memaddr & ::= & addr \\
globaladdr & ::= & addr \\
tagaddr & ::= & addr \\
elemaddr & ::= & addr \\
dataaddr & ::= & addr \\
structaddr & ::= & addr \\
arrayaddr & ::= & addr \\
exnaddr & ::= & addr \\
hostaddr & ::= & addr
\end{array}
$$

An embedder may assign identity to exported store objects corresponding to their addresses, even where this identity is not observable from within WebAssembly code itself (such as for function instances or immutable globals).

Note

Addresses are *dynamic*, globally unique references to runtime objects, in contrast to indices, which are *static*, module-local references to their original definitions. A *memory address* $memaddr$ denotes the abstract address *of* a memory *instance* in the store, not an offset *inside* a memory instance.

There is no specific limit on the number of allocations of store objects, hence logical addresses can be arbitrarily large natural numbers.

Conventions

- The notation $\mathrm{addr}(A)$ denotes the set of addresses from address space $addr$ occurring free in A. We sometimes reinterpret this set as the list of its elements, without assuming any particular order.

4.2.5 External Addresses

An *external address* is the runtime address of an entity that can be imported or exported. It is an address denoting either a function instance, global instance, table instance, memory instance, or tag instance in the shared store.

$$
externaddr ::= \mathsf{tag}\ tagaddr \mid \mathsf{global}\ globaladdr \mid \mathsf{mem}\ memaddr \mid \mathsf{table}\ tableaddr \mid \mathsf{func}\ funcaddr
$$

4.2.6 Module Instances

A *module instance* is the runtime representation of a module. It is created by instantiating a module, and collects runtime representations of all entities that are imported, defined, or exported by the module.

$$
\begin{array}{rcl}
moduleinst & ::= & \{\mathsf{types}\ deftype^* \\
 & & \mathsf{tags}\ tagaddr^* \\
 & & \mathsf{globals}\ globaladdr^* \\
 & & \mathsf{mems}\ memaddr^* \\
 & & \mathsf{tables}\ tableaddr^* \\
 & & \mathsf{funcs}\ funcaddr^* \\
 & & \mathsf{datas}\ dataaddr^* \\
 & & \mathsf{elems}\ elemaddr^* \\
 & & \mathsf{exports}\ exportinst^*\}
\end{array}
$$

Each component references runtime instances corresponding to respective declarations from the original module – whether imported or defined – in the order of their static indices. Function instances, table instances, memory instances, global instances, and tag instances are denoted by their respective addresses in the store.

It is an invariant of the semantics that all export instances in a given module instance have different names.

Note

All record fields except exports are to be considered *private* components of a module instance. They are not accessible to other modules, only to function instances originating from the same module.

4.2.7 Function Instances

A *function instance* is the runtime representation of a function. It effectively is a *closure* of the original function over the runtime module instance of its originating module. The module instance is used to resolve references to other definitions during execution of the function.

$$\begin{array}{rcl} funcinst & ::= & \{\mathsf{type}\ deftype, \mathsf{module}\ moduleinst, \mathsf{code}\ code\} \\ code & ::= & func \mid hostfunc \end{array}$$

A *host function* is a function expressed outside WebAssembly but passed to a module as an import. The definition and behavior of host functions are outside the scope of this specification. For the purpose of this specification, it is assumed that when invoked, a host function behaves non-deterministically, but within certain constraints that ensure the integrity of the runtime.

Note

Function instances are immutable, and their identity is not observable by WebAssembly code. However, an embedder might provide implicit or explicit means for distinguishing their addresses.

4.2.8 Table Instances

A *table instance* is the runtime representation of a table. It records its type and holds a sequence of reference values.

$$tableinst \quad ::= \quad \{\mathsf{type}\ tabletype, \mathsf{refs}\ ref^*\}$$

Table elements can be mutated through table instructions, the execution of an active element segment, or by external means provided by the embedder.

It is an invariant of the semantics that all table elements have a type matching the element type of $tabletype$. It also is an invariant that the length of the element sequence never exceeds the maximum size of $tabletype$.

4.2.9 Memory Instances

A *memory instance* is the runtime representation of a linear memory. It records its type and holds a sequence of bytes.

$$meminst \quad ::= \quad \{\mathsf{type}\ memtype, \mathsf{bytes}\ byte^*\}$$

The length of the sequence always is a multiple of the WebAssembly *page size*, which is defined to be the constant 65536 – abbreviated $64\,\mathrm{Ki}$.

A memory's bytes can be mutated through memory instructions, the execution of an active data segment, or by external means provided by the embedder.

It is an invariant of the semantics that the length of the byte sequence, divided by page size, never exceeds the maximum size of $memtype$.

4.2.10 Global Instances

A *global instance* is the runtime representation of a global variable. It records its type and holds an individual value.

$$globalinst \quad ::= \quad \{\mathsf{type}\ globaltype, \mathsf{value}\ val\}$$

The value of mutable globals can be mutated through variable instructions or by external means provided by the embedder.

It is an invariant of the semantics that the value has a type matching the value type of $globaltype$.

4.2.11 Tag Instances

A *tag instance* is the runtime representation of a tag definition. It records the defined type of the tag.

$$taginst \quad ::= \quad \{\mathsf{type}\ tagtype\}$$

4.2.12 Element Instances

An *element instance* is the runtime representation of an element segment. It holds a list of references and its type.

$$eleminst \quad ::= \quad \{\mathsf{type}\ elemtype, \mathsf{refs}\ ref^*\}$$

It is an invariant of the semantics that all elements of a segment have a type matching $elemtype$.

4.2.13 Data Instances

A *data instance* is the runtime representation of a data segment. It holds a list of bytes.

$$datainst \quad ::= \quad \{\mathsf{bytes}\ byte^*\}$$

4.2.14 Export Instances

An *export instance* is the runtime representation of an export. It defines the export's name and the associated external address.

$$exportinst \quad ::= \quad \{\mathsf{name}\ name, \mathsf{addr}\ externaddr\}$$

Conventions

The following auxiliary functions are assumed on sequences of external addresses. They extract addresses of a specific kind in an order-preserving fashion:

- $\mathrm{funcs}(xa^*)$ extracts all function addresses from xa^*,
- $\mathrm{tables}(xa^*)$ extracts all table addresses from xa^*,
- $\mathrm{mems}(xa^*)$ extracts all memory addresses from xa^*,
- $\mathrm{globals}(xa^*)$ extracts all global addresses from xa^*,
- $\mathrm{tags}(xa^*)$ extracts all tag addresses from xa^*.

4.2.15 Aggregate Instances

A *structure instance* is the runtime representation of a heap object allocated from a structure type. Likewise, an *array instance* is the runtime representation of a heap object allocated from an array type. Both record their respective defined type and hold a list of the values of their *fields*.

$$\begin{array}{rcl} structinst & ::= & \{\mathsf{type}\ deftype, \mathsf{fields}\ fieldval^*\} \\ arrayinst & ::= & \{\mathsf{type}\ deftype, \mathsf{fields}\ fieldval^*\} \\ fieldval & ::= & val \mid packval \\ packval & ::= & packtype.\mathsf{pack}\ iN \end{array}$$

Conventions

- Conversion of a regular value to a field value is defined as follows:

$$\begin{array}{lcl} \mathrm{pack}_{valtype}(val) & = & val \\ \mathrm{pack}_{packtype}(\mathsf{i32.const}\ i) & = & packtype.\mathsf{pack}\ \mathrm{wrap}_{32,|packtype|}(i) \end{array}$$

- The inverse conversion of a field value to a regular value is defined as follows:

$$\begin{array}{lcl} \mathrm{unpack}^{\epsilon}_{valtype}(val) & = & val \\ \mathrm{unpack}^{sx}_{packtype}(packtype.\mathsf{pack}\ i) & = & \mathsf{i32.const}\ \mathrm{extend}^{sx}_{|packtype|,32}(i) \end{array}$$

4.2.16 Exception Instances

An *exception instance* is the runtime representation of an exception produced by a throw instruction. It holds the address of the respective tag and the argument values.

$$exninst \quad ::= \quad \{\mathsf{tag}\ tagaddr, \mathsf{fields}\ val^*\}$$

4.2.17 Stack

Besides the store, most instructions interact with an implicit *stack*. The stack contains the two kinds of entries:

- *Values*: the *operands* of instructions.
- *Control Frames*: currently active control flow structures.

The latter can in turn be one of the following:

- *Labels*: active structured control instructions that can be targeted by branches.
- *(Call) Frames*: the *activation records* of active function calls.
- *Handlers*: active exception handlers.

Note

Where clear from context, *call frame* is abbreviated to just *frame*.

All these entries can occur on the stack in any order during the execution of a program. Stack entries are described by abstract syntax as follows.

Note

It is possible to model the WebAssembly semantics using separate stacks for operands, control constructs, and calls. However, because the stacks are interdependent, additional book keeping about associated stack heights would be required. For the purpose of this specification, an interleaved representation is simpler.

Values

Values are represented by themselves.

Labels

Labels carry an argument arity n and their associated branch *target*, which is expressed syntactically as an instruction sequence:

$$label \quad ::= \quad \mathsf{label}_n\{instr^*\}$$

Intuitively, $instr^*$ is the *continuation* to execute when the branch is taken, in place of the original control construct.

Note

For example, a loop label has the form

$$\mathsf{label}_n\{(\mathsf{loop}\ bt\ \ldots)\}$$

When performing a branch to this label, this executes the loop, effectively restarting it from the beginning. Conversely, a simple block label has the form

$$\mathsf{label}_n\{\epsilon\}$$

When branching, the empty continuation ends the targeted block, such that execution can proceed with consecutive instructions.

Call Frames

Call frames carry the return arity n of the respective function, hold the values of its locals (including arguments) in the order corresponding to their static local indices, and a reference to the function's own module instance:

$$\begin{array}{rcl} callframe & ::= & \mathsf{frame}_n\{frame\} \\ frame & ::= & \{\mathsf{locals}\ (val^?)^*, \mathsf{module}\ moduleinst\} \end{array}$$

Locals may be uninitialized, in which case they are empty. Locals are mutated by respective variable instructions.

Exception Handlers

Exception handlers are installed by try_table instructions and record the corresponding list of catch clauses:

$$handler \quad ::= \quad \mathsf{handler}_n\{catch^*\}$$

The handlers on the stack are searched when an exception is thrown.

Conventions

- The meta variable L ranges over labels where clear from context.
- The meta variable f ranges over frame states where clear from context.
- The meta variable H ranges over exception handlers where clear from context.
- The following auxiliary definition takes a block type and looks up the instruction type that it denotes in the current frame:

$$\begin{array}{lcll} \mathrm{instrtype}_z(x) & = & t_1^* \to t_2^* & \text{if } z.\mathsf{types}[x] \approx \mathsf{func}\ t_1^* \to t_2^* \\ \mathrm{instrtype}_z(t^?) & = & \epsilon \to t^? & \end{array}$$

4.2.18 Administrative Instructions

Note

This section is only relevant for the formal notation.

In order to express the reduction of traps, calls, exception handling, and control instructions, the syntax of instructions is extended to include the following *administrative instructions*:

$$\begin{array}{rcl} \mathit{instr} & ::= & \dots \\ & | & \mathit{ref} \\ & | & \mathsf{label}_n\{\mathit{instr}^*\}\ \mathit{instr}^* \\ & | & \mathsf{frame}_n\{\mathit{frame}\}\ \mathit{instr}^* \\ & | & \mathsf{handler}_n\{\mathit{catch}^*\}\ \mathit{instr}^* \\ & | & \mathsf{trap} \end{array}$$

A reference represents a reference value of respective form "on the stack".

The label, frame, and handler instructions model labels, frames, and active exception handlers, respectively, "on the stack". Moreover, the administrative syntax maintains the nesting structure of the original structured control instruction or function body and their instruction sequences.

The trap instruction represents the occurrence of a trap. Traps are bubbled up through nested instruction sequences, ultimately reducing the entire program to a single trap instruction, signalling abrupt termination.

Note

For example, the reduction rule for block is:

$$(\mathsf{block}\ \mathit{bt}\ \mathit{instr}^*) \hookrightarrow (\mathsf{label}_n\{\epsilon\}\ \mathit{instr}^*)$$

if the block type bt denotes a function type $\mathsf{func}\ t_1^m \to t_2^n$, such that n is the block's result arity. This rule replaces the block with a label instruction, which can be interpreted as "pushing" the label on the stack. When its end is reached, i.e., the inner instruction sequence has been reduced to the empty sequence – or rather, a sequence of n values representing the results – then the label instruction is eliminated courtesy of its own reduction rule:

$$(\mathsf{label}_n\{\mathit{instr}^*\}\ \mathit{val}^*) \hookrightarrow \mathit{val}^*$$

This can be interpreted as removing the label from the stack and only leaving the locally accumulated operand values. Validation guarantees that n matches the number $|\mathit{val}^*|$ of resulting values at this point.

4.2.19 Configurations

A *configuration* describes the current computation. It consists of the computations's *state* and the sequence of instructions left to execute. The state in turn consists of a global store and a current frame referring to the module instance in which the computation runs, i.e., where the current function originates from.

$$\begin{array}{rcl} \mathit{config} & ::= & \mathit{state}; \mathit{instr}^* \\ \mathit{state} & ::= & \mathit{store}; \mathit{frame} \end{array}$$

Note

The current version of WebAssembly is single-threaded, but configurations with multiple threads may be supported in the future.

Conventions

- The meta variable z ranges over frame states where clear from context.
- The following shorthands are defined for accessing a state $z = (s; f)$:
  - $z.\mathsf{module} = f.\mathsf{module}$
  - $z.\mathsf{funcs}[a] = s.\mathsf{funcs}[a]$

  - $z.\mathsf{structs}[a] = s.\mathsf{structs}[a]$
  - $z.\mathsf{arrays}[a] = s.\mathsf{arrays}[a]$
  - $z.\mathsf{exns}[a] = s.\mathsf{exns}[a]$
  - $z.\mathsf{types}[x] = f.\mathsf{module}.\mathsf{types}[x]$
  - $z.\mathsf{tags}[x] = s.\mathsf{tags}[f.\mathsf{module}.\mathsf{tags}[x]]$
  - $z.\mathsf{globals}[x] = s.\mathsf{globals}[f.\mathsf{module}.\mathsf{globals}[x]]$
  - $z.\mathsf{mems}[x] = s.\mathsf{mems}[f.\mathsf{module}.\mathsf{mems}[x]]$
  - $z.\mathsf{tables}[x] = s.\mathsf{tables}[f.\mathsf{module}.\mathsf{tables}[x]]$
  - $z.\mathsf{funcs}[x] = s.\mathsf{funcs}[f.\mathsf{module}.\mathsf{funcs}[x]]$
  - $z.\mathsf{datas}[x] = s.\mathsf{datas}[f.\mathsf{module}.\mathsf{datas}[x]]$
  - $z.\mathsf{elems}[x] = s.\mathsf{elems}[f.\mathsf{module}.\mathsf{elems}[x]]$
  - $z.\mathsf{locals}[x] = f.\mathsf{locals}[x]$
- These shorthands also extend to notation for updating state:
  - $z[.\mathsf{globals}[x].\mathsf{value} = v] = s[.\mathsf{globals}[f.\mathsf{module}.\mathsf{globals}[x]].\mathsf{value} = v]; f$
  - $z[.\mathsf{mems}[x].\mathsf{bytes}[i : j] = b^*] = s[.\mathsf{mems}[f.\mathsf{module}.\mathsf{mems}[x]].\mathsf{bytes}[i : j] = b^*]; f$
  - $z[.\mathsf{tables}[x].\mathsf{refs}[i] = r] = s[.\mathsf{tables}[f.\mathsf{module}.\mathsf{tables}[x]].\mathsf{refs}[i] = r]; f$
  - $z[.\mathsf{locals}[x] = v] = s; f[.\mathsf{locals}[x] = v]$

4.3 Numerics

Numeric primitives are defined in a generic manner, by operators indexed over a bit width N.

Some operators are *non-deterministic*, because they can return one of several possible results (such as different NaN values). Technically, each operator thus returns a *set* of allowed values. For convenience, deterministic results are expressed as plain values, which are assumed to be identified with a respective singleton set.

Some operators are *partial*, because they are not defined on certain inputs. Technically, an empty set of results is returned for these inputs.

In formal notation, each operator is defined by equational clauses that apply in decreasing order of precedence. That is, the first clause that is applicable to the given arguments defines the result. In some cases, similar clauses are combined into one by using the notation $\pm$ or $\mp$. When several of these placeholders occur in a single clause, then they must be resolved consistently: either the upper sign is chosen for all of them or the lower sign.

Note

For example, the fcopysign operator is defined as follows:

$$\begin{array}{lcl} \mathrm{fcopysign}_N(\pm p_1, \pm p_2) & = & \pm p_1 \\ \mathrm{fcopysign}_N(\pm p_1, \mp p_2) & = & \mp p_1 \end{array}$$

This definition is to be read as a shorthand for the following expansion of each clause into two separate ones:

$$\begin{array}{lcl} \mathrm{fcopysign}_N(+p_1, +p_2) & = & +p_1 \\ \mathrm{fcopysign}_N(-p_1, -p_2) & = & -p_1 \\ \mathrm{fcopysign}_N(+p_1, -p_2) & = & -p_1 \\ \mathrm{fcopysign}_N(-p_1, +p_2) & = & +p_1 \end{array}$$

Numeric operators are lifted to input sequences by applying the operator element-wise, returning a sequence of results. When there are multiple inputs, they must be of equal length.

$$op(c_1^n, \dots, c_k^n) \quad = \quad op(c_1^n[0], \dots, c_k^n[0]) \ \dots \ op(c_1^n[n-1], \dots, c_k^n[n-1])$$

Note

For example, the unary operator fabs, when given a sequence of floating-point values, return a sequence of floating-point results:

$$\text{fabs}_N(z^n) \quad = \quad \text{fabs}_N(z[0]) \ \dots \ \text{fabs}_N(z[n])$$

The binary operator iadd, when given two sequences of integers of the same length, n, return a sequence of integer results:

$$\text{iadd}_N(i_1^n, i_2^n) \quad = \quad \text{iadd}_N(i_1[0], i_2[0]) \ \dots \ \text{iadd}_N(i_1[n], i_2[n])$$

Conventions:

- The meta variable d is used to range over single bits.
- The meta variable p is used to range over (signless) magnitudes of floating-point values, including nan and ∞.
- The meta variable q is used to range over (signless) *rational* magnitudes, excluding nan or ∞.
- The notation f^{-1} denotes the inverse of a bijective function f.
- Truncation of rational values is written $\text{trunc}(\pm q)$, with the usual mathematical definition:

$$\text{trunc}(\pm q) \quad = \quad \pm i \qquad (\text{if } i \in \mathbb{N} \wedge +q - 1 < i \leq +q)$$

- Saturation of integers is written $\text{sat_u}_N(i)$ and $\text{sat_s}_N(i)$. The arguments to these two functions range over arbitrary signed integers.
  - Unsigned saturation, $\text{sat_u}_N(i)$ clamps i to between 0 and $2^N - 1$:

$$\begin{array}{lcll} \text{sat_u}_N(i) & = & 0 & (\text{if } i < 0) \\ \text{sat_u}_N(i) & = & 2^N - 1 & (\text{if } i > 2^N - 1) \\ \text{sat_u}_N(i) & = & i & (\text{otherwise}) \end{array}$$

  - Signed saturation, $\text{sat_s}_N(i)$ clamps i to between -2^{N-1} and $2^{N-1} - 1$:

$$\begin{array}{lcll} \text{sat_s}_N(i) & = & -2^{N-1} & (\text{if } i < -2^{N-1}) \\ \text{sat_s}_N(i) & = & 2^{N-1} - 1 & (\text{if } i > 2^{N-1} - 1) \\ \text{sat_s}_N(i) & = & i & (\text{otherwise}) \end{array}$$

4.3.1 Representations

Numbers and numeric vectors have an underlying binary representation as a sequence of bits:

$$\begin{array}{lcl} \text{bits}_{\text{i}N}(i) & = & \text{ibits}_N(i) \\ \text{bits}_{\text{f}N}(z) & = & \text{fbits}_N(z) \\ \text{bits}_{\text{v}N}(i) & = & \text{ibits}_N(i) \end{array}$$

The first case of these applies to representations of both integer value types and packed types.

Each of these functions is a bijection, hence they are invertible.

Integers

Integers are represented as base two unsigned numbers:

$$\text{ibits}_N(i) \quad = \quad d_{N-1} \ \dots \ d_0 \qquad (i = 2^{N-1} \cdot d_{N-1} + \dots + 2^0 \cdot d_0)$$

Boolean operators like $\wedge$, $\vee$, or $\veebar$ are lifted to bit sequences of equal length by applying them pointwise.

Floating-Point

Floating-point values are represented in the respective binary format defined by IEEE 754[22] (Section 3.4):

$$\begin{array}{lcl}
\text{fbits}_N(\pm(1 + m \cdot 2^{-M}) \cdot 2^e) & = & \text{fsign}(\pm)\ \text{ibits}_E(e + \text{fbias}_N)\ \text{ibits}_M(m) \\
\text{fbits}_N(\pm(0 + m \cdot 2^{-M}) \cdot 2^e) & = & \text{fsign}(\pm)\ (0)^E\ \text{ibits}_M(m) \\
\text{fbits}_N(\pm\infty) & = & \text{fsign}(\pm)\ (1)^E\ (0)^M \\
\text{fbits}_N(\pm\mathsf{nan}(n)) & = & \text{fsign}(\pm)\ (1)^E\ \text{ibits}_M(n) \\
\\
\text{fbias}_N & = & 2^{E-1} - 1 \\
\text{fsign}(+) & = & 0 \\
\text{fsign}(-) & = & 1
\end{array}$$

where $M = \text{signif}(N)$ and $E = \text{expon}(N)$.

Vectors

Numeric vectors of type $\text{v}N$ have the same underlying representation as an $\text{i}N$. They can also be interpreted as a sequence of numeric values packed into a $\text{v}N$ with a particular *shape* $t\text{×}M$, provided that $N = |t| \cdot M$.

$$\begin{array}{rcl}
\text{lanes}_{t\text{×}M}(c) & = & c_0 \ \dots \ c_{M-1} \\
\multicolumn{3}{r}{(\text{where } w = |t|/8} \\
\multicolumn{3}{r}{\wedge\ b^* = \text{bytes}_{\text{i}N}(c)} \\
\multicolumn{3}{r}{\wedge\ c_i = \text{bytes}_t^{-1}(b^*[i \cdot w : w]))}
\end{array}$$

This function is a bijection on $\text{i}N$, hence it is invertible.

Numeric values can be *packed* into lanes of a specific lane type and vice versa:

$$\begin{array}{lcl}
\text{pack}_{numtype}(c) & = & c \\
\text{pack}_{packtype}(c) & = & \text{wrap}_{|\text{unpack}(packtype)|,|packtype|}(c) \\
\text{unpack}_{numtype}(c) & = & c \\
\text{unpack}_{packtype}(c) & = & \text{extend}^{\text{u}}_{|packtype|,|\text{unpack}(packtype)|}(c)
\end{array}$$

Storage

When a number is stored into memory, it is converted into a sequence of bytes in little endian[23] byte order:

$$\begin{array}{lcl}
\text{bytes}_t(i) & = & \text{littleendian}(\text{bits}_t(i)) \\
\\
\text{littleendian}(\epsilon) & = & \epsilon \\
\text{littleendian}(d^8\ d'^*\ ) & = & \text{littleendian}(d'^*)\ \text{ibits}_8^{-1}(d^8)
\end{array}$$

Again these functions are invertible bijections.

4.3.2 Integer Operations

Sign Interpretation

Integer operators are defined on $i\text{N}$ values. Operators that use a signed interpretation convert the value using the following definition, which takes the two's complement when the value lies in the upper half of the value range (i.e., its most significant bit is 1):

$$\begin{array}{lcll}
\text{signed}_N(i) & = & i & (0 \le i < 2^{N-1}) \\
\text{signed}_N(i) & = & i - 2^N & (2^{N-1} \le i < 2^N)
\end{array}$$

This function is bijective, and hence invertible.

[22] https://ieeexplore.ieee.org/document/8766229
[23] https://en.wikipedia.org/wiki/Endianness#Little-endian

Boolean Interpretation

The integer result of predicates – i.e., tests and relational operators – is defined with the help of the following auxiliary function producing the value 1 or 0 depending on a condition.

$$\begin{array}{lcll} \mathrm{bool}(C) & = & 1 & (\text{if } C) \\ \mathrm{bool}(C) & = & 0 & (\text{otherwise}) \end{array}$$

$\mathrm{iadd}_N(i_1, i_2)$

- Return the result of adding i_1 and i_2 modulo 2^N.

$$\mathrm{iadd}_N(i_1, i_2) \quad = \quad (i_1 + i_2) \bmod 2^N$$

$\mathrm{isub}_N(i_1, i_2)$

- Return the result of subtracting i_2 from i_1 modulo 2^N.

$$\mathrm{isub}_N(i_1, i_2) \quad = \quad (i_1 - i_2 + 2^N) \bmod 2^N$$

$\mathrm{imul}_N(i_1, i_2)$

- Return the result of multiplying i_1 and i_2 modulo 2^N.

$$\mathrm{imul}_N(i_1, i_2) \quad = \quad (i_1 \cdot i_2) \bmod 2^N$$

$\mathrm{idiv_u}_N(i_1, i_2)$

- If i_2 is 0, then the result is undefined.
- Else, return the result of dividing i_1 by i_2, truncated toward zero.

$$\begin{array}{lcl} \mathrm{idiv_u}_N(i_1, 0) & = & \{\} \\ \mathrm{idiv_u}_N(i_1, i_2) & = & \mathrm{trunc}(i_1 / i_2) \end{array}$$

Note

This operator is partial.

$\mathrm{idiv_s}_N(i_1, i_2)$

- Let j_1 be the signed interpretation of i_1.
- Let j_2 be the signed interpretation of i_2.
- If j_2 is 0, then the result is undefined.
- Else if j_1 divided by j_2 is 2^{N-1}, then the result is undefined.
- Else, return the result of dividing j_1 by j_2, truncated toward zero.

$$\begin{array}{lcll} \mathrm{idiv_s}_N(i_1, 0) & = & \{\} & \\ \mathrm{idiv_s}_N(i_1, i_2) & = & \{\} & (\text{if } \mathrm{signed}_N(i_1) / \mathrm{signed}_N(i_2) = 2^{N-1}) \\ \mathrm{idiv_s}_N(i_1, i_2) & = & \mathrm{signed}_N^{-1}(\mathrm{trunc}(\mathrm{signed}_N(i_1) / \mathrm{signed}_N(i_2))) & \end{array}$$

Note

This operator is partial. Besides division by 0, the result of $(-2^{N-1})/(-1) = +2^{N-1}$ is not representable as an N-bit signed integer.

$\text{irem_u}_N(i_1, i_2)$

- If i_2 is 0, then the result is undefined.
- Else, return the remainder of dividing i_1 by i_2.

$$\begin{array}{lcl} \text{irem_u}_N(i_1, 0) & = & \{\} \\ \text{irem_u}_N(i_1, i_2) & = & i_1 - i_2 \cdot \text{trunc}(i_1 / i_2) \end{array}$$

Note

This operator is partial.

As long as both operators are defined, it holds that $i_1 = i_2 \cdot \text{idiv_u}(i_1, i_2) + \text{irem_u}(i_1, i_2)$.

$\text{irem_s}_N(i_1, i_2)$

- Let j_1 be the signed interpretation of i_1.
- Let j_2 be the signed interpretation of i_2.
- If i_2 is 0, then the result is undefined.
- Else, return the remainder of dividing j_1 by j_2, with the sign of the dividend j_1.

$$\begin{array}{lcl} \text{irem_s}_N(i_1, 0) & = & \{\} \\ \text{irem_s}_N(i_1, i_2) & = & \text{signed}_N^{-1}(j_1 - j_2 \cdot \text{trunc}(j_1 / j_2)) \\ & & (\text{where } j_1 = \text{signed}_N(i_1) \wedge j_2 = \text{signed}_N(i_2)) \end{array}$$

Note

This operator is partial.

As long as both operators are defined, it holds that $i_1 = i_2 \cdot \text{idiv_s}(i_1, i_2) + \text{irem_s}(i_1, i_2)$.

$\text{inot}_N(i)$

- Return the bitwise negation of i.

$$\text{inot}_N(i) \quad = \quad \text{ibits}_N^{-1}(\text{ibits}_N(i) \veebar \text{ibits}_N(2^N - 1))$$

$\text{irev}_N(i)$

- Return the bitwise reversal of i.

$$\text{irev}_N(i) \quad = \quad \text{ibits}_N^{-1}((d^N[N - i])^{i \leq N}) \quad (\text{if } d^N = \text{ibits}_N(i))$$

$\text{iand}_N(i_1, i_2)$

- Return the bitwise conjunction of i_1 and i_2.

$$\text{iand}_N(i_1, i_2) \quad = \quad \text{ibits}_N^{-1}(\text{ibits}_N(i_1) \wedge \text{ibits}_N(i_2))$$

$\text{iandnot}_N(i_1, i_2)$

- Return the bitwise conjunction of i_1 and the bitwise negation of i_2.

$$\text{iandnot}_N(i_1, i_2) \quad = \quad \text{iand}_N(i_1, \text{inot}_N(i_2))$$

$\text{ior}_N(i_1, i_2)$

- Return the bitwise disjunction of i_1 and i_2.

$$\text{ior}_N(i_1, i_2) \quad = \quad \text{ibits}_N^{-1}(\text{ibits}_N(i_1) \vee \text{ibits}_N(i_2))$$

$\text{ixor}_N(i_1, i_2)$

- Return the bitwise exclusive disjunction of i_1 and i_2.

$$\text{ixor}_N(i_1, i_2) \quad = \quad \text{ibits}_N^{-1}(\text{ibits}_N(i_1) \veebar \text{ibits}_N(i_2))$$

$\text{ishl}_N(i_1, i_2)$

- Let k be i_2 modulo N.
- Return the result of shifting i_1 left by k bits, modulo 2^N.

$$\text{ishl}_N(i_1, i_2) \quad = \quad \text{ibits}_N^{-1}(d_2^{N-k}\ 0^k) \quad (\text{if } \text{ibits}_N(i_1) = d_1^k\ d_2^{N-k} \wedge k = i_2 \bmod N)$$

$\text{ishr_u}_N(i_1, i_2)$

- Let k be i_2 modulo N.
- Return the result of shifting i_1 right by k bits, extended with 0 bits.

$$\text{ishr_u}_N(i_1, i_2) \quad = \quad \text{ibits}_N^{-1}(0^k\ d_1^{N-k}) \quad (\text{if } \text{ibits}_N(i_1) = d_1^{N-k}\ d_2^k \wedge k = i_2 \bmod N)$$

$\text{ishr_s}_N(i_1, i_2)$

- Let k be i_2 modulo N.
- Return the result of shifting i_1 right by k bits, extended with the most significant bit of the original value.

$$\text{ishr_s}_N(i_1, i_2) \quad = \quad \text{ibits}_N^{-1}(d_0^{k+1}\ d_1^{N-k-1}) \quad (\text{if } \text{ibits}_N(i_1) = d_0\ d_1^{N-k-1}\ d_2^k \wedge k = i_2 \bmod N)$$

$\text{irotl}_N(i_1, i_2)$

- Let k be i_2 modulo N.
- Return the result of rotating i_1 left by k bits.

$$\text{irotl}_N(i_1, i_2) \quad = \quad \text{ibits}_N^{-1}(d_2^{N-k}\ d_1^k) \quad (\text{if } \text{ibits}_N(i_1) = d_1^k\ d_2^{N-k} \wedge k = i_2 \bmod N)$$

$\text{irotr}_N(i_1, i_2)$

- Let k be i_2 modulo N.
- Return the result of rotating i_1 right by k bits.

$$\text{irotr}_N(i_1, i_2) \quad = \quad \text{ibits}_N^{-1}(d_2^k\ d_1^{N-k}) \quad (\text{if } \text{ibits}_N(i_1) = d_1^{N-k}\ d_2^k \wedge k = i_2 \bmod N)$$

$\text{iclz}_N(i)$

- Return the count of leading zero bits in i; all bits are considered leading zeros if i is 0.

$$\text{iclz}_N(i) \quad = \quad k \quad (\text{if } \text{ibits}_N(i) = 0^k\ (1\ d^*)^?)$$

$\text{ictz}_N(i)$

- Return the count of trailing zero bits in i; all bits are considered trailing zeros if i is 0.

$$\text{ictz}_N(i) \quad = \quad k \quad (\text{if } \text{ibits}_N(i) = (d^*\ 1)^?\ 0^k)$$

$\text{ipopcnt}_N(i)$

- Return the count of non-zero bits in i.

$$\text{ipopcnt}_N(i) \quad = \quad k \quad (\text{if } \text{ibits}_N(i) = (0^* \; 1)^k \; 0^*)$$

$\text{ieqz}_N(i)$

- Return 1 if i is zero, 0 otherwise.

$$\text{ieqz}_N(i) \quad = \quad \text{bool}(i = 0)$$

$\text{inez}_N(i)$

- Return 0 if i is zero, 1 otherwise.

$$\text{inez}_N(i) \quad = \quad \text{bool}(i \neq 0)$$

$\text{ieq}_N(i_1, i_2)$

- Return 1 if i_1 equals i_2, 0 otherwise.

$$\text{ieq}_N(i_1, i_2) \quad = \quad \text{bool}(i_1 = i_2)$$

$\text{ine}_N(i_1, i_2)$

- Return 1 if i_1 does not equal i_2, 0 otherwise.

$$\text{ine}_N(i_1, i_2) \quad = \quad \text{bool}(i_1 \neq i_2)$$

$\text{ilt_u}_N(i_1, i_2)$

- Return 1 if i_1 is less than i_2, 0 otherwise.

$$\text{ilt_u}_N(i_1, i_2) \quad = \quad \text{bool}(i_1 < i_2)$$

$\text{ilt_s}_N(i_1, i_2)$

- Let j_1 be the signed interpretation of i_1.
- Let j_2 be the signed interpretation of i_2.
- Return 1 if j_1 is less than j_2, 0 otherwise.

$$\text{ilt_s}_N(i_1, i_2) \quad = \quad \text{bool}(\text{signed}_N(i_1) < \text{signed}_N(i_2))$$

$\text{igt_u}_N(i_1, i_2)$

- Return 1 if i_1 is greater than i_2, 0 otherwise.

$$\text{igt_u}_N(i_1, i_2) \quad = \quad \text{bool}(i_1 > i_2)$$

$\text{igt_s}_N(i_1, i_2)$

- Let j_1 be the signed interpretation of i_1.
- Let j_2 be the signed interpretation of i_2.
- Return 1 if j_1 is greater than j_2, 0 otherwise.

$$\text{igt_s}_N(i_1, i_2) \quad = \quad \text{bool}(\text{signed}_N(i_1) > \text{signed}_N(i_2))$$

$\text{ile_u}_N(i_1, i_2)$

- Return 1 if i_1 is less than or equal to i_2, 0 otherwise.

$$\text{ile_u}_N(i_1, i_2) \quad = \quad \text{bool}(i_1 \leq i_2)$$

$\text{ile_s}_N(i_1, i_2)$

- Let j_1 be the signed interpretation of i_1.
- Let j_2 be the signed interpretation of i_2.
- Return 1 if j_1 is less than or equal to j_2, 0 otherwise.

$$\text{ile_s}_N(i_1, i_2) \quad = \quad \text{bool}(\text{signed}_N(i_1) \leq \text{signed}_N(i_2))$$

$\text{ige_u}_N(i_1, i_2)$

- Return 1 if i_1 is greater than or equal to i_2, 0 otherwise.

$$\text{ige_u}_N(i_1, i_2) \quad = \quad \text{bool}(i_1 \geq i_2)$$

$\text{ige_s}_N(i_1, i_2)$

- Let j_1 be the signed interpretation of i_1.
- Let j_2 be the signed interpretation of i_2.
- Return 1 if j_1 is greater than or equal to j_2, 0 otherwise.

$$\text{ige_s}_N(i_1, i_2) \quad = \quad \text{bool}(\text{signed}_N(i_1) \geq \text{signed}_N(i_2))$$

$\text{iextend}M\text{_s}_N(i)$

- Let j be the result of computing $\text{wrap}_{N,M}(i)$.
- Return $\text{extend}^{\text{s}}{}_{M,N}(j)$.

$$\text{iextend}M\text{_s}_N(i) \quad = \quad \text{extend}^{\text{s}}{}_{M,N}(\text{wrap}_{N,M}(i))$$

$\text{ibitselect}_N(i_1, i_2, i_3)$

- Let j_1 be the bitwise conjunction of i_1 and i_3.
- Let j_3' be the bitwise negation of i_3.
- Let j_2 be the bitwise conjunction of i_2 and j_3'.
- Return the bitwise disjunction of j_1 and j_2.

$$\text{ibitselect}_N(i_1, i_2, i_3) \quad = \quad \text{ior}_N(\text{iand}_N(i_1, i_3), \text{iand}_N(i_2, \text{inot}_N(i_3)))$$

$\text{iabs}_N(i)$

- Let j be the signed interpretation of i.
- If j is greater than or equal to 0, then return i.
- Else return the negation of j, modulo 2^N.

$$\begin{array}{lcll} \text{iabs}_N(i) & = & i & (\text{if } \text{signed}_N(i) \geq 0) \\ \text{iabs}_N(i) & = & -\text{signed}_N(i) \bmod 2^N & (\text{otherwise}) \end{array}$$

$\text{ineg}_N(i)$

- Return the result of negating i, modulo 2^N.

$$\text{ineg}_N(i) \quad = \quad (2^N - i) \bmod 2^N$$

$\text{imin_u}_N(i_1, i_2)$

- Return i_1 if $\text{ilt_u}_N(i_1, i_2)$ is 1, return i_2 otherwise.

$$\begin{array}{lcll} \text{imin_u}_N(i_1, i_2) & = & i_1 & (\text{if } \text{ilt_u}_N(i_1, i_2) = 1) \\ \text{imin_u}_N(i_1, i_2) & = & i_2 & (\text{otherwise}) \end{array}$$

$\text{imin_s}_N(i_1, i_2)$

- Return i_1 if $\text{ilt_s}_N(i_1, i_2)$ is 1, return i_2 otherwise.

$$\begin{array}{lcll} \text{imin_s}_N(i_1, i_2) & = & i_1 & (\text{if } \text{ilt_s}_N(i_1, i_2) = 1) \\ \text{imin_s}_N(i_1, i_2) & = & i_2 & (\text{otherwise}) \end{array}$$

$\text{imax_u}_N(i_1, i_2)$

- Return i_1 if $\text{igt_u}_N(i_1, i_2)$ is 1, return i_2 otherwise.

$$\begin{array}{lcll} \text{imax_u}_N(i_1, i_2) & = & i_1 & (\text{if } \text{igt_u}_N(i_1, i_2) = 1) \\ \text{imax_u}_N(i_1, i_2) & = & i_2 & (\text{otherwise}) \end{array}$$

$\text{imax_s}_N(i_1, i_2)$

- Return i_1 if $\text{igt_s}_N(i_1, i_2)$ is 1, return i_2 otherwise.

$$\begin{array}{lcll} \text{imax_s}_N(i_1, i_2) & = & i_1 & (\text{if } \text{igt_s}_N(i_1, i_2) = 1) \\ \text{imax_s}_N(i_1, i_2) & = & i_2 & (\text{otherwise}) \end{array}$$

$\text{iadd_sat_u}_N(i_1, i_2)$

- Let i be the result of adding i_1 and i_2.
- Return $\text{sat_u}_N(i)$.

$$\text{iadd_sat_u}_N(i_1, i_2) \quad = \quad \text{sat_u}_N(i_1 + i_2)$$

$\text{iadd_sat_s}_N(i_1, i_2)$

- Let j_1 be the signed interpretation of i_1
- Let j_2 be the signed interpretation of i_2
- Let j be the result of adding j_1 and j_2.
- Return the value whose signed interpretation is $\text{sat_s}_N(j)$.

$$\text{iadd_sat_s}_N(i_1, i_2) \quad = \quad \text{signed}_N^{-1}(\text{sat_s}_N(\text{signed}_N(i_1) + \text{signed}_N(i_2)))$$

$\text{isub_sat_u}_N(i_1, i_2)$

- Let i be the result of subtracting i_2 from i_1.
- Return $\text{sat_u}_N(i)$.

$$\text{isub_sat_u}_N(i_1, i_2) \quad = \quad \text{sat_u}_N(i_1 - i_2)$$

$\text{isub_sat_s}_N(i_1, i_2)$

- Let j_1 be the signed interpretation of i_1
- Let j_2 be the signed interpretation of i_2
- Let j be the result of subtracting j_2 from j_1.
- Return the value whose signed interpretation is $\text{sat_s}_N(j)$.

$$\text{isub_sat_s}_N(i_1, i_2) \quad = \quad \text{signed}_N^{-1}(\text{sat_s}_N(\text{signed}_N(i_1) - \text{signed}_N(i_2)))$$

$\text{iavgr_u}_N(i_1, i_2)$

- Let j be the result of adding i_1, i_2, and 1.
- Return the result of dividing j by 2, truncated toward zero.

$$\text{iavgr_u}_N(i_1, i_2) \quad = \quad \text{trunc}((i_1 + i_2 + 1)/2)$$

$\text{iq15mulrsat_s}_N(i_1, i_2)$

- Return the whose signed interpretation is the result of $\text{sat_s}_N(\text{ishr_s}_N(i_1 \cdot i_2 + 2^{14}, 15))$.

$$\text{iq15mulrsat_s}_N(i_1, i_2) \quad = \quad \text{signed}_N^{-1}(\text{sat_s}_N(\text{ishr_s}_N(i_1 \cdot i_2 + 2^{14}, 15)))$$

4.3.3 Floating-Point Operations

Floating-point arithmetic follows the IEEE 754[24] standard, with the following qualifications:

- All operators use round-to-nearest ties-to-even, except where otherwise specified. Non-default directed rounding attributes are not supported.
- Following the recommendation that operators propagate NaN payloads from their operands is permitted but not required.
- All operators use "non-stop" mode, and floating-point exceptions are not otherwise observable. In particular, neither alternate floating-point exception handling attributes nor operators on status flags are supported. There is no observable difference between quiet and signalling NaNs.

Note

Some of these limitations may be lifted in future versions of WebAssembly.

Rounding

Rounding always is round-to-nearest ties-to-even, in correspondence with IEEE 754[25] (Section 4.3.1).

An *exact* floating-point number is a rational number that is exactly representable as a floating-point number of given bit width N.

A *limit* number for a given floating-point bit width N is a positive or negative number whose magnitude is the smallest power of 2 that is not exactly representable as a floating-point number of width N (that magnitude is 2^{128} for $N = 32$ and 2^{1024} for $N = 64$).

A *candidate* number is either an exact floating-point number or a positive or negative limit number for the given bit width N.

A *candidate pair* is a pair z_1, z_2 of candidate numbers, such that no candidate number exists that lies between the two.

[24] https://ieeexplore.ieee.org/document/8766229
[25] https://ieeexplore.ieee.org/document/8766229

A real number r is converted to a floating-point value of bit width N as follows:

- If r is 0, then return $+0$.
- Else if r is an exact floating-point number, then return r.
- Else if r greater than or equal to the positive limit, then return $+\infty$.
- Else if r is less than or equal to the negative limit, then return $-\infty$.
- Else if z_1 and z_2 are a candidate pair such that $z_1 < r < z_2$, then:
  - If $|r - z_1| < |r - z_2|$, then let z be z_1.
  - Else if $|r - z_1| > |r - z_2|$, then let z be z_2.
  - Else if $|r - z_1| = |r - z_2|$ and the significand of z_1 is even, then let z be z_1.
  - Else, let z be z_2.
- If z is 0, then:
  - If $r < 0$, then return -0.
  - Else, return $+0$.
- Else if z is a limit number, then:
  - If $r < 0$, then return $-\infty$.
  - Else, return $+\infty$.
- Else, return z.

$$
\begin{array}{lcll}
\mathrm{float}_N(0) & = & +0 & \\
\mathrm{float}_N(r) & = & r & (\text{if } r \in \mathrm{exact}_N) \\
\mathrm{float}_N(r) & = & +\infty & (\text{if } r \geq +\mathrm{limit}_N) \\
\mathrm{float}_N(r) & = & -\infty & (\text{if } r \leq -\mathrm{limit}_N) \\
\mathrm{float}_N(r) & = & \mathrm{closest}_N(r, z_1, z_2) & (\text{if } z_1 < r < z_2 \wedge (z_1, z_2) \in \mathrm{candidatepair}_N) \\
\\
\mathrm{closest}_N(r, z_1, z_2) & = & \mathrm{rectify}_N(r, z_1) & (\text{if } |r - z_1| < |r - z_2|) \\
\mathrm{closest}_N(r, z_1, z_2) & = & \mathrm{rectify}_N(r, z_2) & (\text{if } |r - z_1| > |r - z_2|) \\
\mathrm{closest}_N(r, z_1, z_2) & = & \mathrm{rectify}_N(r, z_1) & (\text{if } |r - z_1| = |r - z_2| \wedge \mathrm{even}_N(z_1)) \\
\mathrm{closest}_N(r, z_1, z_2) & = & \mathrm{rectify}_N(r, z_2) & (\text{if } |r - z_1| = |r - z_2| \wedge \mathrm{even}_N(z_2)) \\
\\
\mathrm{rectify}_N(r, \pm\mathrm{limit}_N) & = & \pm\infty & \\
\mathrm{rectify}_N(r, 0) & = & +0 \quad (r \geq 0) & \\
\mathrm{rectify}_N(r, 0) & = & -0 \quad (r < 0) & \\
\mathrm{rectify}_N(r, z) & = & z &
\end{array}
$$

where:

$$
\begin{array}{lcl}
\mathrm{exact}_N & = & f_{N \cap \mathbb{Q}} \\
\mathrm{limit}_N & = & 2^{2^{\mathrm{expon}(N)-1}} \\
\mathrm{candidate}_N & = & \mathrm{exact}_N \cup \{+\mathrm{limit}_N, -\mathrm{limit}_N\} \\
\mathrm{candidatepair}_N & = & \{(z_1, z_2) \in \mathrm{candidate}_N^2 \mid z_1 < z_2 \wedge \forall z \in \mathrm{candidate}_N, z \leq z_1 \vee z \geq z_2\} \\
\mathrm{even}_N((d + m \cdot 2^{-M}) \cdot 2^e) & \Leftrightarrow & m \bmod 2 = 0 \\
\mathrm{even}_N(\pm\mathrm{limit}_N) & \Leftrightarrow & \mathrm{true}
\end{array}
$$

NaN Propagation

When the result of a floating-point operator other than fneg, fabs, or fcopysign is a NaN, then its sign is non-deterministic and the payload is computed as follows:

- If the payload of all NaN inputs to the operator is canonical (including the case that there are no NaN inputs), then the payload of the output is canonical as well.
- Otherwise the payload is picked non-deterministically among all arithmetic NaNs; that is, its most significant bit is 1 and all others are unspecified.

- In the deterministic profile, only positive canonical NaN outputs are produced.

The non-deterministic result is expressed by the following auxiliary function producing a set of allowed outputs from a set of inputs:

$$
\begin{array}{lrcll}
 & \mathrm{nans}_N\{z^*\} & = & \{+\mathsf{nan}(\mathrm{canon}_N)\} & \\
[!\mathrm{DET}] & \mathrm{nans}_N\{z^*\} & = & \{+\mathsf{nan}(n), -\mathsf{nan}(n) \mid n = \mathrm{canon}_N\} & (\text{if } \forall \pm\mathsf{nan}(n) \in z^*,\ n = \mathrm{canon}_N) \\
[!\mathrm{DET}] & \mathrm{nans}_N\{z^*\} & = & \{+\mathsf{nan}(n), -\mathsf{nan}(n) \mid n \geq \mathrm{canon}_N\} & (\text{if } \exists \pm\mathsf{nan}(n) \in z^*,\ n \neq \mathrm{canon}_N)
\end{array}
$$

$\mathrm{fadd}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return an element of $\mathrm{nans}_N\{z_1, z_2\}$.
- Else if both z_1 and z_2 are infinities of opposite signs, then return an element of $\mathrm{nans}_N\{\}$.
- Else if both z_1 and z_2 are infinities of equal sign, then return that infinity.
- Else if either z_1 or z_2 is an infinity, then return that infinity.
- Else if both z_1 and z_2 are zeroes of opposite sign, then return positive zero.
- Else if both z_1 and z_2 are zeroes of equal sign, then return that zero.
- Else if either z_1 or z_2 is a zero, then return the other operand.
- Else if both z_1 and z_2 are values with the same magnitude but opposite signs, then return positive zero.
- Else return the result of adding z_1 and z_2, rounded to the nearest representable value.

$$
\begin{array}{lcl}
\mathrm{fadd}_N(\pm\mathsf{nan}(n), z_2) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n), z_2\} \\
\mathrm{fadd}_N(z_1, \pm\mathsf{nan}(n)) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n), z_1\} \\
\mathrm{fadd}_N(\pm\infty, \mp\infty) & = & \mathrm{nans}_N\{\} \\
\mathrm{fadd}_N(\pm\infty, \pm\infty) & = & \pm\infty \\
\mathrm{fadd}_N(z_1, \pm\infty) & = & \pm\infty \\
\mathrm{fadd}_N(\pm\infty, z_2) & = & \pm\infty \\
\mathrm{fadd}_N(\pm 0, \mp 0) & = & +0 \\
\mathrm{fadd}_N(\pm 0, \pm 0) & = & \pm 0 \\
\mathrm{fadd}_N(z_1, \pm 0) & = & z_1 \\
\mathrm{fadd}_N(\pm 0, z_2) & = & z_2 \\
\mathrm{fadd}_N(\pm q, \mp q) & = & +0 \\
\mathrm{fadd}_N(z_1, z_2) & = & \mathrm{float}_N(z_1 + z_2)
\end{array}
$$

$\mathrm{fsub}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return an element of $\mathrm{nans}_N\{z_1, z_2\}$.
- Else if both z_1 and z_2 are infinities of equal signs, then return an element of $\mathrm{nans}_N\{\}$.
- Else if both z_1 and z_2 are infinities of opposite sign, then return z_1.
- Else if z_1 is an infinity, then return that infinity.
- Else if z_2 is an infinity, then return that infinity negated.
- Else if both z_1 and z_2 are zeroes of equal sign, then return positive zero.
- Else if both z_1 and z_2 are zeroes of opposite sign, then return z_1.
- Else if z_2 is a zero, then return z_1.
- Else if z_1 is a zero, then return z_2 negated.
- Else if both z_1 and z_2 are the same value, then return positive zero.
- Else return the result of subtracting z_2 from z_1, rounded to the nearest representable value.

$$
\begin{array}{lcl}
\mathrm{fsub}_N(\pm\mathsf{nan}(n), z_2) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n), z_2\} \\
\mathrm{fsub}_N(z_1, \pm\mathsf{nan}(n)) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n), z_1\} \\
\mathrm{fsub}_N(\pm\infty, \pm\infty) & = & \mathrm{nans}_N\{\} \\
\mathrm{fsub}_N(\pm\infty, \mp\infty) & = & \pm\infty \\
\mathrm{fsub}_N(z_1, \pm\infty) & = & \mp\infty \\
\mathrm{fsub}_N(\pm\infty, z_2) & = & \pm\infty \\
\mathrm{fsub}_N(\pm 0, \pm 0) & = & +0 \\
\mathrm{fsub}_N(\pm 0, \mp 0) & = & \pm 0 \\
\mathrm{fsub}_N(z_1, \pm 0) & = & z_1 \\
\mathrm{fsub}_N(\pm 0, \pm q_2) & = & \mp q_2 \\
\mathrm{fsub}_N(\pm q, \pm q) & = & +0 \\
\mathrm{fsub}_N(z_1, z_2) & = & \mathrm{float}_N(z_1 - z_2)
\end{array}
$$

Note

Up to the non-determinism regarding NaNs, it always holds that $\mathrm{fsub}_N(z_1, z_2) = \mathrm{fadd}_N(z_1, \mathrm{fneg}_N(z_2))$.

$\mathrm{fmul}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return an element of $\mathrm{nans}_N\{z_1, z_2\}$.
- Else if one of z_1 and z_2 is a zero and the other an infinity, then return an element of $\mathrm{nans}_N\{\}$.
- Else if both z_1 and z_2 are infinities of equal sign, then return positive infinity.
- Else if both z_1 and z_2 are infinities of opposite sign, then return negative infinity.
- Else if either z_1 or z_2 is an infinity and the other a value with equal sign, then return positive infinity.
- Else if either z_1 or z_2 is an infinity and the other a value with opposite sign, then return negative infinity.
- Else if either z_1 or z_2 is a zero and the other a value with the same sign, then return positive zero.
- Else if either z_1 or z_2 is a zero and the other a value with opposite sign, then return negative zero.
- Else if both z_1 and z_2 are zeroes of equal sign, then return positive zero.
- Else if both z_1 and z_2 are zeroes of opposite sign, then return negative zero.
- Else return the result of multiplying z_1 and z_2, rounded to the nearest representable value.

$$
\begin{array}{lcl}
\mathrm{fmul}_N(\pm\mathsf{nan}(n), z_2) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n), z_2\} \\
\mathrm{fmul}_N(z_1, \pm\mathsf{nan}(n)) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n), z_1\} \\
\mathrm{fmul}_N(\pm\infty, \pm 0) & = & \mathrm{nans}_N\{\} \\
\mathrm{fmul}_N(\pm\infty, \mp 0) & = & \mathrm{nans}_N\{\} \\
\mathrm{fmul}_N(\pm 0, \pm\infty) & = & \mathrm{nans}_N\{\} \\
\mathrm{fmul}_N(\pm 0, \mp\infty) & = & \mathrm{nans}_N\{\} \\
\mathrm{fmul}_N(\pm\infty, \pm\infty) & = & +\infty \\
\mathrm{fmul}_N(\pm\infty, \mp\infty) & = & -\infty \\
\mathrm{fmul}_N(\pm q_1, \pm\infty) & = & +\infty \\
\mathrm{fmul}_N(\pm q_1, \mp\infty) & = & -\infty \\
\mathrm{fmul}_N(\pm\infty, \pm q_2) & = & +\infty \\
\mathrm{fmul}_N(\pm\infty, \mp q_2) & = & -\infty \\
\mathrm{fmul}_N(\pm 0, \pm q_2) & = & +0 \\
\mathrm{fmul}_N(\pm 0, \mp q_2) & = & -0 \\
\mathrm{fmul}_N(\pm q_1, \pm 0) & = & +0 \\
\mathrm{fmul}_N(\pm q_1, \mp 0) & = & -0 \\
\mathrm{fmul}_N(z_1, z_2) & = & \mathrm{float}_N(z_1 \cdot z_2)
\end{array}
$$

$\mathrm{fdiv}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return an element of $\mathrm{nans}_N\{z_1, z_2\}$.
- Else if both z_1 and z_2 are infinities, then return an element of $\mathrm{nans}_N\{\}$.
- Else if both z_1 and z_2 are zeroes, then return an element of $\mathrm{nans}_N\{z_1, z_2\}$.
- Else if z_1 is an infinity and z_2 a value with equal sign, then return positive infinity.
- Else if z_1 is an infinity and z_2 a value with opposite sign, then return negative infinity.
- Else if z_2 is an infinity and z_1 a value with equal sign, then return positive zero.
- Else if z_2 is an infinity and z_1 a value with opposite sign, then return negative zero.
- Else if z_1 is a zero and z_2 a value with equal sign, then return positive zero.
- Else if z_1 is a zero and z_2 a value with opposite sign, then return negative zero.
- Else if z_2 is a zero and z_1 a value with equal sign, then return positive infinity.
- Else if z_2 is a zero and z_1 a value with opposite sign, then return negative infinity.
- Else return the result of dividing z_1 by z_2, rounded to the nearest representable value.

$$
\begin{array}{lcl}
\mathrm{fdiv}_N(\pm\mathsf{nan}(n), z_2) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n), z_2\} \\
\mathrm{fdiv}_N(z_1, \pm\mathsf{nan}(n)) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n), z_1\} \\
\mathrm{fdiv}_N(\pm\infty, \pm\infty) & = & \mathrm{nans}_N\{\} \\
\mathrm{fdiv}_N(\pm\infty, \mp\infty) & = & \mathrm{nans}_N\{\} \\
\mathrm{fdiv}_N(\pm 0, \pm 0) & = & \mathrm{nans}_N\{\} \\
\mathrm{fdiv}_N(\pm 0, \mp 0) & = & \mathrm{nans}_N\{\} \\
\mathrm{fdiv}_N(\pm\infty, \pm q_2) & = & +\infty \\
\mathrm{fdiv}_N(\pm\infty, \mp q_2) & = & -\infty \\
\mathrm{fdiv}_N(\pm q_1, \pm\infty) & = & +0 \\
\mathrm{fdiv}_N(\pm q_1, \mp\infty) & = & -0 \\
\mathrm{fdiv}_N(\pm 0, \pm q_2) & = & +0 \\
\mathrm{fdiv}_N(\pm 0, \mp q_2) & = & -0 \\
\mathrm{fdiv}_N(\pm q_1, \pm 0) & = & +\infty \\
\mathrm{fdiv}_N(\pm q_1, \mp 0) & = & -\infty \\
\mathrm{fdiv}_N(z_1, z_2) & = & \mathrm{float}_N(z_1/z_2)
\end{array}
$$

$\mathrm{fma}_N(z_1, z_2, z_3)$

The function fma is the same as *fusedMultiplyAdd* defined by IEEE 754[26] (Section 5.4.1). It computes $(z_1 \cdot z_2) + z_3$ as if with unbounded range and precision, rounding only once for the final result.

- If either z_1 or z_2 or z_3 is a NaN, return an element of $\mathrm{nans}_N\{z_1, z_2, z_3\}$.
- Else if either z_1 or z_2 is a zero and the other is an infinity, then return an element of $\mathrm{nans}_N\{\}$.
- Else if both z_1 and z_2 are infinities of equal sign, and z_3 is a negative infinity, then return an element of $\mathrm{nans}_N\{\}$.
- Else if both z_1 and z_2 are infinities of opposite sign, and z_3 is a positive infinity, then return an element of $\mathrm{nans}_N\{\}$.
- Else if either z_1 or z_2 is an infinity and the other is a value of the same sign, and z_3 is a negative infinity, then return an element of $\mathrm{nans}_N\{\}$.
- Else if either z_1 or z_2 is an infinity and the other is a value of the opposite sign, and z_3 is a positive infinity, then return an element of $\mathrm{nans}_N\{\}$.
- Else if both z_1 and z_2 are zeroes of the same sign and z_3 is a zero, then return positive zero.
- Else if both z_1 and z_2 are zeroes of the opposite sign and z_3 is a positive zero, then return positive zero.
- Else if both z_1 and z_2 are zeroes of the opposite sign and z_3 is a negative zero, then return negative zero.

[26] https://ieeexplore.ieee.org/document/8766229

- Else return the result of multiplying z_1 and z_2, adding z_3 to the intermediate, and the final result rounded to the nearest representable value.

$$
\begin{array}{lcl}
\mathrm{fma}_N(\pm\mathsf{nan}(n), z_2, z_3) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n), z_2, z_3\} \\
\mathrm{fma}_N(z_1, \pm\mathsf{nan}(n), z_3) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n), z_1, z_3\} \\
\mathrm{fma}_N(z_1, z_2, \pm\mathsf{nan}(n)) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n), z_1, z_2\} \\
\mathrm{fma}_N(\pm\infty, \pm 0, z_3) & = & \mathrm{nans}_N\{\} \\
\mathrm{fma}_N(\pm\infty, \mp 0, z_3) & = & \mathrm{nans}_N\{\} \\
\mathrm{fma}_N(\pm\infty, \pm\infty, -\infty) & = & \mathrm{nans}_N\{\} \\
\mathrm{fma}_N(\pm\infty, \mp\infty, +\infty) & = & \mathrm{nans}_N\{\} \\
\mathrm{fma}_N(\pm q_1, \pm\infty, -\infty) & = & \mathrm{nans}_N\{\} \\
\mathrm{fma}_N(\pm q_1, \mp\infty, +\infty) & = & \mathrm{nans}_N\{\} \\
\mathrm{fma}_N(\pm\infty, \pm q_1, -\infty) & = & \mathrm{nans}_N\{\} \\
\mathrm{fma}_N(\mp\infty, \pm q_1, +\infty) & = & \mathrm{nans}_N\{\} \\
\mathrm{fma}_N(\pm 0, \pm 0, \mp 0) & = & +0 \\
\mathrm{fma}_N(\pm 0, \pm 0, \pm 0) & = & +0 \\
\mathrm{fma}_N(\pm 0, \mp 0, +0) & = & +0 \\
\mathrm{fma}_N(\pm 0, \mp 0, -0) & = & -0 \\
\mathrm{fma}_N(z_1, z_2, z_3) & = & \mathrm{float}_N(z_1 \cdot z_2 + z_3)
\end{array}
$$

$\mathrm{fmin}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return an element of $\mathrm{nans}_N\{z_1, z_2\}$.
- Else if either z_1 or z_2 is a negative infinity, then return negative infinity.
- Else if either z_1 or z_2 is a positive infinity, then return the other value.
- Else if both z_1 and z_2 are zeroes of opposite signs, then return negative zero.
- Else return the smaller value of z_1 and z_2.

$$
\begin{array}{lcll}
\mathrm{fmin}_N(\pm\mathsf{nan}(n), z_2) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n), z_2\} & \\
\mathrm{fmin}_N(z_1, \pm\mathsf{nan}(n)) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n), z_1\} & \\
\mathrm{fmin}_N(+\infty, z_2) & = & z_2 & \\
\mathrm{fmin}_N(-\infty, z_2) & = & -\infty & \\
\mathrm{fmin}_N(z_1, +\infty) & = & z_1 & \\
\mathrm{fmin}_N(z_1, -\infty) & = & -\infty & \\
\mathrm{fmin}_N(\pm 0, \mp 0) & = & -0 & \\
\mathrm{fmin}_N(z_1, z_2) & = & z_1 & (\text{if } z_1 \leq z_2) \\
\mathrm{fmin}_N(z_1, z_2) & = & z_2 & (\text{if } z_2 \leq z_1)
\end{array}
$$

$\mathrm{fmax}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return an element of $\mathrm{nans}_N\{z_1, z_2\}$.
- Else if either z_1 or z_2 is a positive infinity, then return positive infinity.
- Else if either z_1 or z_2 is a negative infinity, then return the other value.
- Else if both z_1 and z_2 are zeroes of opposite signs, then return positive zero.
- Else return the larger value of z_1 and z_2.

$$
\begin{array}{lcll}
\mathrm{fmax}_N(\pm\mathsf{nan}(n), z_2) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n), z_2\} & \\
\mathrm{fmax}_N(z_1, \pm\mathsf{nan}(n)) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n), z_1\} & \\
\mathrm{fmax}_N(+\infty, z_2) & = & +\infty & \\
\mathrm{fmax}_N(-\infty, z_2) & = & z_2 & \\
\mathrm{fmax}_N(z_1, +\infty) & = & +\infty & \\
\mathrm{fmax}_N(z_1, -\infty) & = & z_1 & \\
\mathrm{fmax}_N(\pm 0, \mp 0) & = & +0 & \\
\mathrm{fmax}_N(z_1, z_2) & = & z_1 & (\text{if } z_1 \geq z_2) \\
\mathrm{fmax}_N(z_1, z_2) & = & z_2 & (\text{if } z_2 \geq z_1)
\end{array}
$$

$\mathrm{fcopysign}_N(z_1, z_2)$

- If z_1 and z_2 have the same sign, then return z_1.
- Else return z_1 with negated sign.

$$\begin{array}{lcl} \mathrm{fcopysign}_N(\pm p_1, \pm p_2) & = & \pm p_1 \\ \mathrm{fcopysign}_N(\pm p_1, \mp p_2) & = & \mp p_1 \end{array}$$

$\mathrm{fabs}_N(z)$

- If z is a NaN, then return z with positive sign.
- Else if z is an infinity, then return positive infinity.
- Else if z is a zero, then return positive zero.
- Else if z is a positive value, then z.
- Else return z negated.

$$\begin{array}{lcl} \mathrm{fabs}_N(\pm\mathsf{nan}(n)) & = & +\mathsf{nan}(n) \\ \mathrm{fabs}_N(\pm\infty) & = & +\infty \\ \mathrm{fabs}_N(\pm 0) & = & +0 \\ \mathrm{fabs}_N(\pm q) & = & +q \end{array}$$

$\mathrm{fneg}_N(z)$

- If z is a NaN, then return z with negated sign.
- Else if z is an infinity, then return that infinity negated.
- Else if z is a zero, then return that zero negated.
- Else return z negated.

$$\begin{array}{lcl} \mathrm{fneg}_N(\pm\mathsf{nan}(n)) & = & \mp\mathsf{nan}(n) \\ \mathrm{fneg}_N(\pm\infty) & = & \mp\infty \\ \mathrm{fneg}_N(\pm 0) & = & \mp 0 \\ \mathrm{fneg}_N(\pm q) & = & \mp q \end{array}$$

$\mathrm{fsqrt}_N(z)$

- If z is a NaN, then return an element of $\mathrm{nans}_N\{z\}$.
- Else if z is negative infinity, then return an element of $\mathrm{nans}_N\{\}$.
- Else if z is positive infinity, then return positive infinity.
- Else if z is a zero, then return that zero.
- Else if z has a negative sign, then return an element of $\mathrm{nans}_N\{\}$.
- Else return the square root of z.

$$\begin{array}{lcl} \mathrm{fsqrt}_N(\pm\mathsf{nan}(n)) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n)\} \\ \mathrm{fsqrt}_N(-\infty) & = & \mathrm{nans}_N\{\} \\ \mathrm{fsqrt}_N(+\infty) & = & +\infty \\ \mathrm{fsqrt}_N(\pm 0) & = & \pm 0 \\ \mathrm{fsqrt}_N(-q) & = & \mathrm{nans}_N\{\} \\ \mathrm{fsqrt}_N(+q) & = & \mathrm{float}_N\left(\sqrt{q}\right) \end{array}$$

$\mathrm{fceil}_N(z)$

- If z is a NaN, then return an element of $\mathrm{nans}_N\{z\}$.
- Else if z is an infinity, then return z.
- Else if z is a zero, then return z.

- Else if z is smaller than 0 but greater than -1, then return negative zero.
- Else return the smallest integral value that is not smaller than z.

$$
\begin{array}{lcll}
\mathrm{fceil}_N(\pm\mathsf{nan}(n)) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n)\} & \\
\mathrm{fceil}_N(\pm\infty) & = & \pm\infty & \\
\mathrm{fceil}_N(\pm 0) & = & \pm 0 & \\
\mathrm{fceil}_N(-q) & = & -0 & (\text{if } -1 < -q < 0) \\
\mathrm{fceil}_N(\pm q) & = & \mathrm{float}_N(i) & (\text{if } \pm q \leq i < \pm q + 1)
\end{array}
$$

$\mathrm{ffloor}_N(z)$

- If z is a NaN, then return an element of $\mathrm{nans}_N\{z\}$.
- Else if z is an infinity, then return z.
- Else if z is a zero, then return z.
- Else if z is greater than 0 but smaller than 1, then return positive zero.
- Else return the largest integral value that is not larger than z.

$$
\begin{array}{lcll}
\mathrm{ffloor}_N(\pm\mathsf{nan}(n)) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n)\} & \\
\mathrm{ffloor}_N(\pm\infty) & = & \pm\infty & \\
\mathrm{ffloor}_N(\pm 0) & = & \pm 0 & \\
\mathrm{ffloor}_N(+q) & = & +0 & (\text{if } 0 < +q < 1) \\
\mathrm{ffloor}_N(\pm q) & = & \mathrm{float}_N(i) & (\text{if } \pm q - 1 < i \leq \pm q)
\end{array}
$$

$\mathrm{ftrunc}_N(z)$

- If z is a NaN, then return an element of $\mathrm{nans}_N\{z\}$.
- Else if z is an infinity, then return z.
- Else if z is a zero, then return z.
- Else if z is greater than 0 but smaller than 1, then return positive zero.
- Else if z is smaller than 0 but greater than -1, then return negative zero.
- Else return the integral value with the same sign as z and the largest magnitude that is not larger than the magnitude of z.

$$
\begin{array}{lcll}
\mathrm{ftrunc}_N(\pm\mathsf{nan}(n)) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n)\} & \\
\mathrm{ftrunc}_N(\pm\infty) & = & \pm\infty & \\
\mathrm{ftrunc}_N(\pm 0) & = & \pm 0 & \\
\mathrm{ftrunc}_N(+q) & = & +0 & (\text{if } 0 < +q < 1) \\
\mathrm{ftrunc}_N(-q) & = & -0 & (\text{if } -1 < -q < 0) \\
\mathrm{ftrunc}_N(\pm q) & = & \mathrm{float}_N(\pm i) & (\text{if } +q - 1 < i \leq +q)
\end{array}
$$

$\mathrm{fnearest}_N(z)$

- If z is a NaN, then return an element of $\mathrm{nans}_N\{z\}$.
- Else if z is an infinity, then return z.
- Else if z is a zero, then return z.
- Else if z is greater than 0 but smaller than or equal to 0.5, then return positive zero.
- Else if z is smaller than 0 but greater than or equal to -0.5, then return negative zero.
- Else return the integral value that is nearest to z; if two values are equally near, return the even one.

$$
\begin{array}{lcll}
\mathrm{fnearest}_N(\pm\mathsf{nan}(n)) & = & \mathrm{nans}_N\{\pm\mathsf{nan}(n)\} & \\
\mathrm{fnearest}_N(\pm\infty) & = & \pm\infty & \\
\mathrm{fnearest}_N(\pm 0) & = & \pm 0 & \\
\mathrm{fnearest}_N(+q) & = & +0 & (\text{if } 0 < +q \leq 0.5) \\
\mathrm{fnearest}_N(-q) & = & -0 & (\text{if } -0.5 \leq -q < 0) \\
\mathrm{fnearest}_N(\pm q) & = & \mathrm{float}_N(\pm i) & (\text{if } |i-q| < 0.5) \\
\mathrm{fnearest}_N(\pm q) & = & \mathrm{float}_N(\pm i) & (\text{if } |i-q| = 0.5 \wedge i \text{ even})
\end{array}
$$

$\mathrm{feq}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return 0.
- Else if both z_1 and z_2 are zeroes, then return 1.
- Else if both z_1 and z_2 are the same value, then return 1.
- Else return 0.

$$
\begin{array}{lcl}
\mathrm{feq}_N(\pm\mathsf{nan}(n), z_2) & = & 0 \\
\mathrm{feq}_N(z_1, \pm\mathsf{nan}(n)) & = & 0 \\
\mathrm{feq}_N(\pm 0, \mp 0) & = & 1 \\
\mathrm{feq}_N(z_1, z_2) & = & \mathrm{bool}(z_1 = z_2)
\end{array}
$$

$\mathrm{fne}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return 1.
- Else if both z_1 and z_2 are zeroes, then return 0.
- Else if both z_1 and z_2 are the same value, then return 0.
- Else return 1.

$$
\begin{array}{lcl}
\mathrm{fne}_N(\pm\mathsf{nan}(n), z_2) & = & 1 \\
\mathrm{fne}_N(z_1, \pm\mathsf{nan}(n)) & = & 1 \\
\mathrm{fne}_N(\pm 0, \mp 0) & = & 0 \\
\mathrm{fne}_N(z_1, z_2) & = & \mathrm{bool}(z_1 \neq z_2)
\end{array}
$$

$\mathrm{flt}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return 0.
- Else if z_1 and z_2 are the same value, then return 0.
- Else if z_1 is positive infinity, then return 0.
- Else if z_1 is negative infinity, then return 1.
- Else if z_2 is positive infinity, then return 1.
- Else if z_2 is negative infinity, then return 0.
- Else if both z_1 and z_2 are zeroes, then return 0.
- Else if z_1 is smaller than z_2, then return 1.
- Else return 0.

$$
\begin{array}{lcl}
\mathrm{flt}_N(\pm\mathsf{nan}(n), z_2) & = & 0 \\
\mathrm{flt}_N(z_1, \pm\mathsf{nan}(n)) & = & 0 \\
\mathrm{flt}_N(z, z) & = & 0 \\
\mathrm{flt}_N(+\infty, z_2) & = & 0 \\
\mathrm{flt}_N(-\infty, z_2) & = & 1 \\
\mathrm{flt}_N(z_1, +\infty) & = & 1 \\
\mathrm{flt}_N(z_1, -\infty) & = & 0 \\
\mathrm{flt}_N(\pm 0, \mp 0) & = & 0 \\
\mathrm{flt}_N(z_1, z_2) & = & \mathrm{bool}(z_1 < z_2)
\end{array}
$$

$\mathrm{fgt}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return 0.
- Else if z_1 and z_2 are the same value, then return 0.
- Else if z_1 is positive infinity, then return 1.
- Else if z_1 is negative infinity, then return 0.
- Else if z_2 is positive infinity, then return 0.
- Else if z_2 is negative infinity, then return 1.
- Else if both z_1 and z_2 are zeroes, then return 0.
- Else if z_1 is larger than z_2, then return 1.
- Else return 0.

$$
\begin{array}{lcl}
\mathrm{fgt}_N(\pm\mathsf{nan}(n), z_2) & = & 0 \\
\mathrm{fgt}_N(z_1, \pm\mathsf{nan}(n)) & = & 0 \\
\mathrm{fgt}_N(z, z) & = & 0 \\
\mathrm{fgt}_N(+\infty, z_2) & = & 1 \\
\mathrm{fgt}_N(-\infty, z_2) & = & 0 \\
\mathrm{fgt}_N(z_1, +\infty) & = & 0 \\
\mathrm{fgt}_N(z_1, -\infty) & = & 1 \\
\mathrm{fgt}_N(\pm 0, \mp 0) & = & 0 \\
\mathrm{fgt}_N(z_1, z_2) & = & \mathrm{bool}(z_1 > z_2)
\end{array}
$$

$\mathrm{fle}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return 0.
- Else if z_1 and z_2 are the same value, then return 1.
- Else if z_1 is positive infinity, then return 0.
- Else if z_1 is negative infinity, then return 1.
- Else if z_2 is positive infinity, then return 1.
- Else if z_2 is negative infinity, then return 0.
- Else if both z_1 and z_2 are zeroes, then return 1.
- Else if z_1 is smaller than or equal to z_2, then return 1.
- Else return 0.

$$
\begin{array}{lcl}
\mathrm{fle}_N(\pm\mathsf{nan}(n), z_2) & = & 0 \\
\mathrm{fle}_N(z_1, \pm\mathsf{nan}(n)) & = & 0 \\
\mathrm{fle}_N(z, z) & = & 1 \\
\mathrm{fle}_N(+\infty, z_2) & = & 0 \\
\mathrm{fle}_N(-\infty, z_2) & = & 1 \\
\mathrm{fle}_N(z_1, +\infty) & = & 1 \\
\mathrm{fle}_N(z_1, -\infty) & = & 0 \\
\mathrm{fle}_N(\pm 0, \mp 0) & = & 1 \\
\mathrm{fle}_N(z_1, z_2) & = & \mathrm{bool}(z_1 \leq z_2)
\end{array}
$$

$\mathrm{fge}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return 0.
- Else if z_1 and z_2 are the same value, then return 1.
- Else if z_1 is positive infinity, then return 1.
- Else if z_1 is negative infinity, then return 0.

- Else if z_2 is positive infinity, then return 0.
- Else if z_2 is negative infinity, then return 1.
- Else if both z_1 and z_2 are zeroes, then return 1.
- Else if z_1 is larger than or equal to z_2, then return 1.
- Else return 0.

$$\begin{array}{lcl}
\mathrm{fge}_N(\pm\mathsf{nan}(n), z_2) & = & 0 \\
\mathrm{fge}_N(z_1, \pm\mathsf{nan}(n)) & = & 0 \\
\mathrm{fge}_N(z, z) & = & 1 \\
\mathrm{fge}_N(+\infty, z_2) & = & 1 \\
\mathrm{fge}_N(-\infty, z_2) & = & 0 \\
\mathrm{fge}_N(z_1, +\infty) & = & 0 \\
\mathrm{fge}_N(z_1, -\infty) & = & 1 \\
\mathrm{fge}_N(\pm 0, \mp 0) & = & 1 \\
\mathrm{fge}_N(z_1, z_2) & = & \mathrm{bool}(z_1 \geq z_2)
\end{array}$$

$\mathrm{fpmin}_N(z_1, z_2)$

- If z_2 is less than z_1 then return z_2.
- Else return z_1.

$$\begin{array}{lcll}
\mathrm{fpmin}_N(z_1, z_2) & = & z_2 & (\text{if } \mathrm{flt}_N(z_2, z_1) = 1) \\
\mathrm{fpmin}_N(z_1, z_2) & = & z_1 & (\text{otherwise})
\end{array}$$

$\mathrm{fpmax}_N(z_1, z_2)$

- If z_1 is less than z_2 then return z_2.
- Else return z_1.

$$\begin{array}{lcll}
\mathrm{fpmax}_N(z_1, z_2) & = & z_2 & (\text{if } \mathrm{flt}_N(z_1, z_2) = 1) \\
\mathrm{fpmax}_N(z_1, z_2) & = & z_1 & (\text{otherwise})
\end{array}$$

4.3.4 Conversions

$\mathrm{extend}^{\mathsf{u}}{}_{M,N}(i)$

- Return i.

$$\mathrm{extend}^{\mathsf{u}}{}_{M,N}(i) \quad = \quad i$$

Note

In the abstract syntax, unsigned extension just reinterprets the same value.

$\mathrm{extend}^{\mathsf{s}}{}_{M,N}(i)$

- Let j be the signed interpretation of i of size M.
- Return the two's complement of j relative to size N.

$$\mathrm{extend}^{\mathsf{s}}{}_{M,N}(i) \quad = \quad \mathrm{signed}_N^{-1}(\mathrm{signed}_M(i))$$

$\mathrm{wrap}_{M,N}(i)$

- Return i modulo 2^N.

$$\mathrm{wrap}_{M,N}(i) \quad = \quad i \bmod 2^N$$

$\text{trunc}^{\text{u}}_{M,N}(z)$

- If z is a NaN, then the result is undefined.
- Else if z is an infinity, then the result is undefined.
- Else if z is a number and $\text{trunc}(z)$ is a value within range of the target type, then return that value.
- Else the result is undefined.

$$
\begin{array}{lcll}
\text{trunc}^{\text{u}}_{M,N}(\pm\mathsf{nan}(n)) & = & \{\} & \\
\text{trunc}^{\text{u}}_{M,N}(\pm\infty) & = & \{\} & \\
\text{trunc}^{\text{u}}_{M,N}(\pm q) & = & \text{trunc}(\pm q) & (\text{if } -1 < \text{trunc}(\pm q) < 2^N) \\
\text{trunc}^{\text{u}}_{M,N}(\pm q) & = & \{\} & (\text{otherwise})
\end{array}
$$

Note

This operator is partial. It is not defined for NaNs, infinities, or values for which the result is out of range.

$\text{trunc}^{\text{s}}_{M,N}(z)$

- If z is a NaN, then the result is undefined.
- Else if z is an infinity, then the result is undefined.
- If z is a number and $\text{trunc}(z)$ is a value within range of the target type, then return that value.
- Else the result is undefined.

$$
\begin{array}{lcll}
\text{trunc}^{\text{s}}_{M,N}(\pm\mathsf{nan}(n)) & = & \{\} & \\
\text{trunc}^{\text{s}}_{M,N}(\pm\infty) & = & \{\} & \\
\text{trunc}^{\text{s}}_{M,N}(\pm q) & = & \text{trunc}(\pm q) & (\text{if } -2^{N-1} - 1 < \text{trunc}(\pm q) < 2^{N-1}) \\
\text{trunc}^{\text{s}}_{M,N}(\pm q) & = & \{\} & (\text{otherwise})
\end{array}
$$

Note

This operator is partial. It is not defined for NaNs, infinities, or values for which the result is out of range.

$\text{trunc_sat_u}_{M,N}(z)$

- If z is a NaN, then return 0.
- Else if z is negative infinity, then return 0.
- Else if z is positive infinity, then return $2^N - 1$.
- Else, return $\text{sat_u}_N(\text{trunc}(z))$.

$$
\begin{array}{lcl}
\text{trunc_sat_u}_{M,N}(\pm\mathsf{nan}(n)) & = & 0 \\
\text{trunc_sat_u}_{M,N}(-\infty) & = & 0 \\
\text{trunc_sat_u}_{M,N}(+\infty) & = & 2^N - 1 \\
\text{trunc_sat_u}_{M,N}(z) & = & \text{sat_u}_N(\text{trunc}(z))
\end{array}
$$

$\text{trunc_sat_s}_{M,N}(z)$

- If z is a NaN, then return 0.
- Else if z is negative infinity, then return -2^{N-1}.
- Else if z is positive infinity, then return $2^{N-1} - 1$.
- Else, return the value whose signed interpretation is $\text{sat_s}_N(\text{trunc}(z))$.

$$
\begin{array}{lcl}
\text{trunc_sat_s}_{M,N}(\pm\mathsf{nan}(n)) & = & 0 \\
\text{trunc_sat_s}_{M,N}(-\infty) & = & -2^{N-1} \\
\text{trunc_sat_s}_{M,N}(+\infty) & = & 2^{N-1}-1 \\
\text{trunc_sat_s}_{M,N}(z) & = & \text{signed}_N^{-1}(\text{sat_s}_N(\text{trunc}(z)))
\end{array}
$$

$\text{promote}_{M,N}(z)$

- If z is a canonical NaN, then return an element of $\text{nans}_N\{\}$ (i.e., a canonical NaN of size N).
- Else if z is a NaN, then return an element of $\text{nans}_N\{\pm\mathsf{nan}(1)\}$ (i.e., any arithmetic NaN of size N).
- Else, return z.

$$
\begin{array}{lcll}
\text{promote}_{M,N}(\pm\mathsf{nan}(n)) & = & \text{nans}_N\{\} & (\text{if } n = \text{canon}_N) \\
\text{promote}_{M,N}(\pm\mathsf{nan}(n)) & = & \text{nans}_N\{+\mathsf{nan}(1)\} & (\text{otherwise}) \\
\text{promote}_{M,N}(z) & = & z &
\end{array}
$$

$\text{demote}_{M,N}(z)$

- If z is a canonical NaN, then return an element of $\text{nans}_N\{\}$ (i.e., a canonical NaN of size N).
- Else if z is a NaN, then return an element of $\text{nans}_N\{\pm\mathsf{nan}(1)\}$ (i.e., any NaN of size N).
- Else if z is an infinity, then return that infinity.
- Else if z is a zero, then return that zero.
- Else, return $\text{float}_N(z)$.

$$
\begin{array}{lcll}
\text{demote}_{M,N}(\pm\mathsf{nan}(n)) & = & \text{nans}_N\{\} & (\text{if } n = \text{canon}_N) \\
\text{demote}_{M,N}(\pm\mathsf{nan}(n)) & = & \text{nans}_N\{+\mathsf{nan}(1)\} & (\text{otherwise}) \\
\text{demote}_{M,N}(\pm\infty) & = & \pm\infty & \\
\text{demote}_{M,N}(\pm 0) & = & \pm 0 & \\
\text{demote}_{M,N}(\pm q) & = & \text{float}_N(\pm q) &
\end{array}
$$

$\text{convert}^{\mathsf{u}}{}_{M,N}(i)$

- Return $\text{float}_N(i)$.

$$\text{convert}^{\mathsf{u}}{}_{M,N}(i) = \text{float}_N(i)$$

$\text{convert}^{\mathsf{s}}{}_{M,N}(i)$

- Let j be the signed interpretation of i.
- Return $\text{float}_N(j)$.

$$\text{convert}^{\mathsf{s}}{}_{M,N}(i) = \text{float}_N(\text{signed}_M(i))$$

$\text{reinterpret}_{t_1,t_2}(c)$

- Let d^* be the bit sequence $\text{bits}_{t_1}(c)$.
- Return the constant c' for which $\text{bits}_{t_2}(c') = d^*$.

$$\text{reinterpret}_{t_1,t_2}(c) = \text{bits}_{t_2}^{-1}(\text{bits}_{t_1}(c))$$

$\text{narrow}^{\mathsf{s}}{}_{M,N}(i)$

- Let j be the signed interpretation of i of size M.
- Return the value whose signed interpretation is $\text{sat_s}_N(j)$.

$$\text{narrow}^{\mathsf{s}}{}_{M,N}(i) = \text{signed}_N^{-1}(\text{sat_s}_N(\text{signed}_M(i)))$$

$\text{narrow}^{\text{u}}_{M,N}(i)$

- Let j be the signed interpretation of i of size M.
- Return $\text{sat_u}_N(j)$.

$$\text{narrow}^{\text{u}}_{M,N}(i) \quad = \quad \text{sat_u}_N(\text{signed}_M(i))$$

4.3.5 Vector Operations

Most vector operations are performed by applying numeric operations lanewise. However, some operators consider multiple lanes at once.

$\text{ivbitmask}_N(i^m)$

1. For each i_k in i^m, let b_k be the result of computing $\text{ilt_s}_N(i, 0)$.
2. Let b^m be the concatenation of all b_k.
3. Return the result of computing $\text{ibits}^{-1}_{32}((0)^{32-m}\ b^m)$.

$$\text{ivbitmask}_N(i^m) \quad = \quad \text{ibits}^{-1}_{32}((0)^{32-m}\ \text{ilt_s}_N(i, 0)^m)$$

$\text{ivswizzle}(i^n, j^n)$

1. For each j_k in j^n, let r_k be the value $\text{ivswizzle_lane}(i^n, j_k)$.
2. Let r^n be the concatenation of all r_k.
3. Return r^n.

$$\text{ivswizzle}(i^n, j^n) \quad = \quad \text{ivswizzle_lane}(i^n, j)^n$$

where:

$$\begin{array}{lcll} \text{ivswizzle_lane}(i^n, j) & = & i^n[j] & (\text{if } j < n) \\ \text{ivswizzle_lane}(i^n, j) & = & 0 & (\text{otherwise}) \end{array}$$

$\text{ivshuffle}(j^n, i^n_1, i^n_2)$

1. Let i^* ne the concatenation of i^n_1 and i^n_2.
2. For each j_k in j^n, let r_k be $i^*[j_k]$.
3. Let r^n be the concatenation of all r_k.
4. Return r^n.

$$\text{ivshuffle}(j^n, i^n_1, i^n_2) \quad = \quad ((i^n_1\ i^n_2)[j])^n \quad (\text{if } (j < 2 \cdot n)^n)$$

$\text{ivadd_pairwise}_N(i^{2m})$

1. Let $(i_1\ i_2)^m$ be i^{2m}, decomposed into pairwise elements.
2. For each i_{1k} in i^m_1 and corresponding i_{2k} in i^m_2, let r_k be $\text{iadd}_N(i_{1k}, i_{2k})$.
3. Let r^m be the concatenation of all r_k.
4. Return r^m.

$$\text{ivadd_pairwise}_N(i^{2m}) \quad = \quad (\text{iadd}_N(i_1, i_2))^m \quad (\text{if } i^{2m} = (i_1\ i_2)^m)$$

$\text{ivmul}_N(i_1^m, i_2^m)$

1. For each i_{1k} in i_1^m and corresponding i_{2k} in i_2^m, let r_k be $\text{imul}_N(i_{1k}, i_{2k})$.
2. Let r^m be the concatenation of all r_k.
3. Return r^m.

$$\text{ivmul}_N(i_1^m, i_2^m) \quad = \quad (\text{imul}_N(i_1, i_2))^m$$

$\text{ivdot}_N(i_1^{2m}, i_2^{2m})$

1. For each i_{1k} in i_1^{2m} and corresponding i_{2k} in i_2^{2m}, let j_k be $\text{imul}_N(i_{1k}, i_{2k})$.
2. Let j^{2m} be the concatenation of all j_k.
3. Let $(j_1\ j_2)^m$ be j^{2m}, decomposed into pairwise elements.
4. For each i_{1k} in i_1^m and corresponding i_{2k} in i_2^m, let r_k be $\text{iadd}_N(i_{1k}, i_{2k})$.
5. Let r^m be the concatenation of all r_k.
6. Return r^m.

$$\text{ivdot}_N(i_1^{2m}, i_2^{2m}) \quad = \quad (\text{iadd}_N(j_1, j_2))^m \quad (\text{if } (\text{imul}_N(i_1, i_2))^{2m} = (j_1\ j_2)^m)$$

$\text{ivdotsat}_N(i_1^m, i_2^m)$

1. For each i_{1k} in i_1^{2m} and corresponding i_{2k} in i_2^{2m}, let j_k be $\text{imul}_N(i_{1k}, i_{2k})$.
2. Let j^{2m} be the concatenation of all j_k.
3. Let $(j_1\ j_2)^m$ be j^{2m}, decomposed into pairwise elements.
4. For each i_{1k} in i_1^m and corresponding i_{2k} in i_2^m, let r_k be $\text{iadd_sat_s}_N(i_{1k}, i_{2k})$.
5. Let r^m be the concatenation of all r_k.
6. Return r^m.

$$\text{ivdotsat}_N(i_1^{2m}, i_2^{2m}) \quad = \quad (\text{iadd_sat_s}_N(j_1, j_2))^m \quad (\text{if } (\text{imul}_N(i_1, i_2))^{2m} = (j_1\ j_2)^m)$$

The previous operators are lifted to operators on arguments of vector type by wrapping them in corresponding lane projections and injections and intermediate extension operations:

$\mathit{vextunop}_{sh_1, sh_2}(c)$

$$\begin{array}{rcll} \text{extadd_pairwise_}sx_{\text{i}N_1\text{x}M_1,\text{i}N_2\text{x}M_2}(c) & = & \text{lanes}^{-1}_{\text{i}N_2\text{x}M_2}(j^*) & (\text{if } i^* = \text{lanes}_{\text{i}N_1\text{x}M_1}(c) \\ & & & \quad \wedge\ i'^* = \text{extend}^{sx}_{N_1,N_2}(i)^* \\ & & & \quad \wedge\ j^* = \text{ivadd_pairwise}_{N_2}(i'^*) \end{array}$$

$\mathit{vextbinop}_{sh_1, sh_2}(c_1, c_2)$

$$\begin{array}{rcll} \mathit{vextbinop}_{\text{i}N_1\text{x}M_1,\text{i}N_2\text{x}M_2}(c_1, c_2) & = & \text{lanes}^{-1}_{\text{i}N_2\text{x}M_2}(j^*) & (\text{if } i_1^* = \text{lanes}_{\text{i}N_1\text{x}M_1}(c_1)[h : k] \\ & & & \quad \wedge\ i_2^* = \text{lanes}_{\text{i}N_1\text{x}M_1}(c_2)[h : k] \\ & & & \quad \wedge\ i_1'^* = \text{extend}^{sx_1}_{N_1,N_2}(i_1)^* \\ & & & \quad \wedge\ i_2'^* = \text{extend}^{sx_2}_{N_1,N_2}(i_2)^* \\ & & & \quad \wedge\ j^* = f_{N_2}(i_1'^*, i_2'^*) \end{array}$$

where f, sx_1, sx_2, h, and k are instantiated as follows, depending on the operator:

vextbinop	f	sx_1	sx_2	h	k
extmul_low_*sx*	ivmul	*sx*	*sx*	0	M_2
extmul_high_*sx*	ivmul	*sx*	*sx*	M_2	M_2
dot_s	ivdot	s	s	0	M_1
relaxed_dot_s	ivrelaxed_dot	s	relaxed(R_{idot})[s, u]	0	M_1

Note

Relaxed operations and the paramater R_{idot} are introduced below.

$\mathit{vextternop}_{sh_1,sh_2}(c_1, c_2, c_3)$

$$\begin{array}{lcll} \mathsf{relaxed_dot_add_s}_{\mathsf{i}N_1\times M_1,\mathsf{i}N_2\times M_2}(c_1,c_2,c_3) & = & c & (\text{if } N = \mathrm{relaxed}(R_{\text{idot_add}})[2\cdot N_1, N_2] \\ & & & \wedge\ i_1^* = \mathrm{lanes}_{\mathsf{i}N_1\times M_1}(c_1) \\ & & & \wedge\ i_2^* = \mathrm{lanes}_{\mathsf{i}N_1\times M_1}(c_2) \\ & & & \wedge\ {i'_1}^* = \mathrm{extend}^{\mathsf{s}}_{N_1,N}(i_1)^* \\ & & & \wedge\ {i'_2}^* = \mathrm{extend}^{\mathrm{relaxed}(R_{\text{idot}})[\mathsf{s},\mathsf{u}]}_{N_1,N}(i_2)^* \\ & & & \wedge\ j^* = \mathrm{ivrelaxed_dot}_N({i'_1}^*, {i'_2}^*) \\ & & & \wedge\ {j'}^* = \mathrm{extend}^{\mathsf{s}}_{N,N_2}(j)^* \\ & & & \wedge\ c' = \mathrm{lanes}^{-1}_{\mathsf{i}N_2\times M_2}(\mathrm{ivadd_pairwise}_{N_2}({j'}^*)) \\ & & & \wedge\ c \in \mathsf{add}_{\mathsf{i}N_2\times M_2}(c', c_3)) \end{array}$$

Note

Relaxed operations and the parameters R_{idot} and $R_{\text{idot_add}}$ are introduced below.

$\mathsf{narrow_}\mathit{sx}_{sh_1,sh_2}(c_1, c_2)$

$$\begin{array}{lcll} \mathsf{narrow_}\mathit{sx}_{\mathsf{i}N_1\times M_1,\mathsf{i}N_2\times M_2}(c_1,c_2) & = & \mathrm{lanes}^{-1}_{\mathsf{i}N_2\times M_2}(j^*) & (\text{if } i_1^* = \mathrm{lanes}_{\mathsf{i}N_1\times M_1}(c_1) \\ & & & \wedge\ i_2^* = \mathrm{lanes}_{\mathsf{i}N_1\times M_1}(c_2) \\ & & & \wedge\ {i'_1}^* = \mathrm{narrow}^{\mathit{sx}}_{N_1,N_2}(i_1)^* \\ & & & \wedge\ {i'_2}^* = \mathrm{narrow}^{\mathit{sx}}_{N_1,N_2}(i_2)^* \\ & & & \wedge\ j^* = {i'_1}^* \oplus {i'_1}^*) \end{array}$$

$\mathit{vcvtop_half}^?\mathit{_zero}^?_{sh_1,sh_2}(i)$

$$\begin{array}{lcll} \mathit{vcvtop_half}^?\mathit{_zero}^?_{t_1\times M_1,t_2\times M_2}(i) & = & j & (\text{if } \mathit{condition} \\ & & & \wedge\ c^* = \mathrm{lanes}_{t_1\times M_1}(i)[h:k] \\ & & & \wedge\ {c'}^{**} = \times(\mathit{vcvtop}_{|t_1|,|t_2|}(c)^* \oplus (0)^n) \\ & & & \wedge\ j \in \mathrm{lanes}^{-1}_{t_2\times M_2}({c'}^*)^* \end{array}$$

where h, k, n, and $\mathit{condition}$ are instantiated as follows, depending on the operator:

$\mathit{half}^?$	$\mathit{zero}^?$	h	k	n	$\mathit{condition}$
ϵ	ϵ	0	M_1	0	$(M_1 = M_2)$
low	ϵ	0	M_2	0	$(M_1 = 2\cdot M_2)$
high	ϵ	M_2	M_2	0	$(M_1 = 2\cdot M_2)$
ϵ	zero	0	M_1	M_1	$(2\cdot M_1 = M_2)$

while $\times\{x^*\}^N$ transforms a sequence of N sets of non-deterministic values into a set of non-deterministic sequences of N values by computing the set product:

$$\times(S_1 \dots S_N) \quad = \quad \{x_1 \dots x_N \mid x_1 \in S_1 \wedge \dots \wedge x_N \in S_N\}$$

4.3.6 Relaxed Operations

The result of *relaxed* operators are *implementation-dependent*, because the set of possible results may depend on properties of the host environment, such as its hardware. Technically, their behaviour is controlled by a set of *global parameters* to the semantics that an implementation can instantiate in different ways. These choices are fixed, that is, parameters are constant during the execution of any given program.

Every such parameter is an index into a sequence of possible sets of results and must be instantiated to a defined index. In the deterministic profile, every parameter is prescribed to be 0. This behaviour is expressed by the following auxiliary function, where R is a global parameter selecting one of the allowed outcomes:

$$\begin{array}{ll} {}^{[!\text{DET}]} & \text{relaxed}(R)[A_0, \dots, A_n] = A_R \\ & \text{relaxed}(R)[A_0, \dots, A_n] = A_0 \end{array}$$

Note

Each parameter can be thought of as inducing a family of operations that is fixed to one particular choice by an implementation. The fixed operation itself can still be non-deterministic or partial.

Implementations are expexted to either choose the behaviour that is the most efficient on the underlying hardware, or the behaviour of the deterministic profile.

$\text{frelaxed_madd}_N(z_1, z_2, z_3)$

The implementation-specific behaviour of this operation is determined by the global parameter $R_{\text{fmadd}} \in \{0, 1\}$.

- Return $\text{relaxed}(R_{\text{fmadd}})[\text{fadd}_N(\text{fmul}_N(z_1, z_2), z_3), \text{fma}_N(z_1, z_2, z_3)]$.

$$\text{frelaxed_madd}_N(z_1, z_2, z_3) = \text{relaxed}(R_{\text{fmadd}})[\text{fadd}_N(\text{fmul}_N(z_1, z_2), z_3), \text{fma}_N(z_1, z_2, z_3)]$$

Note

Relaxed multiply-add allows for fused or unfused results, which leads to implementation-dependent rounding behaviour. In the deterministic profile, the unfused behaviour is used.

$\text{frelaxed_nmadd}_N(z_1, z_2, z_3)$

- Return $\text{frelaxed_madd}(-z_1, z_2, z_3)$.

$$\text{frelaxed_nmadd}_N(z_1, z_2, z_3) = \text{frelaxed_madd}_N(-z_1, z_2, z_3)$$

Note

This operation is implementation-dependent because frelaxed_madd is implementation-dependent.

$\text{frelaxed_min}_N(z_1, z_2)$

The implementation-specific behaviour of this operation is determined by the global parameter $R_{\text{fmin}} \in \{0, 1, 2, 3\}$.

- If z_1 is a NaN, then return $\text{relaxed}(R_{\text{fmin}})[\text{fmin}_N(z_1, z_2), \mathsf{nan}(n), z_2, z_2]$.
- If z_2 is a NaN, then return $\text{relaxed}(R_{\text{fmin}})[\text{fmin}_N(z_1, z_2), z_1, \mathsf{nan}(n), z_1]$.
- If both z_1 and z_2 are zeroes of opposite sign, then return $\text{relaxed}(R_{\text{fmin}})[\text{fmin}_N(z_1, z_2)$, pm 0, mp 0, -0]`.
- Return $\text{fmin}_N(z_1, z_2)$.

$$\begin{array}{lcll} \text{frelaxed_min}_N(\pm\mathsf{nan}(n), z_2) & = & \text{relaxed}(R_{\text{fmin}})[\text{fmin}_N(\pm\mathsf{nan}(n), z_2), \mathsf{nan}(n), z_2, z_2] & \\ \text{frelaxed_min}_N(z_1, \pm\mathsf{nan}(n)) & = & \text{relaxed}(R_{\text{fmin}})[\text{fmin}_N(z_1, \pm\mathsf{nan}(n)), z_1, \mathsf{nan}(n), z_1] & \\ \text{frelaxed_min}_N(\pm 0, \mp 0) & = & \text{relaxed}(R_{\text{fmin}})[\text{fmin}_N(\pm 0, \mp 0), \pm 0, \mp 0, -0] & \\ \text{frelaxed_min}_N(z_1, z_2) & = & \text{fmin}_N(z_1, z_2) & (\text{otherwise}) \end{array}$$

Note

Relaxed minimum is implementation-dependent for NaNs and for zeroes with different signs. In the deterministic profile, it behaves like regular fmin.

$\text{frelaxed_max}_N(z_1, z_2)$

The implementation-specific behaviour of this operation is determined by the global parameter $R_{\text{fmax}} \in \{0, 1, 2, 3\}$.

- If z_1 is a NaN, then return $\text{relaxed}(R_{\text{fmax}})[\text{fmax}_N(z_1, z_2), \text{nan}(n), z_2, z_2]$.
- If z_2 is a NaN, then return $\text{relaxed}(R_{\text{fmax}})[\text{fmax}_N(z_1, z_2), z_1, \text{nan}(n), z_1]$.
- If both z_1 and z_2 are zeroes of opposite sign, then return $\text{relaxed}(R_{\text{fmax}})[\text{fmax}_N(z_1, z_2)$, pm 0, mp 0, +0]`.
- Return $\text{fmax}_N(z_1, z_2)$.

$$
\begin{array}{lcll}
\text{frelaxed_max}_N(\pm\text{nan}(n), z_2) & = & \text{relaxed}(R_{\text{fmax}})[\text{fmax}_N(\pm\text{nan}(n), z_2), \text{nan}(n), z_2, z_2] & \\
\text{frelaxed_max}_N(z_1, \pm\text{nan}(n)) & = & \text{relaxed}(R_{\text{fmax}})[\text{fmax}_N(z_1, \pm\text{nan}(n)), z_1, \text{nan}(n), z_1] & \\
\text{frelaxed_max}_N(\pm 0, \mp 0) & = & \text{relaxed}(R_{\text{fmax}})[\text{fmax}_N(\pm 0, \mp 0), \pm 0, \mp 0, +0] & \\
\text{frelaxed_max}_N(z_1, z_2) & = & \text{fmax}_N(z_1, z_2) & \text{(otherwise)}
\end{array}
$$

Note

Relaxed maximum is implementation-dependent for NaNs and for zeroes with different signs. In the deterministic profile, it behaves like regular fmax.

$\text{irelaxed_q15mulr_s}_N(i_1, i_2)$

The implementation-specific behaviour of this operation is determined by the global parameter $R_{\text{iq15mulr}} \in \{0, 1\}$.

- If both i_1 and i_2 equal $(\text{signed}_N^{-1}(-2^{N-1})$, then return $\text{relaxed}(R_{\text{iq15mulr}})[2^{N-1} - 1, \text{signed}_N^{-1}(-2^{N-1})]$.
- Return $\text{iq15mulrsat_s}(i_1, i_2)$

$$
\begin{array}{lcl}
\text{irelaxed_q15mulr_s}_N(\text{signed}_N^{-1}(-2^{N-1}), \text{signed}_N^{-1}(-2^{N-1})) & = & \text{relaxed}(R_{\text{iq15mulr}})[2^{N-1} - 1, \text{signed}_N^{-1}(-2^{N-1})] \\
\text{irelaxed_q15mulr_s}_N(i_1, i_2) & = & \text{iq15mulrsat_s}(i_1, i_2)
\end{array}
$$

Note

Relaxed Q15 multiplication is implementation-dependent when the result overflows. In the deterministic profile, it behaves like regular iq15mulrsat_s.

$\text{relaxed_trunc}_{M,N}^{u}(z)$

The implementation-specific behaviour of this operation is determined by the global parameter $R_{\text{trunc_u}} \in \{0, 1\}$.

- If z is normal or subnormal and $\text{trunc}(z)$ is non-negative and less than 2^N, then return $\text{trunc}^{\text{u}}{}_{M,N}(z)$.
- Else, return $\text{relaxed}(R_{\text{trunc_u}})[\text{trunc_sat_u}_{M,N}(z), \mathbf{R}]$.

$$
\begin{array}{lcll}
\text{relaxed_trunc}_{M,N}^{u}(\pm q) & = & \text{trunc}^{\text{u}}{}_{M,N}(\pm q) & (\text{if } 0 \leq \text{trunc}(\pm q) < 2^N) \\
\text{relaxed_trunc}_{M,N}^{u}(z) & = & \text{relaxed}(R_{\text{trunc_u}})[\text{trunc_sat_u}_{M,N}(z), \mathbf{R}] & \text{(otherwise)}
\end{array}
$$

Note

Relaxed unsigned truncation is non-deterministic for NaNs and out-of-range values. In the deterministic profile, it behaves like regular trunc_sat_u.

$\text{relaxed_trunc}^s_{M,N}(z)$

The implementation-specific behaviour of this operation is determined by the global parameter $R_{\text{trunc_s}} \in \{0, 1\}$.

- If z is normal or subnormal and $\text{trunc}(z)$ is greater than or equal to -2^{N-1} and less than 2^{N-1}, then return $\text{trunc}^s_{M,N}(z)$.
- Else, return $\text{relaxed}(R_{\text{trunc_s}})[\text{trunc_sat_s}_{M,N}(z), \mathbf{R}]$.

$$\begin{array}{lcll} \text{relaxed_trunc}^s_{M,N}(\pm q) & = & \text{trunc}^s_{M,N}(\pm q) & (\text{if } -2^{N-1} \leq \text{trunc}(\pm q) < 2^{N-1}) \\ \text{relaxed_trunc}^s_{M,N}(z) & = & \text{relaxed}(R_{\text{trunc_s}})[\text{trunc_sat_s}_{M,N}(z), \mathbf{R}] & (\text{otherwise}) \end{array}$$

Note

Relaxed signed truncation is non-deterministic for NaNs and out-of-range values. In the deterministic profile, it behaves like regular trunc_sat_s.

$\text{ivrelaxed_swizzle}(i^n, j^n)$

The implementation-specific behaviour of this operation is determined by the global parameter $R_{\text{swizzle}} \in \{0, 1\}$.

- For each j_k in j^n, let r_k be the value $\text{ivrelaxed_swizzle_lane}(i^n, j_k)$.
- Let r^n be the concatenation of all r_k.
- Return r^n.

$$\text{ivrelaxed_swizzle}(i^n, j^n) = \text{ivrelaxed_swizzle_lane}(i^n, j)^n$$

where:

$$\begin{array}{lcll} \text{ivrelaxed_swizzle_lane}(i^n, j) & = & i[j] & (\text{if } j < 16) \\ \text{ivrelaxed_swizzle_lane}(i^n, j) & = & 0 & (\text{if } \text{signed}_8(j) < 0) \\ \text{ivrelaxed_swizzle_lane}(i^n, j) & = & \text{relaxed}(R_{\text{swizzle}})[0, i^n[j \bmod n]] & (\text{otherwise}) \end{array}$$

Note

Relaxed swizzle is implementation-dependent if the signed interpretation of any of the 8-bit indices in j^n is larger than or equal to 16. In the deterministic profile, it behaves like regular ivswizzle.

$\text{ivrelaxed_dot}_N(i_1^{2m}, i_2^{2m})$

The implementation-specific behaviour of this operation is determined by the global parameter $R_{\text{idot}} \in \{0, 1\}$. It also affects the behaviour of relaxed_dot specified above, and of relaxed_dot_add specified above, which additionally depends on the global parameter $R_{\text{idot_add}} \in \{0, 1\}$.

- Return $\text{relaxed}(R_{\text{idot}})[\text{ivdot}_N(i_1^{2m}, i_2^{2m}), \text{ivdotsat}_N(i_1^{2m}, i_2^{2m})]$.

$$\text{ivrelaxed_dot}_N(i_1^{2m}, i_2^{2m}) = \text{relaxed}(R_{\text{idot}})[\text{ivdot}_N(i_1^{2m}, i_2^{2m}), \text{ivdotsat}_N(i_1^{2m}, i_2^{2m})]$$

Note

Relaxed dot product is implementation-dependent when the second operand is negative in a signed intepretation. In the deterministic profile, it behaves like signed dot product.

$\text{irelaxed_laneselect}_N(i_1, i_2, i_3)$

The implementation-specific behaviour of this operation is determined by the global parameter $R_{\text{laneselect}} \in \{0, 1\}$.

- If i_3 is smaller than 2^{N-1}, then let i_3' be the value 0, otherwise $2^N - 1$.
- Let i_3'' be $\text{relaxed}(R_{\text{laneselect}})[i_3, i_3']$.
- Return $\text{ibitselect}_N(i_1, i_2, i_3'')$.

$$\text{irelaxed_laneselect}_N(i_1, i_2, i_3) \quad = \quad \text{ibitselect}_N(i_1, i_2, \text{relaxed}(R_{\text{laneselect}})[i_3, \text{extend}^{\text{s}}{}_{1,N}(\text{ishr_u}_N(i_3, N-1))])$$

Note

Relaxed lane selection is non-deterministic when the mask mixes set and cleared bits, since the value of the high bit may or may not be expanded to all bits. In the deterministic profile, it behaves like ibitselect.

4.4 Types

Execution has to check and compare types in a few places, such as executing call_indirect or instantiating modules.

It is an invariant of the semantics that all types occurring during execution are closed.

Note

Runtime type checks generally involve types from multiple modules or types not defined by a module at all, such that any module-local type indices occurring inside them would not generally be meaningful.

4.4.1 Instantiation

Any form of type can be *instantiated* into a closed type inside a module instance by substituting each type index x occurring in it with the corresponding defined type $moduleinst.\text{types}[x]$.

$$\text{clos}_{moduleinst}(t) \quad = \quad t[:= moduleinst.\text{types}]$$

Note

This is the runtime equivalent to type closure, which is applied at validation time.

4.5 Values

4.5.1 Value Typing

For the purpose of checking argument values against the parameter types of exported functions, values are classified by value types. The following auxiliary typing rules specify this typing relation relative to a store S in which possibly referenced addresses live.

Numeric Values

The number value $(nt.\text{const}\ c)$ is valid with the number type nt.

$$\frac{}{s \vdash nt.\text{const}\ c : nt}$$

Vector Values

The vector value $(vt.\mathsf{const}\ c)$ is valid with the vector type vt.

$$\frac{}{s \vdash vt.\mathsf{const}\ c : vt}$$

Null References

The reference value $\mathsf{ref.null}$ is valid with the reference type $(\mathsf{ref\ null\ bot})$.

$$\frac{}{s \vdash \mathsf{ref.null} : \mathsf{ref\ null\ bot}}$$

Scalar References

The reference value $(\mathsf{ref.i31}\ i)$ is valid with the reference type $(\mathsf{ref\ i31})$.

$$\frac{}{s \vdash \mathsf{ref.i31}\ i : \mathsf{ref\ i31}}$$

Structure References

The reference value $(\mathsf{ref.struct}\ a)$ is valid with the reference type $(\mathsf{ref}\ dt)$ if:

- The structure instance $s.\mathsf{structs}[a]$ exists.
- The defined type $s.\mathsf{structs}[a].\mathsf{type}$ is of the form dt.

$$\frac{s.\mathsf{structs}[a].\mathsf{type} = dt}{s \vdash \mathsf{ref.struct}\ a : \mathsf{ref}\ dt}$$

Array References

The reference value $(\mathsf{ref.array}\ a)$ is valid with the reference type $(\mathsf{ref}\ dt)$ if:

- The array instance $s.\mathsf{arrays}[a]$ exists.
- The defined type $s.\mathsf{arrays}[a].\mathsf{type}$ is of the form dt.

$$\frac{s.\mathsf{arrays}[a].\mathsf{type} = dt}{s \vdash \mathsf{ref.array}\ a : \mathsf{ref}\ dt}$$

Exception References

The reference value $(\mathsf{ref.exn}\ a)$ is valid with the reference type $(\mathsf{ref\ exn})$ if:

- The exception instance $s.\mathsf{exns}[a]$ exists.

$$\frac{s.\mathsf{exns}[a] = exn}{s \vdash \mathsf{ref.exn}\ a : \mathsf{ref\ exn}}$$

Function References

The reference value $(\mathsf{ref.func}\ a)$ is valid with the reference type $(\mathsf{ref}\ dt)$ if:

- The function instance $s.\mathsf{funcs}[a]$ exists.
- The defined type $s.\mathsf{funcs}[a].\mathsf{type}$ is of the form dt.

$$\frac{s.\mathsf{funcs}[a].\mathsf{type} = dt}{s \vdash \mathsf{ref.func}\ a : \mathsf{ref}\ dt}$$

Host References

The reference value $(\mathsf{ref.host}\ a)$ is valid with the reference type $(\mathsf{ref\ any})$.

$$\frac{}{s \vdash \mathsf{ref.host}\ a : \mathsf{ref\ any}}$$

Note

A bare host reference is considered internalized.

External References

The reference value $(\mathsf{ref.extern}\ ref)$ is valid with the reference type $(\mathsf{ref\ extern})$ if:

- The reference value ref is valid with the reference type $(\mathsf{ref\ any})$.
- The reference value ref is not of the form $\mathsf{ref.null}$.

$$\frac{s \vdash ref : \mathsf{ref\ any} \qquad ref \neq \mathsf{ref.null}}{s \vdash \mathsf{ref.extern}\ ref : \mathsf{ref\ extern}}$$

Subsumption

The reference value ref is valid with the reference type rt if:

- The reference value ref is valid with the reference type rt'.
- Under the context $\{\mathsf{return}\ \epsilon\}$, the reference type rt is valid.
- The reference type rt' matches the reference type rt.

$$\frac{s \vdash ref : rt' \qquad \{\} \vdash rt : \mathsf{ok} \qquad \{\} \vdash rt' \leq rt}{s \vdash ref : rt}$$

4.5.2 External Typing

For the purpose of checking external address against imports, such values are classified by external types. The following auxiliary typing rules specify this typing relation relative to a store S in which the referenced instances live.

Functions

The external address $(\mathsf{func}\ a)$ is valid with the external type $(\mathsf{func}\ funcinst.\mathsf{type})$ if:

- The function instance $s.\mathsf{funcs}[a]$ exists.
- The function instance $s.\mathsf{funcs}[a]$ is of the form $funcinst$.

$$\frac{s.\mathsf{funcs}[a] = funcinst}{s \vdash \mathsf{func}\ a : \mathsf{func}\ funcinst.\mathsf{type}}$$

Tables

The external address $(\mathsf{table}\ a)$ is valid with the external type $(\mathsf{table}\ tableinst.\mathsf{type})$ if:

- The table instance $s.\mathsf{tables}[a]$ exists.
- The table instance $s.\mathsf{tables}[a]$ is of the form $tableinst$.

$$\frac{s.\mathsf{tables}[a] = tableinst}{s \vdash \mathsf{table}\ a : \mathsf{table}\ tableinst.\mathsf{type}}$$

Memories

The external address $(\mathsf{mem}\ a)$ is valid with the external type $(\mathsf{mem}\ meminst.\mathsf{type})$ if:

- The memory instance $s.\mathsf{mems}[a]$ exists.
- The memory instance $s.\mathsf{mems}[a]$ is of the form $meminst$.

$$\frac{s.\mathsf{mems}[a] = meminst}{s \vdash \mathsf{mem}\ a : \mathsf{mem}\ meminst.\mathsf{type}}$$

Globals

The external address $(\mathsf{global}\ a)$ is valid with the external type $(\mathsf{global}\ globalinst.\mathsf{type})$ if:

- The global instance $s.\mathsf{globals}[a]$ exists.
- The global instance $s.\mathsf{globals}[a]$ is of the form $globalinst$.

$$\frac{s.\mathsf{globals}[a] = globalinst}{s \vdash \mathsf{global}\ a : \mathsf{global}\ globalinst.\mathsf{type}}$$

Tags

The external address $(\mathsf{tag}\ a)$ is valid with the external type $(\mathsf{tag}\ taginst.\mathsf{type})$ if:

- The tag instance $s.\mathsf{tags}[a]$ exists.
- The tag instance $s.\mathsf{tags}[a]$ is of the form $taginst$.

$$\frac{s.\mathsf{tags}[a] = taginst}{s \vdash \mathsf{tag}\ a : \mathsf{tag}\ taginst.\mathsf{type}}$$

Subsumption

The external address $externaddr$ is valid with the external type xt if:

- The external address $externaddr$ is valid with the external type xt'.
- Under the context $\{\mathsf{return}\ \epsilon\}$, the external type xt is valid.
- The external type xt' matches the external type xt.

$$\frac{s \vdash externaddr : xt' \qquad \{\} \vdash xt : \mathsf{ok} \qquad \{\} \vdash xt' \leq xt}{s \vdash externaddr : xt}$$

4.6 Instructions

WebAssembly computation is performed by executing individual instructions.

4.6.1 Parametric Instructions

nop

1. Do nothing.

$$\mathsf{nop} \quad \hookrightarrow \quad \epsilon$$

unreachable

1. Trap.

$$\mathsf{unreachable} \quad \hookrightarrow \quad \mathsf{trap}$$

drop

1. Assert: Due to validation, a value is on the top of the stack.
2. Pop the value val from the stack.

$$val\ \mathsf{drop} \quad \hookrightarrow \quad \epsilon$$

select $(t^*)^?$

1. Assert: Due to validation, a value of number type i32 is on the top of the stack.
2. Pop the value $(\mathsf{i32.const}\ c)$ from the stack.
3. Assert: Due to validation, a value is on the top of the stack.
4. Pop the value val_2 from the stack.
5. Assert: Due to validation, a value is on the top of the stack.
6. Pop the value val_1 from the stack.
7. If $c \neq 0$, then:
   a. Push the value val_1 to the stack.
8. Else:
   a. Push the value val_2 to the stack.

$$\begin{array}{lcll} val_1\ val_2\ (\mathsf{i32.const}\ c)\ (\mathsf{select}\ (t^*)^?) & \hookrightarrow & val_1 & \text{if } c \neq 0 \\ val_1\ val_2\ (\mathsf{i32.const}\ c)\ (\mathsf{select}\ (t^*)^?) & \hookrightarrow & val_2 & \text{if } c = 0 \end{array}$$

Note

In future versions of WebAssembly, select may allow more than one value per choice.

4.6.2 Control Instructions

block $bt\ instr^*$

1. Let z be the current state.
2. Let $t_1^m \to_{localidx_0^*} t_2^n$ be the destructuring of $\mathrm{instrtype}_z(bt)$.
3. Assert: Due to validation, $localidx_0^* = \epsilon$.
4. Assert: Due to validation, there are at least m values on the top of the stack.
5. Pop the values val^m from the stack.
6. Let L be the label whose arity is n and whose continuation is the end of the block.
7. Enter the block $val^m\ instr^*$ with the label L.

$$z; val^m\ (\mathsf{block}\ bt\ instr^*) \quad \hookrightarrow \quad (\mathsf{label}_n\{\epsilon\}\ val^m\ instr^*) \quad \text{if } \mathrm{instrtype}_z(bt) = t_1^m \to t_2^n$$

loop $bt\ instr^*$

1. Let z be the current state.
2. Let $t_1^m \to_{localidx_0^*} t_2^n$ be the destructuring of $\mathrm{instrtype}_z(bt)$.
3. Assert: Due to validation, $localidx_0^* = \epsilon$.
4. Assert: Due to validation, there are at least m values on the top of the stack.
5. Pop the values val^m from the stack.

6. Let L be the label whose arity is m and whose continuation is the start of the block.
7. Enter the block $val^m\ instr^*$ with the label L.

$$z; val^m\ (\mathsf{loop}\ bt\ instr^*) \quad \hookrightarrow \quad (\mathsf{label}_m\{\mathsf{loop}\ bt\ instr^*\}\ val^m\ instr^*) \quad \text{if } \mathrm{instrtype}_z(bt) = t_1^m \to t_2^n$$

$\mathsf{if}\ bt\ instr_1^*\ \mathsf{else}\ instr_2^*$

1. Assert: Due to validation, a value of number type i32 is on the top of the stack.
2. Pop the value $(\mathsf{i32.const}\ c)$ from the stack.
3. If $c \neq 0$, then:
   a. Execute the instruction $(\mathsf{block}\ bt\ instr_1^*)$.
4. Else:
   a. Execute the instruction $(\mathsf{block}\ bt\ instr_2^*)$.

$$\begin{array}{rcll} (\mathsf{i32.const}\ c)\ (\mathsf{if}\ bt\ instr_1^*\ \mathsf{else}\ instr_2^*) & \hookrightarrow & (\mathsf{block}\ bt\ instr_1^*) & \text{if } c \neq 0 \\ (\mathsf{i32.const}\ c)\ (\mathsf{if}\ bt\ instr_1^*\ \mathsf{else}\ instr_2^*) & \hookrightarrow & (\mathsf{block}\ bt\ instr_2^*) & \text{if } c = 0 \end{array}$$

$\mathsf{br}\ l$

1. If the first non-value entry of the stack is a label, then:
   a. Let L be the topmost label.
   b. Let n be the arity of L.
   c. If $l = 0$, then:
      1) Assert: Due to validation, there are at least n values on the top of the stack.
      2) Pop the values val^n from the stack.
      3) Pop all values val'^* from the top of the stack.
      4) Pop the label from the stack.
      5) Push the values val^n to the stack.
      6) Jump to the continuation of L.
   d. Else:
      1) Pop all values val^* from the top of the stack.
      2) Pop the label from the stack.
      3) Push the values val^* to the stack.
      4) Execute the instruction $(\mathsf{br}\ l - 1)$.
2. Else:
   a. Assert: Due to validation, the first non-value entry of the stack is a handler.
   b. Pop all values val^* from the top of the stack.
   c. Pop the handler from the stack.
   d. Push the values val^* to the stack.
   e. Execute the instruction $(\mathsf{br}\ l)$.

$$\begin{array}{rcll} (\mathsf{label}_n\{instr'^*\}\ val'^*\ val^n\ (\mathsf{br}\ l)\ instr^*) & \hookrightarrow & val^n\ instr'^* & \text{if } l = 0 \\ (\mathsf{label}_n\{instr'^*\}\ val^*\ (\mathsf{br}\ l)\ instr^*) & \hookrightarrow & val^*\ (\mathsf{br}\ l - 1) & \text{if } l > 0 \\ (\mathsf{handler}_n\{catch^*\}\ val^*\ (\mathsf{br}\ l)\ instr^*) & \hookrightarrow & val^*\ (\mathsf{br}\ l) & \end{array}$$

br_if l

1. Assert: Due to validation, a value of number type i32 is on the top of the stack.
2. Pop the value $(\mathsf{i32.const}\ c)$ from the stack.
3. If $c \neq 0$, then:
   a. Execute the instruction $(\mathsf{br}\ l)$.
4. Else:
   a. Do nothing.

$$\begin{array}{lcll} (\mathsf{i32.const}\ c)\ (\mathsf{br_if}\ l) & \hookrightarrow & (\mathsf{br}\ l) & \text{if } c \neq 0 \\ (\mathsf{i32.const}\ c)\ (\mathsf{br_if}\ l) & \hookrightarrow & \epsilon & \text{if } c = 0 \end{array}$$

br_table $l^*\ l'$

1. Assert: Due to validation, a value of number type i32 is on the top of the stack.
2. Pop the value $(\mathsf{i32.const}\ i)$ from the stack.
3. If $i < |l^*|$, then:
   a. Execute the instruction $(\mathsf{br}\ l^*[i])$.
4. Else:
   a. Execute the instruction $(\mathsf{br}\ l')$.

$$\begin{array}{lcll} (\mathsf{i32.const}\ i)\ (\mathsf{br_table}\ l^*\ l') & \hookrightarrow & (\mathsf{br}\ l^*[i]) & \text{if } i < |l^*| \\ (\mathsf{i32.const}\ i)\ (\mathsf{br_table}\ l^*\ l') & \hookrightarrow & (\mathsf{br}\ l') & \text{if } i \geq |l^*| \end{array}$$

br_on_null l

1. Assert: Due to validation, a value is on the top of the stack.
2. Pop the value val from the stack.
3. If $val = \mathsf{ref.null}$, then:
   a. Execute the instruction $(\mathsf{br}\ l)$.
4. Else:
   a. Push the value val to the stack.

$$\begin{array}{lcll} val\ (\mathsf{br_on_null}\ l) & \hookrightarrow & (\mathsf{br}\ l) & \text{if } val = \mathsf{ref.null} \\ val\ (\mathsf{br_on_null}\ l) & \hookrightarrow & val & \text{otherwise} \end{array}$$

br_on_non_null l

1. Assert: Due to validation, a value is on the top of the stack.
2. Pop the value val from the stack.
3. If $val = \mathsf{ref.null}$, then:
   a. Do nothing.
4. Else:
   a. Push the value val to the stack.
   b. Execute the instruction $(\mathsf{br}\ l)$.

$$\begin{array}{lcll} val\ (\mathsf{br_on_non_null}\ l) & \hookrightarrow & \epsilon & \text{if } val = \mathsf{ref.null} \\ val\ (\mathsf{br_on_non_null}\ l) & \hookrightarrow & val\ (\mathsf{br}\ l) & \text{otherwise} \end{array}$$

br_on_cast l rt_1 rt_2

1. Let f be the topmost frame.
2. Assert: Due to validation, a reference value is on the top of the stack.
3. Pop the value ref from the stack.
4. Push the value ref to the stack.
5. If ref is valid with type $\mathrm{clos}_{f.\mathsf{module}}(rt_2)$, then:
   a. Execute the instruction $(\mathsf{br}\ l)$.
6. Else:
   a. Do nothing.

$$
\begin{array}{lcll}
s; f; ref\ (\mathsf{br_on_cast}\ l\ rt_1\ rt_2) & \hookrightarrow & ref\ (\mathsf{br}\ l) & \text{if } s \vdash ref : \mathrm{clos}_{f.\mathsf{module}}(rt_2) \\
s; f; ref\ (\mathsf{br_on_cast}\ l\ rt_1\ rt_2) & \hookrightarrow & ref & \text{otherwise}
\end{array}
$$

br_on_cast_fail l rt_1 rt_2

1. Let f be the topmost frame.
2. Assert: Due to validation, a reference value is on the top of the stack.
3. Pop the value ref from the stack.
4. Push the value ref to the stack.
5. If ref is valid with type $\mathrm{clos}_{f.\mathsf{module}}(rt_2)$, then:
   a. Do nothing.
6. Else:
   a. Execute the instruction $(\mathsf{br}\ l)$.

$$
\begin{array}{lcll}
s; f; ref\ (\mathsf{br_on_cast_fail}\ l\ rt_1\ rt_2) & \hookrightarrow & ref & \text{if } s \vdash ref : \mathrm{clos}_{f.\mathsf{module}}(rt_2) \\
s; f; ref\ (\mathsf{br_on_cast_fail}\ l\ rt_1\ rt_2) & \hookrightarrow & ref\ (\mathsf{br}\ l) & \text{otherwise}
\end{array}
$$

return

1. If the first non-value entry of the stack is a frame, then:
   a. Let f be the topmost frame.
   b. Let n be the arity of f.
   c. Assert: Due to validation, there are at least n values on the top of the stack.
   d. Pop the values val^n from the stack.
   e. Pop all values val'^* from the top of the stack.
   f. Pop the frame from the stack.
   g. Push the values val^n to the stack.
2. Else if the first non-value entry of the stack is a label, then:
   a. Pop all values val^* from the top of the stack.
   b. Pop the label from the stack.
   c. Push the values val^* to the stack.
   d. Execute the instruction return.
3. Else:
   a. Assert: Due to validation, the first non-value entry of the stack is a handler.
   b. Pop all values val^* from the top of the stack.

c. Pop the handler from the stack.

d. Push the values val^* to the stack.

e. Execute the instruction return.

$$\begin{array}{rcl}(\mathsf{frame}_n\{f\}\ val'^*\ val^n\ \mathsf{return}\ instr^*) & \hookrightarrow & val^n \\ (\mathsf{label}_n\{instr'^*\}\ val^*\ \mathsf{return}\ instr^*) & \hookrightarrow & val^*\ \mathsf{return} \\ (\mathsf{handler}_n\{catch^*\}\ val^*\ \mathsf{return}\ instr^*) & \hookrightarrow & val^*\ \mathsf{return}\end{array}$$

call x

1. Let z be the current state.
2. Assert: Due to validation, $x < |z.\mathsf{module.funcs}|$.
3. Let a be the address $z.\mathsf{module.funcs}[x]$.
4. Assert: Due to validation, $a < |z.\mathsf{funcs}|$.
5. Push the value $(\mathsf{ref.func}\ a)$ to the stack.
6. Execute the instruction $(\mathsf{call_ref}\ z.\mathsf{funcs}[a].\mathsf{type})$.

$$z; (\mathsf{call}\ x) \quad \hookrightarrow \quad (\mathsf{ref.func}\ a)\ (\mathsf{call_ref}\ z.\mathsf{funcs}[a].\mathsf{type}) \quad \text{if } z.\mathsf{module.funcs}[x] = a$$

call_ref x

Todo

(*) Prose not spliced, for the prose merges the two cases of null and non-null references.

1. Assert: due to validation, a null or function reference is on the top of the stack.
2. Pop the reference value r from the stack.
3. If r is $\mathsf{ref.null}\ ht$, then:
   a. Trap.
4. Assert: due to validation, r is a function reference.
5. Let $\mathsf{ref.func}\ a$ be the reference r.
6. Invoke the function instance at address a.

$$z; (\mathsf{ref.null})\ (\mathsf{call_ref}\ y) \quad \hookrightarrow \quad z; \mathsf{trap}$$

Note

The formal rule for calling a non-null function reference is described below.

call_indirect x y

1. Execute the instruction $(\mathsf{table.get}\ x)$.
2. Execute the instruction $(\mathsf{ref.cast}\ (\mathsf{ref\ null}\ y))$.
3. Execute the instruction $(\mathsf{call_ref}\ y)$.

$$(\mathsf{call_indirect}\ x\ y) \quad \hookrightarrow \quad (\mathsf{table.get}\ x)\ (\mathsf{ref.cast}\ (\mathsf{ref\ null}\ y))\ (\mathsf{call_ref}\ y)$$

return_call x

1. Let z be the current state.
2. Assert: Due to validation, $x < |z.\mathsf{module}.\mathsf{funcs}|$.
3. Let a be the address $z.\mathsf{module}.\mathsf{funcs}[x]$.
4. Assert: Due to validation, $a < |z.\mathsf{funcs}|$.
5. Push the value $(\mathsf{ref.func}\ a)$ to the stack.
6. Execute the instruction $(\mathsf{return_call_ref}\ z.\mathsf{funcs}[a].\mathsf{type})$.

$$z; (\mathsf{return_call}\ x) \quad \hookrightarrow \quad (\mathsf{ref.func}\ a)\ (\mathsf{return_call_ref}\ z.\mathsf{funcs}[a].\mathsf{type}) \quad \text{if } z.\mathsf{module}.\mathsf{funcs}[x] = a$$

return_call_ref y

1. Let z be the current state.
2. If the first non-value entry of the stack is a label, then:
   a. Pop all values val^* from the top of the stack.
   b. Pop the label from the stack.
   c. Push the values val^* to the stack.
   d. Execute the instruction $(\mathsf{return_call_ref}\ y)$.
3. Else if the first non-value entry of the stack is a handler, then:
   a. Pop all values val^* from the top of the stack.
   b. Pop the handler from the stack.
   c. Push the values val^* to the stack.
   d. Execute the instruction $(\mathsf{return_call_ref}\ y)$.
4. Else:
   a. Assert: Due to validation, the first non-value entry of the stack is a frame.
   b. Assert: Due to validation, a value is on the top of the stack.
   c. Pop the value val'' from the stack.
   d. If $val'' = \mathsf{ref.null}$, then:
      1) Trap.
   e. Assert: Due to validation, val'' is some $\mathsf{ref.func}\ funcaddr$.
   f. Let $(\mathsf{ref.func}\ a)$ be the destructuring of val''.
   g. Assert: Due to validation, $a < |z.\mathsf{funcs}|$.
   h. Assert: Due to validation, the expansion of $z.\mathsf{funcs}[a].\mathsf{type}$ is some $\mathsf{func}\ resulttype \to resulttype$.
   i. Let $(\mathsf{func}\ t_1^n \to t_2^m)$ be the destructuring of the expansion of $z.\mathsf{funcs}[a].\mathsf{type}$.
   j. Assert: Due to validation, there are at least n values on the top of the stack.
   k. Pop the values val^n from the stack.
   l. Pop all values val'^* from the top of the stack.
   m. Pop the frame from the stack.
   n. Push the values val^n to the stack.
   o. Push the value $(\mathsf{ref.func}\ a)$ to the stack.
   p. Execute the instruction $(\mathsf{call_ref}\ y)$.

$$
\begin{array}{rcll}
z; (\mathsf{label}_k\{instr'^*\}\ val^*\ (\mathsf{return_call_ref}\ y)\ instr^*) & \hookrightarrow & val^*\ (\mathsf{return_call_ref}\ y) & \\
z; (\mathsf{handler}_k\{catch^*\}\ val^*\ (\mathsf{return_call_ref}\ y)\ instr^*) & \hookrightarrow & val^*\ (\mathsf{return_call_ref}\ y) & \\
z; (\mathsf{frame}_k\{f\}\ val^*\ (\mathsf{ref.null})\ (\mathsf{return_call_ref}\ y)\ instr^*) & \hookrightarrow & \mathsf{trap} & \\
z; (\mathsf{frame}_k\{f\}\ val'^*\ val^n\ (\mathsf{ref.func}\ a)\ (\mathsf{return_call_ref}\ y)\ instr^*) & \hookrightarrow & val^n\ (\mathsf{ref.func}\ a)\ (\mathsf{call_ref}\ y) & \\
 & & \quad \text{if } z.\mathsf{funcs}[a].\mathsf{type} \approx \mathsf{func}\ t_1^n \to t_2^m &
\end{array}
$$

return_call_indirect x y

1. Execute the instruction $(\mathsf{table.get}\ x)$.
2. Execute the instruction $(\mathsf{ref.cast}\ (\mathsf{ref\ null}\ y))$.
3. Execute the instruction $(\mathsf{return_call_ref}\ y)$.

$$(\mathsf{return_call_indirect}\ x\ y) \quad \hookrightarrow \quad (\mathsf{table.get}\ x)\ (\mathsf{ref.cast}\ (\mathsf{ref\ null}\ y))\ (\mathsf{return_call_ref}\ y)$$

throw x

1. Let z be the current state.
2. Assert: Due to validation, $x < |z.\mathsf{module.tags}|$.
3. Assert: Due to validation, the expansion of $z.\mathsf{tags}[x].\mathsf{type}$ is some $\mathsf{func}\ resulttype \to resulttype$.
4. Let $(\mathsf{func}\ t^n \to resulttype_0)$ be the destructuring of the expansion of $z.\mathsf{tags}[x].\mathsf{type}$.
5. Assert: Due to validation, $resulttype_0 = \epsilon$.
6. Let a be the length of $z.\mathsf{exns}$.
7. Assert: Due to validation, there are at least n values on the top of the stack.
8. Pop the values val^n from the stack.
9. Let exn be the exception instance $\{\mathsf{tag}\ z.\mathsf{module.tags}[x],\ \mathsf{fields}\ val^n\}$.
10. Append exn to $s.\mathsf{exns}$.
11. Push the value $(\mathsf{ref.exn}\ a)$ to the stack.
12. Execute the instruction throw_ref.

$$
\begin{array}{rcll}
z; val^n\ (\mathsf{throw}\ x) & \hookrightarrow & z[.\mathsf{exns} =\!\oplus\ exn]; (\mathsf{ref.exn}\ a)\ \mathsf{throw_ref} & \text{if } z.\mathsf{tags}[x].\mathsf{type} \approx \mathsf{func}\ t^n \to \epsilon \\
 & & & \wedge\ a = |z.\mathsf{exns}| \\
 & & & \wedge\ exn = \{\mathsf{tag}\ z.\mathsf{module.tags}[x],\ \mathsf{fields}\ val^n\}
\end{array}
$$

throw_ref

1. Let z be the current state.
2. Assert: Due to validation, a value is on the top of the stack.
3. Pop the value val' from the stack.
4. If $val' = \mathsf{ref.null}$, then:
   a. Trap.
5. If val' is some $\mathsf{ref.exn}\ exnaddr$, then:
   a. Let $(\mathsf{ref.exn}\ a)$ be the destructuring of val'.
   b. Pop all values val^* from the top of the stack.
   c. If $val^* \neq \epsilon$, then:
      1) Push the value $(\mathsf{ref.exn}\ a)$ to the stack.
      2) Execute the instruction throw_ref.
   d. Else if the first non-value entry of the stack is a label, then:
      1) Pop the label from the stack.

2) Push the value $(\mathsf{ref.exn}\ a)$ to the stack.

3) Execute the instruction $\mathsf{throw_ref}$.

e. Else:

1) If the first non-value entry of the stack is a frame, then:

a) Pop the frame from the stack.

b) Push the value $(\mathsf{ref.exn}\ a)$ to the stack.

c) Execute the instruction $\mathsf{throw_ref}$.

2) Else if the first non-value entry of the stack is not a $\mathsf{handler}$, then:

a) Throw the exception val' as a result.

3) Else:

a) Let H be the topmost $\mathsf{handler}$.

b) Let n be the arity of H.

c) Let $catch''^*$ be the catch handler of H.

d) If $catch''^* = \epsilon$, then:

1. Pop the $\mathsf{handler}$ from the stack.
2. Push the value $(\mathsf{ref.exn}\ a)$ to the stack.
3. Execute the instruction $\mathsf{throw_ref}$.

e) Else if $a \geq |z.\mathsf{exns}|$, then:

1. Let $catch_0\ catch'^*$ be $catch''^*$.
2. If $catch_0$ is some $\mathsf{catch_all}\ labelidx$, then:
   a. Let $(\mathsf{catch_all}\ l)$ be the destructuring of $catch_0$.
   b. Pop the $\mathsf{handler}$ from the stack.
   c. Execute the instruction $(\mathsf{br}\ l)$.
3. Else if $catch_0$ is not some $\mathsf{catch_all_ref}\ labelidx$, then:
   a. Let $catch\ catch'^*$ be $catch''^*$.
   b. Pop the $\mathsf{handler}$ from the stack.
   c. Let H' be the $\mathsf{handler}$ whose arity is n and whose catch handler is $catch'^*$.
   d. Push the $\mathsf{handler}$ H'.
   e. Push the value $(\mathsf{ref.exn}\ a)$ to the stack.
   f. Execute the instruction $\mathsf{throw_ref}$.
4. Else:
   a. Let $(\mathsf{catch_all_ref}\ l)$ be the destructuring of $catch_0$.
   b. Pop the $\mathsf{handler}$ from the stack.
   c. Push the value $(\mathsf{ref.exn}\ a)$ to the stack.
   d. Execute the instruction $(\mathsf{br}\ l)$.

f) Else:

1. Let val^* be $z.\mathsf{exns}[a].\mathsf{fields}$.
2. Let $catch_0\ catch'^*$ be $catch''^*$.
3. If $catch_0$ is some $\mathsf{catch}\ tagidx\ labelidx$, then:

a. Let $(\mathsf{catch}\ x\ l)$ be the destructuring of $catch_0$.

b. If $x < |z.\mathsf{module}.\mathsf{tags}|$ and $z.\mathsf{exns}[a].\mathsf{tag} = z.\mathsf{module}.\mathsf{tags}[x]$, then:

1) Pop the handler from the stack.

2) Push the values val^* to the stack.

3) Execute the instruction $(\mathsf{br}\ l)$.

c. Else:

1) Let $catch\ catch'^*$ be $catch''^*$.

2) Pop the handler from the stack.

3) Let H' be the handler whose arity is n and whose catch handler is $catch'^*$.

4) Push the handler H'.

5) Push the value $(\mathsf{ref.exn}\ a)$ to the stack.

6) Execute the instruction throw_ref.

4. Else if $catch_0$ is some $\mathsf{catch_ref}\ tagidx\ labelidx$, then:

a. Let $(\mathsf{catch_ref}\ x\ l)$ be the destructuring of $catch_0$.

b. If $x \geq |z.\mathsf{module}.\mathsf{tags}|$ or $z.\mathsf{exns}[a].\mathsf{tag} \neq z.\mathsf{module}.\mathsf{tags}[x]$, then:

1) Let $catch\ catch'^*$ be $catch''^*$.

2) Pop the handler from the stack.

3) Let H' be the handler whose arity is n and whose catch handler is $catch'^*$.

4) Push the handler H'.

5) Push the value $(\mathsf{ref.exn}\ a)$ to the stack.

6) Execute the instruction throw_ref.

c. Else:

1) Pop the handler from the stack.

2) Push the values val^* to the stack.

3) Push the value $(\mathsf{ref.exn}\ a)$ to the stack.

4) Execute the instruction $(\mathsf{br}\ l)$.

5. Else:

a. If $catch_0$ is some $\mathsf{catch_all}\ labelidx$, then:

1) Let $(\mathsf{catch_all}\ l)$ be the destructuring of $catch_0$.

2) Pop the handler from the stack.

3) Execute the instruction $(\mathsf{br}\ l)$.

b. Else if $catch_0$ is not some $\mathsf{catch_all_ref}\ labelidx$, then:

1) Let $catch\ catch'^*$ be $catch''^*$.

2) Pop the handler from the stack.

3) Let H be the handler whose arity is n and whose catch handler is $catch'^*$.

4) Push the handler H.

5) Push the value $(\mathsf{ref.exn}\ a)$ to the stack.

6) Execute the instruction throw_ref.

c. Else:

1) Let $(\text{catch_all_ref}\ l)$ be the destructuring of $catch_0$.
2) Pop the handler from the stack.
3) Push the value $(\text{ref.exn}\ a)$ to the stack.
4) Execute the instruction $(\text{br}\ l)$.

6. Else:
   a. Assert: Due to validation, the first non-value entry of the stack is not a label.
   b. Assert: Due to validation, the first non-value entry of the stack is not a frame.
   c. Assert: Due to validation, the first non-value entry of the stack is not a handler.
   d. Throw the exception val' as a result.

$$
\begin{array}{rcl}
z; (\text{ref.null})\ \text{throw_ref} & \hookrightarrow & \text{trap} \\
z; val^*\ (\text{ref.exn}\ a)\ \text{throw_ref}\ instr^* & \hookrightarrow & (\text{ref.exn}\ a)\ \text{throw_ref} \\
 & & \quad \text{if } val^* \neq \epsilon \vee instr^* \neq \epsilon \\
z; (\text{label}_n\{instr'^*\}\ (\text{ref.exn}\ a)\ \text{throw_ref}) & \hookrightarrow & (\text{ref.exn}\ a)\ \text{throw_ref} \\
z; (\text{frame}_n\{f\}\ (\text{ref.exn}\ a)\ \text{throw_ref}) & \hookrightarrow & (\text{ref.exn}\ a)\ \text{throw_ref} \\
z; (\text{handler}_n\{\epsilon\}\ (\text{ref.exn}\ a)\ \text{throw_ref}) & \hookrightarrow & (\text{ref.exn}\ a)\ \text{throw_ref} \\
z; (\text{handler}_n\{(\text{catch}\ x\ l)\ catch'^*\}\ (\text{ref.exn}\ a)\ \text{throw_ref}) & \hookrightarrow & val^*\ (\text{br}\ l) \\
 & & \quad \text{if } z.\text{exns}[a].\text{tag} = z.\text{module.tags}[x] \\
 & & \quad \wedge\ val^* = z.\text{exns}[a].\text{fields} \\
z; (\text{handler}_n\{(\text{catch_ref}\ x\ l)\ catch'^*\}\ (\text{ref.exn}\ a)\ \text{throw_ref}) & \hookrightarrow & val^*\ (\text{ref.exn}\ a)\ (\text{br}\ l) \\
 & & \quad \text{if } z.\text{exns}[a].\text{tag} = z.\text{module.tags}[x] \\
 & & \quad \wedge\ val^* = z.\text{exns}[a].\text{fields} \\
z; (\text{handler}_n\{(\text{catch_all}\ l)\ catch'^*\}\ (\text{ref.exn}\ a)\ \text{throw_ref}) & \hookrightarrow & (\text{br}\ l) \\
z; (\text{handler}_n\{(\text{catch_all_ref}\ l)\ catch'^*\}\ (\text{ref.exn}\ a)\ \text{throw_ref}) & \hookrightarrow & (\text{ref.exn}\ a)\ (\text{br}\ l) \\
z; (\text{handler}_n\{catch\ catch'^*\}\ (\text{ref.exn}\ a)\ \text{throw_ref}) & \hookrightarrow & (\text{handler}_n\{catch'^*\}\ (\text{ref.exn}\ a)\ \text{throw_ref}) \\
 & & \quad \text{otherwise}
\end{array}
$$

$\text{try_table}\ bt\ catch^*\ instr^*$

1. Let z be the current state.
2. Let $t_1^m \to_{localidx_0^*} t_2^n$ be the destructuring of $\text{instrtype}_z(bt)$.
3. Assert: Due to validation, $localidx_0^* = \epsilon$.
4. Assert: Due to validation, there are at least m values on the top of the stack.
5. Pop the values val^m from the stack.
6. Let H be the handler whose arity is n and whose catch handler is $catch^*$.
7. Push the handler H.
8. Let L be the label whose arity is n and whose continuation is the end of the block.
9. Enter the block $val^m\ instr^*$ with the label L.

$$
\begin{array}{rcl}
z; val^m\ (\text{try_table}\ bt\ catch^*\ instr^*) & \hookrightarrow & (\text{handler}_n\{catch^*\}\ (\text{label}_n\{\epsilon\}\ val^m\ instr^*)) \\
 & & \quad \text{if } \text{instrtype}_z(bt) = t_1^m \to t_2^n
\end{array}
$$

4.6.3 Blocks

The following auxiliary rules define the semantics of executing an instruction sequence that forms a block.

Entering $instr^*$ with label L and values val^*

1. Push L to the stack.
2. Push the values val^* to the stack.
3. Jump to the start of the instruction sequence $instr^*$.

Note

No formal reduction rule is needed for entering an instruction sequence, because the label L is embedded in the administrative instruction that structured control instructions reduce to directly.

Exiting $instr^*$ with label L

When the end of a block is reached without a jump, exception, or trap aborting it, then the following steps are performed.

1. Pop all values val^* from the top of the stack.
2. Assert: due to validation, the label L is now on the top of the stack.
3. Pop the label from the stack.
4. Push val^* back to the stack.
5. Jump to the position after the end of the structured control instruction associated with the label L.

$$(\mathsf{label}_n\{instr^*\}\ val^*) \quad \hookrightarrow \quad val^*$$

Note

This semantics also applies to the instruction sequence contained in a loop instruction. Therefore, execution of a loop falls off the end, unless a backwards branch is performed explicitly.

4.6.4 Exception Handling

The following auxiliary rules define the semantics of entering and exiting try_table blocks.

Entering $instr^*$ with label L and exception handler H

1. Push H to the stack.
2. Push L onto the stack.
3. Jump to the start of the instruction sequence $instr^*$.

Note

No formal reduction rule is needed for entering an exception handler because it is an administrative instruction that the try_table instruction reduces to directly.

Exiting an exception handler

When the end of a try_table block is reached without a jump, exception, or trap, then the following steps are performed.

1. Let m be the number of values on the top of the stack.
2. Pop the values val^m from the stack.
3. Assert: due to validation, a handler and a label are now on the top of the stack.

4. Pop the label from the stack.
5. Pop the handler H from the stack.
6. Push val^m back to the stack.
7. Jump to the position after the end of the administrative instruction associated with the handler H.

$$(\mathsf{handler}_n\{catch^*\}\ val^*) \quad \hookrightarrow \quad val^*$$

4.6.5 Function Calls

The following auxiliary rules define the semantics of invoking a function instance through one of the call instructions and returning from it.

Invocation of function reference $(\mathsf{ref.func}\ a)$

1. Let z be the current state.
2. Assert: due to validation, $z.\mathsf{funcs}[a]$ exists.
3. Let fi be the function instance, $z.\mathsf{funcs}[a]$.
4. Let $\mathsf{func}\ [t_1^n] \to [t_2^m]$ be the composite type $\mathrm{expand}(fi.\mathsf{type})$.
5. Let $\mathsf{func}\ x\ (\mathsf{local}\ t)^*\ instr^*$ be the function $fi.\mathsf{code}$.
6. Assert: due to validation, n values are on the top of the stack.
7. Pop the values val^n from the stack.
8. Let f be the frame $\{\mathsf{locals}\ val^n\ (\mathrm{default}_t)^*, \mathsf{module}\ fi.\mathsf{module}\}$.
9. Push the activation of f with arity m to the stack.
10. Let L be the label whose arity is m and whose continuation is the end of the function.
11. Enter the instruction sequence $instr^*$ with label L and no values.

$$\begin{array}{lll} z; val^n\ (\mathsf{ref.func}\ a)\ (\mathsf{call_ref}\ y) & \hookrightarrow & z; (\mathsf{frame}_m\{f\}\ (\mathsf{label}_m\{\epsilon\}\ instr^*)) \\ & & \quad \text{if } z.\mathsf{funcs}[a] = fi \\ & & \quad \wedge\ fi.\mathsf{type} \approx \mathsf{func}\ t_1^n \to t_2^m \\ & & \quad \wedge\ fi.\mathsf{code} = \mathsf{func}\ x\ (\mathsf{local}\ t)^*\ (instr^*) \\ & & \quad \wedge\ f = \{\mathsf{locals}\ val^n\ (\mathrm{default}_t)^*,\ \mathsf{module}\ fi.\mathsf{module}\} \end{array}$$

Note

For non-defaultable types, the respective local is left uninitialized by these rules.

Returning from a function

When the end of a function is reached without a jump (including through return), or an exception or trap aborting it, then the following steps are performed.

1. Let F be the current frame.
2. Let n be the arity of the activation of F.
3. Assert: due to validation, there are n values on the top of the stack.
4. Pop the results val^n from the stack.
5. Assert: due to validation, the frame F is now on the top of the stack.
6. Pop the frame from the stack.
7. Push val^n back to the stack.

8. Jump to the instruction after the original call.

$$(\mathsf{frame}_n\{f\}\ val^n) \quad \hookrightarrow \quad val^n$$

Host Functions

Invoking a host function has non-deterministic behavior. It may either terminate with a trap, an exception, or return regularly. However, in the latter case, it must consume and produce the right number and types of WebAssembly values on the stack, according to its function type.

A host function may also modify the store. However, all store modifications must result in an extension of the original store, i.e., they must only modify mutable contents and must not have instances removed. Furthermore, the resulting store must be valid, i.e., all data and code in it is well-typed.

$$\begin{array}{lcl} s; f; val^n\ (\mathsf{ref.func}\ a)\ (\mathsf{call_ref}\ y) & \hookrightarrow & s'; f; result \\ & & \quad \text{if } s.\mathsf{funcs}[a] = fi \\ & & \quad \wedge\ fi.\mathsf{type} \approx \mathsf{func}\ t_1^n \to t_2^m \\ & & \quad \wedge\ fi.\mathsf{code} = hf \\ & & \quad \wedge\ (s', result) \in hf(s, val^n) \\ s; f; val^n\ (\mathsf{ref.func}\ a)\ (\mathsf{call_ref}\ y) & \hookrightarrow & s; f; val^n\ (\mathsf{ref.func}\ a)\ (\mathsf{call_ref}\ y) \\ & & \quad \text{if } s.\mathsf{funcs}[a] = fi \\ & & \quad \wedge\ fi.\mathsf{type} \approx \mathsf{func}\ t_1^n \to t_2^m \\ & & \quad \wedge\ fi.\mathsf{code} = hf \\ & & \quad \wedge\ \bot \in hf(s, val^n) \end{array}$$

Here, $hf(s, val^n)$ denotes the implementation-defined execution of host function hf in current store s with arguments val^n. It yields a set of possible outcomes, where each element is either a pair of a modified store s' and a result or the special value $\bot$ indicating divergence. A host function is non-deterministic if there is at least one argument for which the set of outcomes is not singular.

For a WebAssembly implementation to be sound in the presence of host functions, every host function instance must be valid, which means that it adheres to suitable pre- and post-conditions: under a valid store s, and given arguments val^n matching the ascribed parameter types t_1^n, executing the host function must yield a non-empty set of possible outcomes each of which is either divergence or consists of a valid store s' that is an extension of s and a result matching the ascribed return types t_2^m. All these notions are made precise in the Appendix.

Note

A host function can call back into WebAssembly by invoking a function exported from a module. However, the effects of any such call are subsumed by the non-deterministic behavior allowed for the host function.

4.6.6 Variable Instructions

$\mathsf{local.get}\ x$

1. Let z be the current state.
2. Assert: Due to validation, $z.\mathsf{locals}[x]$ is defined.
3. Let val be $z.\mathsf{locals}[x]$.
4. Push the value val to the stack.

$$z; (\mathsf{local.get}\ x) \quad \hookrightarrow \quad val \quad \text{if } z.\mathsf{locals}[x] = val$$

$\mathsf{local.set}\ x$

1. Let z be the current state.
2. Assert: Due to validation, a value is on the top of the stack.

3. Pop the value val from the stack.
4. Replace $z.\mathsf{locals}[x]$ with val.

$$z; val\ (\mathsf{local.set}\ x) \quad \hookrightarrow \quad z[.\mathsf{locals}[x] = val]; \epsilon$$

$\mathsf{local.tee}\ x$

1. Assert: Due to validation, a value is on the top of the stack.
2. Pop the value val from the stack.
3. Push the value val to the stack.
4. Push the value val to the stack.
5. Execute the instruction $(\mathsf{local.set}\ x)$.

$$val\ (\mathsf{local.tee}\ x) \quad \hookrightarrow \quad val\ val\ (\mathsf{local.set}\ x)$$

$\mathsf{global.get}\ x$

1. Let z be the current state.
2. Let val be the value $z.\mathsf{globals}[x].\mathsf{value}$.
3. Push the value val to the stack.

$$z; (\mathsf{global.get}\ x) \quad \hookrightarrow \quad val \quad \text{if } z.\mathsf{globals}[x].\mathsf{value} = val$$

$\mathsf{global.set}\ x$

1. Let z be the current state.
2. Assert: Due to validation, a value is on the top of the stack.
3. Pop the value val from the stack.
4. Replace $z.\mathsf{globals}[x].\mathsf{value}$ with val.

$$z; val\ (\mathsf{global.set}\ x) \quad \hookrightarrow \quad z[.\mathsf{globals}[x].\mathsf{value} = val]; \epsilon$$

4.6.7 Table Instructions

$\mathsf{table.get}\ x$

1. Let z be the current state.
2. Assert: Due to validation, a number value is on the top of the stack.
3. Pop the value $(at.\mathsf{const}\ i)$ from the stack.
4. If $i \geq |z.\mathsf{tables}[x].\mathsf{refs}|$, then:
   a. Trap.
5. Push the value $z.\mathsf{tables}[x].\mathsf{refs}[i]$ to the stack.

$$\begin{array}{lll} z; (at.\mathsf{const}\ i)\ (\mathsf{table.get}\ x) & \hookrightarrow & \mathsf{trap} \quad \text{if } i \geq |z.\mathsf{tables}[x].\mathsf{refs}| \\ z; (at.\mathsf{const}\ i)\ (\mathsf{table.get}\ x) & \hookrightarrow & z.\mathsf{tables}[x].\mathsf{refs}[i] \quad \text{if } i < |z.\mathsf{tables}[x].\mathsf{refs}| \end{array}$$

table.set x

1. Let z be the current state.
2. Assert: Due to validation, a reference value is on the top of the stack.
3. Pop the value ref from the stack.
4. Assert: Due to validation, a number value is on the top of the stack.
5. Pop the value $(at.\mathsf{const}\ i)$ from the stack.
6. If $i \geq |z.\mathsf{tables}[x].\mathsf{refs}|$, then:
   a. Trap.
7. Replace $z.\mathsf{tables}[x].\mathsf{refs}[i]$ with ref.

$$
\begin{array}{lcll}
z; (at.\mathsf{const}\ i)\ ref\ (\mathsf{table.set}\ x) & \hookrightarrow & z; \mathsf{trap} & \text{if } i \geq |z.\mathsf{tables}[x].\mathsf{refs}| \\
z; (at.\mathsf{const}\ i)\ ref\ (\mathsf{table.set}\ x) & \hookrightarrow & z[.\mathsf{tables}[x].\mathsf{refs}[i] = ref]; \epsilon & \text{if } i < |z.\mathsf{tables}[x].\mathsf{refs}|
\end{array}
$$

table.size x

1. Let z be the current state.
2. Let $(at\ lim\ rt)$ be the destructuring of $z.\mathsf{tables}[x].\mathsf{type}$.
3. Let n be the length of $z.\mathsf{tables}[x].\mathsf{refs}$.
4. Push the value $(at.\mathsf{const}\ n)$ to the stack.

$$
z; (\mathsf{table.size}\ x) \quad \hookrightarrow \quad (at.\mathsf{const}\ n) \quad \begin{array}{l}\text{if } |z.\mathsf{tables}[x].\mathsf{refs}| = n \\ \wedge\ z.\mathsf{tables}[x].\mathsf{type} = at\ lim\ rt\end{array}
$$

table.grow x

1. Let z be the current state.
2. Assert: Due to validation, a number value is on the top of the stack.
3. Pop the value $(at.\mathsf{const}\ n)$ from the stack.
4. Assert: Due to validation, a reference value is on the top of the stack.
5. Pop the value ref from the stack.
6. Either:
   a. Let ti be the table instance $\mathrm{growtable}(z.\mathsf{tables}[x], n, ref)$.
   b. Push the value $(at.\mathsf{const}\ |z.\mathsf{tables}[x].\mathsf{refs}|)$ to the stack.
   c. Replace $z.\mathsf{tables}[x]$ with ti.
7. Or:
   a. Push the value $(at.\mathsf{const}\ \mathrm{signed}^{-1}_{|at|}(-1))$ to the stack.

$$
\begin{array}{lcll}
z; ref\ (at.\mathsf{const}\ n)\ (\mathsf{table.grow}\ x) & \hookrightarrow & z[.\mathsf{tables}[x] = ti]; (at.\mathsf{const}\ |z.\mathsf{tables}[x].\mathsf{refs}|) & \\
 & & \quad \text{if } ti = \mathrm{growtable}(z.\mathsf{tables}[x], n, ref) & \\
z; ref\ (at.\mathsf{const}\ n)\ (\mathsf{table.grow}\ x) & \hookrightarrow & z; (at.\mathsf{const}\ \mathrm{signed}^{-1}_{|at|}(-1)) &
\end{array}
$$

Note

The table.grow instruction is non-deterministic. It may either succeed, returning the old table size sz, or fail, returning -1. Failure *must* occur if the referenced table instance has a maximum size defined that would be exceeded. However, failure *can* occur in other cases as well. In practice, the choice depends on the resources available to the embedder.

table.fill x

1. Let z be the current state.
2. Assert: Due to validation, a number value is on the top of the stack.
3. Pop the value $(at.\mathsf{const}\ n)$ from the stack.
4. Assert: Due to validation, a value is on the top of the stack.
5. Pop the value val from the stack.
6. Assert: Due to validation, a value of number type at is on the top of the stack.
7. Pop the value $(numtype_0.\mathsf{const}\ i)$ from the stack.
8. If $i + n > |z.\mathsf{tables}[x].\mathsf{refs}|$, then:
   a. Trap.
9. If $n = 0$, then:
   a. Do nothing.
10. Else:
    a. Push the value $(at.\mathsf{const}\ i)$ to the stack.
    b. Push the value val to the stack.
    c. Execute the instruction $(\mathsf{table.set}\ x)$.
    d. Push the value $(at.\mathsf{const}\ i + 1)$ to the stack.
    e. Push the value val to the stack.
    f. Push the value $(at.\mathsf{const}\ n - 1)$ to the stack.
    g. Execute the instruction $(\mathsf{table.fill}\ x)$.

$$
\begin{array}{lcll}
z; (at.\mathsf{const}\ i)\ val\ (at.\mathsf{const}\ n)\ (\mathsf{table.fill}\ x) & \hookrightarrow & \mathsf{trap} & \text{if } i + n > |z.\mathsf{tables}[x].\mathsf{refs}| \\
z; (at.\mathsf{const}\ i)\ val\ (at.\mathsf{const}\ n)\ (\mathsf{table.fill}\ x) & \hookrightarrow & \epsilon & \text{otherwise, if } n = 0 \\
z; (at.\mathsf{const}\ i)\ val\ (at.\mathsf{const}\ n)\ (\mathsf{table.fill}\ x) & \hookrightarrow & & \\
\quad (at.\mathsf{const}\ i)\ val\ (\mathsf{table.set}\ x) & & & \text{otherwise} \\
\quad (at.\mathsf{const}\ i + 1)\ val\ (at.\mathsf{const}\ n - 1)\ (\mathsf{table.fill}\ x) & & &
\end{array}
$$

table.copy x_1 x_2

1. Let z be the current state.
2. Assert: Due to validation, a number value is on the top of the stack.
3. Pop the value $(at.\mathsf{const}\ n)$ from the stack.
4. Assert: Due to validation, a number value is on the top of the stack.
5. Pop the value $(at_2.\mathsf{const}\ i_2)$ from the stack.
6. Assert: Due to validation, a number value is on the top of the stack.
7. Pop the value $(at_1.\mathsf{const}\ i_1)$ from the stack.
8. If $i_1 + n > |z.\mathsf{tables}[x_1].\mathsf{refs}|$, then:
   a. Trap.
9. If $i_2 + n > |z.\mathsf{tables}[x_2].\mathsf{refs}|$, then:
   a. Trap.
10. If $n = 0$, then:
    a. Do nothing.
11. Else:

a. If $i_1 \leq i_2$, then:

1) Push the value $(at_1.\mathsf{const}\ i_1)$ to the stack.

2) Push the value $(at_2.\mathsf{const}\ i_2)$ to the stack.

3) Execute the instruction $(\mathsf{table.get}\ x_2)$.

4) Execute the instruction $(\mathsf{table.set}\ x_1)$.

5) Push the value $(at_1.\mathsf{const}\ i_1 + 1)$ to the stack.

6) Push the value $(at_2.\mathsf{const}\ i_2 + 1)$ to the stack.

b. Else:

1) Push the value $(at_1.\mathsf{const}\ i_1 + n - 1)$ to the stack.

2) Push the value $(at_2.\mathsf{const}\ i_2 + n - 1)$ to the stack.

3) Execute the instruction $(\mathsf{table.get}\ x_2)$.

4) Execute the instruction $(\mathsf{table.set}\ x_1)$.

5) Push the value $(at_1.\mathsf{const}\ i_1)$ to the stack.

6) Push the value $(at_2.\mathsf{const}\ i_2)$ to the stack.

c. Push the value $(at.\mathsf{const}\ n - 1)$ to the stack.

d. Execute the instruction $(\mathsf{table.copy}\ x_1\ x_2)$.

$$
\begin{array}{lcll}
z; (at_1.\mathsf{const}\ i_1)\ (at_2.\mathsf{const}\ i_2)\ (at'.\mathsf{const}\ n)\ (\mathsf{table.copy}\ x_1\ x_2) & \hookrightarrow & \mathsf{trap} & \\
\quad \text{if } i_1 + n > |z.\mathsf{tables}[x_1].\mathsf{refs}| \vee i_2 + n > |z.\mathsf{tables}[x_2].\mathsf{refs}| & & & \\
z; (at_1.\mathsf{const}\ i_1)\ (at_2.\mathsf{const}\ i_2)\ (at'.\mathsf{const}\ n)\ (\mathsf{table.copy}\ x\ y) & \hookrightarrow & \epsilon & \text{otherwise, if } n = 0 \\
z; (at_1.\mathsf{const}\ i_1)\ (at_2.\mathsf{const}\ i_2)\ (at'.\mathsf{const}\ n)\ (\mathsf{table.copy}\ x\ y) & \hookrightarrow & & \\
(at_1.\mathsf{const}\ i_1)\ (at_2.\mathsf{const}\ i_2)\ (\mathsf{table.get}\ y)\ (\mathsf{table.set}\ x) & & & \text{otherwise, if } i_1 \leq i_2 \\
(at_1.\mathsf{const}\ i_1 + 1)\ (at_2.\mathsf{const}\ i_2 + 1)\ (at'.\mathsf{const}\ n - 1)\ (\mathsf{table.copy}\ x\ y) & & & \\
z; (at_1.\mathsf{const}\ i_1)\ (at_2.\mathsf{const}\ i_2)\ (at'.\mathsf{const}\ n)\ (\mathsf{table.copy}\ x\ y) & \hookrightarrow & & \\
(at_1.\mathsf{const}\ i_1 + n - 1)\ (at_2.\mathsf{const}\ i_2 + n - 1)\ (\mathsf{table.get}\ y)\ (\mathsf{table.set}\ x) & & & \text{otherwise} \\
(at_1.\mathsf{const}\ i_1)\ (at_2.\mathsf{const}\ i_2)\ (at'.\mathsf{const}\ n - 1)\ (\mathsf{table.copy}\ x\ y) & & &
\end{array}
$$

$\mathsf{table.init}\ x\ y$

1. Let z be the current state.
2. Assert: Due to validation, a value of number type i32 is on the top of the stack.
3. Pop the value $(\mathsf{i32.const}\ n)$ from the stack.
4. Assert: Due to validation, a value of number type i32 is on the top of the stack.
5. Pop the value $(\mathsf{i32.const}\ j)$ from the stack.
6. Assert: Due to validation, a number value is on the top of the stack.
7. Pop the value $(at.\mathsf{const}\ i)$ from the stack.
8. If $i + n > |z.\mathsf{tables}[x].\mathsf{refs}|$, then:
   a. Trap.
9. If $j + n > |z.\mathsf{elems}[y].\mathsf{refs}|$, then:
   a. Trap.
10. If $n = 0$, then:
    a. Do nothing.
11. Else:
    a. Assert: Due to validation, $j < |z.\mathsf{elems}[y].\mathsf{refs}|$.

b. Push the value $(at.\mathsf{const}\ i)$ to the stack.

c. Push the value $z.\mathsf{elems}[y].\mathsf{refs}[j]$ to the stack.

d. Execute the instruction $(\mathsf{table.set}\ x)$.

e. Push the value $(at.\mathsf{const}\ i+1)$ to the stack.

f. Push the value $(\mathsf{i32.const}\ j+1)$ to the stack.

g. Push the value $(\mathsf{i32.const}\ n-1)$ to the stack.

h. Execute the instruction $(\mathsf{table.init}\ x\ y)$.

$$
\begin{array}{lcll}
z;(at.\mathsf{const}\ i)\ (\mathsf{i32.const}\ j)\ (\mathsf{i32.const}\ n)\ (\mathsf{table.init}\ x\ y) & \hookrightarrow & \mathsf{trap} & \\
\quad \text{if } i+n > |z.\mathsf{tables}[x].\mathsf{refs}| \vee j+n > |z.\mathsf{elems}[y].\mathsf{refs}| & & & \\
z;(at.\mathsf{const}\ i)\ (\mathsf{i32.const}\ j)\ (\mathsf{i32.const}\ n)\ (\mathsf{table.init}\ x\ y) & \hookrightarrow & \epsilon & \text{otherwise, if } n = 0 \\
z;(at.\mathsf{const}\ i)\ (\mathsf{i32.const}\ j)\ (\mathsf{i32.const}\ n)\ (\mathsf{table.init}\ x\ y) & \hookrightarrow & & \\
\quad (at.\mathsf{const}\ i)\ z.\mathsf{elems}[y].\mathsf{refs}[j]\ (\mathsf{table.set}\ x) & & & \text{otherwise} \\
\quad (at.\mathsf{const}\ i+1)\ (\mathsf{i32.const}\ j+1)\ (\mathsf{i32.const}\ n-1)\ (\mathsf{table.init}\ x\ y) & & &
\end{array}
$$

$\mathsf{elem.drop}\ x$

1. Let z be the current state.
2. Replace $z.\mathsf{elems}[x].\mathsf{refs}$ with ϵ.

$$z;(\mathsf{elem.drop}\ x) \quad \hookrightarrow \quad z[.\mathsf{elems}[x].\mathsf{refs} = \epsilon];\epsilon$$

4.6.8 Memory Instructions

Note

The alignment $memarg.\mathsf{align}$ in load and store instructions does not affect the semantics. It is a hint that the offset ea at which the memory is accessed is intended to satisfy the property $ea \bmod 2^{memarg.\mathsf{align}} = 0$. A WebAssembly implementation can use this hint to optimize for the intended use. Unaligned access violating that property is still allowed and must succeed regardless of the annotation. However, it may be substantially slower on some hardware.

$nt.\mathsf{load}\ x\ ao$

1. Let z be the current state.
2. Assert: Due to validation, a number value is on the top of the stack.
3. Pop the value $(at.\mathsf{const}\ i)$ from the stack.
4. If $i + ao.\mathsf{offset} + |nt|/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|$, then:
   a. Trap.
5. Let c be the result for which $\mathrm{bytes}_{nt}(c) = z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : |nt|/8]$.
6. Push the value $(nt.\mathsf{const}\ c)$ to the stack.

$$
\begin{array}{lcll}
z;(at.\mathsf{const}\ i)\ (nt.\mathsf{load}\ x\ ao) & \hookrightarrow & \mathsf{trap} & \\
\quad \text{if } i + ao.\mathsf{offset} + |nt|/8 > |z.\mathsf{mems}[x].\mathsf{bytes}| & & & \\
z;(at.\mathsf{const}\ i)\ (nt.\mathsf{load}\ x\ ao) & \hookrightarrow & (nt.\mathsf{const}\ c) & \\
\quad \text{if } \mathrm{bytes}_{nt}(c) = z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : |nt|/8] & & &
\end{array}
$$

$\mathsf{i}N.\mathsf{load}n_sx\ x\ ao$

1. Let z be the current state.
2. Assert: Due to validation, a number value is on the top of the stack.
3. Pop the value $(at.\mathsf{const}\ i)$ from the stack.
4. If $i + ao.\mathsf{offset} + n/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|$, then:
   a. Trap.
5. Let c be the result for which $\mathrm{bytes}_{\mathsf{i}n}(c) = z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : n/8]$.
6. Push the value $(\mathsf{i}N.\mathsf{const}\ \mathrm{extend}^{sx}_{n,|\mathsf{i}N|}(c))$ to the stack.

$$
\begin{array}{lcll}
z; (at.\mathsf{const}\ i)\ (\mathsf{i}N.\mathsf{load}n_sx\ x\ ao) & \hookrightarrow & \mathsf{trap} & \\
 & & \multicolumn{2}{l}{\quad \text{if } i + ao.\mathsf{offset} + n/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|} \\
z; (at.\mathsf{const}\ i)\ (\mathsf{i}N.\mathsf{load}n_sx\ x\ ao) & \hookrightarrow & (\mathsf{i}N.\mathsf{const}\ \mathrm{extend}^{sx}_{n,|\mathsf{i}N|}(c)) & \\
 & & \multicolumn{2}{l}{\quad \text{if } \mathrm{bytes}_{\mathsf{i}n}(c) = z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : n/8]}
\end{array}
$$

$\mathsf{v128}.\mathsf{load}\ x\ ao$

1. Let z be the current state.
2. Assert: Due to validation, a number value is on the top of the stack.
3. Pop the value $(at.\mathsf{const}\ i)$ from the stack.
4. If $i + ao.\mathsf{offset} + |\mathsf{v128}|/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|$, then:
   a. Trap.
5. Let c be the result for which $\mathrm{bytes}_{\mathsf{v128}}(c) = z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : |\mathsf{v128}|/8]$.
6. Push the value $(\mathsf{v128}.\mathsf{const}\ c)$ to the stack.

$$
\begin{array}{lcll}
z; (at.\mathsf{const}\ i)\ (\mathsf{v128}.\mathsf{load}\ x\ ao) & \hookrightarrow & \mathsf{trap} & \text{if } i + ao.\mathsf{offset} + |\mathsf{v128}|/8 > |z.\mathsf{mems}[x].\mathsf{bytes}| \\
z; (at.\mathsf{const}\ i)\ (\mathsf{v128}.\mathsf{load}\ x\ ao) & \hookrightarrow & (\mathsf{v128}.\mathsf{const}\ c) & \text{if } \mathrm{bytes}_{\mathsf{v128}}(c) = z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : |\mathsf{v128}|/8]
\end{array}
$$

$\mathsf{v128}.\mathsf{load}K\mathsf{x}M_sx\ x\ ao$

1. Let z be the current state.
2. Assert: Due to validation, a number value is on the top of the stack.
3. Pop the value $(at.\mathsf{const}\ i)$ from the stack.
4. If $i + ao.\mathsf{offset} + K \cdot M/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|$, then:
   a. Trap.
5. Let j^M be the result for which $(\mathrm{bytes}_{\mathsf{i}K}(j^M) = z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} + k \cdot K/8 : K/8])^{k<M}$.
6. Let $\mathsf{i}N$ be the result for which $N = K \cdot 2$.
7. Let c be $\mathrm{lanes}^{-1}_{\mathsf{i}N\mathsf{x}M}(\mathrm{extend}^{sx}_{K,N}(j)^M)$.
8. Push the value $(\mathsf{v128}.\mathsf{const}\ c)$ to the stack.

$$
\begin{array}{lcll}
z; (at.\mathsf{const}\ i)\ (\mathsf{v128}.\mathsf{load}K\mathsf{x}M_sx\ x\ ao) & \hookrightarrow & \mathsf{trap} & \text{if } i + ao.\mathsf{offset} + K \cdot M/8 > |z.\mathsf{mems}[x].\mathsf{bytes}| \\
z; (at.\mathsf{const}\ i)\ (\mathsf{v128}.\mathsf{load}K\mathsf{x}M_sx\ x\ ao) & \hookrightarrow & (\mathsf{v128}.\mathsf{const}\ c) & \\
\multicolumn{4}{l}{\quad \text{if } (\mathrm{bytes}_{\mathsf{i}K}(j) = z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} + k \cdot K/8 : K/8])^{k<M}} \\
\multicolumn{4}{l}{\quad \wedge\ c = \mathrm{lanes}^{-1}_{\mathsf{i}N\mathsf{x}M}(\mathrm{extend}^{sx}_{K,N}(j)^M) \wedge N = K \cdot 2}
\end{array}
$$

v128.load$N$_splat x ao

1. Let z be the current state.
2. Assert: Due to validation, a number value is on the top of the stack.
3. Pop the value $(at.\mathsf{const}\ i)$ from the stack.
4. If $i + ao.\mathsf{offset} + N/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|$, then:
   a. Trap.
5. Let M be $128/N$.
6. Let $\mathsf{i}N$ be the result for which $|\mathsf{i}N| = N$.
7. Let j be the result for which $\mathrm{bytes}_{\mathsf{i}N}(j) = z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : N/8]$.
8. Let c be $\mathrm{lanes}^{-1}_{\mathsf{i}N \times M}(j^M)$.
9. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$z; (at.\mathsf{const}\ i)\ (\mathsf{v128.load}N\mathsf{_splat}\ x\ ao) \quad \hookrightarrow \quad \mathsf{trap} \qquad \text{if } i + ao.\mathsf{offset} + N/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|$$

$$z; (at.\mathsf{const}\ i)\ (\mathsf{v128.load}N\mathsf{_splat}\ x\ ao) \quad \hookrightarrow \quad (\mathsf{v128.const}\ c) \qquad \text{if } \mathrm{bytes}_{\mathsf{i}N}(j) = z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : N/8] \wedge N = |\mathsf{i}N| \wedge M = 128/N \wedge c = \mathrm{lanes}^{-1}_{\mathsf{i}N \times M}(j^M)$$

v128.load$N$_zero x ao

1. Let z be the current state.
2. Assert: Due to validation, a number value is on the top of the stack.
3. Pop the value $(at.\mathsf{const}\ i)$ from the stack.
4. If $i + ao.\mathsf{offset} + N/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|$, then:
   a. Trap.
5. Let j be the result for which $\mathrm{bytes}_{\mathsf{i}N}(j) = z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : N/8]$.
6. Let c be $\mathrm{extend}^{\mathsf{u}}_{N,128}(j)$.
7. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$z; (at.\mathsf{const}\ i)\ (\mathsf{v128.load}N\mathsf{_zero}\ x\ ao) \quad \hookrightarrow \quad \mathsf{trap} \qquad \text{if } i + ao.\mathsf{offset} + N/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|$$

$$z; (at.\mathsf{const}\ i)\ (\mathsf{v128.load}N\mathsf{_zero}\ x\ ao) \quad \hookrightarrow \quad (\mathsf{v128.const}\ c) \qquad \text{if } \mathrm{bytes}_{\mathsf{i}N}(j) = z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : N/8] \wedge c = \mathrm{extend}^{\mathsf{u}}_{N,128}(j)$$

v128.load$N$_lane x ao j

1. Let z be the current state.
2. Assert: Due to validation, a value of vector type v128 is on the top of the stack.
3. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
4. Assert: Due to validation, a number value is on the top of the stack.
5. Pop the value $(at.\mathsf{const}\ i)$ from the stack.
6. If $i + ao.\mathsf{offset} + N/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|$, then:
   a. Trap.
7. Let M be $|\mathsf{v128}|/N$.
8. Let $\mathsf{i}N$ be the result for which $|\mathsf{i}N| = N$.

9. Let k be the result for which $\mathrm{bytes}_{\mathsf{i}N}(k) = z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : N/8]$.
10. Let c be $\mathrm{lanes}^{-1}_{\mathsf{i}N \times M}(\mathrm{lanes}_{\mathsf{i}N \times M}(c_1)[[j] = k])$.
11. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$
\begin{array}{lcll}
z; (at.\mathsf{const}\ i)\ (\mathsf{v128.const}\ c_1)\ (\mathsf{v128.load}N_\mathsf{lane}\ x\ ao\ j) & \hookrightarrow & \mathsf{trap} & \text{if } i + ao.\mathsf{offset} + N/8 > |z.\mathsf{mems}[x].\mathsf{bytes}| \\
z; (at.\mathsf{const}\ i)\ (\mathsf{v128.const}\ c_1)\ (\mathsf{v128.load}N_\mathsf{lane}\ x\ ao\ j) & \hookrightarrow & (\mathsf{v128.const}\ c) & \\
 & & \multicolumn{2}{l}{\text{if } \mathrm{bytes}_{\mathsf{i}N}(k) = z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : N/8]} \\
 & & \multicolumn{2}{l}{\wedge\ N = |\mathsf{i}N|} \\
 & & \multicolumn{2}{l}{\wedge\ M = |\mathsf{v128}|/N} \\
 & & \multicolumn{2}{l}{\wedge\ c = \mathrm{lanes}^{-1}_{\mathsf{i}N \times M}(\mathrm{lanes}_{\mathsf{i}N \times M}(c_1)[[j] = k])}
\end{array}
$$

$nt.\mathsf{store}storeop^?\ x\ ao$

1. Let z be the current state.
2. Assert: Due to validation, a number value is on the top of the stack.
3. Pop the value $(nt'.\mathsf{const}\ c)$ from the stack.
4. Assert: Due to validation, a value is on the top of the stack.
5. Pop the value $(at.\mathsf{const}\ i)$ from the stack.
6. Assert: Due to validation, $nt = nt'$.
7. If $storeop^?$ is not defined, then:
   a. If $i + ao.\mathsf{offset} + |nt'|/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|$, then:
      1) Trap.
   b. Let b^* be $\mathrm{bytes}_{nt'}(c)$.
   c. Replace $z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : |nt'|/8]$ with b^*.
8. Else:
   a. Assert: Due to validation, nt' is $\mathsf{i}N$.
   b. Let n be $storeop^?$.
   c. If $i + ao.\mathsf{offset} + n/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|$, then:
      1) Trap.
   d. Let b^* be $\mathrm{bytes}_{\mathsf{i}n}(\mathrm{wrap}_{|nt'|,n}(c))$.
   e. Replace $z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : n/8]$ with b^*.

$$
\begin{array}{lcll}
z; (at.\mathsf{const}\ i)\ (nt.\mathsf{const}\ c)\ (nt.\mathsf{store}\ x\ ao) & \hookrightarrow & z; \mathsf{trap} & \\
 & & \multicolumn{2}{l}{\text{if } i + ao.\mathsf{offset} + |nt|/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|} \\
z; (at.\mathsf{const}\ i)\ (nt.\mathsf{const}\ c)\ (nt.\mathsf{store}\ x\ ao) & \hookrightarrow & z[.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : |nt|/8] = b^*]; \epsilon & \\
 & & \multicolumn{2}{l}{\text{if } b^* = \mathrm{bytes}_{nt}(c)} \\
z; (at.\mathsf{const}\ i)\ (\mathsf{i}N.\mathsf{const}\ c)\ (\mathsf{i}N.\mathsf{store}n\ x\ ao) & \hookrightarrow & z; \mathsf{trap} & \\
 & & \multicolumn{2}{l}{\text{if } i + ao.\mathsf{offset} + n/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|} \\
z; (at.\mathsf{const}\ i)\ (\mathsf{i}N.\mathsf{const}\ c)\ (\mathsf{i}N.\mathsf{store}n\ x\ ao) & \hookrightarrow & z[.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : n/8] = b^*]; \epsilon & \\
 & & \multicolumn{2}{l}{\text{if } b^* = \mathrm{bytes}_{\mathsf{i}n}(\mathrm{wrap}_{|\mathsf{i}N|,n}(c))}
\end{array}
$$

$\mathsf{v128.store}\ x\ ao$

1. Let z be the current state.
2. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
3. Pop the value $(\mathsf{v128.const}\ c)$ from the stack.
4. Assert: Due to validation, a number value is on the top of the stack.
5. Pop the value $(at.\mathsf{const}\ i)$ from the stack.

6. If $i + ao.\mathsf{offset} + |\mathsf{v128}|/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|$, then:
   a. Trap.
7. Let b^* be $\mathrm{bytes}_{\mathsf{v128}}(c)$.
8. Replace $z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : |\mathsf{v128}|/8]$ with b^*.

$$z; (at.\mathsf{const}\ i)\ (\mathsf{v128.const}\ c)\ (\mathsf{v128.store}\ x\ ao) \quad \hookrightarrow \quad z; \mathsf{trap} \quad \text{if } i + ao.\mathsf{offset} + |\mathsf{v128}|/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|$$

$$z; (at.\mathsf{const}\ i)\ (\mathsf{v128.const}\ c)\ (\mathsf{v128.store}\ x\ ao) \quad \hookrightarrow \quad z[.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : |\mathsf{v128}|/8] = b^*]; \epsilon \quad \text{if } b^* = \mathrm{bytes}_{\mathsf{v128}}(c)$$

$\mathsf{v128.store}N\mathsf{_lane}\ x\ ao\ j$

1. Let z be the current state.
2. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
3. Pop the value $(\mathsf{v128.const}\ c)$ from the stack.
4. Assert: Due to validation, a number value is on the top of the stack.
5. Pop the value $(at.\mathsf{const}\ i)$ from the stack.
6. If $i + ao.\mathsf{offset} + N/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|$, then:
   a. Trap.
7. Let M be $128/N$.
8. Let $\mathsf{i}N$ be the result for which $|\mathsf{i}N| = N$.
9. Assert: Due to validation, $j < |\mathrm{lanes}_{\mathsf{i}N\mathsf{x}M}(c)|$.
10. Let b^* be $\mathrm{bytes}_{\mathsf{i}N}(\mathrm{lanes}_{\mathsf{i}N\mathsf{x}M}(c)[j])$.
11. Replace $z.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : N/8]$ with b^*.

$$z; (at.\mathsf{const}\ i)\ (\mathsf{v128.const}\ c)\ (\mathsf{v128.store}N\mathsf{_lane}\ x\ ao\ j) \quad \hookrightarrow \quad z; \mathsf{trap} \quad \text{if } i + ao.\mathsf{offset} + N/8 > |z.\mathsf{mems}[x].\mathsf{bytes}|$$

$$z; (at.\mathsf{const}\ i)\ (\mathsf{v128.const}\ c)\ (\mathsf{v128.store}N\mathsf{_lane}\ x\ ao\ j) \quad \hookrightarrow \quad z[.\mathsf{mems}[x].\mathsf{bytes}[i + ao.\mathsf{offset} : N/8] = b^*]; \epsilon \quad \text{if } N = |\mathsf{i}N| \wedge M = 128/N \wedge b^* = \mathrm{bytes}_{\mathsf{i}N}(\mathrm{lanes}_{\mathsf{i}N\mathsf{x}M}(c)[j])$$

$\mathsf{memory.size}\ x$

1. Let z be the current state.
2. Let $(at\ lim\ \mathsf{page})$ be the destructuring of $z.\mathsf{mems}[x].\mathsf{type}$.
3. Let $n \cdot 64\,\mathrm{Ki}$ be the length of $z.\mathsf{mems}[x].\mathsf{bytes}$.
4. Push the value $(at.\mathsf{const}\ n)$ to the stack.

$$z; (\mathsf{memory.size}\ x) \quad \hookrightarrow \quad (at.\mathsf{const}\ n) \quad \text{if } n \cdot 64\,\mathrm{Ki} = |z.\mathsf{mems}[x].\mathsf{bytes}| \wedge z.\mathsf{mems}[x].\mathsf{type} = at\ lim\ \mathsf{page}$$

$\mathsf{memory.grow}\ x$

1. Let z be the current state.
2. Assert: Due to validation, a number value is on the top of the stack.
3. Pop the value $(at.\mathsf{const}\ n)$ from the stack.
4. Either:
   a. Let mi be the memory instance $\mathrm{growmem}(z.\mathsf{mems}[x], n)$.

b. Push the value $(at.\mathsf{const}\ |z.\mathsf{mems}[x].\mathsf{bytes}|/(64\,\mathrm{Ki}))$ to the stack.

c. Replace $s.\mathsf{mems}[x]$ with mi.

5. Or:

a. Push the value $(at.\mathsf{const}\ \mathrm{signed}^{-1}_{|at|}(-1))$ to the stack.

$$
\begin{array}{lcll}
z; (at.\mathsf{const}\ n)\ (\mathsf{memory.grow}\ x) & \hookrightarrow & z[.\mathsf{mems}[x] = mi]; (at.\mathsf{const}\ |z.\mathsf{mems}[x].\mathsf{bytes}|/64\,\mathrm{Ki}) & \\
 & & \quad \text{if } mi = \mathrm{growmem}(z.\mathsf{mems}[x], n) & \\
z; (at.\mathsf{const}\ n)\ (\mathsf{memory.grow}\ x) & \hookrightarrow & z; (at.\mathsf{const}\ \mathrm{signed}^{-1}_{|at|}(-1)) &
\end{array}
$$

Note

The memory.grow instruction is non-deterministic. It may either succeed, returning the old memory size sz, or fail, returning -1. Failure *must* occur if the referenced memory instance has a maximum size defined that would be exceeded. However, failure *can* occur in other cases as well. In practice, the choice depends on the resources available to the embedder.

memory.fill x

1. Let z be the current state.
2. Assert: Due to validation, a number value is on the top of the stack.
3. Pop the value $(at.\mathsf{const}\ n)$ from the stack.
4. Assert: Due to validation, a value is on the top of the stack.
5. Pop the value val from the stack.
6. Assert: Due to validation, a value of number type at is on the top of the stack.
7. Pop the value $(numtype_0.\mathsf{const}\ i)$ from the stack.
8. If $i + n > |z.\mathsf{mems}[x].\mathsf{bytes}|$, then:
   a. Trap.
9. If $n = 0$, then:
   a. Do nothing.
10. Else:
    a. Push the value $(at.\mathsf{const}\ i)$ to the stack.
    b. Push the value val to the stack.
    c. Execute the instruction $(\mathsf{i32.store8}\ x)$.
    d. Push the value $(at.\mathsf{const}\ i + 1)$ to the stack.
    e. Push the value val to the stack.
    f. Push the value $(at.\mathsf{const}\ n - 1)$ to the stack.
    g. Execute the instruction $(\mathsf{memory.fill}\ x)$.

$$
\begin{array}{lcll}
z; (at.\mathsf{const}\ i)\ val\ (at.\mathsf{const}\ n)\ (\mathsf{memory.fill}\ x) & \hookrightarrow & \mathsf{trap} & \text{if } i + n > |z.\mathsf{mems}[x].\mathsf{bytes}| \\
z; (at.\mathsf{const}\ i)\ val\ (at.\mathsf{const}\ n)\ (\mathsf{memory.fill}\ x) & \hookrightarrow & \epsilon & \text{otherwise, if } n = 0 \\
z; (at.\mathsf{const}\ i)\ val\ (at.\mathsf{const}\ n)\ (\mathsf{memory.fill}\ x) & \hookrightarrow & & \\
\quad (at.\mathsf{const}\ i)\ val\ (\mathsf{i32.store8}\ x) & & & \text{otherwise} \\
\quad (at.\mathsf{const}\ i + 1)\ val\ (at.\mathsf{const}\ n - 1)\ (\mathsf{memory.fill}\ x) & & &
\end{array}
$$

memory.copy x_1 x_2

1. Let z be the current state.
2. Assert: Due to validation, a number value is on the top of the stack.
3. Pop the value $(at.\mathsf{const}\ n)$ from the stack.
4. Assert: Due to validation, a number value is on the top of the stack.
5. Pop the value $(at_2.\mathsf{const}\ i_2)$ from the stack.
6. Assert: Due to validation, a number value is on the top of the stack.
7. Pop the value $(at_1.\mathsf{const}\ i_1)$ from the stack.
8. If $i_1 + n > |z.\mathsf{mems}[x_1].\mathsf{bytes}|$, then:
   a. Trap.
9. If $i_2 + n > |z.\mathsf{mems}[x_2].\mathsf{bytes}|$, then:
   a. Trap.
10. If $n = 0$, then:
    a. Do nothing.
11. Else:
    a. If $i_1 \leq i_2$, then:
       1) Push the value $(at_1.\mathsf{const}\ i_1)$ to the stack.
       2) Push the value $(at_2.\mathsf{const}\ i_2)$ to the stack.
       3) Execute the instruction $(\mathsf{i32.load8_u}\ x_2)$.
       4) Execute the instruction $(\mathsf{i32.store8}\ x_1)$.
       5) Push the value $(at_1.\mathsf{const}\ i_1 + 1)$ to the stack.
       6) Push the value $(at_2.\mathsf{const}\ i_2 + 1)$ to the stack.
    b. Else:
       1) Push the value $(at_1.\mathsf{const}\ i_1 + n - 1)$ to the stack.
       2) Push the value $(at_2.\mathsf{const}\ i_2 + n - 1)$ to the stack.
       3) Execute the instruction $(\mathsf{i32.load8_u}\ x_2)$.
       4) Execute the instruction $(\mathsf{i32.store8}\ x_1)$.
       5) Push the value $(at_1.\mathsf{const}\ i_1)$ to the stack.
       6) Push the value $(at_2.\mathsf{const}\ i_2)$ to the stack.
    c. Push the value $(at.\mathsf{const}\ n - 1)$ to the stack.
    d. Execute the instruction $(\mathsf{memory.copy}\ x_1\ x_2)$.

$$z; (at_1.\mathsf{const}\ i_1)\ (at_2.\mathsf{const}\ i_2)\ (at'.\mathsf{const}\ n)\ (\mathsf{memory.copy}\ x_1\ x_2) \quad \hookrightarrow \quad \mathsf{trap} \quad \text{if } i_1 + n > |z.\mathsf{mems}[x_1].\mathsf{bytes}| \vee i_2 + n > |z.\mathsf{mems}[x_2].\mathsf{bytes}|$$

$$z; (at_1.\mathsf{const}\ i_1)\ (at_2.\mathsf{const}\ i_2)\ (at'.\mathsf{const}\ n)\ (\mathsf{memory.copy}\ x_1\ x_2) \quad \hookrightarrow \quad \epsilon \quad \text{otherwise, if } n = 0$$

$$z; (at_1.\mathsf{const}\ i_1)\ (at_2.\mathsf{const}\ i_2)\ (at'.\mathsf{const}\ n)\ (\mathsf{memory.copy}\ x_1\ x_2) \quad \hookrightarrow \quad (at_1.\mathsf{const}\ i_1)\ (at_2.\mathsf{const}\ i_2)\ (\mathsf{i32.load8_u}\ x_2)\ (\mathsf{i32.store8}\ x_1)\ (at_1.\mathsf{const}\ i_1 + 1)\ (at_2.\mathsf{const}\ i_2 + 1)\ (at'.\mathsf{const}\ n - 1)\ (\mathsf{memory.copy}\ x_1\ x_2) \quad \text{otherwise, if } i_1 \leq i_2$$

$$z; (at_1.\mathsf{const}\ i_1)\ (at_2.\mathsf{const}\ i_2)\ (at'.\mathsf{const}\ n)\ (\mathsf{memory.copy}\ x_1\ x_2) \quad \hookrightarrow \quad (at_1.\mathsf{const}\ i_1 + n - 1)\ (at_2.\mathsf{const}\ i_2 + n - 1)\ (\mathsf{i32.load8_u}\ x_2)\ (\mathsf{i32.store8}\ x_1)\ (at_1.\mathsf{const}\ i_1)\ (at_2.\mathsf{const}\ i_2)\ (at'.\mathsf{const}\ n - 1)\ (\mathsf{memory.copy}\ x_1\ x_2) \quad \text{otherwise}$$

memory.init x y

1. Let z be the current state.
2. Assert: Due to validation, a value of number type i32 is on the top of the stack.
3. Pop the value $(\text{i32.const } n)$ from the stack.
4. Assert: Due to validation, a value of number type i32 is on the top of the stack.
5. Pop the value $(\text{i32.const } j)$ from the stack.
6. Assert: Due to validation, a number value is on the top of the stack.
7. Pop the value $(at.\text{const } i)$ from the stack.
8. If $i + n > |z.\text{mems}[x].\text{bytes}|$, then:
   a. Trap.
9. If $j + n > |z.\text{datas}[y].\text{bytes}|$, then:
   a. Trap.
10. If $n = 0$, then:
    a. Do nothing.
11. Else:
    a. Assert: Due to validation, $j < |z.\text{datas}[y].\text{bytes}|$.
    b. Push the value $(at.\text{const } i)$ to the stack.
    c. Push the value $(\text{i32.const } z.\text{datas}[y].\text{bytes}[j])$ to the stack.
    d. Execute the instruction $(\text{i32.store8 } x)$.
    e. Push the value $(at.\text{const } i + 1)$ to the stack.
    f. Push the value $(\text{i32.const } j + 1)$ to the stack.
    g. Push the value $(\text{i32.const } n - 1)$ to the stack.
    h. Execute the instruction $(\text{memory.init } x\ y)$.

$$
\begin{array}{lcll}
z; (at.\text{const } i)\ (\text{i32.const } j)\ (\text{i32.const } n)\ (\text{memory.init } x\ y) & \hookrightarrow & \text{trap} & \\
\quad \text{if } i + n > |z.\text{mems}[x].\text{bytes}| \vee j + n > |z.\text{datas}[y].\text{bytes}| & & & \\
z; (at.\text{const } i)\ (\text{i32.const } j)\ (\text{i32.const } n)\ (\text{memory.init } x\ y) & \hookrightarrow & \epsilon & \text{otherwise, if } n = 0 \\
z; (at.\text{const } i)\ (\text{i32.const } j)\ (\text{i32.const } n)\ (\text{memory.init } x\ y) & \hookrightarrow & & \\
\quad (at.\text{const } i)\ (\text{i32.const } z.\text{datas}[y].\text{bytes}[j])\ (\text{i32.store8 } x) & & & \text{otherwise} \\
\quad (at.\text{const } i + 1)\ (\text{i32.const } j + 1)\ (\text{i32.const } n - 1)\ (\text{memory.init } x\ y) & & &
\end{array}
$$

data.drop x

1. Let z be the current state.
2. Replace $z.\text{datas}[x].$bytes with ϵ.

$$z; (\text{data.drop } x) \quad \hookrightarrow \quad z[.\text{datas}[x].\text{bytes} = \epsilon]; \epsilon$$

4.6.9 Reference Instructions

ref.null ht

1. Push the value ref.null to the stack.

$$z; (\text{ref.null } ht) \quad \hookrightarrow \quad \text{ref.null}$$

ref.func x

1. Let z be the current state.
2. Assert: Due to validation, $x < |z.\mathsf{module.funcs}|$.
3. Push the value $(\mathsf{ref.func}\ z.\mathsf{module.funcs}[x])$ to the stack.

$$z; (\mathsf{ref.func}\ x) \quad \hookrightarrow \quad (\mathsf{ref.func}\ z.\mathsf{module.funcs}[x])$$

ref.is_null

1. Assert: Due to validation, a reference value is on the top of the stack.
2. Pop the value ref from the stack.
3. If $ref = \mathsf{ref.null}$, then:
   a. Push the value $(\mathsf{i32.const}\ 1)$ to the stack.
4. Else:
   a. Push the value $(\mathsf{i32.const}\ 0)$ to the stack.

$$\begin{array}{lcll} ref\ \mathsf{ref.is_null} & \hookrightarrow & (\mathsf{i32.const}\ 1) & \text{if } ref = \mathsf{ref.null} \\ ref\ \mathsf{ref.is_null} & \hookrightarrow & (\mathsf{i32.const}\ 0) & \text{otherwise} \end{array}$$

ref.as_non_null

1. Assert: Due to validation, a reference value is on the top of the stack.
2. Pop the value ref from the stack.
3. If $ref = \mathsf{ref.null}$, then:
   a. Trap.
4. Push the value ref to the stack.

$$\begin{array}{lcll} ref\ \mathsf{ref.as_non_null} & \hookrightarrow & \mathsf{trap} & \text{if } ref = \mathsf{ref.null} \\ ref\ \mathsf{ref.as_non_null} & \hookrightarrow & ref & \text{otherwise} \end{array}$$

ref.eq

1. Assert: Due to validation, a reference value is on the top of the stack.
2. Pop the value ref_2 from the stack.
3. Assert: Due to validation, a reference value is on the top of the stack.
4. Pop the value ref_1 from the stack.
5. If $ref_1 = \mathsf{ref.null}$ and $ref_2 = \mathsf{ref.null}$, then:
   a. Push the value $(\mathsf{i32.const}\ 1)$ to the stack.
6. Else if $ref_1 = ref_2$, then:
   a. Push the value $(\mathsf{i32.const}\ 1)$ to the stack.
7. Else:
   a. Push the value $(\mathsf{i32.const}\ 0)$ to the stack.

$$\begin{array}{lcll} ref_1\ ref_2\ \mathsf{ref.eq} & \hookrightarrow & (\mathsf{i32.const}\ 1) & \text{if } ref_1 = \mathsf{ref.null} \wedge ref_2 = \mathsf{ref.null} \\ ref_1\ ref_2\ \mathsf{ref.eq} & \hookrightarrow & (\mathsf{i32.const}\ 1) & \text{otherwise, if } ref_1 = ref_2 \\ ref_1\ ref_2\ \mathsf{ref.eq} & \hookrightarrow & (\mathsf{i32.const}\ 0) & \text{otherwise} \end{array}$$

ref.test rt

1. Let f be the topmost frame.
2. Assert: Due to validation, a reference value is on the top of the stack.
3. Pop the value ref from the stack.
4. If ref is valid with type $\mathrm{clos}_{f.\mathsf{module}}(rt)$, then:
   a. Push the value $(\mathsf{i32.const}\ 1)$ to the stack.
5. Else:
   a. Push the value $(\mathsf{i32.const}\ 0)$ to the stack.

$$\begin{array}{lcll} s; f; ref\ (\mathsf{ref.test}\ rt) & \hookrightarrow & (\mathsf{i32.const}\ 1) & \text{if } s \vdash ref : \mathrm{clos}_{f.\mathsf{module}}(rt) \\ s; f; ref\ (\mathsf{ref.test}\ rt) & \hookrightarrow & (\mathsf{i32.const}\ 0) & \text{otherwise} \end{array}$$

ref.cast rt

1. Let f be the topmost frame.
2. Assert: Due to validation, a reference value is on the top of the stack.
3. Pop the value ref from the stack.
4. If not ref is valid with type $\mathrm{clos}_{f.\mathsf{module}}(rt)$, then:
   a. Trap.
5. Push the value ref to the stack.

$$\begin{array}{lcll} s; f; ref\ (\mathsf{ref.cast}\ rt) & \hookrightarrow & ref & \text{if } s \vdash ref : \mathrm{clos}_{f.\mathsf{module}}(rt) \\ s; f; ref\ (\mathsf{ref.cast}\ rt) & \hookrightarrow & \mathsf{trap} & \text{otherwise} \end{array}$$

ref.i31

1. Assert: Due to validation, a value of number type i32 is on the top of the stack.
2. Pop the value $(\mathsf{i32.const}\ i)$ from the stack.
3. Push the value $(\mathsf{ref.i31}\ \mathrm{wrap}_{32,31}(i))$ to the stack.

$$(\mathsf{i32.const}\ i)\ \mathsf{ref.i31} \quad \hookrightarrow \quad (\mathsf{ref.i31}\ \mathrm{wrap}_{32,31}(i))$$

i31.get_sx

1. Assert: Due to validation, a value is on the top of the stack.
2. Pop the value val from the stack.
3. If $val = \mathsf{ref.null}$, then:
   a. Trap.
4. Assert: Due to validation, val is some $\mathsf{ref.i31}\ u31$.
5. Let $(\mathsf{ref.i31}\ i)$ be the destructuring of val.
6. Push the value $(\mathsf{i32.const}\ \mathrm{extend}^{sx}_{31,32}(i))$ to the stack.

$$\begin{array}{lcl} (\mathsf{ref.null})\ (\mathsf{i31.get_}sx) & \hookrightarrow & \mathsf{trap} \\ (\mathsf{ref.i31}\ i)\ (\mathsf{i31.get_}sx) & \hookrightarrow & (\mathsf{i32.const}\ \mathrm{extend}^{sx}_{31,32}(i)) \end{array}$$

struct.new x

1. Let z be the current state.
2. Assert: Due to validation, the expansion of $z.\mathsf{types}[x]$ is some struct $list(fieldtype)$.
3. Let (struct $list_0$) be the destructuring of the expansion of $z.\mathsf{types}[x]$.
4. Let $(\mathsf{mut}^?\ zt)^n$ be $list_0$.
5. Let a be the length of $z.\mathsf{structs}$.
6. Assert: Due to validation, there are at least n values on the top of the stack.
7. Pop the values val^n from the stack.
8. Let si be the structure instance $\{\mathsf{type}\ z.\mathsf{types}[x],\ \mathsf{fields}\ \mathrm{pack}_{zt}(val)^n\}$.
9. Push the value $(\mathsf{ref.struct}\ a)$ to the stack.
10. Append si to $s.\mathsf{structs}$.

$$z;\ val^n\ (\mathsf{struct.new}\ x) \quad \hookrightarrow \quad z[.\mathsf{structs} =\oplus si];\ (\mathsf{ref.struct}\ a) \qquad \begin{array}{l}\text{if } z.\mathsf{types}[x] \approx \mathsf{struct}\ (\mathsf{mut}^?\ zt)^n \\ \wedge\ a = |z.\mathsf{structs}| \\ \wedge\ si = \{\mathsf{type}\ z.\mathsf{types}[x],\ \mathsf{fields}\ (\mathrm{pack}_{zt}(val))^n\}\end{array}$$

struct.new_default x

1. Let z be the current state.
2. Assert: Due to validation, the expansion of $z.\mathsf{types}[x]$ is some struct $list(fieldtype)$.
3. Let (struct $list_0$) be the destructuring of the expansion of $z.\mathsf{types}[x]$.
4. Let $(\mathsf{mut}^?\ zt)^*$ be $list_0$.
5. Assert: Due to validation, for all zt in zt^*, $\mathrm{default}_{\mathrm{unpack}(zt)}$ is defined.
6. Let val^* be the value sequence ϵ.
7. For each zt in zt^*, do:
   a. Let val be $\mathrm{default}_{\mathrm{unpack}(zt)}$.
   b. Append val to val^*.
8. Assert: Due to validation, $|val^*| = |zt^*|$.
9. Push the values val^* to the stack.
10. Execute the instruction $(\mathsf{struct.new}\ x)$.

$$z;\ (\mathsf{struct.new_default}\ x) \quad \hookrightarrow \quad val^*\ (\mathsf{struct.new}\ x) \qquad \begin{array}{l}\text{if } z.\mathsf{types}[x] \approx \mathsf{struct}\ (\mathsf{mut}^?\ zt)^* \\ \wedge\ (\mathrm{default}_{\mathrm{unpack}(zt)} = val)^*\end{array}$$

struct.get_$sx^?$ x i

1. Let z be the current state.
2. Assert: Due to validation, a value is on the top of the stack.
3. Pop the value val from the stack.
4. If $val = \mathsf{ref.null}$, then:
   a. Trap.
5. Assert: Due to validation, val is some $\mathsf{ref.struct}\ structaddr$.
6. Let $(\mathsf{ref.struct}\ a)$ be the destructuring of val.
7. Assert: Due to validation, $i < |z.\mathsf{structs}[a].\mathsf{fields}|$.
8. Assert: Due to validation, $a < |z.\mathsf{structs}|$.

9. Assert: Due to validation, the expansion of $z.\mathsf{types}[x]$ is some $\mathsf{struct}\ \mathit{list}(\mathit{fieldtype})$.
10. Let $(\mathsf{struct}\ \mathit{list}_0)$ be the destructuring of the expansion of $z.\mathsf{types}[x]$.
11. Let $(\mathsf{mut}^?\ zt)^*$ be list_0.
12. Assert: Due to validation, $i < |zt^*|$.
13. Push the value $\mathrm{unpack}^{sx^?}_{zt^*[i]}(z.\mathsf{structs}[a].\mathsf{fields}[i])$ to the stack.

$$\begin{array}{rcl} z;(\mathsf{ref.null})\ (\mathsf{struct.get_}sx^?\ x\ i) & \hookrightarrow & \mathsf{trap} \\ z;(\mathsf{ref.struct}\ a)\ (\mathsf{struct.get_}sx^?\ x\ i) & \hookrightarrow & \mathrm{unpack}^{sx^?}_{zt^*[i]}(z.\mathsf{structs}[a].\mathsf{fields}[i]) \\ & & \quad \text{if } z.\mathsf{types}[x] \approx \mathsf{struct}\ (\mathsf{mut}^?\ zt)^* \end{array}$$

$\mathsf{struct.set}\ x\ i$

1. Let z be the current state.
2. Assert: Due to validation, a value is on the top of the stack.
3. Pop the value val from the stack.
4. Assert: Due to validation, a value is on the top of the stack.
5. Pop the value val' from the stack.
6. If $\mathit{val}' = \mathsf{ref.null}$, then:
   a. Trap.
7. Assert: Due to validation, val' is some $\mathsf{ref.struct}\ \mathit{structaddr}$.
8. Let $(\mathsf{ref.struct}\ a)$ be the destructuring of val'.
9. Assert: Due to validation, the expansion of $z.\mathsf{types}[x]$ is some $\mathsf{struct}\ \mathit{list}(\mathit{fieldtype})$.
10. Let $(\mathsf{struct}\ \mathit{list}_0)$ be the destructuring of the expansion of $z.\mathsf{types}[x]$.
11. Let $(\mathsf{mut}^?\ zt)^*$ be list_0.
12. Assert: Due to validation, $i < |zt^*|$.
13. Replace $s.\mathsf{structs}[a].\mathsf{fields}[i]$ with $\mathrm{pack}_{zt^*[i]}(\mathit{val})$.

$$\begin{array}{rcl} z;(\mathsf{ref.null})\ \mathit{val}\ (\mathsf{struct.set}\ x\ i) & \hookrightarrow & z;\mathsf{trap} \\ z;(\mathsf{ref.struct}\ a)\ \mathit{val}\ (\mathsf{struct.set}\ x\ i) & \hookrightarrow & z[.\mathsf{structs}[a].\mathsf{fields}[i] = \mathrm{pack}_{zt^*[i]}(\mathit{val})];\epsilon \\ & & \quad \text{if } z.\mathsf{types}[x] \approx \mathsf{struct}\ (\mathsf{mut}^?\ zt)^* \end{array}$$

$\mathsf{array.new}\ x$

1. Assert: Due to validation, a value of number type $\mathsf{i32}$ is on the top of the stack.
2. Pop the value $(\mathsf{i32.const}\ n)$ from the stack.
3. Assert: Due to validation, a value is on the top of the stack.
4. Pop the value val from the stack.
5. Push the values val^n to the stack.
6. Execute the instruction $(\mathsf{array.new_fixed}\ x\ n)$.

$$\mathit{val}\ (\mathsf{i32.const}\ n)\ (\mathsf{array.new}\ x) \quad \hookrightarrow \quad \mathit{val}^n\ (\mathsf{array.new_fixed}\ x\ n)$$

array.new_default x

1. Let z be the current state.
2. Assert: Due to validation, a value of number type i32 is on the top of the stack.
3. Pop the value $(\text{i32.const}\ n)$ from the stack.
4. Assert: Due to validation, the expansion of $z.\text{types}[x]$ is some array $\mathit{fieldtype}$.
5. Let $(\text{array}\ \mathit{fieldtype}_0)$ be the destructuring of the expansion of $z.\text{types}[x]$.
6. Let $(\text{mut}^?\ zt)$ be the destructuring of $\mathit{fieldtype}_0$.
7. Assert: Due to validation, $\text{default}_{\text{unpack}(zt)}$ is defined.
8. Let val be $\text{default}_{\text{unpack}(zt)}$.
9. Push the values val^n to the stack.
10. Execute the instruction $(\text{array.new_fixed}\ x\ n)$.

$$z; (\text{i32.const}\ n)\ (\text{array.new_default}\ x) \quad \hookrightarrow \quad \mathit{val}^n\ (\text{array.new_fixed}\ x\ n) \quad \begin{array}{l}\text{if } z.\text{types}[x] \approx \text{array}\ (\text{mut}^?\ zt) \\ \wedge\ \text{default}_{\text{unpack}(zt)} = \mathit{val}\end{array}$$

array.new_fixed x n

1. Let z be the current state.
2. Assert: Due to validation, the expansion of $z.\text{types}[x]$ is some array $\mathit{fieldtype}$.
3. Let $(\text{array}\ \mathit{fieldtype}_0)$ be the destructuring of the expansion of $z.\text{types}[x]$.
4. Let $(\text{mut}^?\ zt)$ be the destructuring of $\mathit{fieldtype}_0$.
5. Let a be the length of $z.\text{arrays}$.
6. Assert: Due to validation, there are at least n values on the top of the stack.
7. Pop the values val^n from the stack.
8. Let ai be the array instance $\{\text{type}\ z.\text{types}[x],\ \text{fields}\ \text{pack}_{zt}(\mathit{val})^n\}$.
9. Push the value $(\text{ref.array}\ a)$ to the stack.
10. Append ai to $s.\text{arrays}$.

$$z; \mathit{val}^n\ (\text{array.new_fixed}\ x\ n) \quad \hookrightarrow \quad z[.\text{arrays} =\!\oplus\ ai]; (\text{ref.array}\ a) \quad \begin{array}{l}\text{if } z.\text{types}[x] \approx \text{array}\ (\text{mut}^?\ zt) \\ \wedge\ a = |z.\text{arrays}| \wedge ai = \{\text{type}\ z.\text{types}[x],\ \text{fields}\ (\text{pack}_{zt}(\mathit{val}))^n\}\end{array}$$

array.new_data x y

1. Let z be the current state.
2. Assert: Due to validation, a value of number type i32 is on the top of the stack.
3. Pop the value $(\text{i32.const}\ n)$ from the stack.
4. Assert: Due to validation, a value of number type i32 is on the top of the stack.
5. Pop the value $(\text{i32.const}\ i)$ from the stack.
6. Assert: Due to validation, the expansion of $z.\text{types}[x]$ is some array $\mathit{fieldtype}$.
7. Let $(\text{array}\ \mathit{fieldtype}_0)$ be the destructuring of the expansion of $z.\text{types}[x]$.
8. Let $(\text{mut}^?\ zt)$ be the destructuring of $\mathit{fieldtype}_0$.
9. If $i + n \cdot |zt|/8 > |z.\text{datas}[y].\text{bytes}|$, then:
   a. Trap.

10. Let $byte^{**}$ be the result for which each $byte^{*}$ has length $|zt|/8$, and the concatenation of $byte^{**}$ is $z.\mathsf{datas}[y].\mathsf{bytes}[i : n \cdot |zt|/8]$.
11. Let c^n be the result for which $(\mathrm{bytes}_{zt}(c^n) = byte^{*})^{*}$.
12. Push the values $\mathrm{unpack}(zt).\mathsf{const}\ \mathrm{unpack}_{zt}(c)^n$ to the stack.
13. Execute the instruction $(\mathsf{array.new_fixed}\ x\ n)$.

$$z; (\mathsf{i32.const}\ i)\ (\mathsf{i32.const}\ n)\ (\mathsf{array.new_data}\ x\ y) \quad \hookrightarrow \quad \mathsf{trap} \quad \text{if } z.\mathsf{types}[x] \approx \mathsf{array}\ (\mathsf{mut}^?\ zt) \wedge i + n \cdot |zt|/8 > |z.\mathsf{datas}[y].\mathsf{bytes}|$$

$$z; (\mathsf{i32.const}\ i)\ (\mathsf{i32.const}\ n)\ (\mathsf{array.new_data}\ x\ y) \quad \hookrightarrow \quad (\mathrm{unpack}(zt).\mathsf{const}\ \mathrm{unpack}_{zt}(c))^n\ (\mathsf{array.new_fixed}\ x\ n) \quad \text{if } z.\mathsf{types}[x] \approx \mathsf{array}\ (\mathsf{mut}^?\ zt) \wedge \bigoplus \mathrm{bytes}_{zt}(c)^n = z.\mathsf{datas}[y].\mathsf{bytes}[i : n \cdot |zt|/8]$$

$\mathsf{array.new_elem}\ x\ y$

1. Let z be the current state.
2. Assert: Due to validation, a value of number type $\mathsf{i32}$ is on the top of the stack.
3. Pop the value $(\mathsf{i32.const}\ n)$ from the stack.
4. Assert: Due to validation, a value of number type $\mathsf{i32}$ is on the top of the stack.
5. Pop the value $(\mathsf{i32.const}\ i)$ from the stack.
6. If $i + n > |z.\mathsf{elems}[y].\mathsf{refs}|$, then:
   a. Trap.
7. Let ref^n be $z.\mathsf{elems}[y].\mathsf{refs}[i : n]$.
8. Push the values ref^n to the stack.
9. Execute the instruction $(\mathsf{array.new_fixed}\ x\ n)$.

$$z; (\mathsf{i32.const}\ i)\ (\mathsf{i32.const}\ n)\ (\mathsf{array.new_elem}\ x\ y) \quad \hookrightarrow \quad \mathsf{trap} \quad \text{if } i + n > |z.\mathsf{elems}[y].\mathsf{refs}|$$

$$z; (\mathsf{i32.const}\ i)\ (\mathsf{i32.const}\ n)\ (\mathsf{array.new_elem}\ x\ y) \quad \hookrightarrow \quad ref^n\ (\mathsf{array.new_fixed}\ x\ n) \quad \text{if } ref^n = z.\mathsf{elems}[y].\mathsf{refs}[i : n]$$

$\mathsf{array.get_}sx^?\ x$

1. Let z be the current state.
2. Assert: Due to validation, a value of number type $\mathsf{i32}$ is on the top of the stack.
3. Pop the value $(\mathsf{i32.const}\ i)$ from the stack.
4. Assert: Due to validation, a value is on the top of the stack.
5. Pop the value val from the stack.
6. If $val = \mathsf{ref.null}$, then:
   a. Trap.
7. Assert: Due to validation, val is some $\mathsf{ref.array}\ arrayaddr$.
8. Let $(\mathsf{ref.array}\ a)$ be the destructuring of val.
9. Assert: Due to validation, $a < |z.\mathsf{arrays}|$.
10. If $i \geq |z.\mathsf{arrays}[a].\mathsf{fields}|$, then:
    a. Trap.
11. Assert: Due to validation, the expansion of $z.\mathsf{types}[x]$ is some $\mathsf{array}\ fieldtype$.
12. Let $(\mathsf{array}\ fieldtype_0)$ be the destructuring of the expansion of $z.\mathsf{types}[x]$.
13. Let $(\mathsf{mut}^?\ zt)$ be the destructuring of $fieldtype_0$.

14. Push the value $\mathrm{unpack}_{zt}^{sx^?}(z.\mathsf{arrays}[a].\mathsf{fields}[i])$ to the stack.

$$
\begin{array}{rcll}
z; (\mathsf{ref.null})\ (\mathsf{i32.const}\ i)\ (\mathsf{array.get_}sx^?\ x) & \hookrightarrow & \mathsf{trap} & \\
z; (\mathsf{ref.array}\ a)\ (\mathsf{i32.const}\ i)\ (\mathsf{array.get_}sx^?\ x) & \hookrightarrow & \mathsf{trap} & \text{if } i \geq |z.\mathsf{arrays}[a].\mathsf{fields}| \\
z; (\mathsf{ref.array}\ a)\ (\mathsf{i32.const}\ i)\ (\mathsf{array.get_}sx^?\ x) & \hookrightarrow & \mathrm{unpack}_{zt}^{sx^?}(z.\mathsf{arrays}[a].\mathsf{fields}[i]) & \\
 & & \text{if } z.\mathsf{types}[x] \approx \mathsf{array}\ (\mathsf{mut}^?\ zt) &
\end{array}
$$

$\mathsf{array.set}\ x$

1. Let z be the current state.
2. Assert: Due to validation, a value is on the top of the stack.
3. Pop the value val from the stack.
4. Assert: Due to validation, a value of number type $\mathsf{i32}$ is on the top of the stack.
5. Pop the value $(\mathsf{i32.const}\ i)$ from the stack.
6. Assert: Due to validation, a value is on the top of the stack.
7. Pop the value val' from the stack.
8. If $val' = \mathsf{ref.null}$, then:
   a. Trap.
9. Assert: Due to validation, val' is some $\mathsf{ref.array}\ arrayaddr$.
10. Let $(\mathsf{ref.array}\ a)$ be the destructuring of val'.
11. If $a < |z.\mathsf{arrays}|$ and $i \geq |z.\mathsf{arrays}[a].\mathsf{fields}|$, then:
    a. Trap.
12. Assert: Due to validation, the expansion of $z.\mathsf{types}[x]$ is some $\mathsf{array}\ fieldtype$.
13. Let $(\mathsf{array}\ fieldtype_0)$ be the destructuring of the expansion of $z.\mathsf{types}[x]$.
14. Let $(\mathsf{mut}^?\ zt)$ be the destructuring of $fieldtype_0$.
15. Replace $s.\mathsf{arrays}[a].\mathsf{fields}[i]$ with $\mathrm{pack}_{zt}(val)$.

$$
\begin{array}{rcll}
z; (\mathsf{ref.null})\ (\mathsf{i32.const}\ i)\ val\ (\mathsf{array.set}\ x) & \hookrightarrow & z; \mathsf{trap} & \\
z; (\mathsf{ref.array}\ a)\ (\mathsf{i32.const}\ i)\ val\ (\mathsf{array.set}\ x) & \hookrightarrow & z; \mathsf{trap} & \text{if } i \geq |z.\mathsf{arrays}[a].\mathsf{fields}| \\
z; (\mathsf{ref.array}\ a)\ (\mathsf{i32.const}\ i)\ val\ (\mathsf{array.set}\ x) & \hookrightarrow & z[.\mathsf{arrays}[a].\mathsf{fields}[i] = \mathrm{pack}_{zt}(val)]; \epsilon & \\
 & & \text{if } z.\mathsf{types}[x] \approx \mathsf{array}\ (\mathsf{mut}^?\ zt) &
\end{array}
$$

$\mathsf{array.len}$

1. Let z be the current state.
2. Assert: Due to validation, a value is on the top of the stack.
3. Pop the value val from the stack.
4. If $val = \mathsf{ref.null}$, then:
   a. Trap.
5. Assert: Due to validation, val is some $\mathsf{ref.array}\ arrayaddr$.
6. Let $(\mathsf{ref.array}\ a)$ be the destructuring of val.
7. Assert: Due to validation, $a < |z.\mathsf{arrays}|$.
8. Push the value $(\mathsf{i32.const}\ |z.\mathsf{arrays}[a].\mathsf{fields}|)$ to the stack.

$$
\begin{array}{rcl}
z; (\mathsf{ref.null})\ \mathsf{array.len} & \hookrightarrow & \mathsf{trap} \\
z; (\mathsf{ref.array}\ a)\ \mathsf{array.len} & \hookrightarrow & (\mathsf{i32.const}\ |z.\mathsf{arrays}[a].\mathsf{fields}|)
\end{array}
$$

array.fill x

1. Let z be the current state.
2. Assert: Due to validation, a value of number type i32 is on the top of the stack.
3. Pop the value $(\text{i32.const } n)$ from the stack.
4. Assert: Due to validation, a value is on the top of the stack.
5. Pop the value val from the stack.
6. Assert: Due to validation, a value of number type i32 is on the top of the stack.
7. Pop the value $(\text{i32.const } i)$ from the stack.
8. Assert: Due to validation, a value is on the top of the stack.
9. Pop the value val' from the stack.
10. If $val' = \text{ref.null}$, then:
    a. Trap.
11. Assert: Due to validation, val' is some ref.array $arrayaddr$.
12. Let $(\text{ref.array } a)$ be the destructuring of val'.
13. If $a \geq |z.\text{arrays}|$, then:
    a. Do nothing.
14. Else if $i + n > |z.\text{arrays}[a].\text{fields}|$, then:
    a. Trap.
15. If $n = 0$, then:
    a. Do nothing.
16. Else:
    a. Push the value $(\text{ref.array } a)$ to the stack.
    b. Push the value $(\text{i32.const } i)$ to the stack.
    c. Push the value val to the stack.
    d. Execute the instruction $(\text{array.set } x)$.
    e. Push the value $(\text{ref.array } a)$ to the stack.
    f. Push the value $(\text{i32.const } i + 1)$ to the stack.
    g. Push the value val to the stack.
    h. Push the value $(\text{i32.const } n - 1)$ to the stack.
    i. Execute the instruction $(\text{array.fill } x)$.

$$
\begin{array}{rcll}
z; (\text{ref.null})\ (\text{i32.const } i)\ val\ (\text{i32.const } n)\ (\text{array.fill } x) & \hookrightarrow & \text{trap} & \\
z; (\text{ref.array } a)\ (\text{i32.const } i)\ val\ (\text{i32.const } n)\ (\text{array.fill } x) & \hookrightarrow & \text{trap} & \text{if } i + n > |z.\text{arrays}[a].\text{fields}| \\
z; (\text{ref.array } a)\ (\text{i32.const } i)\ val\ (\text{i32.const } n)\ (\text{array.fill } x) & \hookrightarrow & \epsilon & \text{otherwise, if } n = 0 \\
z; (\text{ref.array } a)\ (\text{i32.const } i)\ val\ (\text{i32.const } n)\ (\text{array.fill } x) & \hookrightarrow & & \\
\quad (\text{ref.array } a)\ (\text{i32.const } i)\ val\ (\text{array.set } x) & & & \text{otherwise} \\
\quad (\text{ref.array } a)\ (\text{i32.const } i + 1)\ val\ (\text{i32.const } n - 1)\ (\text{array.fill } x) & & &
\end{array}
$$

array.copy x_1 x_2

1. Let z be the current state.
2. Assert: Due to validation, a value of number type i32 is on the top of the stack.
3. Pop the value $(\text{i32.const } n)$ from the stack.

4. Assert: Due to validation, a value of number type i32 is on the top of the stack.
5. Pop the value $(\mathsf{i32.const}\ i_2)$ from the stack.
6. Assert: Due to validation, a value is on the top of the stack.
7. Pop the value val from the stack.
8. Assert: Due to validation, a value of number type i32 is on the top of the stack.
9. Pop the value $(\mathsf{i32.const}\ i_1)$ from the stack.
10. Assert: Due to validation, a value is on the top of the stack.
11. Pop the value val' from the stack.
12. If $val' = \mathsf{ref.null}$ and val is reference value, then:
    a. Trap.
13. If $val = \mathsf{ref.null}$ and val' is reference value, then:
    a. Trap.
14. If val' is some $\mathsf{ref.array}\ arrayaddr$, then:
    a. Let $(\mathsf{ref.array}\ a_1)$ be the destructuring of val'.
    b. If val is some $\mathsf{ref.array}\ arrayaddr$, then:
        1) If $a_1 < |z.\mathsf{arrays}|$ and $i_1 + n > |z.\mathsf{arrays}[a_1].\mathsf{fields}|$, then:
            a) Trap.
        2) Let $(\mathsf{ref.array}\ a_2)$ be the destructuring of val.
        3) If $a_2 \geq |z.\mathsf{arrays}|$, then:
            a) Do nothing.
        4) Else if $i_2 + n > |z.\mathsf{arrays}[a_2].\mathsf{fields}|$, then:
            a) Trap.
        5) If $n = 0$, then:
            a) Do nothing.
        6) Else:
            a) Assert: Due to validation, the expansion of $z.\mathsf{types}[x_2]$ is some $\mathsf{array}\ fieldtype$.
            b) Let $(\mathsf{array}\ fieldtype_0)$ be the destructuring of the expansion of $z.\mathsf{types}[x_2]$.
            c) Let $(\mathsf{mut}^?\ zt_2)$ be the destructuring of $fieldtype_0$.
            d) Let $sx^?$ be $sx(zt_2)$.
            e) Push the value $(\mathsf{ref.array}\ a_1)$ to the stack.
            f) If $i_1 \leq i_2$, then:
                1. Push the value $(\mathsf{i32.const}\ i_1)$ to the stack.
                2. Push the value $(\mathsf{ref.array}\ a_2)$ to the stack.
                3. Push the value $(\mathsf{i32.const}\ i_2)$ to the stack.
                4. Execute the instruction $(\mathsf{array.get_}sx^?\ x_2)$.
                5. Execute the instruction $(\mathsf{array.set}\ x_1)$.
                6. Push the value $(\mathsf{ref.array}\ a_1)$ to the stack.
                7. Push the value $(\mathsf{i32.const}\ i_1 + 1)$ to the stack.
                8. Push the value $(\mathsf{ref.array}\ a_2)$ to the stack.

9. Push the value $(\mathsf{i32.const}\ i_2 + 1)$ to the stack.

g) Else:

1. Push the value $(\mathsf{i32.const}\ i_1 + n - 1)$ to the stack.
2. Push the value $(\mathsf{ref.array}\ a_2)$ to the stack.
3. Push the value $(\mathsf{i32.const}\ i_2 + n - 1)$ to the stack.
4. Execute the instruction $(\mathsf{array.get_}sx^?\ x_2)$.
5. Execute the instruction $(\mathsf{array.set}\ x_1)$.
6. Push the value $(\mathsf{ref.array}\ a_1)$ to the stack.
7. Push the value $(\mathsf{i32.const}\ i_1)$ to the stack.
8. Push the value $(\mathsf{ref.array}\ a_2)$ to the stack.
9. Push the value $(\mathsf{i32.const}\ i_2)$ to the stack.

h) Push the value $(\mathsf{i32.const}\ n - 1)$ to the stack.

i) Execute the instruction $(\mathsf{array.copy}\ x_1\ x_2)$.

$$
\begin{array}{rcl}
z; (\mathsf{ref.null})\ (\mathsf{i32.const}\ i_1)\ ref\ (\mathsf{i32.const}\ i_2)\ (\mathsf{i32.const}\ n)\ (\mathsf{array.copy}\ x_1\ x_2) & \hookrightarrow & \mathsf{trap} \\
z; ref\ (\mathsf{i32.const}\ i_1)\ (\mathsf{ref.null})\ (\mathsf{i32.const}\ i_2)\ (\mathsf{i32.const}\ n)\ (\mathsf{array.copy}\ x_1\ x_2) & \hookrightarrow & \mathsf{trap} \\
z; (\mathsf{ref.array}\ a_1)\ (\mathsf{i32.const}\ i_1)\ (\mathsf{ref.array}\ a_2)\ (\mathsf{i32.const}\ i_2)\ (\mathsf{i32.const}\ n)\ (\mathsf{array.copy}\ x_1\ x_2) & \hookrightarrow & \mathsf{trap} \\
\quad \text{if } i_1 + n > |z.\mathsf{arrays}[a_1].\mathsf{fields}| & & \\
z; (\mathsf{ref.array}\ a_1)\ (\mathsf{i32.const}\ i_1)\ (\mathsf{ref.array}\ a_2)\ (\mathsf{i32.const}\ i_2)\ (\mathsf{i32.const}\ n)\ (\mathsf{array.copy}\ x_1\ x_2) & \hookrightarrow & \mathsf{trap} \\
\quad \text{if } i_2 + n > |z.\mathsf{arrays}[a_2].\mathsf{fields}| & & \\
z; (\mathsf{ref.array}\ a_1)\ (\mathsf{i32.const}\ i_1)\ (\mathsf{ref.array}\ a_2)\ (\mathsf{i32.const}\ i_2)\ (\mathsf{i32.const}\ n)\ (\mathsf{array.copy}\ x_1\ x_2) & \hookrightarrow & \epsilon \\
\quad \text{otherwise, if } n = 0 & & \\
z; (\mathsf{ref.array}\ a_1)\ (\mathsf{i32.const}\ i_1)\ (\mathsf{ref.array}\ a_2)\ (\mathsf{i32.const}\ i_2)\ (\mathsf{i32.const}\ n)\ (\mathsf{array.copy}\ x_1\ x_2) & \hookrightarrow & \\
\quad (\mathsf{ref.array}\ a_1)\ (\mathsf{i32.const}\ i_1) & & \\
\quad (\mathsf{ref.array}\ a_2)\ (\mathsf{i32.const}\ i_2) & & \\
\quad (\mathsf{array.get_}sx^?\ x_2)\ (\mathsf{array.set}\ x_1) & & \\
\quad (\mathsf{ref.array}\ a_1)\ (\mathsf{i32.const}\ i_1 + 1)\ (\mathsf{ref.array}\ a_2)\ (\mathsf{i32.const}\ i_2 + 1)\ (\mathsf{i32.const}\ n - 1)\ (\mathsf{array.copy}\ x_1\ x_2) & & \\
\qquad \text{otherwise, if } z.\mathsf{types}[x_2] \approx \mathsf{array}\ (\mathsf{mut}^?\ zt_2) & & \\
\qquad \wedge\ i_1 \leq i_2 \wedge sx^? = sx(zt_2) & & \\
z; (\mathsf{ref.array}\ a_1)\ (\mathsf{i32.const}\ i_1)\ (\mathsf{ref.array}\ a_2)\ (\mathsf{i32.const}\ i_2)\ (\mathsf{i32.const}\ n)\ (\mathsf{array.copy}\ x_1\ x_2) & \hookrightarrow & \\
\quad (\mathsf{ref.array}\ a_1)\ (\mathsf{i32.const}\ i_1 + n - 1) & & \\
\quad (\mathsf{ref.array}\ a_2)\ (\mathsf{i32.const}\ i_2 + n - 1) & & \\
\quad (\mathsf{array.get_}sx^?\ x_2)\ (\mathsf{array.set}\ x_1) & & \\
\quad (\mathsf{ref.array}\ a_1)\ (\mathsf{i32.const}\ i_1)\ (\mathsf{ref.array}\ a_2)\ (\mathsf{i32.const}\ i_2)\ (\mathsf{i32.const}\ n - 1)\ (\mathsf{array.copy}\ x_1\ x_2) & & \\
\qquad \text{otherwise, if } z.\mathsf{types}[x_2] \approx \mathsf{array}\ (\mathsf{mut}^?\ zt_2) & & \\
\qquad \wedge\ sx^? = sx(zt_2) & &
\end{array}
$$

Where:

$$
\begin{array}{rcl}
sx(valtype) & = & \epsilon \\
sx(packtype) & = & \mathsf{s}
\end{array}
$$

$\mathsf{array.init_data}\ x\ y$

1. Let z be the current state.
2. Assert: Due to validation, a value of number type $\mathsf{i32}$ is on the top of the stack.
3. Pop the value $(\mathsf{i32.const}\ n)$ from the stack.
4. Assert: Due to validation, a value of number type $\mathsf{i32}$ is on the top of the stack.
5. Pop the value $(\mathsf{i32.const}\ j)$ from the stack.
6. Assert: Due to validation, a value of number type $\mathsf{i32}$ is on the top of the stack.

7. Pop the value $(\mathsf{i32.const}\ i)$ from the stack.
8. Assert: Due to validation, a value is on the top of the stack.
9. Pop the value val from the stack.
10. If $val = \mathsf{ref.null}$, then:
    a. Trap.
11. Assert: Due to validation, val is some $\mathsf{ref.array}\ arrayaddr$.
12. Let $(\mathsf{ref.array}\ a)$ be the destructuring of val.
13. If $a < |z.\mathsf{arrays}|$ and $i + n > |z.\mathsf{arrays}[a].\mathsf{fields}|$, then:
    a. Trap.
14. If the expansion of $z.\mathsf{types}[x]$ is some $\mathsf{array}\ fieldtype$, then:
    a. Let $(\mathsf{array}\ fieldtype_0)$ be the destructuring of the expansion of $z.\mathsf{types}[x]$.
    b. Let $(\mathsf{mut}^?\ zt)$ be the destructuring of $fieldtype_0$.
    c. If $j + n \cdot |zt|/8 > |z.\mathsf{datas}[y].\mathsf{bytes}|$, then:
        1) Trap.
    d. If $n = 0$, then:
        1) Do nothing.
    e. Else:
        1) Let c be the result for which $\mathrm{bytes}_{zt}(c) = z.\mathsf{datas}[y].\mathsf{bytes}[j : |zt|/8]$.
        2) Push the value $(\mathsf{ref.array}\ a)$ to the stack.
        3) Push the value $(\mathsf{i32.const}\ i)$ to the stack.
        4) Push the value $\mathrm{unpack}(zt).\mathsf{const}\ \mathrm{unpack}_{zt}(c)$ to the stack.
        5) Execute the instruction $(\mathsf{array.set}\ x)$.
        6) Push the value $(\mathsf{ref.array}\ a)$ to the stack.
        7) Push the value $(\mathsf{i32.const}\ i + 1)$ to the stack.
        8) Push the value $(\mathsf{i32.const}\ j + |zt|/8)$ to the stack.
        9) Push the value $(\mathsf{i32.const}\ n - 1)$ to the stack.
        10) Execute the instruction $(\mathsf{array.init_data}\ x\ y)$.
15. Else if $n = 0$, then:
    a. Do nothing.

$$z; (\mathsf{ref.null})\ (\mathsf{i32.const}\ i)\ (\mathsf{i32.const}\ j)\ (\mathsf{i32.const}\ n)\ (\mathsf{array.init_data}\ x\ y) \hookrightarrow \mathsf{trap}$$

$$z; (\mathsf{ref.array}\ a)\ (\mathsf{i32.const}\ i)\ (\mathsf{i32.const}\ j)\ (\mathsf{i32.const}\ n)\ (\mathsf{array.init_data}\ x\ y) \hookrightarrow \mathsf{trap}$$
$$\text{if } i + n > |z.\mathsf{arrays}[a].\mathsf{fields}|$$

$$z; (\mathsf{ref.array}\ a)\ (\mathsf{i32.const}\ i)\ (\mathsf{i32.const}\ j)\ (\mathsf{i32.const}\ n)\ (\mathsf{array.init_data}\ x\ y) \hookrightarrow \mathsf{trap}$$
$$\text{if } z.\mathsf{types}[x] \approx \mathsf{array}\ (\mathsf{mut}^?\ zt)$$
$$\wedge\ j + n \cdot |zt|/8 > |z.\mathsf{datas}[y].\mathsf{bytes}|$$

$$z; (\mathsf{ref.array}\ a)\ (\mathsf{i32.const}\ i)\ (\mathsf{i32.const}\ j)\ (\mathsf{i32.const}\ n)\ (\mathsf{array.init_data}\ x\ y) \hookrightarrow \epsilon$$
$$\text{otherwise, if } n = 0$$

$$z; (\mathsf{ref.array}\ a)\ (\mathsf{i32.const}\ i)\ (\mathsf{i32.const}\ j)\ (\mathsf{i32.const}\ n)\ (\mathsf{array.init_data}\ x\ y) \hookrightarrow$$
$$(\mathsf{ref.array}\ a)\ (\mathsf{i32.const}\ i)\ (\mathrm{unpack}(zt).\mathsf{const}\ \mathrm{unpack}_{zt}(c))\ (\mathsf{array.set}\ x)$$
$$(\mathsf{ref.array}\ a)\ (\mathsf{i32.const}\ i + 1)\ (\mathsf{i32.const}\ j + |zt|/8)\ (\mathsf{i32.const}\ n - 1)\ (\mathsf{array.init_data}\ x\ y)$$
$$\text{otherwise, if } z.\mathsf{types}[x] \approx \mathsf{array}\ (\mathsf{mut}^?\ zt)$$
$$\wedge\ \mathrm{bytes}_{zt}(c) = z.\mathsf{datas}[y].\mathsf{bytes}[j : |zt|/8]$$

array.init_elem x y

1. Let z be the current state.
2. Assert: Due to validation, a value of number type i32 is on the top of the stack.
3. Pop the value (i32.const n) from the stack.
4. Assert: Due to validation, a value of number type i32 is on the top of the stack.
5. Pop the value (i32.const j) from the stack.
6. Assert: Due to validation, a value of number type i32 is on the top of the stack.
7. Pop the value (i32.const i) from the stack.
8. Assert: Due to validation, a value is on the top of the stack.
9. Pop the value val from the stack.
10. If $val = $ ref.null, then:
    a. Trap.
11. Assert: Due to validation, val is some ref.array $arrayaddr$.
12. Let (ref.array a) be the destructuring of val.
13. If $a < |z.\text{arrays}|$ and $i + n > |z.\text{arrays}[a].\text{fields}|$, then:
    a. Trap.
14. If $j + n > |z.\text{elems}[y].\text{refs}|$, then:
    a. Trap.
15. If $n = 0$, then:
    a. Do nothing.
16. Else if $j < |z.\text{elems}[y].\text{refs}|$, then:
    a. Let ref be the reference value $z.\text{elems}[y].\text{refs}[j]$.
    b. Push the value (ref.array a) to the stack.
    c. Push the value (i32.const i) to the stack.
    d. Push the value ref to the stack.
    e. Execute the instruction (array.set x).
    f. Push the value (ref.array a) to the stack.
    g. Push the value (i32.const $i + 1$) to the stack.
    h. Push the value (i32.const $j + 1$) to the stack.
    i. Push the value (i32.const $n - 1$) to the stack.
    j. Execute the instruction (array.init_elem x y).

$$z;\ (\text{ref.null})\ (\text{i32.const } i)\ (\text{i32.const } j)\ (\text{i32.const } n)\ (\text{array.init_elem } x\ y) \quad \hookrightarrow \quad \text{trap}$$

$$z;\ (\text{ref.array } a)\ (\text{i32.const } i)\ (\text{i32.const } j)\ (\text{i32.const } n)\ (\text{array.init_elem } x\ y) \quad \hookrightarrow \quad \text{trap}$$
$$\text{if } i + n > |z.\text{arrays}[a].\text{fields}|$$

$$z;\ (\text{ref.array } a)\ (\text{i32.const } i)\ (\text{i32.const } j)\ (\text{i32.const } n)\ (\text{array.init_elem } x\ y) \quad \hookrightarrow \quad \text{trap}$$
$$\text{if } j + n > |z.\text{elems}[y].\text{refs}|$$

$$z;\ (\text{ref.array } a)\ (\text{i32.const } i)\ (\text{i32.const } j)\ (\text{i32.const } n)\ (\text{array.init_elem } x\ y) \quad \hookrightarrow \quad \epsilon$$
$$\text{otherwise, if } n = 0$$

$$z;\ (\text{ref.array } a)\ (\text{i32.const } i)\ (\text{i32.const } j)\ (\text{i32.const } n)\ (\text{array.init_elem } x\ y) \quad \hookrightarrow$$
$$(\text{ref.array } a)\ (\text{i32.const } i)\ ref\ (\text{array.set } x)$$
$$(\text{ref.array } a)\ (\text{i32.const } i + 1)\ (\text{i32.const } j + 1)\ (\text{i32.const } n - 1)\ (\text{array.init_elem } x\ y)$$
$$\text{otherwise, if } ref = z.\text{elems}[y].\text{refs}[j]$$

any.convert_extern

1. Assert: Due to validation, a value is on the top of the stack.
2. Pop the value val from the stack.
3. If $val = \mathsf{ref.null}$, then:
   a. Push the value ref.null to the stack.
4. If val is some ref.extern ref, then:
   a. Let (ref.extern ref) be the destructuring of val.
   b. Push the value ref to the stack.

$$\begin{array}{rcl} (\mathsf{ref.null})\ \mathsf{any.convert_extern} & \hookrightarrow & \mathsf{ref.null} \\ (\mathsf{ref.extern}\ ref)\ \mathsf{any.convert_extern} & \hookrightarrow & ref \end{array}$$

extern.convert_any

1. Assert: Due to validation, a reference value is on the top of the stack.
2. Pop the value ref from the stack.
3. If $ref = \mathsf{ref.null}$, then:
   a. Push the value ref.null to the stack.
4. Else:
   a. Push the value (ref.extern ref) to the stack.

$$\begin{array}{lcll} ref\ \mathsf{extern.convert_any} & \hookrightarrow & \mathsf{ref.null} & \text{if } ref = \mathsf{ref.null} \\ ref\ \mathsf{extern.convert_any} & \hookrightarrow & (\mathsf{ref.extern}\ ref) & \text{otherwise} \end{array}$$

4.6.10 Numeric Instructions

Numeric instructions are defined in terms of the generic numeric operators. The mapping of numeric instructions to their underlying operators is expressed by the following definition:

$$\begin{array}{lcl} op_{\mathsf{iN}}(i_1, \dots, i_k) & = & \mathrm{i}op_N(i_1, \dots, i_k) \\ op_{\mathsf{fN}}(z_1, \dots, z_k) & = & \mathrm{f}op_N(z_1, \dots, z_k) \end{array}$$

And for conversion operators:

$$cvtop^{sx^?}_{t_1,t_2}(c) \quad = \quad cvtop^{sx^?}_{|t_1|,|t_2|}(c)$$

Where the underlying operators are partial, the corresponding instruction will trap when the result is not defined. Where the underlying operators are non-deterministic, because they may return one of multiple possible NaN values, so are the corresponding instructions.

Note

For example, the result of instruction i32.add applied to operands i_1, i_2 invokes $\mathsf{add}_{\mathsf{i32}}(i_1, i_2)$, which maps to the generic $\mathrm{iadd}_{32}(i_1, i_2)$ via the above definition. Similarly, i64.trunc_f32_s applied to z invokes $\mathsf{trunc}^{\mathsf{s}}_{\mathsf{f32,i64}}(z)$, which maps to the generic $\mathrm{trunc}^{\mathsf{s}}{}_{32,64}(z)$.

nt.const c

1. Push the value (nt.const c) to the stack.

Note

No formal reduction rule is required for this instruction, since const instructions already are values.

nt.unop

1. Assert: Due to validation, a value of number type nt is on the top of the stack.
2. Pop the value $(numtype_0.\mathsf{const}\ c_1)$ from the stack.
3. If $unop_{nt}(c_1)$ is empty, then:
   a. Trap.
4. Let c be an element of $unop_{nt}(c_1)$.
5. Push the value $(nt.\mathsf{const}\ c)$ to the stack.

$$\begin{array}{lcll} (nt.\mathsf{const}\ c_1)\ (nt.unop) & \hookrightarrow & (nt.\mathsf{const}\ c) & \text{if } c \in unop_{nt}(c_1) \\ (nt.\mathsf{const}\ c_1)\ (nt.unop) & \hookrightarrow & \mathsf{trap} & \text{if } unop_{nt}(c_1) = \epsilon \end{array}$$

nt.binop

1. Assert: Due to validation, a value of number type nt is on the top of the stack.
2. Pop the value $(numtype_0.\mathsf{const}\ c_2)$ from the stack.
3. Assert: Due to validation, a number value is on the top of the stack.
4. Pop the value $(numtype_0.\mathsf{const}\ c_1)$ from the stack.
5. If $binop_{nt}(c_1, c_2)$ is empty, then:
   a. Trap.
6. Let c be an element of $binop_{nt}(c_1, c_2)$.
7. Push the value $(nt.\mathsf{const}\ c)$ to the stack.

$$\begin{array}{lcll} (nt.\mathsf{const}\ c_1)\ (nt.\mathsf{const}\ c_2)\ (nt.binop) & \hookrightarrow & (nt.\mathsf{const}\ c) & \text{if } c \in binop_{nt}(c_1, c_2) \\ (nt.\mathsf{const}\ c_1)\ (nt.\mathsf{const}\ c_2)\ (nt.binop) & \hookrightarrow & \mathsf{trap} & \text{if } binop_{nt}(c_1, c_2) = \epsilon \end{array}$$

nt.testop

1. Assert: Due to validation, a value of number type nt is on the top of the stack.
2. Pop the value $(numtype_0.\mathsf{const}\ c_1)$ from the stack.
3. Let c be $testop_{nt}(c_1)$.
4. Push the value $(\mathsf{i32}.\mathsf{const}\ c)$ to the stack.

$$(nt.\mathsf{const}\ c_1)\ (nt.testop) \quad \hookrightarrow \quad (\mathsf{i32}.\mathsf{const}\ c) \quad \text{if } c = testop_{nt}(c_1)$$

nt.relop

1. Assert: Due to validation, a value of number type nt is on the top of the stack.
2. Pop the value $(numtype_0.\mathsf{const}\ c_2)$ from the stack.
3. Assert: Due to validation, a number value is on the top of the stack.
4. Pop the value $(numtype_0.\mathsf{const}\ c_1)$ from the stack.
5. Let c be $relop_{nt}(c_1, c_2)$.
6. Push the value $(\mathsf{i32}.\mathsf{const}\ c)$ to the stack.

$$(nt.\mathsf{const}\ c_1)\ (nt.\mathsf{const}\ c_2)\ (nt.relop) \quad \hookrightarrow \quad (\mathsf{i32}.\mathsf{const}\ c) \quad \text{if } c = relop_{nt}(c_1, c_2)$$

$nt_2.cvtop_nt_1$

1. Assert: Due to validation, a value of number type nt_1 is on the top of the stack.
2. Pop the value $(numtype_0.\mathsf{const}\ c_1)$ from the stack.
3. If $cvtop_{nt_1,nt_2}(c_1)$ is empty, then:
   a. Trap.
4. Let c be an element of $cvtop_{nt_1,nt_2}(c_1)$.
5. Push the value $(nt_2.\mathsf{const}\ c)$ to the stack.

$$\begin{array}{lcll} (nt_1.\mathsf{const}\ c_1)\ (nt_2.cvtop_nt_1) & \hookrightarrow & (nt_2.\mathsf{const}\ c) & \text{if } c \in cvtop_{nt_1,nt_2}(c_1) \\ (nt_1.\mathsf{const}\ c_1)\ (nt_2.cvtop_nt_1) & \hookrightarrow & \mathsf{trap} & \text{if } cvtop_{nt_1,nt_2}(c_1) = \epsilon \end{array}$$

4.6.11 Vector Instructions

Vector instructions that operate bitwise are handled as integer operations of respective bit width.

$$op_{\mathsf{v}N}(i_1, \dots, i_k) \quad = \quad \mathrm{i}op_N(i_1, \dots, i_k)$$

Most other vector instructions are defined in terms of numeric operators that are applied lane-wise according to the given shape.

$$op_{t\mathsf{x}N}(n_1, \dots, n_k) \quad = \quad \mathrm{lanes}^{-1}_{t\mathsf{x}N}(op_t(i_1, \dots, i_k)^*) \qquad (\text{if } i_1^* = \mathrm{lanes}_{t\mathsf{x}N}(n_1) \wedge \dots \wedge i_k^* = \mathrm{lanes}_{t\mathsf{x}N}(n_k)$$

Note

For example, the result of instruction i32x4.add applied to operands v_1, v_2 invokes $\mathsf{add}_{\mathsf{i32x4}}(v_1, v_2)$, which maps to $\mathrm{lanes}^{-1}_{\mathsf{i32x4}}(\mathsf{add}_{\mathsf{i32}}(i_1, i_2)^*)$, where i_1^* and i_2^* are sequences resulting from invoking $\mathrm{lanes}_{\mathsf{i32x4}}(v_1)$ and $\mathrm{lanes}_{\mathsf{i32x4}}(v_2)$ respectively.

For non-deterministic operators this definition is generalized to sets:

$$op_{t\mathsf{x}N}(n_1, \dots, n_k) \quad = \quad \{\mathrm{lanes}^{-1}_{t\mathsf{x}N}(i^*) \mid i^* \in \times(op_t(i_1, \dots, i_k)^*) \wedge i_1^* = \mathrm{lanes}_{t\mathsf{x}N}(n_1) \wedge \dots \wedge i_k^* = \mathrm{lanes}_{t\mathsf{x}N}(n_k)\}$$

where $\times\{x^*\}^N$ transforms a sequence of N sets of values into a set of sequences of N values by computing the set product:

$$\times(S_1 \dots S_N) \quad = \quad \{x_1 \dots x_N \mid x_1 \in S_1 \wedge \dots \wedge x_N \in S_N\}$$

The remaining vector operators use individual definitions.

$\mathsf{v128.const}\ c$

1. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

Note

No formal reduction rule is required for this instruction, since const instructions are already values.

$\mathsf{v128}.vvunop$

1. Assert: Due to validation, a value of vector type v128 is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
3. Assert: Due to validation, $|vvunop_{\mathsf{v128}}(c_1)| > 0$.

4. Let c be an element of $vvunop_{\mathsf{v128}}(c_1)$.
5. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$(\mathsf{v128.const}\ c_1)\ (\mathsf{v128}.vvunop) \quad \hookrightarrow \quad (\mathsf{v128.const}\ c) \quad \text{if } c \in vvunop_{\mathsf{v128}}(c_1)$$

$\mathsf{v128}.vvbinop$

1. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_2)$ from the stack.
3. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
4. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
5. Assert: Due to validation, $|vvbinop_{\mathsf{v128}}(c_1, c_2)| > 0$.
6. Let c be an element of $vvbinop_{\mathsf{v128}}(c_1, c_2)$.
7. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$(\mathsf{v128.const}\ c_1)\ (\mathsf{v128.const}\ c_2)\ (\mathsf{v128}.vvbinop) \quad \hookrightarrow \quad (\mathsf{v128.const}\ c) \quad \text{if } c \in vvbinop_{\mathsf{v128}}(c_1, c_2)$$

$\mathsf{v128}.vvternop$

1. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_3)$ from the stack.
3. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
4. Pop the value $(\mathsf{v128.const}\ c_2)$ from the stack.
5. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
6. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
7. Assert: Due to validation, $|vvternop_{\mathsf{v128}}(c_1, c_2, c_3)| > 0$.
8. Let c be an element of $vvternop_{\mathsf{v128}}(c_1, c_2, c_3)$.
9. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$(\mathsf{v128.const}\ c_1)\ (\mathsf{v128.const}\ c_2)\ (\mathsf{v128.const}\ c_3)\ (\mathsf{v128}.vvternop) \quad \hookrightarrow \quad (\mathsf{v128.const}\ c) \quad \text{if } c \in vvternop_{\mathsf{v128}}(c_1, c_2, c_3)$$

$\mathsf{v128.any_true}$

1. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
3. Let c be $\mathrm{inez}_{|\mathsf{v128}|}(c_1)$.
4. Push the value $(\mathsf{i32.const}\ c)$ to the stack.

$$(\mathsf{v128.const}\ c_1)\ (\mathsf{v128.any_true}) \quad \hookrightarrow \quad (\mathsf{i32.const}\ c) \quad \text{if } c = \mathrm{inez}_{|\mathsf{v128}|}(c_1)$$

$sh.vunop$

1. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
3. If $vunop_{sh}(c_1)$ is empty, then:
   a. Trap.
4. Let c be an element of $vunop_{sh}(c_1)$.
5. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$
\begin{array}{lcll}
(\mathsf{v128.const}\ c_1)\ (sh.vunop) & \hookrightarrow & (\mathsf{v128.const}\ c) & \text{if } c \in vunop_{sh}(c_1) \\
(\mathsf{v128.const}\ c_1)\ (sh.vunop) & \hookrightarrow & \mathsf{trap} & \text{if } vunop_{sh}(c_1) = \epsilon
\end{array}
$$

sh.vbinop

1. Assert: Due to validation, a value of vector type v128 is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_2)$ from the stack.
3. Assert: Due to validation, a value of vector type v128 is on the top of the stack.
4. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
5. If $vbinop_{sh}(c_1, c_2)$ is empty, then:
   a. Trap.
6. Let c be an element of $vbinop_{sh}(c_1, c_2)$.
7. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$
\begin{array}{lcll}
(\mathsf{v128.const}\ c_1)\ (\mathsf{v128.const}\ c_2)\ (sh.vbinop) & \hookrightarrow & (\mathsf{v128.const}\ c) & \text{if } c \in vbinop_{sh}(c_1, c_2) \\
(\mathsf{v128.const}\ c_1)\ (\mathsf{v128.const}\ c_2)\ (sh.vbinop) & \hookrightarrow & \mathsf{trap} & \text{if } vbinop_{sh}(c_1, c_2) = \epsilon
\end{array}
$$

sh.vternop

1. Assert: Due to validation, a value of vector type v128 is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_3)$ from the stack.
3. Assert: Due to validation, a value of vector type v128 is on the top of the stack.
4. Pop the value $(\mathsf{v128.const}\ c_2)$ from the stack.
5. Assert: Due to validation, a value of vector type v128 is on the top of the stack.
6. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
7. If $vternop_{sh}(c_1, c_2, c_3)$ is empty, then:
   a. Trap.
8. Let c be an element of $vternop_{sh}(c_1, c_2, c_3)$.
9. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$
\begin{array}{lcll}
(\mathsf{v128.const}\ c_1)\ (\mathsf{v128.const}\ c_2)\ (\mathsf{v128.const}\ c_3)\ (sh.vternop) & \hookrightarrow & (\mathsf{v128.const}\ c) & \text{if } c \in vternop_{sh}(c_1, c_2, c_3) \\
(\mathsf{v128.const}\ c_1)\ (\mathsf{v128.const}\ c_2)\ (\mathsf{v128.const}\ c_3)\ (sh.vternop) & \hookrightarrow & \mathsf{trap} & \text{if } vternop_{sh}(c_1, c_2, c_3) = \epsilon
\end{array}
$$

i*N*x*M*.all_true

1. Assert: Due to validation, a value of vector type v128 is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
3. Let i^* be $\mathrm{lanes}_{\mathsf{i}N\mathsf{x}M}(c_1)$.
4. Let c be $\Pi\, \mathrm{inez}_N(i)^*$.
5. Push the value $(\mathsf{i32.const}\ c)$ to the stack.

$$
\begin{array}{lcll}
(\mathsf{v128.const}\ c_1)\ (\mathsf{i}N\mathsf{x}M\mathsf{.all_true}) & \hookrightarrow & (\mathsf{i32.const}\ c) & \text{if } i^* = \mathrm{lanes}_{\mathsf{i}N\mathsf{x}M}(c_1) \\
 & & & \wedge\ c = \Pi\, (\mathrm{inez}_N(i)^*)
\end{array}
$$

sh.vrelop

1. Assert: Due to validation, a value of vector type v128 is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_2)$ from the stack.
3. Assert: Due to validation, a value of vector type v128 is on the top of the stack.

4. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
5. Let c be $\mathit{vrelop}_{sh}(c_1, c_2)$.
6. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$(\mathsf{v128.const}\ c_1)\ (\mathsf{v128.const}\ c_2)\ (sh.\mathit{vrelop}) \quad \hookrightarrow \quad (\mathsf{v128.const}\ c) \quad \text{if } c = \mathit{vrelop}_{sh}(c_1, c_2)$$

$sh.\mathit{vshiftop}$

1. Assert: Due to validation, a value of number type $\mathsf{i32}$ is on the top of the stack.
2. Pop the value $(\mathsf{i32.const}\ i)$ from the stack.
3. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
4. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
5. Let c be $\mathit{vshiftop}_{sh}(c_1, i)$.
6. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$(\mathsf{v128.const}\ c_1)\ (\mathsf{i32.const}\ i)\ (sh.\mathit{vshiftop}) \quad \hookrightarrow \quad (\mathsf{v128.const}\ c) \quad \text{if } c = \mathit{vshiftop}_{sh}(c_1, i)$$

$sh.\mathsf{bitmask}$

1. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
3. Let c be $\mathsf{bitmask}_{sh}(c_1)$.
4. Push the value $(\mathsf{i32.const}\ c)$ to the stack.

$$(\mathsf{v128.const}\ c_1)\ (sh.\mathsf{bitmask}) \quad \hookrightarrow \quad (\mathsf{i32.const}\ c) \quad \text{if } c = \mathsf{bitmask}_{sh}(c_1)$$

$sh.\mathit{swizzlop}$

1. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_2)$ from the stack.
3. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
4. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
5. Let c be $\mathit{swizzlop}_{sh}(c_1, c_2)$.
6. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$(\mathsf{v128.const}\ c_1)\ (\mathsf{v128.const}\ c_2)\ (sh.\mathit{swizzlop}) \quad \hookrightarrow \quad (\mathsf{v128.const}\ c) \quad \text{if } c = \mathit{swizzlop}_{sh}(c_1, c_2)$$

$sh.\mathsf{shuffle}\ i^*$

1. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_2)$ from the stack.
3. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
4. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
5. Let c be $\mathsf{shuffle}_{sh}(i^*, c_1, c_2)$.
6. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$(\mathsf{v128.const}\ c_1)\ (\mathsf{v128.const}\ c_2)\ (sh.\mathsf{shuffle}\ i^*) \quad \hookrightarrow \quad (\mathsf{v128.const}\ c) \quad \text{if } c = \mathsf{shuffle}_{sh}(i^*, c_1, c_2)$$

$\mathsf{i}N\mathsf{x}M.\mathsf{splat}$

1. Assert: Due to validation, a value is on the top of the stack.
2. Pop the value $(numtype_0.\mathsf{const}\ c_1)$ from the stack.
3. Assert: Due to validation, $numtype_0 = \mathrm{unpack}(\mathsf{i}N)$.
4. Let c be $\mathrm{lanes}^{-1}_{\mathsf{i}N\mathsf{x}M}(\mathrm{pack}_{\mathsf{i}N}(c_1)^M)$.
5. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$(\mathrm{unpack}(\mathsf{i}N).\mathsf{const}\ c_1)\ (\mathsf{i}N\mathsf{x}M.\mathsf{splat}) \quad \hookrightarrow \quad (\mathsf{v128.const}\ c) \quad \text{if } c = \mathrm{lanes}^{-1}_{\mathsf{i}N\mathsf{x}M}(\mathrm{pack}_{\mathsf{i}N}(c_1)^M)$$

$lanetype\mathsf{x}M.\mathsf{extract_lane_}sx'^?\ i$

1. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
3. If $sx'^?$ is not defined, then:
   a. Assert: Due to validation, $lanetype$ is number type.
   b. Assert: Due to validation, $i < |\mathrm{lanes}_{lanetype\mathsf{x}M}(c_1)|$.
   c. Let c_2 be $\mathrm{lanes}_{lanetype\mathsf{x}M}(c_1)[i]$.
   d. Push the value $(lanetype.\mathsf{const}\ c_2)$ to the stack.
4. Else:
   a. Assert: Due to validation, $lanetype$ is packed type.
   b. Let sx be $sx'^?$.
   c. Assert: Due to validation, $i < |\mathrm{lanes}_{lanetype\mathsf{x}M}(c_1)|$.
   d. Let c_2 be $\mathrm{extend}^{sx}_{|lanetype|,32}(\mathrm{lanes}_{lanetype\mathsf{x}M}(c_1)[i])$.
   e. Push the value $(\mathsf{i32.const}\ c_2)$ to the stack.

$$(\mathsf{v128.const}\ c_1)\ (nt\mathsf{x}M.\mathsf{extract_lane}\ i) \quad \hookrightarrow \quad (nt.\mathsf{const}\ c_2) \quad \text{if } c_2 = \mathrm{lanes}_{nt\mathsf{x}M}(c_1)[i]$$
$$(\mathsf{v128.const}\ c_1)\ (pt\mathsf{x}M.\mathsf{extract_lane_}sx\ i) \quad \hookrightarrow \quad (\mathsf{i32.const}\ c_2) \quad \text{if } c_2 = \mathrm{extend}^{sx}_{|pt|,32}(\mathrm{lanes}_{pt\mathsf{x}M}(c_1)[i])$$

$\mathsf{i}N\mathsf{x}M.\mathsf{replace_lane}\ i$

1. Assert: Due to validation, a value is on the top of the stack.
2. Pop the value $(numtype_0.\mathsf{const}\ c_2)$ from the stack.
3. Assert: Due to validation, $numtype_0 = \mathrm{unpack}(\mathsf{i}N)$.
4. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
5. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
6. Let c be $\mathrm{lanes}^{-1}_{\mathsf{i}N\mathsf{x}M}(\mathrm{lanes}_{\mathsf{i}N\mathsf{x}M}(c_1)[[i] = \mathrm{pack}_{\mathsf{i}N}(c_2)])$.
7. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$(\mathsf{v128.const}\ c_1)\ (\mathrm{unpack}(\mathsf{i}N).\mathsf{const}\ c_2)\ (\mathsf{i}N\mathsf{x}M.\mathsf{replace_lane}\ i) \quad \hookrightarrow \quad (\mathsf{v128.const}\ c)$$
$$\text{if } c = \mathrm{lanes}^{-1}_{\mathsf{i}N\mathsf{x}M}(\mathrm{lanes}_{\mathsf{i}N\mathsf{x}M}(c_1)[[i] = \mathrm{pack}_{\mathsf{i}N}(c_2)])$$

$sh_2.vextunop_sh_1$

1. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
3. Let c be $vextunop_{sh_1,sh_2}(c_1)$.
4. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$(\mathsf{v128.const}\ c_1)\ (sh_2.vextunop_sh_1) \quad \hookrightarrow \quad (\mathsf{v128.const}\ c) \quad \text{if } vextunop_{sh_1,sh_2}(c_1) = c$$

$sh_2.vextbinop_sh_1$

1. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_2)$ from the stack.
3. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
4. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
5. Let c be $vextbinop_{sh_1,sh_2}(c_1, c_2)$.
6. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$(\mathsf{v128.const}\ c_1)\ (\mathsf{v128.const}\ c_2)\ (sh_2.vextbinop_sh_1) \quad \hookrightarrow \quad (\mathsf{v128.const}\ c) \quad \text{if } vextbinop_{sh_1,sh_2}(c_1, c_2) = c$$

$sh_2.vextternop_sh_1$

1. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_3)$ from the stack.
3. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
4. Pop the value $(\mathsf{v128.const}\ c_2)$ from the stack.
5. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
6. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
7. Let c be $vextternop_{sh_1,sh_2}(c_1, c_2, c_3)$.
8. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$(\mathsf{v128.const}\ c_1)\ (\mathsf{v128.const}\ c_2)\ (\mathsf{v128.const}\ c_3)\ (sh_2.vextternop_sh_1) \quad \hookrightarrow \quad (\mathsf{v128.const}\ c) \quad \text{if } vextternop_{sh_1,sh_2}(c_1, c_2, c_3) = c$$

$sh_2.\mathsf{narrow}_sh_1_sx$

1. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_2)$ from the stack.
3. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
4. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
5. Let c be $\mathsf{narrow}^{sx}_{sh_1,sh_2}(c_1, c_2)$.
6. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$(\mathsf{v128.const}\ c_1)\ (\mathsf{v128.const}\ c_2)\ (sh_2.\mathsf{narrow}_sh_1_sx) \quad \hookrightarrow \quad (\mathsf{v128.const}\ c) \quad \text{if } c = \mathsf{narrow}^{sx}_{sh_1,sh_2}(c_1, c_2)$$

$sh_2.vcvtop_sh_1$

1. Assert: Due to validation, a value of vector type $\mathsf{v128}$ is on the top of the stack.
2. Pop the value $(\mathsf{v128.const}\ c_1)$ from the stack.
3. Let c be $vcvtop_{sh_1,sh_2}(vcvtop, c_1)$.
4. Push the value $(\mathsf{v128.const}\ c)$ to the stack.

$$(\mathsf{v128.const}\ c_1)\ (sh_2.vcvtop_sh_1) \quad \hookrightarrow \quad (\mathsf{v128.const}\ c) \quad \text{if } c = vcvtop_{sh_1,sh_2}(vcvtop, c_1)$$

4.6.12 Contextual Rules

The next instruction to execute as well as the current frame is determined according to the following *contextual* reduction rules.

$$
\begin{array}{rcll}
z; val^*\ instr^*\ instr_1^* & \hookrightarrow & z'; val^*\ instr'^*\ instr_1^* & \text{if } z; instr^* \hookrightarrow z'; instr'^* \\
 & & & \wedge\ (val^* \neq \epsilon \vee instr_1^* \neq \epsilon) \\
z; (\mathsf{label}_n\{instr_0^*\}\ instr^*) & \hookrightarrow & z'; (\mathsf{label}_n\{instr_0^*\}\ instr'^*) & \text{if } z; instr^* \hookrightarrow z'; instr'^* \\
z; (\mathsf{handler}_n\{catch^*\}\ instr^*) & \hookrightarrow & z'; (\mathsf{handler}_n\{catch^*\}\ instr'^*) & \text{if } z; instr^* \hookrightarrow z'; instr'^* \\
s; f; (\mathsf{frame}_n\{f'\}\ instr^*) & \hookrightarrow & s'; f; (\mathsf{frame}_n\{f''\}\ instr'^*) & \text{if } s; f'; instr^* \hookrightarrow s'; f''; instr'^*
\end{array}
$$

Note

In the prose formulation, the current instruction pointer and the current frame are implicit.

4.6.13 Expressions

An expression is *evaluated* relative to a current frame pointing to its containing module instance.

Expression evaluation iterates single-step reduction until reaching a sequence of resulting values.

Expressions constituting function bodies are executed during function invocation.

4.7 Modules

For modules, the execution semantics primarily defines instantiation, which allocates instances for a module and its contained definitions, initializes memories and tables from contained data and element segments, and invokes the start function if present. It also includes invocation of exported functions.

4.7.1 Allocation

New instances of tags, globals, memories, tables, functions, data segments, and element segments are *allocated* in a store s, as defined by the following auxiliary functions.

Tags

$\mathrm{alloctag}(s, tagtype)$

1. Let $taginst$ be the tag instance $\{\mathsf{type}\ tagtype\}$.
2. Let a be the length of $s.\mathsf{tags}$.
3. Append $taginst$ to $s.\mathsf{tags}$.
4. Return a.

$$
\mathrm{alloctag}(s, tagtype) \quad = \quad (s \oplus \{\mathsf{tags}\ taginst\}, |s.\mathsf{tags}|) \\
\quad \text{if } taginst = \{\mathsf{type}\ tagtype\}
$$

Globals

$\mathrm{allocglobal}(s, globaltype, val)$

1. Let $globalinst$ be the global instance $\{\mathsf{type}\ globaltype,\ \mathsf{value}\ val\}$.
2. Let a be the length of $s.\mathsf{globals}$.
3. Append $globalinst$ to $s.\mathsf{globals}$.
4. Return a.

$$\begin{array}{l}\text{allocglobal}(s, \mathit{globaltype}, \mathit{val}) \quad = \quad (s \oplus \{\text{globals } \mathit{globalinst}\}, |s.\text{globals}|) \\ \quad \text{if } \mathit{globalinst} = \{\text{type } \mathit{globaltype},\ \text{value } \mathit{val}\}\end{array}$$

Memories

$\text{allocmem}(s, \mathit{at}\ [i\,..\,j^?]\ \text{page})$

1. Let $\mathit{meminst}$ be the memory instance $\{\text{type } (\mathit{at}\ [i\,..\,j^?]\ \text{page}),\ \text{bytes } \texttt{0x00}^{i \cdot 64\,\text{Ki}}\}$.
2. Let a be the length of $s.\text{mems}$.
3. Append $\mathit{meminst}$ to $s.\text{mems}$.
4. Return a.

$$\begin{array}{l}\text{allocmem}(s, \mathit{at}\ [i\,..\,j^?]\ \text{page}) \quad = \quad (s \oplus \{\text{mems } \mathit{meminst}\}, |s.\text{mems}|) \\ \quad \text{if } \mathit{meminst} = \{\text{type } (\mathit{at}\ [i\,..\,j^?]\ \text{page}),\ \text{bytes } (\texttt{0x00})^{i \cdot 64\,\text{Ki}}\}\end{array}$$

Tables

$\text{alloctable}(s, \mathit{at}\ [i\,..\,j^?]\ \mathit{rt}, \mathit{ref})$

1. Let $\mathit{tableinst}$ be the table instance $\{\text{type } (\mathit{at}\ [i\,..\,j^?]\ \mathit{rt}),\ \text{refs } \mathit{ref}^i\}$.
2. Let a be the length of $s.\text{tables}$.
3. Append $\mathit{tableinst}$ to $s.\text{tables}$.
4. Return a.

$$\begin{array}{l}\text{alloctable}(s, \mathit{at}\ [i\,..\,j^?]\ \mathit{rt}, \mathit{ref}) \quad = \quad (s \oplus \{\text{tables } \mathit{tableinst}\}, |s.\text{tables}|) \\ \quad \text{if } \mathit{tableinst} = \{\text{type } (\mathit{at}\ [i\,..\,j^?]\ \mathit{rt}),\ \text{refs } \mathit{ref}^i\}\end{array}$$

Functions

$\text{allocfunc}(s, \mathit{deftype}, \mathit{code}, \mathit{moduleinst})$

1. Let $\mathit{funcinst}$ be the function instance $\{\text{type } \mathit{deftype},\ \text{module } \mathit{moduleinst},\ \text{code } \mathit{code}\}$.
2. Let a be the length of $s.\text{funcs}$.
3. Append $\mathit{funcinst}$ to $s.\text{funcs}$.
4. Return a.

$$\begin{array}{l}\text{allocfunc}(s, \mathit{deftype}, \mathit{code}, \mathit{moduleinst}) \quad = \quad (s \oplus \{\text{funcs } \mathit{funcinst}\}, |s.\text{funcs}|) \\ \quad \text{if } \mathit{funcinst} = \{\text{type } \mathit{deftype},\ \text{module } \mathit{moduleinst},\ \text{code } \mathit{code}\}\end{array}$$

Data segments

$\text{allocdata}(s, \text{ok}, \mathit{byte}^*)$

1. Let $\mathit{datainst}$ be the data instance $\{\text{bytes } \mathit{byte}^*\}$.
2. Let a be the length of $s.\text{datas}$.
3. Append $\mathit{datainst}$ to $s.\text{datas}$.
4. Return a.

$$\begin{array}{l}\text{allocdata}(s, \text{ok}, \mathit{byte}^*) \quad = \quad (s \oplus \{\text{datas } \mathit{datainst}\}, |s.\text{datas}|) \\ \quad \text{if } \mathit{datainst} = \{\text{bytes } \mathit{byte}^*\}\end{array}$$

Element segments

$\text{allocelem}(s, elemtype, ref^*)$

1. Let $eleminst$ be the element instance $\{\text{type } elemtype,\ \text{refs } ref^*\}$.
2. Let a be the length of $s.\text{elems}$.
3. Append $eleminst$ to $s.\text{elems}$.
4. Return a.

$$\text{allocelem}(s, elemtype, ref^*) \quad = \quad (s \oplus \{\text{elems } eleminst\}, |s.\text{elems}|)$$
$$\text{if } eleminst = \{\text{type } elemtype,\ \text{refs } ref^*\}$$

Growing memories

$\text{growmem}(meminst, n)$

1. Let $\{\text{type } (at\ [i\,..\,j^?]\ \text{page}),\ \text{bytes } b^*\}$ be the destructuring of $meminst$.
2. Let i' be $|b^*|/(64\,\text{Ki}) + n$.
3. If not $(i' \leq j)^?$, then:
   a. Fail.
4. If $i' \leq 2^{|at|-16}$, then:
   a. Let $meminst'$ be the memory instance $\{\text{type } (at\ [i'\,..\,j^?]\ \text{page}),\ \text{bytes } b^*\ \texttt{0x00}^{n \cdot 64\,\text{Ki}}\}$.
   b. Return $meminst'$.
5. Fail.

$$\begin{array}{rcll} \text{growmem}(meminst, n) & = & meminst' & \text{if } meminst = \{\text{type } (at\ [i\,..\,j^?]\ \text{page}),\ \text{bytes } b^*\} \\ & & & \wedge\ meminst' = \{\text{type } (at\ [i'\,..\,j^?]\ \text{page}),\ \text{bytes } b^*\ (\texttt{0x00})^{n \cdot 64\,\text{Ki}}\} \\ & & & \wedge\ i' = |b^*|/(64\,\text{Ki}) + n \\ & & & \wedge\ (i' \leq j)^? \\ & & & \wedge\ i' \leq 2^{|at|-16} \end{array}$$

Growing tables

$\text{growtable}(tableinst, n, r)$

1. Let $\{\text{type } (at\ [i\,..\,j^?]\ rt),\ \text{refs } r'^*\}$ be the destructuring of $tableinst$.
2. Let i' be $|r'^*| + n$.
3. If not $(i' \leq j)^?$, then:
   a. Fail.
4. If $i' \leq 2^{|at|} - 1$, then:
   a. Let $tableinst'$ be the table instance $\{\text{type } (at\ [i'\,..\,j^?]\ rt),\ \text{refs } r'^*\ r^n\}$.
   b. Return $tableinst'$.
5. Fail.

$$\begin{array}{rcll} \text{growtable}(tableinst, n, r) & = & tableinst' & \text{if } tableinst = \{\text{type } (at\ [i\,..\,j^?]\ rt),\ \text{refs } r'^*\} \\ & & & \wedge\ tableinst' = \{\text{type } (at\ [i'\,..\,j^?]\ rt),\ \text{refs } r'^*\ r^n\} \\ & & & \wedge\ i' = |r'^*| + n \\ & & & \wedge\ (i' \leq j)^? \\ & & & \wedge\ i' \leq 2^{|at|} - 1 \end{array}$$

Modules

$\text{allocmodule}(s, module, externaddr^*, val_{\mathsf{g}}^*, ref_{\mathsf{t}}^*, ref_{\mathsf{e}}^{**})$

1. Let $(\mathsf{module}\ type^*\ import^*\ tag^*\ global^*\ mem^*\ table^*\ func^*\ data^*\ elem^*\ start^?\ export^*)$ be the destructuring of $module$.
2. Let aa_{i}^* be $\text{tags}(externaddr^*)$.
3. Let ga_{i}^* be $\text{globals}(externaddr^*)$.
4. Let fa_{i}^* be $\text{funcs}(externaddr^*)$.
5. Let ma_{i}^* be $\text{mems}(externaddr^*)$.
6. Let ta_{i}^* be $\text{tables}(externaddr^*)$.
7. Let fa^* be $|s.\mathsf{funcs}| + i_{\mathsf{f}}$ for all i_{f} from 0 to $|func^*| - 1$.
8. Let $tagtype^*$ be the tag type sequence ϵ.
9. For each tag in tag^*, do:
   a. Let $(\mathsf{tag}\ tagtype)$ be the destructuring of tag.
   b. Append $tagtype$ to $tagtype^*$.
10. Let $byte^{**}$ be the byte sequence sequence ϵ.
11. For each $data$ in $data^*$, do:
    a. Let $(\mathsf{data}\ byte^*\ datamode)$ be the destructuring of $data$.
    b. Append $byte^*$ to $byte^{**}$.
12. Let $globaltype^*$ be the global type sequence ϵ.
13. For each $global$ in $global^*$, do:
    a. Let $(\mathsf{global}\ globaltype\ expr_{\mathsf{g}})$ be the destructuring of $global$.
    b. Append $globaltype$ to $globaltype^*$.
14. Let $tabletype^*$ be the table type sequence ϵ.
15. For each $table$ in $table^*$, do:
    a. Let $(\mathsf{table}\ tabletype\ expr_{\mathsf{t}})$ be the destructuring of $table$.
    b. Append $tabletype$ to $tabletype^*$.
16. Let $memtype^*$ be the memory type sequence ϵ.
17. For each mem in mem^*, do:
    a. Let $(\mathsf{memory}\ memtype)$ be the destructuring of mem.
    b. Append $memtype$ to $memtype^*$.
18. Let dt^* be $\text{alloctype}^*(type^*)$.
19. Let $elemtype^*$ be the reference type sequence ϵ.
20. For each $elem$ in $elem^*$, do:
    a. Let $(\mathsf{elem}\ elemtype\ expr_{\mathsf{e}}^*\ elemmode)$ be the destructuring of $elem$.
    b. Append $elemtype$ to $elemtype^*$.
21. Let $expr_{\mathsf{f}}^*$ be the expression sequence ϵ.
22. Let $local^{**}$ be the local sequence sequence ϵ.
23. Let x^* be the type index sequence ϵ.
24. For each $func$ in $func^*$, do:

   a. Let $(\mathsf{func}\ x\ local^*\ expr_{\mathsf{f}})$ be the destructuring of $func$.
   b. Append $expr_{\mathsf{f}}$ to $expr^*_{\mathsf{f}}$.
   c. Append $local^*$ to $local^{**}$.
   d. Append x to x^*.
25. Let aa^* be ϵ.
26. For each $tagtype$ in $tagtype^*$, do:
   a. Let aa be the tag address $\mathrm{alloctag}(s, tagtype[:= dt^*])$.
   b. Append aa to aa^*.
27. Let ga^* be ϵ.
28. For each $globaltype$ in $globaltype^*$ and val_{g} in val^*_{g}, do:
   a. Let ga be the global address $\mathrm{allocglobal}(s, globaltype[:= dt^*], val_{\mathsf{g}})$.
   b. Append ga to ga^*.
29. Let ma^* be ϵ.
30. For each $memtype$ in $memtype^*$, do:
   a. Let ma be the memory address $\mathrm{allocmem}(s, memtype[:= dt^*])$.
   b. Append ma to ma^*.
31. Let ta^* be ϵ.
32. For each $tabletype$ in $tabletype^*$ and ref_{t} in ref^*_{t}, do:
   a. Let ta be the table address $\mathrm{alloctable}(s, tabletype[:= dt^*], ref_{\mathsf{t}})$.
   b. Append ta to ta^*.
33. Let xi^* be ϵ.
34. For each $export$ in $export^*$, do:
   a. Let xi be the export instance $\mathrm{allocexport}(moduleinst, export)$.
   b. Append xi to xi^*.
35. Let da^* be ϵ.
36. For each $byte^*$ in $byte^{**}$, do:
   a. Let da be the data address $\mathrm{allocdata}(s, \mathsf{ok}, byte^*)$.
   b. Append da to da^*.
37. Let ea^* be ϵ.
38. For each $elemtype$ in $elemtype^*$ and ref^*_{e} in ref^{**}_{e}, do:
   a. Let ea be the elem address $\mathrm{allocelem}(s, elemtype[:= dt^*], ref^*_{\mathsf{e}})$.
   b. Append ea to ea^*.
39. Let $moduleinst$ be the module instance $\{\mathsf{types}\ dt^*,\ \mathsf{tags}\ aa^*_{\mathsf{i}}\ aa^*,\ \mathsf{globals}\ ga^*_{\mathsf{i}}\ ga^*,\ \mathsf{mems}\ ma^*_{\mathsf{i}}\ ma^*,\ \mathsf{tables}\ ta^*_{\mathsf{i}}\ ta^*,\ \mathsf{funcs}\ fa^*_{\mathsf{i}}\ fa^*,\ \mathsf{datas}\ da^*,\ \mathsf{elems}\ ea^*,\ \mathsf{exports}\ xi^*\}$.
40. Let $funcaddr^*_0$ be ϵ.
41. For each $expr_{\mathsf{f}}$ in $expr^*_{\mathsf{f}}$ and $local^*$ in $local^{**}$ and x in x^*, do:
   a. Let $funcaddr_0$ be the function address $\mathrm{allocfunc}(s, dt^*[x], \mathsf{func}\ x\ local^*\ expr_{\mathsf{f}}, moduleinst)$.
   b. Append $funcaddr_0$ to $funcaddr^*_0$.
42. Assert: Due to validation, $funcaddr^*_0 = fa^*$.

43. Return $moduleinst$.

$$\begin{array}{l}\text{allocmodule}(s, module, externaddr^*, val_{\mathsf{g}}^*, ref_{\mathsf{t}}^*, (ref_{\mathsf{e}}^*)^*) \quad = \quad (s_7, moduleinst) \\ \quad \text{if } module = \mathsf{module}\ type^*\ import^*\ tag^*\ global^*\ mem^*\ table^*\ func^*\ data^*\ elem^*\ start^?\ export^* \\ \quad \wedge\ tag^* = (\mathsf{tag}\ tagtype)^* \\ \quad \wedge\ global^* = (\mathsf{global}\ globaltype\ expr_{\mathsf{g}})^* \\ \quad \wedge\ mem^* = (\mathsf{memory}\ memtype)^* \\ \quad \wedge\ table^* = (\mathsf{table}\ tabletype\ expr_{\mathsf{t}})^* \\ \quad \wedge\ func^* = (\mathsf{func}\ x\ local^*\ expr_{\mathsf{f}})^* \\ \quad \wedge\ data^* = (\mathsf{data}\ byte^*\ datamode)^* \\ \quad \wedge\ elem^* = (\mathsf{elem}\ elemtype\ expr_{\mathsf{e}}^*\ elemmode)^* \\ \quad \wedge\ aa_{\mathsf{i}}^* = \text{tags}(externaddr^*) \\ \quad \wedge\ ga_{\mathsf{i}}^* = \text{globals}(externaddr^*) \\ \quad \wedge\ ma_{\mathsf{i}}^* = \text{mems}(externaddr^*) \\ \quad \wedge\ ta_{\mathsf{i}}^* = \text{tables}(externaddr^*) \\ \quad \wedge\ fa_{\mathsf{i}}^* = \text{funcs}(externaddr^*) \\ \quad \wedge\ dt^* = \text{alloctype}^*(type^*) \\ \quad \wedge\ fa^* = (|s.\mathsf{funcs}| + i_{\mathsf{f}})^{i_{\mathsf{f}}<|func^*|} \\ \quad \wedge\ (s_1, aa^*) = \text{alloctag}^*(s, tagtype[:= dt^*]^*) \\ \quad \wedge\ (s_2, ga^*) = \text{allocglobal}^*(s_1, globaltype[:= dt^*]^*, val_{\mathsf{g}}^*) \\ \quad \wedge\ (s_3, ma^*) = \text{allocmem}^*(s_2, memtype[:= dt^*]^*) \\ \quad \wedge\ (s_4, ta^*) = \text{alloctable}^*(s_3, tabletype[:= dt^*]^*, ref_{\mathsf{t}}^*) \\ \quad \wedge\ (s_5, da^*) = \text{allocdata}^*(s_4, \mathsf{ok}^{|data^*|}, (byte^*)^*) \\ \quad \wedge\ (s_6, ea^*) = \text{allocelem}^*(s_5, elemtype[:= dt^*]^*, (ref_{\mathsf{e}}^*)^*) \\ \quad \wedge\ (s_7, fa^*) = \text{allocfunc}^*(s_6, dt^*[x]^*, (\mathsf{func}\ x\ local^*\ expr_{\mathsf{f}})^*, moduleinst^{|func^*|}) \\ \quad \wedge\ xi^* = \text{allocexport}^*(\{\mathsf{tags}\ aa_{\mathsf{i}}^*\ aa^*,\ \mathsf{globals}\ ga_{\mathsf{i}}^*\ ga^*,\ \mathsf{mems}\ ma_{\mathsf{i}}^*\ ma^*,\ \mathsf{tables}\ ta_{\mathsf{i}}^*\ ta^*,\ \mathsf{funcs}\ fa_{\mathsf{i}}^*\ fa^*\}, export^*) \\ \quad \wedge\ moduleinst = \{\mathsf{types}\ dt^*, \\ \qquad\qquad \mathsf{tags}\ aa_{\mathsf{i}}^*\ aa^*,\ \mathsf{globals}\ ga_{\mathsf{i}}^*\ ga^*, \\ \qquad\qquad \mathsf{mems}\ ma_{\mathsf{i}}^*\ ma^*, \\ \qquad\qquad \mathsf{tables}\ ta_{\mathsf{i}}^*\ ta^*,\ \mathsf{funcs}\ fa_{\mathsf{i}}^*\ fa^*,\ \mathsf{datas}\ da^*, \\ \qquad\qquad \mathsf{elems}\ ea^*,\ \mathsf{exports}\ xi^*\}\end{array}$$

Here, the notation allocx^* is shorthand for multiple allocations of object kind X, defined as follows:

$$\begin{array}{lcll}\text{allocX}^*(s, \epsilon, \epsilon) & = & (s, \epsilon) & \\ \text{allocX}^*(s, X\ X'^*, Y\ Y'^*) & = & (s_2, a\ a'^*) & \text{if } (s_1, a) = \text{allocX}(X, Y, s, X, Y) \\ & & & \wedge\ (s_2, a'^*) = \text{allocX}^*(s_1, X'^*, Y'^*)\end{array}$$

For types, however, allocation is defined in terms of rolling and substitution of all preceding types to produce a list of closed defined types:

$\text{alloctype}^*(type''^*)$

1. If $type''^* = \epsilon$, then:
   a. Return ϵ.
2. Let $type'^*\ type$ be $type''^*$.
3. Let $(\mathsf{type}\ rectype)$ be the destructuring of $type$.
4. Let $deftype'^*$ be $\text{alloctype}^*(type'^*)$.
5. Let x be the length of $deftype'^*$.
6. Let $deftype^*$ be $\text{roll}_x^*(rectype)[:= deftype'^*]$.
7. Return $deftype'^*\ deftype^*$.

$$\begin{array}{lcll}\text{alloctype}^*(\epsilon) & = & \epsilon & \\ \text{alloctype}^*(type'^*\ type) & = & deftype'^*\ deftype^* & \text{if } deftype'^* = \text{alloctype}^*(type'^*) \\ & & & \wedge\ type = \mathsf{type}\ rectype \\ & & & \wedge\ deftype^* = \text{roll}_x^*(rectype)[:= deftype'^*] \\ & & & \wedge\ x = |deftype'^*|\end{array}$$

Finally, export instances are produced with the help of the following definition:

$\text{allocexport}(moduleinst, \text{export } name\ externidx)$

1. If $externidx$ is some $\text{tag } tagidx$, then:
   a. Let $(\text{tag } x)$ be the destructuring of $externidx$.
   b. Return $\{\text{name } name,\ \text{addr } (\text{tag } moduleinst.\text{tags}[x])\}$.
2. If $externidx$ is some $\text{global } globalidx$, then:
   a. Let $(\text{global } x)$ be the destructuring of $externidx$.
   b. Return $\{\text{name } name,\ \text{addr } (\text{global } moduleinst.\text{globals}[x])\}$.
3. If $externidx$ is some $\text{memory } memidx$, then:
   a. Let $(\text{memory } x)$ be the destructuring of $externidx$.
   b. Return $\{\text{name } name,\ \text{addr } (\text{mem } moduleinst.\text{mems}[x])\}$.
4. If $externidx$ is some $\text{table } tableidx$, then:
   a. Let $(\text{table } x)$ be the destructuring of $externidx$.
   b. Return $\{\text{name } name,\ \text{addr } (\text{table } moduleinst.\text{tables}[x])\}$.
5. Assert: Due to validation, $externidx$ is some $\text{func } funcidx$.
6. Let $(\text{func } x)$ be the destructuring of $externidx$.
7. Return $\{\text{name } name,\ \text{addr } (\text{func } moduleinst.\text{funcs}[x])\}$.

$$
\begin{array}{lcl}
\text{allocexport}(moduleinst, \text{export } name\ (\text{tag } x)) & = & \{\text{name } name,\ \text{addr } (\text{tag } moduleinst.\text{tags}[x])\} \\
\text{allocexport}(moduleinst, \text{export } name\ (\text{global } x)) & = & \{\text{name } name,\ \text{addr } (\text{global } moduleinst.\text{globals}[x])\} \\
\text{allocexport}(moduleinst, \text{export } name\ (\text{memory } x)) & = & \{\text{name } name,\ \text{addr } (\text{mem } moduleinst.\text{mems}[x])\} \\
\text{allocexport}(moduleinst, \text{export } name\ (\text{table } x)) & = & \{\text{name } name,\ \text{addr } (\text{table } moduleinst.\text{tables}[x])\} \\
\text{allocexport}(moduleinst, \text{export } name\ (\text{func } x)) & = & \{\text{name } name,\ \text{addr } (\text{func } moduleinst.\text{funcs}[x])\}
\end{array}
$$

Note

The definition of module allocation is mutually recursive with the allocation of its associated functions, because the resulting module instance is passed to the allocators as an argument, in order to form the necessary closures. In an implementation, this recursion is easily unraveled by mutating one or the other in a secondary step.

4.7.2 Instantiation

Given a store s, a $module$ is instantiated with a list of external addresses $externaddr^*$ supplying the required imports as follows.

Instantiation checks that the module is valid and the provided imports match the declared types, and may *fail* with an error otherwise. Instantiation can also result in an exception or trap when initializing a table or memory from an active segment or when executing the start function. It is up to the embedder to define how such conditions are reported.

$\text{instantiate}(s, module, externaddr^*)$

1. If $module$ is not valid, then:
   a. Fail.
2. Let $xt_\mathsf{i}^* \ \to\ xt_\mathsf{e}^*$ be the destructuring of the type of $module$.
3. Let $(\text{module } type^*\ import^*\ tag^*\ global^*\ mem^*\ table^*\ func^*\ data^*\ elem^*\ start^?\ export^*)$ be the destructuring of $module$.
4. If $|externaddr^*| \neq |xt_\mathsf{i}^*|$, then:
   a. Fail.

5. For all $externaddr$ in $externaddr^*$, and corresponding xt_i in xt_i^*:
   a. If $externaddr$ is not valid with type xt_i, then:
      1) Fail.
6. Let $instr_\mathsf{d}^*$ be the concatenation of $\mathrm{rundata}_{i_\mathsf{d}}(data^*[i_\mathsf{d}])^{i_\mathsf{d}<|data^*|}$.
7. Let $instr_\mathsf{e}^*$ be the concatenation of $\mathrm{runelem}_{i_\mathsf{e}}(elem^*[i_\mathsf{e}])^{i_\mathsf{e}<|elem^*|}$.
8. Let $moduleinst_0$ be the module instance $\{\mathsf{types}\ \mathrm{alloctype}^*(type^*),\ \mathsf{globals}\ \mathrm{globals}(externaddr^*),\ \mathsf{funcs}\ \mathrm{funcs}(externaddr^*)\ (|s.\mathsf{funcs}| + i_\mathsf{f})^{i_\mathsf{f}<|func^*|}\}$.
9. Let $expr_\mathsf{t}^*$ be the expression sequence ϵ.
10. For each $table$ in $table^*$, do:
    a. Let $(\mathsf{table}\ tabletype\ expr_\mathsf{t})$ be the destructuring of $table$.
    b. Append $expr_\mathsf{t}$ to $expr_\mathsf{t}^*$.
11. Let $expr_\mathsf{g}^*$ be the expression sequence ϵ.
12. Let $globaltype^*$ be the global type sequence ϵ.
13. For each $global$ in $global^*$, do:
    a. Let $(\mathsf{global}\ globaltype\ expr_\mathsf{g})$ be the destructuring of $global$.
    b. Append $expr_\mathsf{g}$ to $expr_\mathsf{g}^*$.
    c. Append $globaltype$ to $globaltype^*$.
14. Let $expr_\mathsf{e}^{**}$ be the expression sequence sequence ϵ.
15. For each $elem$ in $elem^*$, do:
    a. Let $(\mathsf{elem}\ reftype\ expr_\mathsf{e}^*\ elemmode)$ be the destructuring of $elem$.
    b. Append $expr_\mathsf{e}^*$ to $expr_\mathsf{e}^{**}$.
16. Let z be the state $(s, \{\mathsf{module}\ moduleinst_0\})$.
17. Let F be the frame $z.\mathsf{frame}$.
18. Push the frame F.
19. Let val_g^* be $\mathrm{evalglobal}^*(z, globaltype^*, expr_\mathsf{g}^*)$.
20. Let ref_t^* be $\mathrm{evalexpr}^*(z, expr_\mathsf{t}^*)$.
21. Let ref_e^{**} be $\mathrm{evalexpr}^{**}(z, expr_\mathsf{e}^{**})$.
22. Pop the frame from the stack.
23. Let (s, f) be the destructuring of z.
24. Let $moduleinst$ be $\mathrm{allocmodule}(s, module, externaddr^*, val_\mathsf{g}^*, ref_\mathsf{t}^*, ref_\mathsf{e}^{**})$.
25. Let F' be the frame $\{\mathsf{module}\ moduleinst\}$.
26. Push the frame F'.
27. Execute the sequence $instr_\mathsf{e}^*$.
28. Execute the sequence $instr_\mathsf{d}^*$.
29. If $start^?$ is defined, then:
    a. Let $(\mathsf{start}\ x)$ be $start^?$.
    b. Let $instr_\mathsf{s}$ be the instruction $(\mathsf{call}\ x)$.
    c. Execute the instruction $instr_\mathsf{s}$.
30. Pop the frame from the stack.

31. Return $moduleinst$.

$$
\begin{array}{l}
\text{instantiate}(s, module, externaddr^*) \quad = \quad s''''; \{\mathsf{module}\ moduleinst\}; instr_{\mathsf{e}}^*\ instr_{\mathsf{d}}^*\ instr_{\mathsf{s}}^? \\
\quad \text{if} \vdash module : xt_{\mathsf{i}}^* \to xt_{\mathsf{e}}^* \\
\quad \wedge\ (s \vdash externaddr : xt_{\mathsf{i}})^* \\
\quad \wedge\ module = \mathsf{module}\ type^*\ import^*\ tag^*\ global^*\ mem^*\ table^*\ func^*\ data^*\ elem^*\ start^?\ export^* \\
\quad \wedge\ global^* = (\mathsf{global}\ globaltype\ expr_{\mathsf{g}})^* \\
\quad \wedge\ table^* = (\mathsf{table}\ tabletype\ expr_{\mathsf{t}})^* \\
\quad \wedge\ data^* = (\mathsf{data}\ byte^*\ datamode)^* \\
\quad \wedge\ elem^* = (\mathsf{elem}\ reftype\ expr_{\mathsf{e}}^*\ elemmode)^* \\
\quad \wedge\ start^? = (\mathsf{start}\ x)^? \\
\quad \wedge\ moduleinst_0 = \{\mathsf{types}\ \text{alloctype}^*(type^*), \\
\qquad\qquad \mathsf{globals}\ \text{globals}(externaddr^*), \\
\qquad\qquad \mathsf{funcs}\ \text{funcs}(externaddr^*)\ (|s.\mathsf{funcs}| + i_{\mathsf{f}})^{i_{\mathsf{f}}<|func^*|}\} \\
\quad \wedge\ z = s; \{\mathsf{module}\ moduleinst_0\} \\
\quad \wedge\ (z', val_{\mathsf{g}}^*) = \text{evalglobal}^*(z, globaltype^*, expr_{\mathsf{g}}^*) \\
\quad \wedge\ (z'', ref_{\mathsf{t}}^*) = \text{evalexpr}^*(z', expr_{\mathsf{t}}^*) \\
\quad \wedge\ (z''', ref_{\mathsf{e}}^{**}) = \text{evalexpr}^{**}(z'', expr_{\mathsf{e}}^{**}) \\
\quad \wedge\ z''' = s'''; f \\
\quad \wedge\ (s'''', moduleinst) = \text{allocmodule}(s''', module, externaddr^*, val_{\mathsf{g}}^*, ref_{\mathsf{t}}^*, (ref_{\mathsf{e}}^*)^*) \\
\quad \wedge\ instr_{\mathsf{d}}^* = \bigoplus \text{rundata}_{i_{\mathsf{d}}}(data^*[i_{\mathsf{d}}])^{i_{\mathsf{d}}<|data^*|} \\
\quad \wedge\ instr_{\mathsf{e}}^* = \bigoplus \text{runelem}_{i_{\mathsf{e}}}(elem^*[i_{\mathsf{e}}])^{i_{\mathsf{e}}<|elem^*|} \\
\quad \wedge\ instr_{\mathsf{s}}^? = (\mathsf{call}\ x)^?
\end{array}
$$

where:

$\text{evalexpr}^*(z, expr''^*)$

1. If $expr''^* = \epsilon$, then:
   a. Return ϵ.
2. Let $expr\ expr'^*$ be $expr''^*$.
3. Let ref be the result of evaluating $expr$ with state z.
4. Let ref'^* be $\text{evalexpr}^*(z, expr'^*)$.
5. Return $ref\ ref'^*$.

$$
\begin{array}{lcl}
\text{evalexpr}^*(z, \epsilon) & = & (z, \epsilon) \\
\text{evalexpr}^*(z, expr\ expr'^*) & = & (z'', ref\ ref'^*) \\
 & & \quad \text{if } z; expr \hookrightarrow^* z'; ref \\
 & & \quad \wedge\ (z'', ref'^*) = \text{evalexpr}^*(z', expr'^*)
\end{array}
$$

and:

$\text{evalglobal}^*(z, globaltype^*, expr''^*)$

1. If $expr''^* = \epsilon$, then:
   a. Assert: Due to validation, $globaltype^* = \epsilon$.
   b. Return ϵ.
2. Else:
   a. Let $expr\ expr'^*$ be $expr''^*$.
   b. Assert: Due to validation, $|globaltype^*| \geq 1$.
   c. Let $gt\ gt'^*$ be $globaltype^*$.
   d. Let val be the result of evaluating $expr$ with state z.
   e. Let (s, f) be the destructuring of z.
   f. Let a be $\text{allocglobal}(s, gt, val)$.

g. Append a to f.module.globals.

h. Let val'^* be $\text{evalglobal}^*((s, f), gt'^*, expr'^*)$.

i. Return $val\ val'^*$.

$$
\begin{array}{lcl}
\text{evalglobal}^*(z, \epsilon, \epsilon) & = & (z, \epsilon) \\
\text{evalglobal}^*(z, gt\ gt'^*, expr\ expr'^*) & = & (z'', val\ val'^*) \\
 & & \quad \text{if } z; expr \hookrightarrow^* z'; val \\
 & & \quad \wedge\ z' = s; f \\
 & & \quad \wedge\ (s', a) = \text{allocglobal}(s, gt, val) \\
 & & \quad \wedge\ (z'', val'^*) = \text{evalglobal}^*((s'; f[\text{.module.globals} =\oplus a]), gt'^*, expr'^*)
\end{array}
$$

$\text{rundata}_x(\text{data}\ b^n\ datamode)$

1. If $datamode = \text{passive}$, then:
   a. Return ϵ.
2. Assert: Due to validation, $datamode$ is some $\text{active}\ memidx\ expr$.
3. Let $(\text{active}\ y\ instr^*)$ be the destructuring of $datamode$.
4. Return $instr^*\ (\text{i32.const}\ 0)\ (\text{i32.const}\ n)\ (\text{memory.init}\ y\ x)\ (\text{data.drop}\ x)$.

$\text{runelem}_x(\text{elem}\ rt\ e^n\ elemmode)$

1. If $elemmode = \text{passive}$, then:
   a. Return ϵ.
2. If $elemmode = \text{declare}$, then:
   a. Return $(\text{elem.drop}\ x)$.
3. Assert: Due to validation, $elemmode$ is some $\text{active}\ tableidx\ expr$.
4. Let $(\text{active}\ y\ instr^*)$ be the destructuring of $elemmode$.
5. Return $instr^*\ (\text{i32.const}\ 0)\ (\text{i32.const}\ n)\ (\text{table.init}\ y\ x)\ (\text{elem.drop}\ x)$.

$$
\begin{array}{lcl}
\text{rundata}_x(\text{data}\ b^n\ (\text{passive})) & = & \epsilon \\
\text{rundata}_x(\text{data}\ b^n\ (\text{active}\ y\ instr^*)) & = & \\
\quad instr^*\ (\text{i32.const}\ 0)\ (\text{i32.const}\ n)\ (\text{memory.init}\ y\ x)\ (\text{data.drop}\ x) & & \\
\text{runelem}_x(\text{elem}\ rt\ e^n\ (\text{passive})) & = & \epsilon \\
\text{runelem}_x(\text{elem}\ rt\ e^n\ (\text{declare})) & = & (\text{elem.drop}\ x) \\
\text{runelem}_x(\text{elem}\ rt\ e^n\ (\text{active}\ y\ instr^*)) & = & \\
\quad instr^*\ (\text{i32.const}\ 0)\ (\text{i32.const}\ n)\ (\text{table.init}\ y\ x)\ (\text{elem.drop}\ x) & &
\end{array}
$$

Note

Checking import types assumes that the module instance has already been allocated to compute the respective closed defined types. However, this forward reference merely is a way to simplify the specification. In practice, implementations will likely allocate or canonicalize types beforehand, when *compiling* a module, in a stage before instantiation and before imports are checked.

Similarly, module allocation and the evaluation of global and table initializers as well as element segments are mutually recursive because the global initialization values val^*_{g}, ref_{t}, and element segment contents ref^{**}_{e} are passed to the module allocator while depending on the module instance $moduleinst$ and store s' returned by allocation. Again, this recursion is just a specification device. In practice, the initialization values can be determined beforehand by staging module allocation such that first, the module's own function instances are pre-allocated in the store, then the initializer expressions are evaluated in order, allocating globals on the way, then the rest of the module instance is allocated, and finally the new function instances' module fields are set to that module instance. This is possible because validation ensures that initialization expressions cannot actually call a function, only take their reference.

All failure conditions are checked before any observable mutation of the store takes place. Store mutation is not atomic; it happens in individual steps that may be interleaved with other threads.

Evaluation of constant expressions does not affect the store.

4.7.3 Invocation

Once a module has been instantiated, any exported function can be *invoked* externally via its function address $funcaddr$ in the store s and an appropriate list val^* of argument values.

Invocation may *fail* with an error if the arguments do not fit the function type. Invocation can also result in an exception or trap. It is up to the embedder to define how such conditions are reported.

Note

If the embedder API performs type checks itself, either statically or dynamically, before performing an invocation, then no failure other than traps or exceptions can occur.

$\text{invoke}(s, funcaddr, val^*)$

1. Assert: Due to validation, the expansion of $s.\text{funcs}[funcaddr].\text{type}$ is some $\text{func}\ resulttype \to resulttype$.
2. Let $(\text{func}\ t_1^*\ \to\ t_2^*)$ be the destructuring of the expansion of $s.\text{funcs}[funcaddr].\text{type}$.
3. If $|t_1^*| \neq |val^*|$, then:
   a. Fail.
4. For all t_1 in t_1^*, and corresponding val in val^*:
   a. If val is not valid with type t_1, then:
      1) Fail.
5. Let k be the length of t_2^*.
6. Let F be the frame $\{\text{module}\ \{\}\}$ whose arity is k.
7. Push the frame F.
8. Push the values val^* to the stack.
9. Push the value $(\text{ref.func}\ funcaddr)$ to the stack.
10. Execute the instruction $(\text{call_ref}\ s.\text{funcs}[funcaddr].\text{type})$.
11. Pop the values ${val'}^k$ from the stack.
12. Pop the frame from the stack.
13. Return ${val'}^k$.

$$\begin{array}{lcl} \text{invoke}(s, funcaddr, val^*) & = & s; \{\text{module}\ \{\}\}; val^*\ (\text{ref.func}\ funcaddr)\ (\text{call_ref}\ s.\text{funcs}[funcaddr].\text{type}) \\ & & \quad \text{if}\ s.\text{funcs}[funcaddr].\text{type} \approx \text{func}\ t_1^* \to t_2^* \\ & & \quad \wedge\ (s \vdash val : t_1)^* \end{array}$$

CHAPTER 5

Binary Format

5.1 Conventions

The binary format for WebAssembly modules is a dense linear *encoding* of their abstract syntax.[28]

The format is defined by an *attribute grammar* whose only terminal symbols are bytes. A byte sequence is a well-formed encoding of a module if and only if it is generated by the grammar.

Each production of this grammar has exactly one synthesized attribute: the abstract syntax that the respective byte sequence encodes. Thus, the attribute grammar implicitly defines a *decoding* function (i.e., a parsing function for the binary format).

Except for a few exceptions, the binary grammar closely mirrors the grammar of the abstract syntax.

Note

Some phrases of abstract syntax have multiple possible encodings in the binary format. For example, numbers may be encoded as if they had optional leading zeros. Implementations of decoders must support all possible alternatives; implementations of encoders can pick any allowed encoding.

The recommended extension for files containing WebAssembly modules in binary format is "`.wasm`" and the recommended Media Type[27] is "`application/wasm`".

5.1.1 Grammar

The following conventions are adopted in defining grammar rules for the binary format. They mirror the conventions used for abstract syntax. In order to distinguish symbols of the binary syntax from symbols of the abstract syntax, `typewriter` font is adopted for the former.

- Terminal symbols are bytes expressed in hexadecimal notation: `0x0F`.
- Nonterminal symbols are written in typewriter font: `valtype`, `instr`.
- B^n is a sequence of $n \geq 0$ iterations of B.
- B^* is a possibly empty sequence of iterations of B. (This is a shorthand for B^n used where n is not relevant.)

[28] Additional encoding layers – for example, introducing compression – may be defined on top of the basic representation defined here. However, such layers are outside the scope of the current specification.

[27] https://www.iana.org/assignments/media-types/media-types.xhtml

- $B^?$ is an optional occurrence of B. (This is a shorthand for B^n where $n \leq 1$.)
- $x{:}B$ denotes the same language as the nonterminal B, but also binds the variable x to the attribute synthesized for B. A pattern may also be used instead of a variable, e.g., $7{:}B$.
- Productions are written $\texttt{sym} ::= B_1 \Rightarrow A_1 \mid \ldots \mid B_n \Rightarrow A_n$, where each A_i is the attribute that is synthesized for `sym` in the given case, usually from attribute variables bound in B_i.
- Large productions may be split into multiple definitions, indicated by ending the first one with explicit ellipses, $\texttt{sym} ::= B_1$, and starting continuations with ellipses, $\texttt{sym} ::= \ldots \mid B_2$.
- Some productions are augmented by side conditions in parentheses, which restrict the applicability of the production. They provide a shorthand for a combinatorial expansion of the production into many separate cases.
- If the same meta variable or non-terminal symbol appears multiple times in a production (in the syntax or in an attribute), then all those occurrences must have the same instantiation. (This is a shorthand for a side condition requiring multiple different variables to be equal.)

Note

For example, the binary grammar for number types is given as follows:

$$\begin{array}{lcll} \texttt{numtype} & ::= & \texttt{0x7C} & \Rightarrow \ \texttt{f64} \\ & \mid & \texttt{0x7D} & \Rightarrow \ \texttt{f32} \\ & \mid & \texttt{0x7E} & \Rightarrow \ \texttt{i64} \\ & \mid & \texttt{0x7F} & \Rightarrow \ \texttt{i32} \end{array}$$

Consequently, the byte `0x7F` encodes the type i32, `0x7E` encodes the type i64, and so forth. No other byte value is allowed as the encoding of a number type.

The binary grammar for limits is defined as follows:

$$\begin{array}{lcll} \texttt{limits} & ::= & \texttt{0x00}\ n{:}\texttt{u64} & \Rightarrow \ (\texttt{i32}, [n \mathbin{..} \epsilon]) \\ & \mid & \texttt{0x01}\ n{:}\texttt{u64}\ m{:}\texttt{u64} & \Rightarrow \ (\texttt{i32}, [n \mathbin{..} m]) \\ & \mid & \texttt{0x04}\ n{:}\texttt{u64} & \Rightarrow \ (\texttt{i64}, [n \mathbin{..} \epsilon]) \\ & \mid & \texttt{0x05}\ n{:}\texttt{u64}\ m{:}\texttt{u64} & \Rightarrow \ (\texttt{i64}, [n \mathbin{..} m]) \end{array}$$

That is, a limits pair is encoded as either the byte `0x00` followed by the encoding of a $u64$ value, or the byte `0x01` followed by two such encodings. The variables n and m name the attributes of the respective u64 nonterminals, which in this case are the actual unsigned integers those decode into. The attribute of the complete production then is the abstract syntax for the limit, expressed in terms of the former values.

5.1.2 Auxiliary Notation

When dealing with binary encodings the following notation is also used:

- ϵ denotes the empty byte sequence.
- $||\texttt{B}||$ is the length of the byte sequence generated from the production `B` in a derivation.

5.1.3 Lists

Lists are encoded with their u32 length followed by the encoding of their element sequence.

$$\texttt{list}(\texttt{X}) \quad ::= \quad n{:}\texttt{u32}\ (el{:}\texttt{X})^n \quad \Rightarrow \quad el^n$$

5.2 Values

5.2.1 Bytes

Bytes encode themselves.

$$\texttt{byte} \quad ::= \quad \texttt{0x00} \mid \ldots \mid \texttt{0xFF}$$

5.2.2 Integers

All integers are encoded using the LEB128[29] variable-length integer encoding, in either unsigned or signed variant.

Unsigned integers are encoded in unsigned LEB128[30] format. As an additional constraint, the total number of bytes encoding a $u N$ value must not exceed $\text{ceil}(N/7)$ bytes.

$$\begin{array}{rcll} \texttt{u}N & ::= & n\texttt{:byte} & \Rightarrow \quad n & \text{if } n < 2^7 \wedge n < 2^N \\ & \mid & n\texttt{:byte}\ m\texttt{:u}(N-7) & \Rightarrow \quad 2^7 \cdot m + (n - 2^7) & \text{if } n \geq 2^7 \wedge N > 7 \end{array}$$

Signed integers are encoded in signed LEB128[31] format, which uses a two's complement representation. As an additional constraint, the total number of bytes encoding an $s N$ value must not exceed $\text{ceil}(N/7)$ bytes.

$$\begin{array}{rcll} \texttt{s}N & ::= & n\texttt{:byte} & \Rightarrow \quad n & \text{if } n < 2^6 \wedge n < 2^{N-1} \\ & \mid & n\texttt{:byte} & \Rightarrow \quad n - 2^7 & \text{if } 2^6 \leq n < 2^7 \wedge n \geq 2^7 - 2^{N-1} \\ & \mid & n\texttt{:byte}\ i\texttt{:s}(N-7) & \Rightarrow \quad 2^7 \cdot i + (n - 2^7) & \text{if } n \geq 2^7 \wedge N > 7 \end{array}$$

Uninterpreted integers are encoded as signed integers.

$$\texttt{i}N \quad ::= \quad i\texttt{:s}N \quad \Rightarrow \quad \text{signed}_N^{-1}(i)$$

Note

The side conditions $N > 7$ in the productions for non-terminal bytes of the $u N$ and $s N$ encodings restrict the encoding's length. However, "trailing zeros" are still allowed within these bounds. For example, `0x03` and `0x83 0x00` are both well-formed encodings for the value 3 as a $u8$. Similarly, either of `0x7E` and `0xFE 0x7F` and `0xFE 0xFF 0x7F` are well-formed encodings of the value -2 as an $s16$.

The side conditions on the value n of terminal bytes further enforce that any unused bits in these bytes must be 0 for positive values and 1 for negative ones. For example, `0x83 0x10` is malformed as a $u8$ encoding. Similarly, both `0x83 0x3E` and `0xFF 0x7B` are malformed as $s8$ encodings.

5.2.3 Floating-Point

Floating-point values are encoded directly by their IEEE 754[32] (Section 3.4) bit pattern in little endian[33] byte order:

$$\texttt{f}N \quad ::= \quad b^*\texttt{:byte}^{N/8} \quad \Rightarrow \quad \text{bytes}_{\texttt{f}N}^{-1}(b^*)$$

[29] https://en.wikipedia.org/wiki/LEB128
[30] https://en.wikipedia.org/wiki/LEB128#Unsigned_LEB128
[31] https://en.wikipedia.org/wiki/LEB128#Signed_LEB128
[32] https://ieeexplore.ieee.org/document/8766229
[33] https://en.wikipedia.org/wiki/Endianness#Little-endian

5.2.4 Names

Names are encoded as a list of bytes containing the Unicode[34] (Section 3.9) UTF-8 encoding of the name's character sequence.

$$\texttt{name} \quad ::= \quad b^*{:}\texttt{list}(\texttt{byte}) \quad \Rightarrow \quad \mathit{name} \quad \text{if } \text{utf8}(\mathit{name}) = b^*$$

The auxiliary utf8 function expressing this encoding is defined as follows:

$$\begin{array}{lcll}
\text{utf8}(ch^*) & = & \bigoplus \text{utf8}(ch)^* & \\
\text{utf8}(ch) & = & b & \text{if } ch < \text{U+80} \\
 & & & \wedge\ ch = b \\
\text{utf8}(ch) & = & b_1\ b_2 & \text{if } \text{U+80} \leq ch < \text{U+0800} \\
 & & & \wedge\ ch = 2^6 \cdot (b_1 - \texttt{0xC0}) + \text{cont}(b_2) \\
\text{utf8}(ch) & = & b_1\ b_2\ b_3 & \text{if } \text{U+0800} \leq ch < \text{U+D800} \vee \text{U+E000} \leq ch < \text{U+10000} \\
 & & & \wedge\ ch = 2^{12} \cdot (b_1 - \texttt{0xE0}) + 2^6 \cdot \text{cont}(b_2) + \text{cont}(b_3) \\
\text{utf8}(ch) & = & b_1\ b_2\ b_3\ b_4 & \text{if } \text{U+10000} \leq ch < \text{U+110000} \\
 & & & \wedge\ ch = 2^{18} \cdot (b_1 - \texttt{0xF0}) + 2^{12} \cdot \text{cont}(b_2) + 2^6 \cdot \text{cont}(b_3) + \text{cont}(b_4)
\end{array}$$

where $\text{cont}(b) = b - \texttt{0x80} \quad \text{if } \texttt{0x80} \leq b < \texttt{0xC0}$

Note

Unlike in some other formats, name strings are not 0-terminated.

5.3 Types

Note

In some places, possible types include both type constructors or types denoted by type indices. Thus, the binary format for type constructors corresponds to the encodings of small negative sN values, such that they can unambiguously occur in the same place as (positive) type indices.

5.3.1 Number Types

Number types are encoded by a single byte.

$$\begin{array}{rcll}
\texttt{numtype} & ::= & \texttt{0x7C} & \Rightarrow \quad \texttt{f64} \\
 & | & \texttt{0x7D} & \Rightarrow \quad \texttt{f32} \\
 & | & \texttt{0x7E} & \Rightarrow \quad \texttt{i64} \\
 & | & \texttt{0x7F} & \Rightarrow \quad \texttt{i32}
\end{array}$$

5.3.2 Vector Types

Vector types are also encoded by a single byte.

$$\texttt{vectype} \quad ::= \quad \texttt{0x7B} \quad \Rightarrow \quad \texttt{v128}$$

[34] https://www.unicode.org/versions/latest/

5.3.3 Heap Types

Heap types are encoded as either a single byte, or as a type index encoded as a positive signed integer.

$$
\begin{array}{rcll}
\texttt{absheaptype} & ::= & \texttt{0x69} & \Rightarrow \ \texttt{exn} \\
& | & \texttt{0x6A} & \Rightarrow \ \texttt{array} \\
& | & \texttt{0x6B} & \Rightarrow \ \texttt{struct} \\
& | & \texttt{0x6C} & \Rightarrow \ \texttt{i31} \\
& | & \texttt{0x6D} & \Rightarrow \ \texttt{eq} \\
& | & \texttt{0x6E} & \Rightarrow \ \texttt{any} \\
& | & \texttt{0x6F} & \Rightarrow \ \texttt{extern} \\
& | & \texttt{0x70} & \Rightarrow \ \texttt{func} \\
& | & \texttt{0x71} & \Rightarrow \ \texttt{none} \\
& | & \texttt{0x72} & \Rightarrow \ \texttt{noextern} \\
& | & \texttt{0x73} & \Rightarrow \ \texttt{nofunc} \\
& | & \texttt{0x74} & \Rightarrow \ \texttt{noexn} \\
\texttt{heaptype} & ::= & ht\texttt{:absheaptype} & \Rightarrow \ ht \\
& | & x\texttt{:s33} & \Rightarrow \ x \quad \text{if } x \geq 0
\end{array}
$$

Note

The heap type bot cannot occur in a module.

5.3.4 Reference Types

Reference types are either encoded by a single byte followed by a heap type, or, as a short form, directly as an abstract heap type.

$$
\begin{array}{rcll}
\texttt{reftype} & ::= & \texttt{0x63}\ ht\texttt{:heaptype} & \Rightarrow \ \texttt{ref null}\ ht \\
& | & \texttt{0x64}\ ht\texttt{:heaptype} & \Rightarrow \ \texttt{ref}\ ht \\
& | & ht\texttt{:absheaptype} & \Rightarrow \ \texttt{ref null}\ ht
\end{array}
$$

5.3.5 Value Types

Value types are encoded with their respective encoding as a number type, vector type, or reference type.

$$
\begin{array}{rcll}
\texttt{valtype} & ::= & nt\texttt{:numtype} & \Rightarrow \ nt \\
& | & vt\texttt{:vectype} & \Rightarrow \ vt \\
& | & rt\texttt{:reftype} & \Rightarrow \ rt
\end{array}
$$

Note

The value type bot cannot occur in a module.

Value types can occur in contexts where type indices are also allowed, such as in the case of block types. Thus, the binary format for types corresponds to the signed LEB128[35] encoding of small negative sN values, so that they can coexist with (positive) type indices in the future.

5.3.6 Result Types

Result types are encoded by the respective lists of value types.

$$
\texttt{resulttype} \ ::= \ t^*\texttt{:list(valtype)} \ \Rightarrow \ t^*
$$

[35] https://en.wikipedia.org/wiki/LEB128#Signed_LEB128

5.3.7 Composite Types

Composite types are encoded by a distinct byte followed by a type encoding of the respective form.

$$
\begin{array}{rcll}
\texttt{mut} & ::= & \texttt{0x00} & \Rightarrow \quad \epsilon \\
& | & \texttt{0x01} & \Rightarrow \quad \texttt{mut} \\
\texttt{comptype} & ::= & \texttt{0x5E}\ ft{:}\texttt{fieldtype} & \Rightarrow \quad \texttt{array}\ ft \\
& | & \texttt{0x5F}\ ft^*{:}\texttt{list(fieldtype)} & \Rightarrow \quad \texttt{struct}\ ft^* \\
& | & \texttt{0x60}\ t_1^*{:}\texttt{resulttype}\ t_2^*{:}\texttt{resulttype} & \Rightarrow \quad \texttt{func}\ t_1^* \to t_2^* \\
\texttt{fieldtype} & ::= & zt{:}\texttt{storagetype}\ \texttt{mut}^?{:}\texttt{mut} & \Rightarrow \quad \texttt{mut}^?\ zt \\
\texttt{storagetype} & ::= & t{:}\texttt{valtype} & \Rightarrow \quad t \\
& | & pt{:}\texttt{packtype} & \Rightarrow \quad pt \\
\texttt{packtype} & ::= & \texttt{0x77} & \Rightarrow \quad \texttt{i16} \\
& | & \texttt{0x78} & \Rightarrow \quad \texttt{i8}
\end{array}
$$

5.3.8 Recursive Types

Recursive types are encoded by the byte 0x4E followed by a list of sub types. Additional shorthands are recognized for unary recursions and sub types without super types.

$$
\begin{array}{rcll}
\texttt{rectype} & ::= & \texttt{0x4E}\ st^*{:}\texttt{list(subtype)} & \Rightarrow \quad \texttt{rec}\ st^* \\
& | & st{:}\texttt{subtype} & \Rightarrow \quad \texttt{rec}\ st \\
\texttt{subtype} & ::= & \texttt{0x4F}\ x^*{:}\texttt{list(typeidx)}\ ct{:}\texttt{comptype} & \Rightarrow \quad \texttt{sub final}\ x^*\ ct \\
& | & \texttt{0x50}\ x^*{:}\texttt{list(typeidx)}\ ct{:}\texttt{comptype} & \Rightarrow \quad \texttt{sub}\ x^*\ ct \\
& | & ct{:}\texttt{comptype} & \Rightarrow \quad \texttt{sub final}\ \epsilon\ ct
\end{array}
$$

5.3.9 Limits

Limits are encoded with a preceding flag indicating whether a maximum is present, and a flag for the address type.

$$
\begin{array}{rcll}
\texttt{limits} & ::= & \texttt{0x00}\ n{:}\texttt{u64} & \Rightarrow \quad (\texttt{i32}, [n\,..\,\epsilon]) \\
& | & \texttt{0x01}\ n{:}\texttt{u64}\ m{:}\texttt{u64} & \Rightarrow \quad (\texttt{i32}, [n\,..\,m]) \\
& | & \texttt{0x04}\ n{:}\texttt{u64} & \Rightarrow \quad (\texttt{i64}, [n\,..\,\epsilon]) \\
& | & \texttt{0x05}\ n{:}\texttt{u64}\ m{:}\texttt{u64} & \Rightarrow \quad (\texttt{i64}, [n\,..\,m])
\end{array}
$$

5.3.10 Tag Types

Tag types are encoded by a type index denoting a function type.

$$
\texttt{tagtype} \quad ::= \quad \texttt{0x00}\ x{:}\texttt{typeidx} \quad \Rightarrow \quad x
$$

Note

In future versions of WebAssembly, the preceding zero byte may encode additional attributes.

5.3.11 Global Types

Global types are encoded by their value type and a flag for their mutability.

$$
\texttt{globaltype} \quad ::= \quad t{:}\texttt{valtype}\ \texttt{mut}^?{:}\texttt{mut} \quad \Rightarrow \quad \texttt{mut}^?\ t
$$

5.3.12 Memory Types

Memory types are encoded with their limits.

$$\texttt{memtype} \quad ::= \quad (at, lim)\texttt{:limits} \quad \Rightarrow \quad at\ lim\ \texttt{page}$$

5.3.13 Table Types

Table types are encoded with their limits and the encoding of their element reference type.

$$\texttt{tabletype} \quad ::= \quad rt\texttt{:reftype}\ (at, lim)\texttt{:limits} \quad \Rightarrow \quad at\ lim\ rt$$

5.3.14 External Types

External types are encoded by a distiguishing byte followed by an encoding of the respective form of type.

$$\begin{array}{lcll} \texttt{externtype} & ::= & \texttt{0x00}\ x\texttt{:typeidx} & \Rightarrow \quad \texttt{func}\ x \\ & | & \texttt{0x01}\ tt\texttt{:tabletype} & \Rightarrow \quad \texttt{table}\ tt \\ & | & \texttt{0x02}\ mt\texttt{:memtype} & \Rightarrow \quad \texttt{mem}\ mt \\ & | & \texttt{0x03}\ gt\texttt{:globaltype} & \Rightarrow \quad \texttt{global}\ gt \\ & | & \texttt{0x04}\ jt\texttt{:tagtype} & \Rightarrow \quad \texttt{tag}\ jt \end{array}$$

5.4 Instructions

Instructions are encoded by *opcodes*. Each opcode is represented by a single byte, and is followed by the instruction's immediate arguments, where present. The only exception are structured control instructions, which consist of several opcodes bracketing their nested instruction sequences.

Note

The byte codes chosen to encode instructions are historical and do not follow a consistent pattern. In this section, instructions are hence not presented in opcode order, but instead grouped consistently with other sections in this document. An instruction index ordered by opcode can be found in the Appendix.

Gaps in the byte code ranges are reserved for future extensions.

5.4.1 Parametric Instructions

Parametric instructions are represented by single byte codes, possibly followed by a type annotation.

$$\begin{array}{lcll} \texttt{instr} & ::= & \texttt{0x00} & \Rightarrow \quad \texttt{unreachable} \\ & | & \texttt{0x01} & \Rightarrow \quad \texttt{nop} \\ & | & \texttt{0x1A} & \Rightarrow \quad \texttt{drop} \\ & | & \texttt{0x1B} & \Rightarrow \quad \texttt{select} \\ & | & \texttt{0x1C}\ t^*\texttt{:list(valtype)} & \Rightarrow \quad \texttt{select}\ t^* \end{array}$$

5.4.2 Control Instructions

Control instructions have varying encodings. For structured instructions, the instruction sequences forming nested blocks are delimited with explicit opcodes for end and else.

Block types are encoded in special compressed form, by either the byte 0x40 indicating the empty type, as a single value type, or as a type index encoded as a positive signed integer.

$$
\begin{array}{lcll}
\text{blocktype} & ::= & \texttt{0x40} & \Rightarrow \epsilon \\
& | & t{:}\texttt{valtype} & \Rightarrow t \\
& | & i{:}\texttt{s33} & \Rightarrow i \\
\text{instr} & ::= & \ldots & \\
& | & \texttt{0x02}\ bt{:}\texttt{blocktype}\ (in{:}\texttt{instr})^*\ \texttt{0x0B} & \Rightarrow \texttt{block}\ bt\ in^* \\
& | & \texttt{0x03}\ bt{:}\texttt{blocktype}\ (in{:}\texttt{instr})^*\ \texttt{0x0B} & \Rightarrow \texttt{loop}\ bt\ in^* \\
& | & \texttt{0x04}\ bt{:}\texttt{blocktype}\ (in{:}\texttt{instr})^*\ \texttt{0x0B} & \Rightarrow \texttt{if}\ bt\ in^*\ \texttt{else}\ \epsilon \\
& | & \texttt{0x04}\ bt{:}\texttt{blocktype}\ (in_1{:}\texttt{instr})^* & \Rightarrow \texttt{if}\ bt\ in_1^*\ \texttt{else}\ in_2^* \\
& & \texttt{0x05}\ (in_2{:}\texttt{instr})^*\ \texttt{0x0B} & \\
& | & \texttt{0x08}\ x{:}\texttt{tagidx} & \Rightarrow \texttt{throw}\ x \\
& | & \texttt{0x0A} & \Rightarrow \texttt{throw_ref} \\
& | & \texttt{0x0C}\ l{:}\texttt{labelidx} & \Rightarrow \texttt{br}\ l \\
& | & \texttt{0x0D}\ l{:}\texttt{labelidx} & \Rightarrow \texttt{br_if}\ l \\
& | & \texttt{0x0E}\ l^*{:}\texttt{list(labelidx)}\ l_n{:}\texttt{labelidx} & \Rightarrow \texttt{br_table}\ l^*\ l_n \\
& | & \texttt{0x0F} & \Rightarrow \texttt{return} \\
& | & \texttt{0x10}\ x{:}\texttt{funcidx} & \Rightarrow \texttt{call}\ x \\
& | & \texttt{0x11}\ y{:}\texttt{typeidx}\ x{:}\texttt{tableidx} & \Rightarrow \texttt{call_indirect}\ x\ y \\
& | & \texttt{0x12}\ x{:}\texttt{funcidx} & \Rightarrow \texttt{return_call}\ x \\
& | & \texttt{0x13}\ y{:}\texttt{typeidx}\ x{:}\texttt{tableidx} & \Rightarrow \texttt{return_call_indirect}\ x\ y \\
& | & \texttt{0x14}\ x{:}\texttt{typeidx} & \Rightarrow \texttt{call_ref}\ x \\
& | & \texttt{0x15}\ x{:}\texttt{typeidx} & \Rightarrow \texttt{return_call_ref}\ x \\
& | & \texttt{0x1F}\ bt{:}\texttt{blocktype}\ c^*{:}\texttt{list(catch)}\ (in{:}\texttt{instr})^*\ \texttt{0x0B} & \Rightarrow \texttt{try_table}\ bt\ c^*\ in^* \\
& | & \texttt{0xD5}\ l{:}\texttt{labelidx} & \Rightarrow \texttt{br_on_null}\ l \\
& | & \texttt{0xD6}\ l{:}\texttt{labelidx} & \Rightarrow \texttt{br_on_non_null}\ l \\
& | & \texttt{0xFB}\ 24{:}\texttt{u32}\ (\texttt{null}_1^?, \texttt{null}_2^?){:}\texttt{castop} & \Rightarrow \texttt{br_on_cast}\ l\ (\texttt{ref null}_1^?\ ht_1)\ (\texttt{ref null}_2^?\ ht_2) \\
& & l{:}\texttt{labelidx}\ ht_1{:}\texttt{heaptype}\ ht_2{:}\texttt{heaptype} & \\
& | & \texttt{0xFB}\ 25{:}\texttt{u32}\ (\texttt{null}_1^?, \texttt{null}_2^?){:}\texttt{castop} & \Rightarrow \texttt{br_on_cast_fail}\ l\ (\texttt{ref null}_1^?\ ht_1)\ (\texttt{ref null}_2^?\ \\
& & l{:}\texttt{labelidx}\ ht_1{:}\texttt{heaptype}\ ht_2{:}\texttt{heaptype} & \\
\text{catch} & ::= & \texttt{0x00}\ x{:}\texttt{tagidx}\ l{:}\texttt{labelidx} & \Rightarrow \texttt{catch}\ x\ l \\
& | & \texttt{0x01}\ x{:}\texttt{tagidx}\ l{:}\texttt{labelidx} & \Rightarrow \texttt{catch_ref}\ x\ l \\
& | & \texttt{0x02}\ l{:}\texttt{labelidx} & \Rightarrow \texttt{catch_all}\ l \\
& | & \texttt{0x03}\ l{:}\texttt{labelidx} & \Rightarrow \texttt{catch_all_ref}\ l \\
\text{castop} & ::= & \texttt{0x00} & \Rightarrow (\epsilon, \epsilon) \\
& | & \texttt{0x01} & \Rightarrow (\texttt{null}, \epsilon) \\
& | & \texttt{0x02} & \Rightarrow (\epsilon, \texttt{null}) \\
& | & \texttt{0x03} & \Rightarrow (\texttt{null}, \texttt{null})
\end{array}
$$

Note

The else opcode 0x05 in the encoding of an if instruction can be omitted if the following instruction sequence is empty.

Unlike any other occurrence, the type index in a block type is encoded as a positive signed integer, so that its signed LEB128 bit pattern cannot collide with the encoding of value types or the special code 0x40, which correspond to the LEB128 encoding of negative integers. To avoid any loss in the range of allowed indices, it is treated as a 33 bit signed integer.

5.4.3 Variable Instructions

Variable instructions are represented by byte codes followed by the encoding of the respective index.

```
instr ::= ...
        | 0x20 x:localidx    ⇒ local.get x
        | 0x21 x:localidx    ⇒ local.set x
        | 0x22 x:localidx    ⇒ local.tee x
        | 0x23 x:globalidx   ⇒ global.get x
        | 0x24 x:globalidx   ⇒ global.set x
```

5.4.4 Table Instructions

Table instructions are represented either by a single byte or a one byte prefix followed by a variable-length unsigned integer.

```
instr ::= ...
        | 0x25 x:tableidx                        ⇒ table.get x
        | 0x26 x:tableidx                        ⇒ table.set x
        | 0xFC 12:u32 y:elemidx x:tableidx       ⇒ table.init x y
        | 0xFC 13:u32 x:elemidx                  ⇒ elem.drop x
        | 0xFC 14:u32 x1:tableidx x2:tableidx    ⇒ table.copy x1 x2
        | 0xFC 15:u32 x:tableidx                  ⇒ table.grow x
        | 0xFC 16:u32 x:tableidx                  ⇒ table.size x
        | 0xFC 17:u32 x:tableidx                  ⇒ table.fill x
```

5.4.5 Memory Instructions

Each variant of memory instruction is encoded with a different byte code. Loads and stores are followed by the encoding of their *memarg* immediate, which includes the memory index if bit 6 of the flags field containing alignment is set; the memory index defaults to 0 otherwise.

```
memarg ::= n:u32 m:u64                          ⇒ (0, {align n, offset m})          if n < 2^6
         | n:u32 x:memidx m:u64                 ⇒ (x, {align (n − 2^6), offset m})  if 2^6 ≤ n < 2^7

instr  ::= ...
         | 0x28 (x, ao):memarg                  ⇒ i32.load x ao
         | 0x29 (x, ao):memarg                  ⇒ i64.load x ao
         | 0x2A (x, ao):memarg                  ⇒ f32.load x ao
         | 0x2B (x, ao):memarg                  ⇒ f64.load x ao
         | 0x2C (x, ao):memarg                  ⇒ i32.load8_s x ao
         | 0x2D (x, ao):memarg                  ⇒ i32.load8_u x ao
         | 0x2E (x, ao):memarg                  ⇒ i32.load16_s x ao
         | 0x2F (x, ao):memarg                  ⇒ i32.load16_u x ao
         | 0x30 (x, ao):memarg                  ⇒ i64.load8_s x ao
         | 0x31 (x, ao):memarg                  ⇒ i64.load8_u x ao
         | 0x32 (x, ao):memarg                  ⇒ i64.load16_s x ao
         | 0x33 (x, ao):memarg                  ⇒ i64.load16_u x ao
         | 0x34 (x, ao):memarg                  ⇒ i64.load32_s x ao
         | 0x35 (x, ao):memarg                  ⇒ i64.load32_u x ao
         | 0x36 (x, ao):memarg                  ⇒ i32.store x ao
         | 0x37 (x, ao):memarg                  ⇒ i64.store x ao
         | 0x38 (x, ao):memarg                  ⇒ f32.store x ao
         | 0x39 (x, ao):memarg                  ⇒ f64.store x ao
         | 0x3A (x, ao):memarg                  ⇒ i32.store8 x ao
         | 0x3B (x, ao):memarg                  ⇒ i32.store16 x ao
         | 0x3C (x, ao):memarg                  ⇒ i64.store8 x ao
         | 0x3D (x, ao):memarg                  ⇒ i64.store16 x ao
         | 0x3E (x, ao):memarg                  ⇒ i64.store32 x ao
         | 0x3F x:memidx                        ⇒ memory.size x
         | 0x40 x:memidx                        ⇒ memory.grow x
         | 0xFC 8:u32 y:dataidx x:memidx        ⇒ memory.init x y
         | 0xFC 9:u32 x:dataidx                 ⇒ data.drop x
         | 0xFC 10:u32 x1:memidx x2:memidx      ⇒ memory.copy x1 x2
         | 0xFC 11:u32 x:memidx                 ⇒ memory.fill x
```

5.4.6 Reference Instructions

Generic reference instructions are represented by single byte codes, others use prefixes and type operands.

```
instr ::= ...
        | 0xD0 ht:heaptype          ⇒ ref.null ht
        | 0xD1                      ⇒ ref.is_null
        | 0xD2 x:funcidx            ⇒ ref.func x
        | 0xD3                      ⇒ ref.eq
        | 0xD4                      ⇒ ref.as_non_null
        | 0xFB 20:u32 ht:heaptype   ⇒ ref.test (ref ht)
        | 0xFB 21:u32 ht:heaptype   ⇒ ref.test (ref null ht)
        | 0xFB 22:u32 ht:heaptype   ⇒ ref.cast (ref ht)
        | 0xFB 23:u32 ht:heaptype   ⇒ ref.cast (ref null ht)
```

5.4.7 Aggregate Instructions

Aggregate instructions all use a prefix.

```
instr ::= ...
        | 0xFB 0:u32 x:typeidx                ⇒ struct.new x
        | 0xFB 1:u32 x:typeidx                ⇒ struct.new_default x
        | 0xFB 2:u32 x:typeidx i:fieldidx     ⇒ struct.get x i
        | 0xFB 3:u32 x:typeidx i:fieldidx     ⇒ struct.get_s x i
        | 0xFB 4:u32 x:typeidx i:fieldidx     ⇒ struct.get_u x i
        | 0xFB 5:u32 x:typeidx i:fieldidx     ⇒ struct.set x i
        | 0xFB 6:u32 x:typeidx                ⇒ array.new x
        | 0xFB 7:u32 x:typeidx                ⇒ array.new_default x
        | 0xFB 8:u32 x:typeidx n:u32          ⇒ array.new_fixed x n
        | 0xFB 9:u32 x:typeidx y:dataidx      ⇒ array.new_data x y
        | 0xFB 10:u32 x:typeidx y:elemidx     ⇒ array.new_elem x y
        | 0xFB 11:u32 x:typeidx               ⇒ array.get x
        | 0xFB 12:u32 x:typeidx               ⇒ array.get_s x
        | 0xFB 13:u32 x:typeidx               ⇒ array.get_u x
        | 0xFB 14:u32 x:typeidx               ⇒ array.set x
        | 0xFB 15:u32                         ⇒ array.len
        | 0xFB 16:u32 x:typeidx               ⇒ array.fill x
        | 0xFB 17:u32 x1:typeidx x2:typeidx   ⇒ array.copy x1 x2
        | 0xFB 18:u32 x:typeidx y:dataidx     ⇒ array.init_data x y
        | 0xFB 19:u32 x:typeidx y:elemidx     ⇒ array.init_elem x y
        | 0xFB 26:u32                         ⇒ any.convert_extern
        | 0xFB 27:u32                         ⇒ extern.convert_any
        | 0xFB 28:u32                         ⇒ ref.i31
        | 0xFB 29:u32                         ⇒ i31.get_s
        | 0xFB 30:u32                         ⇒ i31.get_u
```

5.4.8 Numeric Instructions

All variants of numeric instructions are represented by separate byte codes.

The const instructions are followed by the respective literal.

```
instr ::= ...
        | 0x41 i:i32   ⇒ i32.const i
        | 0x42 i:i64   ⇒ i64.const i
        | 0x43 p:f32   ⇒ f32.const p
        | 0x44 p:f64   ⇒ f64.const p
```

All other numeric instructions are plain opcodes without any immediates.

```
instr ::= ...
        | 0x45 ⇒ i32.eqz
        | 0x46 ⇒ i32.eq
        | 0x47 ⇒ i32.ne
        | 0x48 ⇒ i32.lt_s
        | 0x49 ⇒ i32.lt_u
        | 0x4A ⇒ i32.gt_s
        | 0x4B ⇒ i32.gt_u
        | 0x4C ⇒ i32.le_s
        | 0x4D ⇒ i32.le_u
        | 0x4E ⇒ i32.ge_s
        | 0x4F ⇒ i32.ge_u
        | 0x50 ⇒ i64.eqz
        | 0x51 ⇒ i64.eq
        | 0x52 ⇒ i64.ne
        | 0x53 ⇒ i64.lt_s
        | 0x54 ⇒ i64.lt_u
        | 0x55 ⇒ i64.gt_s
        | 0x56 ⇒ i64.gt_u
        | 0x57 ⇒ i64.le_s
        | 0x58 ⇒ i64.le_u
        | 0x59 ⇒ i64.ge_s
        | 0x5A ⇒ i64.ge_u

instr ::= ...
        | 0x5B ⇒ f32.eq
        | 0x5C ⇒ f32.ne
        | 0x5D ⇒ f32.lt
        | 0x5E ⇒ f32.gt
        | 0x5F ⇒ f32.le
        | 0x60 ⇒ f32.ge
        | 0x61 ⇒ f64.eq
        | 0x62 ⇒ f64.ne
        | 0x63 ⇒ f64.lt
        | 0x64 ⇒ f64.gt
        | 0x65 ⇒ f64.le
        | 0x66 ⇒ f64.ge
```

```
instr ::= ...
        | 0x67 ⇒ i32.clz
        | 0x68 ⇒ i32.ctz
        | 0x69 ⇒ i32.popcnt
        | 0x6A ⇒ i32.add
        | 0x6B ⇒ i32.sub
        | 0x6C ⇒ i32.mul
        | 0x6D ⇒ i32.div_s
        | 0x6E ⇒ i32.div_u
        | 0x6F ⇒ i32.rem_s
        | 0x70 ⇒ i32.rem_u
        | 0x71 ⇒ i32.and
        | 0x72 ⇒ i32.or
        | 0x73 ⇒ i32.xor
        | 0x74 ⇒ i32.shl
        | 0x75 ⇒ i32.shr_s
        | 0x76 ⇒ i32.shr_u
        | 0x77 ⇒ i32.rotl
        | 0x78 ⇒ i32.rotr
        | 0x79 ⇒ i64.clz
        | 0x7A ⇒ i64.ctz
        | 0x7B ⇒ i64.popcnt
        | 0x7C ⇒ i64.add
        | 0x7D ⇒ i64.sub
        | 0x7E ⇒ i64.mul
        | 0x7F ⇒ i64.div_s
        | 0x80 ⇒ i64.div_u
        | 0x81 ⇒ i64.rem_s
        | 0x82 ⇒ i64.rem_u
        | 0x83 ⇒ i64.and
        | 0x84 ⇒ i64.or
        | 0x85 ⇒ i64.xor
        | 0x86 ⇒ i64.shl
        | 0x87 ⇒ i64.shr_s
        | 0x88 ⇒ i64.shr_u
        | 0x89 ⇒ i64.rotl
        | 0x8A ⇒ i64.rotr
```

```
instr ::= ...
        | 0x8B ⇒ f32.abs
        | 0x8C ⇒ f32.neg
        | 0x8D ⇒ f32.ceil
        | 0x8E ⇒ f32.floor
        | 0x8F ⇒ f32.trunc
        | 0x90 ⇒ f32.nearest
        | 0x91 ⇒ f32.sqrt
        | 0x92 ⇒ f32.add
        | 0x93 ⇒ f32.sub
        | 0x94 ⇒ f32.mul
        | 0x95 ⇒ f32.div
        | 0x96 ⇒ f32.min
        | 0x97 ⇒ f32.max
        | 0x98 ⇒ f32.copysign
        | 0x99 ⇒ f64.abs
        | 0x9A ⇒ f64.neg
        | 0x9B ⇒ f64.ceil
        | 0x9C ⇒ f64.floor
        | 0x9D ⇒ f64.trunc
        | 0x9E ⇒ f64.nearest
        | 0x9F ⇒ f64.sqrt
        | 0xA0 ⇒ f64.add
        | 0xA1 ⇒ f64.sub
        | 0xA2 ⇒ f64.mul
        | 0xA3 ⇒ f64.div
        | 0xA4 ⇒ f64.min
        | 0xA5 ⇒ f64.max
        | 0xA6 ⇒ f64.copysign
```

```
instr ::= ...
        | 0xA7 ⇒ i32.wrap_i64
        | 0xA8 ⇒ i32.trunc_s_f32
        | 0xA9 ⇒ i32.trunc_u_f32
        | 0xAA ⇒ i32.trunc_s_f64
        | 0xAB ⇒ i32.trunc_u_f64
        | 0xAC ⇒ i64.extend_s_i32
        | 0xAD ⇒ i64.extend_u_i32
        | 0xAE ⇒ i64.trunc_s_f32
        | 0xAF ⇒ i64.trunc_u_f32
        | 0xB0 ⇒ i64.trunc_s_f64
        | 0xB1 ⇒ i64.trunc_u_f64
        | 0xB2 ⇒ f32.convert_s_i32
        | 0xB3 ⇒ f32.convert_u_i32
        | 0xB4 ⇒ f32.convert_s_i64
        | 0xB5 ⇒ f32.convert_u_i64
        | 0xB6 ⇒ f32.demote_f64
        | 0xB7 ⇒ f64.convert_s_i32
        | 0xB8 ⇒ f64.convert_u_i32
        | 0xB9 ⇒ f64.convert_s_i64
        | 0xBA ⇒ f64.convert_u_i64
        | 0xBB ⇒ f64.promote_f32
        | 0xBC ⇒ i32.reinterpret_f32
        | 0xBD ⇒ i64.reinterpret_f64
        | 0xBE ⇒ f32.reinterpret_i32
        | 0xBF ⇒ f64.reinterpret_i64
```

```
instr ::= ...
        | 0xC0 ⇒ i32.extend8_s
        | 0xC1 ⇒ i32.extend16_s
        | 0xC2 ⇒ i64.extend8_s
        | 0xC3 ⇒ i64.extend16_s
        | 0xC4 ⇒ i64.extend32_s
```

The saturating truncation instructions all have a one byte prefix, whereas the actual opcode is encoded by a variable-length unsigned integer.

```
instr ::= ...
        | 0xFC 0:u32 ⇒ i32.trunc_sat_s_f32
        | 0xFC 1:u32 ⇒ i32.trunc_sat_u_f32
        | 0xFC 2:u32 ⇒ i32.trunc_sat_s_f64
        | 0xFC 3:u32 ⇒ i32.trunc_sat_u_f64
        | 0xFC 4:u32 ⇒ i64.trunc_sat_s_f32
        | 0xFC 5:u32 ⇒ i64.trunc_sat_u_f32
        | 0xFC 6:u32 ⇒ i64.trunc_sat_s_f64
        | 0xFC 7:u32 ⇒ i64.trunc_sat_u_f64
```

5.4.9 Vector Instructions

All variants of vector instructions are represented by separate byte codes. They all have a one byte prefix, whereas the actual opcode is encoded by a variable-length unsigned integer.

Vector loads and stores are followed by the encoding of their *memarg* immediate.

```
laneidx ::= l:byte                                  ⇒ l
  instr ::= ...
          | 0xFD 0:u32 (x, ao):memarg               ⇒ v128.load x ao
          | 0xFD 1:u32 (x, ao):memarg               ⇒ v128.load8x8_s x ao
          | 0xFD 2:u32 (x, ao):memarg               ⇒ v128.load8x8_u x ao
          | 0xFD 3:u32 (x, ao):memarg               ⇒ v128.load16x4_s x ao
          | 0xFD 4:u32 (x, ao):memarg               ⇒ v128.load16x4_u x ao
          | 0xFD 5:u32 (x, ao):memarg               ⇒ v128.load32x2_s x ao
          | 0xFD 6:u32 (x, ao):memarg               ⇒ v128.load32x2_u x ao
          | 0xFD 7:u32 (x, ao):memarg               ⇒ v128.load8_splat x ao
          | 0xFD 8:u32 (x, ao):memarg               ⇒ v128.load16_splat x ao
          | 0xFD 9:u32 (x, ao):memarg               ⇒ v128.load32_splat x ao
          | 0xFD 10:u32 (x, ao):memarg              ⇒ v128.load64_splat x ao
          | 0xFD 11:u32 (x, ao):memarg              ⇒ v128.store x ao
          | 0xFD 84:u32 (x, ao):memarg i:laneidx    ⇒ v128.load8_lane x ao i
          | 0xFD 85:u32 (x, ao):memarg i:laneidx    ⇒ v128.load16_lane x ao i
          | 0xFD 86:u32 (x, ao):memarg i:laneidx    ⇒ v128.load32_lane x ao i
          | 0xFD 87:u32 (x, ao):memarg i:laneidx    ⇒ v128.load64_lane x ao i
          | 0xFD 88:u32 (x, ao):memarg i:laneidx    ⇒ v128.store8_lane x ao i
          | 0xFD 89:u32 (x, ao):memarg i:laneidx    ⇒ v128.store16_lane x ao i
          | 0xFD 90:u32 (x, ao):memarg i:laneidx    ⇒ v128.store32_lane x ao i
          | 0xFD 91:u32 (x, ao):memarg i:laneidx    ⇒ v128.store64_lane x ao i
          | 0xFD 92:u32 (x, ao):memarg              ⇒ v128.load32_zero x ao
          | 0xFD 93:u32 (x, ao):memarg              ⇒ v128.load64_zero x ao
```

The const instruction for vectors is followed by 16 immediate bytes, which are converted into an $u128$ in littleendian byte order:

```
instr ::= ...
        | 0xFD 12:u32 (b:byte)^16  ⇒  v128.const bytes_i128^-1((b)^16)
```

The shuffle instruction is also followed by the encoding of 16 *laneidx* immediates.

$$
\begin{array}{rcll}
\text{instr} & ::= & \dots & \\
& | & \texttt{0xFD}\ 13{:}\texttt{u32}\ (l{:}\texttt{laneidx})^{16} & \Rightarrow \quad \text{i8x16.shuffle}\ l^{16} \\
& | & \texttt{0xFD}\ 14{:}\texttt{u32} & \Rightarrow \quad \text{i8x16.swizzle} \\
& | & \texttt{0xFD}\ 256{:}\texttt{u32} & \Rightarrow \quad \text{i8x16.relaxed_swizzle}
\end{array}
$$

Lane instructions are followed by the encoding of a *laneidx* immediate.

$$
\begin{array}{rcll}
\text{instr} & ::= & \dots & \\
& | & \texttt{0xFD}\ 21{:}\texttt{u32}\ l{:}\texttt{laneidx} & \Rightarrow \quad \text{i8x16.extract_lane_s}\ l \\
& | & \texttt{0xFD}\ 22{:}\texttt{u32}\ l{:}\texttt{laneidx} & \Rightarrow \quad \text{i8x16.extract_lane_u}\ l \\
& | & \texttt{0xFD}\ 23{:}\texttt{u32}\ l{:}\texttt{laneidx} & \Rightarrow \quad \text{i8x16.replace_lane}\ l \\
& | & \texttt{0xFD}\ 24{:}\texttt{u32}\ l{:}\texttt{laneidx} & \Rightarrow \quad \text{i16x8.extract_lane_s}\ l \\
& | & \texttt{0xFD}\ 25{:}\texttt{u32}\ l{:}\texttt{laneidx} & \Rightarrow \quad \text{i16x8.extract_lane_u}\ l \\
& | & \texttt{0xFD}\ 26{:}\texttt{u32}\ l{:}\texttt{laneidx} & \Rightarrow \quad \text{i16x8.replace_lane}\ l \\
& | & \texttt{0xFD}\ 27{:}\texttt{u32}\ l{:}\texttt{laneidx} & \Rightarrow \quad \text{i32x4.extract_lane}\ l \\
& | & \texttt{0xFD}\ 28{:}\texttt{u32}\ l{:}\texttt{laneidx} & \Rightarrow \quad \text{i32x4.replace_lane}\ l \\
& | & \texttt{0xFD}\ 29{:}\texttt{u32}\ l{:}\texttt{laneidx} & \Rightarrow \quad \text{i64x2.extract_lane}\ l \\
& | & \texttt{0xFD}\ 30{:}\texttt{u32}\ l{:}\texttt{laneidx} & \Rightarrow \quad \text{i64x2.replace_lane}\ l \\
& | & \texttt{0xFD}\ 31{:}\texttt{u32}\ l{:}\texttt{laneidx} & \Rightarrow \quad \text{f32x4.extract_lane}\ l \\
& | & \texttt{0xFD}\ 32{:}\texttt{u32}\ l{:}\texttt{laneidx} & \Rightarrow \quad \text{f32x4.replace_lane}\ l \\
& | & \texttt{0xFD}\ 33{:}\texttt{u32}\ l{:}\texttt{laneidx} & \Rightarrow \quad \text{f64x2.extract_lane}\ l \\
& | & \texttt{0xFD}\ 34{:}\texttt{u32}\ l{:}\texttt{laneidx} & \Rightarrow \quad \text{f64x2.replace_lane}\ l
\end{array}
$$

All other vector instructions are plain opcodes without any immediates.

$$
\begin{array}{rcll}
\text{instr} & ::= & \dots & \\
& | & \texttt{0xFD}\ 15{:}\texttt{u32} & \Rightarrow \quad \text{i8x16.splat} \\
& | & \texttt{0xFD}\ 16{:}\texttt{u32} & \Rightarrow \quad \text{i16x8.splat} \\
& | & \texttt{0xFD}\ 17{:}\texttt{u32} & \Rightarrow \quad \text{i32x4.splat} \\
& | & \texttt{0xFD}\ 18{:}\texttt{u32} & \Rightarrow \quad \text{i64x2.splat} \\
& | & \texttt{0xFD}\ 19{:}\texttt{u32} & \Rightarrow \quad \text{f32x4.splat} \\
& | & \texttt{0xFD}\ 20{:}\texttt{u32} & \Rightarrow \quad \text{f64x2.splat}
\end{array}
$$

```
instr ::= ...
        | 0xFD 35:u32  ⇒ i8x16.eq
        | 0xFD 36:u32  ⇒ i8x16.ne
        | 0xFD 37:u32  ⇒ i8x16.lt_s
        | 0xFD 38:u32  ⇒ i8x16.lt_u
        | 0xFD 39:u32  ⇒ i8x16.gt_s
        | 0xFD 40:u32  ⇒ i8x16.gt_u
        | 0xFD 41:u32  ⇒ i8x16.le_s
        | 0xFD 42:u32  ⇒ i8x16.le_u
        | 0xFD 43:u32  ⇒ i8x16.ge_s
        | 0xFD 44:u32  ⇒ i8x16.ge_u
        | 0xFD 45:u32  ⇒ i16x8.eq
        | 0xFD 46:u32  ⇒ i16x8.ne
        | 0xFD 47:u32  ⇒ i16x8.lt_s
        | 0xFD 48:u32  ⇒ i16x8.lt_u
        | 0xFD 49:u32  ⇒ i16x8.gt_s
        | 0xFD 50:u32  ⇒ i16x8.gt_u
        | 0xFD 51:u32  ⇒ i16x8.le_s
        | 0xFD 52:u32  ⇒ i16x8.le_u
        | 0xFD 53:u32  ⇒ i16x8.ge_s
        | 0xFD 54:u32  ⇒ i16x8.ge_u
        | 0xFD 55:u32  ⇒ i32x4.eq
        | 0xFD 56:u32  ⇒ i32x4.ne
        | 0xFD 57:u32  ⇒ i32x4.lt_s
        | 0xFD 58:u32  ⇒ i32x4.lt_u
        | 0xFD 59:u32  ⇒ i32x4.gt_s
        | 0xFD 60:u32  ⇒ i32x4.gt_u
        | 0xFD 61:u32  ⇒ i32x4.le_s
        | 0xFD 62:u32  ⇒ i32x4.le_u
        | 0xFD 63:u32  ⇒ i32x4.ge_s
        | 0xFD 64:u32  ⇒ i32x4.ge_u
        | 0xFD 214:u32 ⇒ i64x2.eq
        | 0xFD 215:u32 ⇒ i64x2.ne
        | 0xFD 216:u32 ⇒ i64x2.lt_s
        | 0xFD 217:u32 ⇒ i64x2.gt_s
        | 0xFD 218:u32 ⇒ i64x2.le_s
        | 0xFD 219:u32 ⇒ i64x2.ge_s
```

```
instr ::= ...
        | 0xFD 65:u32  ⇒ f32x4.eq
        | 0xFD 66:u32  ⇒ f32x4.ne
        | 0xFD 67:u32  ⇒ f32x4.lt
        | 0xFD 68:u32  ⇒ f32x4.gt
        | 0xFD 69:u32  ⇒ f32x4.le
        | 0xFD 70:u32  ⇒ f32x4.ge
        | 0xFD 71:u32  ⇒ f64x2.eq
        | 0xFD 72:u32  ⇒ f64x2.ne
        | 0xFD 73:u32  ⇒ f64x2.lt
        | 0xFD 74:u32  ⇒ f64x2.gt
        | 0xFD 75:u32  ⇒ f64x2.le
        | 0xFD 76:u32  ⇒ f64x2.ge
```

```
instr ::= ...
        | 0xFD 77:u32  ⇒ v128.not
        | 0xFD 78:u32  ⇒ v128.and
        | 0xFD 79:u32  ⇒ v128.andnot
        | 0xFD 80:u32  ⇒ v128.or
        | 0xFD 81:u32  ⇒ v128.xor
        | 0xFD 82:u32  ⇒ v128.bitselect
        | 0xFD 83:u32  ⇒ v128.any_true
```

```
instr ::= ...
        | 0xFD 96:u32   ⇒ i8x16.abs
        | 0xFD 97:u32   ⇒ i8x16.neg
        | 0xFD 98:u32   ⇒ i8x16.popcnt
        | 0xFD 99:u32   ⇒ i8x16.all_true
        | 0xFD 100:u32  ⇒ i8x16.bitmask
        | 0xFD 101:u32  ⇒ i8x16.narrow_i16x8_s
        | 0xFD 102:u32  ⇒ i8x16.narrow_i16x8_u
        | 0xFD 107:u32  ⇒ i8x16.shl
        | 0xFD 108:u32  ⇒ i8x16.shr_s
        | 0xFD 109:u32  ⇒ i8x16.shr_u
        | 0xFD 110:u32  ⇒ i8x16.add
        | 0xFD 111:u32  ⇒ i8x16.add_sat_s
        | 0xFD 112:u32  ⇒ i8x16.add_sat_u
        | 0xFD 113:u32  ⇒ i8x16.sub
        | 0xFD 114:u32  ⇒ i8x16.sub_sat_s
        | 0xFD 115:u32  ⇒ i8x16.sub_sat_u
        | 0xFD 118:u32  ⇒ i8x16.min_s
        | 0xFD 119:u32  ⇒ i8x16.min_u
        | 0xFD 120:u32  ⇒ i8x16.max_s
        | 0xFD 121:u32  ⇒ i8x16.max_u
        | 0xFD 123:u32  ⇒ i8x16.avgr_u
```

```
instr ::= ...
        | 0xFD 124:u32 ⇒ i16x8.extadd_pairwise_s_i8x16
        | 0xFD 125:u32 ⇒ i16x8.extadd_pairwise_u_i8x16
        | 0xFD 128:u32 ⇒ i16x8.abs
        | 0xFD 129:u32 ⇒ i16x8.neg
        | 0xFD 131:u32 ⇒ i16x8.all_true
        | 0xFD 132:u32 ⇒ i16x8.bitmask
        | 0xFD 133:u32 ⇒ i16x8.narrow_i32x4_s
        | 0xFD 134:u32 ⇒ i16x8.narrow_i32x4_u
        | 0xFD 135:u32 ⇒ i16x8.extend_low_s_i8x16
        | 0xFD 136:u32 ⇒ i16x8.extend_high_s_i8x16
        | 0xFD 137:u32 ⇒ i16x8.extend_low_u_i8x16
        | 0xFD 138:u32 ⇒ i16x8.extend_high_u_i8x16
        | 0xFD 139:u32 ⇒ i16x8.shl
        | 0xFD 140:u32 ⇒ i16x8.shr_s
        | 0xFD 141:u32 ⇒ i16x8.shr_u
        | 0xFD 130:u32 ⇒ i16x8.q15mulr_sat_s
        | 0xFD 142:u32 ⇒ i16x8.add
        | 0xFD 143:u32 ⇒ i16x8.add_sat_s
        | 0xFD 144:u32 ⇒ i16x8.add_sat_u
        | 0xFD 145:u32 ⇒ i16x8.sub
        | 0xFD 146:u32 ⇒ i16x8.sub_sat_s
        | 0xFD 147:u32 ⇒ i16x8.sub_sat_u
        | 0xFD 149:u32 ⇒ i16x8.mul
        | 0xFD 150:u32 ⇒ i16x8.min_s
        | 0xFD 151:u32 ⇒ i16x8.min_u
        | 0xFD 152:u32 ⇒ i16x8.max_s
        | 0xFD 153:u32 ⇒ i16x8.max_u
        | 0xFD 155:u32 ⇒ i16x8.avgr_u
        | 0xFD 273:u32 ⇒ i16x8.relaxed_q15mulr_s
        | 0xFD 156:u32 ⇒ i16x8.extmul_low_s_i8x16
        | 0xFD 157:u32 ⇒ i16x8.extmul_high_s_i8x16
        | 0xFD 158:u32 ⇒ i16x8.extmul_low_u_i8x16
        | 0xFD 159:u32 ⇒ i16x8.extmul_high_u_i8x16
        | 0xFD 274:u32 ⇒ i16x8.relaxed_dot_s_i8x16
```

```
instr ::= ...
        | 0xFD 126:u32 ⇒ i32x4.extadd_pairwise_s_i16x8
        | 0xFD 127:u32 ⇒ i32x4.extadd_pairwise_u_i16x8
        | 0xFD 160:u32 ⇒ i32x4.abs
        | 0xFD 161:u32 ⇒ i32x4.neg
        | 0xFD 163:u32 ⇒ i32x4.all_true
        | 0xFD 164:u32 ⇒ i32x4.bitmask
        | 0xFD 167:u32 ⇒ i32x4.extend_low_s_i16x8
        | 0xFD 168:u32 ⇒ i32x4.extend_high_s_i16x8
        | 0xFD 169:u32 ⇒ i32x4.extend_low_u_i16x8
        | 0xFD 170:u32 ⇒ i32x4.extend_high_u_i16x8
        | 0xFD 171:u32 ⇒ i32x4.shl
        | 0xFD 172:u32 ⇒ i32x4.shr_s
        | 0xFD 173:u32 ⇒ i32x4.shr_u
        | 0xFD 174:u32 ⇒ i32x4.add
        | 0xFD 177:u32 ⇒ i32x4.sub
        | 0xFD 181:u32 ⇒ i32x4.mul
        | 0xFD 182:u32 ⇒ i32x4.min_s
        | 0xFD 183:u32 ⇒ i32x4.min_u
        | 0xFD 184:u32 ⇒ i32x4.max_s
        | 0xFD 185:u32 ⇒ i32x4.max_u
        | 0xFD 186:u32 ⇒ i32x4.dot_s_i16x8
        | 0xFD 188:u32 ⇒ i32x4.extmul_low_s_i16x8
        | 0xFD 189:u32 ⇒ i32x4.extmul_high_s_i16x8
        | 0xFD 190:u32 ⇒ i32x4.extmul_low_u_i16x8
        | 0xFD 191:u32 ⇒ i32x4.extmul_high_u_i16x8
        | 0xFD 275:u32 ⇒ i32x4.relaxed_dot_add_s_i8x16

instr ::= ...
        | 0xFD 192:u32 ⇒ i64x2.abs
        | 0xFD 193:u32 ⇒ i64x2.neg
        | 0xFD 195:u32 ⇒ i64x2.all_true
        | 0xFD 196:u32 ⇒ i64x2.bitmask
        | 0xFD 199:u32 ⇒ i64x2.extend_low_s_i32x4
        | 0xFD 200:u32 ⇒ i64x2.extend_high_s_i32x4
        | 0xFD 201:u32 ⇒ i64x2.extend_low_u_i32x4
        | 0xFD 202:u32 ⇒ i64x2.extend_high_u_i32x4
        | 0xFD 203:u32 ⇒ i64x2.shl
        | 0xFD 204:u32 ⇒ i64x2.shr_s
        | 0xFD 205:u32 ⇒ i64x2.shr_u
        | 0xFD 206:u32 ⇒ i64x2.add
        | 0xFD 209:u32 ⇒ i64x2.sub
        | 0xFD 213:u32 ⇒ i64x2.mul
        | 0xFD 220:u32 ⇒ i64x2.extmul_low_s_i32x4
        | 0xFD 221:u32 ⇒ i64x2.extmul_high_s_i32x4
        | 0xFD 222:u32 ⇒ i64x2.extmul_low_u_i32x4
        | 0xFD 223:u32 ⇒ i64x2.extmul_high_u_i32x4
```

```
instr ::= ...
        | 0xFD 103:u32 ⇒ f32x4.ceil
        | 0xFD 104:u32 ⇒ f32x4.floor
        | 0xFD 105:u32 ⇒ f32x4.trunc
        | 0xFD 106:u32 ⇒ f32x4.nearest
        | 0xFD 224:u32 ⇒ f32x4.abs
        | 0xFD 225:u32 ⇒ f32x4.neg
        | 0xFD 227:u32 ⇒ f32x4.sqrt
        | 0xFD 228:u32 ⇒ f32x4.add
        | 0xFD 229:u32 ⇒ f32x4.sub
        | 0xFD 230:u32 ⇒ f32x4.mul
        | 0xFD 231:u32 ⇒ f32x4.div
        | 0xFD 232:u32 ⇒ f32x4.min
        | 0xFD 233:u32 ⇒ f32x4.max
        | 0xFD 234:u32 ⇒ f32x4.pmin
        | 0xFD 235:u32 ⇒ f32x4.pmax
        | 0xFD 269:u32 ⇒ f32x4.relaxed_min
        | 0xFD 270:u32 ⇒ f32x4.relaxed_max
        | 0xFD 261:u32 ⇒ f32x4.relaxed_madd
        | 0xFD 262:u32 ⇒ f32x4.relaxed_nmadd

instr ::= ...
        | 0xFD 116:u32 ⇒ f64x2.ceil
        | 0xFD 117:u32 ⇒ f64x2.floor
        | 0xFD 122:u32 ⇒ f64x2.trunc
        | 0xFD 148:u32 ⇒ f64x2.nearest
        | 0xFD 236:u32 ⇒ f64x2.abs
        | 0xFD 237:u32 ⇒ f64x2.neg
        | 0xFD 239:u32 ⇒ f64x2.sqrt
        | 0xFD 240:u32 ⇒ f64x2.add
        | 0xFD 241:u32 ⇒ f64x2.sub
        | 0xFD 242:u32 ⇒ f64x2.mul
        | 0xFD 243:u32 ⇒ f64x2.div
        | 0xFD 244:u32 ⇒ f64x2.min
        | 0xFD 245:u32 ⇒ f64x2.max
        | 0xFD 246:u32 ⇒ f64x2.pmin
        | 0xFD 247:u32 ⇒ f64x2.pmax
        | 0xFD 271:u32 ⇒ f64x2.relaxed_min
        | 0xFD 272:u32 ⇒ f64x2.relaxed_max
        | 0xFD 263:u32 ⇒ f64x2.relaxed_madd
        | 0xFD 264:u32 ⇒ f64x2.relaxed_nmadd
        | 0xFD 265:u32 ⇒ i8x16.relaxed_laneselect
        | 0xFD 266:u32 ⇒ i16x8.relaxed_laneselect
        | 0xFD 267:u32 ⇒ i32x4.relaxed_laneselect
        | 0xFD 268:u32 ⇒ i64x2.relaxed_laneselect
```

```
instr ::= ...
        | 0xFD 94:u32  ⇒ f32x4.demote_zero_f64x2
        | 0xFD 95:u32  ⇒ f64x2.promote_low_f32x4
        | 0xFD 248:u32 ⇒ i32x4.trunc_sat_s_f32x4
        | 0xFD 249:u32 ⇒ i32x4.trunc_sat_u_f32x4
        | 0xFD 250:u32 ⇒ f32x4.convert_s_i32x4
        | 0xFD 251:u32 ⇒ f32x4.convert_u_i32x4
        | 0xFD 252:u32 ⇒ i32x4.trunc_sat_s_zero_f64x2
        | 0xFD 253:u32 ⇒ i32x4.trunc_sat_u_zero_f64x2
        | 0xFD 254:u32 ⇒ f64x2.convert_low_s_i32x4
        | 0xFD 255:u32 ⇒ f64x2.convert_low_u_i32x4
        | 0xFD 257:u32 ⇒ i32x4.relaxed_trunc_s_f32x4
        | 0xFD 258:u32 ⇒ i32x4.relaxed_trunc_u_f32x4
        | 0xFD 259:u32 ⇒ i32x4.relaxed_trunc_s_zero_f64x2
        | 0xFD 260:u32 ⇒ i32x4.relaxed_trunc_u_zero_f64x2
```

5.4.10 Expressions

Expressions are encoded by their instruction sequence terminated with an explicit `0x0B` opcode for end.

$$\texttt{expr} ::= (in{:}\texttt{instr})^* \ \texttt{0x0B} \Rightarrow in^*$$

5.5 Modules

The binary encoding of modules is organized into *sections*. Most sections correspond to one component of a module record, except that function definitions are split into two sections, separating their type declarations in the function section from their bodies in the code section.

Note

This separation enables *parallel* and *streaming* compilation of the functions in a module.

5.5.1 Indices

All basic indices are encoded with their respective value.

$$\begin{array}{rcll}
\texttt{typeidx} & ::= & x{:}\texttt{u32} & \Rightarrow x \\
\texttt{funcidx} & ::= & x{:}\texttt{u32} & \Rightarrow x \\
\texttt{tableidx} & ::= & x{:}\texttt{u32} & \Rightarrow x \\
\texttt{memidx} & ::= & x{:}\texttt{u32} & \Rightarrow x \\
\texttt{globalidx} & ::= & x{:}\texttt{u32} & \Rightarrow x \\
\texttt{tagidx} & ::= & x{:}\texttt{u32} & \Rightarrow x \\
\texttt{elemidx} & ::= & x{:}\texttt{u32} & \Rightarrow x \\
\texttt{dataidx} & ::= & x{:}\texttt{u32} & \Rightarrow x \\
\texttt{localidx} & ::= & x{:}\texttt{u32} & \Rightarrow x \\
\texttt{fieldidx} & ::= & x{:}\texttt{u32} & \Rightarrow x \\
\texttt{labelidx} & ::= & l{:}\texttt{u32} & \Rightarrow l
\end{array}$$

External indices are encoded by a distiguishing byte followed by an encoding of their respective value.

$$\begin{array}{rcll}
\texttt{externidx} & ::= & \texttt{0x00}\ x{:}\texttt{funcidx} & \Rightarrow \text{func } x \\
& | & \texttt{0x01}\ x{:}\texttt{tableidx} & \Rightarrow \text{table } x \\
& | & \texttt{0x02}\ x{:}\texttt{memidx} & \Rightarrow \text{memory } x \\
& | & \texttt{0x03}\ x{:}\texttt{globalidx} & \Rightarrow \text{global } x \\
& | & \texttt{0x04}\ x{:}\texttt{tagidx} & \Rightarrow \text{tag } x
\end{array}$$

5.5.2 Sections

Each section consists of

- a one-byte section *id*,
- the $u32$ *length* of the contents, in bytes,
- the actual *contents*, whose structure is dependent on the section id.

Every section is optional; an omitted section is equivalent to the section being present with empty contents.

The following parameterized grammar rule defines the generic structure of a section with id N and contents described by the grammar $\texttt{X}$.

$$\begin{array}{rcllll} \texttt{section}_N(\texttt{X}) & ::= & N{:}\texttt{byte}\ \ len{:}\texttt{u32}\ \ en^*{:}\texttt{X} & \Rightarrow & en^* & \text{if } len = ||\texttt{X}|| \\ & | & \epsilon & \Rightarrow & \epsilon & \end{array}$$

For most sections, the contents $\texttt{X}$ encodes a list. In these cases, the empty result ϵ is interpreted as the empty list.

Note

Other than for unknown custom sections, the *size* is not required for decoding, but can be used to skip sections when navigating through a binary. The module is malformed if the size does not match the length of the binary contents $\texttt{X}$.

The following section ids are used:

Id	Section
0	custom section
1	type section
2	import section
3	function section
4	table section
5	memory section
6	global section
7	export section
8	start section
9	element section
10	code section
11	data section
12	data count section
13	tag section

Note

Section ids do not always correspond to the order of sections in the encoding of a module.

5.5.3 Custom Section

Custom sections have the id 0. They are intended to be used for debugging information or third-party extensions, and are ignored by the WebAssembly semantics. Their contents consist of a name further identifying the custom section, followed by an uninterpreted sequence of bytes for custom use.

$$\begin{array}{rcl} \texttt{customsec} & ::= & \texttt{section}_0(\texttt{custom}) \\ \texttt{custom} & ::= & \texttt{name byte}^* \end{array}$$

Note

If an implementation interprets the data of a custom section, then errors in that data, or the placement of the section, must not invalidate the module.

5.5.4 Type Section

The *type section* has the id 1. It decodes into the list of recursive types of a module.

$$\begin{array}{rcll} \texttt{typesec} & ::= & ty^*{:}\texttt{section}_1(\texttt{list}(\texttt{type})) & \Rightarrow \quad ty^* \\ \texttt{type} & ::= & qt{:}\texttt{rectype} & \Rightarrow \quad \texttt{type}\ qt \end{array}$$

5.5.5 Import Section

The *import section* has the id 2. It decodes into the list of imports of a module.

$$\begin{array}{rcll} \texttt{importsec} & ::= & im^*{:}\texttt{section}_2(\texttt{list}(\texttt{import})) & \Rightarrow \quad im^* \\ \texttt{import} & ::= & nm_1{:}\texttt{name}\ nm_2{:}\texttt{name}\ xt{:}\texttt{externtype} & \Rightarrow \quad \texttt{import}\ nm_1\ nm_2\ xt \end{array}$$

5.5.6 Function Section

The *function section* has the id 3. It decodes into a list of type indices that classify the functions defined by a module. The bodies of the respective functions are encoded separately in the code section.

$$\texttt{funcsec} \quad ::= \quad x^*{:}\texttt{section}_3(\texttt{list}(\texttt{typeidx})) \quad \Rightarrow \quad x^*$$

5.5.7 Table Section

The *table section* has the id 4. It decodes into the list of tables defined by a module.

$$\begin{array}{rcll} \texttt{tablesec} & ::= & tab^*{:}\texttt{section}_4(\texttt{list}(\texttt{table})) & \Rightarrow \quad tab^* \\ \texttt{table} & ::= & tt{:}\texttt{tabletype} & \Rightarrow \quad \texttt{table}\ tt\ (\texttt{ref.null}\ ht) \quad \text{if } tt = at\ lim\ (\texttt{ref null}^?\ ht) \\ & | & \texttt{0x40 0x00}\ tt{:}\texttt{tabletype}\ e{:}\texttt{expr} & \Rightarrow \quad \texttt{table}\ tt\ e \end{array}$$

Note

The encoding of a table type cannot start with byte 0x40, hence decoding is unambiguous. The zero byte following it is reserved for future extensions.

5.5.8 Memory Section

The *memory section* has the id 5. It decodes into the list of memories defined by a module.

$$\begin{array}{rcll} \texttt{memsec} & ::= & mem^*{:}\texttt{section}_5(\texttt{list}(\texttt{mem})) & \Rightarrow \quad mem^* \\ \texttt{mem} & ::= & mt{:}\texttt{memtype} & \Rightarrow \quad \texttt{memory}\ mt \end{array}$$

5.5.9 Global Section

The *global section* has the id 6. It decodes into the list of globals defined by a module.

$$\begin{array}{rcll} \texttt{globalsec} & ::= & glob^*{:}\texttt{section}_6(\texttt{list}(\texttt{global})) & \Rightarrow \quad glob^* \\ \texttt{global} & ::= & gt{:}\texttt{globaltype}\ e{:}\texttt{expr} & \Rightarrow \quad \texttt{global}\ gt\ e \end{array}$$

5.5.10 Export Section

The *export section* has the id 7. It decodes into the list of exports of a module.

$$\begin{array}{rcll} \texttt{exportsec} & ::= & ex^*{:}\texttt{section}_7(\texttt{list}(\texttt{export})) & \Rightarrow \quad ex^* \\ \texttt{export} & ::= & nm{:}\texttt{name}\ \ xx{:}\texttt{externidx} & \Rightarrow \quad \text{export}\ nm\ xx \end{array}$$

5.5.11 Start Section

The *start section* has the id 8. It decodes into the optional start function of a module.

$$\begin{array}{rcll} \texttt{startsec} & ::= & start^?{:}\texttt{section}_8(\texttt{start}) & \Rightarrow \quad start^? \\ \texttt{start} & ::= & x{:}\texttt{funcidx} & \Rightarrow \quad (\text{start}\ x) \end{array}$$

5.5.12 Element Section

The *element section* has the id 9. It decodes into the list of element segments defined by a module.

$$\begin{array}{rcll} \texttt{elemsec} & ::= & elem^*{:}\texttt{section}_9(\texttt{list}(\texttt{elem})) & \Rightarrow \quad elem^* \\ \texttt{elemkind} & ::= & \texttt{0x00} & \Rightarrow \quad \text{ref func} \\ \texttt{elem} & ::= & 0{:}\texttt{u32}\ e_o{:}\texttt{expr}\ y^*{:}\texttt{list}(\texttt{funcidx}) & \Rightarrow \\ & & \quad \text{elem (ref func)}\ (\text{ref.func}\ y)^*\ (\text{active}\ 0\ e_o) & \\ & | & 1{:}\texttt{u32}\ rt{:}\texttt{elemkind}\ y^*{:}\texttt{list}(\texttt{funcidx}) & \Rightarrow \\ & & \quad \text{elem}\ rt\ (\text{ref.func}\ y)^*\ \text{passive} & \\ & | & 2{:}\texttt{u32}\ x{:}\texttt{tableidx}\ e{:}\texttt{expr}\ rt{:}\texttt{elemkind}\ y^*{:}\texttt{list}(\texttt{funcidx}) & \Rightarrow \\ & & \quad \text{elem}\ rt\ (\text{ref.func}\ y)^*\ (\text{active}\ x\ e) & \\ & | & 3{:}\texttt{u32}\ rt{:}\texttt{elemkind}\ y^*{:}\texttt{list}(\texttt{funcidx}) & \Rightarrow \\ & & \quad \text{elem}\ rt\ (\text{ref.func}\ y)^*\ \text{declare} & \\ & | & 4{:}\texttt{u32}\ e_\texttt{o}{:}\texttt{expr}\ e^*{:}\texttt{list}(\texttt{expr}) & \Rightarrow \\ & & \quad \text{elem (ref null func)}\ e^*\ (\text{active}\ 0\ e_\texttt{o}) & \\ & | & 5{:}\texttt{u32}\ rt{:}\texttt{reftype}\ e^*{:}\texttt{list}(\texttt{expr}) & \Rightarrow \\ & & \quad \text{elem}\ rt\ e^*\ \text{passive} & \\ & | & 6{:}\texttt{u32}\ x{:}\texttt{tableidx}\ e_\texttt{o}{:}\texttt{expr}\ rt{:}\texttt{reftype}\ e^*{:}\texttt{list}(\texttt{expr}) & \Rightarrow \\ & & \quad \text{elem}\ rt\ e^*\ (\text{active}\ x\ e_\texttt{o}) & \\ & | & 7{:}\texttt{u32}\ rt{:}\texttt{reftype}\ e^*{:}\texttt{list}(\texttt{expr}) & \Rightarrow \\ & & \quad \text{elem}\ rt\ e^*\ \text{declare} & \end{array}$$

Note

The initial integer can be interpreted as a bitfield. Bit 0 distinguishes a passive or declarative segment from an active segment, bit 1 indicates the presence of an explicit table index for an active segment and otherwise distinguishes passive from declarative segments, bit 2 indicates the use of element type and element expressions instead of element kind and element indices.

Additional element kinds may be added in future versions of WebAssembly.

5.5.13 Code Section

The *code section* has the id 10. It decodes into the list of *code* entries that are pairs of lists of locals and expressions. They represent the body of the functions defined by a module. The types of the respective functions are encoded separately in the function section.

The encoding of each code entry consists of

- the *u32 length* of the function code in bytes,
- the actual *function code*, which in turn consists of

- the declaration of *locals*,
- the function *body* as an expression.

Local declarations are compressed into a list whose entries consist of

- a *u32* *count*,
- a value type,

denoting *count* locals of the same value type.

$$
\begin{array}{rcll}
\texttt{codesec} & ::= & code^*{:}\texttt{section}_{10}(\texttt{list}(\texttt{code})) & \Rightarrow & code^* & \\
\texttt{code} & ::= & len{:}\texttt{u32}\ code{:}\texttt{func} & \Rightarrow & code & \text{if } len = ||\texttt{func}|| \\
\texttt{func} & ::= & loc^{**}{:}\texttt{list}(\texttt{locals})\ e{:}\texttt{expr} & \Rightarrow & (\bigoplus loc^{**}, e) & \text{if } |\bigoplus loc^{**}| < 2^{32} \\
\texttt{locals} & ::= & n{:}\texttt{u32}\ t{:}\texttt{valtype} & \Rightarrow & (\texttt{local}\ t)^n &
\end{array}
$$

Here, *code* ranges over pairs $(local^*, expr)$. Any code for which the length of the resulting sequence is out of bounds of the maximum size of a list is malformed.

Note

Like with sections, the code *size* is not needed for decoding, but can be used to skip functions when navigating through a binary. The module is malformed if a size does not match the length of the respective function code.

5.5.14 Data Section

The *data section* has the id 11. It decodes into the list of data segments defined by a module.

$$
\begin{array}{rcll}
\texttt{datasec} & ::= & data^*{:}\texttt{section}_{11}(\texttt{list}(\texttt{data})) & \Rightarrow & data^* \\
\texttt{data} & ::= & 0{:}\texttt{u32}\ e{:}\texttt{expr}\ b^*{:}\texttt{list}(\texttt{byte}) & \Rightarrow & \texttt{data}\ b^*\ (\texttt{active}\ 0\ e) \\
& | & 1{:}\texttt{u32}\ b^*{:}\texttt{list}(\texttt{byte}) & \Rightarrow & \texttt{data}\ b^*\ \texttt{passive} \\
& | & 2{:}\texttt{u32}\ x{:}\texttt{memidx}\ e{:}\texttt{expr}\ b^*{:}\texttt{list}(\texttt{byte}) & \Rightarrow & \texttt{data}\ b^*\ (\texttt{active}\ x\ e)
\end{array}
$$

Note

The initial integer can be interpreted as a bitfield. Bit 0 indicates a passive segment, bit 1 indicates the presence of an explicit memory index for an active segment.

5.5.15 Data Count Section

The *data count section* has the id 12. It decodes into an optional *u32* count that represents the number of data segments in the data section. If this count does not match the length of the data segment list, the module is malformed.

$$
\begin{array}{rcll}
\texttt{datacntsec} & ::= & n^?{:}\texttt{section}_{12}(\texttt{datacnt}) & \Rightarrow & n^? \\
\texttt{datacnt} & ::= & n{:}\texttt{u32} & \Rightarrow & n
\end{array}
$$

Note

The data count section is used to simplify single-pass validation. Since the data section occurs after the code section, the memory.init and data.drop instructions would not be able to check whether the data segment index is valid until the data section is read. The data count section occurs before the code section, so a single-pass validator can use this count instead of deferring validation.

5.5.16 Tag Section

The *tag section* has the id 13. It decodes into the list of tags defined by a module.

$$\begin{array}{rcll} \texttt{tagsec} & ::= & tag^*{:}\texttt{section}_{13}(\texttt{list}(\texttt{tag})) & \Rightarrow \quad tag^* \\ \texttt{tag} & ::= & jt{:}\texttt{tagtype} & \Rightarrow \quad \texttt{tag}\ jt \end{array}$$

5.5.17 Modules

The encoding of a module starts with a preamble containing a 4-byte magic number (the string '`\0asm`') and a version field. The current version of the WebAssembly binary format is 1.

The preamble is followed by a sequence of sections. Custom sections may be inserted at any place in this sequence, while other sections must occur at most once and in the prescribed order. All sections can be empty.

The lengths of lists produced by the (possibly empty) function and code section must match up.

Similarly, the optional data count must match the length of the data segment list. Furthermore, it must be present if any data index occurs in the code section.

$$\begin{array}{rcl} \texttt{magic} & ::= & \texttt{0x00 0x61 0x73 0x6D} \\ \texttt{version} & ::= & \texttt{0x01 0x00 0x00 0x00} \\ \texttt{module} & ::= & \texttt{magic version} \quad \Rightarrow \\ & & \texttt{customsec}^*\ type^*{:}\texttt{typesec} \\ & & \texttt{customsec}^*\ import^*{:}\texttt{importsec} \\ & & \texttt{customsec}^*\ typeidx^*{:}\texttt{funcsec} \\ & & \texttt{customsec}^*\ table^*{:}\texttt{tablesec} \\ & & \texttt{customsec}^*\ mem^*{:}\texttt{memsec} \\ & & \texttt{customsec}^*\ tag^*{:}\texttt{tagsec} \\ & & \texttt{customsec}^*\ global^*{:}\texttt{globalsec} \\ & & \texttt{customsec}^*\ export^*{:}\texttt{exportsec} \\ & & \texttt{customsec}^*\ start^?{:}\texttt{startsec} \\ & & \texttt{customsec}^*\ elem^*{:}\texttt{elemsec} \\ & & \texttt{customsec}^*\ n^?{:}\texttt{datacntsec} \\ & & \texttt{customsec}^*\ (local^*, expr)^*{:}\texttt{codesec} \\ & & \texttt{customsec}^*\ data^*{:}\texttt{datasec} \\ & & \texttt{customsec}^* \\ & & \quad \texttt{module}\ type^*\ import^*\ tag^*\ global^*\ mem^*\ table^*\ func^*\ data^*\ elem^*\ start^?\ export^* \\ & & \qquad \text{if } (n = |data^*|)^? \\ & & \qquad \wedge\ (n^? \neq \epsilon \vee \text{dataidx}(func^*) = \epsilon) \\ & & \qquad \wedge\ (func = \texttt{func}\ typeidx\ local^*\ expr)^* \end{array}$$

Note

The version of the WebAssembly binary format may increase in the future if backward-incompatible changes have to be made to the format. However, such changes are expected to occur very infrequently, if ever. The binary format is intended to be extensible, such that future features can be added without incrementing its version.

CHAPTER 6

Text Format

6.1 Conventions

The textual format for WebAssembly modules is a rendering of their abstract syntax into S-expressions[36].

Like the binary format, the text format is defined by an *attribute grammar*. A text string is a well-formed description of a module if and only if it is generated by the grammar. Each production of this grammar has at most one synthesized attribute: the abstract syntax that the respective character sequence expresses. Thus, the attribute grammar implicitly defines a *parsing* function. Some productions also take a context as an inherited attribute that records bound identifiers.

Except for a few exceptions, the core of the text grammar closely mirrors the grammar of the abstract syntax. However, it also defines a number of *abbreviations* that are "syntactic sugar" over the core syntax.

The recommended extension for files containing WebAssembly modules in text format is "`.wat`". Files with this extension are assumed to be encoded in UTF-8, as per Unicode[37] (Section 2.5).

6.1.1 Grammar

The following conventions are adopted in defining grammar rules of the text format. They mirror the conventions used for abstract syntax and for the binary format. In order to distinguish symbols of the textual syntax from symbols of the abstract syntax, `typewriter` font is adopted for the former.

- Terminal symbols are either literal strings of characters enclosed in quotes or expressed as Unicode[38] scalar values: '`module`', U+0A. (All characters written literally are unambiguously drawn from the 7-bit ASCII[39] subset of Unicode.)
- Nonterminal symbols are written in typewriter font: `valtype`, `instr`.
- T^n is a sequence of $n \geq 0$ iterations of T.
- T^* is a possibly empty sequence of iterations of T. (This is a shorthand for T^n used where n is not relevant.)
- T^+ is a non-empty sequence of iterations of T. (This is a shorthand for T^n where $n \geq 1$.)
- $T^?$ is an optional occurrence of T. (This is a shorthand for T^n where $n \leq 1$.)

[36] https://en.wikipedia.org/wiki/S-expression
[37] https://www.unicode.org/versions/latest/
[38] https://www.unicode.org/versions/latest/
[39] https://webstore.ansi.org/RecordDetail.aspx?sku=INCITS+4-1986%5bR2012%5d

- x:T denotes the same language as the nonterminal T, but also binds the variable x to the attribute synthesized for T. A pattern may also be used instead of a variable, e.g., 7:T.
- Productions are written $\texttt{sym} ::= T_1 \Rightarrow A_1 \mid \ldots \mid T_n \Rightarrow A_n$, where each A_i is the attribute that is synthesized for `sym` in the given case, usually from attribute variables bound in T_i.
- Large productions may be split into multiple definitions, indicated by ending the first one with explicit ellipses, $\texttt{sym} ::= B_1$, and starting continuations with ellipses, $\texttt{sym} ::= \ldots \mid B_2$.
- Some productions are augmented by side conditions in parentheses, which restrict the applicability of the production. They provide a shorthand for a combinatorial expansion of the production into many separate cases.
- If the same meta variable or non-terminal symbol appears multiple times in a production (in the syntax or in an attribute), then all those occurrences must have the same instantiation.
- A distinction is made between *lexical* and *syntactic* productions. For the latter, arbitrary white space is allowed in any place where the grammar contains spaces. The productions defining lexical syntax and the syntax of values are considered lexical, all others are syntactic.

Note

For example, the textual grammar for number types is given as follows:

$$\begin{array}{rcll} \texttt{numtype} & ::= & \texttt{'i32'} & \Rightarrow \ \texttt{i32} \\ & \mid & \texttt{'i64'} & \Rightarrow \ \texttt{i64} \\ & \mid & \texttt{'f32'} & \Rightarrow \ \texttt{f32} \\ & \mid & \texttt{'f64'} & \Rightarrow \ \texttt{f64} \end{array}$$

The textual grammar for limits is defined as follows:

$$\begin{array}{rcll} \texttt{limits} & ::= & n{:}\texttt{u64} & \Rightarrow \ [n \,..\, \epsilon] \\ & \mid & n{:}\texttt{u64}\ m{:}\texttt{u64} & \Rightarrow \ [n \,..\, m] \end{array}$$

The variables n and m name the attributes of the respective `u64` nonterminals, which in this case are the actual unsigned integers those parse into. The attribute of the complete production then is the abstract syntax for the limit, expressed in terms of the former values.

6.1.2 Abbreviations

In addition to the core grammar, which corresponds directly to the abstract syntax, the textual syntax also defines a number of *abbreviations* that can be used for convenience and readability.

Abbreviations are defined by *rewrite rules* specifying their expansion into the core syntax:

$$\texttt{sym} \ ::= \ \textit{abbreviated syntax} \ \equiv \ \textit{expanded syntax}$$

These expansions are assumed to be applied, recursively and in order of appearance, before applying the core grammar rules to construct the abstract syntax.

6.1.3 Contexts

The text format allows the use of symbolic identifiers in place of indices. To resolve these identifiers into concrete indices, some grammar productions are indexed by an *identifier context* I as a synthesized attribute that records the declared identifiers in each index space. In addition, the context records the types defined in the module, so that parameter indices can be computed for functions.

It is convenient to define identifier contexts as records I with abstract syntax as follows:

$$\begin{array}{rcl} I & ::= & \{\mathsf{types}\ (name^?)^* \\ & & \ \mathsf{tags}\ (name^?)^* \\ & & \ \mathsf{globals}\ (name^?)^* \\ & & \ \mathsf{mems}\ (name^?)^* \\ & & \ \mathsf{tables}\ (name^?)^* \\ & & \ \mathsf{funcs}\ (name^?)^* \\ & & \ \mathsf{datas}\ (name^?)^* \\ & & \ \mathsf{elems}\ (name^?)^* \\ & & \ \mathsf{locals}\ (name^?)^* \\ & & \ \mathsf{labels}\ (name^?)^* \\ & & \ \mathsf{fields}\ ((name^?)^*)^* \\ & & \ \mathsf{typedefs}\ (deftype^?)^*\} \end{array}$$

For each index space, such a context contains the list of names assigned to the defined indices, which were denoted by the corresponding identifiers. Unnamed indices are associated with empty (ϵ) entries in these lists. Fields have *dependent* name spaces, and hence a separate list of field identifiers per type.

In addition, the field typedefs records the defined type associated with each type index. They are needed to look up the number of parameters of function types when used in a function definition, in order to produce the correct indices for locals.

An identifier context is *well-formed* if no index space contains duplicate identifiers. For fields, names need only be unique within a single type.

Conventions

To avoid unnecessary clutter, empty components are omitted when writing out identifier contexts. For example, the record $\{\}$ is shorthand for an identifier context whose components are all empty.

6.1.4 Lists

Lists are written as plain sequences, but with a restriction on the length of these sequence.

$$\texttt{list}(\texttt{X}) \quad ::= \quad (el{:}\texttt{X})^* \quad \Rightarrow \quad el^* \quad \text{if } |el^*| < 2^{32}$$

6.2 Lexical Format

6.2.1 Characters

The text format assigns meaning to *source text*, which consists of a sequence of *characters*. Characters are assumed to be represented as valid Unicode[40] (Section 2.4) *scalar values*.

$$\begin{array}{rcl} \texttt{source} & ::= & \texttt{char}^* \\ \texttt{char} & ::= & \text{U+00} \mid \ldots \mid \text{U+D7FF} \mid \text{U+E000} \mid \ldots \mid \text{U+10FFFF} \end{array}$$

Note

While source text may contain any Unicode character in comments or string literals, the rest of the grammar is formed exclusively from the characters supported by the 7-bit ASCII[41] subset of Unicode.

[40] https://www.unicode.org/versions/latest/
[41] https://webstore.ansi.org/RecordDetail.aspx?sku=INCITS+4-1986%5bR2012%5d

6.2.2 Tokens

The character stream in the source text is divided, from left to right, into a sequence of *tokens*, as defined by the following grammar.

$$\begin{array}{rcl} \texttt{token} & ::= & \texttt{keyword} \mid \texttt{u} \mid \texttt{s} \mid \texttt{f} \mid \texttt{string} \mid \texttt{id} \mid \text{`('} \mid \text{`)'} \mid \texttt{reserved} \\ \texttt{keyword} & ::= & (\text{`a'} \mid \ldots \mid \text{`z'})\ \texttt{idchar}^* \\ \texttt{reserved} & ::= & (\texttt{idchar} \mid \texttt{string} \mid \text{`,'} \mid \text{`;'} \mid \text{`[']} \mid \text{`]'} \mid \text{`\{'} \mid \text{`\}'})^+ \end{array}$$

Tokens are formed from the input character stream according to the *longest match* rule. That is, the next token always consists of the longest possible sequence of characters that is recognized by the above lexical grammar. Tokens can be separated by white space, but except for strings, they cannot themselves contain whitespace.

Keyword tokens always start with a lower-case letter. The set of keywords is defined implicitly: only those tokens are defined to be keywords that occur as a terminal symbol in literal form, such as '`keyword`', in a syntactic production of this chapter.

Any token that does not fall into any of the other categories is considered *reserved*, and cannot occur in source text.

Note

The effect of defining the set of reserved tokens is that all tokens must be separated by either parentheses, white space, or comments. For example, '`0$x`' is a single reserved token, as is '`''a''''b''`'. Consequently, they are not recognized as two separate tokens '`0`' and '`$x`', or '`''a''`' and '`''b''`', respectively, but instead disallowed. This property of tokenization is not affected by the fact that the definition of reserved tokens overlaps with other token classes.

6.2.3 White Space

White space is any sequence of literal space characters, formatting characters, comments, or annotations. The allowed formatting characters correspond to a subset of the ASCII[42] *format effectors*, namely, *horizontal tabulation* (U+09), *line feed* (U+0A), and *carriage return* (U+0D).

$$\begin{array}{rcl} \texttt{space} & ::= & (\text{` '} \mid \texttt{format} \mid \texttt{comment} \mid \texttt{annot})^* \\ \texttt{format} & ::= & \texttt{newline} \mid \text{U+09} \\ \texttt{newline} & ::= & \text{U+0A} \mid \text{U+0D}\ \text{U+0A} \end{array}$$

The only relevance of white space is to separate tokens. It is otherwise ignored.

6.2.4 Comments

A *comment* can either be a *line comment*, started with a double semicolon '`;;`' and extending to the end of the line, or a *block comment*, enclosed in delimiters '`(;`' ... '`;)`'. Block comments can be nested.

$$\begin{array}{rcll} \texttt{comment} & ::= & \texttt{linecomment} \mid \texttt{blockcomment} & \\ \texttt{linecomment} & ::= & \text{`;;'}\ \texttt{linechar}^*\ (\texttt{newline} \mid \texttt{eof}) & \\ \texttt{linechar} & ::= & c{:}\texttt{char} & \text{if } c \neq \text{U+0A} \wedge c \neq \text{U+0D} \\ \texttt{blockcomment} & ::= & \text{`(;'}\ \texttt{blockchar}^*\ \text{`;)'} & \\ \texttt{blockchar} & ::= & c{:}\texttt{char} & \text{if } c \neq \text{`;'} \wedge c \neq \text{`('} \\ & \mid & \text{`;'}^+\ c{:}\texttt{char} & \text{if } c \neq \text{`;'} \wedge c \neq \text{`)'} \\ & \mid & \text{`('}^+\ c{:}\texttt{char} & \text{if } c \neq \text{`;'} \wedge c \neq \text{`('} \\ & \mid & \texttt{blockcomment} & \end{array}$$

Here, the pseudo token `eof` indicates the end of the input. The *look-ahead* restrictions on the productions for `blockchar` disambiguate the grammar such that only well-bracketed uses of block comment delimiters are allowed.

[42] https://webstore.ansi.org/RecordDetail.aspx?sku=INCITS+4-1986%5bR2012%5d

Note

Any formatting and control characters are allowed inside comments.

6.2.5 Annotations

An *annotation* is a bracketed token sequence headed by an *annotation id* of the form '`@id`' or '`@''...''`'. No space is allowed between the opening parenthesis and this id. Annotations are intended to be used for third-party extensions; they can appear anywhere in a program but are ignored by the WebAssembly semantics itself, which treats them as white space.

Annotations can contain other parenthesized token sequences (including nested annotations), as long as they are well-nested. String literals and comments occurring in an annotation must also be properly nested and closed.

$$\begin{array}{rcl} \texttt{annot} & ::= & \text{`}\texttt{(@}\text{'}\ \texttt{annotid}\ (\texttt{space} \mid \texttt{token})^*\ \text{`}\texttt{)}\text{'} \\ \texttt{annotid} & ::= & \texttt{idchar}^+ \mid \texttt{name} \end{array}$$

Note

The annotation id is meant to be an identifier categorising the extension, and plays a role similar to the name of a custom section. By convention, annotations corresponding to a custom section should use the custom section's name as an id.

Implementations are expected to ignore annotations with ids that they do not recognize. On the other hand, they may impose restrictions on annotations that they do recognize, e.g., requiring a specific structure by superimposing a more concrete grammar. It is up to an implementation how it deals with errors in such annotations.

6.3 Values

The grammar productions in this section define *lexical syntax*, hence no white space is allowed.

6.3.1 Integers

All integers can be written in either decimal or hexadecimal notation. In both cases, digits can optionally be separated by underscores.

$$\begin{array}{rcl} \texttt{sign} & ::= & \epsilon \Rightarrow +1 \mid \text{`}\texttt{+}\text{'} \Rightarrow +1 \mid \text{`}\texttt{-}\text{'} \Rightarrow -1 \\ \texttt{digit} & ::= & \text{`}\texttt{0}\text{'} \Rightarrow 0 \mid \ldots \mid \text{`}\texttt{9}\text{'} \Rightarrow 9 \\ \texttt{hexdigit} & ::= & d{:}\texttt{digit} \Rightarrow d \\ & \mid & \text{`}\texttt{A}\text{'} \Rightarrow 10 \mid \ldots \mid \text{`}\texttt{F}\text{'} \Rightarrow 15 \\ & \mid & \text{`}\texttt{a}\text{'} \Rightarrow 10 \mid \ldots \mid \text{`}\texttt{f}\text{'} \Rightarrow 15 \\ \texttt{num} & ::= & d{:}\texttt{digit} \Rightarrow d \\ & \mid & n{:}\texttt{num}\ \text{`}\texttt{_}\text{'}^?\ d{:}\texttt{digit} \Rightarrow 10\,n + d \\ \texttt{hexnum} & ::= & h{:}\texttt{hexdigit} \Rightarrow h \\ & \mid & n{:}\texttt{hexnum}\ \text{`}\texttt{_}\text{'}^?\ h{:}\texttt{hexdigit} \Rightarrow 16\,n + h \end{array}$$

The allowed syntax for integer literals depends on size and signedness. Moreover, their value must lie within the range of the respective type.

$$\begin{array}{rcl} \texttt{u}N & ::= & n{:}\texttt{num} \Rightarrow n \quad \text{if } n < 2^N \\ & \mid & \text{`}\texttt{0x}\text{'}\ n{:}\texttt{hexnum} \Rightarrow n \quad \text{if } n < 2^N \\ \texttt{s}N & ::= & s{:}\texttt{sign}\ n{:}\texttt{u}N \Rightarrow s \cdot n \quad \text{if } -2^{N-1} \leq s \cdot n < 2^{N-1} \end{array}$$

Uninterpreted integers can be written as either signed or unsigned, and are normalized to unsigned in the abstract syntax.

$$\begin{array}{rcl} \texttt{i}N & ::= & n{:}\texttt{u}N \Rightarrow n \\ & \mid & i{:}\texttt{s}N \Rightarrow \text{signed}_N^{-1}(i) \end{array}$$

6.3.2 Floating-Point

Floating-point values can be represented in either decimal or hexadecimal notation.

$$
\begin{array}{rcll}
\texttt{frac} & ::= & d{:}\texttt{digit} & \Rightarrow \; d/10 \\
& | & d{:}\texttt{digit}\ \texttt{'_'}^{?}\ p{:}\texttt{frac} & \Rightarrow \; (d + p/10)/10 \\
\texttt{hexfrac} & ::= & h{:}\texttt{hexdigit} & \Rightarrow \; h/16 \\
& | & h{:}\texttt{hexdigit}\ \texttt{'_'}^{?}\ p{:}\texttt{hexfrac} & \Rightarrow \; (h + p/16)/16 \\
\texttt{mant} & ::= & p{:}\texttt{num}\ \texttt{'.'}^{?} & \Rightarrow \; p \\
& | & p{:}\texttt{num}\ \texttt{'.'}\ q{:}\texttt{frac} & \Rightarrow \; p + q \\
\texttt{hexmant} & ::= & p{:}\texttt{hexnum}\ \texttt{'.'}^{?} & \Rightarrow \; p \\
& | & p{:}\texttt{hexnum}\ \texttt{'.'}\ q{:}\texttt{hexfrac} & \Rightarrow \; p + q \\
\texttt{float} & ::= & p{:}\texttt{mant}\ (\texttt{'E'} \mid \texttt{'e'})\ s{:}\texttt{sign}\ e{:}\texttt{num} & \Rightarrow \; p \cdot 10^{s \cdot e} \\
\texttt{hexfloat} & ::= & \texttt{'0x'}\ p{:}\texttt{hexmant}\ (\texttt{'P'} \mid \texttt{'p'})\ s{:}\texttt{sign}\ e{:}\texttt{num} & \Rightarrow \; p \cdot 2^{s \cdot e}
\end{array}
$$

The value of a literal must not lie outside the representable range of the corresponding IEEE 754[43] type (that is, a numeric value must not overflow to $\pm\infty$), but it may be rounded to the nearest representable value.

Note

Rounding can be prevented by using hexadecimal notation with no more significant bits than supported by the required type.

Floating-point values may also be written as constants for *infinity* or *canonical NaN* (*not a number*). Furthermore, arbitrary NaN values may be expressed by providing an explicit payload value.

$$
\begin{array}{rcllll}
\texttt{f}N & ::= & (+1){:}\texttt{sign}\ q{:}\texttt{f}N\texttt{mag} & \Rightarrow & +q & \\
& | & (-1){:}\texttt{sign}\ q{:}\texttt{f}N\texttt{mag} & \Rightarrow & -q & \\
\texttt{f}N\texttt{mag} & ::= & q{:}\texttt{float} & \Rightarrow & \mathrm{float}_N(q) & \text{if } \mathrm{float}_N(q) \neq \infty \\
& | & q{:}\texttt{hexfloat} & \Rightarrow & \mathrm{float}_N(q) & \text{if } \mathrm{float}_N(q) \neq \infty \\
& | & \texttt{'inf'} & \Rightarrow & \infty & \\
& | & \texttt{'nan'} & \Rightarrow & \texttt{nan}(\mathrm{canon}_N) & \\
& | & \texttt{'nan:0x'}\ n{:}\texttt{hexnum} & \Rightarrow & \texttt{nan}(n) & \text{if } 1 \leq n < 2^{\mathrm{signif}(N)}
\end{array}
$$

6.3.3 Strings

Strings denote sequences of bytes that can represent both textual and binary data. They are enclosed in quotation marks and may contain any character other than ASCII[44] control characters, quotation marks (`''`), or backslash (`\`), except when expressed with an *escape sequence*.

$$
\begin{array}{rcllll}
\texttt{string} & ::= & \texttt{''}\ (b^{*}{:}\texttt{stringelem})^{*}\ \texttt{''} & \Rightarrow & \bigoplus b^{**} & \text{if } |\bigoplus b^{**}| < 2^{32} \\
\texttt{stringelem} & ::= & c{:}\texttt{stringchar} & \Rightarrow & \mathrm{utf8}(c) & \\
& | & \texttt{'\'}\ h_1{:}\texttt{hexdigit}\ h_2{:}\texttt{hexdigit} & \Rightarrow & 16\,h_1 + h_2 &
\end{array}
$$

[43] https://ieeexplore.ieee.org/document/8766229
[44] https://webstore.ansi.org/RecordDetail.aspx?sku=INCITS+4-1986%5bR2012%5d

Each character in a string literal represents the byte sequence corresponding to its UTF-8 Unicode[45] (Section 2.5) encoding, except for hexadecimal escape sequences '\hh', which represent raw bytes of the respective value.

$$
\begin{array}{lcll}
\texttt{stringchar} & ::= & c{:}\texttt{char} & \Rightarrow \quad c \quad \text{if } c \geq \text{U+20} \wedge c \neq \text{U+7F} \wedge c \neq \text{`''} \wedge c \neq \text{`\'} \\
& | & \text{`\texttt{t}'} & \Rightarrow \quad \text{U+09} \\
& | & \text{`\texttt{n}'} & \Rightarrow \quad \text{U+0A} \\
& | & \text{`\texttt{r}'} & \Rightarrow \quad \text{U+0D} \\
& | & \text{`\''} & \Rightarrow \quad \text{U+22} \\
& | & \text{`\'} & \Rightarrow \quad \text{U+27} \\
& | & \text{`\\'} & \Rightarrow \quad \text{U+5C} \\
& | & \text{`\texttt{u\{}'}\ n{:}\texttt{hexnum}\ \text{`\texttt{\}}'} & \Rightarrow \quad n \quad \text{if } n < \texttt{0xD800} \vee \texttt{0xE000} \leq n < \texttt{0x110000}
\end{array}
$$

6.3.4 Names

Names are strings denoting a literal character sequence. A name string must form a valid UTF-8 encoding as defined by Unicode[46] (Section 2.5) and is interpreted as a string of Unicode scalar values.

$$\texttt{name} \quad ::= \quad b^*{:}\texttt{string} \quad \Rightarrow \quad c^* \quad \text{if } b^* = \text{utf8}(c^*)$$

Note

Presuming the source text is itself encoded correctly, strings that do not contain any uses of hexadecimal byte escapes are always valid names.

6.3.5 Identifiers

Indices can be given in both numeric and symbolic form. Symbolic *identifiers* that stand in lieu of indices start with '$', followed by either a sequence of printable ASCII[47] characters that does not contain a space, quotation mark, comma, semicolon, or bracket, or by a quoted name.

$$
\begin{array}{lcll}
\texttt{id} & ::= & \text{`\$'}\ c^*{:}\texttt{idchar}^+ & \Rightarrow \quad c^* \\
& | & \text{`\$'}\ c^*{:}\texttt{name} & \Rightarrow \quad c^* \quad \text{if } |c^*| > 0 \\
\texttt{idchar} & ::= & \text{`\texttt{0}'} \mid \ldots \mid \text{`\texttt{9}'} & \\
& | & \text{`\texttt{A}'} \mid \ldots \mid \text{`\texttt{Z}'} & \\
& | & \text{`\texttt{a}'} \mid \ldots \mid \text{`\texttt{z}'} & \\
& | & \text{`\texttt{!}'} \mid \text{`\texttt{\#}'} \mid \text{`\texttt{\$}'} \mid \text{`\texttt{\%}'} \mid \text{`\texttt{\&}'} \mid \text{`\texttt{'}'} \mid \text{`\texttt{*}'} \mid \text{`\texttt{+}'} \mid \text{`\texttt{-}'} \mid \text{`\texttt{.}'} \mid \text{`\texttt{/}'} & \\
& | & \text{`\texttt{:}'} \mid \text{`\texttt{<}'} \mid \text{`\texttt{=}'} \mid \text{`\texttt{>}'} \mid \text{`\texttt{?}'} \mid \text{`\texttt{@}'} \mid \text{`\texttt{\textbackslash}'} \mid \text{`\texttt{\^{}}'} \mid \text{`\texttt{\_}'} \mid \text{`\texttt{`}'} \mid \text{`\texttt{|}'} \mid \text{`\texttt{\~{}}'} &
\end{array}
$$

Note

The value of an identifier character is the Unicode codepoint denoting it.

Conventions

The expansion rules of some abbreviations require insertion of a *fresh* identifier. That may be any syntactically valid identifier that does not already occur in the given source text.

[45] https://www.unicode.org/versions/latest/
[46] https://www.unicode.org/versions/latest/
[47] https://webstore.ansi.org/RecordDetail.aspx?sku=INCITS+4-1986%5bR2012%5d

6.4 Types

6.4.1 Number Types

```
numtype ::= 'i32'  ⇒ i32
          | 'i64'  ⇒ i64
          | 'f32'  ⇒ f32
          | 'f64'  ⇒ f64
```

6.4.2 Vector Types

```
vectype ::= 'v128' ⇒ v128
```

6.4.3 Heap Types

```
absheaptype ::= 'any'      ⇒ any
              | 'eq'       ⇒ eq
              | 'i31'      ⇒ i31
              | 'struct'   ⇒ struct
              | 'array'    ⇒ array
              | 'none'     ⇒ none
              | 'func'     ⇒ func
              | 'nofunc'   ⇒ nofunc
              | 'exn'      ⇒ exn
              | 'noexn'    ⇒ noexn
              | 'extern'   ⇒ extern
              | 'noextern' ⇒ noextern

heaptype_I ::= ht:absheaptype ⇒ ht
             | x:typeidx_I    ⇒ x
```

6.4.4 Reference Types

```
null       ::= 'null'                                    ⇒ null
reftype_I  ::= '(' 'ref' null?:null? ht:heaptype_I ')'   ⇒ ref null? ht
```

Abbreviations

There are shorthands for references to abstract heap types.

```
reftype_I ::= ...
            | 'anyref'      ≡ '(' 'ref' 'null' 'any' ')'
            | 'eqref'       ≡ '(' 'ref' 'null' 'eq' ')'
            | 'i31ref'      ≡ '(' 'ref' 'null' 'i31' ')'
            | 'structref'   ≡ '(' 'ref' 'null' 'struct' ')'
            | 'arrayref'    ≡ '(' 'ref' 'null' 'array' ')'
            | 'nullref'     ≡ '(' 'ref' 'null' 'none' ')'
            | 'funcref'     ≡ '(' 'ref' 'null' 'func' ')'
            | 'nullfuncref' ≡ '(' 'ref' 'null' 'nofunc' ')'
            | 'exnref'      ≡ '(' 'ref' 'null' 'exn' ')'
            | 'nullexnref'  ≡ '(' 'ref' 'null' 'noexn' ')'
            | 'externref'   ≡ '(' 'ref' 'null' 'extern' ')'
            | 'nullexternref' ≡ '(' 'ref' 'null' 'noextern' ')'
```

6.4.5 Value Types

$$\begin{array}{rcll} \texttt{valtype}_I & ::= & nt{:}\texttt{numtype} & \Rightarrow & nt \\ & | & vt{:}\texttt{vectype} & \Rightarrow & vt \\ & | & rt{:}\texttt{reftype}_I & \Rightarrow & rt \end{array}$$

6.4.6 Composite Types

Composite types are parsed into their respective abstract representation, paired with the local identifier context generated by their bound field or parameter identifiers:

$$\begin{array}{rcll} \texttt{comptype}_I & ::= & \text{`(' `struct'} \ (ft, id^?)^*{:}\texttt{list}(\texttt{field}_I) \ \text{`)'} & \Rightarrow & (\mathsf{struct}\ ft^*, \{\mathsf{fields}\ ((id^?)^*)\}) \\ & | & \text{`(' `array'} \ ft{:}\texttt{fieldtype}_I \ \text{`)'} & \Rightarrow & (\mathsf{array}\ ft, \{\mathsf{fields}\ (\epsilon)\}) \\ & | & \text{`(' `func'} \ (t_1, id^?)^*{:}\texttt{list}(\texttt{param}_I) \ t_2^*{:}\texttt{list}(\texttt{result}_I) \ \text{`)'} & \Rightarrow & (\mathsf{func}\ t_1^* \to t_2^*, \{\mathsf{fields}\ (\epsilon)\}) \\ \texttt{field}_I & ::= & \text{`(' `field'} \ id^?{:}\texttt{id}^? \ ft{:}\texttt{fieldtype}_I \ \text{`)'} & \Rightarrow & (ft, id^?) \\ \texttt{param}_I & ::= & \text{`(' `param'} \ id^?{:}\texttt{id}^? \ t{:}\texttt{valtype}_I \ \text{`)'} & \Rightarrow & (t, id^?) \\ \texttt{result}_I & ::= & \text{`(' `result'} \ t{:}\texttt{valtype}_I \ \text{`)'} & \Rightarrow & t \\ \texttt{fieldtype}_I & ::= & zt{:}\texttt{storagetype}_I & \Rightarrow & zt \\ & | & \text{`(' `mut'} \ zt{:}\texttt{storagetype}_I \ \text{`)'} & \Rightarrow & \mathsf{mut}\ zt \\ \texttt{storagetype}_I & ::= & t{:}\texttt{valtype}_I & \Rightarrow & t \\ & | & pt{:}\texttt{packtype} & \Rightarrow & pt \\ \texttt{packtype} & ::= & \text{`i8'} & \Rightarrow & \mathsf{i8} \\ & | & \text{`i16'} & \Rightarrow & \mathsf{i16} \end{array}$$

Note

The optional identifier names for parameters in a function type only have documentation purpose. They cannot be referenced from anywhere.

Abbreviations

Multiple anonymous structure fields or parameters or multiple results may be combined into a single declaration:

$$\begin{array}{rcll} \texttt{field}_I & ::= & \ldots \mid \text{`(' `field'} \ \texttt{fieldtype}_I^* \ \text{`)'} & \equiv & (\text{`(' `field'} \ \texttt{fieldtype}_I \ \text{`)'})^* \\ \texttt{param}_I & ::= & \ldots \mid \text{`(' `param'} \ \texttt{valtype}_I^* \ \text{`)'} & \equiv & (\text{`(' `param'} \ \texttt{valtype}_I \ \text{`)'})^* \\ \texttt{result}_I & ::= & \ldots \mid \text{`(' `result'} \ \texttt{valtype}_I^* \ \text{`)'} & \equiv & (\text{`(' `result'} \ \texttt{valtype}_I \ \text{`)'})^* \end{array}$$

6.4.7 Recursive Types

Recursive types are parsed into their respective abstract representation, paired with the identifier context generated by their bound identifiers:

$$\begin{array}{rcll} \texttt{final} & ::= & \text{`final'} & \Rightarrow \ \mathsf{final} \\ \texttt{subtype}_I & ::= & \text{`(' `sub'} \ fin^?{:}\texttt{final}^? \ x^*{:}\texttt{list}(\texttt{typeidx}_I) \ (ct, I'){:}\texttt{comptype}_I \ \text{`)'} & \Rightarrow \ (\mathsf{sub}\ fin^?\ x^*\ ct, I') \\ \texttt{typedef}_I & ::= & \text{`(' `type'} \ id^?{:}\texttt{id}^? \ (st, I'){:}\texttt{subtype}_I \ \text{`)'} & \Rightarrow \ (st, I' \oplus \{\mathsf{types}\ (id^?)\}) \\ \texttt{rectype}_I & ::= & \text{`(' `rec'} \ (st, I')^*{:}\texttt{list}(\texttt{typedef}_I) \ \text{`)'} & \Rightarrow \ (\mathsf{rec}\ st^*, \bigoplus I'^*) \end{array}$$

Abbreviations

Final sub types with no super-types can omit the 'sub' keyword and its arguments:

$$\texttt{subtype}_I \ ::= \ \ldots \mid \texttt{comptype}_I \ \equiv \ \text{`(' `sub' `final'} \ \texttt{comptype}_I \ \text{`)'}$$

Similarly, singular recursive types can omit the 'rec' keyword:

$$\texttt{rectype}_I \ ::= \ \ldots \mid \texttt{typedef}_I \ \equiv \ \text{`(' `rec'} \ \texttt{typedef}_I \ \text{`)'}$$

6.4.8 Address Types

$$\begin{array}{rcll} \texttt{addrtype} & ::= & \text{`}\texttt{i32}\text{'} & \Rightarrow & \texttt{i32} \\ & | & \text{`}\texttt{i64}\text{'} & \Rightarrow & \texttt{i64} \end{array}$$

Abbreviations

The address type can be omitted, in which case it defaults i32:

$$\texttt{addrtype} ::= \ldots \mid \epsilon \equiv \text{`}\texttt{i32}\text{'}$$

6.4.9 Limits

$$\begin{array}{rcll} \texttt{limits} & ::= & n{:}\texttt{u64} & \Rightarrow & [n\,..\,\epsilon] \\ & | & n{:}\texttt{u64}\ m{:}\texttt{u64} & \Rightarrow & [n\,..\,m] \end{array}$$

6.4.10 Tag Types

$$\texttt{tagtype}_I ::= (x, I'){:}\texttt{typeuse}_I \Rightarrow x$$

6.4.11 Global Types

$$\begin{array}{rcll} \texttt{globaltype}_I & ::= & t{:}\texttt{valtype}_I & \Rightarrow & t \\ & | & \text{`(' `}\texttt{mut}\text{'}\ t{:}\texttt{valtype}_I\ \text{`)'} & \Rightarrow & \mathsf{mut}\ t \end{array}$$

6.4.12 Memory Types

$$\texttt{memtype}_I ::= at{:}\texttt{addrtype}\ lim{:}\texttt{limits} \Rightarrow at\ lim\ \mathsf{page}$$

6.4.13 Table Types

$$\texttt{tabletype}_I ::= at{:}\texttt{addrtype}\ lim{:}\texttt{limits}\ rt{:}\texttt{reftype}_I \Rightarrow at\ lim\ rt$$

6.4.14 External Types

$$\begin{array}{rcll} \texttt{externtype}_I & ::= & \text{`(' `}\texttt{tag}\text{'}\ id^?{:}\texttt{id}^?\ jt{:}\texttt{tagtype}_I\ \text{`)'} & \Rightarrow & (\mathsf{tag}\ jt, \{\mathsf{tags}\ (id^?)\}) \\ & | & \text{`(' `}\texttt{global}\text{'}\ id^?{:}\texttt{id}^?\ gt{:}\texttt{globaltype}_I\ \text{`)'} & \Rightarrow & (\mathsf{global}\ gt, \{\mathsf{globals}\ (id^?)\}) \\ & | & \text{`(' `}\texttt{memory}\text{'}\ id^?{:}\texttt{id}^?\ mt{:}\texttt{memtype}_I\ \text{`)'} & \Rightarrow & (\mathsf{mem}\ mt, \{\mathsf{mems}\ (id^?)\}) \\ & | & \text{`(' `}\texttt{table}\text{'}\ id^?{:}\texttt{id}^?\ tt{:}\texttt{tabletype}_I\ \text{`)'} & \Rightarrow & (\mathsf{table}\ tt, \{\mathsf{tables}\ (id^?)\}) \\ & | & \text{`(' `}\texttt{func}\text{'}\ id^?{:}\texttt{id}^?\ (x, I'){:}\texttt{typeuse}_I\ \text{`)'} & \Rightarrow & (\mathsf{func}\ x, \{\mathsf{funcs}\ (id^?)\}) \end{array}$$

6.4.15 Type Uses

A *type use* is a reference to a type definition. Where it is required to reference a function type, it may optionally be augmented by explicit inlined parameter and result declarations. That allows binding symbolic identifiers to name the local indices of parameters. If inline declarations are given, then their types must match the referenced function type.

$$
\begin{array}{rcll}
\texttt{typeuse}_I & ::= & \text{`('}\ \text{`type'}\ x{:}\texttt{typeidx}_I\ \text{`)'} & \Rightarrow \quad (x, I') \\
& & \quad \text{if } I.\mathsf{typedefs}[x] = (\mathsf{rec}\ st^*).i \\
& & \quad \wedge\ st^*[i] = \mathsf{sub\ final}\ (\mathsf{func}\ t_1^* \to t_2^*) \\
& & \quad \wedge\ I' = \{\mathsf{locals}\ (\epsilon)^{|t_1^*|}\} \\
& | & \text{`('}\ \text{`type'}\ x{:}\texttt{typeidx}_I\ \text{`)'}\ (t_1, id^?)^*{:}\texttt{param}_I^*\ t_2^*{:}\texttt{result}_I^* & \Rightarrow \quad (x, I') \\
& & \quad \text{if } I.\mathsf{typedefs}[x] = (\mathsf{rec}\ st^*).i \\
& & \quad \wedge\ st^*[i] = \mathsf{sub\ final}\ (\mathsf{func}\ t_1^* \to t_2^*) \\
& & \quad \wedge\ I' = \{\mathsf{locals}\ (id^?)^*\} \\
& & \quad \wedge \vdash I' : \mathsf{ok}
\end{array}
$$

Note

If inline declarations are given, their types must be *syntactically* equal to the types from the indexed definition; possible type substitutions from other definitions that might make them equal are not taken into account. This is to simplify syntactic pre-processing.

The synthesized attribute of a `typeuse` is a pair consisting of both the used type index and the local identifier context containing possible parameter identifiers.

Note

Both productions overlap for the case that the function type is $\mathsf{func}\ \epsilon \to \epsilon$. However, in that case, they also produce the same results, so that the choice is immaterial.

The well-formedness condition on I' ensures that the parameters do not contain duplicate identifiers.

Abbreviations

A type use may also be replaced entirely by inline parameter and result declarations. In that case, a type index is automatically inserted:

$$
\begin{array}{rcll}
\texttt{typeuse}_I & ::= & \ldots \mid (t_1, id^?)^*{:}\texttt{param}_I^*\ t_2^*{:}\texttt{result}_I^* & \equiv \quad \text{`('}\ \text{`type'}\ x{:}\texttt{typeidx}_I\ \text{`)'}\ \texttt{param}_I^*\ \texttt{result}_I^* \\
& & \quad \text{if } I.\mathsf{typedefs}[x] = (\mathsf{rec}\ (\mathsf{sub\ final}\ (\mathsf{func}\ t_1^* \to t_2^*))).0 \\
& & \quad \wedge\ (I.\mathsf{typedefs}[i] \neq (\mathsf{rec}\ (\mathsf{sub\ final}\ (\mathsf{func}\ t_1^* \to t_2^*))).0)^{i<x}
\end{array}
$$

where x is the smallest existing type index whose recursive type definition parses into a singular, final function type with the same parameters and results. If no such index exists, then a new recursive type of the same form is inserted at the end of the module.

Abbreviations are expanded in the order they appear, such that previously inserted type definitions are reused by consecutive expansions.

6.5 Instructions

Instructions are syntactically distinguished into *plain* and *structured* instructions.

$$
\begin{array}{rcll}
\texttt{instr}_I & ::= & in{:}\texttt{plaininstr}_I & \Rightarrow \quad in \\
& | & in{:}\texttt{blockinstr}_I & \Rightarrow \quad in \\
\texttt{instrs}_I & ::= & in^*{:}\texttt{instr}_I^* & \Rightarrow \quad in^*
\end{array}
$$

In addition, as a syntactic abbreviation, instructions can be written as S-expressions in folded form, to group them visually.

6.5.1 Labels

Structured control instructions can be annotated with a symbolic label identifier. They are the only symbolic identifiers that can be bound locally in an instruction sequence. The following grammar handles the corresponding update to the identifier context by composing the context with an additional label entry.

$$
\begin{array}{lcll}
\texttt{label}_I & ::= & \epsilon & \Rightarrow & (\epsilon, \{\mathsf{labels}\ \epsilon\} \oplus I) & \\
& | & id\texttt{:id} & \Rightarrow & (id, \{\mathsf{labels}\ id\} \oplus I) & \text{if } id \notin I.\mathsf{labels} \\
& | & id\texttt{:id} & \Rightarrow & (id, \{\mathsf{labels}\ id\} \oplus I[.\mathsf{labels}[x] = \epsilon]) & \text{if } id = I.\mathsf{labels}[x]
\end{array}
$$

Note

The new label entry is inserted at the *beginning* of the label list in the identifier context. This effectively shifts all existing labels up by one, mirroring the fact that control instructions are indexed relatively not absolutely.

If a label with the same name already exists, then it is shadowed and the earlier label becomes inaccessible.

6.5.2 Parametric Instructions

$$
\begin{array}{lcll}
\texttt{plaininstr}_I & ::= & \text{`}\texttt{unreachable}\text{'} & \Rightarrow & \mathsf{unreachable} \\
& | & \text{`}\texttt{nop}\text{'} & \Rightarrow & \mathsf{nop} \\
& | & \text{`}\texttt{drop}\text{'} & \Rightarrow & \mathsf{drop} \\
& | & \text{`}\texttt{select}\text{'}\ (t^*\texttt{:result}_I^*)^? & \Rightarrow & \mathsf{select}\ (t^*)^?
\end{array}
$$

6.5.3 Control Instructions

Structured control instructions can bind an optional symbolic label identifier. The same label identifier may optionally be repeated after the corresponding ‘`end`’ or ‘`else`’ keywords, to indicate the matching delimiters.

Their block type is given as a type use, analogous to the type of functions. However, the special case of a type use that is syntactically empty or consists of only a single result is not regarded as an abbreviation for an inline function

type, but is parsed directly into an optional value type.

$$
\begin{array}{rcll}
\texttt{blocktype}_I & ::= & t^?{:}\texttt{result}_I^? & \Rightarrow\ t^? \\
& | & (x, I'){:}\texttt{typeuse}_I & \Rightarrow\ x \\
& & \quad \text{if } I' = \{\text{locals } (\epsilon)^*\} & \\
\texttt{blockinstr}_I & ::= & \text{`block'}\ (id^?, I'){:}\texttt{label}_I\ bt{:}\texttt{blocktype}_I & \Rightarrow\ \text{block } bt\ in^* \\
& & in^*{:}\texttt{instrs}_{I'} & \\
& & \text{`end'}\ id'^?{:}\texttt{id}^? & \\
& & \quad \text{if } id'^? = \epsilon \vee id'^? = id^? & \\
& | & \text{`loop'}\ (id^?, I'){:}\texttt{label}_I\ bt{:}\texttt{blocktype}_I & \Rightarrow\ \text{loop } bt\ in^* \\
& & in^*{:}\texttt{instrs}_{I'} & \\
& & \text{`end'}\ id'^?{:}\texttt{id}^? & \\
& & \quad \text{if } id'^? = \epsilon \vee id'^? = id^? & \\
& | & \text{`if'}\ (id^?, I'){:}\texttt{label}_I\ bt{:}\texttt{blocktype}_I & \Rightarrow\ \text{if } bt\ in_1^* \text{ else } in_2^* \\
& & in_1^*{:}\texttt{instrs}_{I'} & \\
& & \text{`else'}\ id_1^?{:}\texttt{id}^? & \\
& & in_2^*{:}\texttt{instrs}_{I'} & \\
& & \text{`end'}\ id_2^?{:}\texttt{id}^? & \\
& & \quad \text{if } (id_1^? = \epsilon \vee id_1^? = id^?) \wedge (id_2^? = \epsilon \vee id_2^? = id^?) & \\
& | & \text{`try_table'}\ (id^?, I'){:}\texttt{label}_I\ bt{:}\texttt{blocktype}_I & \Rightarrow\ \text{try_table } bt\ c^*\ in^* \\
& & c^*{:}\texttt{catch}_I^* & \\
& & in^*{:}\texttt{instrs}_{I'} & \\
& & \text{`end'}\ id'^?{:}\texttt{id}^? & \\
& & \quad \text{if } id'^? = \epsilon \vee id'^? = id^? & \\
\texttt{catch}_I & ::= & \text{`('}\ \text{`catch'}\ x{:}\texttt{tagidx}_I\ l{:}\texttt{labelidx}_I\ \text{`)'} & \Rightarrow\ \text{catch } x\ l \\
& | & \text{`('}\ \text{`catch_ref'}\ x{:}\texttt{tagidx}_I\ l{:}\texttt{labelidx}_I\ \text{`)'} & \Rightarrow\ \text{catch_ref } x\ l \\
& | & \text{`('}\ \text{`catch_all'}\ l{:}\texttt{labelidx}_I\ \text{`)'} & \Rightarrow\ \text{catch_all } l \\
& | & \text{`('}\ \text{`catch_all_ref'}\ l{:}\texttt{labelidx}_I\ \text{`)'} & \Rightarrow\ \text{catch_all_ref } l
\end{array}
$$

Note

The side condition stating that the identifier context I' must only contain unnamed entries in the rule for `typeuse` block types enforces that no identifier can be bound in any `param` declaration for a block type.

All other control instruction are represented verbatim.

$$
\begin{array}{rcll}
\texttt{plaininstr}_I & ::= & \ldots & \\
& | & \text{`br'}\ l{:}\texttt{labelidx}_I & \Rightarrow\ \text{br } l \\
& | & \text{`br_if'}\ l{:}\texttt{labelidx}_I & \Rightarrow\ \text{br_if } l \\
& | & \text{`br_table'}\ l^*{:}\texttt{labelidx}_I^*\ l'{:}\texttt{labelidx}_I & \Rightarrow\ \text{br_table } l^*\ l' \\
& | & \text{`br_on_null'}\ l{:}\texttt{labelidx}_I & \Rightarrow\ \text{br_on_null } l \\
& | & \text{`br_on_non_null'}\ l{:}\texttt{labelidx}_I & \Rightarrow\ \text{br_on_non_null } l \\
& | & \text{`br_on_cast'}\ l{:}\texttt{labelidx}_I\ rt_1{:}\texttt{reftype}_I\ rt_2{:}\texttt{reftype}_I & \Rightarrow\ \text{br_on_cast } l\ rt_1\ rt_2 \\
& | & \text{`br_on_cast_fail'}\ l{:}\texttt{labelidx}_I\ rt_1{:}\texttt{reftype}_I\ rt_2{:}\texttt{reftype}_I & \Rightarrow\ \text{br_on_cast_fail } l\ rt_1\ rt_2 \\
& | & \text{`call'}\ x{:}\texttt{funcidx}_I & \Rightarrow\ \text{call } x \\
& | & \text{`call_ref'}\ x{:}\texttt{typeidx}_I & \Rightarrow\ \text{call_ref } x \\
& | & \text{`call_indirect'}\ x{:}\texttt{tableidx}_I\ (y, I'){:}\texttt{typeuse}_I & \Rightarrow\ \text{call_indirect } x\ y \quad \text{if } I' = \\
& | & \text{`return'} & \Rightarrow\ \text{return} \\
& | & \text{`return_call'}\ x{:}\texttt{funcidx}_I & \Rightarrow\ \text{return_call } x \\
& | & \text{`return_call_ref'}\ x{:}\texttt{typeidx}_I & \Rightarrow\ \text{return_call_ref } x \\
& | & \text{`return_call_indirect'}\ x{:}\texttt{tableidx}_I\ (y, I'){:}\texttt{typeuse}_I & \Rightarrow\ \text{return_call_indirect } x\ y \quad \text{if } I' = \\
& | & \text{`throw'}\ x{:}\texttt{tagidx}_I & \Rightarrow\ \text{throw } x \\
& | & \text{`throw_ref'} & \Rightarrow\ \text{throw_ref}
\end{array}
$$

Note

The side condition stating that the identifier context I' must only contain unnamed entries in the rule for call_indirect enforces that no identifier can be bound in any `param` declaration appearing in the type annotation.

Abbreviations

The '`else`' keyword of an '`if`' instruction can be omitted if the following instruction sequence is empty.

$$\begin{array}{rcl} \mathit{blockinstr}_I & ::= & \dots \\ & | & \texttt{'if'}\ \mathit{label}_I\ \mathit{blocktype}_I\ \mathit{instrs}_I\ \texttt{'end'}\ \mathit{id}^? \quad \equiv \\ & & \quad \texttt{'if'}\ \mathit{label}_I\ \mathit{blocktype}_I\ \mathit{instrs}_I\ \texttt{'else'}\ \texttt{'end'}\ \mathit{id}^? \end{array}$$

Also, for backwards compatibility, the table index to '`call_indirect`' and '`return_call_indirect`' can be omitted, defaulting to 0.

$$\begin{array}{rcll} \mathit{plaininstr}_I & ::= & \dots & \\ & | & \texttt{'call_indirect'}\ \mathit{typeuse}_I & \equiv \quad \texttt{'call_indirect'}\ \texttt{'0'}\ \mathit{typeuse}_I \\ & | & \texttt{'return_call_indirect'}\ \mathit{typeuse}_I & \equiv \quad \texttt{'return_call_indirect'}\ \texttt{'0'}\ \mathit{typeuse}_I \end{array}$$

6.5.4 Variable Instructions

$$\begin{array}{rcll} \mathit{plaininstr}_I & ::= & \dots & \\ & | & \texttt{'local.get'}\ x{:}\mathit{localidx}_I & \Rightarrow \quad \text{local.get}\ x \\ & | & \texttt{'local.set'}\ x{:}\mathit{localidx}_I & \Rightarrow \quad \text{local.set}\ x \\ & | & \texttt{'local.tee'}\ x{:}\mathit{localidx}_I & \Rightarrow \quad \text{local.tee}\ x \\ & | & \texttt{'global.get'}\ x{:}\mathit{globalidx}_I & \Rightarrow \quad \text{global.get}\ x \\ & | & \texttt{'global.set'}\ x{:}\mathit{globalidx}_I & \Rightarrow \quad \text{global.set}\ x \end{array}$$

6.5.5 Table Instructions

$$\begin{array}{rcll} \mathit{plaininstr}_I & ::= & \dots & \\ & | & \texttt{'table.get'}\ x{:}\mathit{tableidx}_I & \Rightarrow \quad \text{table.get}\ x \\ & | & \texttt{'table.set'}\ x{:}\mathit{tableidx}_I & \Rightarrow \quad \text{table.set}\ x \\ & | & \texttt{'table.size'}\ x{:}\mathit{tableidx}_I & \Rightarrow \quad \text{table.size}\ x \\ & | & \texttt{'table.grow'}\ x{:}\mathit{tableidx}_I & \Rightarrow \quad \text{table.grow}\ x \\ & | & \texttt{'table.fill'}\ x{:}\mathit{tableidx}_I & \Rightarrow \quad \text{table.fill}\ x \\ & | & \texttt{'table.copy'}\ x_1{:}\mathit{tableidx}_I\ x_2{:}\mathit{tableidx}_I & \Rightarrow \quad \text{table.copy}\ x_1\ x_2 \\ & | & \texttt{'table.init'}\ x{:}\mathit{tableidx}_I\ y{:}\mathit{elemidx}_I & \Rightarrow \quad \text{table.init}\ x\ y \\ & | & \texttt{'elem.drop'}\ x{:}\mathit{elemidx}_I & \Rightarrow \quad \text{elem.drop}\ x \end{array}$$

Abbreviations

For backwards compatibility, all table indices may be omitted from table instructions, defaulting to 0.

$$\begin{array}{rcll} \mathit{plaininstr}_I & ::= & \dots & \\ & | & \texttt{'table.get'} & \equiv \quad \texttt{'table.get'}\ \texttt{'0'} \\ & | & \texttt{'table.set'} & \equiv \quad \texttt{'table.set'}\ \texttt{'0'} \\ & | & \texttt{'table.size'} & \equiv \quad \texttt{'table.size'}\ \texttt{'0'} \\ & | & \texttt{'table.grow'} & \equiv \quad \texttt{'table.grow'}\ \texttt{'0'} \\ & | & \texttt{'table.fill'} & \equiv \quad \texttt{'table.fill'}\ \texttt{'0'} \\ & | & \texttt{'table.copy'} & \equiv \quad \texttt{'table.copy'}\ \texttt{'0'}\ \texttt{'0'} \\ & | & \texttt{'table.init'}\ \mathit{elemidx}_I & \equiv \quad \texttt{'table.init'}\ \texttt{'0'}\ \mathit{elemidx}_I \end{array}$$

6.5.6 Memory Instructions

The offset and alignment immediates to memory instructions are optional. The offset defaults to 0, the alignment to the storage size of the respective memory access, which is its *natural alignment*. Lexically, an `offset` or `align` phrase is considered a single keyword token, so no white space is allowed around the '='.

$\texttt{memarg}_N$	::=	m:`offset` n:$\texttt{align}_N$	$\Rightarrow$	$\{\mathsf{align}\ n,\ \mathsf{offset}\ m\}$	
`offset`	::=	'`offset=`' m:`u64`	$\Rightarrow$	m	
	$\vert$	ϵ	$\Rightarrow$	0	
$\texttt{align}_N$	::=	'`align=`' m:`u64`	$\Rightarrow$	n	if $m = 2^n$
	$\vert$	ϵ	$\Rightarrow$	n	if $N = 2^n$
`laneidx`	::=	i:`u8`	$\Rightarrow$	i	

plaininstr_I ::= ...
| 'i32.load' x:memidx_I ao:memarg_4 ⇒ i32.load x ao
| 'i64.load' x:memidx_I ao:memarg_8 ⇒ i64.load x ao
| 'f32.load' x:memidx_I ao:memarg_4 ⇒ f32.load x ao
| 'f64.load' x:memidx_I ao:memarg_8 ⇒ f64.load x ao
| 'i32.load8_s' x:memidx_I ao:memarg_1 ⇒ i32.load8_s x ao
| 'i32.load8_u' x:memidx_I ao:memarg_1 ⇒ i32.load8_u x ao
| 'i32.load16_s' x:memidx_I ao:memarg_2 ⇒ i32.load16_s x ao
| 'i32.load16_u' x:memidx_I ao:memarg_2 ⇒ i32.load16_u x ao
| 'i64.load8_s' x:memidx_I ao:memarg_1 ⇒ i64.load8_s x ao
| 'i64.load8_u' x:memidx_I ao:memarg_1 ⇒ i64.load8_u x ao
| 'i64.load16_s' x:memidx_I ao:memarg_2 ⇒ i64.load16_s x ao
| 'i64.load16_u' x:memidx_I ao:memarg_2 ⇒ i64.load16_u x ao
| 'i64.load32_s' x:memidx_I ao:memarg_4 ⇒ i64.load32_s x ao
| 'i64.load32_u' x:memidx_I ao:memarg_4 ⇒ i64.load32_u x ao
| 'v128.load' x:memidx_I ao:memarg_{16} ⇒ v128.load x ao
| 'v128.load8x8_s' x:memidx_I ao:memarg_8 ⇒ v128.load8x8_s x ao
| 'v128.load8x8_u' x:memidx_I ao:memarg_8 ⇒ v128.load8x8_u x ao
| 'v128.load16x4_s' x:memidx_I ao:memarg_8 ⇒ v128.load16x4_s x ao
| 'v128.load16x4_u' x:memidx_I ao:memarg_8 ⇒ v128.load16x4_u x ao
| 'v128.load32x2_s' x:memidx_I ao:memarg_8 ⇒ v128.load32x2_s x ao
| 'v128.load32x2_u' x:memidx_I ao:memarg_8 ⇒ v128.load32x2_u x ao
| 'v128.load8_splat' x:memidx_I ao:memarg_1 ⇒ v128.load8_splat x ao
| 'v128.load16_splat' x:memidx_I ao:memarg_2 ⇒ v128.load16_splat x ao
| 'v128.load32_splat' x:memidx_I ao:memarg_4 ⇒ v128.load32_splat x ao
| 'v128.load64_splat' x:memidx_I ao:memarg_8 ⇒ v128.load64_splat x ao
| 'v128.load32_zero' x:memidx_I ao:memarg_4 ⇒ v128.load32_zero x ao
| 'v128.load64_zero' x:memidx_I ao:memarg_8 ⇒ v128.load64_zero x ao
| 'v128.load8_lane' x:memidx_I ao:memarg_1 i:laneidx ⇒ v128.load8_lane x ao i
| 'v128.load16_lane' x:memidx_I ao:memarg_2 i:laneidx ⇒ v128.load16_lane x ao i
| 'v128.load32_lane' x:memidx_I ao:memarg_4 i:laneidx ⇒ v128.load32_lane x ao i
| 'v128.load64_lane' x:memidx_I ao:memarg_8 i:laneidx ⇒ v128.load64_lane x ao i
| 'i32.store' x:memidx_I ao:memarg_4 ⇒ i32.store x ao
| 'i64.store' x:memidx_I ao:memarg_8 ⇒ i64.store x ao
| 'f32.store' x:memidx_I ao:memarg_4 ⇒ f32.store x ao
| 'f64.store' x:memidx_I ao:memarg_8 ⇒ f64.store x ao
| 'i32.store8' x:memidx_I ao:memarg_1 ⇒ i32.store8 x ao
| 'i32.store16' x:memidx_I ao:memarg_2 ⇒ i32.store16 x ao
| 'i64.store8' x:memidx_I ao:memarg_1 ⇒ i64.store8 x ao
| 'i64.store16' x:memidx_I ao:memarg_2 ⇒ i64.store16 x ao
| 'i64.store32' x:memidx_I ao:memarg_4 ⇒ i64.store32 x ao
| 'v128.store' x:memidx_I ao:memarg_{16} ⇒ v128.store x ao
| 'v128.store8_lane' x:memidx_I ao:memarg_1 i:laneidx ⇒ v128.store8_lane x ao i
| 'v128.store16_lane' x:memidx_I ao:memarg_2 i:laneidx ⇒ v128.store16_lane x ao i
| 'v128.store32_lane' x:memidx_I ao:memarg_4 i:laneidx ⇒ v128.store32_lane x ao i
| 'v128.store64_lane' x:memidx_I ao:memarg_8 i:laneidx ⇒ v128.store64_lane x ao i
| 'memory.size' x:memidx_I ⇒ memory.size x
| 'memory.grow' x:memidx_I ⇒ memory.grow x
| 'memory.fill' x:memidx_I ⇒ memory.fill x
| 'memory.copy' x_1:memidx_I x_2:memidx_I ⇒ memory.copy x_1 x_2
| 'memory.init' x:memidx_I y:dataidx$_I$ ⇒ memory.init x y
| 'data.drop' x:dataidx$_I$ ⇒ data.drop x

Abbreviations

As an abbreviation, the memory index can be omitted in all memory instructions, defaulting to 0.

$$
\begin{array}{lcll}
\text{plaininstr}_I & ::= & \dots & \\
& | & \text{'i32.load'}\ \text{memarg}_4 & \equiv \text{'i32.load'}\ \text{'0'}\ \text{memarg}_4 \\
& | & \text{'i64.load'}\ \text{memarg}_8 & \equiv \text{'i64.load'}\ \text{'0'}\ \text{memarg}_8 \\
& | & \text{'f32.load'}\ \text{memarg}_4 & \equiv \text{'f32.load'}\ \text{'0'}\ \text{memarg}_4 \\
& | & \text{'f64.load'}\ \text{memarg}_8 & \equiv \text{'f64.load'}\ \text{'0'}\ \text{memarg}_8 \\
& | & \text{'i32.load8_s'}\ \text{memarg}_1 & \equiv \text{'i32.load8_s'}\ \text{'0'}\ \text{memarg}_1 \\
& | & \text{'i32.load8_u'}\ \text{memarg}_1 & \equiv \text{'i32.load8_u'}\ \text{'0'}\ \text{memarg}_1 \\
& | & \text{'i32.load16_s'}\ \text{memarg}_2 & \equiv \text{'i32.load16_s'}\ \text{'0'}\ \text{memarg}_2 \\
& | & \text{'i32.load16_u'}\ \text{memarg}_2 & \equiv \text{'i32.load16_u'}\ \text{'0'}\ \text{memarg}_2 \\
& | & \text{'i64.load8_s'}\ \text{memarg}_1 & \equiv \text{'i64.load8_s'}\ \text{'0'}\ \text{memarg}_1 \\
& | & \text{'i64.load8_u'}\ \text{memarg}_1 & \equiv \text{'i64.load8_u'}\ \text{'0'}\ \text{memarg}_1 \\
& | & \text{'i64.load16_s'}\ \text{memarg}_2 & \equiv \text{'i64.load16_s'}\ \text{'0'}\ \text{memarg}_2 \\
& | & \text{'i64.load16_u'}\ \text{memarg}_2 & \equiv \text{'i64.load16_u'}\ \text{'0'}\ \text{memarg}_2 \\
& | & \text{'i64.load32_s'}\ \text{memarg}_4 & \equiv \text{'i64.load32_s'}\ \text{'0'}\ \text{memarg}_4 \\
& | & \text{'i64.load32_u'}\ \text{memarg}_4 & \equiv \text{'i64.load32_u'}\ \text{'0'}\ \text{memarg}_4 \\
& | & \text{'v128.load'}\ \text{memarg}_{16} & \equiv \text{'v128.load'}\ \text{'0'}\ \text{memarg}_{16} \\
& | & \text{'v128.load8x8_s'}\ \text{memarg}_8 & \equiv \text{'v128.load8x8_s'}\ \text{'0'}\ \text{memarg}_8 \\
& | & \text{'v128.load8x8_u'}\ \text{memarg}_8 & \equiv \text{'v128.load8x8_u'}\ \text{'0'}\ \text{memarg}_8 \\
& | & \text{'v128.load16x4_s'}\ \text{memarg}_8 & \equiv \text{'v128.load16x4_s'}\ \text{'0'}\ \text{memarg}_8 \\
& | & \text{'v128.load16x4_u'}\ \text{memarg}_8 & \equiv \text{'v128.load16x4_u'}\ \text{'0'}\ \text{memarg}_8 \\
& | & \text{'v128.load32x2_s'}\ \text{memarg}_8 & \equiv \text{'v128.load32x2_s'}\ \text{'0'}\ \text{memarg}_8 \\
& | & \text{'v128.load32x2_u'}\ \text{memarg}_8 & \equiv \text{'v128.load32x2_u'}\ \text{'0'}\ \text{memarg}_8 \\
& | & \text{'v128.load8_splat'}\ \text{memarg}_1 & \equiv \text{'v128.load8_splat'}\ \text{'0'}\ \text{memarg}_1 \\
& | & \text{'v128.load16_splat'}\ \text{memarg}_2 & \equiv \text{'v128.load16_splat'}\ \text{'0'}\ \text{memarg}_2 \\
& | & \text{'v128.load32_splat'}\ \text{memarg}_4 & \equiv \text{'v128.load32_splat'}\ \text{'0'}\ \text{memarg}_4 \\
& | & \text{'v128.load64_splat'}\ \text{memarg}_8 & \equiv \text{'v128.load64_splat'}\ \text{'0'}\ \text{memarg}_8 \\
& | & \text{'v128.load32_zero'}\ \text{memarg}_4 & \equiv \text{'v128.load32_zero'}\ \text{'0'}\ \text{memarg}_4 \\
& | & \text{'v128.load64_zero'}\ \text{memarg}_8 & \equiv \text{'v128.load64_zero'}\ \text{'0'}\ \text{memarg}_8 \\
& | & \text{'v128.load8_lane'}\ \text{memarg}_1\ \text{laneidx} & \equiv \text{'v128.load8_lane'}\ \text{'0'}\ \text{memarg}_1\ \text{laneidx} \\
& | & \text{'v128.load16_lane'}\ \text{memarg}_2\ \text{laneidx} & \equiv \text{'v128.load16_lane'}\ \text{'0'}\ \text{memarg}_2\ \text{laneidx} \\
& | & \text{'v128.load32_lane'}\ \text{memarg}_4\ \text{laneidx} & \equiv \text{'v128.load32_lane'}\ \text{'0'}\ \text{memarg}_4\ \text{laneidx} \\
& | & \text{'v128.load64_lane'}\ \text{memarg}_8\ \text{laneidx} & \equiv \text{'v128.load64_lane'}\ \text{'0'}\ \text{memarg}_8\ \text{laneidx} \\
& | & \text{'i32.store'}\ \text{memarg}_4 & \equiv \text{'i32.store'}\ \text{'0'}\ \text{memarg}_4 \\
& | & \text{'i64.store'}\ \text{memarg}_8 & \equiv \text{'i64.store'}\ \text{'0'}\ \text{memarg}_8 \\
& | & \text{'f32.store'}\ \text{memarg}_4 & \equiv \text{'f32.store'}\ \text{'0'}\ \text{memarg}_4 \\
& | & \text{'f64.store'}\ \text{memarg}_8 & \equiv \text{'f64.store'}\ \text{'0'}\ \text{memarg}_8 \\
& | & \text{'i32.store8'}\ \text{memarg}_1 & \equiv \text{'i32.store8'}\ \text{'0'}\ \text{memarg}_1 \\
& | & \text{'i32.store16'}\ \text{memarg}_2 & \equiv \text{'i32.store16'}\ \text{'0'}\ \text{memarg}_2 \\
& | & \text{'i64.store8'}\ \text{memarg}_1 & \equiv \text{'i64.store8'}\ \text{'0'}\ \text{memarg}_1 \\
& | & \text{'i64.store16'}\ \text{memarg}_2 & \equiv \text{'i64.store16'}\ \text{'0'}\ \text{memarg}_2 \\
& | & \text{'i64.store32'}\ \text{memarg}_4 & \equiv \text{'i64.store32'}\ \text{'0'}\ \text{memarg}_4 \\
& | & \text{'v128.store'}\ \text{memarg}_{16} & \equiv \text{'v128.store'}\ \text{'0'}\ \text{memarg}_{16} \\
& | & \text{'v128.store8_lane'}\ \text{memarg}_1\ \text{laneidx} & \equiv \text{'v128.store8_lane'}\ \text{'0'}\ \text{memarg}_1\ \text{laneidx} \\
& | & \text{'v128.store16_lane'}\ \text{memarg}_2\ \text{laneidx} & \equiv \text{'v128.store16_lane'}\ \text{'0'}\ \text{memarg}_2\ \text{laneidx} \\
& | & \text{'v128.store32_lane'}\ \text{memarg}_4\ \text{laneidx} & \equiv \text{'v128.store32_lane'}\ \text{'0'}\ \text{memarg}_4\ \text{laneidx} \\
& | & \text{'v128.store64_lane'}\ \text{memarg}_8\ \text{laneidx} & \equiv \text{'v128.store64_lane'}\ \text{'0'}\ \text{memarg}_8\ \text{laneidx} \\
& | & \text{'memory.size'} & \equiv \text{'memory.size'}\ \text{'0'} \\
& | & \text{'memory.grow'} & \equiv \text{'memory.grow'}\ \text{'0'} \\
& | & \text{'memory.fill'} & \equiv \text{'memory.fill'}\ \text{'0'} \\
& | & \text{'memory.copy'} & \equiv \text{'memory.copy'}\ \text{'0'}\ \text{'0'} \\
& | & \text{'memory.init'}\ \text{dataidx}_I & \equiv \text{'memory.init'}\ \text{'0'}\ \text{dataidx}_I
\end{array}
$$

6.5.7 Reference Instructions

$$
\begin{array}{rcll}
\text{plaininstr}_I & ::= & \dots & \\
& | & \text{'ref.null'}\ ht{:}\text{heaptype}_I & \Rightarrow\ \text{ref.null}\ ht \\
& | & \text{'ref.func'}\ x{:}\text{funcidx}_I & \Rightarrow\ \text{ref.func}\ x \\
& | & \text{'ref.is_null'} & \Rightarrow\ \text{ref.is_null} \\
& | & \text{'ref.as_non_null'} & \Rightarrow\ \text{ref.as_non_null} \\
& | & \text{'ref.eq'} & \Rightarrow\ \text{ref.eq} \\
& | & \text{'ref.test'}\ rt{:}\text{reftype}_I & \Rightarrow\ \text{ref.test}\ rt \\
& | & \text{'ref.cast'}\ rt{:}\text{reftype}_I & \Rightarrow\ \text{ref.cast}\ rt
\end{array}
$$

6.5.8 Aggregate Instructions

$$
\begin{array}{rcll}
\text{plaininstr}_I & ::= & \dots & \\
& | & \text{'ref.i31'} & \Rightarrow\ \text{ref.i31} \\
& | & \text{'i31.get_s'} & \Rightarrow\ \text{i31.get_s} \\
& | & \text{'i31.get_u'} & \Rightarrow\ \text{i31.get_u} \\
& | & \text{'struct.new'}\ x{:}\text{typeidx}_I & \Rightarrow\ \text{struct.new}\ x \\
& | & \text{'struct.new_default'}\ x{:}\text{typeidx}_I & \Rightarrow\ \text{struct.new_default}\ x \\
& | & \text{'struct.get'}\ x{:}\text{typeidx}_I\ i{:}\text{fieldidx}_{I,x} & \Rightarrow\ \text{struct.get}\ x\ i \\
& | & \text{'struct.get_s'}\ x{:}\text{typeidx}_I\ i{:}\text{fieldidx}_{I,x} & \Rightarrow\ \text{struct.get_s}\ x\ i \\
& | & \text{'struct.get_u'}\ x{:}\text{typeidx}_I\ i{:}\text{fieldidx}_{I,x} & \Rightarrow\ \text{struct.get_u}\ x\ i \\
& | & \text{'struct.set'}\ x{:}\text{typeidx}_I\ i{:}\text{fieldidx}_{I,x} & \Rightarrow\ \text{struct.set}\ x\ i \\
& | & \text{'array.new'}\ x{:}\text{typeidx}_I & \Rightarrow\ \text{array.new}\ x \\
& | & \text{'array.new_default'}\ x{:}\text{typeidx}_I & \Rightarrow\ \text{array.new_default}\ x \\
& | & \text{'array.new_fixed'}\ x{:}\text{typeidx}_I\ n{:}\text{u32} & \Rightarrow\ \text{array.new_fixed}\ x\ n \\
& | & \text{'array.new_data'}\ x{:}\text{typeidx}_I\ y{:}\text{dataidx}_I & \Rightarrow\ \text{array.new_data}\ x\ y \\
& | & \text{'array.new_elem'}\ x{:}\text{typeidx}_I\ y{:}\text{elemidx}_I & \Rightarrow\ \text{array.new_elem}\ x\ y \\
& | & \text{'array.get'}\ x{:}\text{typeidx}_I & \Rightarrow\ \text{array.get}\ x \\
& | & \text{'array.get_s'}\ x{:}\text{typeidx}_I & \Rightarrow\ \text{array.get_s}\ x \\
& | & \text{'array.get_u'}\ x{:}\text{typeidx}_I & \Rightarrow\ \text{array.get_u}\ x \\
& | & \text{'array.set'}\ x{:}\text{typeidx}_I & \Rightarrow\ \text{array.set}\ x \\
& | & \text{'array.len'} & \Rightarrow\ \text{array.len} \\
& | & \text{'array.fill'}\ x{:}\text{typeidx}_I & \Rightarrow\ \text{array.fill}\ x \\
& | & \text{'array.copy'}\ x_1{:}\text{typeidx}_I\ x_2{:}\text{typeidx}_I & \Rightarrow\ \text{array.copy}\ x_1\ x_2 \\
& | & \text{'array.init_data'}\ x{:}\text{typeidx}_I\ y{:}\text{dataidx}_I & \Rightarrow\ \text{array.init_data}\ x\ y \\
& | & \text{'array.init_elem'}\ x{:}\text{typeidx}_I\ y{:}\text{elemidx}_I & \Rightarrow\ \text{array.init_elem}\ x\ y \\
& | & \text{'any.convert_extern'} & \Rightarrow\ \text{any.convert_extern} \\
& | & \text{'extern.convert_any'} & \Rightarrow\ \text{extern.convert_any}
\end{array}
$$

6.5.9 Numeric Instructions

$$
\begin{array}{rcll}
\text{plaininstr}_I & ::= & \dots & \\
& | & \text{'i32.const'}\ c{:}\text{i32} & \Rightarrow\ \text{i32.const}\ c \\
& | & \text{'i64.const'}\ c{:}\text{i64} & \Rightarrow\ \text{i64.const}\ c \\
& | & \text{'f32.const'}\ c{:}\text{f32} & \Rightarrow\ \text{f32.const}\ c \\
& | & \text{'f64.const'}\ c{:}\text{f64} & \Rightarrow\ \text{f64.const}\ c
\end{array}
$$

plaininstr_I ::= ...
| 'i32.eqz' ⇒ i32.eqz
| 'i32.eq' ⇒ i32.eq
| 'i32.ne' ⇒ i32.ne
| 'i32.lt_s' ⇒ i32.lt_s
| 'i32.lt_u' ⇒ i32.lt_u
| 'i32.gt_s' ⇒ i32.gt_s
| 'i32.gt_u' ⇒ i32.gt_u
| 'i32.le_s' ⇒ i32.le_s
| 'i32.le_u' ⇒ i32.le_u
| 'i32.ge_s' ⇒ i32.ge_s
| 'i32.ge_u' ⇒ i32.ge_u
| 'i32.clz' ⇒ i32.clz
| 'i32.ctz' ⇒ i32.ctz
| 'i32.popcnt' ⇒ i32.popcnt
| 'i32.extend8_s' ⇒ i32.extend8_s
| 'i32.extend16_s' ⇒ i32.extend16_s
| 'i32.add' ⇒ i32.add
| 'i32.sub' ⇒ i32.sub
| 'i32.mul' ⇒ i32.mul
| 'i32.div_s' ⇒ i32.div_s
| 'i32.div_u' ⇒ i32.div_u
| 'i32.rem_s' ⇒ i32.rem_s
| 'i32.rem_u' ⇒ i32.rem_u
| 'i32.and' ⇒ i32.and
| 'i32.or' ⇒ i32.or
| 'i32.xor' ⇒ i32.xor
| 'i32.shl' ⇒ i32.shl
| 'i32.shr_s' ⇒ i32.shr_s
| 'i32.shr_u' ⇒ i32.shr_u
| 'i32.rotl' ⇒ i32.rotl
| 'i32.rotr' ⇒ i32.rotr

$$
\begin{array}{lcll}
\text{plaininstr}_I & ::= & \ldots & \\
& | & \text{'i64.eqz'} & \Rightarrow \quad \text{i64.eqz} \\
& | & \text{'i64.eq'} & \Rightarrow \quad \text{i64.eq} \\
& | & \text{'i64.ne'} & \Rightarrow \quad \text{i64.ne} \\
& | & \text{'i64.lt_s'} & \Rightarrow \quad \text{i64.lt_s} \\
& | & \text{'i64.lt_u'} & \Rightarrow \quad \text{i64.lt_u} \\
& | & \text{'i64.gt_s'} & \Rightarrow \quad \text{i64.gt_s} \\
& | & \text{'i64.gt_u'} & \Rightarrow \quad \text{i64.gt_u} \\
& | & \text{'i64.le_s'} & \Rightarrow \quad \text{i64.le_s} \\
& | & \text{'i64.le_u'} & \Rightarrow \quad \text{i64.le_u} \\
& | & \text{'i64.ge_s'} & \Rightarrow \quad \text{i64.ge_s} \\
& | & \text{'i64.ge_u'} & \Rightarrow \quad \text{i64.ge_u} \\
& | & \text{'i64.clz'} & \Rightarrow \quad \text{i64.clz} \\
& | & \text{'i64.ctz'} & \Rightarrow \quad \text{i64.ctz} \\
& | & \text{'i64.popcnt'} & \Rightarrow \quad \text{i64.popcnt} \\
& | & \text{'i64.extend8_s'} & \Rightarrow \quad \text{i64.extend}_8\text{_s} \\
& | & \text{'i64.extend16_s'} & \Rightarrow \quad \text{i64.extend}_{16}\text{_s} \\
& | & \text{'i64.extend32_s'} & \Rightarrow \quad \text{i64.extend}_{32}\text{_s} \\
& | & \text{'i64.add'} & \Rightarrow \quad \text{i64.add} \\
& | & \text{'i64.sub'} & \Rightarrow \quad \text{i64.sub} \\
& | & \text{'i64.mul'} & \Rightarrow \quad \text{i64.mul} \\
& | & \text{'i64.div_s'} & \Rightarrow \quad \text{i64.div_s} \\
& | & \text{'i64.div_u'} & \Rightarrow \quad \text{i64.div_u} \\
& | & \text{'i64.rem_s'} & \Rightarrow \quad \text{i64.rem_s} \\
& | & \text{'i64.rem_u'} & \Rightarrow \quad \text{i64.rem_u} \\
& | & \text{'i64.and'} & \Rightarrow \quad \text{i64.and} \\
& | & \text{'i64.or'} & \Rightarrow \quad \text{i64.or} \\
& | & \text{'i64.xor'} & \Rightarrow \quad \text{i64.xor} \\
& | & \text{'i64.shl'} & \Rightarrow \quad \text{i64.shl} \\
& | & \text{'i64.shr_s'} & \Rightarrow \quad \text{i64.shr_s} \\
& | & \text{'i64.shr_u'} & \Rightarrow \quad \text{i64.shr_u} \\
& | & \text{'i64.rotl'} & \Rightarrow \quad \text{i64.rotl} \\
& | & \text{'i64.rotr'} & \Rightarrow \quad \text{i64.rotr}
\end{array}
$$

$$
\begin{array}{lcll}
\text{plaininstr}_I & ::= & \ldots & \\
& | & \text{'f32.eq'} & \Rightarrow \quad \text{f32.eq} \\
& | & \text{'f32.ne'} & \Rightarrow \quad \text{f32.ne} \\
& | & \text{'f32.lt'} & \Rightarrow \quad \text{f32.lt} \\
& | & \text{'f32.gt'} & \Rightarrow \quad \text{f32.gt} \\
& | & \text{'f32.le'} & \Rightarrow \quad \text{f32.le} \\
& | & \text{'f32.ge'} & \Rightarrow \quad \text{f32.ge} \\
& | & \text{'f32.abs'} & \Rightarrow \quad \text{f32.abs} \\
& | & \text{'f32.neg'} & \Rightarrow \quad \text{f32.neg} \\
& | & \text{'f32.sqrt'} & \Rightarrow \quad \text{f32.sqrt} \\
& | & \text{'f32.ceil'} & \Rightarrow \quad \text{f32.ceil} \\
& | & \text{'f32.floor'} & \Rightarrow \quad \text{f32.floor} \\
& | & \text{'f32.trunc'} & \Rightarrow \quad \text{f32.trunc} \\
& | & \text{'f32.nearest'} & \Rightarrow \quad \text{f32.nearest} \\
& | & \text{'f32.add'} & \Rightarrow \quad \text{f32.add} \\
& | & \text{'f32.sub'} & \Rightarrow \quad \text{f32.sub} \\
& | & \text{'f32.mul'} & \Rightarrow \quad \text{f32.mul} \\
& | & \text{'f32.div'} & \Rightarrow \quad \text{f32.div} \\
& | & \text{'f32.min'} & \Rightarrow \quad \text{f32.min} \\
& | & \text{'f32.max'} & \Rightarrow \quad \text{f32.max} \\
& | & \text{'f32.copysign'} & \Rightarrow \quad \text{f32.copysign}
\end{array}
$$

$$
\begin{array}{rcll}
\mathit{plaininstr}_I & ::= & \ldots & \\
& | & \text{`f64.eq'} & \Rightarrow \quad \mathsf{f64.eq} \\
& | & \text{`f64.ne'} & \Rightarrow \quad \mathsf{f64.ne} \\
& | & \text{`f64.lt'} & \Rightarrow \quad \mathsf{f64.lt} \\
& | & \text{`f64.gt'} & \Rightarrow \quad \mathsf{f64.gt} \\
& | & \text{`f64.le'} & \Rightarrow \quad \mathsf{f64.le} \\
& | & \text{`f64.ge'} & \Rightarrow \quad \mathsf{f64.ge} \\
& | & \text{`f64.abs'} & \Rightarrow \quad \mathsf{f64.abs} \\
& | & \text{`f64.neg'} & \Rightarrow \quad \mathsf{f64.neg} \\
& | & \text{`f64.sqrt'} & \Rightarrow \quad \mathsf{f64.sqrt} \\
& | & \text{`f64.ceil'} & \Rightarrow \quad \mathsf{f64.ceil} \\
& | & \text{`f64.floor'} & \Rightarrow \quad \mathsf{f64.floor} \\
& | & \text{`f64.trunc'} & \Rightarrow \quad \mathsf{f64.trunc} \\
& | & \text{`f64.nearest'} & \Rightarrow \quad \mathsf{f64.nearest} \\
& | & \text{`f64.add'} & \Rightarrow \quad \mathsf{f64.add} \\
& | & \text{`f64.sub'} & \Rightarrow \quad \mathsf{f64.sub} \\
& | & \text{`f64.mul'} & \Rightarrow \quad \mathsf{f64.mul} \\
& | & \text{`f64.div'} & \Rightarrow \quad \mathsf{f64.div} \\
& | & \text{`f64.min'} & \Rightarrow \quad \mathsf{f64.min} \\
& | & \text{`f64.max'} & \Rightarrow \quad \mathsf{f64.max} \\
& | & \text{`f64.copysign'} & \Rightarrow \quad \mathsf{f64.copysign}
\end{array}
$$

$$
\begin{array}{rcll}
\mathit{plaininstr}_I & ::= & \ldots & \\
& | & \text{`i32.wrap_i64'} & \Rightarrow \quad \mathsf{i32.wrap_i64} \\
& | & \text{`i32.trunc_f32_s'} & \Rightarrow \quad \mathsf{i32.trunc_s_f32} \\
& | & \text{`i32.trunc_f32_u'} & \Rightarrow \quad \mathsf{i32.trunc_u_f32} \\
& | & \text{`i32.trunc_f64_s'} & \Rightarrow \quad \mathsf{i32.trunc_s_f64} \\
& | & \text{`i32.trunc_f64_u'} & \Rightarrow \quad \mathsf{i32.trunc_u_f64} \\
& | & \text{`i32.trunc_sat_f32_s'} & \Rightarrow \quad \mathsf{i32.trunc_sat_s_f32} \\
& | & \text{`i32.trunc_sat_f32_u'} & \Rightarrow \quad \mathsf{i32.trunc_sat_u_f32} \\
& | & \text{`i32.trunc_sat_f64_s'} & \Rightarrow \quad \mathsf{i32.trunc_sat_s_f64} \\
& | & \text{`i32.trunc_sat_f64_u'} & \Rightarrow \quad \mathsf{i32.trunc_sat_u_f64} \\
& | & \text{`i64.extend_i32_s'} & \Rightarrow \quad \mathsf{i64.extend_s_i32} \\
& | & \text{`i64.extend_i32_u'} & \Rightarrow \quad \mathsf{i64.extend_u_i32} \\
& | & \text{`i64.trunc_f32_s'} & \Rightarrow \quad \mathsf{i64.trunc_s_f32} \\
& | & \text{`i64.trunc_f32_u'} & \Rightarrow \quad \mathsf{i64.trunc_u_f32} \\
& | & \text{`i64.trunc_f64_s'} & \Rightarrow \quad \mathsf{i64.trunc_s_f64} \\
& | & \text{`i64.trunc_f64_u'} & \Rightarrow \quad \mathsf{i64.trunc_u_f64} \\
& | & \text{`i64.trunc_sat_f32_s'} & \Rightarrow \quad \mathsf{i64.trunc_sat_s_f32} \\
& | & \text{`i64.trunc_sat_f32_u'} & \Rightarrow \quad \mathsf{i64.trunc_sat_u_f32} \\
& | & \text{`i64.trunc_sat_f64_s'} & \Rightarrow \quad \mathsf{i64.trunc_sat_s_f64} \\
& | & \text{`i64.trunc_sat_f64_u'} & \Rightarrow \quad \mathsf{i64.trunc_sat_u_f64} \\
& | & \text{`f32.demote_f64'} & \Rightarrow \quad \mathsf{f32.demote_f64} \\
& | & \text{`f32.convert_i32_s'} & \Rightarrow \quad \mathsf{f32.convert_s_i32} \\
& | & \text{`f32.convert_i32_u'} & \Rightarrow \quad \mathsf{f32.convert_u_i32} \\
& | & \text{`f32.convert_i64_s'} & \Rightarrow \quad \mathsf{f32.convert_s_i64} \\
& | & \text{`f32.convert_i64_u'} & \Rightarrow \quad \mathsf{f32.convert_u_i64} \\
& | & \text{`f64.promote_f32'} & \Rightarrow \quad \mathsf{f64.promote_f32} \\
& | & \text{`f64.convert_i32_s'} & \Rightarrow \quad \mathsf{f64.convert_s_i32} \\
& | & \text{`f64.convert_i32_u'} & \Rightarrow \quad \mathsf{f64.convert_u_i32} \\
& | & \text{`f64.convert_i64_s'} & \Rightarrow \quad \mathsf{f64.convert_s_i64} \\
& | & \text{`f64.convert_i64_u'} & \Rightarrow \quad \mathsf{f64.convert_u_i64} \\
& | & \text{`i32.reinterpret_f32'} & \Rightarrow \quad \mathsf{i32.reinterpret_f32} \\
& | & \text{`i64.reinterpret_f64'} & \Rightarrow \quad \mathsf{i64.reinterpret_f64} \\
& | & \text{`f32.reinterpret_i32'} & \Rightarrow \quad \mathsf{f32.reinterpret_i32} \\
& | & \text{`f64.reinterpret_i64'} & \Rightarrow \quad \mathsf{f64.reinterpret_i64}
\end{array}
$$

6.5.10 Vector Instructions

Vector constant instructions have a mandatory shape descriptor, which determines how the following values are parsed.

$$
\begin{array}{lcll}
\text{plaininstr}_I & ::= & \dots & \\
& | & \text{'v128.const' 'i8x16'} \ c^*{:}\text{i8}^{16} & \Rightarrow \ \text{v128.const}\ \text{bytes}_{\text{i128}}^{-1}(\bigoplus \text{bytes}_{\text{i8}}(c)^*) \\
& | & \text{'v128.const' 'i16x8'} \ c^*{:}\text{i16}^{8} & \Rightarrow \ \text{v128.const}\ \text{bytes}_{\text{i128}}^{-1}(\bigoplus \text{bytes}_{\text{i16}}(c)^*) \\
& | & \text{'v128.const' 'i32x4'} \ c^*{:}\text{i32}^{4} & \Rightarrow \ \text{v128.const}\ \text{bytes}_{\text{i128}}^{-1}(\bigoplus \text{bytes}_{\text{i32}}(c)^*) \\
& | & \text{'v128.const' 'i64x2'} \ c^*{:}\text{i64}^{2} & \Rightarrow \ \text{v128.const}\ \text{bytes}_{\text{i128}}^{-1}(\bigoplus \text{bytes}_{\text{i64}}(c)^*) \\
& | & \text{'v128.const' 'f32x4'} \ c^*{:}\text{f32}^{4} & \Rightarrow \ \text{v128.const}\ \text{bytes}_{\text{i128}}^{-1}(\bigoplus \text{bytes}_{\text{f32}}(c)^*) \\
& | & \text{'v128.const' 'f64x2'} \ c^*{:}\text{f64}^{2} & \Rightarrow \ \text{v128.const}\ \text{bytes}_{\text{i128}}^{-1}(\bigoplus \text{bytes}_{\text{f64}}(c)^*)
\end{array}
$$

$$
\begin{array}{lcll}
\text{plaininstr}_I & ::= & \dots & \\
& | & \text{'i8x16.shuffle'} \ i^*{:}\text{laneidx}^{16} & \Rightarrow \ \text{i8x16.shuffle}\ i^* \\
& | & \text{'i8x16.swizzle'} & \Rightarrow \ \text{i8x16.swizzle} \\
& | & \text{'i8x16.relaxed_swizzle'} & \Rightarrow \ \text{i8x16.relaxed_swizzle} \\
& | & \text{'i8x16.splat'} & \Rightarrow \ \text{i8x16.splat} \\
& | & \text{'i16x8.splat'} & \Rightarrow \ \text{i16x8.splat} \\
& | & \text{'i32x4.splat'} & \Rightarrow \ \text{i32x4.splat} \\
& | & \text{'i64x2.splat'} & \Rightarrow \ \text{i64x2.splat} \\
& | & \text{'f32x4.splat'} & \Rightarrow \ \text{f32x4.splat} \\
& | & \text{'f64x2.splat'} & \Rightarrow \ \text{f64x2.splat} \\
& | & \text{'i8x16.extract_lane_s'} \ i{:}\text{laneidx} & \Rightarrow \ \text{i8x16.extract_lane_s}\ i \\
& | & \text{'i8x16.extract_lane_u'} \ i{:}\text{laneidx} & \Rightarrow \ \text{i8x16.extract_lane_u}\ i \\
& | & \text{'i16x8.extract_lane_s'} \ i{:}\text{laneidx} & \Rightarrow \ \text{i16x8.extract_lane_s}\ i \\
& | & \text{'i16x8.extract_lane_u'} \ i{:}\text{laneidx} & \Rightarrow \ \text{i16x8.extract_lane_u}\ i \\
& | & \text{'i32x4.extract_lane'} \ i{:}\text{laneidx} & \Rightarrow \ \text{i32x4.extract_lane}\ i \\
& | & \text{'i64x2.extract_lane'} \ i{:}\text{laneidx} & \Rightarrow \ \text{i64x2.extract_lane}\ i \\
& | & \text{'f32x4.extract_lane'} \ i{:}\text{laneidx} & \Rightarrow \ \text{f32x4.extract_lane}\ i \\
& | & \text{'f64x2.extract_lane'} \ i{:}\text{laneidx} & \Rightarrow \ \text{f64x2.extract_lane}\ i \\
& | & \text{'i8x16.replace_lane'} \ i{:}\text{laneidx} & \Rightarrow \ \text{i8x16.replace_lane}\ i \\
& | & \text{'i16x8.replace_lane'} \ i{:}\text{laneidx} & \Rightarrow \ \text{i16x8.replace_lane}\ i \\
& | & \text{'i32x4.replace_lane'} \ i{:}\text{laneidx} & \Rightarrow \ \text{i32x4.replace_lane}\ i \\
& | & \text{'i64x2.replace_lane'} \ i{:}\text{laneidx} & \Rightarrow \ \text{i64x2.replace_lane}\ i \\
& | & \text{'f32x4.replace_lane'} \ i{:}\text{laneidx} & \Rightarrow \ \text{f32x4.replace_lane}\ i \\
& | & \text{'f64x2.replace_lane'} \ i{:}\text{laneidx} & \Rightarrow \ \text{f64x2.replace_lane}\ i
\end{array}
$$

$$
\begin{array}{lcll}
\text{plaininstr}_I & ::= & \dots & \\
& | & \text{'v128.any_true'} & \Rightarrow \ \text{v128.any_true} \\
& | & \text{'v128.not'} & \Rightarrow \ \text{v128.not} \\
& | & \text{'v128.and'} & \Rightarrow \ \text{v128.and} \\
& | & \text{'v128.andnot'} & \Rightarrow \ \text{v128.andnot} \\
& | & \text{'v128.or'} & \Rightarrow \ \text{v128.or} \\
& | & \text{'v128.xor'} & \Rightarrow \ \text{v128.xor} \\
& | & \text{'v128.bitselect'} & \Rightarrow \ \text{v128.bitselect}
\end{array}
$$

plaininstr_I ::= ...
| 'i8x16.all_true' ⇒ i8x16.all_true
| 'i8x16.eq' ⇒ i8x16.eq
| 'i8x16.ne' ⇒ i8x16.ne
| 'i8x16.lt_s' ⇒ i8x16.lt_s
| 'i8x16.lt_u' ⇒ i8x16.lt_u
| 'i8x16.gt_s' ⇒ i8x16.gt_s
| 'i8x16.gt_u' ⇒ i8x16.gt_u
| 'i8x16.le_s' ⇒ i8x16.le_s
| 'i8x16.le_u' ⇒ i8x16.le_u
| 'i8x16.ge_s' ⇒ i8x16.ge_s
| 'i8x16.ge_u' ⇒ i8x16.ge_u
| 'i8x16.abs' ⇒ i8x16.abs
| 'i8x16.neg' ⇒ i8x16.neg
| 'i8x16.popcnt' ⇒ i8x16.popcnt
| 'i8x16.add' ⇒ i8x16.add
| 'i8x16.add_sat_s' ⇒ i8x16.add_sat_s
| 'i8x16.add_sat_u' ⇒ i8x16.add_sat_u
| 'i8x16.sub' ⇒ i8x16.sub
| 'i8x16.sub_sat_s' ⇒ i8x16.sub_sat_s
| 'i8x16.sub_sat_u' ⇒ i8x16.sub_sat_u
| 'i8x16.min_s' ⇒ i8x16.min_s
| 'i8x16.min_u' ⇒ i8x16.min_u
| 'i8x16.max_s' ⇒ i8x16.max_s
| 'i8x16.max_u' ⇒ i8x16.max_u
| 'i8x16.avgr_u' ⇒ i8x16.avgr_u
| 'i8x16.relaxed_laneselect' ⇒ i8x16.relaxed_laneselect
| 'i8x16.shl' ⇒ i8x16.shl
| 'i8x16.shr_s' ⇒ i8x16.shr_s
| 'i8x16.shr_u' ⇒ i8x16.shr_u
| 'i8x16.bitmask' ⇒ i8x16.bitmask
| 'i8x16.narrow_i16x8_s' ⇒ i8x16.narrow_i16x8_s
| 'i8x16.narrow_i16x8_u' ⇒ i8x16.narrow_i16x8_u

plaininstr_I ::=
| ... |
| 'i16x8.all_true' ⇒ i16x8.all_true |
| 'i16x8.eq' ⇒ i16x8.eq |
| 'i16x8.ne' ⇒ i16x8.ne |
| 'i16x8.lt_s' ⇒ i16x8.lt_s |
| 'i16x8.lt_u' ⇒ i16x8.lt_u |
| 'i16x8.gt_s' ⇒ i16x8.gt_s |
| 'i16x8.gt_u' ⇒ i16x8.gt_u |
| 'i16x8.le_s' ⇒ i16x8.le_s |
| 'i16x8.le_u' ⇒ i16x8.le_u |
| 'i16x8.ge_s' ⇒ i16x8.ge_s |
| 'i16x8.ge_u' ⇒ i16x8.ge_u |
| 'i16x8.abs' ⇒ i16x8.abs |
| 'i16x8.neg' ⇒ i16x8.neg |
| 'i16x8.add' ⇒ i16x8.add |
| 'i16x8.add_sat_s' ⇒ i16x8.add_sat_s |
| 'i16x8.add_sat_u' ⇒ i16x8.add_sat_u |
| 'i16x8.sub' ⇒ i16x8.sub |
| 'i16x8.sub_sat_s' ⇒ i16x8.sub_sat_s |
| 'i16x8.sub_sat_u' ⇒ i16x8.sub_sat_u |
| 'i16x8.mul' ⇒ i16x8.mul |
| 'i16x8.min_s' ⇒ i16x8.min_s |
| 'i16x8.min_u' ⇒ i16x8.min_u |
| 'i16x8.max_s' ⇒ i16x8.max_s |
| 'i16x8.max_u' ⇒ i16x8.max_u |
| 'i16x8.avgr_u' ⇒ i16x8.avgr_u |
| 'i16x8.q15mulr_sat_s' ⇒ i16x8.q15mulr_sat_s |
| 'i16x8.relaxed_q15mulr_s' ⇒ i16x8.relaxed_q15mulr_s |
| 'i16x8.relaxed_laneselect' ⇒ i16x8.relaxed_laneselect |
| 'i16x8.shl' ⇒ i16x8.shl |
| 'i16x8.shr_s' ⇒ i16x8.shr_s |
| 'i16x8.shr_u' ⇒ i16x8.shr_u |
| 'i16x8.bitmask' ⇒ i16x8.bitmask |
| 'i16x8.narrow_i32x4_s' ⇒ i16x8.narrow_i32x4_s |
| 'i16x8.narrow_i32x4_u' ⇒ i16x8.narrow_i32x4_u |

plaininstr_I ::= ...
| 'i32x4.all_true' ⇒ i32x4.all_true
| 'i32x4.eq' ⇒ i32x4.eq
| 'i32x4.ne' ⇒ i32x4.ne
| 'i32x4.lt_s' ⇒ i32x4.lt_s
| 'i32x4.lt_u' ⇒ i32x4.lt_u
| 'i32x4.gt_s' ⇒ i32x4.gt_s
| 'i32x4.gt_u' ⇒ i32x4.gt_u
| 'i32x4.le_s' ⇒ i32x4.le_s
| 'i32x4.le_u' ⇒ i32x4.le_u
| 'i32x4.ge_s' ⇒ i32x4.ge_s
| 'i32x4.ge_u' ⇒ i32x4.ge_u
| 'i32x4.abs' ⇒ i32x4.abs
| 'i32x4.neg' ⇒ i32x4.neg
| 'i32x4.add' ⇒ i32x4.add
| 'i32x4.sub' ⇒ i32x4.sub
| 'i32x4.mul' ⇒ i32x4.mul
| 'i32x4.min_s' ⇒ i32x4.min_s
| 'i32x4.min_u' ⇒ i32x4.min_u
| 'i32x4.max_s' ⇒ i32x4.max_s
| 'i32x4.max_u' ⇒ i32x4.max_u
| 'i32x4.relaxed_laneselect' ⇒ i32x4.relaxed_laneselect
| 'i32x4.shl' ⇒ i32x4.shl
| 'i32x4.shr_s' ⇒ i32x4.shr_s
| 'i32x4.shr_u' ⇒ i32x4.shr_u
| 'i32x4.bitmask' ⇒ i32x4.bitmask

plaininstr_I ::= ...
| 'i64x2.all_true' ⇒ i64x2.all_true
| 'i64x2.eq' ⇒ i64x2.eq
| 'i64x2.ne' ⇒ i64x2.ne
| 'i64x2.lt_s' ⇒ i64x2.lt_s
| 'i64x2.gt_s' ⇒ i64x2.gt_s
| 'i64x2.le_s' ⇒ i64x2.le_s
| 'i64x2.ge_s' ⇒ i64x2.ge_s
| 'i64x2.abs' ⇒ i64x2.abs
| 'i64x2.neg' ⇒ i64x2.neg
| 'i64x2.add' ⇒ i64x2.add
| 'i64x2.sub' ⇒ i64x2.sub
| 'i64x2.mul' ⇒ i64x2.mul
| 'i64x2.relaxed_laneselect' ⇒ i64x2.relaxed_laneselect
| 'i64x2.shl' ⇒ i64x2.shl
| 'i64x2.shr_s' ⇒ i64x2.shr_s
| 'i64x2.shr_u' ⇒ i64x2.shr_u
| 'i64x2.bitmask' ⇒ i64x2.bitmask

plaininstr_I ::= ...
| 'f32x4.eq' ⇒ f32x4.eq
| 'f32x4.ne' ⇒ f32x4.ne
| 'f32x4.lt' ⇒ f32x4.lt
| 'f32x4.gt' ⇒ f32x4.gt
| 'f32x4.le' ⇒ f32x4.le
| 'f32x4.ge' ⇒ f32x4.ge
| 'f32x4.abs' ⇒ f32x4.abs
| 'f32x4.neg' ⇒ f32x4.neg
| 'f32x4.sqrt' ⇒ f32x4.sqrt
| 'f32x4.ceil' ⇒ f32x4.ceil
| 'f32x4.floor' ⇒ f32x4.floor
| 'f32x4.trunc' ⇒ f32x4.trunc
| 'f32x4.nearest' ⇒ f32x4.nearest
| 'f32x4.add' ⇒ f32x4.add
| 'f32x4.sub' ⇒ f32x4.sub
| 'f32x4.mul' ⇒ f32x4.mul
| 'f32x4.div' ⇒ f32x4.div
| 'f32x4.min' ⇒ f32x4.min
| 'f32x4.max' ⇒ f32x4.max
| 'f32x4.pmin' ⇒ f32x4.pmin
| 'f32x4.pmax' ⇒ f32x4.pmax
| 'f32x4.relaxed_min' ⇒ f32x4.relaxed_min
| 'f32x4.relaxed_max' ⇒ f32x4.relaxed_max
| 'f32x4.relaxed_madd' ⇒ f32x4.relaxed_madd
| 'f32x4.relaxed_nmadd' ⇒ f32x4.relaxed_nmadd

plaininstr_I ::= ...
| 'f64x2.eq' ⇒ f64x2.eq
| 'f64x2.ne' ⇒ f64x2.ne
| 'f64x2.lt' ⇒ f64x2.lt
| 'f64x2.gt' ⇒ f64x2.gt
| 'f64x2.le' ⇒ f64x2.le
| 'f64x2.ge' ⇒ f64x2.ge
| 'f64x2.abs' ⇒ f64x2.abs
| 'f64x2.neg' ⇒ f64x2.neg
| 'f64x2.sqrt' ⇒ f64x2.sqrt
| 'f64x2.ceil' ⇒ f64x2.ceil
| 'f64x2.floor' ⇒ f64x2.floor
| 'f64x2.trunc' ⇒ f64x2.trunc
| 'f64x2.nearest' ⇒ f64x2.nearest
| 'f64x2.add' ⇒ f64x2.add
| 'f64x2.sub' ⇒ f64x2.sub
| 'f64x2.mul' ⇒ f64x2.mul
| 'f64x2.div' ⇒ f64x2.div
| 'f64x2.min' ⇒ f64x2.min
| 'f64x2.max' ⇒ f64x2.max
| 'f64x2.pmin' ⇒ f64x2.pmin
| 'f64x2.pmax' ⇒ f64x2.pmax
| 'f64x2.relaxed_min' ⇒ f64x2.relaxed_min
| 'f64x2.relaxed_max' ⇒ f64x2.relaxed_max
| 'f64x2.relaxed_madd' ⇒ f64x2.relaxed_madd
| 'f64x2.relaxed_nmadd' ⇒ f64x2.relaxed_nmadd

```
plaininstr_I ::= ...
               | 'i16x8.extend_low_i8x16_s'           ⇒ i16x8.extend_low_s_i8x16
               | 'i16x8.extend_low_i8x16_u'           ⇒ i16x8.extend_low_u_i8x16
               | 'i16x8.extend_high_i8x16_s'          ⇒ i16x8.extend_high_s_i8x16
               | 'i16x8.extend_high_i8x16_u'          ⇒ i16x8.extend_high_u_i8x16
               | 'i32x4.extend_low_i16x8_s'           ⇒ i32x4.extend_low_s_i16x8
               | 'i32x4.extend_low_i16x8_u'           ⇒ i32x4.extend_low_u_i16x8
               | 'i32x4.extend_high_i16x8_s'          ⇒ i32x4.extend_high_s_i16x8
               | 'i32x4.extend_high_i16x8_u'          ⇒ i32x4.extend_high_u_i16x8
               | 'i32x4.trunc_sat_f32x4_s'            ⇒ i32x4.trunc_sat_s_f32x4
               | 'i32x4.trunc_sat_f32x4_u'            ⇒ i32x4.trunc_sat_u_f32x4
               | 'i32x4.trunc_sat_f64x2_s_zero'       ⇒ i32x4.trunc_sat_s_zero_f64x2
               | 'i32x4.trunc_sat_f64x2_u_zero'       ⇒ i32x4.trunc_sat_u_zero_f64x2
               | 'i32x4.relaxed_trunc_f32x4_s'        ⇒ i32x4.relaxed_trunc_s_f32x4
               | 'i32x4.relaxed_trunc_f32x4_u'        ⇒ i32x4.relaxed_trunc_u_f32x4
               | 'i32x4.relaxed_trunc_f64x2_s_zero'   ⇒ i32x4.relaxed_trunc_s_zero_f64x2
               | 'i32x4.relaxed_trunc_f64x2_u_zero'   ⇒ i32x4.relaxed_trunc_u_zero_f64x2
               | 'i64x2.extend_low_i32x4_s'           ⇒ i64x2.extend_low_s_i32x4
               | 'i64x2.extend_low_i32x4_u'           ⇒ i64x2.extend_low_u_i32x4
               | 'i64x2.extend_high_i32x4_s'          ⇒ i64x2.extend_high_s_i32x4
               | 'i64x2.extend_high_i32x4_u'          ⇒ i64x2.extend_high_u_i32x4
               | 'f32x4.demote_f64x2_zero'            ⇒ f32x4.demote_zero_f64x2
               | 'f32x4.convert_i32x4_s'              ⇒ f32x4.convert_s_i32x4
               | 'f32x4.convert_i32x4_u'              ⇒ f32x4.convert_u_i32x4
               | 'f64x2.promote_low_f32x4'            ⇒ f64x2.promote_low_f32x4
               | 'f64x2.convert_low_i32x4_s'          ⇒ f64x2.convert_low_s_i32x4
               | 'f64x2.convert_low_i32x4_u'          ⇒ f64x2.convert_low_u_i32x4

plaininstr_I ::= ...
               | 'i16x8.extadd_pairwise_i8x16_s'
               | 'i16x8.extadd_pairwise_i8x16_u'
               | 'i16x8.extmul_low_i8x16_s'
               | 'i16x8.extmul_low_i8x16_u'
               | 'i16x8.extmul_high_i8x16_s'
               | 'i16x8.extmul_high_i8x16_u'
               | 'i16x8.relaxed_dot_i8x16_i7x16_s'
               | 'i32x4.extadd_pairwise_i16x8_s'
               | 'i32x4.extadd_pairwise_i16x8_u'
               | 'i32x4.extmul_low_i16x8_s'
               | 'i32x4.extmul_low_i16x8_u'
               | 'i32x4.extmul_high_i16x8_s'
               | 'i32x4.extmul_high_i16x8_u'
               | 'i32x4.dot_i16x8_s'
               | 'i32x4.relaxed_dot_i8x16_i7x16_add_s' ⇒ i32x4.relaxed_dot_add_s_i8x16 | 'i64x2.extmul_low_
               | 'i64x2.extmul_low_i32x4_u'
               | 'i64x2.extmul_high_i32x4_s'
               | 'i64x2.extmul_high_i32x4_u'
```

6.5.11 Folded Instructions

Instructions can be written as S-expressions by grouping them into *folded* form. In that notation, an instruction is wrapped in parentheses and optionally includes nested folded instructions to indicate its operands.

In the case of block instructions, the folded form omits the '`end`' delimiter. For `if` instructions, both branches have to be wrapped into nested S-expressions, headed by the keywords '`then`' and '`else`'.

The set of all phrases defined by the following abbreviations recursively forms the auxiliary syntactic class

`foldedinstr`. Such a folded instruction can appear anywhere a regular instruction can.

$$
\begin{array}{lcll}
\texttt{foldedinstr}_I & ::= & \text{`(' } \texttt{plaininstr}_I\ \texttt{instrs}_I \text{ `)'} & \equiv \\
 & & \quad \texttt{instrs}_I\ \texttt{plaininstr}_I & \\
 & | & \text{`(' `block' } \texttt{label}_I\ \texttt{blocktype}_I\ \texttt{instrs}_{I'} \text{ `)'} & \equiv \\
 & & \quad \text{`block' } \texttt{label}_I\ \texttt{blocktype}_I\ \texttt{instrs}_{I'} \text{ `end'} & \\
 & | & \text{`(' `loop' } \texttt{label}_I\ \texttt{blocktype}_I\ \texttt{instrs}_{I'} \text{ `)'} & \equiv \\
 & & \quad \text{`loop' } \texttt{label}_I\ \texttt{blocktype}_I\ \texttt{instrs}_{I'} \text{ `end'} & \\
 & | & \text{`(' `if' } \texttt{label}_I\ \texttt{blocktype}_I\ \texttt{foldedinstr}_I^* & \equiv \\
 & & \text{`(' `then' } in_1^*\texttt{:instrs}_{I'} \text{ `)'} \ (\text{`(' `else' } in_2^*\texttt{:instrs}_{I'} \text{ `)'})^? \text{ `)'} & \\
 & & \quad \texttt{foldedinstr}_I^* \text{ `if' } \texttt{label}_I\ \texttt{blocktype}_I\ in_1^*\texttt{:instrs}_{I'}\ (\text{`else' } in_2^*\texttt{:instrs}_{I'})^? \text{ `end'} & \\
 & | & \text{`(' `try_table' } \texttt{label}_I\ \texttt{blocktype}_I\ \texttt{catch}_I^*\ \texttt{instrs}_{I'} \text{ `)'} & \equiv \\
 & & \quad \text{`try\_table' } \texttt{label}_I\ \texttt{blocktype}_I\ \texttt{catch}_I^*\ \texttt{instrs}_{I'} \text{ `end'} &
\end{array}
$$

Note

For example, the instruction sequence

$$(\texttt{local.get \$x})\ (\texttt{i32.const 2})\ \texttt{i32.add}\ (\texttt{i32.const 3})\ \texttt{i32.mul}$$

can be folded into

$$(\texttt{i32.mul}\ (\texttt{i32.add}\ (\texttt{local.get \$x})\ (\texttt{i32.const 2}))\ (\texttt{i32.const 3}))$$

Folded instructions are solely syntactic sugar, no additional syntactic or type-based checking is implied.

6.5.12 Expressions

Expressions are written as instruction sequences.

$$\texttt{expr}_I \quad ::= \quad in^*\texttt{:instrs}_I \quad \Rightarrow \quad in^*$$

6.6 Modules

Modules consist of a sequence of declarations. The grammar rules for each declaration construct produce a pair, consisting of not just the abstract syntax representing the respective declaration, but also an identifier context recording the new symbolic identifiers bound by the construct, for use in the remainder of the module.

6.6.1 Indices

Indices can be given either in raw numeric form or as symbolic identifiers when bound by a respective construct. Such identifiers are looked up in the suitable space of the identifier context I.

$$
\begin{array}{rcll}
\texttt{idx}_{ids} & ::= & x\texttt{:u32} & \Rightarrow\ x \\
 & | & id\texttt{:id} & \Rightarrow\ x \quad \text{if } ids[x] = id \\
\\
\texttt{typeidx}_I & ::= & \texttt{idx}_{I.\texttt{types}} & \\
\texttt{tagidx}_I & ::= & \texttt{idx}_{I.\texttt{tags}} & \\
\texttt{globalidx}_I & ::= & \texttt{idx}_{I.\texttt{globals}} & \\
\texttt{memidx}_I & ::= & \texttt{idx}_{I.\texttt{mems}} & \\
\texttt{tableidx}_I & ::= & \texttt{idx}_{I.\texttt{tables}} & \\
\texttt{funcidx}_I & ::= & \texttt{idx}_{I.\texttt{funcs}} & \\
\texttt{dataidx}_I & ::= & \texttt{idx}_{I.\texttt{datas}} & \\
\texttt{elemidx}_I & ::= & \texttt{idx}_{I.\texttt{elems}} & \\
\texttt{localidx}_I & ::= & \texttt{idx}_{I.\texttt{locals}} & \\
\texttt{labelidx}_I & ::= & \texttt{idx}_{I.\texttt{labels}} & \\
\texttt{fieldidx}_{I,x} & ::= & \texttt{idx}_{I.\texttt{fields}[x]} &
\end{array}
$$

6.6.2 Types

A type definition consists of a recursive type. The identifier context produced for the local bindings is further extended with the respective sequence of defined types that the recursive type generates.

$$\begin{array}{lcll} \texttt{type}_I & ::= & (qt, I') \texttt{:rectype}_I & \Rightarrow \quad (\text{type } qt, I' \oplus I'') \quad \text{if } qt = \text{rec } st^n \\ & & & \quad \wedge\ I'' = \{\text{typedefs } (qt.i)^{i<n}\} \end{array}$$

6.6.3 Tags

A tag definition can bind a symbolic tag identifier.

$$\texttt{tag}_I \quad ::= \quad \text{`(' `tag'} \ id^?\texttt{:id}^? \ jt\texttt{:tagtype}_I \ \text{`)'} \quad \Rightarrow \quad (\text{tag } jt, \{\text{tags } (id^?)\})$$

Abbreviations

Tags can be defined as imports or exports inline:

$$\begin{array}{lcl} \texttt{import}_I & ::= & \ldots \\ & | & \text{`(' `tag'} \ \texttt{id}^? \ \text{`(' `import'} \ \texttt{name}^2 \ \text{`)'} \ \texttt{tagtype}_I \ \text{`)'} \quad \equiv \\ & & \quad \text{`(' `import'} \ \texttt{name}^2 \ \text{`(' `tag'} \ \texttt{id}^? \ \texttt{tagtype}_I \ \text{`)' `)'} \\ \texttt{export}_I & ::= & \text{`(' `tag'} \ id^?\texttt{:id}^? \ \text{`(' `export'} \ \texttt{name} \ \text{`)'} \ \ldots \ \text{`)'} \quad \equiv \\ & & \quad \text{`(' `tag'} \ id'\texttt{:id} \ \ldots \ \text{`)'} \\ & & \quad \text{`(' `export'} \ \texttt{name} \ \text{`(' `tag'} \ \texttt{id} \ \text{`)' `)'} \\ & & \qquad \text{if } id^? = id' \vee id^? = \epsilon \wedge id' \notin I.\text{tags} \end{array}$$

Note

The latter abbreviation can be applied repeatedly, if "..." contains additional export clauses. Consequently, a memory declaration can contain any number of exports, possibly followed by an import.

6.6.4 Globals

Global definitions can bind a symbolic global identifier.

$$\texttt{global}_I \quad ::= \quad \text{`(' `global'} \ id^?\texttt{:id}^? \ gt\texttt{:globaltype}_I \ e\texttt{:expr}_I \ \text{`)'} \quad \Rightarrow \quad (\text{global } gt \ e, \{\text{globals } (id^?)\})$$

Abbreviations

Globals can be defined as imports or exports inline:

$$\begin{array}{lcl} \texttt{import}_I & ::= & \ldots \\ & | & \text{`(' `global'} \ \texttt{id}^? \ \text{`(' `import'} \ \texttt{name}^2 \ \text{`)'} \ \texttt{globaltype}_I \ \text{`)'} \quad \equiv \\ & & \quad \text{`(' `import'} \ \texttt{name}^2 \ \text{`(' `global'} \ \texttt{id}^? \ \texttt{globaltype}_I \ \text{`)' `)'} \\ \texttt{export}_I & ::= & \text{`(' `global'} \ id^?\texttt{:id}^? \ \text{`(' `export'} \ \texttt{name} \ \text{`)'} \ \ldots \ \text{`)'} \quad \equiv \\ & & \quad \text{`(' `global'} \ id'\texttt{:id} \ \ldots \ \text{`)'} \\ & & \quad \text{`(' `export'} \ \texttt{name} \ \text{`(' `global'} \ \texttt{id} \ \text{`)' `)'} \\ & & \qquad \text{if } id^? = id' \vee id^? = \epsilon \wedge id' \notin I.\text{globals} \end{array}$$

Note

The latter abbreviation can be applied repeatedly, if "..." contains additional export clauses. Consequently, a global declaration can contain any number of exports, possibly followed by an import.

6.6.5 Memories

Memory definitions can bind a symbolic memory identifier.

$$\texttt{mem}_I \quad ::= \quad \text{`(' `memory'} \; id^?{:}\texttt{id}^? \; mt{:}\texttt{memtype}_I \; \text{`)'} \quad \Rightarrow \quad (\mathsf{memory}\; mt, \{\mathsf{mems}\; (id^?)\})$$

Abbreviations

A data segment can be given inline with a memory definition, in which case its offset is 0 and the limits of the memory type are inferred from the length of the data, rounded up to page size:

$$\begin{array}{rcl}
\texttt{mem}_I & ::= & \text{`(' `memory'} \; id^?{:}\texttt{id}^? \; at^?{:}\texttt{addrtype}^? \; \text{`(' `data'} \; b^*{:}\texttt{datastring} \; \text{`)' `)'} \quad \equiv \\
& & \text{`(' `memory'} \; id'{:}\texttt{id} \; at^?{:}\texttt{addrtype}^? \; n{:}\texttt{u64} \; n{:}\texttt{u64} \; \text{`)'} \\
& & \text{`(' `data' `(' `memory'} \; id'{:}\texttt{id} \; \text{`)' `('} \; at'{:}\texttt{addrtype} \; \text{`.const' `0' `)'} \; \texttt{datastring} \; \text{`)'} \\
& & \quad \text{if } id^? = id' \vee id^? = \epsilon \wedge id' \notin I.\mathsf{mems} \\
& & \quad \wedge \; at^? = at' \vee at^? = \epsilon \wedge at' = \mathsf{i32} \\
& & \quad \wedge \; n = \mathrm{ceil}(|b^*|/64 \cdot \mathrm{Ki})
\end{array}$$

Memories can be defined as imports or exports inline:

$$\begin{array}{rcl}
\texttt{import}_I & ::= & \ldots \\
& | & \text{`(' `memory'} \; \texttt{id}^? \; \text{`(' `import'} \; \texttt{name}^2 \; \text{`)'} \; \texttt{memtype}_I \; \text{`)'} \quad \equiv \\
& & \text{`(' `import'} \; \texttt{name}^2 \; \text{`(' `memory'} \; \texttt{id}^? \; \texttt{memtype}_I \; \text{`)' `)'} \\
\texttt{export}_I & ::= & \text{`(' `memory'} \; id^?{:}\texttt{id}^? \; \text{`(' `export'} \; \texttt{name} \; \text{`)'} \; \ldots \; \text{`)'} \quad \equiv \\
& & \text{`(' `memory'} \; id'{:}\texttt{id} \; \ldots \; \text{`)'} \\
& & \text{`(' `export'} \; \texttt{name} \; \text{`(' `memory'} \; \texttt{id} \; \text{`)' `)'} \\
& & \quad \text{if } id^? = id' \vee id^? = \epsilon \wedge id' \notin I.\mathsf{mems}
\end{array}$$

Note

The latter abbreviation can be applied repeatedly, if "..." contains additional export clauses. Consequently, a memory declaration can contain any number of exports, possibly followed by an import.

6.6.6 Tables

Table definitions can bind a symbolic table identifier.

$$\texttt{table}_I \quad ::= \quad \text{`(' `table'} \; id^?{:}\texttt{id}^? \; tt{:}\texttt{tabletype}_I \; e{:}\texttt{expr}_I \; \text{`)'} \quad \Rightarrow \quad (\mathsf{table}\; tt\; e, \{\mathsf{tables}\; (id^?)\})$$

Abbreviations

A table's initialization expression can be omitted, in which case it defaults to ref.null:

$$\begin{array}{rcl}
\texttt{table}_I & ::= & \ldots \\
& | & \text{`(' `table'} \; \texttt{id}^? \; tt{:}\texttt{tabletype}_I \; \text{`)'} \quad \equiv \\
& & \text{`(' `table'} \; \texttt{id}^? \; tt{:}\texttt{tabletype}_I \; \text{`(' `ref.null'} \; ht{:}\texttt{heaptype}_I \; \text{`)' `)'} \\
& & \quad \text{if } tt = at\; lim\; (\mathsf{ref\ null}^?\; ht)
\end{array}$$

An element segment can be given inline with a table definition, in which case its offset is 0 and the limits of the table type are inferred from the length of the given segment:

$$
\begin{array}{lcl}
\texttt{table}_I & ::= & \text{`(' `table'}\ id^?{:}\texttt{id}^?\ at^?{:}\texttt{addrtype}^?\ \texttt{reftype}_I\ \text{`(' `elem'}\ (rt, e^*){:}\texttt{elemlist}_I\ \text{`)' `)'} \\
& & \quad \equiv\ \text{`(' `table'}\ id'{:}\texttt{id}\ at^?{:}\texttt{addrtype}^?\ n{:}\texttt{u64}\ n{:}\texttt{u64}\ \texttt{reftype}_I\ \text{`)'} \\
& & \quad\ \text{`(' `elem'}\ \text{`(' `table'}\ id'{:}\texttt{id}\ \text{`)'}\ \text{`(' }at'{:}\texttt{addrtype}\ \text{`.const' `0' `)'}\ \texttt{elemlist}_I\ \text{`)'} \\
& & \qquad \text{if } id^? = id' \vee id^? = \epsilon \wedge id' \notin I.\mathsf{tables} \\
& & \qquad \wedge\ at^? = at' \vee at^? = \epsilon \wedge at' = \mathsf{i32} \\
& & \qquad \wedge\ n = |e^*|
\end{array}
$$

Tables can be defined as imports or exports inline:

$$
\begin{array}{lcl}
\texttt{import}_I & ::= & \ldots \\
& & \quad |\ \text{`(' `table'}\ \texttt{id}^?\ \text{`(' `import'}\ \texttt{name}^2\ \text{`)'}\ \texttt{tabletype}_I\ \text{`)'} \\
& & \quad \equiv\ \text{`(' `import'}\ \texttt{name}^2\ \text{`(' `table'}\ \texttt{id}^?\ \texttt{tabletype}_I\ \text{`)' `)'} \\
\texttt{export}_I & ::= & \text{`(' `table'}\ id^?{:}\texttt{id}^?\ \text{`(' `export'}\ \texttt{name}\ \text{`)'}\ \ldots\ \text{`)'} \\
& & \quad \equiv\ \text{`(' `table'}\ id'{:}\texttt{id}\ \ldots\ \text{`)'} \\
& & \quad\ \text{`(' `export'}\ \texttt{name}\ \text{`(' `table'}\ \texttt{id}\ \text{`)' `)'} \\
& & \qquad \text{if } id^? = id' \vee id^? = \epsilon \wedge id' \notin I.\mathsf{tables}
\end{array}
$$

Note

The latter abbreviation can be applied repeatedly, if "…" contains additional export clauses. Consequently, a table declaration can contain any number of exports, possibly followed by an import.

6.6.7 Functions

Function definitions can bind a symbolic function identifier, and local identifiers for its parameters and locals.

$$
\begin{array}{lcl}
\texttt{func}_I & ::= & \text{`(' `func'}\ id^?{:}\texttt{id}^?\ (x, I_1){:}\texttt{typeuse}_I\ ((loc^*, I_2){:}\texttt{local}_I)^*\ e{:}\texttt{expr}_{I'}\ \text{`)'} \\
& & \quad \Rightarrow\ (\mathsf{func}\ x\ (\bigoplus loc^{**})\ e, \{\mathsf{funcs}\ (id^?)\}) \\
& & \qquad \text{if } I' = I \oplus I_1 \oplus \bigoplus I_2^* \\
& & \qquad \wedge \vdash I' : \mathsf{ok}
\end{array}
$$

$$
\begin{array}{lcl}
\texttt{local}_I & ::= & \text{`(' `local'}\ id^?{:}\texttt{id}^?\ t{:}\texttt{valtype}_I\ \text{`)'}\ \Rightarrow\ (\mathsf{local}\ t, \{\mathsf{locals}\ (id^?)\})
\end{array}
$$

Note

The well-formedness condition on I' ensures that parameters and locals do not contain duplicate identifiers.

Abbreviations

Multiple anonymous locals may be combined into a single declaration:

$$
\texttt{local}_I\ ::=\ \ldots\ |\ \text{`(' `local'}\ \texttt{valtype}_I^*\ \text{`)'}\ \equiv\ (\text{`(' `local'}\ \texttt{valtype}_I\ \text{`)'})^*
$$

Functions can be defined as imports or exports inline:

$$
\begin{array}{lcl}
\texttt{import}_I & ::= & \ldots \\
& & \quad |\ \text{`(' `func'}\ \texttt{id}^?\ \text{`(' `import'}\ \texttt{name}^2\ \text{`)'}\ \texttt{typeuse}_I\ \text{`)'} \\
& & \quad \equiv\ \text{`(' `import'}\ \texttt{name}^2\ \text{`(' `func'}\ \texttt{id}^?\ \texttt{typeuse}_I\ \text{`)' `)'} \\
\texttt{export}_I & ::= & \text{`(' `func'}\ id^?{:}\texttt{id}^?\ \text{`(' `export'}\ \texttt{name}\ \text{`)'}\ \ldots\ \text{`)'} \\
& & \quad \equiv\ \text{`(' `func'}\ id'{:}\texttt{id}\ \ldots\ \text{`)'} \\
& & \quad\ \text{`(' `export'}\ \texttt{name}\ \text{`(' `func'}\ \texttt{id}\ \text{`)' `)'} \\
& & \qquad \text{if } id^? = id' \vee id^? = \epsilon \wedge id' \notin I.\mathsf{funcs}
\end{array}
$$

Note

The latter abbreviation can be applied repeatedly, if "..." contains additional export clauses. Consequently, a function declaration can contain any number of exports, possibly followed by an import.

6.6.8 Data Segments

Data segments allow for an optional memory index to identify the memory to initialize. The data is written as a string, which may be split up into a possibly empty sequence of individual string literals.

$$
\begin{array}{rcll}
\text{data}_I & ::= & \text{`(' `data'}\ id^?{:}\text{id}^?\ b^*{:}\text{datastring}\ \text{`)'} & \Rightarrow \\
 & & \quad (\text{data}\ b^*\ \text{passive}, \{\text{datas}\ (id^?)\}) & \\
 & | & \text{`(' `data'}\ id^?{:}\text{id}^?\ x{:}\text{memuse}_I\ e{:}\text{offsetexpr}_I\ b^*{:}\text{datastring}\ \text{`)'} & \Rightarrow \\
 & & \quad (\text{data}\ b^*\ (\text{active}\ x\ e), \{\text{datas}\ (id^?)\}) & \\
\text{datastring} & ::= & b^{**}{:}\text{string}^* & \Rightarrow \bigoplus b^{**} \\
\text{memuse}_I & ::= & \text{`(' `memory'}\ x{:}\text{memidx}_I\ \text{`)'} & \Rightarrow x \\
\text{offsetexpr}_I & ::= & \text{`(' `offset'}\ e{:}\text{expr}_I\ \text{`)'} & \Rightarrow e
\end{array}
$$

Abbreviations

As an abbreviation, a single folded instruction may occur in place of the offset of an active segment:

$$\text{offsetexpr}_I\ ::=\ \ldots \mid \text{foldedinstr}_I\ \equiv\ \text{`(' `offset'}\ \text{foldedinstr}_I\ \text{`)'}$$

Also, a memory use can be omitted, defaulting to 0.

$$\text{memuse}_I\ ::=\ \ldots \mid \epsilon\ \equiv\ \text{`(' `memory' `0' `)'}$$

As another abbreviation, data segments may also be specified inline with memory definitions; see the respective section.

6.6.9 Element Segments

Element segments allow for an optional table index to identify the table to initialize.

$$
\begin{array}{rcll}
\text{elem}_I & ::= & \text{`(' `elem'}\ id^?{:}\text{id}^?\ (rt, e^*){:}\text{elemlist}_I\ \text{`)'} & \Rightarrow \\
 & & \quad (\text{elem}\ rt\ e^*\ \text{passive}, \{\text{elems}\ (id^?)\}) & \\
 & | & \text{`(' `elem'}\ id^?{:}\text{id}^?\ x{:}\text{tableuse}_I\ e'{:}\text{offsetexpr}_I\ (rt, e^*){:}\text{elemlist}_I\ \text{`)'} & \Rightarrow \\
 & & \quad (\text{elem}\ rt\ e^*\ (\text{active}\ x\ e'), \{\text{elems}\ (id^?)\}) & \\
 & | & \text{`(' `elem'}\ id^?{:}\text{id}^?\ \text{`declare'}\ (rt, e^*){:}\text{elemlist}_I\ \text{`)'} & \Rightarrow \\
 & & \quad (\text{elem}\ rt\ e^*\ \text{declare}, \{\text{elems}\ (id^?)\}) & \\
\text{elemlist}_I & ::= & rt{:}\text{reftype}_I\ e^*{:}\text{list}(\text{elemexpr}_I) & \Rightarrow (rt, e^*) \\
\text{elemexpr}_I & ::= & \text{`(' `item'}\ e{:}\text{expr}_I\ \text{`)'} & \Rightarrow e \\
\text{tableuse}_I & ::= & \text{`(' `table'}\ x{:}\text{tableidx}_I\ \text{`)'} & \Rightarrow x
\end{array}
$$

Abbreviations

As an abbreviation, a single folded instruction may occur in place of the offset of an active element segment or as an element expression:

$$\text{elemexpr}_I\ ::=\ \ldots \mid \text{foldedinstr}_I\ \equiv\ \text{`(' `item'}\ \text{foldedinstr}_I\ \text{`)'}$$

Also, the element list may be written as just a sequence of function indices:

$$\text{elemlist}_I\ ::=\ \ldots \mid \text{`func'}\ x^*{:}\text{funcidx}_I^*\ \equiv\ \text{`(' `ref' `func' `)'}\ (\text{`(' `ref.func'}\ \text{funcidx}_I\ \text{`)'})^*$$

A table use can be omitted, defaulting to 0.

$$\texttt{tableuse}_I \;::=\; \ldots \mid \epsilon \;\equiv\; \text{`(' `table' `0' `)'}$$

Furthermore, for backwards compatibility with earlier versions of WebAssembly, if the table use is omitted, the '`func`' keyword can be omitted as well.

$$\begin{array}{rcl}\texttt{elem}_I & ::= & \ldots \\ & \mid & \text{`(' `elem'}\ \texttt{offsetexpr}_I\ \texttt{list}(\texttt{funcidx}_I)\ \text{`)'} \;\equiv \\ & & \quad \text{`(' `elem'}\ \texttt{offsetexpr}_I\ \text{`func'}\ \texttt{list}(\texttt{funcidx}_I)\ \text{`)'}\end{array}$$

As yet another abbreviation, element segments may also be specified inline with table definitions; see the respective section.

6.6.10 Start Function

A start function is defined in terms of its index.

$$\texttt{start}_I \;::=\; \text{`(' `start'}\ x{:}\texttt{funcidx}_I\ \text{`)'} \;\Rightarrow\; (\text{start}\ x, \{\})$$

Note

At most one start function may occur in a module, which is ensured by a suitable side condition on the `module` grammar.

6.6.11 Imports

The external type in imports can bind a symbolic tag, global, memory, or function identifier.

$$\texttt{import}_I \;::=\; \text{`(' `import'}\ nm_1{:}\texttt{name}\ nm_2{:}\texttt{name}\ (xt, I'){:}\texttt{externtype}_I\ \text{`)'} \;\Rightarrow\; (\text{import}\ nm_1\ nm_2\ xt, I')$$

Abbreviations

As an abbreviation, imports may also be specified inline with tag, global, memory, table, or function definitions; see the respective sections.

6.6.12 Exports

The syntax for exports mirrors their abstract syntax directly.

$$\begin{array}{rcll}\texttt{export}_I & ::= & \text{`(' `export'}\ nm{:}\texttt{name}\ xx{:}\texttt{externidx}_I\ \text{`)'} & \Rightarrow\ (\text{export}\ nm\ xx, \{\}) \\ \texttt{externidx}_I & ::= & \text{`(' `tag'}\ x{:}\texttt{tagidx}_I\ \text{`)'} & \Rightarrow\ \text{tag}\ x \\ & \mid & \text{`(' `global'}\ x{:}\texttt{globalidx}_I\ \text{`)'} & \Rightarrow\ \text{global}\ x \\ & \mid & \text{`(' `memory'}\ x{:}\texttt{memidx}_I\ \text{`)'} & \Rightarrow\ \text{memory}\ x \\ & \mid & \text{`(' `table'}\ x{:}\texttt{tableidx}_I\ \text{`)'} & \Rightarrow\ \text{table}\ x \\ & \mid & \text{`(' `func'}\ x{:}\texttt{funcidx}_I\ \text{`)'} & \Rightarrow\ \text{func}\ x\end{array}$$

Abbreviations

As an abbreviation, exports may also be specified inline with tag, global, memory, table, or function definitions; see the respective sections.

6.6.13 Modules

A module consists of a sequence of *declarations* that can occur in any order.

$$\mathit{decl} \;::=\; \mathit{type} \mid \mathit{import} \mid \mathit{tag} \mid \mathit{global} \mid \mathit{mem} \mid \mathit{table} \mid \mathit{func} \mid \mathit{data} \mid \mathit{elem} \mid \mathit{start} \mid \mathit{export}$$

All declarations and their respective bound identifiers scope over the entire module, including the text preceding them. A module itself may optionally bind an identifier that names the module. The name serves a documentary role only.

Note

Tools may include the module name in the name section of the binary format.

$$\begin{array}{rcl} \texttt{decl}_I & ::= & \texttt{type}_I \mid \texttt{import}_I \mid \texttt{tag}_I \mid \texttt{global}_I \mid \texttt{mem}_I \mid \texttt{table}_I \\ & \mid & \texttt{func}_I \mid \texttt{data}_I \mid \texttt{elem}_I \mid \texttt{start}_I \mid \texttt{export}_I \end{array}$$

$$\begin{array}{rcl} \texttt{module} & ::= & \texttt{`('}\ \texttt{`module'}\ \texttt{id}^?\ (\mathit{decl}, I)^*{:}\texttt{decl}_{I'}^*\ \texttt{`)'} \\ & & \Rightarrow \\ & & \quad \mathsf{module}\ \mathit{type}^*\ \mathit{import}^*\ \mathit{tag}^*\ \mathit{global}^*\ \mathit{mem}^*\ \mathit{table}^*\ \mathit{func}^*\ \mathit{data}^*\ \mathit{elem}^*\ \mathit{start}^?\ \mathit{export}^* \\ & & \qquad \text{if } I' = \bigoplus I^* \\ & & \qquad \wedge \vdash I' : \mathsf{ok} \\ & & \qquad \wedge\ \mathit{type}^* = \mathrm{types}(\mathit{decl}^*) \\ & & \qquad \wedge\ \mathit{import}^* = \mathrm{imports}(\mathit{decl}^*) \\ & & \qquad \wedge\ \mathit{tag}^* = \mathrm{tags}(\mathit{decl}^*) \\ & & \qquad \wedge\ \mathit{global}^* = \mathrm{globals}(\mathit{decl}^*) \\ & & \qquad \wedge\ \mathit{mem}^* = \mathrm{mems}(\mathit{decl}^*) \\ & & \qquad \wedge\ \mathit{table}^* = \mathrm{tables}(\mathit{decl}^*) \\ & & \qquad \wedge\ \mathit{func}^* = \mathrm{funcs}(\mathit{decl}^*) \\ & & \qquad \wedge\ \mathit{data}^* = \mathrm{datas}(\mathit{decl}^*) \\ & & \qquad \wedge\ \mathit{elem}^* = \mathrm{elems}(\mathit{decl}^*) \\ & & \qquad \wedge\ \mathit{start}^? = \mathrm{starts}(\mathit{decl}^*) \\ & & \qquad \wedge\ \mathit{export}^* = \mathrm{exports}(\mathit{decl}^*) \\ & & \qquad \wedge\ \mathrm{ordered}(\mathit{decl}^*) \end{array}$$

where $\mathrm{types}(\mathit{decl}^*)$, $\mathrm{imports}(\mathit{decl}^*)$, $\mathrm{tags}(\mathit{decl}^*)$, etc., extract the sequence of types, imports, tags, etc., contained in decl^*, respectively. The auxiliary predicate ordered checks that no imports occur after the first definition of a tag, global, memory, table, or function in a sequence of declarations:

$$\begin{array}{l} \mathrm{ordered}(\mathit{decl}^*) \quad = \quad \mathsf{true} \quad \text{if } \mathrm{imports}(\mathit{decl}^*) = \epsilon \\ \mathrm{ordered}(\mathit{decl}_1^*\ \mathit{import}\ \mathit{decl}_2^*) \quad = \\ \quad \mathrm{imports}(\mathit{decl}_1^*) = \epsilon \wedge \mathrm{tags}(\mathit{decl}_1^*) = \epsilon \wedge \mathrm{globals}(\mathit{decl}_1^*) = \epsilon \wedge \mathrm{mems}(\mathit{decl}_1^*) = \epsilon \wedge \mathrm{tables}(\mathit{decl}_1^*) = \epsilon \wedge \mathrm{funcs}(\mathit{decl}_1^*) = \epsilon \end{array}$$

Abbreviations

In a source file, the toplevel `'('` `'module'` ... `')'` surrounding the module body may be omitted.

$$\texttt{module} \quad ::= \quad \ldots \mid \texttt{decl}_I^* \quad \equiv \quad \texttt{`(`}\ \texttt{`module'}\ \texttt{decl}_I^*\ \texttt{`)'}$$

CHAPTER 7

Appendix

7.1 Embedding

A WebAssembly implementation will typically be *embedded* into a *host* environment. An *embedder* implements the connection between such a host environment and the WebAssembly semantics as defined in the main body of this specification. An embedder is expected to interact with the semantics in well-defined ways.

This section defines a suitable interface to the WebAssembly semantics in the form of entry points through which an embedder can access it. The interface is intended to be complete, in the sense that an embedder does not need to reference other functional parts of the WebAssembly specification directly.

Note

On the other hand, an embedder does not need to provide the host environment with access to all functionality defined in this interface. For example, an implementation may not support parsing of the text format.

7.1.1 Types

In the description of the embedder interface, syntactic classes from the abstract syntax and the runtime's abstract machine are used as names for variables that range over the possible objects from that class. Hence, these syntactic classes can also be interpreted as types.

For numeric parameters, notation like $i : u64$ is used to specify a symbolic name in addition to the respective value range.

7.1.2 Booleans

Interface operation that are predicates return Boolean values:

$$bool \quad ::= \quad false \mid true$$

7.1.3 Exceptions and Errors

Invoking an exported function may throw or propagate exceptions, expressed by an auxiliary syntactic class:

$$exception \quad ::= \quad \mathsf{exception}\ exnaddr$$

The exception address $exnaddr$ identifies the exception thrown.

Failure of an interface operation is also indicated by an auxiliary syntactic class:

$$\mathit{error} \quad ::= \quad \text{error}$$

In addition to the error conditions specified explicitly in this section, such as invalid arguments or exceptions and traps resulting from execution, implementations may also return errors when specific implementation limitations are reached.

Note

Errors are abstract and unspecific with this definition. Implementations can refine it to carry suitable classifications and diagnostic messages.

7.1.4 Pre- and Post-Conditions

Some operations state *pre-conditions* about their arguments or *post-conditions* about their results. It is the embedder's responsibility to meet the pre-conditions. If it does, the post conditions are guaranteed by the semantics.

In addition to pre- and post-conditions explicitly stated with each operation, the specification adopts the following conventions for runtime objects (store, $\mathit{moduleinst}$, addresses):

- Every runtime object passed as a parameter must be valid per an implicit pre-condition.
- Every runtime object returned as a result is valid per an implicit post-condition.

Note

As long as an embedder treats runtime objects as abstract and only creates and manipulates them through the interface defined here, all implicit pre-conditions are automatically met.

7.1.5 Store

$\text{store_init}() : \mathit{store}$

1. Return the empty store.

$$\text{store_init}() \quad = \quad \{\}$$

7.1.6 Modules

$\text{module_decode}(\mathit{byte}^*) : \mathit{module} \mid \mathit{error}$

1. If there exists a derivation for the byte sequence byte^* as a `module` according to the binary grammar for modules, yielding a module m, then return m.
2. Else, return error.

$$\begin{array}{lcll} \text{module_decode}(b^*) & = & m & (\text{if } \texttt{module} \stackrel{*}{\Longrightarrow} m{:}b^*) \\ \text{module_decode}(b^*) & = & \text{error} & (\text{otherwise}) \end{array}$$

$\text{module_parse}(\mathit{char}^*) : \mathit{module} \mid \mathit{error}$

1. If there exists a derivation for the source char^* as a `module` according to the text grammar for modules, yielding a module m, then return m.
2. Else, return error.

$$\begin{array}{lcll} \text{module_parse}(c^*) & = & m & (\text{if } \texttt{module} \stackrel{*}{\Longrightarrow} m{:}c^*) \\ \text{module_parse}(c^*) & = & \text{error} & (\text{otherwise}) \end{array}$$

$\text{module_validate}(module) : error^?$

1. If $module$ is valid, then return nothing.
2. Else, return error.

$$\begin{array}{lcll} \text{module_validate}(m) & = & \epsilon & (\text{if } \vdash m : externtype^* \to externtype'^*) \\ \text{module_validate}(m) & = & \mathsf{error} & (\text{otherwise}) \end{array}$$

$\text{module_instantiate}(store, module, externaddr^*) : (store, moduleinst \mid exception \mid error)$

1. Try instantiating $module$ in $store$ with external addresses $externaddr^*$ as imports:

a. If it succeeds with a module instance $moduleinst$, then let $result$ be $moduleinst$.

b. Else, let $result$ be error.

2. Return the new store paired with $result$.

$$\begin{array}{lcll} \text{module_instantiate}(S, m, ev^*) & = & (S', F.\mathsf{module}) & (\text{if } \text{instantiate}(S, m, ev^*) \hookrightarrow {}^*S'; F; \epsilon) \\ \text{module_instantiate}(S, m, ev^*) & = & (S', \mathsf{error}) & (\text{otherwise, if } \text{instantiate}(S, m, ev^*) \hookrightarrow {}^*S'; F; result) \end{array}$$

Note

The store may be modified even in case of an error.

$\text{module_imports}(module) : (name, name, externtype)^*$

1. Pre-condition: $module$ is valid with the external import types $externtype^*$ and external export types $externtype'^*$.
2. Let $import^*$ be the imports of $module$.
3. Assert: the length of $import^*$ equals the length of $externtype^*$.
4. For each $import_i$ in $import^*$ and corresponding $externtype_i$ in $externtype^*$, do:

a. Let $\mathsf{import}\ nm_{i1}\ nm_{i2}\ xt_i$ be the deconstruction of $import_i$.

b. Let $result_i$ be the triple $(nm_{i1}, nm_{i2}, externtype_i)$.

5. Return the concatenation of all $result_i$, in index order.
6. Post-condition: each $externtype_i$ is valid under the empty context.

$$\begin{array}{lcl} \text{module_imports}(m) & = & (nm_1, nm_2, externtype)^* \\ & & \quad (\text{if } (\mathsf{import}\ nm_1\ nm_2\ xt^*)^* \in m \wedge \vdash m : externtype^* \to externtype'^*) \end{array}$$

$\text{module_exports}(module) : (name, externtype)^*$

1. Pre-condition: $module$ is valid with the external import types $externtype^*$ and external export types $externtype'^*$.
2. Let $export^*$ be the exports of $module$.
3. Assert: the length of $export^*$ equals the length of $externtype'^*$.
4. For each $export_i$ in $export^*$ and corresponding $externtype'_i$ in $externtype'^*$, do:

a. Let $\mathsf{export}\ nm_i\ externidx_i$ be the deconstruction of $export_i$.

b. Let $result_i$ be the pair $(nm_i, externtype'_i)$.

5. Return the concatenation of all $result_i$, in index order.

6. Post-condition: each $externtype'_i$ is valid under the empty context.

$$\begin{array}{lcl}\text{module_exports}(m) & = & (\\ mathitnm, externtype')^* & & \\ & & (\text{if } (\text{export } nm\ xt^*)^* \in m \wedge \vdash m : externtype^* \to externtype'^*)\end{array}$$

7.1.7 Module Instances

$\text{instance_export}(moduleinst, name) : externaddr \mid error$

1. Assert: due to validity of the module instance $moduleinst$, all its export names are different.
2. If there exists an $exportinst_i$ in $moduleinst.\text{exports}$ such that name $exportinst_i.\text{name}$ equals $name$, then:
   a. Return the external address $exportinst_i.\text{addr}$.
3. Else, return error.

$$\begin{array}{lcll}\text{instance_export}(m, name) & = & m.\text{exports}[i].\text{addr} & (\text{if } m.\text{exports}[i].\text{name} = name)\\ \text{instance_export}(m, name) & = & \text{error} & (\text{otherwise})\end{array}$$

7.1.8 Functions

$\text{func_alloc}(store, deftype, hostfunc) : (store, funcaddr)$

1. Pre-condition: the defined type $deftype$ is valid under the empty context and expands to a function type.
2. Let $funcaddr$ be the result of allocating a host function in $store$ with defined type $deftype$, host function code $hostfunc$ and an empty module instance.
3. Return the new store paired with $funcaddr$.

$$\text{func_alloc}(S, dt, code) = (S', a) \quad (\text{if } \text{allocfunc}(S, dt, code, \{\}) = S', a)$$

Note

This operation assumes that $hostfunc$ satisfies the pre- and post-conditions required for a function instance with type $deftype$.

Regular (non-host) function instances can only be created indirectly through module instantiation.

$\text{func_type}(store, funcaddr) : deftype$

1. Let $deftype$ be the definedn type $S.\text{funcs}[a].\text{type}$.
2. Return $deftype$.
3. Post-condition: the returned defined type is valid and expands to a function type.

$$\text{func_type}(S, a) = S.\text{funcs}[a].\text{type}$$

$\text{func_invoke}(store, funcaddr, val^*) : (store, val^* \mid exception \mid error)$

1. Try invoking the function $funcaddr$ in $store$ with values val^* as arguments:

a. If it succeeds with values val'^* as results, then let $result$ be val'^*.

b. Else if the outcome is an exception with a thrown exception $\text{ref.exn } exnaddr$ as the result, then let $result$ be $\text{exception } exnaddr$

c. Else it has trapped, hence let $result$ be error.

2. Return the new store paired with $result$.

$$\begin{array}{lcll} \text{func_invoke}(S, a, v^*) & = & (S', v'^*) & (\text{if } \text{invoke}(S, a, v^*) \hookrightarrow {}^*S'; F; v'^*) \\ \text{func_invoke}(S, a, v^*) & = & (S', \text{exception } a') & (\text{if } \text{invoke}(S, a, v^*) \hookrightarrow {}^*S'; F; (\text{ref.exn } a')\ \text{throw_ref} \\ \text{func_invoke}(S, a, v^*) & = & (S', \text{error}) & (\text{if } \text{invoke}(S, a, v^*) \hookrightarrow {}^*S'; F; \text{trap}) \end{array}$$

Note

The store may be modified even in case of an error.

7.1.9 Tables

$\text{table_alloc}(store, tabletype, ref) : (store, tableaddr)$

1. Pre-condition: the $tabletype$ is valid under the empty context.
2. Let $tableaddr$ be the result of allocating a table in $store$ with table type $tabletype$ and initialization value ref.
3. Return the new store paired with $tableaddr$.

$$\text{table_alloc}(S, tt, r) = (S', a) \quad (\text{if } \text{alloctable}(S, tt, r) = S', a)$$

$\text{table_type}(store, tableaddr) : tabletype$

1. Return $S.\text{tables}[a].\text{type}$.
2. Post-condition: the returned table type is valid under the empty context.

$$\text{table_type}(S, a) = S.\text{tables}[a].\text{type}$$

$\text{table_read}(store, tableaddr, i : u64) : ref \mid error$

1. Let ti be the table instance $store.\text{tables}[tableaddr]$.
2. If i is larger than or equal to the length of $ti.\text{refs}$, then return error.
3. Else, return the reference value $ti.\text{refs}[i]$.

$$\begin{array}{lcll} \text{table_read}(S, a, i) & = & r & (\text{if } S.\text{tables}[a].\text{refs}[i] = r) \\ \text{table_read}(S, a, i) & = & \text{error} & (\text{otherwise}) \end{array}$$

$\text{table_write}(store, tableaddr, i : u64, ref) : store \mid error$

1. Let ti be the table instance $store.\text{tables}[tableaddr]$.
2. If i is larger than or equal to the length of $ti.\text{refs}$, then return error.
3. Replace $ti.\text{refs}[i]$ with the reference value ref.
4. Return the updated store.

$$\begin{array}{lcll} \text{table_write}(S, a, i, r) & = & S' & (\text{if } S' = S \text{ with } \text{tables}[a].\text{refs}[i] = r) \\ \text{table_write}(S, a, i, r) & = & \text{error} & (\text{otherwise}) \end{array}$$

$\text{table_size}(store, tableaddr) : u64$

1. Return the length of $store.\text{tables}[tableaddr].\text{refs}$.

$$\text{table_size}(S, a) = n \quad (\text{if } |S.\text{tables}[a].\text{refs}| = n)$$

table_grow$(store, tableaddr, n : u64, ref) : store \mid error$

1. Try growing the table instance $store.\mathsf{tables}[tableaddr]$ by n elements with initialization value ref:
   a. If it succeeds, return the updated store.
   b. Else, return error.

$$\begin{array}{lcll} \text{table_grow}(S, a, n, r) & = & S' & (\text{if } S' = S \text{ with } \mathsf{tables}[a] = \text{growtable}(S.\mathsf{tables}[a], n, r)) \\ \text{table_grow}(S, a, n, r) & = & \mathsf{error} & (\text{otherwise}) \end{array}$$

7.1.10 Memories

mem_alloc$(store, memtype) : (store, memaddr)$

1. Pre-condition: the $memtype$ is valid under the empty context.
2. Let $memaddr$ be the result of allocating a memory in $store$ with memory type $memtype$.
3. Return the new store paired with $memaddr$.

$$\text{mem_alloc}(S, mt) = (S', a) \quad (\text{if } \text{allocmem}(S, mt) = S', a)$$

mem_type$(store, memaddr) : memtype$

1. Return $S.\mathsf{mems}[a].\mathsf{type}$.
2. Post-condition: the returned memory type is valid under the empty context.

$$\text{mem_type}(S, a) = S.\mathsf{mems}[a].\mathsf{type}$$

mem_read$(store, memaddr, i : u64) : byte \mid error$

1. Let mi be the memory instance $store.\mathsf{mems}[memaddr]$.
2. If i is larger than or equal to the length of $mi.\mathsf{bytes}$, then return error.
3. Else, return the byte $mi.\mathsf{bytes}[i]$.

$$\begin{array}{lcll} \text{mem_read}(S, a, i) & = & b & (\text{if } S.\mathsf{mems}[a].\mathsf{bytes}[i] = b) \\ \text{mem_read}(S, a, i) & = & \mathsf{error} & (\text{otherwise}) \end{array}$$

mem_write$(store, memaddr, i : u64, byte) : store \mid error$

1. Let mi be the memory instance $store.\mathsf{mems}[memaddr]$.
2. If i is larger than or equal to the length of $mi.\mathsf{bytes}$, then return error.
3. Replace $mi.\mathsf{bytes}[i]$ with $byte$.
4. Return the updated store.

$$\begin{array}{lcll} \text{mem_write}(S, a, i, b) & = & S' & (\text{if } S' = S \text{ with } \mathsf{mems}[a].\mathsf{bytes}[i] = b) \\ \text{mem_write}(S, a, i, b) & = & \mathsf{error} & (\text{otherwise}) \end{array}$$

mem_size$(store, memaddr) : u64$

1. Return the length of $store.\mathsf{mems}[memaddr].\mathsf{bytes}$ divided by the page size.

$$\text{mem_size}(S, a) = n \quad (\text{if } |S.\mathsf{mems}[a].\mathsf{bytes}| = n \cdot 64\,\text{Ki})$$

$\text{mem_grow}(\mathit{store}, \mathit{memaddr}, n : \mathit{u64}) : \mathit{store} \mid \mathit{error}$

1. Try growing the memory instance $\mathit{store}.\mathsf{mems}[\mathit{memaddr}]$ by n pages:
   a. If it succeeds, return the updated store.
   b. Else, return error.

$$\begin{array}{lcll} \text{mem_grow}(S, a, n) & = & S' & (\text{if } S' = S \text{ with } \mathsf{mems}[a] = \text{growmem}(S.\mathsf{mems}[a], n)) \\ \text{mem_grow}(S, a, n) & = & \mathsf{error} & (\text{otherwise}) \end{array}$$

7.1.11 Tags

$\text{tag_alloc}(\mathit{store}, \mathit{tagtype}) : (\mathit{store}, \mathit{tagaddr})$

1. Pre-condition: $\mathit{tagtype}$ is valid.
2. Let $\mathit{tagaddr}$ be the result of allocating a tag in store with tag type $\mathit{tagtype}$.
3. Return the new store paired with $\mathit{tagaddr}$.

$$\text{tag_alloc}(S, tt) \quad = \quad (S', a) \qquad (\text{if } \text{alloctag}(S, tt) = S', a)$$

$\text{tag_type}(\mathit{store}, \mathit{tagaddr}) : \mathit{tagtype}$

1. Return $S.\mathsf{tags}[a].\mathsf{type}$.
2. Post-condition: the returned tag type is valid.

$$\text{tag_type}(S, a) \quad = \quad S.\mathsf{tags}[a].\mathsf{type}$$

7.1.12 Exceptions

$\text{exn_alloc}(\mathit{store}, \mathit{tagaddr}, \mathit{val}^*) : (\mathit{store}, \mathit{exnaddr})$

1. Pre-condition: $\mathit{tagaddr}$ is an allocated tag address.
2. Let $\mathit{exnaddr}$ be the result of allocating an exception instance in store with tag address $\mathit{tagaddr}$ and initialization values val^*.
3. Return the new store paired with $\mathit{exnaddr}$.

$$\text{exn_alloc}(S, \mathit{tagaddr}, \mathit{val}^*) \quad = \quad (S \oplus \{\mathsf{exns}\ \mathit{exninst}\}, |S.\mathsf{exns}|) \quad (\text{if } \mathit{exninst} = \{\mathsf{tag}\ \mathit{tagaddr}, \mathsf{fields}\ \mathit{val}^*\}$$

$\text{exn_tag}(\mathit{store}, \mathit{exnaddr}) : \mathit{tagaddr}$

1. Let $\mathit{exninst}$ be the exception instance $\mathit{store}.\mathsf{exns}[\mathit{exnaddr}]$.
2. Return the tag address $\mathit{exninst}.\mathsf{tag}$.

$$\text{exn_tag}(S, a) \quad = \quad \mathit{exninst}.\mathsf{tag} \quad (\text{if } \mathit{exninst} = S.\mathsf{exns}[a])$$

$\text{exn_read}(\mathit{store}, \mathit{exnaddr}) : \mathit{val}^*$

1. Let $\mathit{exninst}$ be the exception instance $\mathit{store}.\mathsf{exns}[\mathit{exnaddr}]$.
2. Return the values $\mathit{exninst}.\mathsf{fields}$.

$$\text{exn_read}(S, a) \quad = \quad \mathit{exninst}.\mathsf{fields} \quad (\text{if } \mathit{exninst} = S.\mathsf{exns}[a])$$

7.1.13 Globals

$\text{global_alloc}(\mathit{store}, \mathit{globaltype}, \mathit{val}) : (\mathit{store}, \mathit{globaladdr})$

1. Pre-condition: the $\mathit{globaltype}$ is valid under the empty context.
2. Let $\mathit{globaladdr}$ be the result of allocating a global in store with global type $\mathit{globaltype}$ and initialization value val.
3. Return the new store paired with $\mathit{globaladdr}$.

$$\text{global_alloc}(S, gt, v) \quad = \quad (S', a) \qquad (\text{if } \text{allocglobal}(S, gt, v) = S', a)$$

$\text{global_type}(\mathit{store}, \mathit{globaladdr}) : \mathit{globaltype}$

1. Return $S.\text{globals}[a].\text{type}$.
2. Post-condition: the returned global type is valid under the empty context.

$$\text{global_type}(S, a) \quad = \quad S.\text{globals}[a].\text{type}$$

$\text{global_read}(\mathit{store}, \mathit{globaladdr}) : \mathit{val}$

1. Let gi be the global instance $\mathit{store}.\text{globals}[\mathit{globaladdr}]$.
2. Return the value $gi.\text{value}$.

$$\text{global_read}(S, a) \quad = \quad v \qquad (\text{if } S.\text{globals}[a].\text{value} = v)$$

$\text{global_write}(\mathit{store}, \mathit{globaladdr}, \mathit{val}) : \mathit{store} \mid \mathit{error}$

1. Let gi be the global instance $\mathit{store}.\text{globals}[\mathit{globaladdr}]$.
2. Let $\mathit{mut}\ t$ be the structure of the global type $gi.\text{type}$.
3. If mut is empty, then return error.
4. Replace $gi.\text{value}$ with the value val.
5. Return the updated store.

$$\begin{array}{lcll} \text{global_write}(S, a, v) & = & S' & (\text{if } S.\text{globals}[a].\text{type} = \text{mut}\ t \wedge S' = S \text{ with globals}[a].\text{value} = v) \\ \text{global_write}(S, a, v) & = & \text{error} & (\text{otherwise}) \end{array}$$

7.1.14 Values

$\text{ref_type}(\mathit{store}, \mathit{ref}) : \mathit{reftype}$

1. Pre-condition: the reference ref is valid under store S.
2. Return the reference type t with which ref is valid.
3. Post-condition: the returned reference type is valid under the empty context.

$$\text{ref_type}(S, r) \quad = \quad t \qquad (\text{if } S \vdash r : t)$$

Note

In future versions of WebAssembly, not all references may carry precise type information at run time. In such cases, this function may return a less precise supertype.

$\text{val_default}(\mathit{valtype}) : \mathit{val}$

1. If $\text{default}_{\mathit{valtype}}$ is not defined, then return error.
1. Else, return the value $\text{default}_{\mathit{valtype}}$.

$$\begin{array}{lcll} \text{val_default}(t) & = & v & (\text{if } \text{default}_t = v) \\ \text{val_default}(t) & = & \text{error} & (\text{if } \text{default}_t = \epsilon) \end{array}$$

7.1.15 Matching

$\text{match_valtype}(\mathit{valtype}_1, \mathit{valtype}_2) : \mathit{bool}$

1. Pre-condition: the value types $\mathit{valtype}_1$ and $\mathit{valtype}_2$ are valid under the empty context.
2. If $\mathit{valtype}_1$ matches $\mathit{valtype}_2$, then return true.
3. Else, return false.

$$\begin{array}{lcll} \text{match_reftype}(t_1, t_2) & = & \mathit{true} & (\text{if } \vdash t_1 \leq t_2) \\ \text{match_reftype}(t_1, t_2) & = & \mathit{false} & (\text{otherwise}) \end{array}$$

$\text{match_externtype}(\mathit{externtype}_1, \mathit{externtype}_2) : \mathit{bool}$

1. Pre-condition: the extern types $\mathit{externtype}_1$ and $\mathit{externtype}_2$ are valid under the empty context.
2. If $\mathit{externtype}_1$ matches $\mathit{externtype}_2$, then return true.
3. Else, return false.

$$\begin{array}{lcll} \text{match_externtype}(et_1, et_2) & = & \mathit{true} & (\text{if } \vdash et_1 \leq et_2) \\ \text{match_externtype}(et_1, et_2) & = & \mathit{false} & (\text{otherwise}) \end{array}$$

7.2 Profiles

To enable the use of WebAssembly in as many environments as possible, *profiles* specify coherent language subsets that fit constraints imposed by common classes of host environments. A host platform can thereby decide to support the language only under a restricted profile, or even the intersection of multiple profiles.

7.2.1 Conventions

A profile modification is specified by decorating selected rules in the main body of this specification with a *profile annotation* that defines them as conditional on the choice of profile.

For that purpose, every profile defines a *profile marker*, an alphanumeric short-hand like ABC. A profile annotation of the form $^{[!\text{ABC XYZ}]}$ on a rule indicates that this rule is *excluded* for either of the profiles whose marker is ABC or XYZ.

There are two ways of subsetting the language in a profile:

- *Syntactic*, by *omitting* a feature, in which case certain constructs are removed from the syntax altogether.
- *Semantic*, by *restricting* a feature, in which case certain constructs are still present but some behaviours are ruled out.

Syntax Annotations

To omit a construct from a profile syntactically, respective productions in the grammar of the abstract syntax are annotated with an associated profile marker. This is defined to have the following implications:

1. Any production in the binary or textual syntax that produces abstract syntax with a marked construct is omitted by extension.
2. Any validation or execution rule that handles a marked construct is omitted by extension.

The overall effect is that the respective construct is no longer part of the language under a respective profile.

Note

For example, a "busy" profile marked BUSY could rule out the nop instruction by marking the production for it in the abstract syntax as follows:

$$\begin{array}{lll} \textit{instr} & ::= & \dots \\ {\scriptstyle[!\text{BUSY}]} & | & \mathsf{nop} \\ & | & \mathsf{unreachable} \end{array}$$

A rule may be annotated by multiple markers, which could be the case if a construct is in the intersection of multiple features.

Semantics Annotations

To restrict certain behaviours in a profile, individual validation or reduction rules or auxiliary definitions are annotated with an associated marker.

This has the consequence that the respective rule is no longer applicable under the given profile.

Note

For example, an "infinite" profile marked INF could define that growing memory never fails:

$$\begin{array}{lll} & S; F; (\mathsf{i32.const}\ n)\ \mathsf{memory.grow}\ x & \hookrightarrow \quad S'; F; (\mathsf{i32.const}\ sz) \\ & & \quad (\text{if } F.\mathsf{module.mems}[x] = a \\ & & \quad \wedge\ sz = |S.\mathsf{mems}[a].\mathsf{datas}|/64\,\text{Ki} \\ & & \quad \wedge\ S' = S\ \mathsf{with}\ \mathsf{mems}[a] = \mathrm{growmem}(S.\mathsf{mems}[a], n)) \\ {\scriptstyle[!\text{INF}]} & S; F; (\mathsf{i32.const}\ n)\ \mathsf{memory.grow}\ x & \hookrightarrow \quad S; F; (\mathsf{i32.const}\ \mathrm{signed}_{32}^{-1}(-1)) \end{array}$$

Properties

All profiles are defined such that the following properties are preserved:

- All profiles represent syntactic and semantic subsets of the full profile, i.e., they do not *add* syntax or *alter* behaviour.
- All profiles are mutually compatible, i.e., no two profiles subset semantic behaviour in inconsistent or ambiguous ways, and any intersection of profiles preserves the properties described here.
- Profiles do not violate soundness, i.e., all configurations valid under that profile still have well-defined execution behaviour.

Note

Tools are generally expected to handle and produce code for the full profile by default. In particular, producers should not generate code that *depends* on specific profiles. Instead, all code should preserve correctness when executed under the full profile.

Moreover, profiles should be considered static and fixed for a given platform or ecosystem. Runtime conditioning on the "current" profile is not intended and should be avoided.

7.2.2 Defined Profiles

Note

The number of defined profiles is expected to remain small in the future. Profiles are intended for broad and permanent use cases only. In particular, profiles are not intended for language versioning.

Full Profile (FUL)

The *full* profile contains the complete language and all possible behaviours. It imposes no restrictions, i.e., all rules and definitions are active. All other profiles define sub-languages of this profile.

Deterministic Profile (DET)

The *deterministic* profile excludes all rules marked $^{[!\text{DET}]}$. It defines a sub-language that does not exhibit any incidental non-deterministic behaviour:

- All NaN values generated by floating-point instructions are canonical and positive.
- All relaxed vector instructions have a fixed behaviour that does not depend on the implementation.

Even under this profile, the memory.grow and table.grow instructions technically remain non-deterministic, in order to be able to indicate resource exhaustion.

Note

In future versions of WebAssembly, new non-deterministic behaviour may be added to the language, such that the deterministic profile will induce additional restrictions.

7.3 Implementation Limitations

Implementations typically impose additional restrictions on a number of aspects of a WebAssembly module or execution. These may stem from:

- physical resource limits,
- constraints imposed by the embedder or its environment,
- limitations of selected implementation strategies.

This section lists allowed limitations. Where restrictions take the form of numeric limits, no minimum requirements are given, nor are the limits assumed to be concrete, fixed numbers. However, it is expected that all implementations have "reasonably" large limits to enable common applications.

Note

A conforming implementation is not allowed to leave out individual *features*. However, designated subsets of WebAssembly may be specified in the future.

7.3.1 Syntactic Limits

Structure

An implementation may impose restrictions on the following dimensions of a module:

- the number of types in a module
- the number of functions in a module, including imports
- the number of tables in a module, including imports
- the number of memories in a module, including imports

- the number of globals in a module, including imports
- the number of tags in a module, including imports
- the number of element segments in a module
- the number of data segments in a module
- the number of imports to a module
- the number of exports from a module
- the number of sub types in a recursive type
- the subtyping depth of a sub type
- the number of fields in a structure type
- the number of parameters in a function type
- the number of results in a function type
- the number of parameters in a block type
- the number of results in a block type
- the number of locals in a function
- the number of instructions in a function body
- the number of instructions in a structured control instruction
- the number of structured control instructions in a function
- the nesting depth of structured control instructions
- the number of label indices in a `br_table` instruction
- the number of instructions in a constant expression
- the number of instructions between a branch instruction and its target
- the nesting depth of blocks between a branch instruction and its target
- the length of the array in a `array.new_fixed` instruction
- the length of an element segment
- the length of a data segment
- the length of a name
- the range of characters in a name

If the limits of an implementation are exceeded for a given module, then the implementation may reject the validation, compilation, or instantiation of that module with an implementation-specific error.

Note

The last item allows embedders that operate in limited environments without support for Unicode[48] to limit the names of imports and exports to common subsets like ASCII[49].

Binary Format

For a module given in binary format, additional limitations may be imposed on the following dimensions:

- the size of a module
- the size of any section

[48] https://www.unicode.org/versions/latest/
[49] https://webstore.ansi.org/RecordDetail.aspx?sku=INCITS+4-1986%5bR2012%5d

- the size of an individual function's code
- the size of a structured control instruction
- the size of an individual constant expression's instruction sequence
- the number of sections

Text Format

For a module given in text format, additional limitations may be imposed on the following dimensions:

- the size of the source text
- the size of any syntactic element
- the size of an individual token
- the nesting depth of folded instructions
- the length of symbolic identifiers
- the range of literal characters allowed in the source text

7.3.2 Validation

An implementation may defer validation of individual functions until they are first invoked.

If a function turns out to be invalid, then the invocation, and every consecutive call to the same function, results in a trap.

Note

This is to allow implementations to use interpretation or just-in-time compilation for functions. The function must still be fully validated before execution of its body begins.

7.3.3 Execution

Restrictions on the following dimensions may be imposed during execution of a WebAssembly program:

- the number of allocated module instances
- the number of allocated function instances
- the number of allocated table instances
- the number of allocated memory instances
- the number of allocated global instances
- the number of allocated tag instances
- the number of allocated structure instances
- the number of allocated array instances
- the number of allocated exception instances
- the size of a table instance
- the size of a memory instance
- the size of an array instance
- the number of frames on the stack
- the number of labels on the stack
- the number of values on the stack

- the number of handlers on the stack
- the number of labels dropped from the stack by a branch instruction
- the number of values dropped from the stack by a branch instruction

If the runtime limits of an implementation are exceeded during execution of a computation, then it may terminate that computation and report an implementation-specific error to the invoking code.

Some of the above limits may already be verified during validation or instantiation, in which case an implementation may report exceedance in the same manner as for syntactic limits.

Note

Concrete limits are usually not fixed but may be dependent on specifics, interdependent, vary over time, or depend on other implementation- or embedder-specific situations or events.

7.4 Type Soundness

The type system of WebAssembly is *sound*, implying both *type safety* and *memory safety* with respect to the WebAssembly semantics. For example:

- All types declared and derived during validation are respected at run time; e.g., every local or global variable will only contain type-correct values, every instruction will only be applied to operands of the expected type, and every function invocation always evaluates to a result of the right type (if it does not diverge, throw an exception, or trap).
- No memory location will be read or written except those explicitly defined by the program, i.e., as a local, a global, an element in a table, or a location within a linear memory.
- There is no undefined behavior, i.e., the execution rules cover all possible cases that can occur in a valid program, and the rules are mutually consistent.

Soundness also is instrumental in ensuring additional properties, most notably, *encapsulation* of function and module scopes: no locals can be accessed outside their own function and no module components can be accessed outside their own module unless they are explicitly exported or imported.

The typing rules defining WebAssembly validation only cover the *static* components of a WebAssembly program. In order to state and prove soundness precisely, the typing rules must be extended to the *dynamic* components of the abstract runtime, that is, the store, configurations, and administrative instructions.[50]

7.4.1 Contexts

In order to check rolled up recursive types, the context is locally extended with an additional component that records the sub type corresponding to each recursive type index within the current group of recursive types:

$$context \quad ::= \quad \{\dots, \mathsf{recs}\ subtype^*\}$$

7.4.2 Types

Well-formedness for extended type forms is defined as follows.

The type use $(\mathsf{rec}.i)$ is valid if:

- The recursive type $C.\mathsf{recs}[i]$ exists.

[50] The formalization and theorems are derived from the following article: Andreas Haas, Andreas Rossberg, Derek Schuff, Ben Titzer, Dan Gohman, Luke Wagner, Alon Zakai, JF Bastien, Michael Holman. Bringing the Web up to Speed with WebAssembly[51]. Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017). ACM 2017.

[51] https://dl.acm.org/citation.cfm?doid=3062341.3062363

$$\frac{C.\mathsf{recs}[i] = st}{C \vdash \mathsf{rec}.i : \mathsf{ok}}$$

The heap type bot is always valid.

$$\frac{}{C \vdash \mathsf{bot} : \mathsf{ok}}$$

The value type bot is always valid.

$$\frac{}{C \vdash \mathsf{bot} : \mathsf{ok}}$$

The recursive type $(\mathsf{rec}\ subtype^*)$ is valid for i if:

- Either:
  - The sub type sequence $subtype^*$ is empty.
- Or:
  - The sub type sequence $subtype^*$ is of the form $subtype_1\ subtype'^*$.
  - The sub type $subtype_1$ is valid for i.
  - The recursive type $(\mathsf{rec}\ subtype'^*)$ is valid for $i+1$.

$$\frac{}{C \vdash \mathsf{rec}\ \epsilon : \mathsf{ok}(i)} \qquad \frac{C \vdash subtype_1 : \mathsf{ok}(i) \qquad C \vdash \mathsf{rec}\ subtype^* : \mathsf{ok}(i+1)}{C \vdash \mathsf{rec}\ (subtype_1\ subtype^*) : \mathsf{ok}(i)}$$

The sub type $(\mathsf{sub}\ \mathsf{final}^?\ typeuse^*\ comptype)$ is valid for i if:

- The length of $typeuse^*$ is less than or equal to 1.
- For all $typeuse$ in $typeuse^*$:
  - The type use $typeuse$ is valid.
  - $typeuse \prec i$ is true.
  - The sub type $\mathrm{unroll}_C(typeuse)$ is of the form $(\mathsf{sub}\ typeuse'^*\ comptype')$.
- $comptype'^*$ is the concatenation of all such $comptype'$.
- The composite type $comptype$ is valid.
- For all $comptype'$ in $comptype'^*$:
  - The composite type $comptype$ matches the composite type $comptype'$.

$$\frac{\begin{array}{c}|typeuse^*| \leq 1 \qquad (C \vdash typeuse : \mathsf{ok})^* \qquad (typeuse \prec i)^* \\ (\mathrm{unroll}_C(typeuse) = \mathsf{sub}\ typeuse'^*\ comptype')^* \\ C \vdash comptype : \mathsf{ok} \qquad (C \vdash comptype \leq comptype')^*\end{array}}{C \vdash \mathsf{sub}\ \mathsf{final}^?\ typeuse^*\ comptype : \mathsf{ok}(i)}$$

where:

$$\begin{array}{lcll}\mathrm{unroll}_C(deftype) & = & \mathrm{unroll}(deftype) & \\ \mathrm{unroll}_C(typeidx) & = & \mathrm{unroll}(C.\mathsf{types}[typeidx]) & \\ \mathrm{unroll}_C(\mathsf{rec}.i) & = & C.\mathsf{recs}[i] & \\ \mathsf{rec}.j \prec i & = & j < i & \\ typeuse \prec i & = & \mathsf{true} & \text{otherwise}\end{array}$$

Note

The new rules for recursive types and sub types complement the ones previously given, which only allowed regular type indices as supertypes. They define validity of rolled-up recursive types, like they occur in defined types, in turn needed to define validity of contexts. None of these rules are needed in the implementation of a validator.

The defined type $(\mathit{rectype}.i)$ is valid if:

- Let C' be the same context as C, but with the sub type sequence $\mathit{subtype}^n$ prepended to the field recs.
- Under the context C', the recursive type $\mathit{rectype}$ is valid for 0.
- The recursive type $\mathit{rectype}$ is of the form $(\mathsf{rec}\ \mathit{subtype}^n)$.
- i is less than n.

$$\frac{C, \mathsf{recs}\ \mathit{subtype}^n \vdash \mathit{rectype} : \mathsf{ok}(0) \qquad \mathit{rectype} = \mathsf{rec}\ \mathit{subtype}^n \qquad i < n}{C \vdash \mathit{rectype}.i : \mathsf{ok}}$$

7.4.3 Subtyping

Inside a rolled-up recursive type, a recursive type index can match another heap type.

$$\frac{C.\mathsf{recs}[i] = \mathsf{sub}\ \mathsf{final}^?\ (\mathsf{struct}\ \mathit{fieldtype}^*)}{C \vdash \mathsf{rec}.i \leq \mathsf{struct}}$$

$$\frac{C.\mathsf{recs}[i] = \mathsf{sub}\ \mathsf{final}^?\ (\mathsf{array}\ \mathit{fieldtype})}{C \vdash \mathsf{rec}.i \leq \mathsf{array}}$$

$$\frac{C.\mathsf{recs}[i] = \mathsf{sub}\ \mathsf{final}^?\ (\mathsf{func}\ t_1^* \to t_2^*)}{C \vdash \mathsf{rec}.i \leq \mathsf{func}}$$

$$\frac{C.\mathsf{recs}[i] = \mathsf{sub}\ \mathsf{final}^?\ \mathit{typeuse}^*\ \mathit{ct}}{C \vdash \mathsf{rec}.i \leq \mathit{typeuse}^*[j]}$$

Note

These rules complement the previously given rules for matching heap types. They are only invoked when checking validity of rolled-up recursive types.

7.4.4 Results

Results can be classified by result types as follows.

Results val^*

- For each value val_i in val^*:
  - The value val_i is valid with some value type t_i.
- Let t^* be the concatenation of all t_i.
- Then the result is valid with result type $[t^*]$.

$$\frac{(S \vdash \mathit{val} : t)^*}{S \vdash \mathit{val}^* : [t^*]}$$

Results $(\mathsf{ref.exn}\ a)\ \mathsf{throw_ref}$

- The value ref.exn a must be valid.
- Then the result is valid with result type $[t^*]$, for any valid closed result types.

$$\frac{S \vdash \mathsf{ref.exn}\ a : \mathsf{ref\ exn} \qquad \vdash [t^*] : \mathsf{ok}}{S \vdash (\mathsf{ref.exn}\ a)\ \mathsf{throw_ref} : [t^*]}$$

Results trap

- The result is valid with result type $[t^*]$, for any valid closed result types.

$$\frac{\vdash [t^*] : \mathsf{ok}}{S \vdash \mathsf{trap} : [t^*]}$$

7.4.5 Store Validity

The following typing rules specify when a runtime store S is *valid*. A valid store must consist of tag, global, memory, table, function, data, element, structure, array, exception, and module instances that are themselves valid, relative to S.

To that end, each kind of instance is classified by a respective tag, global, memory, table, function, or element, type, or just ok in the case of data structures, arrays, or exceptions. Module instances are classified by *module contexts*, which are regular contexts repurposed as module types describing the index spaces defined by a module.

Store S

- Each tag instance $taginst_i$ in $S.\mathsf{tags}$ must be valid with some tag type $tagtype_i$.
- Each global instance $globalinst_i$ in $S.\mathsf{globals}$ must be valid with some global type $globaltype_i$.
- Each memory instance $meminst_i$ in $S.\mathsf{mems}$ must be valid with some memory type $memtype_i$.
- Each table instance $tableinst_i$ in $S.\mathsf{tables}$ must be valid with some table type $tabletype_i$.
- Each function instance $funcinst_i$ in $S.\mathsf{funcs}$ must be valid with some defined type $deftype_i$.
- Each data instance $datainst_i$ in $S.\mathsf{datas}$ must be valid.
- Each element instance $eleminst_i$ in $S.\mathsf{elems}$ must be valid with some reference type $reftype_i$.
- Each structure instance $structinst_i$ in $S.\mathsf{structs}$ must be valid.
- Each array instance $arrayinst_i$ in $S.\mathsf{arrays}$ must be valid.
- Each exception instance $exninst_i$ in $S.\mathsf{exns}$ must be valid.
- No reference to a bound structure address must be reachable from itself through a path consisting only of indirections through immutable structure, or array fields or fields of exception instances.
- No reference to a bound array address must be reachable from itself through a path consisting only of indirections through immutable structure or array fields or fields of exception instances.
- No reference to a bound exception address must be reachable from itself through a path consisting only of indirections through immutable structure or array fields or fields of exception instances.
- Then the store is valid.

$$\frac{\begin{array}{c}(S \vdash taginst : tagtype)^* \quad (S \vdash globalinst : globaltype)^* \\ (S \vdash meminst : memtype)^* \quad (S \vdash tableinst : tabletype)^* \\ (S \vdash funcinst : deftype)^* \quad (S \vdash datainst : \mathsf{ok})^* \quad (S \vdash eleminst : reftype)^* \\ (S \vdash structinst : \mathsf{ok})^* \quad (S \vdash arrayinst : \mathsf{ok})^* \quad (S \vdash exninst : \mathsf{ok})^* \\ S = \{\mathsf{tags}\ taginst^*, \mathsf{globals}\ globalinst^*, \mathsf{mems}\ meminst^*, \mathsf{tables}\ tableinst^*, \mathsf{funcs}\ funcinst^*, \\ \mathsf{datas}\ datainst^*, \mathsf{elems}\ eleminst^*, \mathsf{structs}\ structinst^*, \mathsf{arrays}\ arrayinst^*, \mathsf{exns}\ exninst^*\} \\ (S.\mathsf{structs}[a_\mathrm{s}] = structinst)^* \quad ((\mathsf{ref.struct}\ a_\mathrm{s}) \not\gg_S^+ (\mathsf{ref.struct}\ a_\mathrm{s}))^* \\ (S.\mathsf{arrays}[a_\mathrm{a}] = arrayinst)^* \quad ((\mathsf{ref.array}\ a_\mathrm{a}) \not\gg_S^+ (\mathsf{ref.array}\ a_\mathrm{a}))^* \\ (S.\mathsf{exns}[a_\mathrm{e}] = exninst)^* \quad ((\mathsf{ref.exn}\ a_\mathrm{e}) \not\gg_S^+ (\mathsf{ref.exn}\ a_\mathrm{e}))^*\end{array}}{\vdash S : \mathsf{ok}}$$

where $val_1 \gg_S^+ val_2$ denotes the transitive closure of the following *immutable reachability* relation on values:

$$\begin{array}{lll}(\mathsf{ref.struct}\ a) & \gg_S & S.\mathsf{structs}[a].\mathsf{fields}[i] \quad \text{if } \mathrm{expand}(S.\mathsf{structs}[a].\mathsf{type}) = \mathsf{struct}\ ft_1^i\ st\ ft_2^* \\ (\mathsf{ref.array}\ a) & \gg_S & S.\mathsf{arrays}[a].\mathsf{fields}[i] \quad \text{if } \mathrm{expand}(S.\mathsf{arrays}[a].\mathsf{type}) = \mathsf{array}\ st \\ (\mathsf{ref.exn}\ a) & \gg_S & S.\mathsf{exns}[a].\mathsf{fields}[i] \\ (\mathsf{ref.extern}\ ref) & \gg_S & ref\end{array}$$

Note

The constraint on reachability through immutable fields prevents the presence of cyclic data structures that can not be constructed in the language. Cycles can only be formed using mutation.

Tag Instances $\{\text{type } tagtype\}$

- The tag type $tagtype$ must be valid under the empty context.
- Then the tag instance is valid with tag type $tagtype$.

$$\frac{\vdash tagtype : \mathsf{ok}}{S \vdash \{\text{type } tagtype\} : tagtype}$$

Global Instances $\{\text{type } mut\ t, \text{value } val\}$

- The global type $mut\ t$ must be valid under the empty context.
- The value val must be valid with some value type t'.
- The value type t' must match the value type t.
- Then the global instance is valid with global type $mut\ t$.

$$\frac{\vdash mut\ t : \mathsf{ok} \qquad S \vdash val : t' \qquad \vdash t' \leq t}{S \vdash \{\text{type } mut\ t, \text{value } val\} : mut\ t}$$

Memory Instances $\{\text{type } (addrtype\ limits), \text{bytes } b^*\}$

- The memory type $addrtype\ limits$ must be valid under the empty context.
- Let $limits$ be $[n\,..\,m]$.
- The length of b^* must equal m multiplied by the page size $64\,\mathrm{Ki}$.
- Then the memory instance is valid with memory type $addrtype\ limits$.

$$\frac{\vdash addrtype\ [n\,..\,m] : \mathsf{ok} \qquad |b^*| = n \cdot 64\,\mathrm{Ki}}{S \vdash \{\text{type } (addrtype\ [n\,..\,m]), \text{bytes } b^*\} : addrtype\ [n\,..\,m]}$$

Table Instances $\{\text{type } (addrtype\ limits\ t), \text{refs } ref^*\}$

- The table type $addrtype\ limits\ t$ must be valid under the empty context.
- Let $limits$ be $[n\,..\,m]$.
- The length of ref^* must equal n.
- For each reference ref_i in the table's elements ref^n:
  - The reference ref_i must be valid with some reference type t'_i.
  - The reference type t'_i must match the reference type t.
- Then the table instance is valid with table type $addrtype\ limits\ t$.

$$\frac{\vdash addrtype\ [n\,..\,m]\ t : \mathsf{ok} \qquad |ref^*| = n \qquad (S \vdash ref : t')^* \qquad (\vdash t' \leq t)^*}{S \vdash \{\text{type } (addrtype\ [n\,..\,m]\ t), \text{refs } ref^*\} : addrtype\ [n\,..\,m]\ t}$$

Function Instances $\{\mathsf{type}\ deftype, \mathsf{module}\ moduleinst, \mathsf{code}\ func\}$

- The defined type $deftype$ must be valid under an empty context.
- The module instance $moduleinst$ must be valid with some context C.
- Under context C:
  - The function $func$ must be valid with some defined type $deftype'$.
  - The defined type $deftype'$ must match $deftype$.
- Then the function instance is valid with defined type $deftype$.

$$\frac{\vdash deftype : \mathsf{ok} \qquad S \vdash moduleinst : C \qquad C \vdash func : deftype' \qquad C \vdash deftype' \leq deftype}{S \vdash \{\mathsf{type}\ deftype, \mathsf{module}\ moduleinst, \mathsf{code}\ func\} : deftype}$$

Host Function Instances $\{\mathsf{type}\ deftype, \mathsf{hostfunc}\ hf\}$

- The defined type $deftype$ must be valid under an empty context.
- The expansion of defined type $deftype$ must be some function type $\mathsf{func}\ [t_1^*] \to [t_2^*]$.
- For every valid store S_1 extending S and every sequence val^* of values whose types coincide with t_1^*:
  - Executing hf in store S_1 with arguments val^* has a non-empty set of possible outcomes.
  - For every element R of this set:
    - Either R must be $\bot$ (i.e., divergence).
    - Or R consists of a valid store S_2 extending S_1 and a result $result$ whose type coincides with $[t_2^*]$.
- Then the function instance is valid with defined type $deftype$.

$$\frac{\begin{array}{c}\forall S_1, val^*, \vdash S_1 : \mathsf{ok} \wedge \vdash S \preceq S_1 \wedge S_1 \vdash val^* : [t_1^*] \implies \\ hf(S_1; val^*) \supset \emptyset \wedge \\ \forall R \in hf(S_1; val^*),\ R = \bot \vee \\ \exists S_2, result, \vdash S_2 : \mathsf{ok} \wedge \vdash S_1 \preceq S_2 \wedge S_2 \vdash result : [t_2^*] \wedge R = (S_2; result)\end{array} \qquad \begin{array}{c}\vdash deftype : \mathsf{ok} \\ deftype \approx \mathsf{func}\ [t_1^*] \to [t_2^*]\end{array}}{S \vdash \{\mathsf{type}\ deftype, \mathsf{hostfunc}\ hf\} : deftype}$$

Note

This rule states that, if appropriate pre-conditions about store and arguments are satisfied, then executing the host function must satisfy appropriate post-conditions about store and results. The post-conditions match the ones in the execution rule for invoking host functions.

Any store under which the function is invoked is assumed to be an extension of the current store. That way, the function itself is able to make sufficient assumptions about future stores.

Data Instances $\{\mathsf{bytes}\ b^*\}$

- The data instance is valid.

$$\frac{}{S \vdash \{\mathsf{bytes}\ b^*\} : \mathsf{ok}}$$

Element Instances $\{\mathsf{type}\ t, \mathsf{refs}\ ref^*\}$

- The reference type t must be valid under the empty context.
- For each reference ref_i in the elements ref^n:
  - The reference ref_i must be valid with some reference type t_i'.

  - The reference type t'_i must match the reference type t.
- Then the element instance is valid with reference type t.

$$\frac{\vdash t : \mathsf{ok} \qquad (S \vdash ref : t')^* \qquad (\vdash t' \leq t)^*}{S \vdash \{\mathsf{type}\ t, \mathsf{refs}\ ref^*\} : t}$$

Structure Instances $\{\mathsf{type}\ deftype, \mathsf{fields}\ fieldval^*\}$

- The defined type $deftype$ must be valid under the empty context.
- The expansion of $deftype$ must be a structure type $\mathsf{struct}\ fieldtype^*$.
- The length of the sequence of field values $fieldval^*$ must be the same as the length of the sequence of field types $fieldtype^*$.
- For each field value $fieldval_i$ in $fieldval^*$ and corresponding field type $fieldtype_i$ in $fieldtype^*$:
  - Let $fieldtype_i$ be $mut\ storagetype_i$.
  - The field value $fieldval_i$ must be valid with storage type $storagetype_i$.
- Then the structure instance is valid.

$$\frac{\vdash dt : \mathsf{ok} \qquad \mathrm{expand}(dt) = \mathsf{struct}\ (mut\ st)^* \qquad (S \vdash fv : st)^*}{S \vdash \{\mathsf{type}\ dt, \mathsf{fields}\ fv^*\} : \mathsf{ok}}$$

Array Instances $\{\mathsf{type}\ deftype, \mathsf{fields}\ fieldval^*\}$

- The defined type $deftype$ must be valid under the empty context.
- The expansion of $deftype$ must be an array type $\mathsf{array}\ fieldtype$.
- Let $fieldtype$ be $mut\ storagetype$.
- For each field value $fieldval_i$ in $fieldval^*$:
  - The field value $fieldval_i$ must be valid with storage type $storagetype$.
- Then the array instance is valid.

$$\frac{\vdash dt : \mathsf{ok} \qquad \mathrm{expand}(dt) = \mathsf{array}\ (mut\ st) \qquad (S \vdash fv : st)^*}{S \vdash \{\mathsf{type}\ dt, \mathsf{fields}\ fv^*\} : \mathsf{ok}}$$

Field Values $fieldval$

- If $fieldval$ is a value val, then:
  - The value val must be valid with value type t.
  - Then the field value is valid with value type t.
- Else, $fieldval$ is a packed value $packval$:
  - Let $packtype.\mathsf{pack}\ i$ be the field value $fieldval$.
  - Then the field value is valid with packed type $packtype$.

$$\frac{}{S \vdash pt.\mathsf{pack}\ i : pt}$$

Exception Instances $\{\mathsf{tag}\ a, \mathsf{fields}\ val^*\}$

- The store entry $S.\mathsf{tags}[a]$ must exist.
- The expansion of the tag type $S.\mathsf{tags}[a].\mathsf{type}$ must be some function type $\mathsf{func}\ [t^*] \to [t'^*]$.
- The result type $[t'^*]$ must be empty.
- The sequence $val^a\,st$ of values must have the same length as the sequence t^* of value types.
- For each value val_i in $val^a\,st$ and corresponding value type t_i in t^*, the value val_i must be valid with type t_i.
- Then the exception instance is valid.

$$\frac{S.\mathsf{tags}[a].\mathsf{type} \approx \mathsf{func}\ [t^*] \to [] \qquad (S \vdash val : t)^*}{S \vdash \{\mathsf{tag}\ a, \mathsf{fields}\ val^*\} : \mathsf{ok}}$$

Export Instances $\{\mathsf{name}\ name, \mathsf{addr}\ externaddr\}$

- The external address $externaddr$ must be valid with some external type $externtype$.
- Then the export instance is valid.

$$\frac{S \vdash externaddr : externtype}{S \vdash \{\mathsf{name}\ name, \mathsf{addr}\ externaddr\} : \mathsf{ok}}$$

Module Instances $moduleinst$

- Each defined type $deftype_i$ in $moduleinst.\mathsf{types}$ must be valid under the empty context.
- For each tag address $tagaddr_i$ in $moduleinst.\mathsf{tags}$, the external address tag $tagaddr_i$ must be valid with some external type tag $tagtype_i$.
- For each global address $globaladdr_i$ in $moduleinst.\mathsf{globals}$, the external address global $globaladdr_i$ must be valid with some external type global $globaltype_i$.
- For each memory address $memaddr_i$ in $moduleinst.\mathsf{mems}$, the external address mem $memaddr_i$ must be valid with some external type mem $memtype_i$.
- For each table address $tableaddr_i$ in $moduleinst.\mathsf{tables}$, the external address table $tableaddr_i$ must be valid with some external type table $tabletype_i$.
- For each function address $funcaddr_i$ in $moduleinst.\mathsf{funcs}$, the external address func $funcaddr_i$ must be valid with some external type func $deftype_{\mathsf{F}i}$.
- For each data address $dataaddr_i$ in $moduleinst.\mathsf{datas}$, the data instance $S.\mathsf{datas}[dataaddr_i]$ must be valid with ok_i.
- For each element address $elemaddr_i$ in $moduleinst.\mathsf{elems}$, the element instance $S.\mathsf{elems}[elemaddr_i]$ must be valid with some reference type $reftype_i$.
- Each export instance $exportinst_i$ in $moduleinst.\mathsf{exports}$ must be valid.
- For each export instance $exportinst_i$ in $moduleinst.\mathsf{exports}$, the name $exportinst_i.\mathsf{name}$ must be different from any other name occurring in $moduleinst.\mathsf{exports}$.
- Let $deftype^*$ be the concatenation of all $deftype_i$ in order.
- Let $tagtype^*$ be the concatenation of all $tagtype_i$ in order.
- Let $globaltype^*$ be the concatenation of all $globaltype_i$ in order.
- Let $memtype^*$ be the concatenation of all $memtype_i$ in order.
- Let $tabletype^*$ be the concatenation of all $tabletype_i$ in order.
- Let $deftype^*_{\mathsf{F}}$ be the concatenation of all $deftype_{\mathsf{F}i}$ in order.

- Let $reftype^*$ be the concatenation of all $reftype_i$ in order.
- Let ok^* be the concatenation of all ok_i in order.
- Let m be the length of $moduleinst$.funcs.
- Let x^* be the sequence of function indices from 0 to $k-1$ for some k that is smaller than or equal to the number of functions in the instance.
- Then the module instance is valid with context $\{\mathsf{types}\ deftype^*,\ \mathsf{tags}\ tagtype^*,\ \mathsf{globals}\ globaltype^*,\ \mathsf{mems}\ memtype^*,\ \mathsf{tables}\ tabletype^*,\ \mathsf{funcs}\ deftype_\mathsf{F}^*,\ \mathsf{datas}\ ok^*,\ \mathsf{elems}\ reftype^*,\ \mathsf{refs}\ x^*\}$.

$$\frac{\begin{array}{c}(\vdash deftype : \mathsf{ok})^* \qquad (S \vdash \mathsf{tag}\ tagaddr : \mathsf{tag}\ tagtype)^* \\ (S \vdash \mathsf{global}\ globaladdr : \mathsf{global}\ globaltype)^* \qquad (S \vdash \mathsf{func}\ funcaddr : \mathsf{func}\ deftype_\mathsf{F})^* \\ (S \vdash \mathsf{mem}\ memaddr : \mathsf{mem}\ memtype)^* \qquad (S \vdash \mathsf{table}\ tableaddr : \mathsf{table}\ tabletype)^* \\ (S \vdash S.\mathsf{datas}[dataaddr] : ok)^* \qquad (S \vdash S.\mathsf{elems}[elemaddr] : reftype)^* \\ (S \vdash exportinst : \mathsf{ok})^* \qquad (exportinst.\mathsf{name})^*\ \text{disjoint} \\ k \leq |funcaddr^*|\end{array}}{\begin{array}{l}S \vdash \{\mathsf{types}\ deftype^*, \\ \quad \mathsf{tags}\ tagaddr^*, \\ \quad \mathsf{globals}\ globaladdr^*, \\ \quad \mathsf{mems}\ memaddr^*, \\ \quad \mathsf{tables}\ tableaddr^*, \\ \quad \mathsf{funcs}\ funcaddr^*, \\ \quad \mathsf{datas}\ dataaddr^*, \\ \quad \mathsf{elems}\ elemaddr^*, \\ \quad \mathsf{exports}\ exportinst^*\ \} : \{\mathsf{types}\ deftype^*, \\ \qquad\qquad\qquad\qquad \mathsf{tags}\ tagtype^*, \\ \qquad\qquad\qquad\qquad \mathsf{globals}\ globaltype^*, \\ \qquad\qquad\qquad\qquad \mathsf{mems}\ memtype^*, \\ \qquad\qquad\qquad\qquad \mathsf{tables}\ tabletype^*, \\ \qquad\qquad\qquad\qquad \mathsf{funcs}\ deftype_\mathsf{F}^*, \\ \qquad\qquad\qquad\qquad \mathsf{datas}\ ok^*, \\ \qquad\qquad\qquad\qquad \mathsf{elems}\ reftype^*, \\ \qquad\qquad\qquad\qquad \mathsf{refs}\ 0 \ldots k-1\ \}\end{array}}$$

Note

The field C.refs from the resulting context is meant to be constructed to contain all function indices from the module. However, modules and their instances can contain more than 2^{32} functions (at most 2^{32} definitions plus 2^{32} imports), while the highest representable function index is $2^{32}-1$. The variable k in the rule hence allows picking an upper limit for C.refs that is smaller than the total number of functions, in case that is necessary for C.refs to be syntactically well-formed. In practice, $k = \min(|funcaddr^*|, 2^{32})$ always is the maximally permissive choice.

7.4.6 Configuration Validity

To relate the WebAssembly type system to its execution semantics, the typing rules for instructions must be extended to configurations $S;T$, which relates the store to execution threads.

Configurations and threads are classified by their result type. In addition to the store S, threads are typed under a *return type* $resulttype^?$, which controls whether and with which type a return instruction is allowed. This type is absent (ϵ) except for instruction sequences inside an administrative frame instruction.

Finally, frames are classified with *frame contexts*, which extend the module contexts of a frame's associated module instance with the locals that the frame contains.

Configurations $S; T$

- The store S must be valid.
- Under no allowed return type, the thread T must be valid with some result type $[t^*]$.
- Then the configuration is valid with the result type $[t^*]$.

$$\frac{\vdash S : \mathsf{ok} \qquad S; \epsilon \vdash T : [t^*]}{\vdash S; T : [t^*]}$$

Threads $F; instr^*$

- Let $resulttype^?$ be the current allowed return type.
- The frame F must be valid with a context C.
- Let C' be the same context as C, but with return set to $resulttype^?$.
- Under context C', the instruction sequence $instr^*$ must be valid with some type $[] \to [t^*]$.
- Then the thread is valid with the result type $[t^*]$.

$$\frac{S \vdash F : C \qquad S; C, \mathsf{return}\ resulttype^? \vdash instr^* : [] \to [t^*]}{S; resulttype^? \vdash F; instr^* : [t^*]}$$

Frames $\{\mathsf{locals}\ val^*, \mathsf{module}\ moduleinst\}$

- The module instance $moduleinst$ must be valid with some module context C.
- Each value val_i in val^* must be valid with some value type t_i.
- Let t^* be the concatenation of all t_i in order.
- Let C' be the same context as C, but with the value types t^* prepended to the locals list.
- Then the frame is valid with frame context C'.

$$\frac{S \vdash moduleinst : C \qquad (S \vdash val : t)^*}{S \vdash \{\mathsf{locals}\ val^*, \mathsf{module}\ moduleinst\} : (C, \mathsf{locals}\ t^*)}$$

7.4.7 Administrative Instructions

Typing rules for administrative instructions are specified as follows. In addition to the context C, typing of these instructions is defined under a given store S.

To that end, all previous typing judgements $C \vdash prop$ are generalized to include the store, as in $S; C \vdash prop$, by implicitly adding S to all rules – S is never modified by the pre-existing rules, but it is accessed in the extra rules for administrative instructions given below.

trap

- The instruction is valid with any valid instruction type of the form $[t_1^*] \to [t_2^*]$.

$$\frac{C \vdash [t_1^*] \to [t_2^*] : \mathsf{ok}}{S; C \vdash \mathsf{trap} : [t_1^*] \to [t_2^*]}$$

val

- The value val must be valid with value type t.
- Then it is valid as an instruction with type $[] \to [t]$.

$$\frac{S \vdash val : t}{S; C \vdash val : [] \to [t]}$$

$\mathsf{label}_n\{instr_0^*\}\ instr^*$

- The instruction sequence $instr_0^*$ must be valid with some type $[t_1^n] \to_{x^*} [t_2^*]$.
- Let C' be the same context as C, but with the result type $[t_1^n]$ prepended to the labels list.
- Under context C', the instruction sequence $instr^*$ must be valid with type $[] \to_{x'^*} [t_2^*]$.
- Then the compound instruction is valid with type $[] \to [t_2^*]$.

$$\frac{S; C \vdash instr_0^* : [t_1^n] \to_{x^*} [t_2^*] \qquad S; C, \mathsf{labels}\,[t_1^n] \vdash instr^* : [] \to_{x'^*} [t_2^*]}{S; C \vdash \mathsf{label}_n\{instr_0^*\}\ instr^* : [] \to [t_2^*]}$$

$\mathsf{frame}_n\{F\}\ instr^*$

- Under the valid return type $[t^n]$, the thread $F; instr^*$ must be valid with result type $[t^n]$.
- Then the compound instruction is valid with type $[] \to [t^n]$.

$$\frac{\{\} \vdash [t^n] : \mathsf{ok} \qquad S; [t^n] \vdash F; instr^* : [t^n]}{S; C \vdash \mathsf{frame}_n\{F\}\ instr^* : [] \to [t^n]}$$

$\mathsf{handler}_n\{catch^*\}\ instr^*$

- For every catch clause $catch_i$ in $catch^*$, $catch_i$ must be valid.
- The instruction sequence $instr^*$ must be valid with some type $[t_1^*] \to [t_2^*]$.
- Then the compound instruction is valid with type $[t_1^*] \to [t_2^*]$.

$$\frac{(C \vdash catch : \mathsf{ok})^* \qquad S; C \vdash instr^* : [] \to [t^*]}{S; C \vdash \mathsf{handler}_n\{catch^*\}\ instr^* : [] \to [t^*]}$$

7.4.8 Store Extension

Programs can mutate the store and its contained instances. Any such modification must respect certain invariants, such as not removing allocated instances or changing immutable definitions. While these invariants are inherent to the execution semantics of WebAssembly instructions and modules, host functions do not automatically adhere to them. Consequently, the required invariants must be stated as explicit constraints on the invocation of host functions. Soundness only holds when the embedder ensures these constraints.

The necessary constraints are codified by the notion of store *extension*: a store state S' extends state S, written $S \preceq S'$, when the following rules hold.

Note

Extension does not imply that the new store is valid, which is defined separately above.

Store S

- The length of $S.\mathsf{tags}$ must not shrink.
- The length of $S.\mathsf{globals}$ must not shrink.
- The length of $S.\mathsf{mems}$ must not shrink.
- The length of $S.\mathsf{tables}$ must not shrink.
- The length of $S.\mathsf{funcs}$ must not shrink.
- The length of $S.\mathsf{datas}$ must not shrink.
- The length of $S.\mathsf{elems}$ must not shrink.
- The length of $S.\mathsf{structs}$ must not shrink.
- The length of $S.\mathsf{arrays}$ must not shrink.
- The length of $S.\mathsf{exns}$ must not shrink.
- For each tag instance $taginst_i$ in the original $S.\mathsf{tags}$, the new tag instance must be an extension of the old.
- For each global instance $globalinst_i$ in the original $S.\mathsf{globals}$, the new global instance must be an extension of the old.
- For each memory instance $meminst_i$ in the original $S.\mathsf{mems}$, the new memory instance must be an extension of the old.
- For each table instance $tableinst_i$ in the original $S.\mathsf{tables}$, the new table instance must be an extension of the old.
- For each function instance $funcinst_i$ in the original $S.\mathsf{funcs}$, the new function instance must be an extension of the old.
- For each data instance $datainst_i$ in the original $S.\mathsf{datas}$, the new data instance must be an extension of the old.
- For each element instance $eleminst_i$ in the original $S.\mathsf{elems}$, the new element instance must be an extension of the old.
- For each structure instance $structinst_i$ in the original $S.\mathsf{structs}$, the new structure instance must be an extension of the old.
- For each array instance $arrayinst_i$ in the original $S.\mathsf{arrays}$, the new array instance must be an extension of the old.
- For each exception instance $exninst_i$ in the original $S.\mathsf{exns}$, the new exception instance must be an extension of the old.

$$\frac{\begin{array}{ccc}
S_1.\mathsf{tags} = taginst_1^* & S_2.\mathsf{tags} = taginst_1'^{\,*}\ taginst_2^* & (\vdash taginst_1 \preceq taginst_1')^* \\
S_1.\mathsf{globals} = globalinst_1^* & S_2.\mathsf{globals} = globalinst_1'^{\,*}\ globalinst_2^* & (\vdash globalinst_1 \preceq globalinst_1')^* \\
S_1.\mathsf{mems} = meminst_1^* & S_2.\mathsf{mems} = meminst_1'^{\,*}\ meminst_2^* & (\vdash meminst_1 \preceq meminst_1')^* \\
S_1.\mathsf{tables} = tableinst_1^* & S_2.\mathsf{tables} = tableinst_1'^{\,*}\ tableinst_2^* & (\vdash tableinst_1 \preceq tableinst_1')^* \\
S_1.\mathsf{funcs} = funcinst_1^* & S_2.\mathsf{funcs} = funcinst_1'^{\,*}\ funcinst_2^* & (\vdash funcinst_1 \preceq funcinst_1')^* \\
S_1.\mathsf{datas} = datainst_1^* & S_2.\mathsf{datas} = datainst_1'^{\,*}\ datainst_2^* & (\vdash datainst_1 \preceq datainst_1')^* \\
S_1.\mathsf{elems} = eleminst_1^* & S_2.\mathsf{elems} = eleminst_1'^{\,*}\ eleminst_2^* & (\vdash eleminst_1 \preceq eleminst_1')^* \\
S_1.\mathsf{structs} = structinst_1^* & S_2.\mathsf{structs} = structinst_1'^{\,*}\ structinst_2^* & (\vdash structinst_1 \preceq structinst_1')^* \\
S_1.\mathsf{arrays} = arrayinst_1^* & S_2.\mathsf{arrays} = arrayinst_1'^{\,*}\ arrayinst_2^* & (\vdash arrayinst_1 \preceq arrayinst_1')^* \\
S_1.\mathsf{exns} = exninst_1^* & S_2.\mathsf{exns} = exninst_1'^{\,*}\ exninst_2^* & (\vdash exninst_1 \preceq exninst_1')^*
\end{array}}{\vdash S_1 \preceq S_2}$$

Tag Instance $taginst$

- A tag instance must remain unchanged.

$$\frac{}{\vdash taginst \preceq taginst}$$

Global Instance $globalinst$

- The global type $globalinst.\mathsf{type}$ must remain unchanged.
- Let $mut\ t$ be the structure of $globalinst.\mathsf{type}$.
- If mut is empty, then the value $globalinst.\mathsf{value}$ must remain unchanged.

$$\frac{mut = \mathsf{mut} \vee val_1 = val_2}{\vdash \{\mathsf{type}\ (mut\ t), \mathsf{value}\ val_1\} \preceq \{\mathsf{type}\ (mut\ t), \mathsf{value}\ val_2\}}$$

Memory Instance $meminst$

- The memory type $meminst.\mathsf{type}$ must remain unchanged.
- The length of $meminst.\mathsf{bytes}$ must not shrink.

$$\frac{n_1 \leq n_2}{\vdash \{\mathsf{type}\ mt, \mathsf{bytes}\ b_1^{n_1}\} \preceq \{\mathsf{type}\ mt, \mathsf{bytes}\ b_2^{n_2}\}}$$

Table Instance $tableinst$

- The table type $tableinst.\mathsf{type}$ must remain unchanged.
- The length of $tableinst.\mathsf{refs}$ must not shrink.

$$\frac{n_1 \leq n_2}{\vdash \{\mathsf{type}\ tt, \mathsf{refs}\ (fa_1^?)^{n_1}\} \preceq \{\mathsf{type}\ tt, \mathsf{refs}\ (fa_2^?)^{n_2}\}}$$

Function Instance $funcinst$

- A function instance must remain unchanged.

$$\frac{}{\vdash funcinst \preceq funcinst}$$

Data Instance $datainst$

- The list $datainst.\mathsf{bytes}$ must:
  - either remain unchanged,
  - or shrink to length 0.

$$\frac{}{\vdash \{\mathsf{bytes}\ b^*\} \preceq \{\mathsf{bytes}\ b^*\}}$$

$$\frac{}{\vdash \{\mathsf{bytes}\ b^*\} \preceq \{\mathsf{bytes}\ \epsilon\}}$$

Element Instance $eleminst$

- The reference type $eleminst$.type must remain unchanged.
- The list $eleminst$.refs must:
  - either remain unchanged,
  - or shrink to length 0.

$$\frac{}{\vdash \{\mathsf{type}\ t, \mathsf{refs}\ a^*\} \preceq \{\mathsf{type}\ t, \mathsf{refs}\ a^*\}}$$

$$\frac{}{\vdash \{\mathsf{type}\ t, \mathsf{refs}\ a^*\} \preceq \{\mathsf{type}\ t, \mathsf{refs}\ \epsilon\}}$$

Structure Instance $structinst$

- The defined type $structinst$.type must remain unchanged.
- Assert: due to store well-formedness, the expansion of $structinst$.type is a structure type.
- Let struct $fieldtype^*$ be the expansion of $structinst$.type.
- The length of the list $structinst$.fields must remain unchanged.
- Assert: due to store well-formedness, the length of $structinst$.fields is the same as the length of $fieldtype^*$.
- For each field value $fieldval_i$ in $structinst$.fields and corresponding field type $fieldtype_i$ in $fieldtype^*$:
  - Let $mut_i\ st_i$ be the structure of $fieldtype_i$.
  - If mut_i is empty, then the field value $fieldval_i$ must remain unchanged.

$$\frac{(mut = \mathsf{mut} \vee fieldval_1 = fieldval_2)^*}{\vdash \{\mathsf{type}\ (mut\ st)^*, \mathsf{fields}\ fieldval_1^*\} \preceq \{\mathsf{type}\ (mut\ st)^*, \mathsf{fields}\ fieldval_2^*\}}$$

Array Instance $arrayinst$

- The defined type $arrayinst$.type must remain unchanged.
- Assert: due to store well-formedness, the expansion of $arrayinst$.type is an array type.
- Let array $fieldtype$ be the expansion of $arrayinst$.type.
- The length of the list $arrayinst$.fields must remain unchanged.
- Let $mut\ st$ be the structure of $fieldtype$.
- If mut is empty, then the sequence of field values $arrayinst$.fields must remain unchanged.

$$\frac{mut = \mathsf{mut} \vee fieldval_1^* = fieldval_2^*}{\vdash \{\mathsf{type}\ (mut\ st), \mathsf{fields}\ fieldval_1^*\} \preceq \{\mathsf{type}\ (mut\ st), \mathsf{fields}\ fieldval_2^*\}}$$

Exception Instance $exninst$

- An exception instance must remain unchanged.

$$\frac{}{\vdash exninst \preceq exninst}$$

7.4.9 Theorems

Given the definition of valid configurations, the standard soundness theorems hold.[52][54]

Theorem (Preservation). If a configuration $S;T$ is valid with result type $[t^*]$ (i.e., $\vdash S;T : [t^*]$), and steps to $S';T'$ (i.e., $S;T \hookrightarrow S';T'$), then $S';T'$ is a valid configuration with the same result type (i.e., $\vdash S';T' : [t^*]$). Furthermore, S' is an extension of S (i.e., $\vdash S \preceq S'$).

A *terminal* thread is one whose sequence of instructions is a result. A terminal configuration is a configuration whose thread is terminal.

Theorem (Progress). If a configuration $S;T$ is valid (i.e., $\vdash S;T : [t^*]$ for some result type $[t^*]$), then either it is terminal, or it can step to some configuration $S';T'$ (i.e., $S;T \hookrightarrow S';T'$).

From Preservation and Progress the soundness of the WebAssembly type system follows directly.

Corollary (Soundness). If a configuration $S;T$ is valid (i.e., $\vdash S;T : [t^*]$ for some result type $[t^*]$), then it either diverges or takes a finite number of steps to reach a terminal configuration $S';T'$ (i.e., $S;T \hookrightarrow {}^*S';T'$) that is valid with the same result type (i.e., $\vdash S';T' : [t^*]$) and where S' is an extension of S (i.e., $\vdash S \preceq S'$).

In other words, every thread in a valid configuration either runs forever, traps, throws an exception, or terminates with a result that has the expected type. Consequently, given a valid store, no computation defined by instantiation or invocation of a valid module can "crash" or otherwise (mis)behave in ways not covered by the execution semantics given in this specification.

7.5 Type System Properties

7.5.1 Principal Types

The type system of WebAssembly features both subtyping and simple forms of polymorphism for instruction types. That has the effect that every instruction or instruction sequence can be classified with multiple different instruction types.

However, the typing rules still allow deriving *principal types* for instruction sequences. That is, every valid instruction sequence has one particular type scheme, possibly containing some unconstrained place holder *type variables*, that is a subtype of all its valid instruction types, after substituting its type variables with suitable specific types.

Moreover, when deriving an instruction type in a "forward" manner, i.e., the *input* of the instruction sequence is already fixed to specific types, then it has a principal *output* type expressible without type variables, up to a possibly polymorphic stack bottom representable with one single variable. In other words, "forward" principal types are effectively *closed*.

Note

For example, in isolation, the instruction ref.as_non_null has the type $[(\mathsf{ref\ null}\ ht)] \to [(\mathsf{ref}\ ht)]$ for any choice of valid heap type ht. Moreover, if the input type $[(\mathsf{ref\ null}\ ht)]$ is already determined, i.e., a specific ht is given, then the output type $[(\mathsf{ref}\ ht)]$ is fully determined as well.

The implication of the latter property is that a validator for *complete* instruction sequences (as they occur in valid modules) can be implemented with a simple left-to-right algorithm that does not require the introduction of type variables.

A typing algorithm capable of handling *partial* instruction sequences (as might be considered for program analysis or program manipulation) needs to introduce type variables and perform substitutions, but it does not need to perform backtracking or record any non-syntactic constraints on these type variables.

[52] A machine-verified version of the formalization and soundness proof of the PLDI 2017 paper is described in the following article: Conrad Watt. Mechanising and Verifying the WebAssembly Specification[53]. Proceedings of the 7th ACM SIGPLAN Conference on Certified Programs and Proofs (CPP 2018). ACM 2018.

[53] https://dl.acm.org/citation.cfm?id=3167082

[54] Machine-verified formalizations and soundness proofs of the semantics from the official specification are described in the following article: Conrad Watt, Xiaojia Rao, Jean Pichon-Pharabod, Martin Bodin, Philippa Gardner. Two Mechanisations of WebAssembly 1.0[55]. Proceedings of the 24th International Symposium on Formal Methods (FM 2021). Springer 2021.

[55] https://link.springer.com/chapter/10.1007/978-3-030-90870-6_4

Technically, the syntax of heap, value, and result types can be enriched with type variables as follows:

$$
\begin{array}{lcl}
null & ::= & \mathsf{null}^? \mid \alpha_{null} \\
heaptype & ::= & \ldots \mid \alpha_{heaptype} \\
reftype & ::= & \mathsf{ref}\ null\ heaptype \\
valtype & ::= & \ldots \mid \alpha_{valtype} \mid \alpha_{numvectype} \\
resulttype & ::= & [\alpha^?_{valtype^*}\ valtype^*]
\end{array}
$$

where each α_{xyz} ranges over a set of type variables for syntactic class xyz, respectively. The special class $numvectype$ is defined as $numtype \mid vectype \mid \mathsf{bot}$, and is only needed to handle unannotated select instructions.

A type is *closed* when it does not contain any type variables, and *open* otherwise. A *type substitution* σ is a finite mapping from type variables to closed types of the respective syntactic class. When applied to an open type, it replaces the type variables α from its domain with the respective $\sigma(\alpha)$.

Theorem (Principal Types). If an instruction sequence $instr^*$ is valid with some closed instruction type $instrtype$ (i.e., $C \vdash instr^* : instrtype$), then it is also valid with a possibly open instruction type $instrtype_{\min}$ (i.e., $C \vdash instr^* : instrtype_{\min}$), such that for *every* closed type $instrtype'$ with which $instr^*$ is valid (i.e., for all $C \vdash instr^* : instrtype'$), there exists a substitution σ, such that $\sigma(instrtype_{\min})$ is a subtype of $instrtype'$ (i.e., $C \vdash \sigma(instrtype_{\min}) \leq instrtype'$). Furthermore, $instrtype_{\min}$ is unique up to the choice of type variables.

Theorem (Closed Principal Forward Types). If closed input type $[t_1^*]$ is given and the instruction sequence $instr^*$ is valid with instruction type $[t_1^*] \to_{x^*} [t_2^*]$ (i.e., $C \vdash instr^* : [t_1^*] \to_{x^*} [t_2^*]$), then it is also valid with instruction type $[t_1^*] \to_{x^*} [\alpha_{valtype^*}\ t^*]$ (i.e., $C \vdash instr^* : [t_1^*] \to_{x^*} [\alpha_{valtype^*}\ t^*]$), where all t^* are closed, such that for *every* closed result type $[{t'_2}^*]$ with which $instr^*$ is valid (i.e., for all $C \vdash instr^* : [t_1^*] \to_{x^*} [{t'_2}^*]$), there exists a substitution σ, such that $[{t'_2}^*] = [\sigma(\alpha_{valtype^*})\ t^*]$.

7.5.2 Type Lattice

The Principal Types property depends on the existence of a *greatest lower bound* for any pair of types.

Theorem (Greatest Lower Bounds for Value Types). For any two value types t_1 and t_2 that are valid (i.e., $C \vdash t_1 : \mathsf{ok}$ and $C \vdash t_2 : \mathsf{ok}$), there exists a valid value type t that is a subtype of both t_1 and t_2 (i.e., $C \vdash t : \mathsf{ok}$ and $C \vdash t \leq t_1$ and $C \vdash t \leq t_2$), such that *every* valid value type t' that also is a subtype of both t_1 and t_2 (i.e., for all $C \vdash t' : \mathsf{ok}$ and $C \vdash t' \leq t_1$ and $C \vdash t' \leq t_2$), is a subtype of t (i.e., $C \vdash t' \leq t$).

Note

The greatest lower bound of two types may be bot.

Theorem (Conditional Least Upper Bounds for Value Types). Any two value types t_1 and t_2 that are valid (i.e., $C \vdash t_1 : \mathsf{ok}$ and $C \vdash t_2 : \mathsf{ok}$) either have no common supertype, or there exists a valid value type t that is a supertype of both t_1 and t_2 (i.e., $C \vdash t : \mathsf{ok}$ and $C \vdash t_1 \leq t$ and $C \vdash t_2 \leq t$), such that *every* valid value type t' that also is a supertype of both t_1 and t_2 (i.e., for all $C \vdash t' : \mathsf{ok}$ and $C \vdash t_1 \leq t'$ and $C \vdash t_2 \leq t'$), is a supertype of t (i.e., $C \vdash t \leq t'$).

Note

If a top type was added to the type system, a least upper bound would exist for any two types.

Corollary (Type Lattice). Assuming the addition of a provisional top type, value types form a lattice with respect to their subtype relation.

Finally, value types can be partitioned into multiple disjoint hierarchies that are not related by subtyping, except through bot.

Theorem (Disjoint Subtype Hierarchies). The greatest lower bound of two value types is bot or ref bot if and only if they do not have a least upper bound.

In other words, types that do not have common supertypes, do not have common subtypes either (other than bot or ref bot), and vice versa.

Note

Types from disjoint hierarchies can safely be represented in mutually incompatible ways in an implementation, because their values can never flow to the same place.

7.5.3 Compositionality

Valid instruction sequences can be freely *composed*, as long as their types match up.

Theorem (Composition). If two instruction sequences $instr_1^*$ and $instr_2^*$ are valid with types $[t_1^*] \to_{x_1^*} [t^*]$ and $[t^*] \to_{x_2^*} [t_2^*]$, respectively (i.e., $C \vdash instr_1^* : [t_1^*] \to_{x_1^*} [t^*]$ and $C \vdash instr_1^* : [t^*] \to_{x_2^*} [t_2^*]$), then the concatenated instruction sequence $(instr_1^*\ instr_2^*)$ is valid with type $[t_1^*] \to_{x_1^*\ x_2^*} [t_2^*]$ (i.e., $C \vdash instr_1^*\ instr_2^* : [t_1^*] \to_{x_1^*\ x_2^*} [t_2^*]$).

Note

More generally, instead of a shared type $[t^*]$, it suffices if the output type of $instr_1^*$ is a subtype of the input type of $instr_1^*$, since the subtype can always be weakened to its supertype by subsumption.

Inversely, valid instruction sequences can also freely be *decomposed*, that is, splitting them anywhere produces two instruction sequences that are both valid.

Theorem (Decomposition). If an instruction sequence $instr^*$ that is valid with type $[t_1^*] \to_{x^*} [t_2^*]$ (i.e., $C \vdash instr^* : [t_1^*] \to_{x^*} [t_2^*]$) is split into two instruction sequences $instr_1^*$ and $instr_2^*$ at any point (i.e., $instr^* = instr_1^*\ instr_2^*$), then these are separately valid with some types $[t_1^*] \to_{x_1^*} [t^*]$ and $[t^*] \to_{x_2^*} [t_2^*]$, respectively (i.e., $C \vdash instr_1^* : [t_1^*] \to_{x_1^*} [t^*]$ and $C \vdash instr_1^* : [t^*] \to_{x_2^*} [t_2^*]$), where $x^* = x_1^*\ x_2^*$.

Note

This property holds because validation is required even for unreachable code. Without that, $instr_2^*$ might not be valid in isolation.

7.6 Validation Algorithm

The specification of WebAssembly validation is purely *declarative*. It describes the constraints that must be met by a module or instruction sequence to be valid.

This section sketches the skeleton of a sound and complete *algorithm* for effectively validating code, i.e., sequences of instructions. (Other aspects of validation are straightforward to implement.)

In fact, the algorithm is expressed over the flat sequence of opcodes as occurring in the binary format, and performs only a single pass over it. Consequently, it can be integrated directly into a decoder.

The algorithm is expressed in typed pseudo code whose semantics is intended to be self-explanatory.

7.6.1 Data Structures

Types

Value types are representable as sets of enumerations:

```
type num_type = I32 | I64 | F32 | F64
type vec_type = V128
type heap_type =
  Any | Eq | I31 | Struct | Array | None |
  Func | Nofunc | Exn | Noexn | Extern | Noextern | Bot |
  Def(def : def_type)
```

(continues on next page)

(continued from previous page)

```
type ref_type = Ref(heap : heap_type, null : bool)
type val_type = num_type | vec_type | ref_type | Bot

func is_num(t : val_type) : bool =
  return t = I32 || t = I64 || t = F32 || t = F64 || t = Bot

func is_vec(t : val_type) : bool =
  return t = V128 || t = Bot

func is_ref(t : val_type) : bool =
  return not (is_num t || is_vec t) || t = Bot
```

Similarly, defined types `def_type` can be represented:

```
type pack_type = I8 | I16
type field_type = Field(val : val_type | pack_type, mut : bool)

type struct_type = Struct(fields : list(field_type))
type array_type = Array(fields : field_type)
type func_type = Func(params : list(val_type), results : list(val_type))
type comp_type = struct_type | array_type | func_type

type sub_type = Sub(super : list(def_type), body : comp_type, final : bool)
type rec_type = Rec(types : list(sub_type))

type def_type = Def(rec : rec_type, proj : int32)

func unpack_field(t : field_type) : val_type =
  if (it = I8 || t = I16) return I32
  return t

func expand_def(t : def_type) : comp_type =
  return t.rec.types[t.proj].body
```

These representations assume that all types have been closed by substituting all type indices (in concrete heap types and in sub types) with their respective defined types. This includes *recursive* references to enclosing defined types, such that type representations form graphs and may be *cyclic* for recursive types.

We assume that all types have been *canonicalized*, such that equality on two type representations holds if and only if their closures are syntactically equivalent, making it a constant-time check.

Note

For the purpose of type canonicalization, recursive references from a heap type to an enclosing recursive type (i.e., forward edges in the graph that form a cycle) need to be distinguished from references to previously defined types. However, this distinction does not otherwise affect validation, so is ignored here. In the graph representation, all recursive types are effectively infinitely unrolled.

We further assume that validation and subtyping checks are defined on value types, as well as a few auxiliary functions on composite types:

```
func validate_val_type(t : val_type)
func validate_ref_type(t : ref_type)

func matches_val(t1 : val_type, t2 : val_type) : bool
func matches_ref(t1 : val_type, t2 : val_type) : bool
```

(continues on next page)

(continued from previous page)

```
func is_func(t : comp_type) : bool
func is_struct(t : comp_type) : bool
func is_array(t : comp_type) : bool
```

Finally, the following function computes the least precise supertype of a given heap type (its corresponding top type):

```
func top_heap_type(t : heap_type) : heap_type =
  switch (t)
    case (Any | Eq | I31 | Struct | Array | None)
      return Any
    case (Func | Nofunc)
      return Func
    case (Extern | Noextern)
      return Extern
    case (Def(dt))
      switch (dt.rec.types[dt.proj].body)
        case (Struct(_) | Array(_))
          return Any
        case (Func(_))
          return Func
    case (Bot)
      raise CannotOccurInSource
```

Context

Validation requires a context for checking uses of indices. For the purpose of presenting the algorithm, it is maintained in a set of global variables:

```
var return_type : list(val_type)
var types : array(def_type)
var locals : array(val_type)
var locals_init : array(bool)
var globals : array(global_type)
var funcs : array(func_type)
var tables : array(table_type)
var mems : array(mem_type)
```

This assumes suitable representations for the various types besides `val_type`, which are omitted here.

For locals, there is an additional array recording the initialization status of each local.

Stacks

The algorithm uses three separate stacks: the *value stack*, the *control stack*, and the *initialization stack*. The value stack tracks the types of operand values on the stack. The control stack tracks surrounding structured control instructions and their associated blocks. The initialization stack records all locals that have been initialized since the beginning of the function.

```
type val_stack = stack(val_type)
type init_stack = stack(u32)

type ctrl_stack = stack(ctrl_frame)
type ctrl_frame = {
  opcode : opcode
  start_types : list(val_type)
```

(continues on next page)

(continued from previous page)

```
  end_types : list(val_type)
  val_height : nat
  init_height : nat
  unreachable : bool
}
```

For each entered block, the control stack records a *control frame* with the originating opcode, the types on the top of the operand stack at the start and end of the block (used to check its result as well as branches), the height of the operand stack at the start of the block (used to check that operands do not underflow the current block), the height of the initialization stack at the start of the block (used to reset initialization status at the end of the block), and a flag recording whether the remainder of the block is unreachable (used to handle stack-polymorphic typing after branches).

For the purpose of presenting the algorithm, these stacks are simply maintained as global variables:

```
var vals : val_stack
var inits : init_stack
var ctrls : ctrl_stack
```

However, these variables are not manipulated directly by the main checking function, but through a set of auxiliary functions:

```
func push_val(type : val_type) =
  vals.push(type)

func pop_val() : val_type =
  if (vals.size() = ctrls[0].val_height && ctrls[0].unreachable) return Bot
  error_if(vals.size() = ctrls[0].val_height)
  return vals.pop()

func pop_val(expect : val_type) : val_type =
  let actual = pop_val()
  error_if(not matches_val(actual, expect))
  return actual

func pop_num() : num_type | Bot =
  let actual = pop_val()
  error_if(not is_num(actual))
  return actual

func pop_ref() : ref_type =
  let actual = pop_val()
  error_if(not is_ref(actual))
  if (actual = Bot) return Ref(Bot, false)
  return actual

func push_vals(types : list(val_type)) = foreach (t in types) push_val(t)
func pop_vals(types : list(val_type)) : list(val_type) =
  var popped := []
  foreach (t in reverse(types)) popped.prepend(pop_val(t))
  return popped
```

Pushing an operand value simply pushes the respective type to the value stack.

Popping an operand value checks that the value stack does not underflow the current block and then removes one type. But first, a special case is handled where the block contains no known values, but has been marked as unreachable. That can occur after an unconditional branch, when the stack is typed polymorphically. In that case, the `Bot` type is returned, because that is a *principal* choice trivially satisfying all use constraints.

A second function for popping an operand value takes an expected type, which the actual operand type is checked against. The types may differ by subtyping, including the case where the actual type is `Bot`, and thereby matches unconditionally. The function returns the actual type popped from the stack.

Finally, there are accumulative functions for pushing or popping multiple operand types.

Note

The notation `stack[i]` is meant to index the stack from the top, so that, e.g., `ctrls[0]` accesses the element pushed last.

The initialization stack and the initialization status of locals is manipulated through the following functions:

```
func get_local(idx : u32) =
  error_if(not locals_init[idx])

func set_local(idx : u32) =
  if (not locals_init[idx])
    inits.push(idx)
    locals_init[idx] := true

func reset_locals(height : nat) =
  while (inits.size() > height)
    locals_init[inits.pop()] := false
```

Getting a local verifies that it is known to be initialized. When a local is set that was not set already, then its initialization status is updated and the change is recorded in the initialization stack. Thus, the initialization status of all locals can be reset to a previous state by denoting a specific height in the initialization stack.

The size of the initialization stack is bounded by the number of (non-defaultable) locals in a function, so can be preallocated by an algorithm.

The control stack is likewise manipulated through auxiliary functions:

```
func push_ctrl(opcode : opcode, in : list(val_type), out : list(val_type)) =
  let frame = ctrl_frame(opcode, in, out, vals.size(), inits.size(), false)
  ctrls.push(frame)
  push_vals(in)

func pop_ctrl() : ctrl_frame =
  error_if(ctrls.is_empty())
  let frame = ctrls[0]
  pop_vals(frame.end_types)
  error_if(vals.size() =/= frame.val_height)
  reset_locals(frame.init_height)
  ctrls.pop()
  return frame

func label_types(frame : ctrl_frame) : list(val_types) =
  return (if (frame.opcode = loop) frame.start_types else frame.end_types)

func unreachable() =
  vals.resize(ctrls[0].val_height)
  ctrls[0].unreachable := true
```

Pushing a control frame takes the types of the label and result values. It allocates a new frame record recording them along with the current height of the operand stack and marks the block as reachable.

Popping a frame first checks that the control stack is not empty. It then verifies that the operand stack contains the right types of values expected at the end of the exited block and pops them off the operand stack. Afterwards, it

checks that the stack has shrunk back to its initial height. Finally, it undoes all changes to the initialization status of locals that happened inside the block.

The type of the label associated with a control frame is either that of the stack at the start or the end of the frame, determined by the opcode that it originates from.

Finally, the current frame can be marked as unreachable. In that case, all existing operand types are purged from the value stack, in order to allow for the stack-polymorphism logic in `pop_val` to take effect. Because every function has an implicit outermost label that corresponds to an implicit block frame, it is an invariant of the validation algorithm that there always is at least one frame on the control stack when validating an instruction, and hence, *ctrls[0]* is always defined.

Note

Even with the unreachable flag set, consecutive operands are still pushed to and popped from the operand stack. That is necessary to detect invalid examples like (unreachable (i32.const) i64.add). However, a polymorphic stack cannot underflow, but instead generates `Bot` types as needed.

7.6.2 Validation of Opcode Sequences

The following function shows the validation of a number of representative instructions that manipulate the stack. Other instructions are checked in a similar manner.

```
func validate(opcode) =
  switch (opcode)
    case (i32.add)
      pop_val(I32)
      pop_val(I32)
      push_val(I32)

    case (drop)
      pop_val()

    case (select)
      pop_val(I32)
      let t1 = pop_val()
      let t2 = pop_val()
      error_if(not (is_num(t1) && is_num(t2) || is_vec(t1) && is_vec(t2)))
      error_if(t1 =/= t2 && t1 =/= Bot && t2 =/= Bot)
      push_val(if (t1 = Bot) t2 else t1)

    case (select t)
      pop_val(I32)
      pop_val(t)
      pop_val(t)
      push_val(t)

    case (ref.is_null)
      pop_ref()
      push_val(I32)

    case (ref.as_non_null)
      let rt = pop_ref()
      push_val(Ref(rt.heap, false))

    case (ref.test rt)
      validate_ref_type(rt)
```

(continues on next page)

(continued from previous page)

```
    pop_val(Ref(top_heap_type(rt), true))
    push_val(I32)

  case (local.get x)
    get_local(x)
    push_val(locals[x])

  case (local.set x)
    pop_val(locals[x])
    set_local(x)

  case (unreachable)
    unreachable()

  case (block t1*->t2*)
    pop_vals([t1*])
    push_ctrl(block, [t1*], [t2*])

  case (loop t1*->t2*)
    pop_vals([t1*])
    push_ctrl(loop, [t1*], [t2*])

  case (if t1*->t2*)
    pop_val(I32)
    pop_vals([t1*])
    push_ctrl(if, [t1*], [t2*])

  case (end)
    let frame = pop_ctrl()
    push_vals(frame.end_types)

  case (else)
    let frame = pop_ctrl()
    error_if(frame.opcode =/= if)
    push_ctrl(else, frame.start_types, frame.end_types)

  case (br n)
    error_if(ctrls.size() < n)
    pop_vals(label_types(ctrls[n]))
    unreachable()

  case (br_if n)
    error_if(ctrls.size() < n)
    pop_val(I32)
    pop_vals(label_types(ctrls[n]))
    push_vals(label_types(ctrls[n]))

  case (br_table n* m)
    pop_val(I32)
    error_if(ctrls.size() < m)
    let arity = label_types(ctrls[m]).size()
    foreach (n in n*)
      error_if(ctrls.size() < n)
      error_if(label_types(ctrls[n]).size() =/= arity)
      push_vals(pop_vals(label_types(ctrls[n])))
    pop_vals(label_types(ctrls[m]))
```

(continues on next page)

(continued from previous page)

```
    unreachable()

  case (br_on_null n)
    error_if(ctrls.size() < n)
    let rt = pop_ref()
    pop_vals(label_types(ctrls[n]))
    push_vals(label_types(ctrls[n]))
    push_val(Ref(rt.heap, false))

  case (br_on_cast n rt1 rt2)
    validate_ref_type(rt1)
    validate_ref_type(rt2)
    pop_val(rt1)
    push_val(rt2)
    pop_vals(label_types(ctrls[n]))
    push_vals(label_types(ctrls[n]))
    pop_val(rt2)
    push_val(diff_ref_type(rt2, rt1))

  case (return)
    pop_vals(return_types)
    unreachable()

  case (call_ref x)
    let t = expand_def(types[x])
    error_if(not is_func(t))
    pop_vals(t.params)
    pop_val(Ref(Def(types[x])))
    push_vals(t.results)

  case (return_call_ref x)
    let t = expand_def(types[x])
    error_if(not is_func(t))
    pop_vals(t.params)
    pop_val(Ref(Def(types[x])))
    error_if(t.results.len() =/= return_types.len())
    push_vals(t.results)
    pop_vals(return_types)
    unreachable()

  case (struct.new x)
    let t = expand_def(types[x])
    error_if(not is_struct(t))
    for (ti in reverse(t.fields))
      pop_val(unpack_field(ti))
    push_val(Ref(Def(types[x])))

  case (struct.set x n)
    let t = expand_def(types[x])
    error_if(not is_struct(t) || n >= t.fields.len())
    pop_val(Ref(Def(types[x])))
    pop_val(unpack_field(st.fields[n]))

  case (throw x)
     pop_vals(tags[x].type.params)
     unreachable()
```

(continues on next page)

(continued from previous page)

```
  case (try_table t1*->t2* handler*)
    pop_vals([t1*])
    foreach (handler in handler*)
      error_if(ctrls.size() < handler.label)
      push_ctrl(catch, [], label_types(ctrls[handler.label]))
      switch (handler.clause)
        case (catch x)
          push_vals(tags[x].type.params)
        case (catch_ref x)
          push_vals(tags[x].type.params)
          push_val(Exnref)
        case (catch_all)
          skip
        case (catch_all_ref)
          push_val(Exnref)
      pop_ctrl()
    push_ctrl(try_table, [t1*], [t2*])
```

Note

It is an invariant under the current WebAssembly instruction set that an operand of `Bot` type is never duplicated on the stack. This would change if the language were extended with stack instructions like `dup`. Under such an extension, the above algorithm would need to be refined by replacing the `Bot` type with proper *type variables* to ensure that all uses are consistent.

7.7 Custom Sections and Annotations

This appendix defines dedicated custom sections for WebAssembly's binary format and annotations for the text format. Such sections or annotations do not contribute to, or otherwise affect, the WebAssembly semantics, and may be ignored by an implementation. However, they provide useful meta data that implementations can make use of to improve user experience or take compilation hints.

7.7.1 Name Section

The *name section* is a custom section whose name string is itself '`name`'. The name section should appear only once in a module, and only after the data section.

The purpose of this section is to attach printable names to definitions in a module, which e.g. can be used by a debugger or when parts of the module are to be rendered in text form.

Note

All names are represented in Unicode[56] encoded in UTF-8. Names need not be unique.

Subsections

The data of a name section consists of a sequence of *subsections*. Each subsection consists of a

- a one-byte subsection *id*,
- the $u32$ *size* of the contents, in bytes,
- the actual *contents*, whose structure is dependent on the subsection id.

[56] https://www.unicode.org/versions/latest/

$$
\begin{array}{lcll}
\texttt{namesec} & ::= & \texttt{section}_0(\texttt{namedata}) & \\
\texttt{namedata} & ::= & n{:}\texttt{name} & (\text{if } n = \text{`}\texttt{name}\text{'}) \\
 & & \texttt{modulenamesubsec}^? & \\
 & & \texttt{funcnamesubsec}^? & \\
 & & \texttt{localnamesubsec}^? & \\
 & & \texttt{typenamesubsec}^? & \\
 & & \texttt{fieldnamesubsec}^? & \\
 & & \texttt{tagnamesubsec}^? & \\
\texttt{namesubsection}_N(\texttt{B}) & ::= & N{:}\texttt{byte}\ \mathit{size}{:}\texttt{u32}\ \texttt{B} & (\text{if } \mathit{size} = ||\texttt{B}||)
\end{array}
$$

The following subsection ids are used:

Id	Subsection
0	module name
1	function names
2	local names
4	type names
10	field names
11	tag names

Each subsection may occur at most once, and in order of increasing id.

Name Maps

A *name map* assigns names to indices in a given index space. It consists of a list of index/name pairs in order of increasing index value. Each index must be unique, but the assigned names need not be.

$$
\begin{array}{lcl}
\texttt{namemap} & ::= & \texttt{list}(\texttt{nameassoc}) \\
\texttt{nameassoc} & ::= & \texttt{idx}\ \texttt{name}
\end{array}
$$

An *indirect name map* assigns names to a two-dimensional index space, where secondary indices are *grouped* by primary indices. It consists of a list of primary index/name map pairs in order of increasing index value, where each name map in turn maps secondary indices to names. Each primary index must be unique, and likewise each secondary index per individual name map.

$$
\begin{array}{lcl}
\texttt{indirectnamemap} & ::= & \texttt{list}(\texttt{indirectnameassoc}) \\
\texttt{indirectnameassoc} & ::= & \texttt{idx}\ \texttt{namemap}
\end{array}
$$

Module Names

The *module name subsection* has the id 0. It simply consists of a single name that is assigned to the module itself.

$$
\texttt{modulenamesubsec} ::= \texttt{namesubsection}_0(\texttt{name})
$$

Function Names

The *function name subsection* has the id 1. It consists of a name map assigning function names to function indices.

$$
\texttt{funcnamesubsec} ::= \texttt{namesubsection}_1(\texttt{namemap})
$$

Local Names

The *local name subsection* has the id 2. It consists of an indirect name map assigning local names to local indices grouped by function indices.

$$
\texttt{localnamesubsec} ::= \texttt{namesubsection}_2(\texttt{indirectnamemap})
$$

Type Names

The *type name subsection* has the id 4. It consists of a name map assigning type names to type indices.

$$\texttt{typenamesubsec} \;::=\; \texttt{namesubsection}_4(\texttt{namemap})$$

Field Names

The *field name subsection* has the id 10. It consists of an indirect name map assigning field names to field indices grouped by the type indices of their respective structure types.

$$\texttt{fieldnamesubsec} \;::=\; \texttt{namesubsection}_{10}(\texttt{indirectnamemap})$$

Tag Names

The *tag name subsection* has the id 11. It consists of a name map assigning tag names to tag indices.

$$\texttt{tagnamesubsec} \;::=\; \texttt{namesubsection}_{11}(\texttt{namemap})$$

7.7.2 Name Annotations

Name annotations are the textual analogue to the name section and provide a textual representation for it. Consequently, their id is `@name`.

Analogous to the name section, name annotations are allowed on modules, functions, and locals (including parameters). They can be placed where the text format allows binding occurrences of respective identifiers. If both an identifier and a name annotation are given, the annotation is expected *after* the identifier. In that case, the annotation takes precedence over the identifier as a textual representation of the binding's name. At most one name annotation may be given per binding.

All name annotations have the following format:

$$\texttt{nameannot} \;::=\; \text{`(}\texttt{@name}\text{'}\ \texttt{string}\ \text{`)'}$$

Note

All name annotations can be arbitrary UTF-8 strings. Names need not be unique.

Module Names

A *module name annotation* must be placed on a module definition, directly after the '`module`' keyword, or if present, after the following module identifier.

$$\texttt{modulenameannot} \;::=\; \texttt{nameannot}$$

Function Names

A *function name annotation* must be placed on a function definition or function import, directly after the '`func`' keyword, or if present, after the following function identifier or.

$$\texttt{funcnameannot} \;::=\; \texttt{nameannot}$$

Parameter Names

A *parameter name annotation* must be placed on a parameter declaration, directly after the '`param`' keyword, or if present, after the following parameter identifier. It may only be placed on a declaration that declares exactly one parameter.

```
paramnameannot ::= nameannot
```

Local Names

A *local name annotation* must be placed on a local declaration, directly after the '`local`' keyword, or if present, after the following local identifier. It may only be placed on a declaration that declares exactly one local.

```
localnameannot ::= nameannot
```

Type Names

A *type name annotation* must be placed on a type declaration, directly after the '`type`' keyword, or if present, after the following type identifier.

```
typenameannot ::= nameannot
```

Field Names

A *field name annotation* must be placed on the field of a structure type, directly after the '`field`' keyword, or if present, after the following field identifier. It may only be placed on a declaration that declares exactly one field.

```
fieldnameannot ::= nameannot
```

Tag Names

A *tag name annotation* must be placed on a tag declaration or tag import, directly after the '`tag`' keyword, or if present, after the following tag identifier.

```
tagnameannot ::= nameannot
```

7.7.3 Custom Annotations

Custom annotations are a generic textual representation for any custom section. Their id is `@custom`. By generating custom annotations, tools converting between binary format and text format can maintain and round-trip the content of custom sections even when they do not recognize them.

Custom annotations must be placed inside a module definition. They must occur anywhere after the '`module`'

keyword, or if present, after the following module identifier. They must not be nested into other constructs.

```
customannot  ::=  '(@custom' string customplace? datastring ')'
customplace  ::=  '(' 'before' 'first' ')'
                |  '(' 'before' sec ')'
                |  '(' 'after' sec ')'
                |  '(' 'after' 'last' ')'
sec          ::=  'type'
                |  'import'
                |  'func'
                |  'table'
                |  'memory'
                |  'global'
                |  'export'
                |  'start'
                |  'elem'
                |  'code'
                |  'data'
                |  'datacount'
```

The first string in a custom annotation denotes the name of the custom section it represents. The remaining strings collectively represent the section's payload data, written as a data string, which can be split up into a possibly empty sequence of individual string literals (similar to data segments).

An arbitrary number of custom annotations (even of the same name) may occur in a module, each defining a separate custom section when converting to binary format. Placement of the sections in the binary can be customized via explicit *placement* directives, that position them either directly before or directly after a known section. That section must exist and be non-empty in the binary encoding of the annotated module. The placements (`before first`) and (`after last`) denote virtual sections before the first and after the last known section, respectively. When the placement directive is omitted, it defaults to (`after last`).

If multiple placement directives appear for the same position, then the sections are all placed there, in order of their appearance in the text. For this purpose, the position `after` a section is considered different from the position `before` the consecutive section, and the former occurs before the latter.

Note

Future versions of WebAssembly may introduce additional sections between others or at the beginning or end of a module. Using `first` and `last` guarantees that placement will still go before or after any future section, respectively.

If a custom section with a specific section id is given as well as annotations representing the same custom section (e.g., `@name` annotations as well as a `@custom` annotation for a `name` section), then two sections are assumed to be created. Their relative placement will depend on the placement directive given for the `@custom` annotation as well as the implicit placement requirements of the custom section, which are applied to the other annotation.

Note

For example, the following module,

```
(module
  (@custom "A" "aaa")
  (type $t (func))
  (@custom "B" (after func) "bbb")
  (@custom "C" (before func) "ccc")
  (@custom "D" (after last) "ddd")
  (table 10 funcref)
  (func (type $t))
  (@custom "E" (after import) "eee")
```

```
  (@custom "F" (before type) "fff")
  (@custom "G" (after data) "ggg")
  (@custom "H" (after code) "hhh")
  (@custom "I" (after func) "iii")
  (@custom "J" (before func) "jjj")
  (@custom "K" (before first) "kkk")
)
```

will result in the following section ordering:

```
custom section "K"
custom section "F"
type section
custom section "E"
custom section "C"
custom section "J"
function section
custom section "B"
custom section "I"
table section
code section
custom section "H"
custom section "G"
custom section "A"
custom section "D"
```

7.8 Change History

Since the original release 1.0 of the WebAssembly specification, a number of proposals for extensions have been integrated. The following sections provide an overview of what has changed.

All present and future versions of WebAssembly are intended to be *backwards-compatible* with all previous versions. Concretely:

1. All syntactically well-formed (in binary or text format) and valid modules remain well-formed and valid with an equivalent module type (or a subtype).

   Note

   This allows previously malformed or invalid modules to become legal, e.g., by adding new features or by relaxing typing rules.

   It also allows reclassifying previously malformed modules as well-formed but invalid, or vice versa.

   And it allows refining the typing of imports and exports, such that previously unlinkable modules become linkable.

   Historically, minor breaking changes to the *text format* have been allowed that turned previously possible valid modules invalid, as long as they were unlikely to occur in practice.

2. All non-trapping executions of a valid program retain their behaviour with an equivalent set of possible results (or a non-empty subset).

   Note

   This allows previously malformed or invalid programs to become executable.

   It also allows program executions that previously trapped to execute successfully, although the

> intention is to only exercise this where the possibility of such an extension has been previously noted.
>
> And it allows reducing the set of observable behaviours of a program execution, e.g., by reducing non-determinism.
>
> In a program linking prior modules with modules using new features, a prior module may encounter new behaviours, e.g., new forms of control flow or side effects when calling into a latter module.

In addition, future versions of WebAssembly will not allocate the opcode `0xFF` to represent an instruction or instruction prefix.

7.8.1 Release 2.0

Sign Extension Instructions

Added new numeric instructions for performing sign extension within integer representations.[57]

- New numeric instructions:
  - $\mathsf{i}nn.\mathsf{extend}N_\mathsf{s}$

Non-trapping Float-to-Int Conversions

Added new conversion instructions that avoid trapping when converting a floating-point number to an integer.[58]

- New numeric instructions:
  - $\mathsf{i}nn.\mathsf{trunc_sat_f}mm_sx$

Multiple Values

Generalized the result type of blocks and functions to allow for multiple values; in addition, introduced the ability to have block parameters.[59]

- Function types allow more than one result
- Block types can be arbitrary function types

Reference Types

Added funcref and externref as new value types and respective instructions.[60]

- New reference value types:
  - funcref
  - externref
- New reference instructions:
  - ref.null
  - ref.func
  - ref.is_null
- Extended parametric instruction:
  - select with optional type immediate
- New declarative form of element segment

[57] https://github.com/WebAssembly/spec/tree/main/proposals/sign-extension-ops/
[58] https://github.com/WebAssembly/spec/tree/main/proposals/nontrapping-float-to-int-conversion/
[59] https://github.com/WebAssembly/spec/tree/main/proposals/multi-value/
[60] https://github.com/WebAssembly/spec/tree/main/proposals/reference-types/

Table Instructions

Added instructions to directly access and modify tables.[Page 284, 60]

- Table types allow any reference type as element type
- New table instructions:
  - table.get
  - table.set
  - table.size
  - table.grow

Multiple Tables

Added the ability to use multiple tables per module.[Page 284, 60]

- Modules may
  - define multiple tables
  - import multiple tables
  - export multiple tables
- Table instructions take a table index immediate:
  - table.get
  - table.set
  - table.size
  - table.grow
  - call_indirect
- Element segments take a table index

Bulk Memory and Table Instructions

Added instructions that modify ranges of memory or table entries.[Page 284, 6061]

- New memory instructions:
  - memory.fill
  - memory.init
  - memory.copy
  - data.drop
- New table instructions:
  - table.fill
  - table.init
  - table.copy
  - elem.drop
- New passive form of data segment
- New passive form of element segment
- New data count section in binary format
- Active data and element segments boundaries are no longer checked at compile time but may trap instead

[61] https://github.com/WebAssembly/spec/tree/main/proposals/bulk-memory-operations/

Vector Instructions

Added vector type and instructions that manipulate multiple numeric values in parallel (also known as *SIMD*, single instruction multiple data)[62]

- New value type:
  - v128
- New memory instructions:
  - v128.load
  - v128.load*N*x*M*_*sx*
  - v128.load*N*_zero
  - v128.load*N*_splat
  - v128.load*N*_lane
  - v128.store
  - v128.store*N*_lane
- New constant vector instruction:
  - v128.const
- New unary vector instructions:
  - v128.not
  - i*N*x*M*.abs
  - i*N*x*M*.neg
  - i8x16.popcnt
  - f*N*x*M*.abs
  - f*N*x*M*.neg
  - f*N*x*M*.sqrt
  - f*N*x*M*.ceil
  - f*N*x*M*.floor
  - f*N*x*M*.trunc
  - f*N*x*M*.nearest
- New binary vector instructions:
  - v128.and
  - v128.andnot
  - v128.or
  - v128.xor
  - i*N*x*M*.add
  - i*N*x*M*.sub
  - i*N*x*M*.mul
  - i*N*x*M*.add_sat_*sx*
  - i*N*x*M*.sub_sat_*sx*
  - i*N*x*M*.min_*sx*

[62] https://github.com/WebAssembly/spec/tree/main/proposals/simd/

  - i*N*x*M*.max_*sx*
  - i*N*x*M*.shl
  - i*N*x*M*.shr_*sx*
  - f*N*x*M*.add
  - f*N*x*M*.sub
  - f*N*x*M*.mul
  - f*N*x*M*.div
  - i16x8.extadd_pairwise_i8x16_*sx*
  - i32x4.extadd_pairwise_i16x8_*sx*
  - i*N*x*M*.extmul_*half*_i*N*′x*M*′_*sx*
  - i16x8.q15mulr_sat_s
  - i32x4.dot_i16x8_s
  - i8x16.avgr_u
  - i16x8.avgr_u
  - f*N*x*M*.min
  - f*N*x*M*.max
  - f*N*x*M*.pmin
  - f*N*x*M*.pmax
- New ternary vector instruction:
  - v128.bitselect
- New test vector instructions:
  - v128.any_true
  - i*N*x*M*.all_true
- New relational vector instructions:
  - i*N*x*M*.eq
  - i*N*x*M*.ne
  - i*N*x*M*.lt_*sx*
  - i*N*x*M*.gt_*sx*
  - i*N*x*M*.le_*sx*
  - i*N*x*M*.ge_*sx*
  - f*N*x*M*.eq
  - f*N*x*M*.ne
  - f*N*x*M*.lt
  - f*N*x*M*.gt
  - f*N*x*M*.le
  - f*N*x*M*.ge
- New conversion vector instructions:
  - i32x4.trunc_sat_f32x4_*sx*
  - i32x4.trunc_sat_f64x2_*sx*_zero

  - f32x4.convert_i32x4_sx
  - f32x4.demote_f64x2_zero
  - f64x2.convert_low_i32x4_sx
  - f64x2.promote_low_f32x4
- New lane access vector instructions:
  - iNxM.extract_lane_sx?
  - iNxM.replace_lane
  - fNxM.extract_lane
  - fNxM.replace_lane
- New lane splitting/combining vector instructions:
  - iNxM.extend_$half$_iN'xM'_sx
  - i8x16.narrow_i16x8_sx
  - i16x8.narrow_i32x4_sx
- New byte reordering vector instructions:
  - i8x16.shuffle
  - i8x16.swizzle
- New injection/projection vector instructions:
  - iNxM.splat
  - fNxM.splat
  - iNxM.bitmask

7.8.2 Release 3.0

Extended Constant Expressions

Allowed basic numeric computations in constant expressions.[63]

- Extended set of constant instructions with:
  - inn.add
  - inn.sub
  - inn.mul
  - global.get for any previously declared immutable global

Note

The garbage collection extension added further constant instructions.

Tail Calls

Added instructions to perform tail calls.[64]

- New control instructions:
  - return_call
  - return_call_indirect

[63] https://github.com/WebAssembly/spec/tree/main/proposals/extended-const/
[64] https://github.com/WebAssembly/spec/tree/main/proposals/tail-call/

Exception Handling

Added tag definitions, imports, and exports, and instructions to throw and catch exceptions.[65]

- Modules may
  - define tags
  - import tags
  - export tags
- New heap types:
  - exn
  - noexn
- New reference type short-hands:
  - exnref
  - nullexnref
- New control instructions:
  - throw
  - throw_ref
  - try_table
- New tag section in binary format.

Multiple Memories

Added the ability to use multiple memories per module.[66]

- Modules may
  - define multiple memories
  - import multiple memories
  - export multiple memories
- Memory instructions take a memory index immediate:
  - memory.size
  - memory.grow
  - memory.fill
  - memory.copy
  - memory.init
  - t.load
  - t.store
  - t.load$N$_sx
  - t.store$N$
  - v128.load$N$xM_sx
  - v128.load$N$_zero
  - v128.load$N$_splat
  - v128.load$N$_lane

[65] https://github.com/WebAssembly/spec/tree/main/proposals/exception-handling/
[66] https://github.com/WebAssembly/spec/tree/main/proposals/multi-memory/

  - v128.store$N$_lane

- Data segments take a memory index

64-bit Address Space

Added the ability to declare an i64 address type for tables and memories.[67]

- Address types denote a subset of the integral number types
- Table types include an address type
- Memory types include an address type
- Operand types of table and memory instructions now depend on the subject's declared address type:
  - table.get
  - table.set
  - table.size
  - table.grow
  - table.fill
  - table.copy
  - table.init
  - memory.size
  - memory.grow
  - memory.fill
  - memory.copy
  - memory.init
  - t.load
  - t.store
  - t.load$N$_sx
  - t.store$N$
  - v128.load$N$xM_sx
  - v128.load$N$_zero
  - v128.load$N$_splat
  - v128.load$N$_lane
  - v128.store$N$_lane

Typeful References

Added more precise types for references.[68]

- New generalised form of reference types:
  - (ref null$^?$ $heaptype$)
- New class of heap types:
  - func
  - extern

[67] https://github.com/WebAssembly/spec/tree/main/proposals/memory64/
[68] https://github.com/WebAssembly/spec/tree/main/proposals/function-references/

  - *typeidx*
- Basic subtyping on reference and value types
- New reference instructions:
  - ref.as_non_null
  - br_on_null
  - br_on_non_null
- New control instruction:
  - call_ref
- Refined typing of reference instruction:
  - ref.func with more precise result type
- Refined typing of local instructions and instruction sequences to track the initialization status of locals with non-defaultable type
- Refined decoding of active element segments with implicit element type and plain function indices (opcode 0) to produce non-null reference type
- Extended table definitions with optional initializer expression

Garbage Collection

Added managed reference types.[69]

- New forms of heap types:
  - any
  - eq
  - i31
  - struct
  - array
  - none
  - nofunc
  - noextern
- New reference type short-hands:
  - anyref
  - eqref
  - i31ref
  - structref
  - arrayref
  - nullref
  - nullfuncref
  - nullexternref
- New forms of type definitions:
  - structure
  - array types

[69] https://github.com/WebAssembly/spec/tree/main/proposals/gc/

  - sub types
  - recursive types
- Enriched subtyping based on explicitly declared sub types and the new heap types
- New generic reference instructions:
  - ref.eq
  - ref.test
  - ref.cast
  - br_on_cast
  - br_on_cast_fail
- New reference instructions for unboxed scalars:
  - ref.i31
  - i31.get_sx
- New reference instructions for structure types:
  - struct.new
  - struct.new_default
  - struct.get_$sx^?$
  - struct.set
- New reference instructions for array types:
  - array.new
  - array.new_default
  - array.new_fixed
  - array.new_data
  - array.new_elem
  - array.get_$sx^?$
  - array.set
  - array.len
  - array.fill
  - array.copy
  - array.init_data
  - array.init_elem
- New reference instructions for converting external types:
  - any.convert_extern
  - extern.convert_any
- Extended set of constant instructions with:
  - ref.i31
  - struct.new
  - struct.new_default
  - array.new
  - array.new_default

  - array.new_fixed
  - any.convert_extern
  - extern.convert_any

Relaxed Vector Instructions

Added new *relaxed* vector instructions, whose behaviour is non-deterministic and implementation-dependent.[70]

- New binary vector instruction:
  - f*N*x*M*.relaxed_min
  - f*N*x*M*.relaxed_max
  - i16x8.relaxed_q15mulr_s
  - i16x8.relaxed_dot_i8x16_i7x16_s
- New ternary vector instruction:
  - f*N*x*M*.relaxed_madd
  - f*N*x*M*.relaxed_nmadd
  - i*N*x*M*.relaxed_laneselect
  - i32x4.relaxed_dot_i8x16_i7x16_add_s
- New conversion vector instructions:
  - i32x4.relaxed_trunc_f32x4_*sx*
  - i32x4.relaxed_trunc_f64x2_*sx*_zero
- New byte reordering vector instruction:
  - i8x16.relaxed_swizzle

Profiles

Introduced the concept of profile for specifying language subsets.

- A new profile defining a deterministic mode of execution.

Custom Annotations

Added generic syntax for custom annotations in the text format, mirroring the role of custom sections in the binary format.[71]

- Annotations of the form '(`@id` ...)' are allowed anywhere in the text format
- Identifiers can be escaped as '@"..."' with arbitrary names
- Defined name annotations '(`@name` "...")' for:
  - module names
  - type names
  - function names
  - local names
  - field names
- Defined custom annotation '(`@custom` "...")' to represent arbitrary custom sections in the text format

[70] https://github.com/WebAssembly/spec/tree/main/proposals/relaxed-simd/
[71] https://github.com/WebAssembly/spec/tree/main/proposals/annotations/

7.9 Index of Types

Category	Constructor	Binary Opcode
Type index	x	(positive number as s32 or u32)
Number type	i32	0x7F (-1 as s7)
Number type	i64	0x7E (-2 as s7)
Number type	f32	0x7D (-3 as s7)
Number type	f64	0x7C (-4 as s7)
Vector type	v128	0x7B (-5 as s7)
(reserved)		0x7A .. 0x79
Packed type	i8	0x78 (-8 as s7)
Packed type	i16	0x77 (-9 as s7)
(reserved)		0x78 .. 0x75
Heap type	noexn	0x74 (-12 as s7)
Heap type	nofunc	0x73 (-13 as s7)
Heap type	noextern	0x72 (-14 as s7)
Heap type	none	0x71 (-15 as s7)
Heap type	func	0x70 (-16 as s7)
Heap type	extern	0x6F (-17 as s7)
Heap type	any	0x6E (-18 as s7)
Heap type	eq	0x6D (-19 as s7)
Heap type	i31	0x6C (-20 as s7)
Heap type	struct	0x6B (-21 as s7)
Heap type	array	0x6A (-22 as s7)
Heap type	exn	0x69 (-23 as s7)
(reserved)		0x68 .. 0x65
Reference type	ref	0x64 (-28 as s7)
Reference type	ref null	0x63 (-29 as s7)
(reserved)		0x62 .. 0x61
Composite type	func $[valtype^*] \to [valtype^*]$	0x60 (-32 as s7)
Composite type	struct $fieldtype^*$	0x5F (-33 as s7)
Composite type	array $fieldtype$	0x5E (-34 as s7)
(reserved)		0x5D .. 0x51
Sub type	sub $typeidx^*$ $comptype$	0x50 (-48 as s7)
Sub type	sub final $typeidx^*$ $comptype$	0x4F (-49 as s7)
Recursive type	rec $subtype^*$	0x4E (-50 as s7)
(reserved)		0x4D .. 0x41
Result type	$[\epsilon]$	0x40 (-64 as s7)
Tag type	$typeuse$	(none)
Global type	mut $valtype$	(none)
Memory type	$addrtype$ $limits$	(none)
Table type	$addrtype$ $limits$ $reftype$	(none)

7.10 Index of Instructions

Instruction	Binary Opcode	Type	Validation	Executio
unreachable	0x00	$[t_1^*] \to [t_2^*]$	validation	execution
nop	0x01	$[] \to []$	validation	execution
block bt	0x02	$[t_1^*] \to [t_2^*]$	validation	execution
loop bt	0x03	$[t_1^*] \to [t_2^*]$	validation	execution
if bt	0x04	$[t_1^*$ i32$] \to [t_2^*]$	validation	execution
else	0x05			

continues on

Table 2 – continued from previous page

Instruction	Binary Opcode	Type	Validation	Executio
(reserved)	0x06			
(reserved)	0x07			
throw x	0x08	$[t_1^*\ t_x^*] \to [t_2^*]$	validation	execution
(reserved)	0x09			
throw_ref	0x0A	$[t_1^*\ \mathsf{exnref}] \to [t_2^*]$	validation	execution
end	0x0B			
br l	0x0C	$[t_1^*\ t^*] \to [t_2^*]$	validation	execution
br_if l	0x0D	$[t^*\ \mathsf{i32}] \to [t^*]$	validation	execution
br_table $l^*\ l$	0x0E	$[t_1^*\ t^*\ \mathsf{i32}] \to [t_2^*]$	validation	execution
return	0x0F	$[t_1^*\ t^*] \to [t_2^*]$	validation	execution
call x	0x10	$[t_1^*] \to [t_2^*]$	validation	execution
call_indirect $x\ y$	0x11	$[t_1^*\ \mathsf{i32}] \to [t_2^*]$	validation	execution
return_call x	0x12	$[t_1^*] \to [t_2^*]$	validation	execution
return_call_indirect $x\ y$	0x13	$[t_1^*\ \mathsf{i32}] \to [t_2^*]$	validation	execution
call_ref x	0x14	$[t_1^*\ (\mathsf{ref\ null}\ x)] \to [t_2^*]$	validation	execution
return_call_ref x	0x15	$[t_1^*\ (\mathsf{ref\ null}\ x)] \to [t_2^*]$	validation	execution
(reserved)	0x16			
(reserved)	0x17			
(reserved)	0x18			
(reserved)	0x19			
drop	0x1A	$[t] \to []$	validation	execution
select	0x1B	$[t\ t\ \mathsf{i32}] \to [t]$	validation	execution
select t	0x1C	$[t\ t\ \mathsf{i32}] \to [t]$	validation	execution
(reserved)	0x1D			
(reserved)	0x1E			
try_table bt	0x1F	$[t_1^*] \to [t_2^*]$	validation	execution
local.get x	0x20	$[] \to [t]$	validation	execution
local.set x	0x21	$[t] \to []$	validation	execution
local.tee x	0x22	$[t] \to [t]$	validation	execution
global.get x	0x23	$[] \to [t]$	validation	execution
global.set x	0x24	$[t] \to []$	validation	execution
table.get x	0x25	$[at] \to [t]$	validation	execution
table.set x	0x26	$[at\ t] \to []$	validation	execution
(reserved)	0x27			
i32.load x *memarg*	0x28	$[at] \to [\mathsf{i32}]$	validation	execution
i64.load x *memarg*	0x29	$[at] \to [\mathsf{i64}]$	validation	execution
f32.load x *memarg*	0x2A	$[at] \to [\mathsf{f32}]$	validation	execution
f64.load x *memarg*	0x2B	$[at] \to [\mathsf{f64}]$	validation	execution
i32.load8_s x *memarg*	0x2C	$[at] \to [\mathsf{i32}]$	validation	execution
i32.load8_u x *memarg*	0x2D	$[at] \to [\mathsf{i32}]$	validation	execution
i32.load16_s x *memarg*	0x2E	$[at] \to [\mathsf{i32}]$	validation	execution
i32.load16_u x *memarg*	0x2F	$[at] \to [\mathsf{i32}]$	validation	execution
i64.load8_s x *memarg*	0x30	$[at] \to [\mathsf{i64}]$	validation	execution
i64.load8_u x *memarg*	0x31	$[at] \to [\mathsf{i64}]$	validation	execution
i64.load16_s x *memarg*	0x32	$[at] \to [\mathsf{i64}]$	validation	execution
i64.load16_u x *memarg*	0x33	$[at] \to [\mathsf{i64}]$	validation	execution
i64.load32_s x *memarg*	0x34	$[at] \to [\mathsf{i64}]$	validation	execution
i64.load32_u x *memarg*	0x35	$[at] \to [\mathsf{i64}]$	validation	execution
i32.store x *memarg*	0x36	$[at\ \mathsf{i32}] \to []$	validation	execution
i64.store x *memarg*	0x37	$[at\ \mathsf{i64}] \to []$	validation	execution
f32.store x *memarg*	0x38	$[at\ \mathsf{f32}] \to []$	validation	execution
f64.store x *memarg*	0x39	$[at\ \mathsf{f64}] \to []$	validation	execution
i32.store8 x *memarg*	0x3A	$[at\ \mathsf{i32}] \to []$	validation	execution
i32.store16 x *memarg*	0x3B	$[at\ \mathsf{i32}] \to []$	validation	execution
i64.store8 x *memarg*	0x3C	$[at\ \mathsf{i64}] \to []$	validation	execution

continues on

Table 2 – continued from previous page

Instruction	Binary Opcode	Type	Validation	Executio
i64.store16 x *memarg*	0x3D	$[at\ \text{i64}] \to []$	validation	execution
i64.store32 x *memarg*	0x3E	$[at\ \text{i64}] \to []$	validation	execution
memory.size x	0x3F	$[] \to [at]$	validation	execution
memory.grow x	0x40	$[at] \to [at]$	validation	execution
i32.const $i32$	0x41	$[] \to [\text{i32}]$	validation	execution
i64.const $i64$	0x42	$[] \to [\text{i64}]$	validation	execution
f32.const $f32$	0x43	$[] \to [\text{f32}]$	validation	execution
f64.const $f64$	0x44	$[] \to [\text{f64}]$	validation	execution
i32.eqz	0x45	$[\text{i32}] \to [\text{i32}]$	validation	execution
i32.eq	0x46	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.ne	0x47	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.lt_s	0x48	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.lt_u	0x49	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.gt_s	0x4A	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.gt_u	0x4B	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.le_s	0x4C	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.le_u	0x4D	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.ge_s	0x4E	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.ge_u	0x4F	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i64.eqz	0x50	$[\text{i64}] \to [\text{i32}]$	validation	execution
i64.eq	0x51	$[\text{i64}\ \text{i64}] \to [\text{i32}]$	validation	execution
i64.ne	0x52	$[\text{i64}\ \text{i64}] \to [\text{i32}]$	validation	execution
i64.lt_s	0x53	$[\text{i64}\ \text{i64}] \to [\text{i32}]$	validation	execution
i64.lt_u	0x54	$[\text{i64}\ \text{i64}] \to [\text{i32}]$	validation	execution
i64.gt_s	0x55	$[\text{i64}\ \text{i64}] \to [\text{i32}]$	validation	execution
i64.gt_u	0x56	$[\text{i64}\ \text{i64}] \to [\text{i32}]$	validation	execution
i64.le_s	0x57	$[\text{i64}\ \text{i64}] \to [\text{i32}]$	validation	execution
i64.le_u	0x58	$[\text{i64}\ \text{i64}] \to [\text{i32}]$	validation	execution
i64.ge_s	0x59	$[\text{i64}\ \text{i64}] \to [\text{i32}]$	validation	execution
i64.ge_u	0x5A	$[\text{i64}\ \text{i64}] \to [\text{i32}]$	validation	execution
f32.eq	0x5B	$[\text{f32}\ \text{f32}] \to [\text{i32}]$	validation	execution
f32.ne	0x5C	$[\text{f32}\ \text{f32}] \to [\text{i32}]$	validation	execution
f32.lt	0x5D	$[\text{f32}\ \text{f32}] \to [\text{i32}]$	validation	execution
f32.gt	0x5E	$[\text{f32}\ \text{f32}] \to [\text{i32}]$	validation	execution
f32.le	0x5F	$[\text{f32}\ \text{f32}] \to [\text{i32}]$	validation	execution
f32.ge	0x60	$[\text{f32}\ \text{f32}] \to [\text{i32}]$	validation	execution
f64.eq	0x61	$[\text{f64}\ \text{f64}] \to [\text{i32}]$	validation	execution
f64.ne	0x62	$[\text{f64}\ \text{f64}] \to [\text{i32}]$	validation	execution
f64.lt	0x63	$[\text{f64}\ \text{f64}] \to [\text{i32}]$	validation	execution
f64.gt	0x64	$[\text{f64}\ \text{f64}] \to [\text{i32}]$	validation	execution
f64.le	0x65	$[\text{f64}\ \text{f64}] \to [\text{i32}]$	validation	execution
f64.ge	0x66	$[\text{f64}\ \text{f64}] \to [\text{i32}]$	validation	execution
i32.clz	0x67	$[\text{i32}] \to [\text{i32}]$	validation	execution
i32.ctz	0x68	$[\text{i32}] \to [\text{i32}]$	validation	execution
i32.popcnt	0x69	$[\text{i32}] \to [\text{i32}]$	validation	execution
i32.add	0x6A	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.sub	0x6B	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.mul	0x6C	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.div_s	0x6D	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.div_u	0x6E	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.rem_s	0x6F	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.rem_u	0x70	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.and	0x71	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.or	0x72	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution
i32.xor	0x73	$[\text{i32}\ \text{i32}] \to [\text{i32}]$	validation	execution

continues on

Table 2 – continued from previous page

Instruction	Binary Opcode	Type	Validation	Executio
i32.shl	0x74	[i32 i32] → [i32]	validation	execution
i32.shr_s	0x75	[i32 i32] → [i32]	validation	execution
i32.shr_u	0x76	[i32 i32] → [i32]	validation	execution
i32.rotl	0x77	[i32 i32] → [i32]	validation	execution
i32.rotr	0x78	[i32 i32] → [i32]	validation	execution
i64.clz	0x79	[i64] → [i64]	validation	execution
i64.ctz	0x7A	[i64] → [i64]	validation	execution
i64.popcnt	0x7B	[i64] → [i64]	validation	execution
i64.add	0x7C	[i64 i64] → [i64]	validation	execution
i64.sub	0x7D	[i64 i64] → [i64]	validation	execution
i64.mul	0x7E	[i64 i64] → [i64]	validation	execution
i64.div_s	0x7F	[i64 i64] → [i64]	validation	execution
i64.div_u	0x80	[i64 i64] → [i64]	validation	execution
i64.rem_s	0x81	[i64 i64] → [i64]	validation	execution
i64.rem_u	0x82	[i64 i64] → [i64]	validation	execution
i64.and	0x83	[i64 i64] → [i64]	validation	execution
i64.or	0x84	[i64 i64] → [i64]	validation	execution
i64.xor	0x85	[i64 i64] → [i64]	validation	execution
i64.shl	0x86	[i64 i64] → [i64]	validation	execution
i64.shr_s	0x87	[i64 i64] → [i64]	validation	execution
i64.shr_u	0x88	[i64 i64] → [i64]	validation	execution
i64.rotl	0x89	[i64 i64] → [i64]	validation	execution
i64.rotr	0x8A	[i64 i64] → [i64]	validation	execution
f32.abs	0x8B	[f32] → [f32]	validation	execution
f32.neg	0x8C	[f32] → [f32]	validation	execution
f32.ceil	0x8D	[f32] → [f32]	validation	execution
f32.floor	0x8E	[f32] → [f32]	validation	execution
f32.trunc	0x8F	[f32] → [f32]	validation	execution
f32.nearest	0x90	[f32] → [f32]	validation	execution
f32.sqrt	0x91	[f32] → [f32]	validation	execution
f32.add	0x92	[f32 f32] → [f32]	validation	execution
f32.sub	0x93	[f32 f32] → [f32]	validation	execution
f32.mul	0x94	[f32 f32] → [f32]	validation	execution
f32.div	0x95	[f32 f32] → [f32]	validation	execution
f32.min	0x96	[f32 f32] → [f32]	validation	execution
f32.max	0x97	[f32 f32] → [f32]	validation	execution
f32.copysign	0x98	[f32 f32] → [f32]	validation	execution
f64.abs	0x99	[f64] → [f64]	validation	execution
f64.neg	0x9A	[f64] → [f64]	validation	execution
f64.ceil	0x9B	[f64] → [f64]	validation	execution
f64.floor	0x9C	[f64] → [f64]	validation	execution
f64.trunc	0x9D	[f64] → [f64]	validation	execution
f64.nearest	0x9E	[f64] → [f64]	validation	execution
f64.sqrt	0x9F	[f64] → [f64]	validation	execution
f64.add	0xA0	[f64 f64] → [f64]	validation	execution
f64.sub	0xA1	[f64 f64] → [f64]	validation	execution
f64.mul	0xA2	[f64 f64] → [f64]	validation	execution
f64.div	0xA3	[f64 f64] → [f64]	validation	execution
f64.min	0xA4	[f64 f64] → [f64]	validation	execution
f64.max	0xA5	[f64 f64] → [f64]	validation	execution
f64.copysign	0xA6	[f64 f64] → [f64]	validation	execution
i32.wrap_i64	0xA7	[i64] → [i32]	validation	execution
i32.trunc_f32_s	0xA8	[f32] → [i32]	validation	execution
i32.trunc_f32_u	0xA9	[f32] → [i32]	validation	execution
i32.trunc_f64_s	0xAA	[f64] → [i32]	validation	execution

continues on

Table 2 – continued from previous page

Instruction	Binary Opcode	Type	Validation	Executio
i32.trunc_f64_u	0xAB	[f64] → [i32]	validation	execution
i64.extend_i32_s	0xAC	[i32] → [i64]	validation	execution
i64.extend_i32_u	0xAD	[i32] → [i64]	validation	execution
i64.trunc_f32_s	0xAE	[f32] → [i64]	validation	execution
i64.trunc_f32_u	0xAF	[f32] → [i64]	validation	execution
i64.trunc_f64_s	0xB0	[f64] → [i64]	validation	execution
i64.trunc_f64_u	0xB1	[f64] → [i64]	validation	execution
f32.convert_i32_s	0xB2	[i32] → [f32]	validation	execution
f32.convert_i32_u	0xB3	[i32] → [f32]	validation	execution
f32.convert_i64_s	0xB4	[i64] → [f32]	validation	execution
f32.convert_i64_u	0xB5	[i64] → [f32]	validation	execution
f32.demote_f64	0xB6	[f64] → [f32]	validation	execution
f64.convert_i32_s	0xB7	[i32] → [f64]	validation	execution
f64.convert_i32_u	0xB8	[i32] → [f64]	validation	execution
f64.convert_i64_s	0xB9	[i64] → [f64]	validation	execution
f64.convert_i64_u	0xBA	[i64] → [f64]	validation	execution
f64.promote_f32	0xBB	[f32] → [f64]	validation	execution
i32.reinterpret_f32	0xBC	[f32] → [i32]	validation	execution
i64.reinterpret_f64	0xBD	[f64] → [i64]	validation	execution
f32.reinterpret_i32	0xBE	[i32] → [f32]	validation	execution
f64.reinterpret_i64	0xBF	[i64] → [f64]	validation	execution
i32.extend8_s	0xC0	[i32] → [i32]	validation	execution
i32.extend16_s	0xC1	[i32] → [i32]	validation	execution
i64.extend8_s	0xC2	[i64] → [i64]	validation	execution
i64.extend16_s	0xC3	[i64] → [i64]	validation	execution
i64.extend32_s	0xC4	[i64] → [i64]	validation	execution
(reserved)	0xC5			
(reserved)	0xC6			
(reserved)	0xC7			
(reserved)	0xC8			
(reserved)	0xC9			
(reserved)	0xCA			
(reserved)	0xCB			
(reserved)	0xCC			
(reserved)	0xCD			
(reserved)	0xCE			
(reserved)	0xCF			
ref.null ht	0xD0	[] → [(ref null ht)]	validation	execution
ref.is_null	0xD1	[(ref null ht)] → [i32]	validation	execution
ref.func x	0xD2	[] → [ref ht]	validation	execution
ref.eq	0xD3	[eqref eqref] → [i32]	validation	execution
ref.as_non_null	0xD4	[(ref null ht)] → [(ref ht)]	validation	execution
br_on_null l	0xD5	[t^* (ref null ht)] → [t^* (ref ht)]	validation	execution
br_on_non_null l	0xD6	[t^* (ref null ht)] → [t^*]	validation	execution
(reserved)	0xD7			
(reserved)	0xD8			
(reserved)	0xD9			
(reserved)	0xDA			
(reserved)	0xDB			
(reserved)	0xDC			
(reserved)	0xDD			
(reserved)	0xDE			
(reserved)	0xDF			
(reserved)	0xE0			
(reserved)	0xE1			

continues on

Table 2 – continued from previous page

Instruction	Binary Opcode	Type	Validation	Executio
(reserved)	0xE2			
(reserved)	0xE3			
(reserved)	0xE4			
(reserved)	0xE5			
(reserved)	0xE6			
(reserved)	0xE7			
(reserved)	0xE8			
(reserved)	0xE9			
(reserved)	0xEA			
(reserved)	0xEB			
(reserved)	0xEC			
(reserved)	0xED			
(reserved)	0xEE			
(reserved)	0xEF			
(reserved)	0xF0			
(reserved)	0xF1			
(reserved)	0xF2			
(reserved)	0xF3			
(reserved)	0xF4			
(reserved)	0xF5			
(reserved)	0xF6			
(reserved)	0xF7			
(reserved)	0xF8			
(reserved)	0xF9			
(reserved)	0xFA			
struct.new x	0xFB 0x00	$[t^*] \to [(\text{ref } x)]$	validation	execution
struct.new_default x	0xFB 0x01	$[] \to [(\text{ref } x)]$	validation	execution
struct.get $x\ y$	0xFB 0x02	$[(\text{ref null } x)] \to [t]$	validation	execution
struct.get_s $x\ y$	0xFB 0x03	$[(\text{ref null } x)] \to [\text{i32}]$	validation	execution
struct.get_u $x\ y$	0xFB 0x04	$[(\text{ref null } x)] \to [\text{i32}]$	validation	execution
struct.set $x\ y$	0xFB 0x05	$[(\text{ref null } x)\ t] \to []$	validation	execution
array.new x	0xFB 0x06	$[t\ \text{i32}] \to [(\text{ref } x)]$	validation	execution
array.new_default x	0xFB 0x07	$[\text{i32}] \to [(\text{ref } x)]$	validation	execution
array.new_fixed $x\ n$	0xFB 0x08	$[t^n] \to [(\text{ref } x)]$	validation	execution
array.new_data $x\ y$	0xFB 0x09	$[\text{i32 i32}] \to [(\text{ref } x)]$	validation	execution
array.new_elem $x\ y$	0xFB 0x0A	$[\text{i32 i32}] \to [(\text{ref } x)]$	validation	execution
array.get x	0xFB 0x0B	$[(\text{ref null } x)\ \text{i32}] \to [t]$	validation	execution
array.get_s x	0xFB 0x0C	$[(\text{ref null } x)\ \text{i32}] \to [\text{i32}]$	validation	execution
array.get_u x	0xFB 0x0D	$[(\text{ref null } x)\ \text{i32}] \to [\text{i32}]$	validation	execution
array.set x	0xFB 0x0E	$[(\text{ref null } x)\ \text{i32}\ t] \to []$	validation	execution
array.len	0xFB 0x0F	$[(\text{ref null array})] \to [\text{i32}]$	validation	execution
array.fill x	0xFB 0x10	$[(\text{ref null } x)\ \text{i32}\ t\ \text{i32}] \to []$	validation	execution
array.copy $x\ y$	0xFB 0x11	$[(\text{ref null } x)\ \text{i32}\ (\text{ref null } y)\ \text{i32 i32}] \to []$	validation	execution
array.init_data $x\ y$	0xFB 0x12	$[(\text{ref null } x)\ \text{i32 i32 i32}] \to []$	validation	execution
array.init_elem $x\ y$	0xFB 0x13	$[(\text{ref null } x)\ \text{i32 i32 i32}] \to []$	validation	execution
ref.test (ref t)	0xFB 0x14	$[(\text{ref } t')] \to [\text{i32}]$	validation	execution
ref.test (ref null t)	0xFB 0x15	$[(\text{ref null } t')] \to [\text{i32}]$	validation	execution
ref.cast (ref t)	0xFB 0x16	$[(\text{ref } t')] \to [(\text{ref } t)]$	validation	execution
ref.cast (ref null t)	0xFB 0x17	$[(\text{ref null } t')] \to [(\text{ref null } t)]$	validation	execution
br_on_cast $t_1\ t_2$	0xFB 0x18	$[t_1] \to [t_1 \setminus t_2]$	validation	execution
br_on_cast_fail $t_1\ t_2$	0xFB 0x19	$[t_1] \to [t_2]$	validation	execution
any.convert_extern	0xFB 0x1A	$[(\text{ref null extern})] \to [(\text{ref null any})]$	validation	execution
extern.convert_any	0xFB 0x1B	$[(\text{ref null any})] \to [(\text{ref null extern})]$	validation	execution
ref.i31	0xFB 0x1C	$[\text{i32}] \to [(\text{ref i31})]$	validation	execution
i31.get_s	0xFB 0x1D	$[\text{i31ref}] \to [\text{i32}]$	validation	execution

continues on

Table 2 – continued from previous page

Instruction	Binary Opcode	Type	Validation	Executio
i31.get_u	0xFB 0x1E	[i31ref] → [i32]	validation	execution
(reserved)	0xFB 0x1F ...			
i32.trunc_sat_f32_s	0xFC 0x00	[f32] → [i32]	validation	execution
i32.trunc_sat_f32_u	0xFC 0x01	[f32] → [i32]	validation	execution
i32.trunc_sat_f64_s	0xFC 0x02	[f64] → [i32]	validation	execution
i32.trunc_sat_f64_u	0xFC 0x03	[f64] → [i32]	validation	execution
i64.trunc_sat_f32_s	0xFC 0x04	[f32] → [i64]	validation	execution
i64.trunc_sat_f32_u	0xFC 0x05	[f32] → [i64]	validation	execution
i64.trunc_sat_f64_s	0xFC 0x06	[f64] → [i64]	validation	execution
i64.trunc_sat_f64_u	0xFC 0x07	[f64] → [i64]	validation	execution
memory.init x y	0xFC 0x08	[at i32 i32] → []	validation	execution
data.drop x	0xFC 0x09	[] → []	validation	execution
memory.copy x y	0xFC 0x0A	[at_1 at_2 at] → []	validation	execution
memory.fill y	0xFC 0x0B	[at i32 at] → []	validation	execution
table.init x y	0xFC 0x0C	[at i32 i32] → []	validation	execution
elem.drop x	0xFC 0x0D	[] → []	validation	execution
table.copy x y	0xFC 0x0E	[at_1 at_2 at] → []	validation	execution
table.grow x	0xFC 0x0F	[t at] → [at]	validation	execution
table.size x	0xFC 0x10	[] → [at]	validation	execution
table.fill x	0xFC 0x11	[at t at] → []	validation	execution
(reserved)	0xFC 0x12 ...			
v128.load x *memarg*	0xFD 0x00	[at] → [v128]	validation	execution
v128.load8x8_s x *memarg*	0xFD 0x01	[at] → [v128]	validation	execution
v128.load8x8_u x *memarg*	0xFD 0x02	[at] → [v128]	validation	execution
v128.load16x4_s x *memarg*	0xFD 0x03	[at] → [v128]	validation	execution
v128.load16x4_u x *memarg*	0xFD 0x04	[at] → [v128]	validation	execution
v128.load32x2_s x *memarg*	0xFD 0x05	[at] → [v128]	validation	execution
v128.load32x2_u x *memarg*	0xFD 0x06	[at] → [v128]	validation	execution
v128.load8_splat x *memarg*	0xFD 0x07	[at] → [v128]	validation	execution
v128.load16_splat x *memarg*	0xFD 0x08	[at] → [v128]	validation	execution
v128.load32_splat x *memarg*	0xFD 0x09	[at] → [v128]	validation	execution
v128.load64_splat x *memarg*	0xFD 0x0A	[at] → [v128]	validation	execution
v128.store x *memarg*	0xFD 0x0B	[at v128] → []	validation	execution
v128.const $i128$	0xFD 0x0C	[] → [v128]	validation	execution
i8x16.shuffle $laneidx^{16}$	0xFD 0x0D	[v128 v128] → [v128]	validation	execution
i8x16.swizzle	0xFD 0x0E	[v128 v128] → [v128]	validation	execution
i8x16.splat	0xFD 0x0F	[i32] → [v128]	validation	execution
i16x8.splat	0xFD 0x10	[i32] → [v128]	validation	execution
i32x4.splat	0xFD 0x11	[i32] → [v128]	validation	execution
i64x2.splat	0xFD 0x12	[i64] → [v128]	validation	execution
f32x4.splat	0xFD 0x13	[f32] → [v128]	validation	execution
f64x2.splat	0xFD 0x14	[f64] → [v128]	validation	execution
i8x16.extract_lane_s $laneidx$	0xFD 0x15	[v128] → [i32]	validation	execution
i8x16.extract_lane_u $laneidx$	0xFD 0x16	[v128] → [i32]	validation	execution
i8x16.replace_lane $laneidx$	0xFD 0x17	[v128 i32] → [v128]	validation	execution
i16x8.extract_lane_s $laneidx$	0xFD 0x18	[v128] → [i32]	validation	execution
i16x8.extract_lane_u $laneidx$	0xFD 0x19	[v128] → [i32]	validation	execution
i16x8.replace_lane $laneidx$	0xFD 0x1A	[v128 i32] → [v128]	validation	execution
i32x4.extract_lane $laneidx$	0xFD 0x1B	[v128] → [i32]	validation	execution
i32x4.replace_lane $laneidx$	0xFD 0x1C	[v128 i32] → [v128]	validation	execution
i64x2.extract_lane $laneidx$	0xFD 0x1D	[v128] → [i64]	validation	execution
i64x2.replace_lane $laneidx$	0xFD 0x1E	[v128 i64] → [v128]	validation	execution
f32x4.extract_lane $laneidx$	0xFD 0x1F	[v128] → [f32]	validation	execution
f32x4.replace_lane $laneidx$	0xFD 0x20	[v128 f32] → [v128]	validation	execution
f64x2.extract_lane $laneidx$	0xFD 0x21	[v128] → [f64]	validation	execution

continues on

Table 2 – continued from previous page

Instruction	Binary Opcode	Type	Validation	Executio
f64x2.replace_lane *laneidx*	0xFD 0x22	[v128 f64] → [v128]	validation	execution
i8x16.eq	0xFD 0x23	[v128 v128] → [v128]	validation	execution
i8x16.ne	0xFD 0x24	[v128 v128] → [v128]	validation	execution
i8x16.lt_s	0xFD 0x25	[v128 v128] → [v128]	validation	execution
i8x16.lt_u	0xFD 0x26	[v128 v128] → [v128]	validation	execution
i8x16.gt_s	0xFD 0x27	[v128 v128] → [v128]	validation	execution
i8x16.gt_u	0xFD 0x28	[v128 v128] → [v128]	validation	execution
i8x16.le_s	0xFD 0x29	[v128 v128] → [v128]	validation	execution
i8x16.le_u	0xFD 0x2A	[v128 v128] → [v128]	validation	execution
i8x16.ge_s	0xFD 0x2B	[v128 v128] → [v128]	validation	execution
i8x16.ge_u	0xFD 0x2C	[v128 v128] → [v128]	validation	execution
i16x8.eq	0xFD 0x2D	[v128 v128] → [v128]	validation	execution
i16x8.ne	0xFD 0x2E	[v128 v128] → [v128]	validation	execution
i16x8.lt_s	0xFD 0x2F	[v128 v128] → [v128]	validation	execution
i16x8.lt_u	0xFD 0x30	[v128 v128] → [v128]	validation	execution
i16x8.gt_s	0xFD 0x31	[v128 v128] → [v128]	validation	execution
i16x8.gt_u	0xFD 0x32	[v128 v128] → [v128]	validation	execution
i16x8.le_s	0xFD 0x33	[v128 v128] → [v128]	validation	execution
i16x8.le_u	0xFD 0x34	[v128 v128] → [v128]	validation	execution
i16x8.ge_s	0xFD 0x35	[v128 v128] → [v128]	validation	execution
i16x8.ge_u	0xFD 0x36	[v128 v128] → [v128]	validation	execution
i32x4.eq	0xFD 0x37	[v128 v128] → [v128]	validation	execution
i32x4.ne	0xFD 0x38	[v128 v128] → [v128]	validation	execution
i32x4.lt_s	0xFD 0x39	[v128 v128] → [v128]	validation	execution
i32x4.lt_u	0xFD 0x3A	[v128 v128] → [v128]	validation	execution
i32x4.gt_s	0xFD 0x3B	[v128 v128] → [v128]	validation	execution
i32x4.gt_u	0xFD 0x3C	[v128 v128] → [v128]	validation	execution
i32x4.le_s	0xFD 0x3D	[v128 v128] → [v128]	validation	execution
i32x4.le_u	0xFD 0x3E	[v128 v128] → [v128]	validation	execution
i32x4.ge_s	0xFD 0x3F	[v128 v128] → [v128]	validation	execution
i32x4.ge_u	0xFD 0x40	[v128 v128] → [v128]	validation	execution
f32x4.eq	0xFD 0x41	[v128 v128] → [v128]	validation	execution
f32x4.ne	0xFD 0x42	[v128 v128] → [v128]	validation	execution
f32x4.lt	0xFD 0x43	[v128 v128] → [v128]	validation	execution
f32x4.gt	0xFD 0x44	[v128 v128] → [v128]	validation	execution
f32x4.le	0xFD 0x45	[v128 v128] → [v128]	validation	execution
f32x4.ge	0xFD 0x46	[v128 v128] → [v128]	validation	execution
f64x2.eq	0xFD 0x47	[v128 v128] → [v128]	validation	execution
f64x2.ne	0xFD 0x48	[v128 v128] → [v128]	validation	execution
f64x2.lt	0xFD 0x49	[v128 v128] → [v128]	validation	execution
f64x2.gt	0xFD 0x4A	[v128 v128] → [v128]	validation	execution
f64x2.le	0xFD 0x4B	[v128 v128] → [v128]	validation	execution
f64x2.ge	0xFD 0x4C	[v128 v128] → [v128]	validation	execution
v128.not	0xFD 0x4D	[v128] → [v128]	validation	execution
v128.and	0xFD 0x4E	[v128 v128] → [v128]	validation	execution
v128.andnot	0xFD 0x4F	[v128 v128] → [v128]	validation	execution
v128.or	0xFD 0x50	[v128 v128] → [v128]	validation	execution
v128.xor	0xFD 0x51	[v128 v128] → [v128]	validation	execution
v128.bitselect	0xFD 0x52	[v128 v128 v128] → [v128]	validation	execution
v128.any_true	0xFD 0x53	[v128] → [i32]	validation	execution
v128.load8_lane *memarg laneidx*	0xFD 0x54	[*at* v128] → [v128]	validation	execution
v128.load16_lane *memarg laneidx*	0xFD 0x55	[*at* v128] → [v128]	validation	execution
v128.load32_lane *memarg laneidx*	0xFD 0x56	[*at* v128] → [v128]	validation	execution
v128.load64_lane *memarg laneidx*	0xFD 0x57	[*at* v128] → [v128]	validation	execution
v128.store8_lane *memarg laneidx*	0xFD 0x58	[*at* v128] → []	validation	execution

continues on

Table 2 – continued from previous page

Instruction	Binary Opcode	Type	Validation	Executio
v128.store16_lane *memarg laneidx*	0xFD 0x59	$[at\ \text{v128}] \to []$	validation	execution
v128.store32_lane *memarg laneidx*	0xFD 0x5A	$[at\ \text{v128}] \to []$	validation	execution
v128.store64_lane *memarg laneidx*	0xFD 0x5B	$[at\ \text{v128}] \to []$	validation	execution
v128.load32_zero *memarg*	0xFD 0x5C	$[at] \to [\text{v128}]$	validation	execution
v128.load64_zero *memarg*	0xFD 0x5D	$[at] \to [\text{v128}]$	validation	execution
f32x4.demote_f64x2_zero	0xFD 0x5E	$[\text{v128}] \to [\text{v128}]$	validation	execution
f64x2.promote_low_f32x4	0xFD 0x5F	$[\text{v128}] \to [\text{v128}]$	validation	execution
i8x16.abs	0xFD 0x60	$[\text{v128}] \to [\text{v128}]$	validation	execution
i8x16.neg	0xFD 0x61	$[\text{v128}] \to [\text{v128}]$	validation	execution
i8x16.popcnt	0xFD 0x62	$[\text{v128}] \to [\text{v128}]$	validation	execution
i8x16.all_true	0xFD 0x63	$[\text{v128}] \to [\text{i32}]$	validation	execution
i8x16.bitmask	0xFD 0x64	$[\text{v128}] \to [\text{i32}]$	validation	execution
i8x16.narrow_i16x8_s	0xFD 0x65	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution
i8x16.narrow_i16x8_u	0xFD 0x66	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution
f32x4.ceil	0xFD 0x67	$[\text{v128}] \to [\text{v128}]$	validation	execution
f32x4.floor	0xFD 0x68	$[\text{v128}] \to [\text{v128}]$	validation	execution
f32x4.trunc	0xFD 0x69	$[\text{v128}] \to [\text{v128}]$	validation	execution
f32x4.nearest	0xFD 0x6A	$[\text{v128}] \to [\text{v128}]$	validation	execution
i8x16.shl	0xFD 0x6B	$[\text{v128}\ \text{i32}] \to [\text{v128}]$	validation	execution
i8x16.shr_s	0xFD 0x6C	$[\text{v128}\ \text{i32}] \to [\text{v128}]$	validation	execution
i8x16.shr_u	0xFD 0x6D	$[\text{v128}\ \text{i32}] \to [\text{v128}]$	validation	execution
i8x16.add	0xFD 0x6E	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution
i8x16.add_sat_s	0xFD 0x6F	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution
i8x16.add_sat_u	0xFD 0x70	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution
i8x16.sub	0xFD 0x71	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution
i8x16.sub_sat_s	0xFD 0x72	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution
i8x16.sub_sat_u	0xFD 0x73	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution
f64x2.ceil	0xFD 0x74	$[\text{v128}] \to [\text{v128}]$	validation	execution
f64x2.floor	0xFD 0x75	$[\text{v128}] \to [\text{v128}]$	validation	execution
i8x16.min_s	0xFD 0x76	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution
i8x16.min_u	0xFD 0x77	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution
i8x16.max_s	0xFD 0x78	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution
i8x16.max_u	0xFD 0x79	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution
f64x2.trunc	0xFD 0x7A	$[\text{v128}] \to [\text{v128}]$	validation	execution
i8x16.avgr_u	0xFD 0x7B	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution
i16x8.extadd_pairwise_i8x16_s	0xFD 0x7C	$[\text{v128}] \to [\text{v128}]$	validation	execution
i16x8.extadd_pairwise_i8x16_u	0xFD 0x7D	$[\text{v128}] \to [\text{v128}]$	validation	execution
i32x4.extadd_pairwise_i16x8_s	0xFD 0x7E	$[\text{v128}] \to [\text{v128}]$	validation	execution
i32x4.extadd_pairwise_i16x8_u	0xFD 0x7F	$[\text{v128}] \to [\text{v128}]$	validation	execution
i16x8.abs	0xFD 0x80 0x01	$[\text{v128}] \to [\text{v128}]$	validation	execution
i16x8.neg	0xFD 0x81 0x01	$[\text{v128}] \to [\text{v128}]$	validation	execution
i16x8.q15mulr_sat_s	0xFD 0x82 0x01	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution
i16x8.all_true	0xFD 0x83 0x01	$[\text{v128}] \to [\text{i32}]$	validation	execution
i16x8.bitmask	0xFD 0x84 0x01	$[\text{v128}] \to [\text{i32}]$	validation	execution
i16x8.narrow_i32x4_s	0xFD 0x85 0x01	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution
i16x8.narrow_i32x4_u	0xFD 0x86 0x01	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution
i16x8.extend_low_i8x16_s	0xFD 0x87 0x01	$[\text{v128}] \to [\text{v128}]$	validation	execution
i16x8.extend_high_i8x16_s	0xFD 0x88 0x01	$[\text{v128}] \to [\text{v128}]$	validation	execution
i16x8.extend_low_i8x16_u	0xFD 0x89 0x01	$[\text{v128}] \to [\text{v128}]$	validation	execution
i16x8.extend_high_i8x16_u	0xFD 0x8A 0x01	$[\text{v128}] \to [\text{v128}]$	validation	execution
i16x8.shl	0xFD 0x8B 0x01	$[\text{v128}\ \text{i32}] \to [\text{v128}]$	validation	execution
i16x8.shr_s	0xFD 0x8C 0x01	$[\text{v128}\ \text{i32}] \to [\text{v128}]$	validation	execution
i16x8.shr_u	0xFD 0x8D 0x01	$[\text{v128}\ \text{i32}] \to [\text{v128}]$	validation	execution
i16x8.add	0xFD 0x8E 0x01	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution
i16x8.add_sat_s	0xFD 0x8F 0x01	$[\text{v128}\ \text{v128}] \to [\text{v128}]$	validation	execution

continues on

Table 2 – continued from previous page

Instruction	Binary Opcode	Type	Validation	Executio
i16x8.add_sat_u	0xFD 0x90 0x01	[v128 v128] → [v128]	validation	execution
i16x8.sub	0xFD 0x91 0x01	[v128 v128] → [v128]	validation	execution
i16x8.sub_sat_s	0xFD 0x92 0x01	[v128 v128] → [v128]	validation	execution
i16x8.sub_sat_u	0xFD 0x93 0x01	[v128 v128] → [v128]	validation	execution
f64x2.nearest	0xFD 0x94 0x01	[v128] → [v128]	validation	execution
i16x8.mul	0xFD 0x95 0x01	[v128 v128] → [v128]	validation	execution
i16x8.min_s	0xFD 0x96 0x01	[v128 v128] → [v128]	validation	execution
i16x8.min_u	0xFD 0x97 0x01	[v128 v128] → [v128]	validation	execution
i16x8.max_s	0xFD 0x98 0x01	[v128 v128] → [v128]	validation	execution
i16x8.max_u	0xFD 0x99 0x01	[v128 v128] → [v128]	validation	execution
(reserved)	0xFD 0x9A 0x01			
i16x8.avgr_u	0xFD 0x9B 0x01	[v128 v128] → [v128]	validation	execution
i16x8.extmul_low_i8x16_s	0xFD 0x9C 0x01	[v128 v128] → [v128]	validation	execution
i16x8.extmul_high_i8x16_s	0xFD 0x9D 0x01	[v128 v128] → [v128]	validation	execution
i16x8.extmul_low_i8x16_u	0xFD 0x9E 0x01	[v128 v128] → [v128]	validation	execution
i16x8.extmul_high_i8x16_u	0xFD 0x9F 0x01	[v128 v128] → [v128]	validation	execution
i32x4.abs	0xFD 0xA0 0x01	[v128] → [v128]	validation	execution
i32x4.neg	0xFD 0xA1 0x01	[v128] → [v128]	validation	execution
(reserved)	0xFD 0xA2 0x01			
i32x4.all_true	0xFD 0xA3 0x01	[v128] → [i32]	validation	execution
i32x4.bitmask	0xFD 0xA4 0x01	[v128] → [i32]	validation	execution
(reserved)	0xFD 0xA5 0x01			
(reserved)	0xFD 0xA6 0x01			
i32x4.extend_low_i16x8_s	0xFD 0xA7 0x01	[v128] → [v128]	validation	execution
i32x4.extend_high_i16x8_s	0xFD 0xA8 0x01	[v128] → [v128]	validation	execution
i32x4.extend_low_i16x8_u	0xFD 0xA9 0x01	[v128] → [v128]	validation	execution
i32x4.extend_high_i16x8_u	0xFD 0xAA 0x01	[v128] → [v128]	validation	execution
i32x4.shl	0xFD 0xAB 0x01	[v128 i32] → [v128]	validation	execution
i32x4.shr_s	0xFD 0xAC 0x01	[v128 i32] → [v128]	validation	execution
i32x4.shr_u	0xFD 0xAD 0x01	[v128 i32] → [v128]	validation	execution
i32x4.add	0xFD 0xAE 0x01	[v128 v128] → [v128]	validation	execution
(reserved)	0xFD 0xAF 0x01			
(reserved)	0xFD 0xB0 0x01			
i32x4.sub	0xFD 0xB1 0x01	[v128 v128] → [v128]	validation	execution
(reserved)	0xFD 0xB2 0x01			
(reserved)	0xFD 0xB3 0x01			
(reserved)	0xFD 0xB4 0x01			
i32x4.mul	0xFD 0xB5 0x01	[v128 v128] → [v128]	validation	execution
i32x4.min_s	0xFD 0xB6 0x01	[v128 v128] → [v128]	validation	execution
i32x4.min_u	0xFD 0xB7 0x01	[v128 v128] → [v128]	validation	execution
i32x4.max_s	0xFD 0xB8 0x01	[v128 v128] → [v128]	validation	execution
i32x4.max_u	0xFD 0xB9 0x01	[v128 v128] → [v128]	validation	execution
i32x4.dot_i16x8_s	0xFD 0xBA 0x01	[v128 v128] → [v128]	validation	execution
i32x4.extmul_low_i16x8_s	0xFD 0xBC 0x01	[v128 v128] → [v128]	validation	execution
i32x4.extmul_high_i16x8_s	0xFD 0xBD 0x01	[v128 v128] → [v128]	validation	execution
i32x4.extmul_low_i16x8_u	0xFD 0xBE 0x01	[v128 v128] → [v128]	validation	execution
i32x4.extmul_high_i16x8_u	0xFD 0xBF 0x01	[v128 v128] → [v128]	validation	execution
i64x2.abs	0xFD 0xC0 0x01	[v128] → [v128]	validation	execution
i64x2.neg	0xFD 0xC1 0x01	[v128] → [v128]	validation	execution
(reserved)	0xFD 0xC2 0x01			
i64x2.all_true	0xFD 0xC3 0x01	[v128] → [i32]	validation	execution
i64x2.bitmask	0xFD 0xC4 0x01	[v128] → [i32]	validation	execution
(reserved)	0xFD 0xC5 0x01			
(reserved)	0xFD 0xC6 0x01			
i64x2.extend_low_i32x4_s	0xFD 0xC7 0x01	[v128] → [v128]	validation	execution

continues on

Table 2 – continued from previous page

Instruction	Binary Opcode	Type	Validation	Executio
i64x2.extend_high_i32x4_s	0xFD 0xC8 0x01	[v128] → [v128]	validation	execution
i64x2.extend_low_i32x4_u	0xFD 0xC9 0x01	[v128] → [v128]	validation	execution
i64x2.extend_high_i32x4_u	0xFD 0xCA 0x01	[v128] → [v128]	validation	execution
i64x2.shl	0xFD 0xCB 0x01	[v128 i32] → [v128]	validation	execution
i64x2.shr_s	0xFD 0xCC 0x01	[v128 i32] → [v128]	validation	execution
i64x2.shr_u	0xFD 0xCD 0x01	[v128 i32] → [v128]	validation	execution
i64x2.add	0xFD 0xCE 0x01	[v128 v128] → [v128]	validation	execution
(reserved)	0xFD 0xCF 0x01			
(reserved)	0xFD 0xD0 0x01			
i64x2.sub	0xFD 0xD1 0x01	[v128 v128] → [v128]	validation	execution
(reserved)	0xFD 0xD2 0x01			
(reserved)	0xFD 0xD3 0x01			
(reserved)	0xFD 0xD4 0x01			
i64x2.mul	0xFD 0xD5 0x01	[v128 v128] → [v128]	validation	execution
i64x2.eq	0xFD 0xD6 0x01	[v128 v128] → [v128]	validation	execution
i64x2.ne	0xFD 0xD7 0x01	[v128 v128] → [v128]	validation	execution
i64x2.lt_s	0xFD 0xD8 0x01	[v128 v128] → [v128]	validation	execution
i64x2.gt_s	0xFD 0xD9 0x01	[v128 v128] → [v128]	validation	execution
i64x2.le_s	0xFD 0xDA 0x01	[v128 v128] → [v128]	validation	execution
i64x2.ge_s	0xFD 0xDB 0x01	[v128 v128] → [v128]	validation	execution
i64x2.extmul_low_i32x4_s	0xFD 0xDC 0x01	[v128 v128] → [v128]	validation	execution
i64x2.extmul_high_i32x4_s	0xFD 0xDD 0x01	[v128 v128] → [v128]	validation	execution
i64x2.extmul_low_i32x4_u	0xFD 0xDE 0x01	[v128 v128] → [v128]	validation	execution
i64x2.extmul_high_i32x4_u	0xFD 0xDF 0x01	[v128 v128] → [v128]	validation	execution
f32x4.abs	0xFD 0xE0 0x01	[v128] → [v128]	validation	execution
f32x4.neg	0xFD 0xE1 0x01	[v128] → [v128]	validation	execution
(reserved)	0xFD 0xE2 0x01			
f32x4.sqrt	0xFD 0xE3 0x01	[v128] → [v128]	validation	execution
f32x4.add	0xFD 0xE4 0x01	[v128 v128] → [v128]	validation	execution
f32x4.sub	0xFD 0xE5 0x01	[v128 v128] → [v128]	validation	execution
f32x4.mul	0xFD 0xE6 0x01	[v128 v128] → [v128]	validation	execution
f32x4.div	0xFD 0xE7 0x01	[v128 v128] → [v128]	validation	execution
f32x4.min	0xFD 0xE8 0x01	[v128 v128] → [v128]	validation	execution
f32x4.max	0xFD 0xE9 0x01	[v128 v128] → [v128]	validation	execution
f32x4.pmin	0xFD 0xEA 0x01	[v128 v128] → [v128]	validation	execution
f32x4.pmax	0xFD 0xEB 0x01	[v128 v128] → [v128]	validation	execution
f64x2.abs	0xFD 0xEC 0x01	[v128] → [v128]	validation	execution
f64x2.neg	0xFD 0xED 0x01	[v128] → [v128]	validation	execution
(reserved)	0xFD 0xEE 0x01			
f64x2.sqrt	0xFD 0xEF 0x01	[v128] → [v128]	validation	execution
f64x2.add	0xFD 0xF0 0x01	[v128 v128] → [v128]	validation	execution
f64x2.sub	0xFD 0xF1 0x01	[v128 v128] → [v128]	validation	execution
f64x2.mul	0xFD 0xF2 0x01	[v128 v128] → [v128]	validation	execution
f64x2.div	0xFD 0xF3 0x01	[v128 v128] → [v128]	validation	execution
f64x2.min	0xFD 0xF4 0x01	[v128 v128] → [v128]	validation	execution
f64x2.max	0xFD 0xF5 0x01	[v128 v128] → [v128]	validation	execution
f64x2.pmin	0xFD 0xF6 0x01	[v128 v128] → [v128]	validation	execution
f64x2.pmax	0xFD 0xF7 0x01	[v128 v128] → [v128]	validation	execution
i32x4.trunc_sat_f32x4_s	0xFD 0xF8 0x01	[v128] → [v128]	validation	execution
i32x4.trunc_sat_f32x4_u	0xFD 0xF9 0x01	[v128] → [v128]	validation	execution
f32x4.convert_i32x4_s	0xFD 0xFA 0x01	[v128] → [v128]	validation	execution
f32x4.convert_i32x4_u	0xFD 0xFB 0x01	[v128] → [v128]	validation	execution
i32x4.trunc_sat_f64x2_s_zero	0xFD 0xFC 0x01	[v128] → [v128]	validation	execution
i32x4.trunc_sat_f64x2_u_zero	0xFD 0xFD 0x01	[v128] → [v128]	validation	execution
f64x2.convert_low_i32x4_s	0xFD 0xFE 0x01	[v128] → [v128]	validation	execution

continues on

Table 2 – continued from previous page

Instruction	Binary Opcode	Type	Validation	Executio
f64x2.convert_low_i32x4_u	0xFD 0xFF 0x01	[v128] → [v128]	validation	execution
i8x16.relaxed_swizzle	0xFD 0x80 0x02	[v128 v128] → [v128]	validation	execution
i32x4.relaxed_trunc_f32x4_s	0xFD 0x81 0x02	[v128] → [v128]	validation	execution
i32x4.relaxed_trunc_f32x4_u	0xFD 0x82 0x02	[v128] → [v128]	validation	execution
i32x4.relaxed_trunc_f64x2_s	0xFD 0x83 0x02	[v128] → [v128]	validation	execution
i32x4.relaxed_trunc_f64x2_u	0xFD 0x84 0x02	[v128] → [v128]	validation	execution
f32x4.relaxed_madd	0xFD 0x85 0x02	[v128 v128 v128] → [v128]	validation	execution
f32x4.relaxed_nmadd	0xFD 0x86 0x02	[v128 v128 v128] → [v128]	validation	execution
f64x2.relaxed_madd	0xFD 0x87 0x02	[v128 v128 v128] → [v128]	validation	execution
f64x2.relaxed_nmadd	0xFD 0x88 0x02	[v128 v128 v128] → [v128]	validation	execution
i8x16.relaxed_laneselect	0xFD 0x89 0x02	[v128 v128 v128] → [v128]	validation	execution
i16x8.relaxed_laneselect	0xFD 0x8A 0x02	[v128 v128 v128] → [v128]	validation	execution
i32x4.relaxed_laneselect	0xFD 0x8B 0x02	[v128 v128 v128] → [v128]	validation	execution
i64x2.relaxed_laneselect	0xFD 0x8C 0x02	[v128 v128 v128] → [v128]	validation	execution
f32x4.relaxed_min	0xFD 0x8D 0x02	[v128 v128] → [v128]	validation	execution
f32x4.relaxed_max	0xFD 0x8E 0x02	[v128 v128] → [v128]	validation	execution
f64x2.relaxed_min	0xFD 0x8F 0x02	[v128 v128] → [v128]	validation	execution
f64x2.relaxed_max	0xFD 0x90 0x02	[v128 v128] → [v128]	validation	execution
i16x8.relaxed_q15mulr_s	0xFD 0x91 0x02	[v128 v128] → [v128]	validation	execution
i16x8.relaxed_dot_i8x16_i7x16_s	0xFD 0x92 0x02	[v128 v128] → [v128]	validation	execution
i32x4.relaxed_dot_i8x16_i7x16_add_s	0xFD 0x93 0x02	[v128 v128 v128] → [v128]	validation	execution
(reserved)	0xFD 0x94 0x02 ...			
(reserved)	0xFE			
(reserved)	0xFF			

Note

Multi-byte opcodes are given with the shortest possible encoding in the table. However, what is following the first byte is actually a u32 with variable-length encoding and consequently has multiple possible representations.

7.11 Index of Semantic Rules

7.11.1 Well-formedness of Types

Construct	Judgement
Numeric type	$C \vdash numtype : \mathsf{ok}$
Vector type	$C \vdash vectype : \mathsf{ok}$
Heap type	$C \vdash heaptype : \mathsf{ok}$
Reference type	$C \vdash reftype : \mathsf{ok}$
Value type	$C \vdash valtype : \mathsf{ok}$
Packed type	$C \vdash packtype : \mathsf{ok}$
Storage type	$C \vdash storagetype : \mathsf{ok}$
Field type	$C \vdash fieldtype : \mathsf{ok}$
Result type	$C \vdash resulttype : \mathsf{ok}$
Instruction type	$C \vdash instrtype : \mathsf{ok}$
Composite type	$C \vdash comptype : \mathsf{ok}$
Sub type	$C \vdash subtype : \mathsf{ok}$
Recursive type	$C \vdash rectype : \mathsf{ok}$
Defined type	$C \vdash deftype : \mathsf{ok}$
Block type	$C \vdash blocktype : instrtype$
Tag type	$C \vdash tagtype : \mathsf{ok}$
Global type	$C \vdash globaltype : \mathsf{ok}$
Memory type	$C \vdash memtype : \mathsf{ok}$
Table type	$C \vdash tabletype : \mathsf{ok}$
External type	$C \vdash externtype : \mathsf{ok}$
Type definitions	$C \vdash type^* : \mathsf{ok}$

7.11.2 Typing of Static Constructs

Construct	Judgement
Instruction	$S; C \vdash instr : instrtype$
Instruction sequence	$S; C \vdash instr^* : instrtype$
Catch clause	$C \vdash catch : \mathsf{ok}$
Expression	$C \vdash expr : resulttype$
Limits	$C \vdash limits : k$
Tag	$C \vdash tag : tagtype$
Global	$C \vdash global : globaltype$
Memory	$C \vdash mem : memtype$
Table	$C \vdash table : tabletype$
Function	$C \vdash func : deftype$
Local	$C \vdash local : localtype$
Element segment	$C \vdash elem : reftype$
Element mode	$C \vdash elemmode : reftype$
Data segment	$C \vdash data : \mathsf{ok}$
Data mode	$C \vdash datamode : \mathsf{ok}$
Start function	$C \vdash start : \mathsf{ok}$
Import	$C \vdash import : externtype$
Export	$C \vdash export : externtype$
Module	$\vdash module : externtype^* \to externtype^*$

7.11.3 Typing of Runtime Constructs

Construct	Judgement
Value	$S \vdash val : valtype$
Result	$S \vdash result : resulttype$
Packed value	$S \vdash packval : packtype$
Field value	$S \vdash fieldval : storagetype$
External address	$S \vdash externaddr : externtype$
Tag instance	$S \vdash taginst : tagtype$
Global instance	$S \vdash globalinst : globaltype$
Memory instance	$S \vdash meminst : memtype$
Table instance	$S \vdash tableinst : tabletype$
Function instance	$S \vdash funcinst : deftype$
Data instance	$S \vdash datainst : \mathsf{ok}$
Element instance	$S \vdash eleminst : t$
Structure instance	$S \vdash structinst : \mathsf{ok}$
Array instance	$S \vdash arrayinst : \mathsf{ok}$
Export instance	$S \vdash exportinst : \mathsf{ok}$
Module instance	$S \vdash moduleinst : C$
Store	$\vdash store : \mathsf{ok}$
Configuration	$\vdash config : [t^*]$
Thread	$S; resulttype^? \vdash thread : resulttype$
Frame	$S \vdash frame : C$

7.11.4 Constantness

Construct	Judgement
Constant expression	$C \vdash expr\mathsf{const}$
Constant instruction	$C \vdash instr\mathsf{const}$

7.11.5 Matching

Construct	Judgement
Number type	$C \vdash numtype_1 \leq numtype_2$
Vector type	$C \vdash vectype_1 \leq vectype_2$
Heap type	$C \vdash heaptype_1 \leq heaptype_2$
Reference type	$C \vdash reftype_1 \leq reftype_2$
Value type	$C \vdash valtype_1 \leq valtype_2$
Packed type	$C \vdash packtype_1 \leq packtype_2$
Storage type	$C \vdash storagetype_1 \leq storagetype_2$
Field type	$C \vdash fieldtype_1 \leq fieldtype_2$
Result type	$C \vdash resulttype_1 \leq resulttype_2$
Instruction type	$C \vdash instrtype_1 \leq instrtype_2$
Composite type	$C \vdash comptype_1 \leq comptype_2$
Defined type	$C \vdash deftype_1 \leq deftype_2$
Limits	$C \vdash limits_1 \leq limits_2$
Tag type	$C \vdash tagtype_1 \leq tagtype_2$
Global type	$C \vdash globaltype_1 \leq globaltype_2$
Memory type	$C \vdash memtype_1 \leq memtype_2$
Table type	$C \vdash tabletype_1 \leq tabletype_2$
External type	$C \vdash externtype_1 \leq externtype_2$

7.11.6 Store Extension

Construct	Judgement
Tag instance	$\vdash taginst_1 \preceq taginst_2$
Global instance	$\vdash globalinst_1 \preceq globalinst_2$
Memory instance	$\vdash meminst_1 \preceq meminst_2$
Table instance	$\vdash tableinst_1 \preceq tableinst_2$
Function instance	$\vdash funcinst_1 \preceq funcinst_2$
Data instance	$\vdash datainst_1 \preceq datainst_2$
Element instance	$\vdash eleminst_1 \preceq eleminst_2$
Structure instance	$\vdash structinst_1 \preceq structinst_2$
Array instance	$\vdash arrayinst_1 \preceq arrayinst_2$
Store	$\vdash store_1 \preceq store_2$

7.11.7 Execution

Construct	Judgement
Instruction	$S; F; instr^* \hookrightarrow S'; F'; instr'^*$
Expression	$S; F; expr \hookrightarrow S'; F'; expr'$

Index

Symbols

: abstract syntax
    administrative instruction, 89

A

abbreviations, **208**
abstract syntax, **5**, 179, 207, 249, 251
    array address, 84
    array instance, 87
    array type, 12, 36
    block type, 11, 15, 36
    byte, 7
    composite type, 12, 36
    data, 25, 75
    data address, 84
    data index, 23
    data instance, 87
    data type, 13
    defined type, 30, 45
    element, 26, 76
    element address, 84
    element index, 23
    element instance, 87
    element mode, 26
    element type, 13
    exception address, 84
    exception instance, 88
    export, 27, 77
    export instance, 87
    expression, 23, 71, 168
    external address, 85
    external index, 27
    external type, 13, 39
    field index, 23
    field type, 12, 36
    field value, 87
    floating-point number, 7
    frame, 88
    function, 25, 74
    function address, 84
    function index, 23
    function instance, 86
    function type, 12, 36
    global, 24, 73
    global address, 84
    global index, 23
    global instance, 86
    global type, 13, 38
    grammar, 5
    handler, 88
    heap type, 9, 29, 35
    host address, 84
    import, 26, 76
    instruction, 14–20, 48, 49, 55–57, 61, 66, 67, 122, 123, 135, 136, 140, 147, 160, 162
    instruction type, 31, 36
    integer, 7
    label, 88
    label index, 23
    limits, 12, 38
    list, 6
    local, 25, 75
    local index, 23
    local type, 32
    memory, 25, 74
    memory address, 84
    memory index, 23
    memory instance, 86
    memory type, 13, 38
    module, 23, 78
    module instance, 85
    mutability, 13
    name, 8
    notation, 5
    number type, 9, 34
    packed type, 12, 36
    packed value, 87
    recursive type, 12, 37
    recursive type index, 29
    reference type, 10, 35
    result, 84
    result type, 11, 36
    signed integer, 7
    start function, 26, 76
    storage type, 12, 36
    store, 84
    structure address, 84

    structure instance, 87
    structure type, 12, 36
    sub type, 12, 29, 37
    table, 25, 74
    table address, 84
    table index, 23
    table instance, 86
    table type, 13, 38
    tag, 24, 73
    tag address, 84
    tag index, 23
    tag instance, 87
    tag type, 13, 38
    type, 9, 73
    type definition, 24
    type index, 23
    type use, 9, 35
    uninterpreted integer, 7
    unsigned integer, 7
    value, 7, 83
    value type, 11, 29, 35, 36
    vector, 8
    vector type, 35
abstract type, **9**, 29
activation, 88
active, 25, **26**
address, **84**, 123, 135, 136, 140, 168
    array, 84
    data, 84
    element, 84
    exception, 84
    external, 85
    function, 84
    global, 84
    host, 84
    memory, 84
    structure, 84
    table, 84
    tag, 84
address type, 215, 290
    text format, 215
administrative instruction, 262, 263
    : abstract syntax, 89
administrative instructions, **89**
aggregate instruction, 188, 224
aggregate reference, 62
aggregate type, **12**, 24, 36, 43, 183, 215
    binary format, 183
    text format, 215
    validation, 36
algorithm, 270
allocation, 84, **168**, 242, 253
annotation, **211**, 278, 293
arithmetic NaN, 7
array, 12, 83, 119
    address, 84
    instance, 87
    type, 12
array address
    abstract syntax, 84
array instance, 84, **87**, 119, 257, 260, 267
    abstract syntax, 87
array type, **12**, 36, 39, 43, 87, 119, 183, 215, 257, 270, 291
    abstract syntax, 12
    binary format, 183
    text format, 215
    validation, 36
ASCII, 209, 210, 212

B

binary format, 8, **179**, 242, 249, 252, 270, 278
    aggregate type, 183
    array type, 183
    block type, 185
    byte, 181
    composite type, 183
    custom section, 201
    data, 204
    data count, 204
    data index, 200
    element, 203
    element index, 200
    export, 202
    expression, 200
    external type, 185
    field index, 200
    field type, 183
    floating-point number, 181
    function, 202, 203
    function index, 200
    function type, 183
    global, 202
    global index, 200
    global type, 184
    grammar, 179
    heap type, 182
    import, 202
    instruction, 185–189, 193
    integer, 181
    label index, 200
    limits, 184
    list, 180
    local, 203
    local index, 200
    memory, 202
    memory index, 200
    memory type, 184
    module, 205
    mutability, 184
    name, 181
    notation, 179
    number type, 182
    packed type, 183
    recursive type, 184
    reference type, 183

result type, 183
section, 200
signed integer, 181
start function, 203
storage type, 183
structure type, 183
sub type, 184
table, 202
table index, 200
table type, 185
tag, 204
tag index, 200
tag type, 184
type, 182
type index, 200
type section, 202
uninterpreted integer, 181
unsigned integer, 181
value, 180
value type, 183
vector type, 182
bit, 92
bit width, 7, 9, 12, 91, 140
block, 11, **15**, 49, 123, 132, 185, 218, 284
type, 11, 15
block type, **11**, **15**, 36, 49, 185
abstract syntax, 11, 15
binary format, 185
validation, 36
Boolean, 3, 92, 93
bottom type, **29**, 254
branch, **15**, 49, 123, 185, 218
byte, **7**, 8, 25, 75, 86, 87, 93, 169, 179, 181, 204, 212, 213, 236, 238, 246, 258, 259
abstract syntax, 7
binary format, 181
text format, 212

C

call, 88, 89, **134**, 288
call frame, **88**
canonical NaN, 7
cast, 18
caught, 89
caught exception, 89
changes, **283**
character, 2, 8, **209**, 209, 210, 212, 213, 251, 253
text format, 209
closed type, **29**
closure, 86
code, 14, 252
section, 203
code section, **203**
comment, 209, **210**
composite type, **12**, 12, 36, 37, 183, 184, 215, 270, 291
abstract syntax, 12
binary format, 183
text format, 215
validation, 36
composite types, 43
compositionality, **270**
concepts, 3
concrete type, **9**, 29
configuration, 82, **90**, 262, 267
constant, 23–26, **71**, 73–76, 83, 288
context, **32**, 47, 49, 55–57, 78, 205, 254, 261, 263
contextual rules, **167**
control frame, **88**
control instruction, **15**
control instructions, 49, 123, 185, 218
custom annotation, **281**
custom section, **201**, 278, 281, 293
binary format, 201

D

data, 23, **25**, 25, 75, 89, 169, 204, 205, 236, 238, 251
abstract syntax, 25
address, 84
binary format, 204
index, 23
instance, 87
section, 204
segment, 25, 75, 204, 236, 238
text format, 236, 238
type, 13
validation, 75
data address, 85, 169, 170
abstract syntax, 84
data count, 204
binary format, 204
section, 204
data count section, **204**
data index, **23**, 25, 200, 234
abstract syntax, 23
binary format, 200
text format, 234
data instance, 84, 85, **87**, 169, 170, 257, 259, 266
abstract syntax, 87
data section, **204**
data segment, 78, 86, 87, 170, 204, 239, 285, 289
data type, **13**
abstract syntax, 13
declaration, **239**
declarative, **26**
decoding, 4
default value, **83**
defaultable, 74
defined type, 13, **30**, 31, 39, 45, 73, 87, 119, 256–260, 270
abstract syntax, 30, 45
design goals, 1
determinism, 91, 101, 137, 144, 160, 251, 293
deterministic profile, 251
dynamic type, 119

E

element, 13, 23, 25, **26**, 76, 89, 169, 203, 205, 236, 238, 245, 251
  abstract syntax, 26
  address, 84
  binary format, 203
  index, 23
  instance, 87
  mode, 26
  section, 203
  segment, 26, 76, 203, 236, 238
  text format, 236, 238
  type, 13
  validation, 76
element address, 85, 136, 169, 170
  abstract syntax, 84
element expression, 87
element index, **23**, 26, 200, 234
  abstract syntax, 23
  binary format, 200
  text format, 234
element instance, 84, 85, **87**, 136, 169, 170, 257, 259, 266
  abstract syntax, 87
element mode, **26**
  abstract syntax, 26
element section, **203**
element segment, 78, 86, 87, 170, 239, 284, 285
element type, **13**, 46
  abstract syntax, 13
embedder, 2, **3**, 84, 86, 87, 241
embedding, **241**
evaluation context, 82
exception, **15**, 84, 88, 89, 132, 133, 174, 178, 247, 256, 288
  address, 84
  instance, 88
exception address, 84, 247
  abstract syntax, 84
exception handling, 185
exception instance, 84, **88**, 247, 260, 267
  abstract syntax, 88
exception tag, 13, 38, 73, 87, 122, 133, 184, 204, 235
exception type, 247
execution, 4, 9, 11, **81**, 167, 249, 253
  expression, 168
  instruction, 122, 123, 135, 136, 140, 147, 160, 162
expand, 256
expansion, **31**
exponent, 7, 93
export, 23, **27**, 77, 78, 87, 170, 178, 202, 205, 235–237, 239, 243, 244, 251, 285, 289
  abstract syntax, 27
  binary format, 202
  instance, 87
  section, 202
  text format, 235–237, 239
  validation, 77
export instance, 85, **87**, 170, 244, 261
  abstract syntax, 87
export section, **202**
expression, **23**, 24–26, 71, 73–76, 168, 200, 202–204, 234–236, 238, 288
  abstract syntax, 23
  binary format, 200
  constant, 23, 71, 200, 234
  execution, 168
  text format, 234
  validation, 71
external
  address, 85
  type, 13
external address, 13, **85**, 87, 121, 170, 261
  abstract syntax, 85
external index, 27, 239
  abstract syntax, 27
external reference, 66, 83
external type, **13**, 39, 46, 121, 170, 185, 216, 249, 261
  abstract syntax, 13
  binary format, 185
  text format, 216
  validation, 39

F

field, 23, 280, 281
  index, 23
field index, **23**, 200, 280
  abstract syntax, 23
  binary format, 200
field type, **12**, 36, 43, 44, 183, 215, 260, 267, 270, 291
  abstract syntax, 12
  binary format, 183
  text format, 215
  validation, 36
field value, **87**, 260, 267
  abstract syntax, 87
file extension, 179, 207
final, **12**, 37
floating point, 2
floating-point, 3, **7**, 8, 9, 19, 83, 91, 92, 100, 284
floating-point number, 181, 212
  abstract syntax, 7
  binary format, 181
  text format, 212
folded instruction, **233**
frame, **88**, 89, 90, 123, 134–136, 140, 253, 262–264, 270
  abstract syntax, 88
full profile, 251
funciton type, 43
function, 2, 3, 10–12, 15, 23, **25**, 26, 27, 32, 74, 78, 85, 86, 88, 89, 119, 134, 169, 170, 178, 202,

203, 205, 237, 239, 244, 251–253, 279–281, 284, 288, 290, 293
abstract syntax, 25, 74
address, 84
binary format, 202, 203
export, 27
import, 26
index, 23
instance, 86
section, 202
text format, 237
type, 12
function address, 85, 86, 89, 121, 169, 170, 178, 244, 245, 258
abstract syntax, 84
function index, 15, **23**, 25–27, 49, 74, 76, 77, 123, 170, 185, 200, 202, 203, 218, 234, 236–239, 279
abstract syntax, 23
binary format, 200
text format, 234
function instance, 84, 85, **86**, 89, 119, 134, 169, 170, 178, 244, 253, 257–259, 264, 266
abstract syntax, 86
function section, **202**
function type, 10, 11, **12**, 13, 15, 24, 26, 29, 32, 36, 38, 39, 46, 73, 74, 76, 85, 119, 121, 122, 169, 178, 183, 184, 202, 203, 205, 215, 216, 235, 237, 239, 244, 247, 257–259, 270, 290
abstract syntax, 12
binary format, 183
text format, 215
validation, 36
function type index, 204

G

global, 13, 16, 23, **24**, 26, 27, 32, 73, 78, 85, 86, 168, 170, 202, 205, 235, 239, 247, 251
abstract syntax, 24
address, 84
binary format, 202
export, 27
import, 26
index, 23
instance, 86
mutability, 13
section, 202
text format, 235
type, 13
validation, 73
global address, 85, 122, 135, 168, 170, 247
abstract syntax, 84
global index, 16, **23**, 24, 26, 27, 55, 77, 135, 170, 186, 200, 202, 220, 234, 235, 239
abstract syntax, 23
binary format, 200
text format, 234
global instance, 84, 85, **86**, 135, 168, 170, 247, 253, 257, 258, 264, 266
abstract syntax, 86
global section, **202**
global type, **13**, 13, 24, 26, 29, 32, 38, 39, 46, 73, 76, 122, 168, 184, 202, 216, 235, 239, 247, 257, 258
abstract syntax, 13
binary format, 184
text format, 216
validation, 38
grammar notation, **5**, 179, 207
greatest lower bound, 269
grow, 170

H

handler, **88**, 89, 133, 264, 288
abstract syntax, 88
heap type, **9**, 10, 18, 29, 35, 39, 182, 214, 236, 254, 256, 290, 291
abstract syntax, 9, 29
binary format, 182
text format, 214
validation, 35
host, 2, 84, 241
address, 84
host address, **83**
abstract syntax, 84
host function, **86**, 135, 244, 259

I

identifier, 207, 208, 234–237, 239, 253, 293
identifier context, **208**, 239
identifiers, **213**
text format, 213
IEEE 754, 2, 3, 7, 9, 93, 100
implementation, 241, 251
implementation limitations, **251**
import, 2, 13, 23–25, **26**, 74, 76, 78, 121, 170, 202, 205, 235–237, 239, 243, 251, 285, 289
abstract syntax, 26
binary format, 202
section, 202
text format, 235–237, 239
validation, 76
import section, **202**
index, **23**, 26, 27, 77, 85, 200, 202, 208, 217, 234–237, 239, 279
data, 23
element, 23
field, 23
function, 23
global, 23
label, 23
local, 23
memory, 23
table, 23
tag, 23

type, 23
index space, **23**, 26, 29, 32, 208, 279
instance, **85**, 174
array, 87
data, 87
element, 87
exception, 88
export, 87
function, 86
global, 86
memory, 86
module, 85
structure, 87
table, 86
tag, 87
instantiation, 4, 9, 26, 27, **174**, 243, 267
instantiation. module, 29
instruction, 3, 11, **14**, 23, 31, 47, 70, 86, 88–90, 122, 132, 185, 217, 251, 263, 264, 268, 270, 284, 285, 288–290, 293, 294
abstract syntax, 14–20
binary format, 185–189, 193
execution, 122, 123, 135, 136, 140, 147, 160, 162
text format, 218, 220, 223, 224, 227
type, 31
validation, 48, 49, 55–57, 61, 66, 67
instruction sequence, 70, 132
instruction type, **31**, 36, 43, **47**, 85, 268–270, 290
abstract syntax, 31
validation, 36
instructions, 285
integer, 3, **7**, 8, 9, 19, 83, 91–93, 136, 140, 181, 211, 284
abstract syntax, 7
binary format, 181
signed, 7
text format, 211
uninterpreted, 7
unsigned, 7
invocation, 4, 86, **178**, 244, 267

K

keyword, **209**

L

label, **15**, 49, **88**, 89, 123, 134, 185, 218, 253, 264, 270
abstract syntax, 88
index, 23
label index, 15, **23**, 49, 123, 185, 200, 217, 218, 234
abstract syntax, 23
binary format, 200
text format, 217, 234
lane, 8, 93
least upper bound, 269
LEB128, 181, 185
lexical format, 209
limits, **12**, 13, 25, 38, 45, 46, 136, 140, 169, 170, 184, 185, 216, 258
abstract syntax, 12
binary format, 184
memory, 13
table, 13
text format, 216
validation, 38
linear memory, 3
list, **6**, 11, 12, 15, 25, 26, 49, 123, 180, 185, 209, 218
abstract syntax, 6
binary format, 180
text format, 209
little endian, 17, 93, 181
local, 16, 23, **25**, 31, 32, 74, 75, 88, 203, 237, 251, 263, 279, 281, 290, 293
abstract syntax, 25
binary format, 203
index, 23
text format, 237
type, 32
validation, 75
local index, 16, **23**, 25, 31, 32, 55, 74, 135, 186, 200, 220, 234, 279
abstract syntax, 23
binary format, 200
text format, 234
local type, **32**, 32, 70, 74, 75, 290
abstract syntax, 32

M

magnitude, 7
mantissa, 212
matching, **39**, 170, 290
memmory argument, 57
memory, 3, 9, 13, 17, 23, **25**, 25–27, 32, 74, 75, 78, 85, 86, 89, 93, 169, 170, 202, 204, 205, 236, 238, 239, 246, 251, 285, 289, 290
abstract syntax, 25
address, 84
binary format, 202
data, 25, 75, 204, 236, 238
export, 27
import, 26
index, 23
instance, 86
limits, 12, 13
section, 202
text format, 236
type, 13
validation, 74
memory address, 85, 121, 140, 169, 170, 246
abstract syntax, 84
memory index, 17, **23**, 25–27, 57, 75, 77, 140, 170, 187, 200, 202, 204, 220, 234, 236, 238, 239, 289
abstract syntax, 23
binary format, 200

text format, 234
memory instance, 84, 85, **86**, 89, 140, 169, 170, 246, 253, 257, 258, 264, 266
abstract syntax, 86
memory instruction, **17**, 57, 140, 187, 220
memory section, **202**
memory type, 12, **13**, 13, 25, 26, 29, 32, 38, 39, 46, 74, 76, 86, 121, 169, 184, 202, 216, 236, 239, 246, 257, 258
abstract syntax, 13
binary format, 184
text format, 216
validation, 38
module, 2, 3, **23**, 32, 78, 84, 86, 170, 174, 178, 179, 205, 239, 242, 244, 251, 252, 267, 270, 279, 280, 293
abstract syntax, 23
binary format, 205
instance, 85
text format, 239
validation, 78
module instance, 86, 88, 119, 169, 170, 178, 243, 244, 253, 261, 263
abstract syntax, 85
module instruction, 90
mutability, **13**, 13, 24, 36, 38, 44, 46, 86, 122, 168, 183, 184, 215, 216, 258, 266
abstract syntax, 13
binary format, 184
global, 13
text format, 216

N

name, 2, **8**, 26, 27, 76, 77, 85, 87, 181, 202, 213, 235–237, 239, 251, 261, 278, 280
abstract syntax, 8
binary format, 181
text format, 213
name annotation, **280**
name map, **279**
name section, 239, **278**
NaN, **7**, 91, 101, 160
arithmetic, 7
canonical, 7
payload, 7
non-determinism, 91, 101, 137, 144, 160, 251, 293
notation, 5, 179, 207
abstract syntax, 5
binary format, 179
text format, 207
null, 10, 18
null reference, 119
number, 20, 83
type, 9
number type, **9**, 11, 34, 35, 39, 42, 83, 182, 183, 214, 270, 290
abstract syntax, 9
binary format, 182
text format, 214
validation, 34
numeric instruction, **19**, 66, 160, 189, 224
numeric vector, **8**, 20, 92, 93

O

offset, 23
opcode, **185**, 270, 275
operand, 14
operand stack, 14, 47

P

packed type, **12**, 36, 44, 92, 183, 215, 260, 270
abstract syntax, 12
binary format, 183
text format, 215
validation, 36
packed value, **87**, 260
abstract syntax, 87
page size, 13, 17, 25, **86**, 184, 216, 236
parameter, 12, 23, 251, 280
parametric instruction, **14**, 122, 185, 218
parametric instructions, 48
passive, 25, **26**
payload, 7
phases, 4
polymorphism, **47**, 48, 49, 185, 218, 268
portability, 1
preservation, **267**
principal types, **268**
profile, **249**
deterministic, 251
full, 251
profiles, 293
progress, **267**

R

reachability, 257
recursive type, **12**, 30, 31, 37, 45, 73, 78, 184, 202, 215, 234, 239, 254, 256, 270, 291
abstract syntax, 12, 37
binary format, 184
text format, 215
recursive type index, 12, **29**, 254, 256
abstract syntax, 29
reduction rules, **82**
reference, 10, 18, 83, 136, 147, 218, 248, 259, 284, 290, 291
type, 10
reference instruction, **18**, 18, 188, 223
reference instructions, 61, 147
reference type, **10**, 11, 13, 18, 35, 38, 42, 61, 74, 83, 136, 183, 185, 214, 216, 236, 248, 254, 270, 284, 288, 290, 291
abstract syntax, 10
binary format, 183
text format, 214
validation, 35

reftype, 89
result, 12, **84**, 244, 251, 256
    abstract syntax, 84
    type, 11
result type, **11**, 12, 29, 31, 32, 36, 43, 49, 71, 123, 183, 185, 215, 218, 256, 262–264, 284
    abstract syntax, 11
    binary format, 183
    validation, 36
rewrite rule, 208
roll, **12**
rolling, 29, **31**
rounding, 100
runtime, **83**, 306

S

S-expression, 207, 233
scalar reference, 66, 119
section, **200**, 205, 252, 278
    binary format, 200
    code, 203
    custom, 201
    data, 204
    data count, 204
    element, 203
    export, 202
    function, 202
    global, 202
    import, 202
    memory, 202
    name, 239
    start, 203
    table, 202
    tag, 204
    type, 202
security, **2**
segment, 89
shape, 93
sign, 93
signed integer, **7**, 93, 181, 211
    abstract syntax, 7
    binary format, 181
    text format, 211
significand, 7, 93
SIMD, 8, 9, 20, 285, 293
soundness, **254**, 267
source text, **209**, 209, 253
stack, 81, **88**, 178, 270
stack machine, 14
stack type, 15
start function, 23, **26**, 76, 78, 203, 205, 239
    abstract syntax, 26
    binary format, 203
    section, 203
    text format, 239
    validation, 76
start section, **203**
state, **90**
storage type, **12**, 36, 44, 183, 215, 260, 291
    abstract syntax, 12
    binary format, 183
    text format, 215
    validation, 36
store, 9, 81, **84**, 84, 85, 88, 90, 119, 121–123, 135, 136, 140, 168, 174, 178, 242, 244–247, 257, 260, 262–264
    abstract syntax, 84
store extension, **264**
string, **212**
    text format, 212
structure, 12, 83, 119
    address, 84
    instance, 87
    type, 12
structure address
    abstract syntax, 84
structure field, 293
structure instance, 84, **87**, 119, 257, 260, 267
    abstract syntax, 87
structure type, **12**, 36, 39, 43, 87, 119, 183, 215, 257, 270, 281, 291
    abstract syntax, 12
    binary format, 183
    text format, 215
    validation, 36
structured control, **15**, 49, 123, 185, 218
structured control instruction, 251
sub type, **12**, 29, 31, 37, 184, 215, 254, 270, 291
    abstract syntax, 12, 29, 37
    binary format, 184
    text format, 215
substitution, **30**
subtyping, 12, 29, 37, **39**, 249, 268–270, 290
syntax, 268

T

table, 3, 10, 13, 15, 16, 23, **25**, 26, 27, 32, 74, 76, 78, 85, 86, 89, 169, 170, 202, 205, 236, 239, 245, 251, 284, 285, 290
    abstract syntax, 25
    address, 84
    binary format, 202
    element, 26, 76, 203, 236, 238
    export, 27
    import, 26
    index, 23
    instance, 86
    limits, 12, 13
    section, 202
    text format, 236
    type, 13
    validation, 74
table address, 85, 121, 123, 136, 169, 170, 245
    abstract syntax, 84
table index, 16, **23**, 25–27, 56, 76, 77, 136, 170, 187, 200, 202, 203, 220, 234, 236, 238, 239, 285

    abstract syntax, 23
    binary format, 200
    text format, 234
table instance, 84, 85, **86**, 89, 123, 136, 169, 170, 245, 253, 257, 258, 264, 266
    abstract syntax, 86
table instruction, **16**, 56, 136, 187, 220
table section, **202**
table type, 12, **13**, 13, 25, 26, 29, 32, 38, 39, 46, 74, 76, 86, 121, 169, 185, 202, 216, 236, 239, 245, 257, 258, 284
    abstract syntax, 13
    binary format, 185
    text format, 216
    validation, 38
tag, 13, 15, 23, **24**, 26, 27, 32, 73, 78, 85, 87–89, 133, 168, 170, 204, 205, 235, 239, 247, 251, 260, 280, 281, 288
    abstract syntax, 24
    address, 84
    binary format, 204
    export, 27
    import, 26
    index, 23
    instance, 87
    section, 204
    text format, 235
    type, 13
    validation, 73
tag address, 85, 88, 89, 122, 168, 170, 247, 260
    abstract syntax, 84
tag index, **23**, 26, 27, 49, 77, 170, 185, 200, 202, 218, 234, 235, 239, 280
    abstract syntax, 23
    binary format, 200
    text format, 234
tag instance, 84, 85, **87**, 89, 168, 170, 247, 257, 258, 265
    abstract syntax, 87
tag section, **204**
tag type, **13**, 13, 15, 24, 26, 29, 32, 38, 45, 46, 73, 76, 87, 122, 168, 184, 202, 204, 216, 235, 239, 247, 257, 258, 288
    abstract syntax, 13
    binary format, 184
    text format, 216
    validation, 38
terminal configuration, 267
text format, 2, **207**, 242, 249, 253, 278, 293
    address type, 215
    aggregate type, 215
    annotation, 211
    array type, 215
    byte, 212
    character, 209
    comment, 210
    composite type, 215
    data, 236, 238
    data index, 234
    element, 236, 238
    element index, 234
    export, 235–237, 239
    expression, 234
    external type, 216
    field type, 215
    floating-point number, 212
    function, 237
    function index, 234
    function type, 215
    global, 235
    global index, 234
    global type, 216
    grammar, 207
    heap type, 214
    identifiers, 213
    import, 235–237, 239
    instruction, 218, 220, 223, 224, 227
    integer, 211
    label index, 217, 234
    limits, 216
    list, 209
    local, 237
    local index, 234
    memory, 236
    memory index, 234
    memory type, 216
    module, 239
    mutability, 216
    name, 213
    notation, 207
    number type, 214
    packed type, 215
    recursive type, 215
    reference type, 214
    signed integer, 211
    start function, 239
    storage type, 215
    string, 212
    structure type, 215
    sub type, 215
    table, 236
    table index, 234
    table type, 216
    tag, 235
    tag index, 234
    tag type, 216
    token, 209
    type, 213, 234
    type index, 234
    type use, 216
    uninterpreted integer, 211
    unsigned integer, 211
    value, 211
    value type, 214
    vector type, 214
    white space, 210

thread, **90**, 262, 263, 267
throw, 256
throw context, 133, 264
token, **209**, 253
trap, 3, 15–17, 84, 89, 132, 160, 174, 178, 256, 263, 284
try block, 15
two's complement, 3, 7, 19, 93, 181
type, **9**, 73, 119, 170, 182, 213, 234, 251, 279–281, 293, 306
    abstract syntax, 9, 73
    array, 12
    binary format, 182
    block, 11, 15
    data, 13
    element, 13
    external, 13
    function, 12
    global, 13
    index, 23
    instruction, 31
    local, 32
    memory, 13
    number, 9
    reference, 10
    result, 11
    section, 202
    structure, 12
    table, 13
    tag, 13
    text format, 213, 234
    value, 11
type closure, **33**
type definition, 23, **24**, 78, 202, 205, 239
    abstract syntax, 24
type equivalence, 31, 45
type index, 9, 11, 15, **23**, 24–26, 29, 35, 49, 73, 74, 119, 123, 185, 200, 202, 203, 218, 234, 237, 279, 280
    abstract syntax, 23
    binary format, 200
    text format, 234
type instance, 84, 85
type instantiation, 119
type lattice, **269**
type section, **202**
    binary format, 202
type system, **29**, 254, 268
type use, **9**, 9, 13, 35, 216
    abstract syntax, 9
    text format, 216
    validation, 35
typing rules, **33**

U

unboxed scalar, **9**, 83
unboxed scalar type, 39
Unicode, 2, 8, 181, 207, 209, 212, 251
unicode, 253
Unicode UTF-8, 278, 280
uninterpreted integer, **7**, 93, 181, 211
    abstract syntax, 7
    binary format, 181
    text format, 211
unroll, **12**, 45, 256
unrolling, 29, **31**
unsigned integer, **7**, 93, 181, 211
    abstract syntax, 7
    binary format, 181
    text format, 211
unwinding, **15**
UTF-8, 2, 8, **181**, 207, 212

V

validation, 4, 9, **29**, 119, 121, 122, 242, 249, 253, 260, 270, 306
    aggregate type, 36
    array type, 36
    block type, 36
    composite type, 36
    data, 75
    element, 76
    export, 77
    expression, 71
    external type, 39
    field type, 36
    function type, 36
    global, 73
    global type, 38
    heap type, 35
    import, 76
    instruction, 48, 49, 55–57, 61, 66, 67
    instruction type, 36
    limits, 38
    local, 75
    memory, 74
    memory type, 38
    module, 78
    number type, 34
    packed type, 36
    reference type, 35
    result type, 36
    start function, 76
    storage type, 36
    structure type, 36
    table, 74
    table type, 38
    tag, 73
    tag type, 38
    type use, 35
    value type, 35
    vector type, 35
validity, 267
value, 3, **7**, 19, 20, 24, 47, **83**, 84, 86, 88, 91, 119, 122, 135, 136, 140, 160, 168, 178, 180, 211, 244, 247, 248, 253, 256, 258, 263, 266

abstract syntax, 7, 83
binary format, 180
text format, 211
type, 11
value type, **11**, 11, 13–15, 19, 20, 25, 29, 31, 32, 35, 36, 38, 42–44, 46, 48, 74, 75, 83, 92, 119, 122, 140, 160, 168, 183–185, 214–216, 218, 248, 249, 254, 256, 263, 270, 284, 285, 290
abstract syntax, 11, 29
binary format, 183
text format, 214
validation, 35
variable instruction, **16**
variable instructions, 55, 135, 186, 220
vector
abstract syntax, 8
vector instruction, **20**, 67, 162, 193, 227, 293
vector number, 83
vector type, **9**, 11, 35, 39, 83, 182, 214, 270, 285
binary format, 182
text format, 214
validation, 35
version, 205

W

white space, 209, **210**